

A man in a tan polo shirt is smiling and looking down at a spiral-bound notebook he is holding a pen over. In the background, two other men are standing and talking near a computer monitor, and a woman is sitting at a desk with a laptop. The setting appears to be a training or office environment.

SIEMENS

SITRAIN: Training for Industry

TIA Portal Programming 2
(TIA-PRO2)

[siemens.com/sitrain](https://www.siemens.com/sitrain)

SIEMENS

SITRAIN

Training for Industry

SIMATIC S7

TIA Portal Programming 2

Course TIA-PRO2

Name: _____

Course from: _____ to: _____

Instructor: _____

Location: _____

This document was produced for training purposes. SIEMENS assumes no responsibility for its contents. The reproduction, transmission or use of this document or its contents is not permitted without express written authority. Offenders will be liable to damages.

Copyright © Siemens AG 2015. All rights, including rights created by patent grant or registration of a utility model or design, are reserved.

SITRAIN course offer on the Internet: www.siemens.com/sitrain

Course folder Version: 13.01.00 (for STEP7 Version 13 SP1)

1 Training Devices

2 Hardware Commissioning

3 Program Design Methods

4 Jump and Accumulator Functions

5 Analog Value Processing and Arithmetic

6 FCs, FBs and Multiple Instances

7 Complex Data and Addressing

8 Optimized Blocks

9 HMI Alarm Messages

10 System Diagnostics and Error OBs

11 Introduction to SCL

12 Introduction to S7-Graph (1)

13 Integrating and Commissioning a Drive with Startdrive

14 Training and Support

15

Contents

1.	Training Devices and Addressing.....	1-2
1.1.	Training Area Setup with S7-1500.....	1-3
1.2.	Positioning the Modular S7 Controllers	1-4
1.3.	SIMATIC S7-1500: Modular Controller for the Mid to Upper Performance Range.....	1-5
1.3.1.	SIMATIC S7-1500: Modules	1-6
1.3.2.	SIMATIC S7-1500: CPU-Display → Menu and Colors	1-8
1.3.3.	SIMATIC S7-1200/1500: Technology Functions	1-9
1.3.4.	SIMATIC S7-1200/1500: Memory Card.....	1-10
1.4.	Distributed I/O Systems	1-11
1.5.	Configuration of the S7-1500 Training Device	1-12
1.6.	Configuration of the ET200SP Training Device	1-13
1.7.	Operating and Display Elements of the Training Device	1-14
1.8.	Setup and Connection of the Conveyor Model.....	1-15
1.8.1.	Connection to Central I/Os of the S7-1500.....	1-15
1.8.2.	Connection to Distributed I/Os of the ET200SP	1-15
1.9.	Networking and IP Addresses of the Modules.....	1-16
1.10.	Training Area as Plant with Distribution Conveyor and Touchpanel	1-16

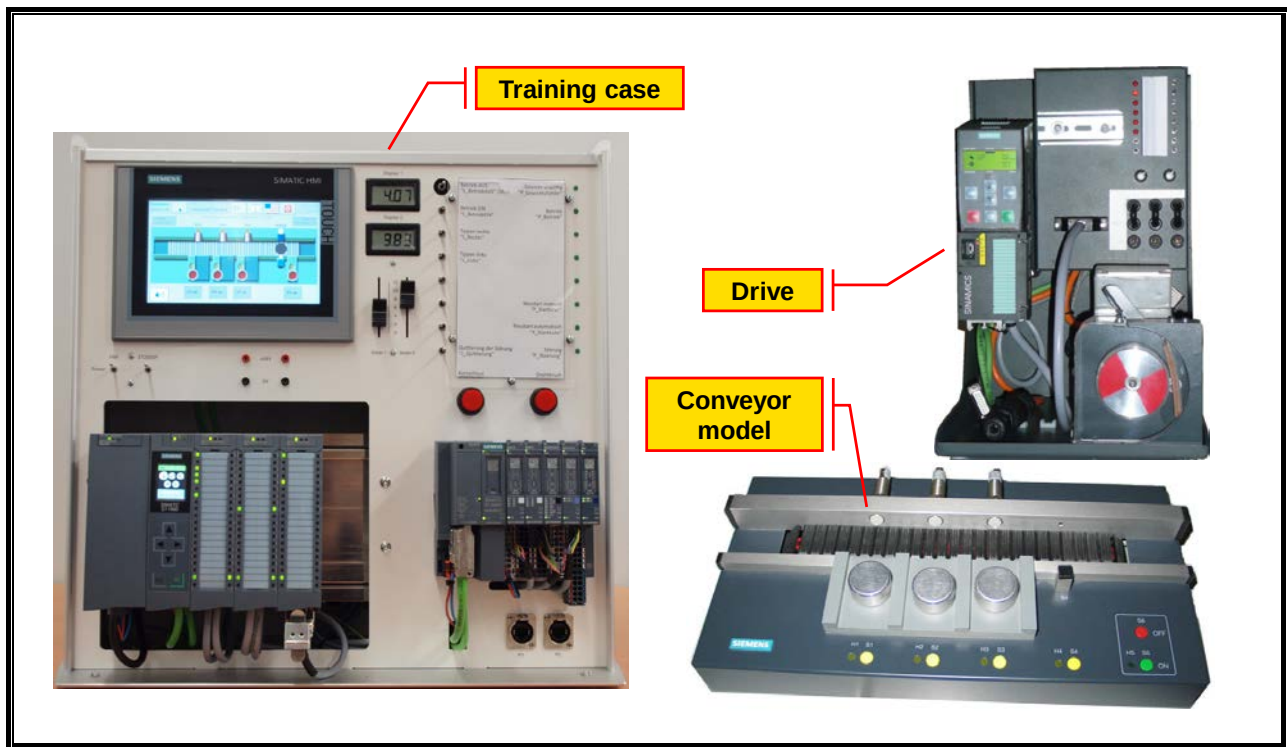
1. Training Devices and Addressing

At the end of the chapter the participant will ...



- ... be familiar with the principle setup of an S7-1500
- ... be familiar with the configuration of the training area
- ... be familiar with the wiring of the training area components

1.1. Training Area Setup with S7-1500

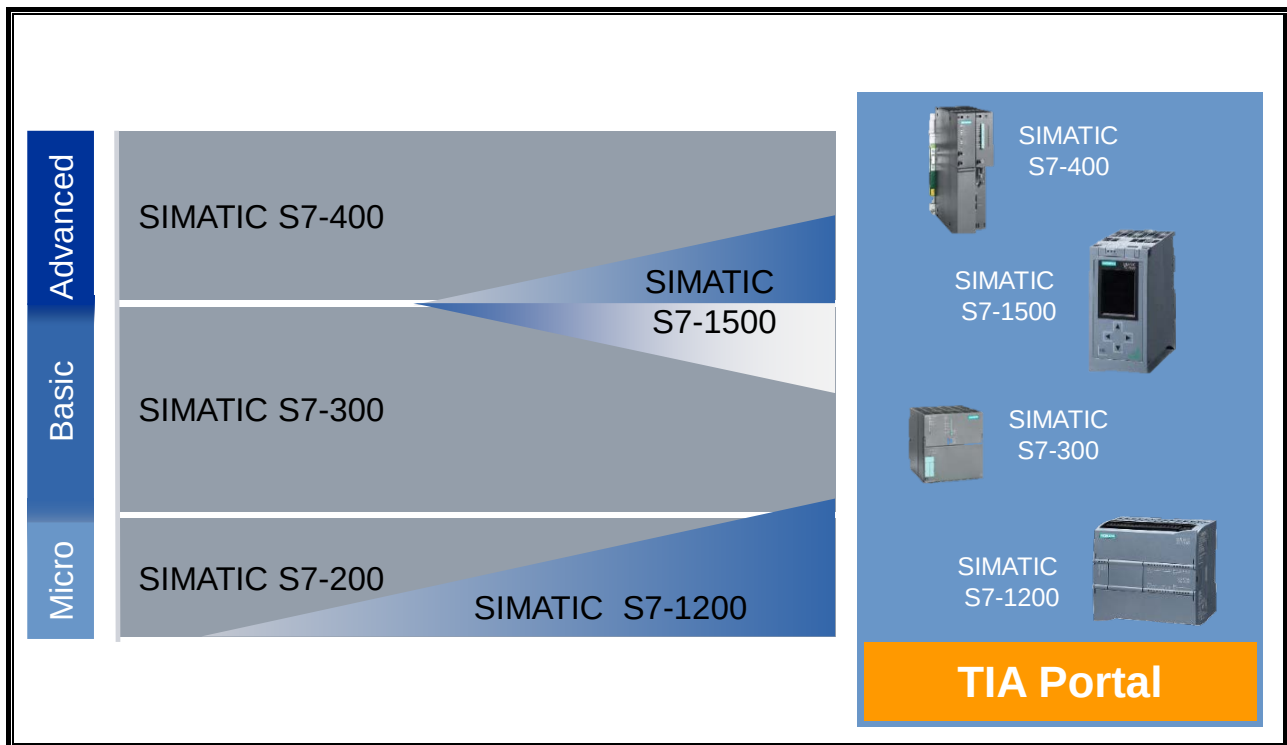


Components of the Training Area with S7-1500

The training area for this course contains the following components:

- SIMATIC Field PG
- Training case with S7-1500, ET200SP, touchpanel, operating elements (switches/pushbuttons and slide controls) and display elements (lights (LEDs) and voltage displays)
- Sinamics G120 drive
- Conveyor model

1.2. Positioning the Modular S7 Controllers



SIMATIC S7

The programmable logic controllers can be divided into the micro PLC (S7-1200) and the mid/upper (S7-1500) performance ranges.

The existing lower/mid performance range (S7-300) and the mid/upper performance range (S7-400) will be covered in future by an S7-1500 system.

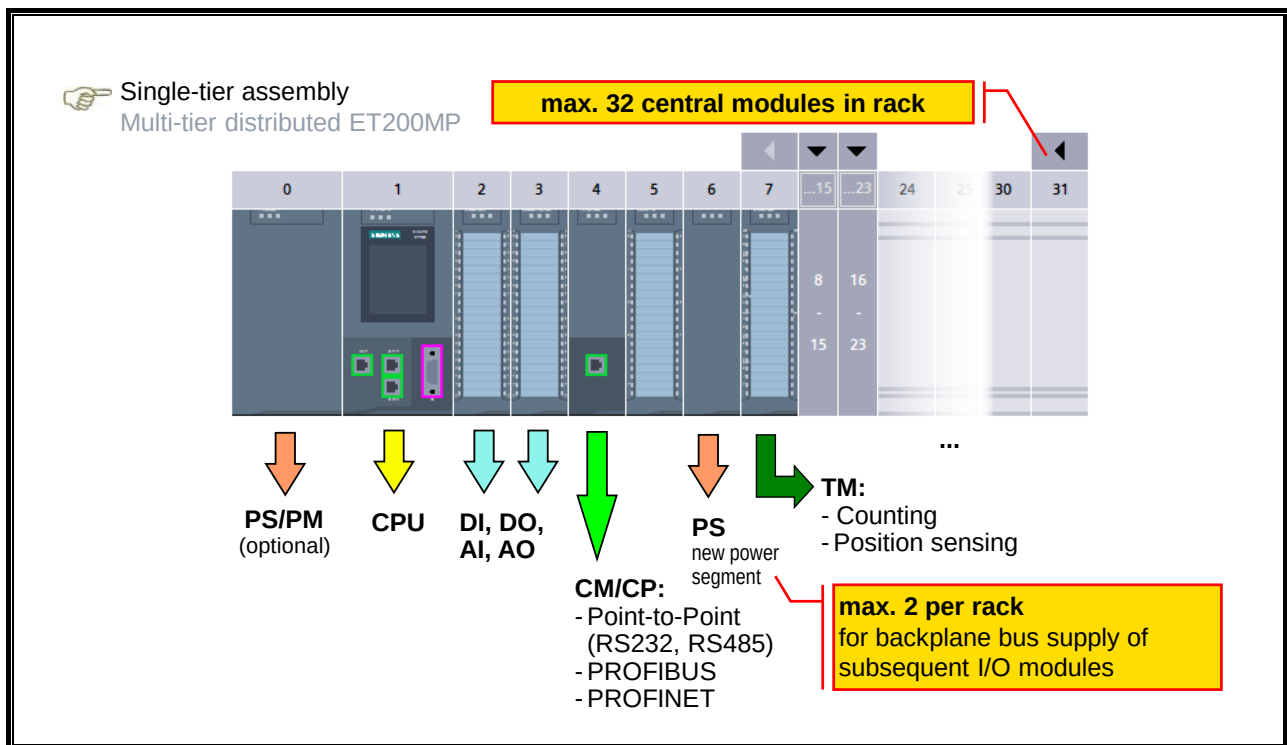
1.3. SIMATIC S7-1500: Modular Controller for the Mid to Upper Performance Range



Features

- Modular control system for the mid to upper performance range
- Scaled CPU range
- Display for basic CPU settings (system time, interfaces...) and for calling diagnostic and status information (diagnostic buffer, message display, CPU state...)
- Extensive range of modules
- High performance I/O bus for efficient process interfacing via central I/O
- Can be expanded to up to 32 modules in one tier
 - 👉 Currently only single-tier assembly possible. Multi-tier assembly using distributed ET200MP.
- Can be networked with PROFIBUS or PROFINET
- Slot rules
 - left of the CPU: 1x power supply (PM or PS)
 - right of the CPU: signal modules (digital, analog), technology modules, communication modules and further power supplies
- No slot rules for modules to the right of the CPU

1.3.1. SIMATIC S7-1500: Modules



Slot Rules

- 1x PS/PM Slot 0
- 1x CPU in Slot 1
- As of Slot 2 any

Signal Modules

- Digital input modules: 24VDC, 230VAC
- Digital output modules: 24VDC, 230VAC
- Analog input modules: voltage, current, resistance, thermocouple
- Analog output modules: voltage, current

Communication Modules (CP - Communication Processor, CM - Communication Module)

- Point-to-Point connection
- PROFIBUS
- PROFINET



CPs and CMs are both communication modules. CPs have, as a rule, somewhat more functionality than CMs (e.g. own web server, firewall, or the like).

Technology Modules (TM - Technology Module)

- Counting
- Position sensing

Power Supply

I/O modules in the central rack of the S7-1500 require a system power supply via the backplane bus (communication connection to the CPU) and a load power supply (input or output circuits for sensors/encoders and actuators).

- **PM - Power Module → Load Power Supply**

supplies modules with 24VDC for input and output circuits as well as sensors/encoders and actuators



If the CPU is supplied 24V via a load power supply (PM), it supplies the system power supply of 12W for the first inserted I/O modules.

- **PS - Power System → System Power Supply**

supplies S7-1500 modules in the central rack via the backplane bus



Each CPU offers a system power supply of 12W for the first inserted I/O modules. Depending on the I/O modules used, further power segments have to be set up, as required.



A system power supply (PS) can also supply the load circuit for 24VDC modules in addition to the CPU.

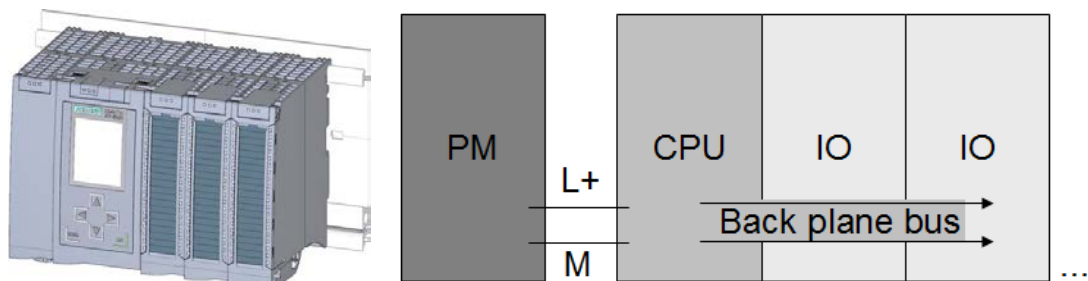
Power Supply and Power Segments of the I/O Modules

It is necessary to set up power segments in the central rack for larger configurations or configurations with greater I/O module power requirements (as a rule, when using CP, CM, TM).

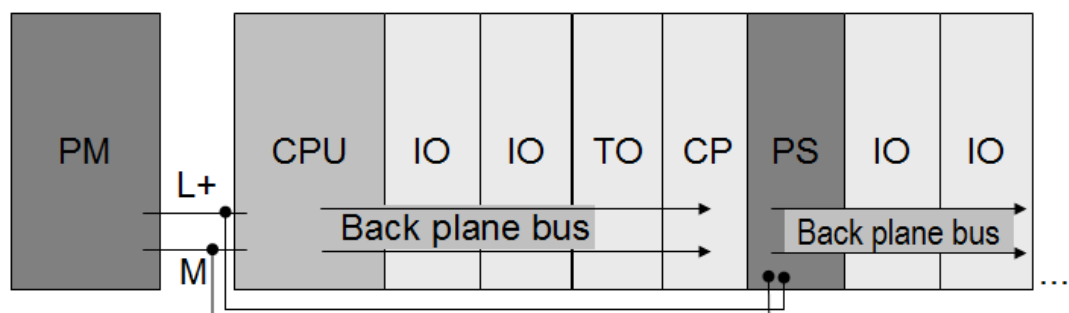
A maximum of 3 power segments can be set up per rack (1xCPU segment plus 2 more).

If the configuration includes additional power segments, additional system power supply modules (PS) are inserted to the right next to the CPU. The CPU continues to control all modules of the rack. Only the system power supply of the I/O modules is subdivided here.

Example of a Small S7-1500 Configuration



Example of an S7-1500 Configuration with a 2nd Power Segment



Interface Modules for Expansion Rack

A central multi-tier assembly is not planned. An expansion can be realized using the distributed ET200MP I/O system.

1.3.2. SIMATIC S7-1500: CPU-Display → Menu and Colors



Main menu items and their meaning:

-  **Overview** (CPU status: name, type, MLFB, version)
-  **Diagnostics** (display diagnostics buffer, message display)
-  **Settings** (CPU settings: addresses, time, operating mode, CPU Reset, protection level, unlock Display when password set)
-  **Modules** (status information of modules in the system)
-  **Display** (display settings: brightness, display language, energy-saving times, MLFB, version)

Colors of status information and their meaning:

- green** RUN of CPU, RUN with pending alarm message
- yellow** STOP or CPU HOLD
- red** Error
- white** Connection setup or connection to CPU lost



Depending on the current CPU state, the color on the Display varies.

Available Display languages are the available user interface languages of STEP7.

Additional Symbols in the Status Information



Password is configured but not entered



Password is configured and entered



An Alarm exists



A Force job is active on the CPU

1.3.3. SIMATIC S7-1200/1500: Technology Functions

The screenshot displays the SIMATIC TIA Portal Technology Functions library. The left pane shows the 'Instructions' tree with 'Technology' expanded, listing 'High_Speed_Counter', 'PID Control', 'Time-based IO', and 'Motion Control'. The right pane shows the 'Technology' list with 'PID Control' and 'Motion Control' expanded. The 'PID Control' section includes 'Compact PID' (PID_Compact, PID_3Step, PID_Temp) and 'PID Basic functions' (CONT_C, CONT_S, PULSEGEN, TCONT_CP, TCONT_S). The 'Motion Control' section includes 'S7-1500 Motion Contr...' (MC_Power, MC_Reset, MC_Home, MC_Halt, MC_MoveAbsolute, MC_MoveRelative, MC_MoveVelocity, MC_MoveJog, MC_GearIn, MC_MoveSuperim...) and 'Time-based IO'. A central diagram shows a 'PID Control' block with a feedback loop. A red arrow points from the 'PID Control' block to the 'PID Control' section in the right pane. Another red arrow points from the 'High-Speed Counter' block to the 'High_Speed_Counter' entry in the left pane. A third red arrow points from the 'Motion Control' block to the 'Motion Control' section in the right pane. The 'Motion Control' block also features a 'PLCopen motion control' logo.

High-speed Counters

High-speed counters, (e.g. with 100 kHz, depending on the used hardware) are available for precise monitoring of incremental encoders, frequency counting or counting of highly frequent process events.

Motion

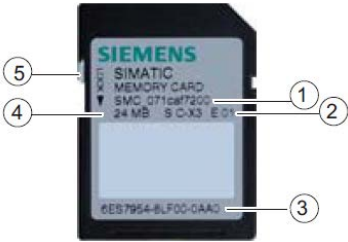
To control rotary speed, position or pulse duty factor, PWM outputs (pulse width modulation) are available. Application examples are, for example, controlling the speed of a motor, position of a valve, or the mark-to-space ratio for a heating element. For rotary speed and position controls, PTO (pulse train output) outputs at 100 kHz (depending on the used hardware) are available.

It supplies a pulse train for controlling speed and position of stepper or servo motors.

PID

For simple closed-loop control tasks, PID control circuits with automatic PID adjustment and tuning control panel are available.

1.3.4. SIMATIC S7-1200/1500: Memory Card



① Serial number of the SMC
 ② Product version
 ③ Order (Article) number
 ④ Card size
 ⑤ Slide switch for write-protect
 (must not be write-protected)

SIMATIC Memory Card in the S7-1200 / S7-1500:

- Load memory
- Distribution of programs (only S7-1200)
- Firmware update
- Documentation
- Memory Card Binding
- Unlinked DBs
- Archiving of data
- Module exchange without PG

Write with:

- Commercially available SD card reader
- Field PG

Memory Card Binding – Copy Protection

The executability of the program can be bound to the serial number of the card.

Load Memory

- S7-1500
This has no integrated load memory and therefore it is imperative that a card is inserted.
- S7-1200
This has an integrated load memory. Here, an inserted memory card can replace the integrated load memory or the card can be used for program updates (distribution of programs).

Distribution of Programs ← only S7-1200

The use as Transfer card (card mode = "Transfer") is only supported by the S7-1200. Here, a program can be downloaded into the CPU without a PG if a card is inserted.

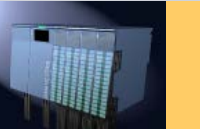
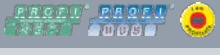
Archiving of Data

It is possible to archive process values on the card.

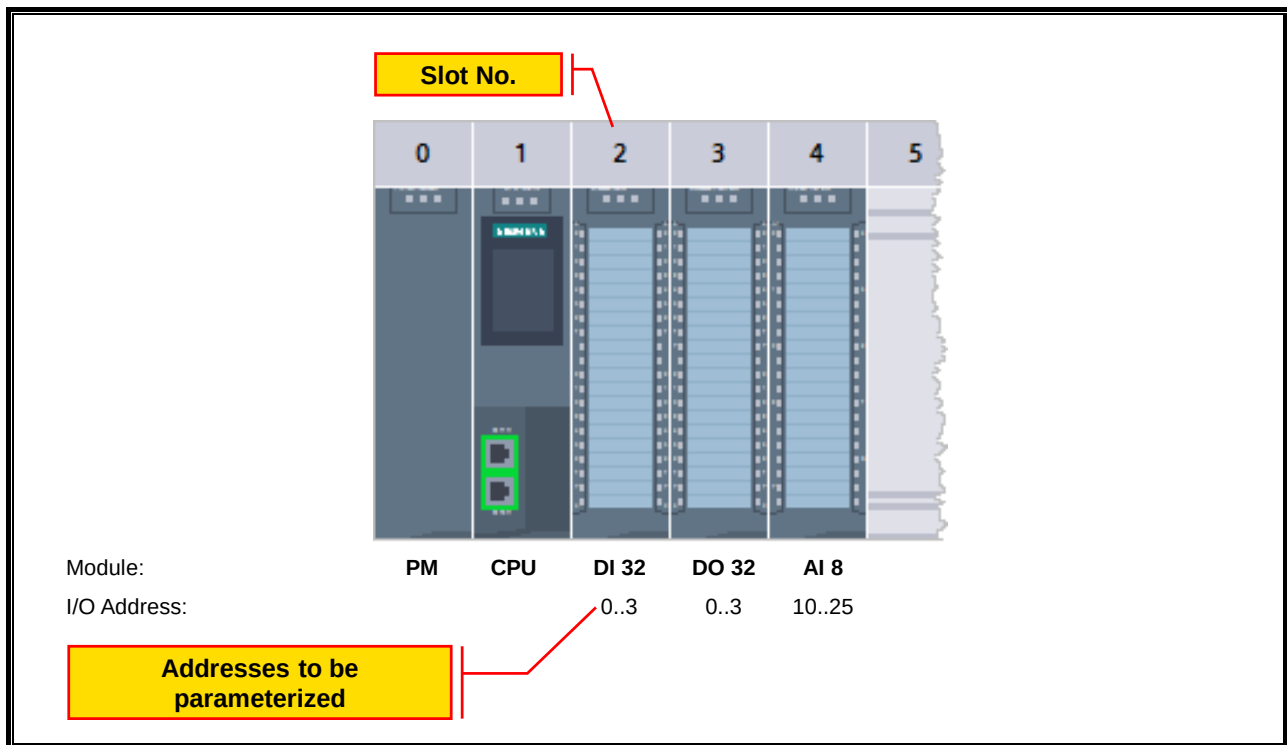


The use of this functionality affects the operating life of the Memory Card

1.4. Distributed I/O Systems

ET 200M	 	ET 200MP	 	Distributed Installation With modules of the central installation - Front connector - Single-wiring technology - Optimized width
ET 200S	 	ET 200SP	 	Distributed Installation - Fixed wiring and multi-wiring technology - Tool-free, fast connection technology - Optimized size One distributed I/O for various market segments.
ET 200iSP	 	ET 200iSP To be continued		Distributed Installation - 24 VDC and 230 VAC power supply Functionality for process automation Installation in applications for Ex-Zone 1

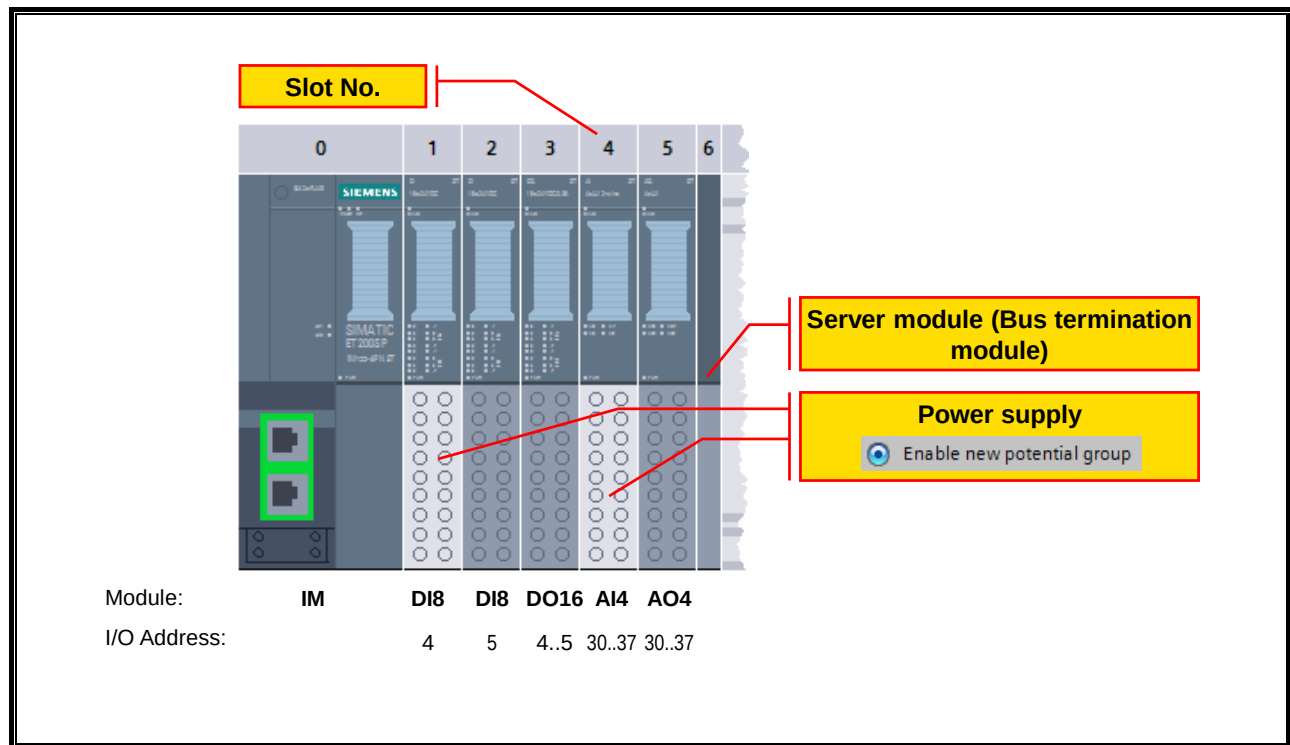
1.5. Configuration of the S7-1500 Training Device



Addresses of the Central S7-1500 I/O Modules

Two digital 32-channel modules are available as central I/O. These are to begin as of Address =0. Since digital channels are also available on the distributed I/O, the analog module of the central I/O is to begin as of Address =10.

1.6. Configuration of the ET200SP Training Device

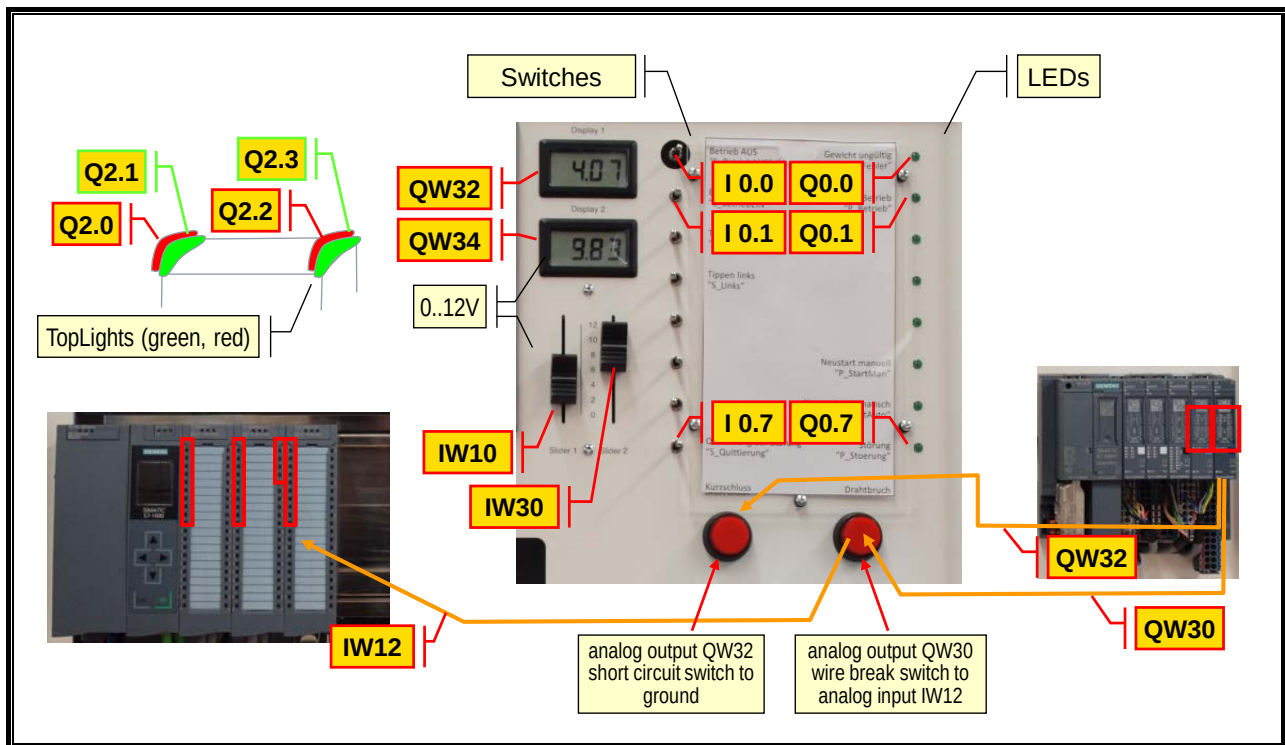


Addresses of the Distributed ET200SP I/O Modules

Three digital modules are available as distributed I/O. These are to follow the central digital I/O in the address space as of Address =4.

The analog distributed I/O is to begin as of Address =30.

1.7. Operating and Display Elements of the Training Device



Operating Elements

In addition to the touchpanel, separate operating elements are also available for operating the system:

- 8 switches
- 2 potentiometers for setting or simulating analog input signals
- Wire break switch that interrupts the connection AO1 distributed I/O to AI2 central I/O
- Short circuit switch that short-circuits the AO2 of the distributed I/O to ground

Display Elements

In addition to the touchpanel, separate display elements are also available for the visualization of process information:

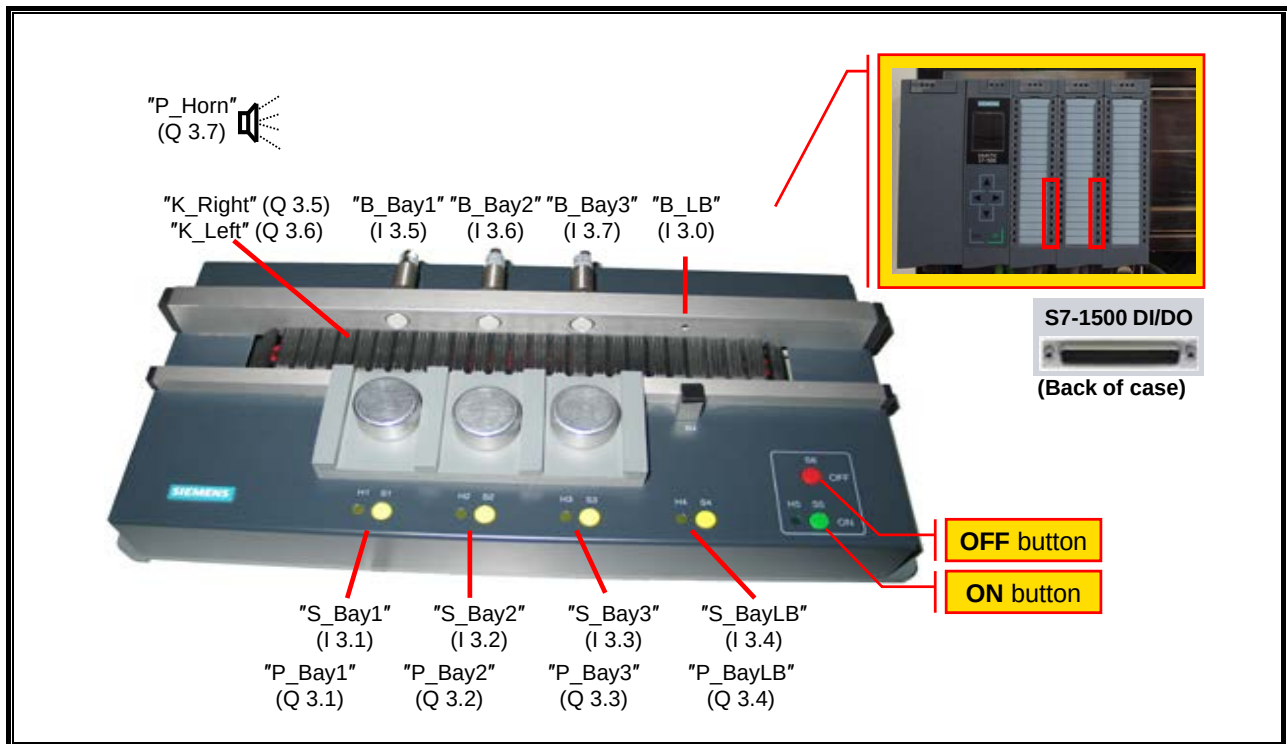
- 8 LEDs
- 2 digital voltage displays for displaying analog output signals
- On top of the training device there are, to the right and left, 2 LED bars "TopLights" (2x green, 2x red). These can be controlled by means of 4 DOs.

Addressing

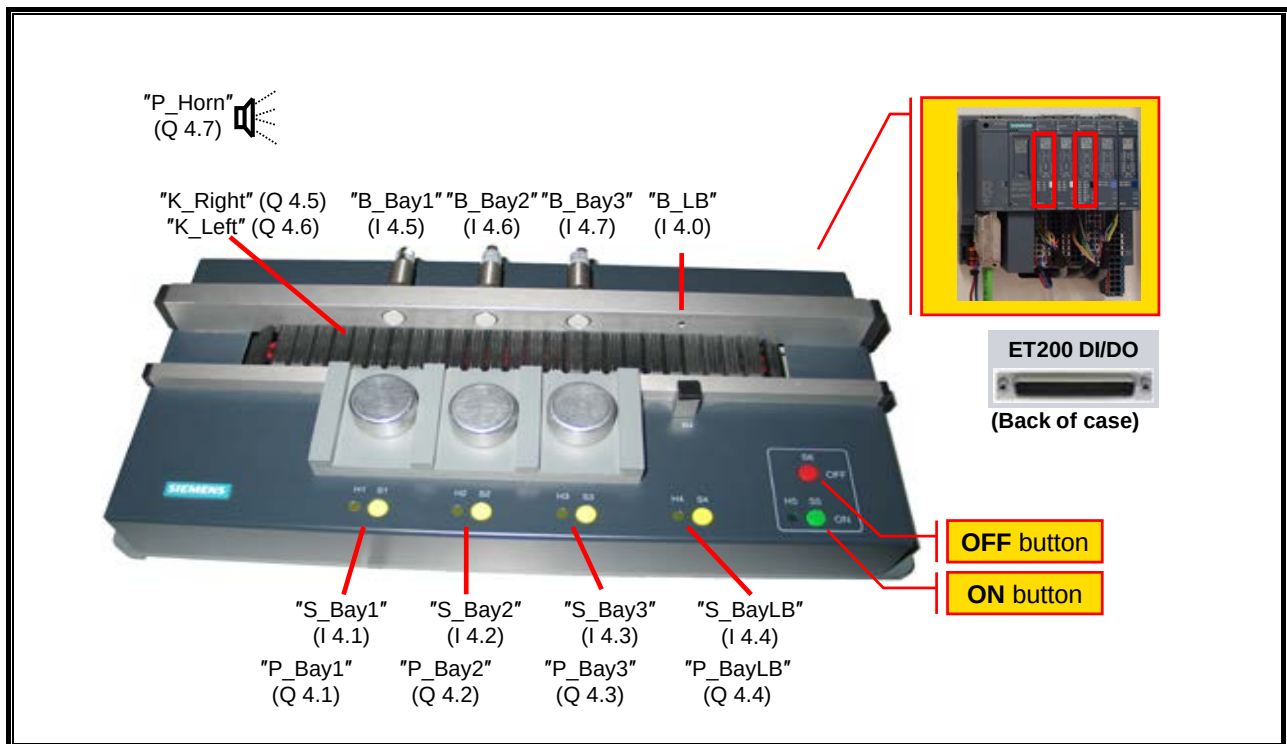
For the addressing shown in the picture, the relevant module address settings must be made in the device configuration.

1.8. Setup and Connection of the Conveyor Model

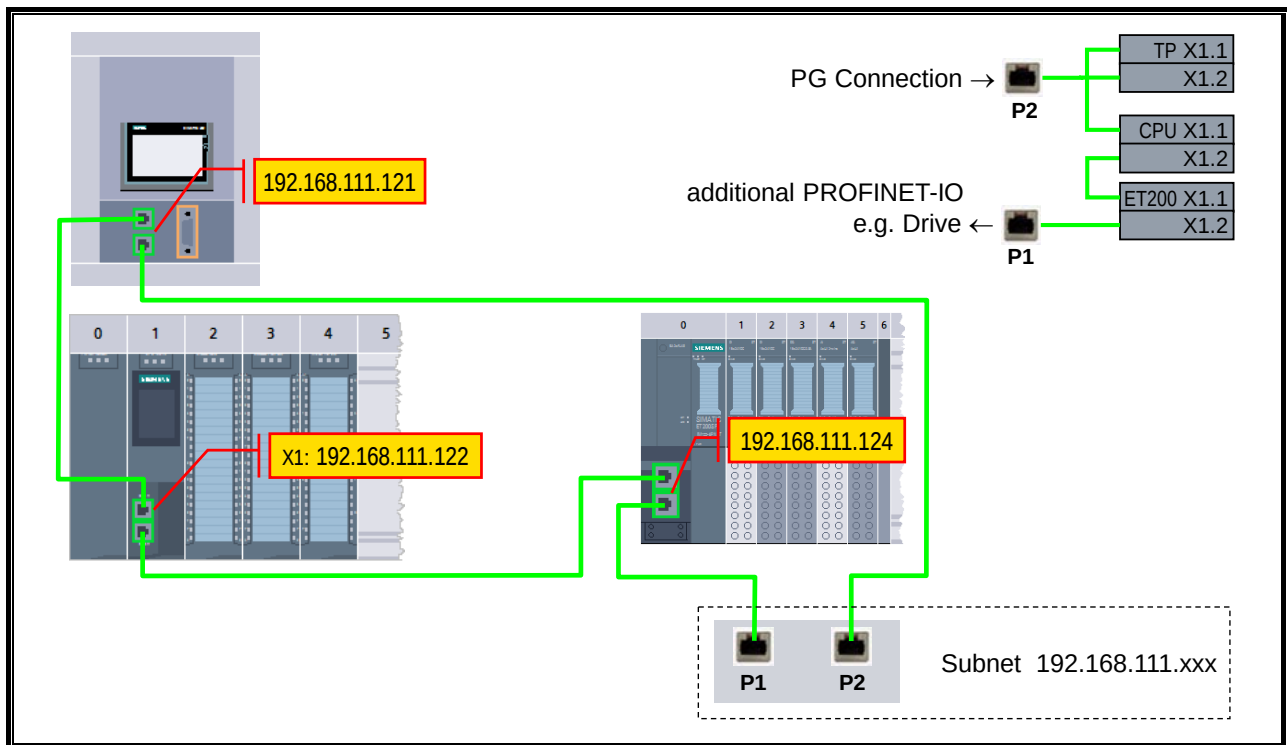
1.8.1. Connection to Central I/Os of the S7-1500



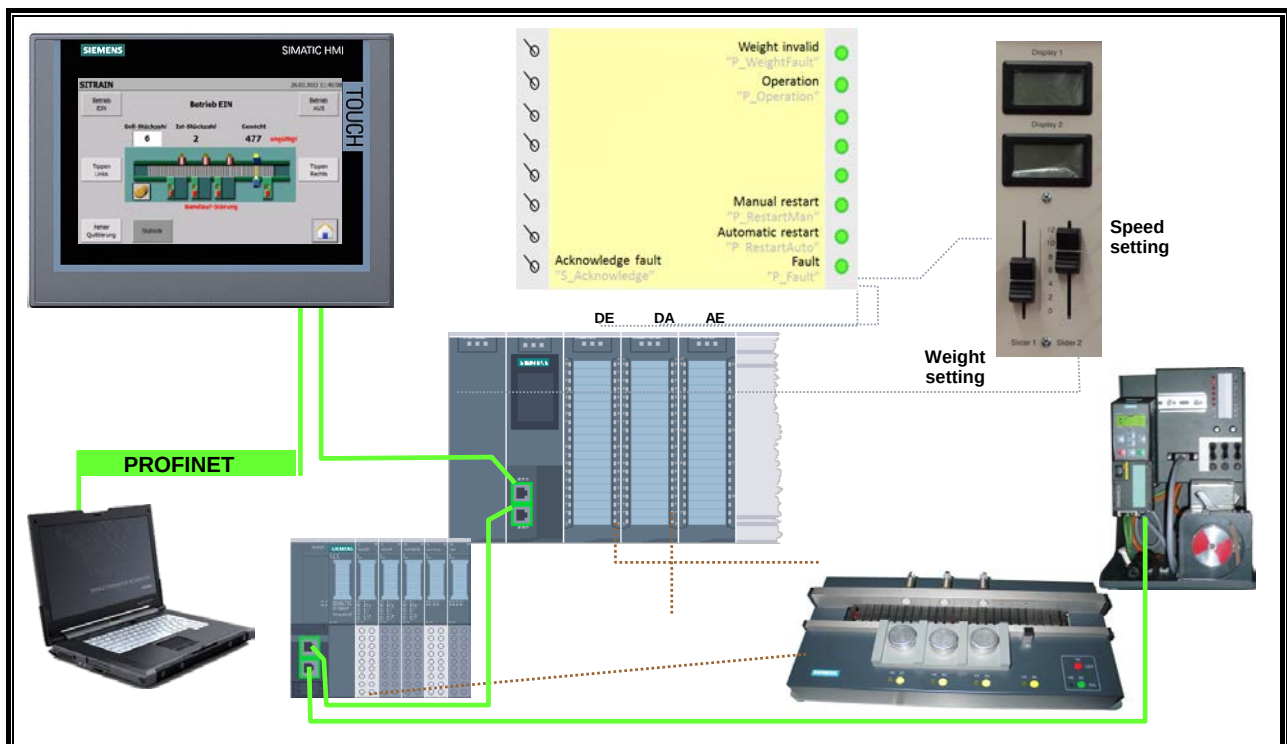
1.8.2. Connection to Distributed I/Os of the ET200SP



1.9. Networking and IP Addresses of the Modules



1.10. Training Area as Plant with Distribution Conveyor and Touchpanel



A functional description follows on the next page

Functional Description

The distribution conveyor is used to transport parts and can be operated in two different operating modes.

For now, the simulator switches are used to select the operating mode and later, the associated buttons on the touchpanel.

Operation Switched Off "P_Operation" (Q0.1) = Off

Now, the conveyor motor can be jogged to the right and left. For now, the simulator switches are to be used for this, later, the associated buttons on the touchpanel.

Operation Switched On "P_Operation" (Q0.1) = On

When operation is switched on, parts are placed on the conveyor at the light barrier bay, weighed (weight simulation via the left slider on the simulator) and, after pressing the bay pushbutton, are transported to the next free Bay 1, 2 or 3 if a bay is free and the weight is in the allowed range. When a part reaches the bay, the indicator light signals with a flashing light that the part can be removed. After removing the part, a continuous light signals that the bay is still occupied. By acknowledging using the associated bay pushbutton, the bay indicator light goes dark and the bay is once again free.

Every transport sequence is monitored for time. If it takes longer than 6 seconds, there is a fault and the conveyor motor is automatically switched off. Only after acknowledging the fault via "S_Acknowledge" or on the touchpanel, can a new transport sequence be started.

The transported parts are counted for each bay and the statistical data - total quantity, the utilization of the 3 bays, the current weight, the total weight of all transported parts as well as the maximum, minimum and average weight - is determined and displayed.

If none of the Bays 1 to 3 is free or the weight storage for saving the part weight is full or the simulated weight is too high, no part transportation can be started. These statuses as well as an occurring I/O Device failure are output on the touchpanel as a message.

The maximum quantity of parts to be transported can be set. If this is reached, it is also not possible to start part transportation.

Only after operation has been switched off and subsequently switched back on and, with that, the *Actual Quantity* has been reset to 0, all stored part weights have been overwritten with 0 as well as the statistic values have been reset to 0, can new part transports once again be started.

Integrating the G120 Drive

The G120 drive is controlled as if it would drive the conveyor. The speed of the motor can be preset via the right slider on the simulator.

Contents

2.	Hardware Commissioning	2-3
2.1.	Task Description Hardware Commissioning: Commissioning an S7-1500, ET200SP and Touchpanel	2-4
2.2.	TIA Portal: Portal View and Project View.....	2-5
2.2.1.	Portal View	2-6
2.2.2.	Project View	2-7
2.3.	Components of the "Devices & Networks" Editor	2-8
2.3.1.	Devices & Networks: Device View (Hardware Configuration)	2-9
2.3.2.	Hardware Catalog	2-10
2.4.	Setpoint and Actual Configuration	2-11
2.4.1.	Setpoint Configuration: Creating a Hardware Station.....	2-12
2.4.2.	Inserting / Deleting a Module	2-13
2.4.3.	Symbols of the Module Channels	2-14
2.4.4.	Changing a Module.....	2-15
2.4.5.	Uploading the Actual Configuration into the Project (1): Inserting an Unspecified CPU ...	2-16
2.4.6.	Uploading the Actual Configuration into the Project (2): Detecting Accessible Devices ...	2-17
2.4.7.	Uploading the Actual Configuration into the Project (3): Reading-out the Actual Configuration.....	2-18
2.5.	Uploading a Device as a New Station (1) (Hardware with Parameterization and Software)2-19	
2.5.1.	Uploading a Device as a New Station (2) (Hardware with Parameterization and Software)2-20	
2.6.	Compiling the Hardware Configuration and Downloading It into the CPU	2-21
2.7.	Online Tools	2-22
2.7.1.	Online Connection via Industrial Ethernet: IP Address and Subnet Mask	2-23
2.7.2.	Establishing an Online Connection: Assigning an IP Address for the PG.....	2-24
2.7.2.1.	Windows7 Operating System.....	2-24
2.7.3.	Online Access: Accessible Devices in the Portal View.....	2-25
2.7.4.	Accessible Devices in the Project View: CPU Online Access: Reading-out the Diagnostics Buffer.....	2-26
2.7.5.	Accessible Devices in the Project View: CPU Online Access: CPU-RUN/STOP, Memory Reset (MRES)	2-27
2.7.6.	Online Access to the CPU: IP Address, Name, Time, FW Update, Format Memory Card	2-28
2.8.	CPU Memory Reset (MRES) using the Mode Selector Switch	2-29
2.8.1.	SIMATIC S7-1200/1500: Memory Concept for CPU Memory Reset.....	2-30
2.8.2.	SIMATIC S7-1200/1500: Memory Concept for CPU Reset to Factory Settings.....	2-31
2.9.	SIMATIC Card Reader	2-32
2.10.	Task Description: Creating a Project with an S7-1500 Station.....	2-33
2.10.1.	Exercise 1: Deleting Old Projects	2-34
2.10.2.	Exercise 2: Connecting the PG and Setting the IP Address of the PG	2-35
2.10.3.	Exercise 3: Erasing the SIMATIC Memory Card of the CPU.....	2-36
2.10.4.	Exercise 4: Resetting the CPU using the Mode Selector Switch.....	2-37
2.10.5.	Exercise 5: Determining the CPU Firmware Version and Assigning the IP Address (Node Initialization)	2-38
2.10.6.	Exercise 6: Creating a New Project	2-39
2.10.7.	Exercise 7: Creating the S7-1500 Station.....	2-40
2.10.8.	Exercise 8: Reading-out the Actual Configuration	2-41
2.10.9.	Exercise 9: CPU Properties: Parameterizing the Clock Memory Byte	2-42
2.10.10.	Exercise 10: CPU Properties: Parameterizing the Display Language and Display Protection2-43	
2.10.11.	Exercise 11: Addresses of the DI Module.....	2-44

2.10.12.	Exercise 12: Addresses of the DO Module	2-45
2.10.13.	Exercise 13: Addresses of the AI Module	2-46
2.10.14.	Exercise 14: Compiling the Device Configuration and Downloading It into the CPU	2-47
2.11.	Task Description: Commissioning the ET200SP	2-48
2.12.	Fieldbus Systems for SIMATIC S7	2-49
2.13.	Components of the PROFINET Standard	2-50
2.13.1.	PROFINET IO Device Types	2-51
2.13.2.	PROFINET Addresses	2-52
2.13.3.	Inserting and Networking Distributed I/O	2-53
2.13.4.	PROFINET IO Device ET200SP: Assigning the IP Address and Device Name <u>OFFLINE</u>	2-54
2.13.5.	PROFINET IO Device ET200SP: Assigning the Device Name <u>ONLINE</u>	2-55
2.13.6.	Exercise 15: ET200SP: Reset to Factory Settings	2-56
2.13.7.	Exercise 16: Reading-out the Firmware Version of the ET200SP	2-57
2.13.8.	Exercise 17: Offline Project: Adding the ET200SP	2-58
2.13.9.	Exercise 18: Networking the ET200SP	2-59
2.13.10.	Exercise 19: Configuring and Parameterizing the ET200SP	2-60
2.13.11.	Exercise 20: ET200SP: Assigning the IP Address / PROFINET Name <u>OFFLINE</u>	2-61
2.13.12.	Exercise 21: ET200SP: Assigning the PROFINET Name <u>ONLINE</u>	2-62
2.13.13.	Exercise 22: Copying a PLC Tag Table	2-63
2.13.14.	Exercise 23: Compiling the HW Configuration and Downloading It into the CPU	2-64
2.13.15.	Exercise 24: Testing the Wiring of the Conveyor Model's Sensors and Actuators	2-65
2.14.	Task Description: Commissioning the Touchpanel	2-66
2.15.	Adding an HMI Device	2-67
2.16.	Configuring the IP Address of a Touchpanel	2-68
2.16.1.	Networking a Touchpanel	2-69
2.16.2.	Configuring an HMI Connection	2-70
2.16.3.	Setting the IP Address on the Touchpanel	2-71
2.16.4.	Downloading the HMI Project into the Touchpanel	2-72
2.17.	Exercise 25: Copying the Touchpanel Project and the Interface Data Block from the Library	2-73
2.17.1.	Exercise 26: Networking the Touchpanel	2-74
2.17.2.	Exercise 27: Configuring the HMI Connection	2-75
2.17.3.	Exercise 28: Compiling and Saving the HMI Project	2-76
2.17.4.	Exercise 29: Setting the IP Address on the TP	2-77
2.17.5.	Exercise 30: Downloading the HMI Project into the Touchpanel	2-78
2.17.6.	Exercise 31: Carrying Out a Function Test	2-79

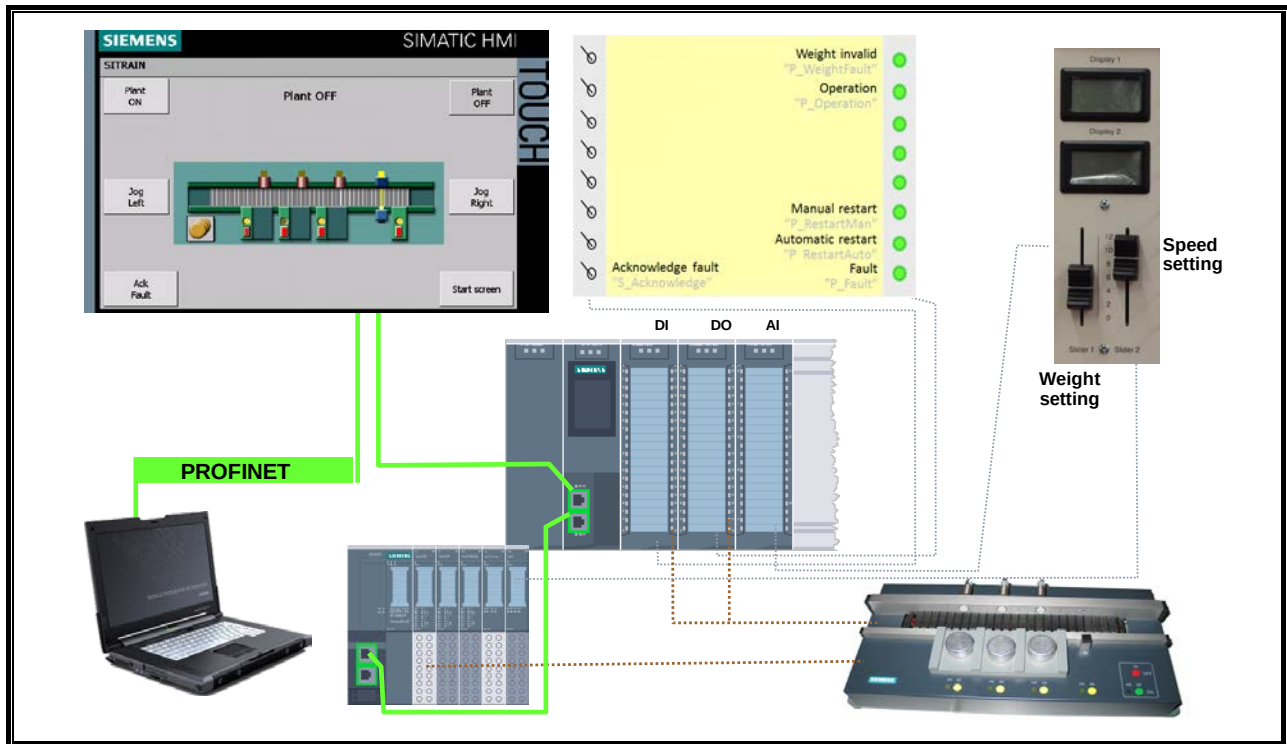
2. Hardware Commissioning

At the end of the chapter the participant will ...

- ... be able to establish an online connection to the controller
- ... be able to create a hardware station and assign parameters to it
- ... be able to integrate a PROFINET station and assign parameters to it
- ... be able to carry out a wiring test using the "Monitor/Modify Variables" tool
- ... be able to commission a touchpanel via PROFINET



2.1. Task Description Hardware Commissioning: Commissioning an S7-1500, ET200SP and Touchpanel



Task Description

The hardware components of the training area are to be commissioned. These are

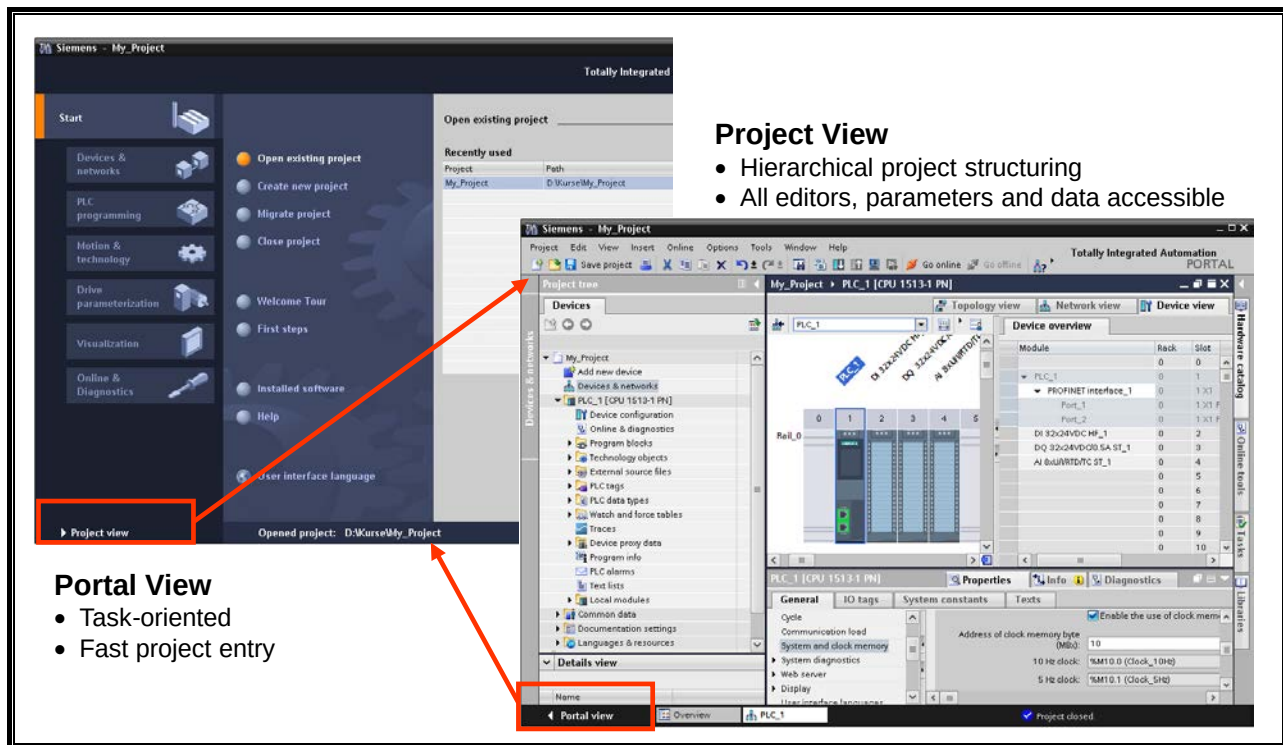
- S7-1500 station central device
- ET200SP distributed I/O station
- TP700 Comfort touchpanel
- Conveyor model

The wiring of the conveyor model to the ET200SP and the communication of the touchpanel with the S7-1500-Station is also to be checked.

Note:

The G120 drive, which is also a part of the training area, will be commissioned later.

2.2. TIA Portal: Portal View and Project View



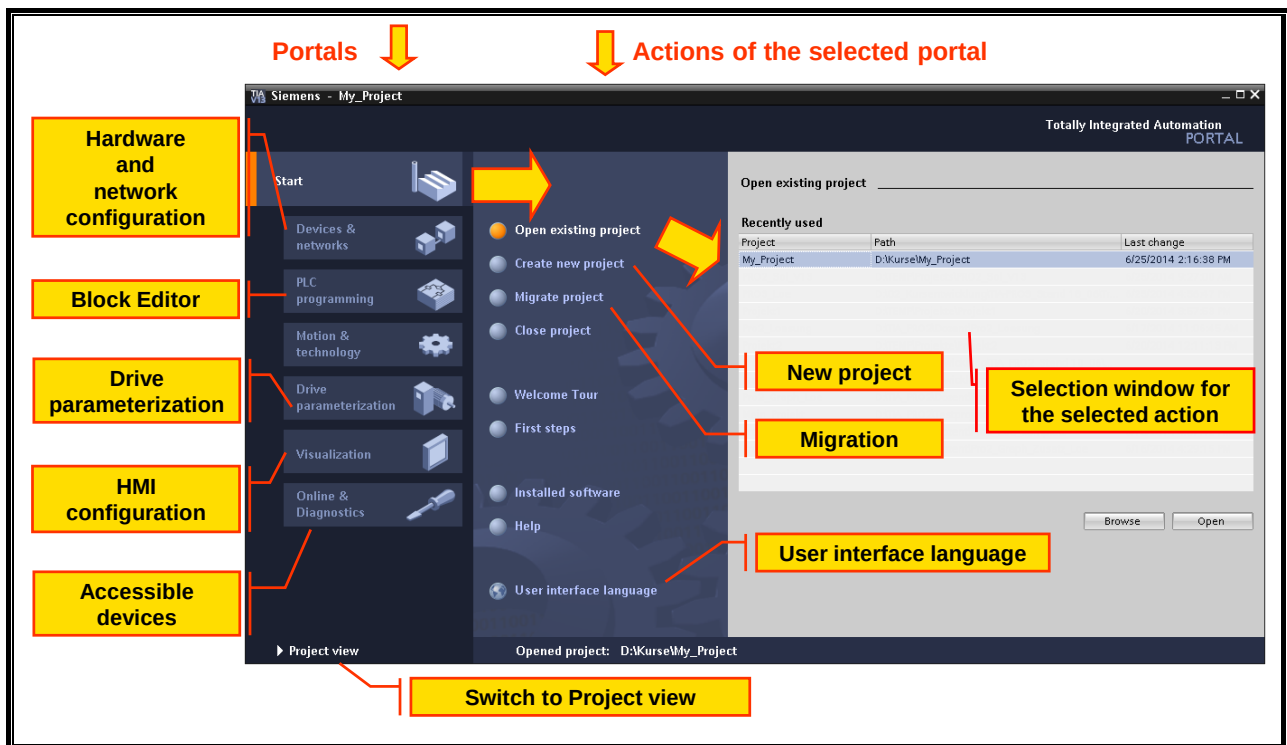
Portal View

- Task-oriented mode of working
- Fast project entry through user guidance

Project View

- Hierarchical structuring of the project
- The necessary editors open according to the task in hand
- All editors, parameters and data are found in one view

2.2.1. Portal View



Layout of the Portal View:

- Portals for the different tasks
- Actions for the selected portal
- Selection window for the selected action

Portals

Access to devices, components and their connections.

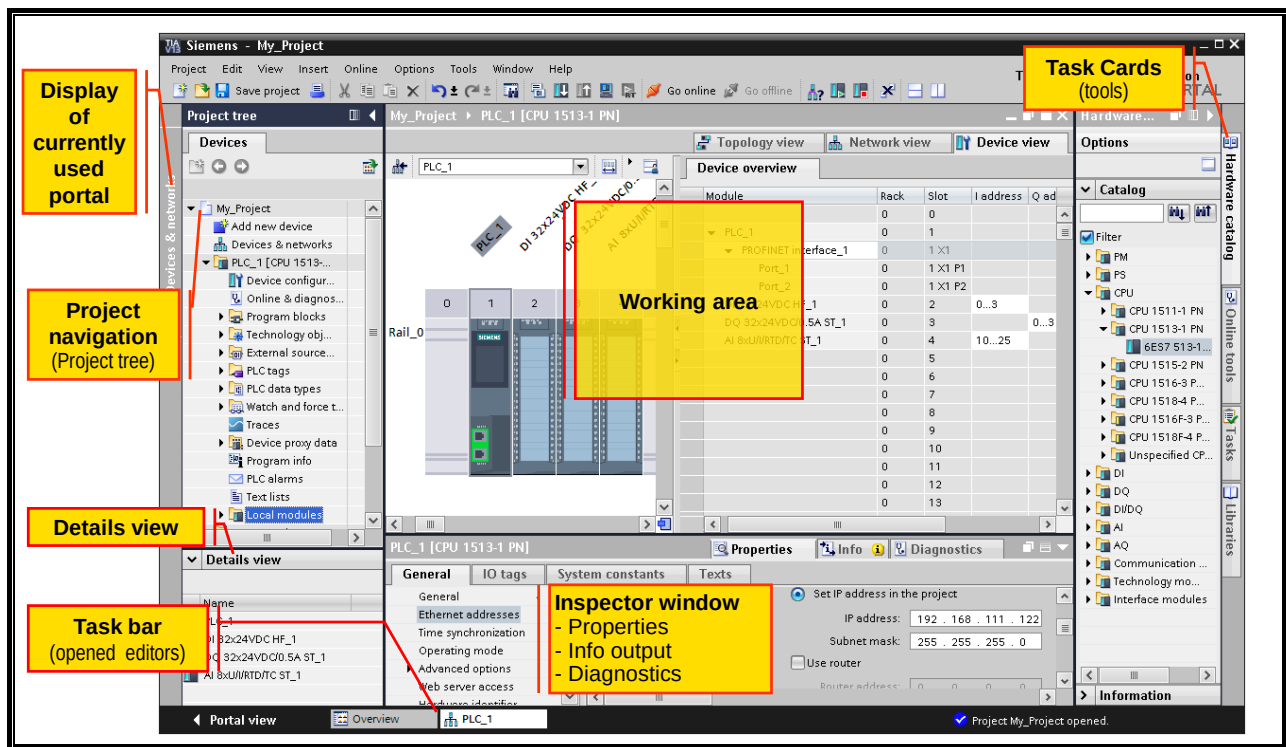
Actions

Depending on the selected portal, actions are available here that can be executed in the selected portal. Context-sensitive help is available in every portal.

Selection Window

The selection window is available in all portals. The content of the window adapts to your current selection.

2.2.2. Project View



Project Navigation (Tree)

The Project tree contains all components and project data of an automation solution. All components can be opened from there.

Working Area

The objects opened for editing are displayed in the working area. These objects include, for example hardware components, blocks, PLC tag tables, screens of HMI devices etc. If several objects are open at the same time, they are displayed as tabs in the task bar.

Task Cards

These provide tools for configuring/programming. The content of the Task cards depends on the object displayed in the working area.

If a hardware station is open, the Hardware catalog, for example, is available as a Task card. If a program block is open, there is a Task card with Instructions.

Inspector Window

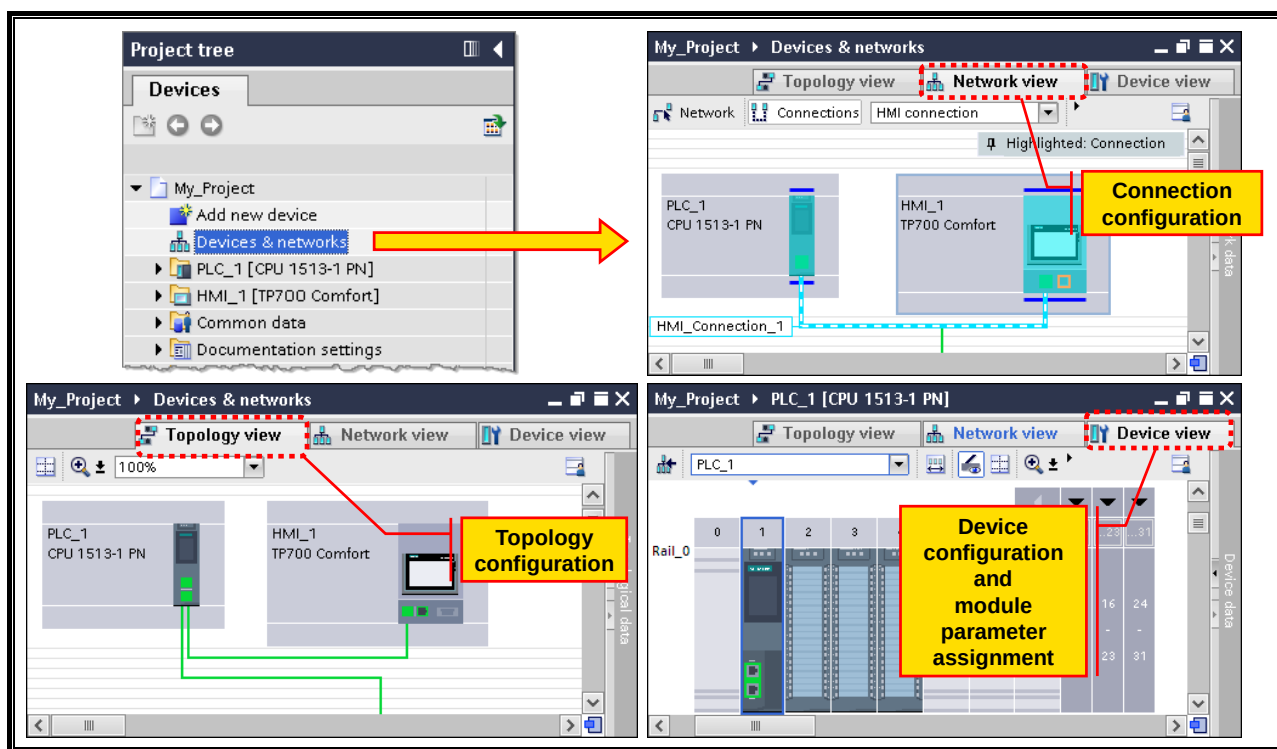
Additional information on a selected object or on executed actions is displayed in the Inspector window. The available properties of the selected objects can also be edited here (for example, properties of screens, screen objects, tags).

The Inspector window displays all system messages from the engineering, for example, those resulting from generating a project. This window should always be checked for any errors and warnings after a generation is completed.

Details View

The Details view is a help window. Here, the elements of the configuration object selected in the Project tree are displayed. These can be used in the active working area (by dragging them to the working area using drag & drop). This enables fast access to the required objects (for example, tags).

2.3. Components of the "Devices & Networks" Editor



Components of the Devices & Networks Editor

The "Devices & networks" editor consists of a Device, Network and Topology view.

Network View

The network view is used for networking devices.

- Connection configuration

Device View

The device view is used for configuring devices.

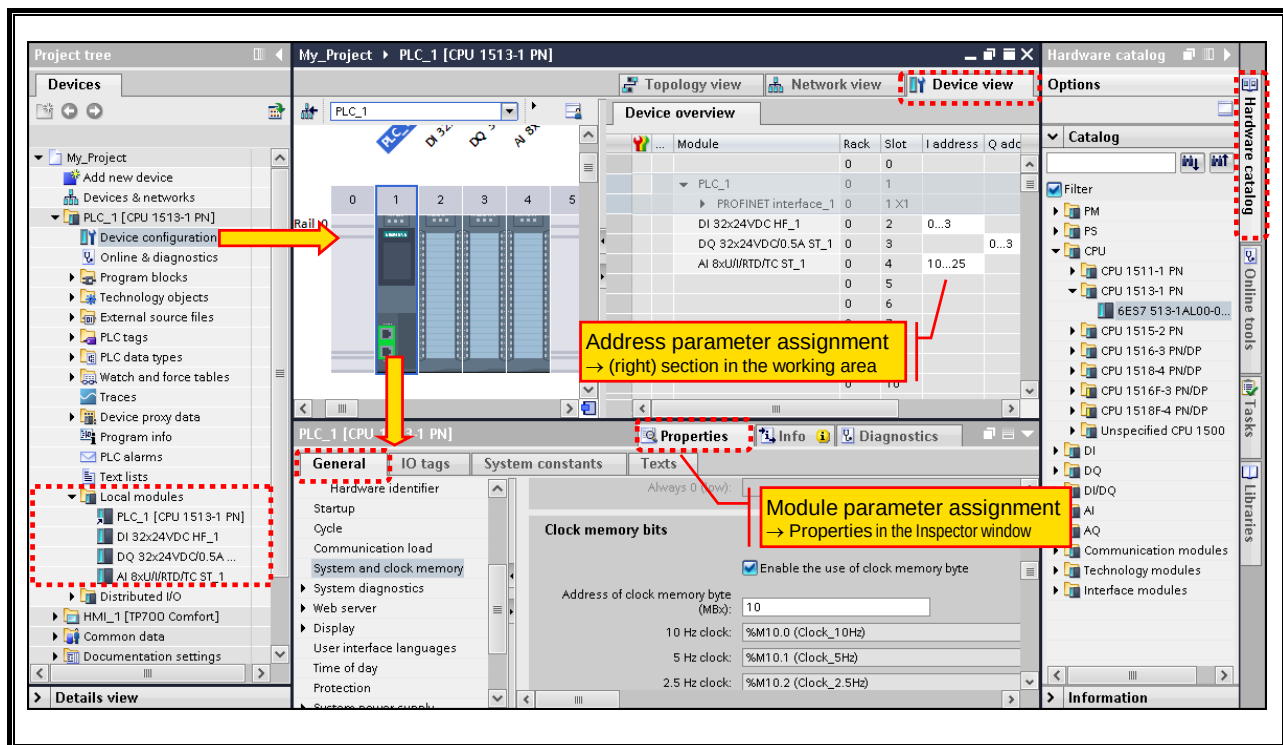
- Hardware configuration
- Module parameter assignment

Topology View

The topology view is used to determine the physical structure of networks.

- Configuration of the interface assignment and the relationship between devices

2.3.1. Devices & Networks: Device View (Hardware Configuration)



Creating/Editing a Hardware Configuration

- "Devices & networks" editor → select module → select "Device view" tab
- Open the "Device configuration" editor of the module from the Project tree

Components of the "Device configuration" Editor

- **"Device view" section** in the working area
This editor is made up of 2 sections. The right section can be opened/closed by double-clicking the



buttons, or, the splitting of the two sections can be dragged to the desired width using the mouse.

- left area = module configuration
- right area = address parameter assignment of the configured modules

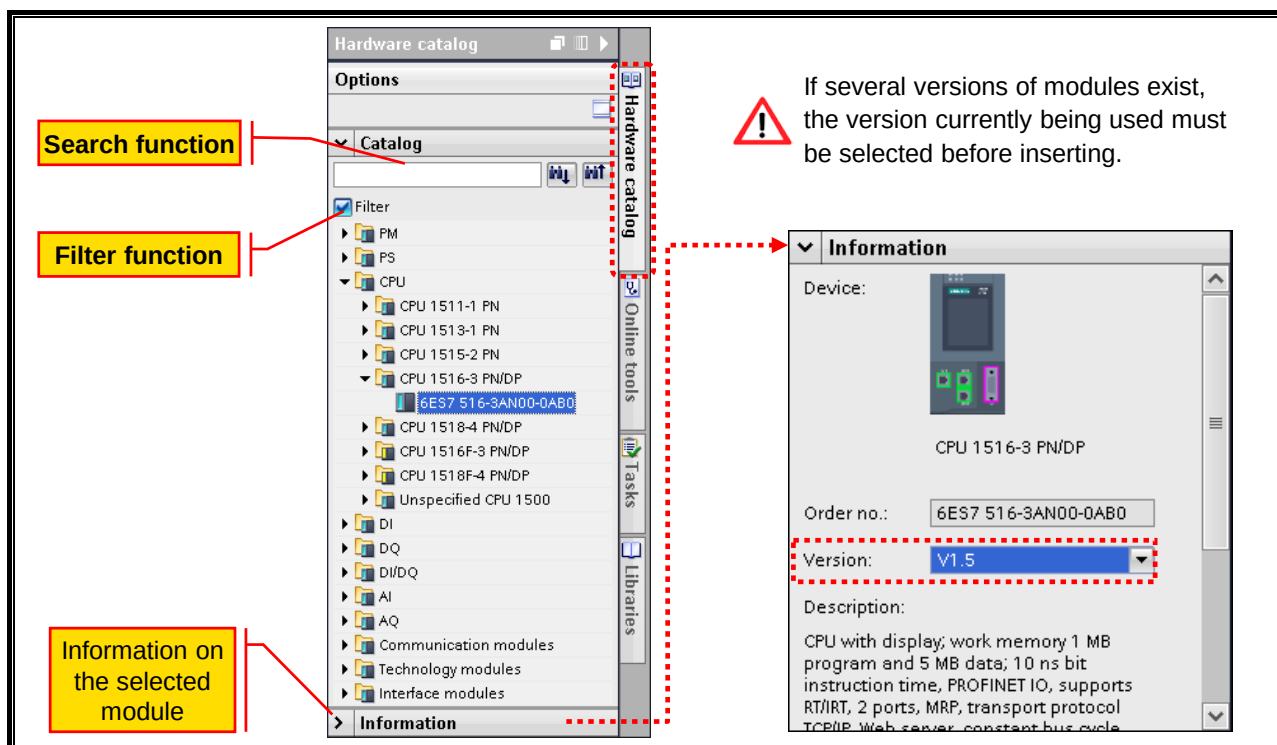
- **"Properties" tab** in the Inspector window
This tab is used to assign parameters to the module selected in the working area. Here, all the properties or parameters of the selected module are displayed and can also be modified. In the left-hand part of the Properties tab, there is a navigation section in which the parameters are arranged in groups.

- **"Hardware catalog" task card**
Module catalog for the configuration (module grouping) in the working area

Project tree → "Local modules"

In the Project tree, the modules along with their parameter assignments (for example, addresses) are stored under the relevant device in the "Local modules" folder.

2.3.2. Hardware Catalog



The Hardware catalog contains all devices and hardware components in a tree structure. From the catalog, selected devices or modules can be dragged to the graphic work area of the "Devices & networks" editor.

Search Function

This allows a convenient search for specific hardware components. The search also includes the module description texts.

Filter Function

☒ Filter enabled

Only modules that match the current context (match the working area) are displayed.

☐ Filter disabled

All existing objects of the catalog are displayed

Contents of the Hardware Catalog

- Network view → only objects that can be networked
- Device view → all modules that belong to the current device in the working area

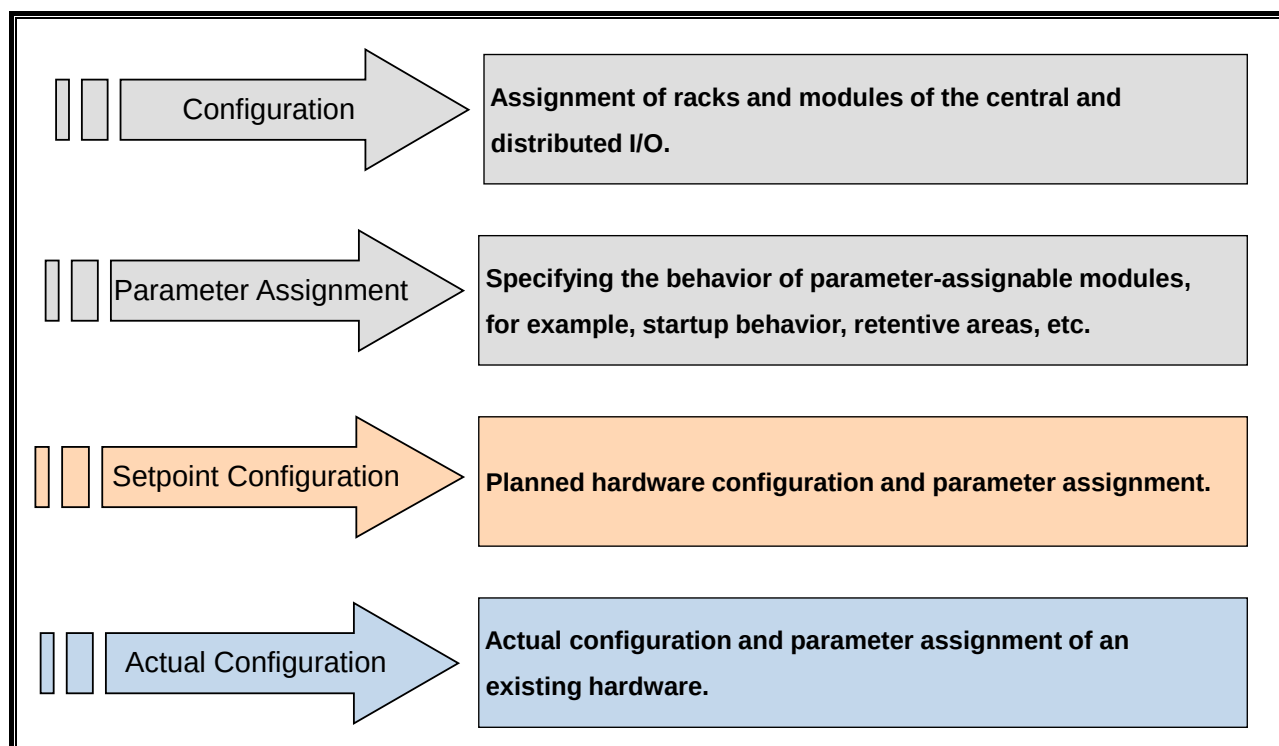
When changing from the Network to the Device view, the view of the filtered objects is adapted to the device currently selected in the Network view.

Information

The "Information" pane shows detailed information about the objects selected in the catalog.

- Schematic diagram
- Name
- Order (Article) number
- Version number

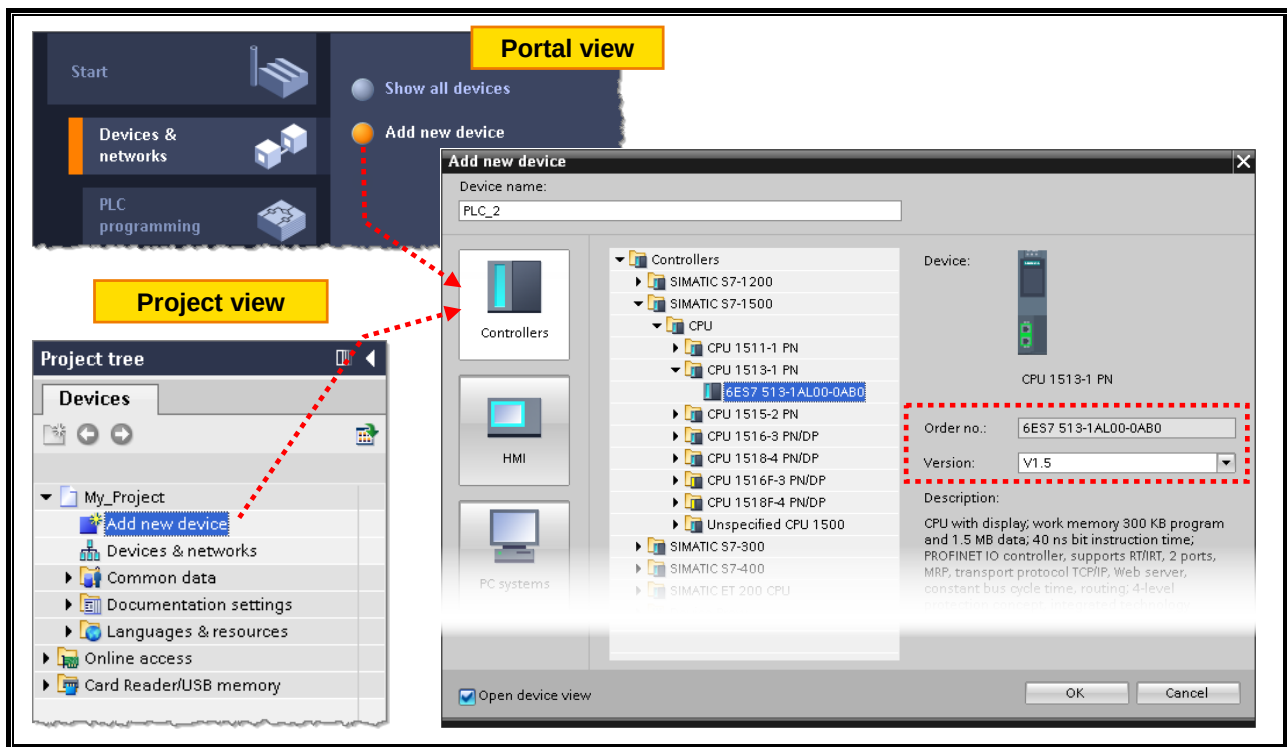
2.4. Setpoint and Actual Configuration



Setpoint and Actual Configuration

When you configure a system, a setpoint configuration is created. It contains a hardware station with the planned modules and the associated parameters. The PLC system is assembled according to the setpoint configuration. During commissioning, the setpoint configuration is downloaded to the CPU.

2.4.1. Setpoint Configuration: Creating a Hardware Station



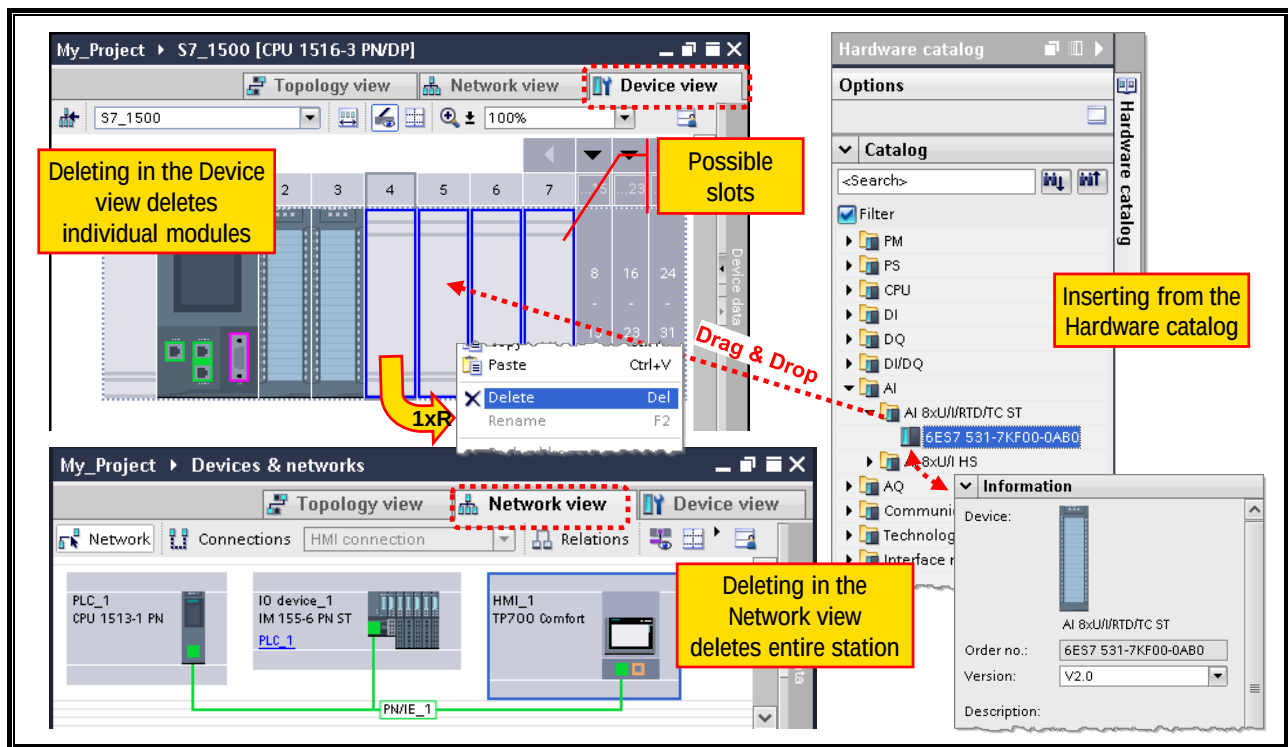
Device Configuration

With the device configuration, the user specifies the arrangement of modules in the rack.

When a new device is created, a suitable rack is also created automatically. The selected device is inserted into the first permitted slot in the rack.

Regardless of the method selected, the added device is visible in the Device view and the Network view of the "Devices & networks" editor.

2.4.2. Inserting / Deleting a Module



Inserting a Module

A new module can also be inserted by dragging and dropping it between two modules that have already been plugged in. To do this, the new module is dragged above and between the two already existing modules. An insert mark then appears, via which the new module can be inserted. All modules that have already been inserted are moved one slot to the right; if there are too many modules, the excess modules are moved to the "area for modules not plugged in".

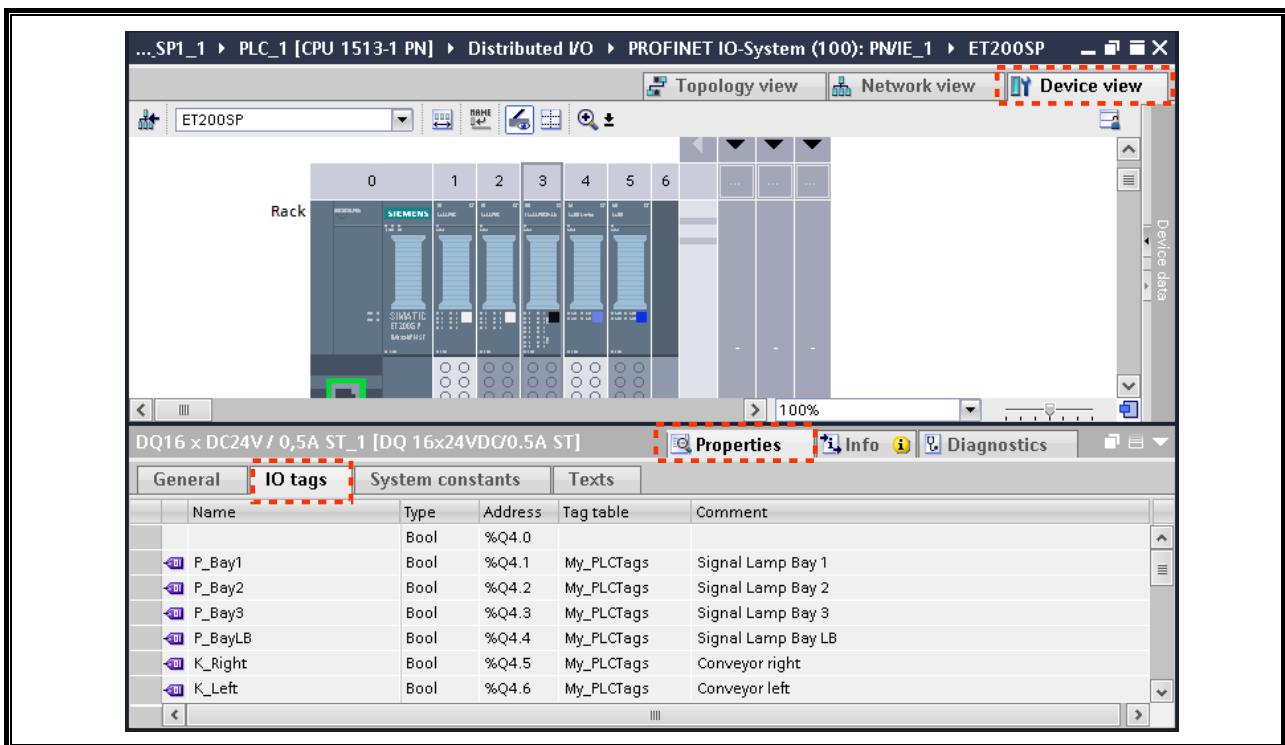
Deleting a Module

Deleted hardware components are removed from the system and assigned addresses are released again.

Selecting a Version

When selecting a module, you must pay attention to the correct version. If the module is selected (highlighted) in the Task Card "Hardware catalog > Catalog", the version can be selected in the Task Card "Hardware catalog > Information".

2.4.3. Symbols of the Module Channels

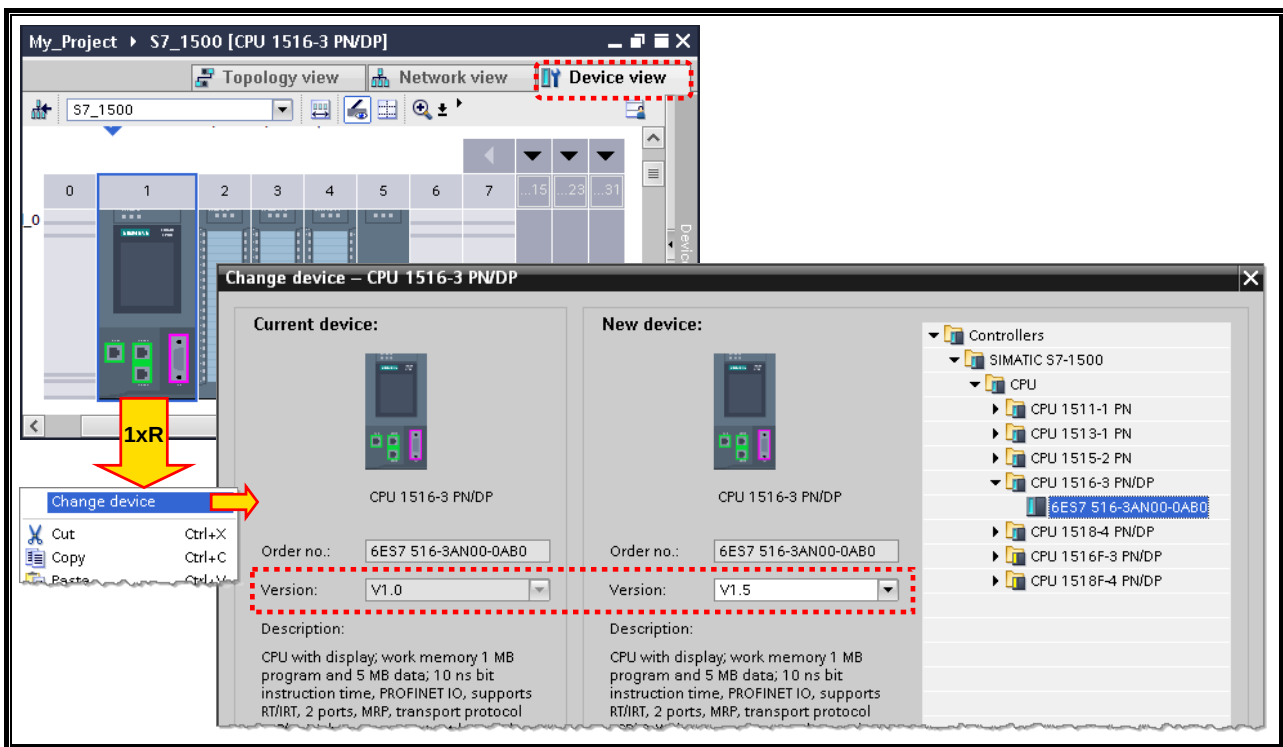


Symbols of the Module Channels

During the configuration or parameter assignment of the I/O modules, the symbols for the digital and/or analog input and output channels can already be defined by the user.

Already declared symbols are displayed (can be edited) with the PLC Tag table in which they are declared. Newly declared symbols are always assigned to the PLC default tag table.

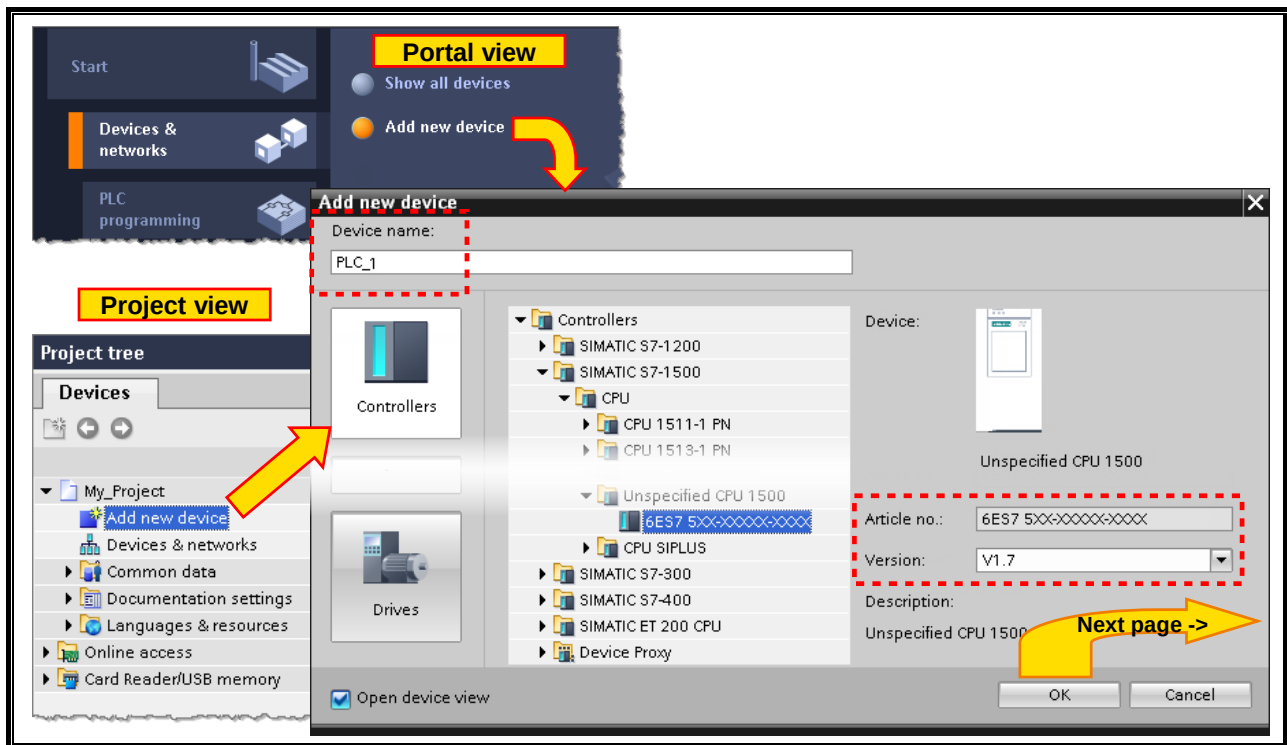
2.4.4. Changing a Module



Changing a Module

Compared to deleting and then inserting a new module, the advantage of changing is that when a module is changed (replaced), all the parameters of the old module are adopted on the new module. A module exchange can, for example, then be necessary when the CPU version in the offline project is to be adapted to the CPU version (online) following a firmware update. Hardware components can only be exchanged if the components are compatible.

2.4.5. Uploading the Actual Configuration into the Project (1): Inserting an Unspecified CPU

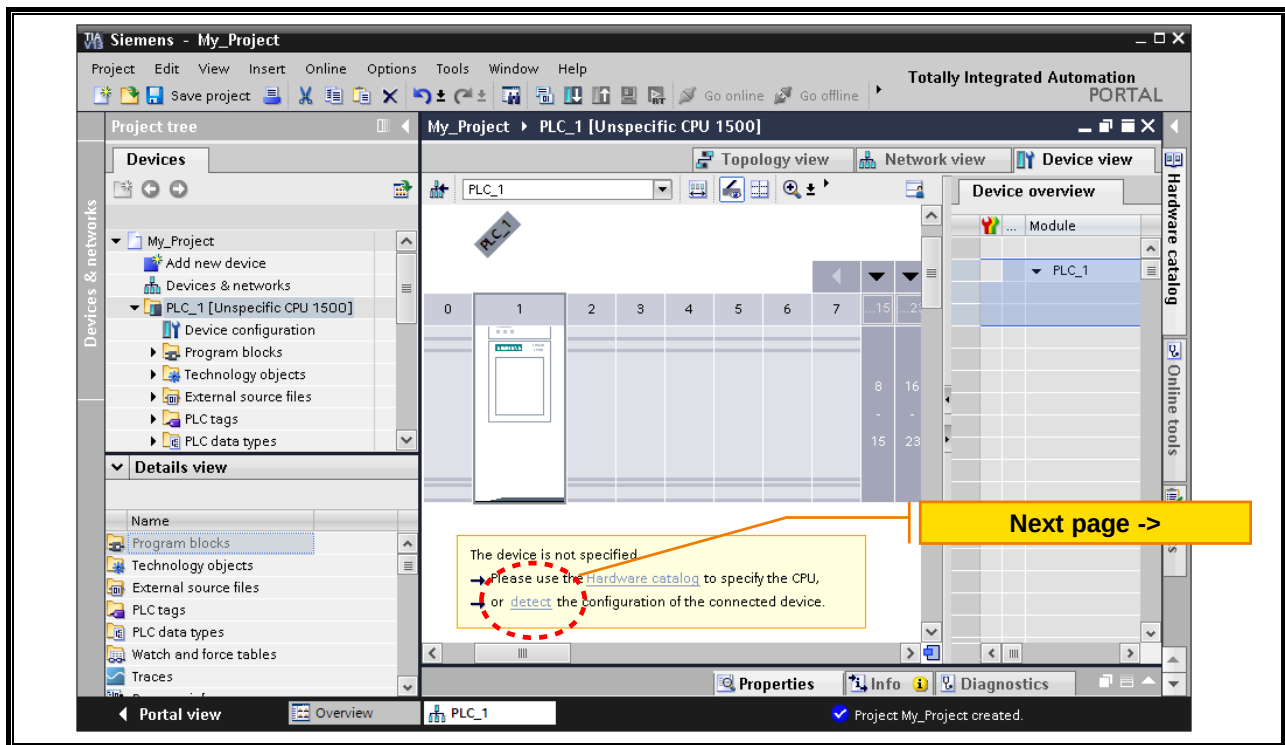


Uploading the Actual Configuration into the PG

The available configuration (without module parameters!) can be read out from an actually existing station.

This is then necessary, for example, when no matching offline project exists on the PG. After the actual configuration has been read out, the modules can be assigned parameters, saved and reloaded into the CPU. For this, an "Unspecified CPU" must be created in the offline project as a first step.

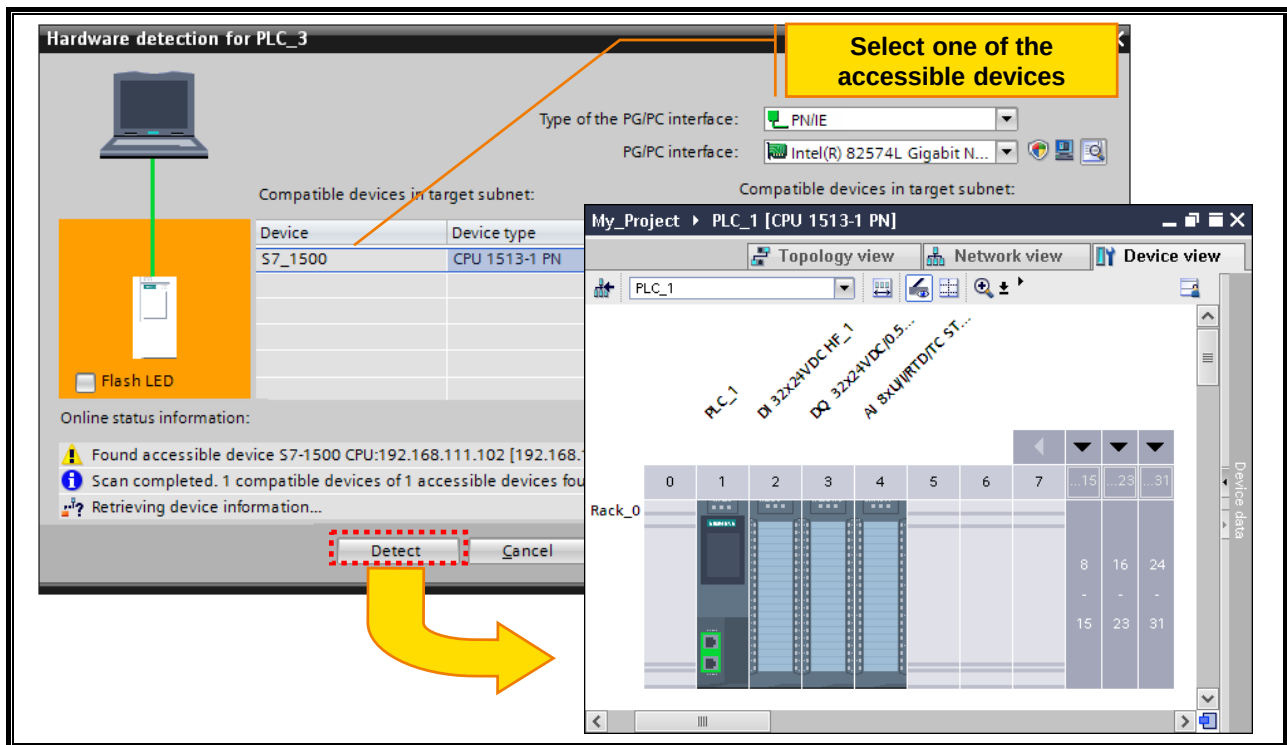
2.4.6. Uploading the Actual Configuration into the Project (2): Detecting Accessible Devices



Detecting the Online Station

As soon as the "Unspecified CPU" is created, all accessible devices can be detected in the dialog shown via "detect".

2.4.7. Uploading the Actual Configuration into the Project (3): Reading-out the Actual Configuration



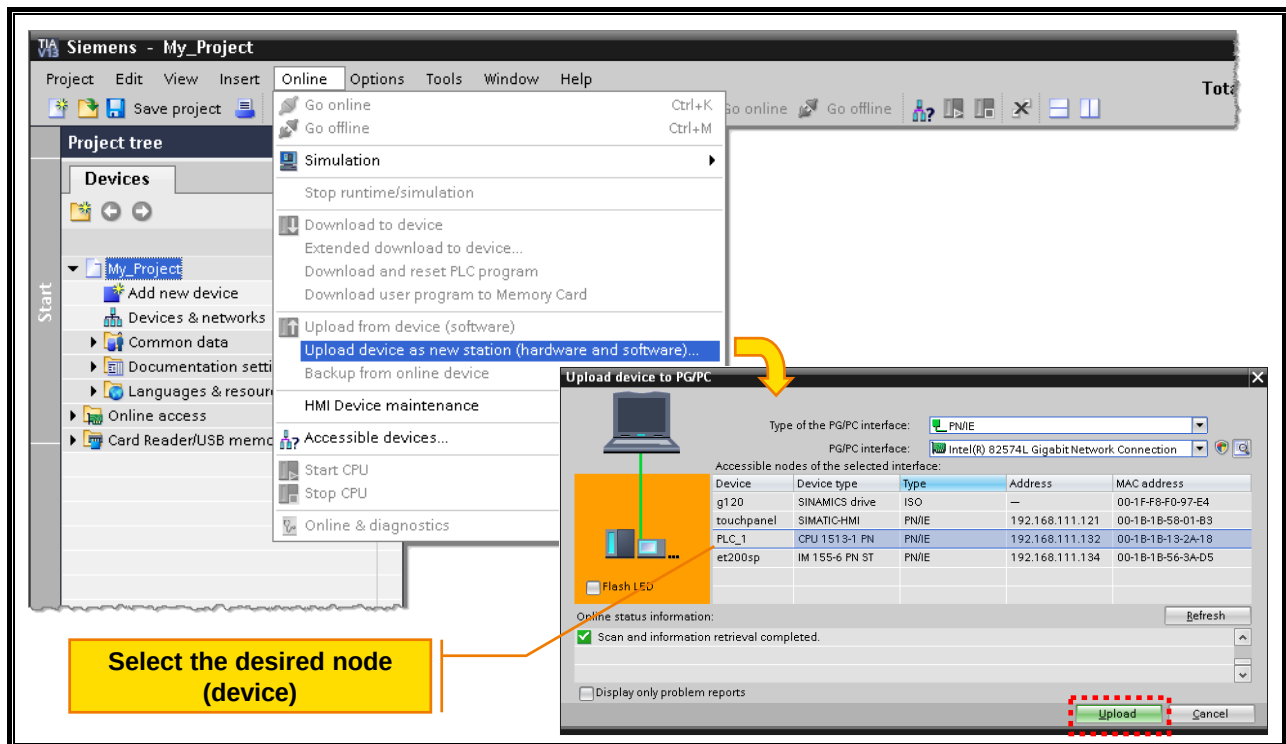
Uploading the Actual Configuration

The actual configuration of the selected device is read out and stored in the offline/project.

Note:

During read-out, only the hardware configuration is uploaded, no hardware parameterization and also no S7 program blocks.

2.5. Uploading a Device as a New Station (1) (Hardware with Parameterization and Software)



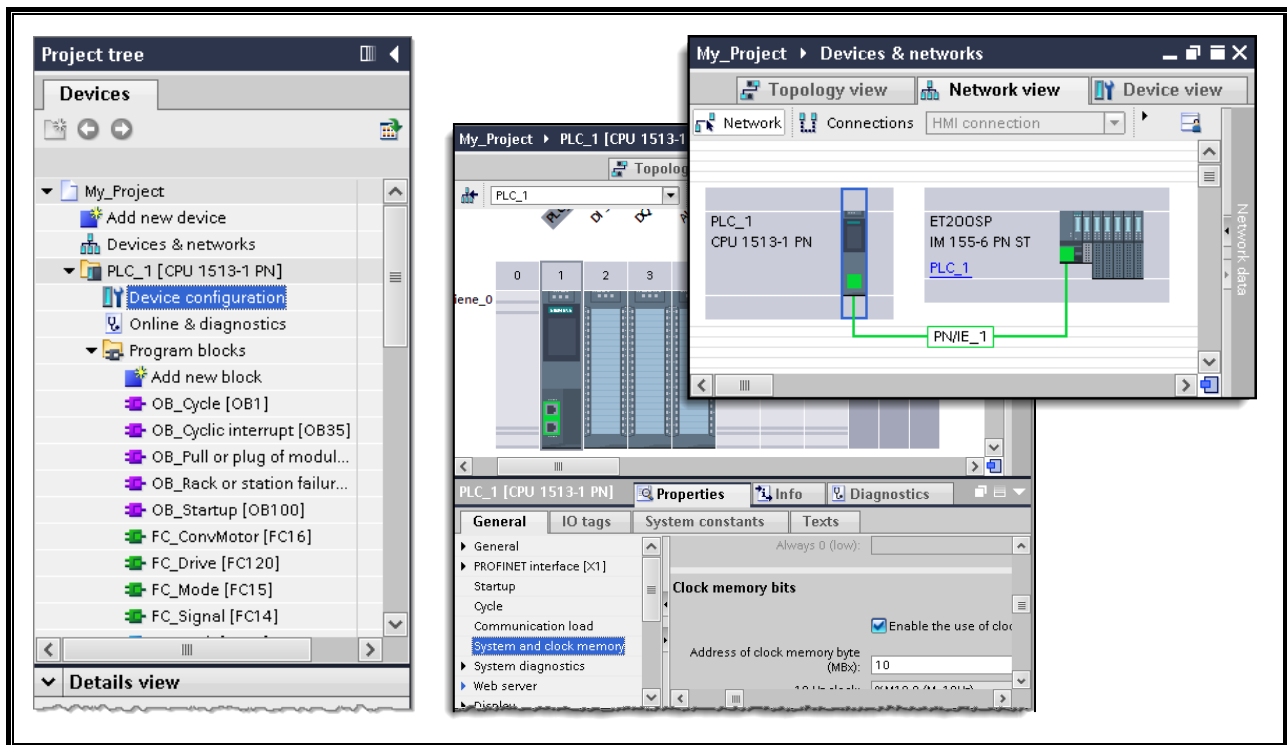
Unlike the option “Upload actual configuration without parameterization”, an already existing configuration of an S7 station can be read out using the function “Upload device as new station”.

This is then necessary when the appropriate offline station **doesn't** exist on the PG. After reading out the S7 station, the hardware as well as the program can be adjusted or modified, saved and downloaded into the CPU.

Requirement:

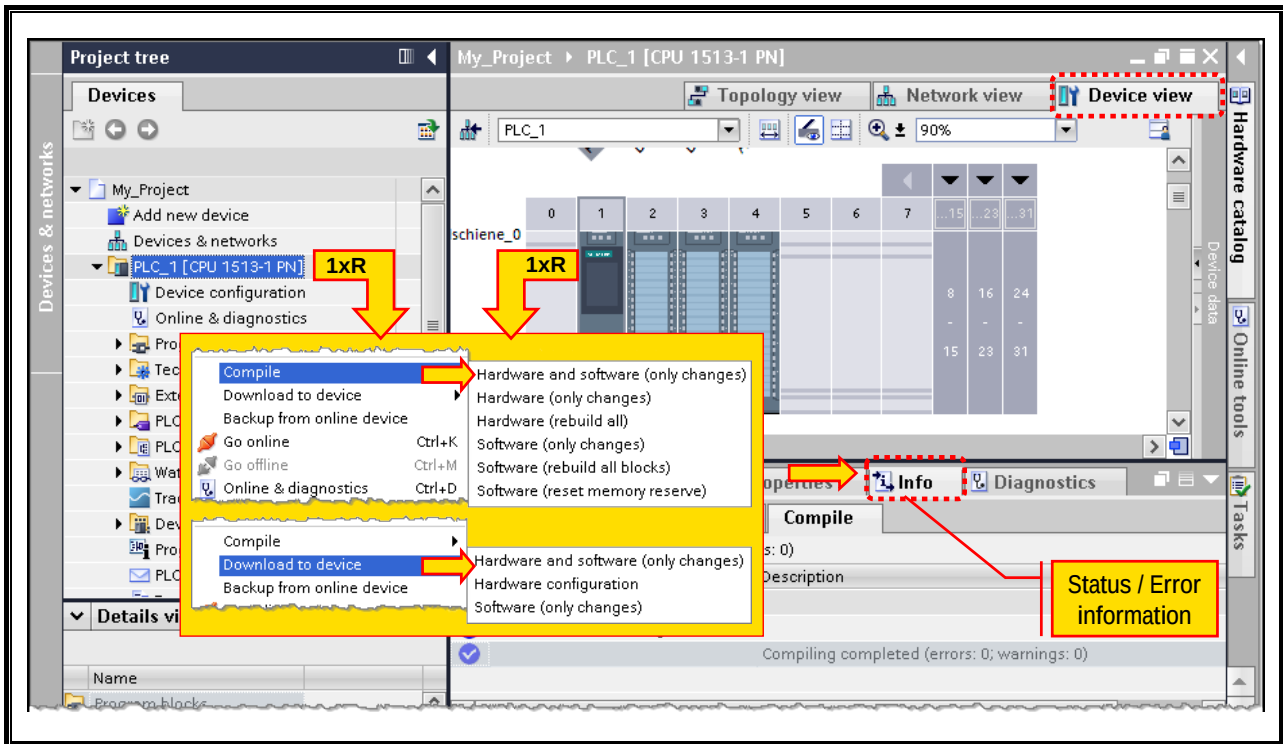
The station already has a configuration.

2.5.1. Uploading a Device as a New Station (2) (Hardware with Parameterization and Software)



After the device is uploaded, the entire station (central and distributed hardware with parameterization, the entire program with comments and symbols) is available to the user offline for further processing.

2.6. Compiling the Hardware Configuration and Downloading It into the CPU

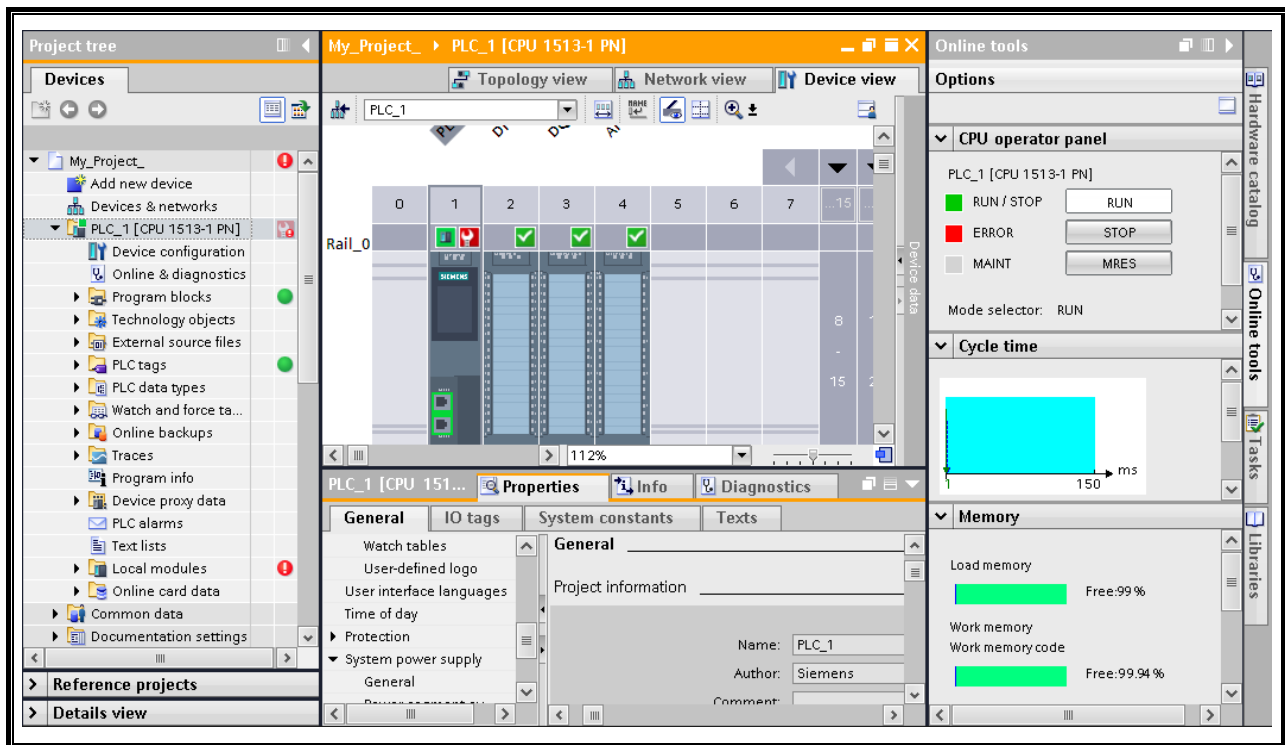


Compiling / Downloading the Hardware Configuration

The following components of a hardware station can be compiled and downloaded:

- **Hardware and software (only changes)**
The entire hardware configuration and parameter assignment as well as all changes to the user program are compiled/downloaded.
- **Hardware (only changes)**
Only the changes are compiled.
- **Hardware (rebuild all) / Hardware configuration**
The entire hardware configuration and parameter assignment is compiled/downloaded.
- **Software (only changes)**
Only the modified blocks of the user program are compiled/downloaded.
- **Software (rebuild all blocks)**
All blocks of the user program are compiled.
- **Software (reset memory reserve)**
Compiling all modified blocks of the user program and re-initialization of the data blocks for which memory reserves are used after changes, that is, the current data of the modified blocks in the CPU are overwritten with the start values during the next download and the memory reserves are enabled even if a memory reserve (for loading a DB structure change with value retention) is set in the properties of the data block.

2.7. Online Tools



Online Tools

If it is possible to establish an online connection to the CPU, diagnostics and status information of all modules can be called.

With CPUs that can be accessed online, the mode can also be controlled using the "Online tools" task card and further status information (cycle time statistics and memory load) can be called.

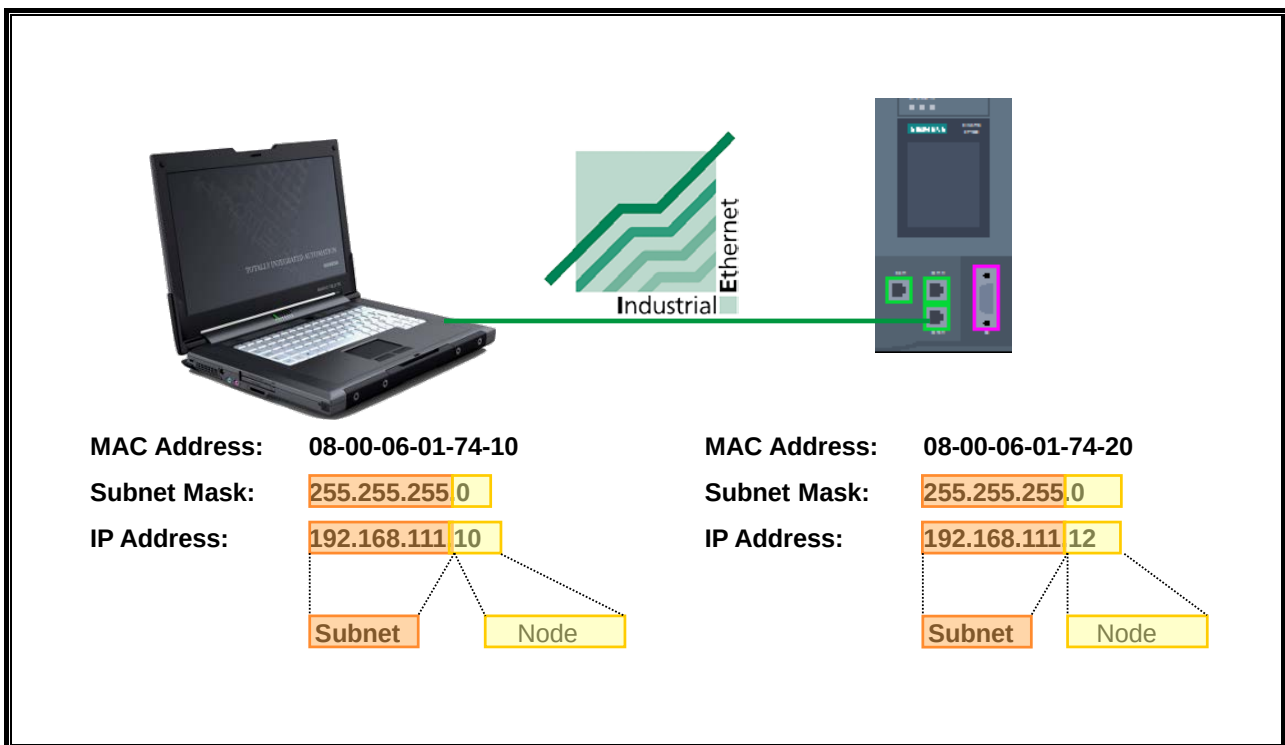
Configuring and Parameterizing the Hardware

Almost all devices or components of an automation solution such as PLCs or touchpanels can be assigned parameters. The parameter assignment of the devices and network settings required for commissioning is handled using the "Devices & networks" editor.

With this, for example, all components of an Ethernet network are assigned IP addresses via which they communicate during later operation.

But even inside the automation device, address areas of the I/O modules must be specified and the cycle monitoring time of the CPU must be set, for example.

2.7.1. Online Connection via Industrial Ethernet: IP Address and Subnet Mask



Internet Protocol

The Internet Protocol (IP) is the basis for all TCP/IP networks. It creates the so-called datagrams (data packets specially tailored to the Internet protocol) and handles their transport within the local subnet or their "routing" (forwarding) to other subnets.

IP Addresses

IP addresses are not assigned to a specific computer, but rather to the network interfaces of the computer. A computer with several network connections (for example routers) must therefore be assigned an IP address for each connection.

IP addresses consist of 4 bytes. With the dot notation, each byte of the IP address is expressed by a decimal number between 0 and 255. The four decimal numbers are separated by dots (see picture).

MAC Address

Every Ethernet interface is assigned a fixed address by the manufacturer that is unique worldwide. This address is referred to as the hardware or MAC address (Media Access Control). It is stored on the network card and uniquely identifies the Ethernet interface in a local network. Cooperation among the manufacturers ensures that the address is unique worldwide.

Subnet Mask

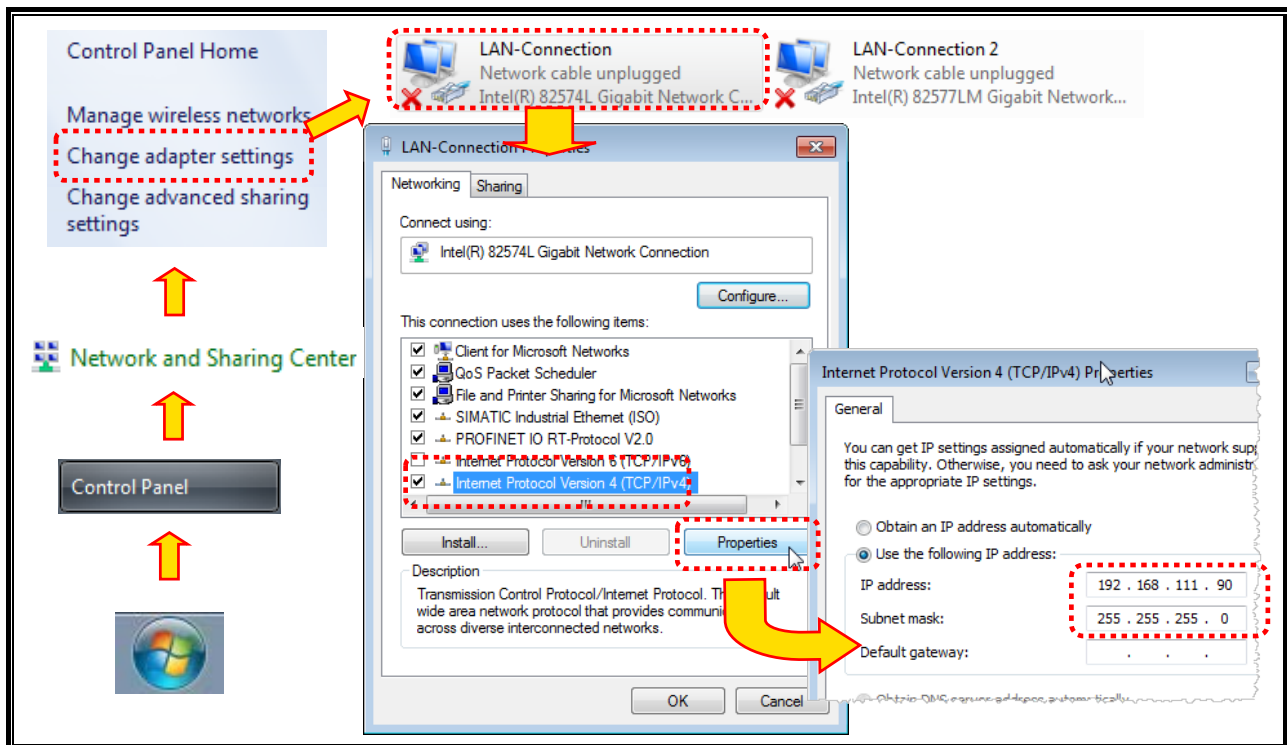
The subnet mask specifies which IP addresses in the local network can be accessed. It separates the IP address into the network and device part.

Only IP addresses whose network part is the same can be accessed.

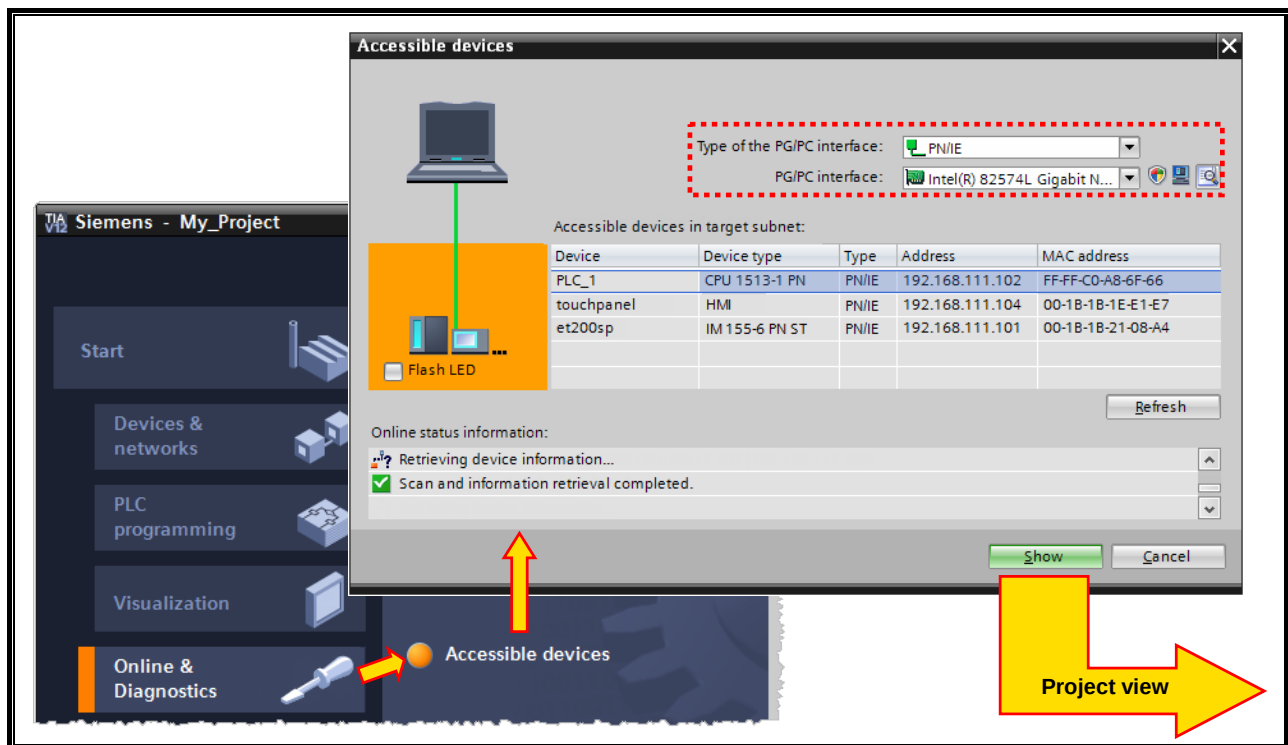
e.g.: Subnet mask = **255.255.255.0** and IP address = **192.168.111.10**
accessible IP addresses: **192.168.111.1** to **192.168.111.254**

2.7.2. Establishing an Online Connection: Assigning an IP Address for the PG

2.7.2.1. Windows7 Operating System



2.7.3. Online Access: Accessible Devices in the Portal View



Accessible Devices in the Portal View

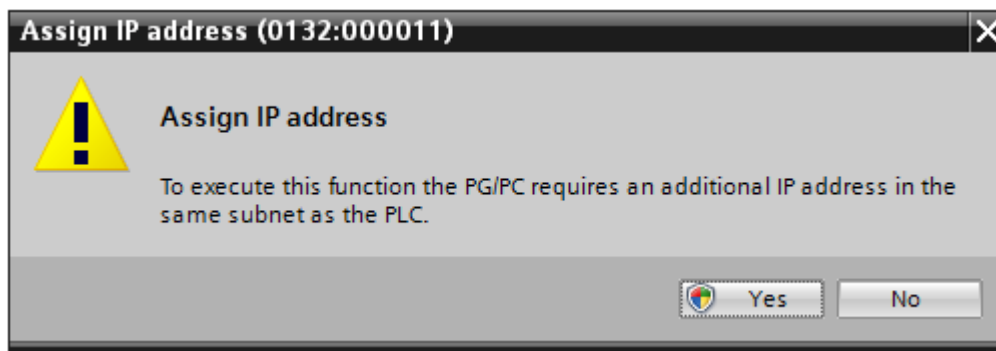
This function provides the option of fast access (for example for service purposes) even when there is no offline project data for the target systems on the PG.

All accessible, programmable modules (CPUs, FMs CPs, HMI devices) are listed in the Portal view, even if they are located in other subnets.

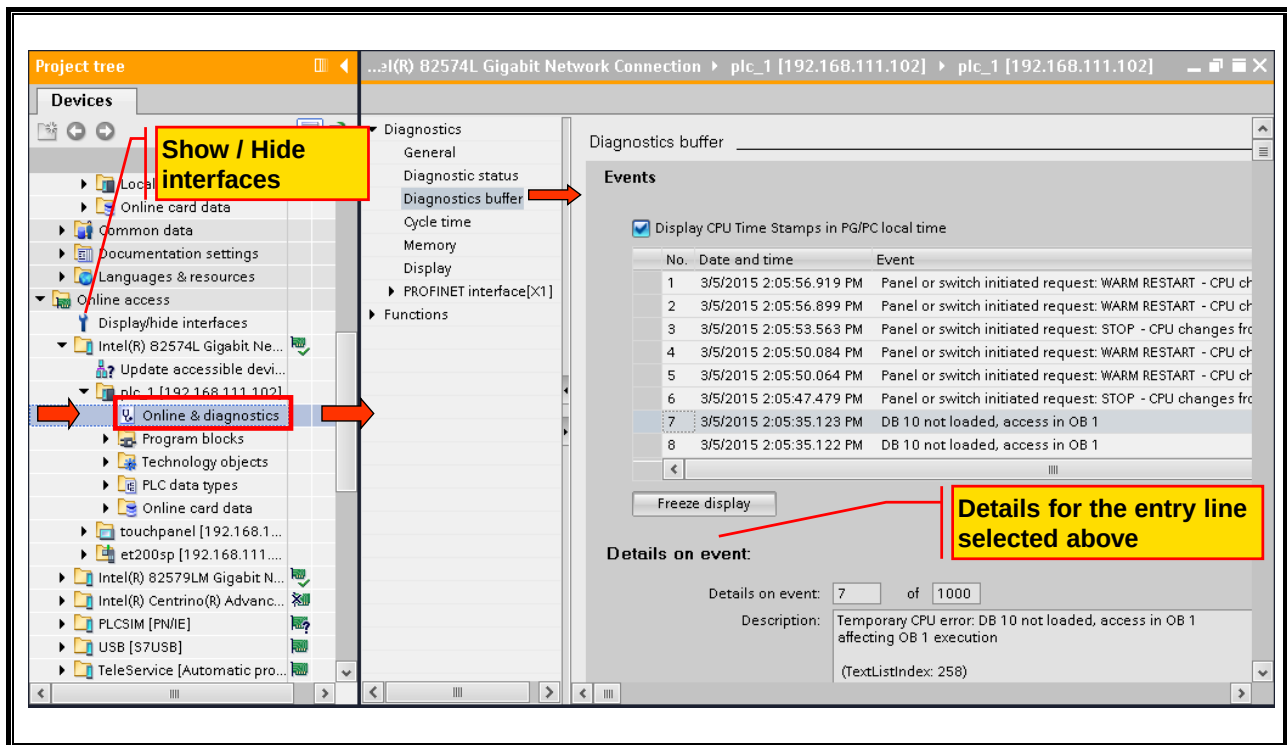
Access Online Functions → Button

Whenever there is an attempt to access a module online with the "Show" button and this is located in a different subnet from the PG, a dialog opens asking whether an additional IP address should be assigned.

Following confirmation, an additional IP address is assigned to the PG that is located in the same subnet as the address of the CPU. After that, all online functions can be used.



2.7.4. Accessible Devices in the Project View: CPU Online Access: Reading-out the Diagnostics Buffer



Online Access to the CPU

If the PG and the target system (for example CPU) are located in the same subnet, various Online & diagnostics functions are available in the "Accessible devices" function.

- in the working area of the TIA Portal
- in the "Online tools" task card

Diagnostics Buffer

The diagnostics buffer is a buffered memory area on the CPU organized as a circular buffer. It contains all diagnostics events (error alarms, diagnostics interrupts, start-up information etc.) of the CPU in the order in which they occurred. The highest entry is the last event to occur.

All events can be displayed on the programming device in plain language and in the order in which they occurred.



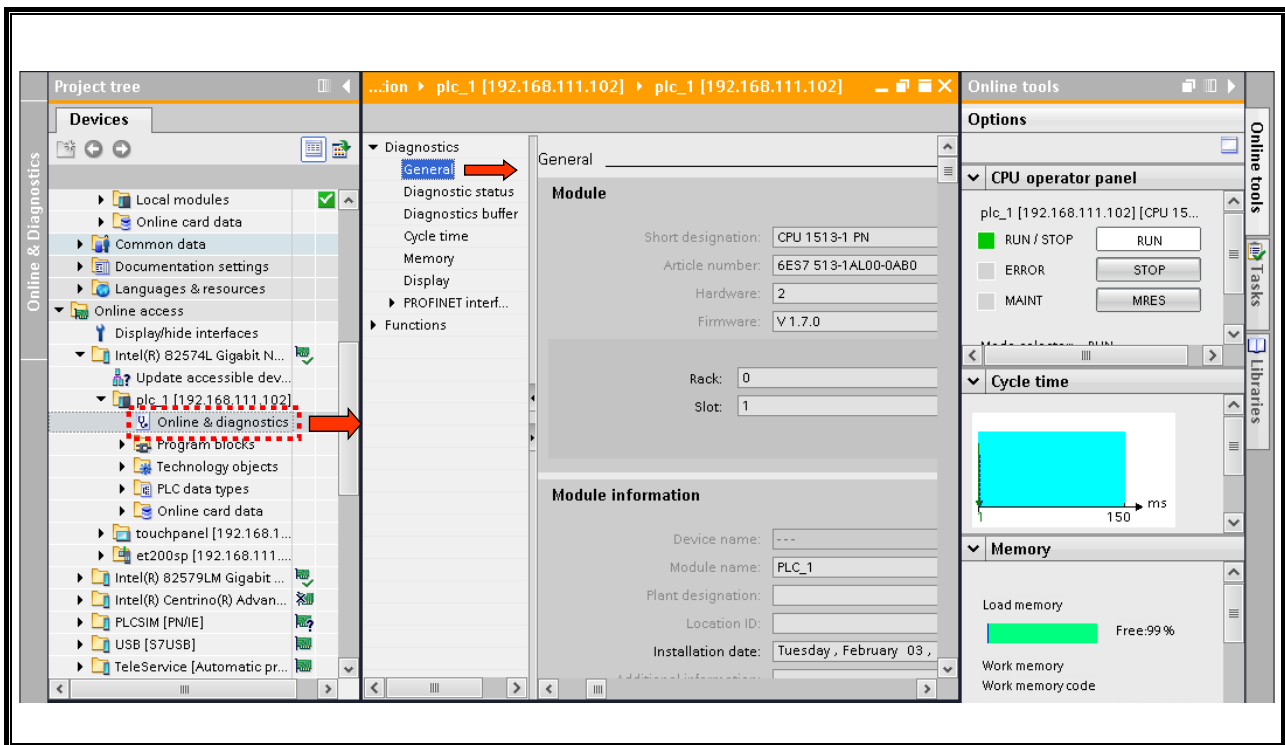
The size of the diagnostics buffer depends on the CPU. As well, not all of the diagnostics buffer is buffered with PowerOFF (only a part is retentive).

Details on Event

Some additional information is also provided for the selected event in the "Details on event" box:

- Event name and number
- Additional information depending on the event, such as, the address of the instruction that caused the event etc.

2.7.5. Accessible Devices in the Project View: CPU Online Access: CPU-RUN/STOP, Memory Reset (MRES)



CPU Operator Panel: Mode Selector Switch

The operating mode of the CPU can be changed.

- RUN → STOP:
If there is a change from RUN to STOP, the CPU terminates the running user program.
- STOP → RUN:
If there is a change from STOP to RUN, the CPU performs a restart.

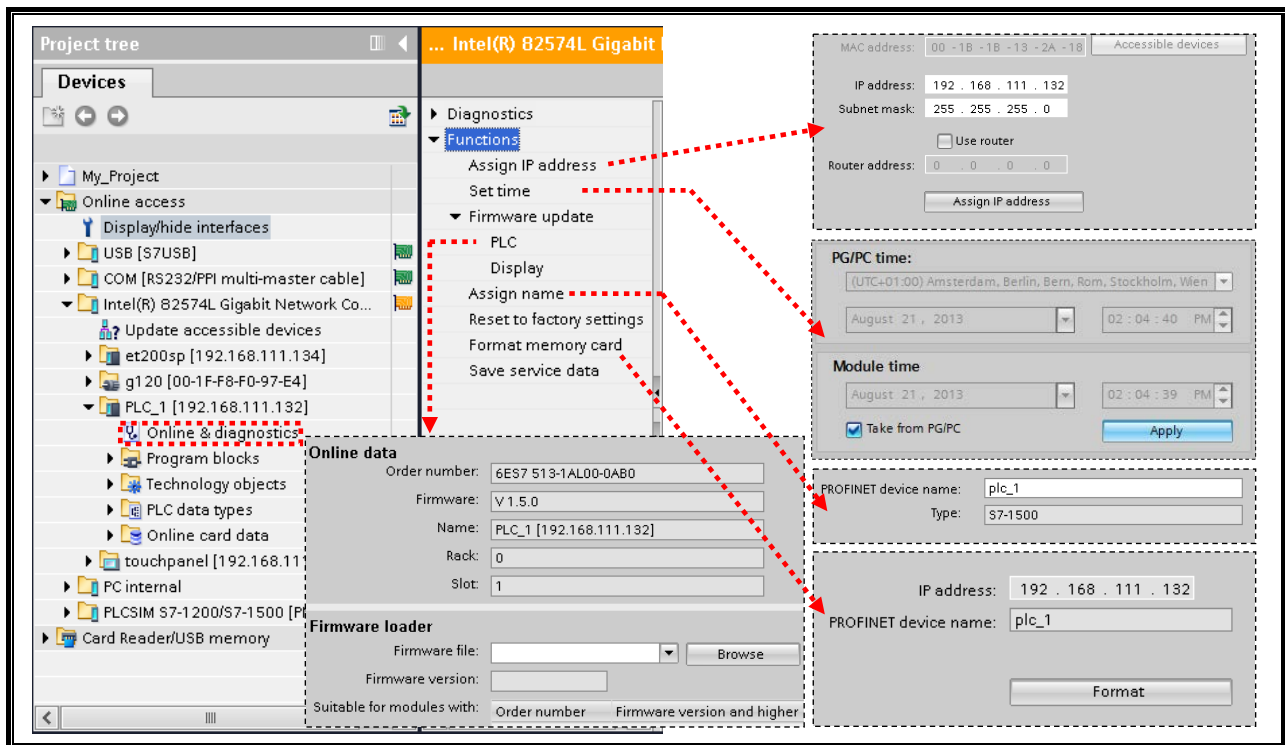
Cycle Time:

"Shortest", "Current" and "Longest" are the cycle times since the last CPU restart

With a Memory Reset (MRES), a CPU reset is carried out:

- All user data (even the retentive) is deleted (delete work memory)
(process images, memory bits, timers, counters, all program/data blocks)
- Retained are: parameter assignment of the X1 (Ethernet) interface, the retentive part of the diagnostics buffer, time-of-day

2.7.6. Online Access to the CPU: IP Address, Name, Time, FW Update, Format Memory Card



- **Set Time (of Day)**
Each S7 CPU has a real-time clock that can be set here.
- **Assign IP Address**
As long as no IP address has been specified already by a hardware configuration that was downloaded earlier, this can be assigned or modified here (this function is also available when the PG/PC and the CPU are not assigned to the same subnet).
- **Reset to Factory Settings**
Unlike the "memory reset", all the memory areas of the CPU (work, load and retentive memory, diagnostics buffer and time) are deleted. Optionally (see dialog in the picture), the IP address can also be deleted so that the CPU then only has a MAC address (Media Access Control).
- **Format Memory Card**
The CPU memory card can also be deleted in the CPU via this online function. After that, the CPU only has its IP address. All other data (including the device configuration) is deleted. The card cannot be deleted in the card reader via the Project tree. Device configuration and blocks have a gray background, that is, are write-protected (only status information or open with a double-click).
- **Assign Name**
In PROFINET, each device must be assigned a unique device name that is stored retentively on the device. The device name identifies a distributed I/O module (PROFINET IO) and allows module replacement without a PG/PC.
- **Firmware Update**
Here the firmware version of the CPU can be updated. Under "Diagnostics -> General", the current firmware version of the CPU is displayed in the module data.

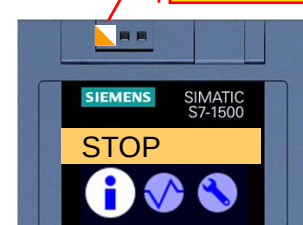


- **Attention!**
If both components have to be updated, then update the Display first and only after that the CPU.

2.8. CPU Memory Reset (MRES) using the Mode Selector Switch



1. Set the **mode selector switch** to **STOP**
2. Press and hold the mode selector switch in the **MRES** position until the **RUN/STOP LED** has flashed 2x slowly
then let go again
within 3 sec !!!
3. Press and hold the mode selector switch in the **MRES** position until the **RUN/STOP LED** begins to flash quickly
then let go again
4. Set the **mode selector switch** to **RUN**
A CPU restart is carried out



RUN/STOP LED of the S7-1500

Result:

With inserted PROGRAM card
→ **Memory reset**

Without inserted card
→ **Reset to factory settings**

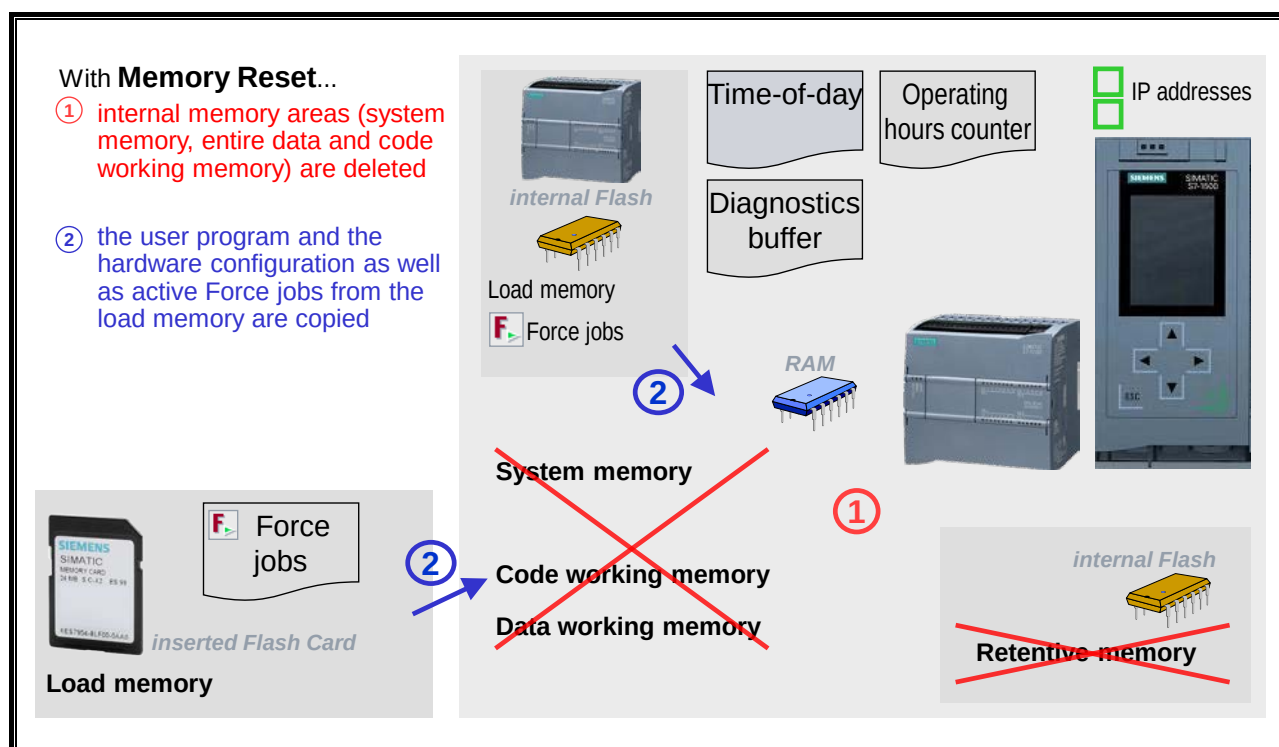
Particular Feature for CPU Memory Reset (MRES) using the Mode Selector Switch:

- **when Memory Card (MC) is inserted => Memory Reset**
 - All user data is deleted (work memory, retentive memory) (process images, memory bits, counters, timers, all program/data blocks)
 - Retained are: parameter assignment of the X1 (Ethernet) interface, the retentive part of the diagnostics buffer
 - The CPU copies all load memory data relevant for execution (memory card) into the internal RAM work memory. (Data relevant for execution: device configuration, program blocks, data blocks).
- **when no Memory Card (MC) is inserted => Reset to factory settings**
 - All memory areas of the CPU (work memory, retentive memory, diagnostics buffer, time-of-day) and the IP address are deleted.

After the MC is inserted, the load memory data relevant for execution is reloaded into the internal RAM work memory from the memory card:

Device configuration (with IP address), program blocks, data blocks

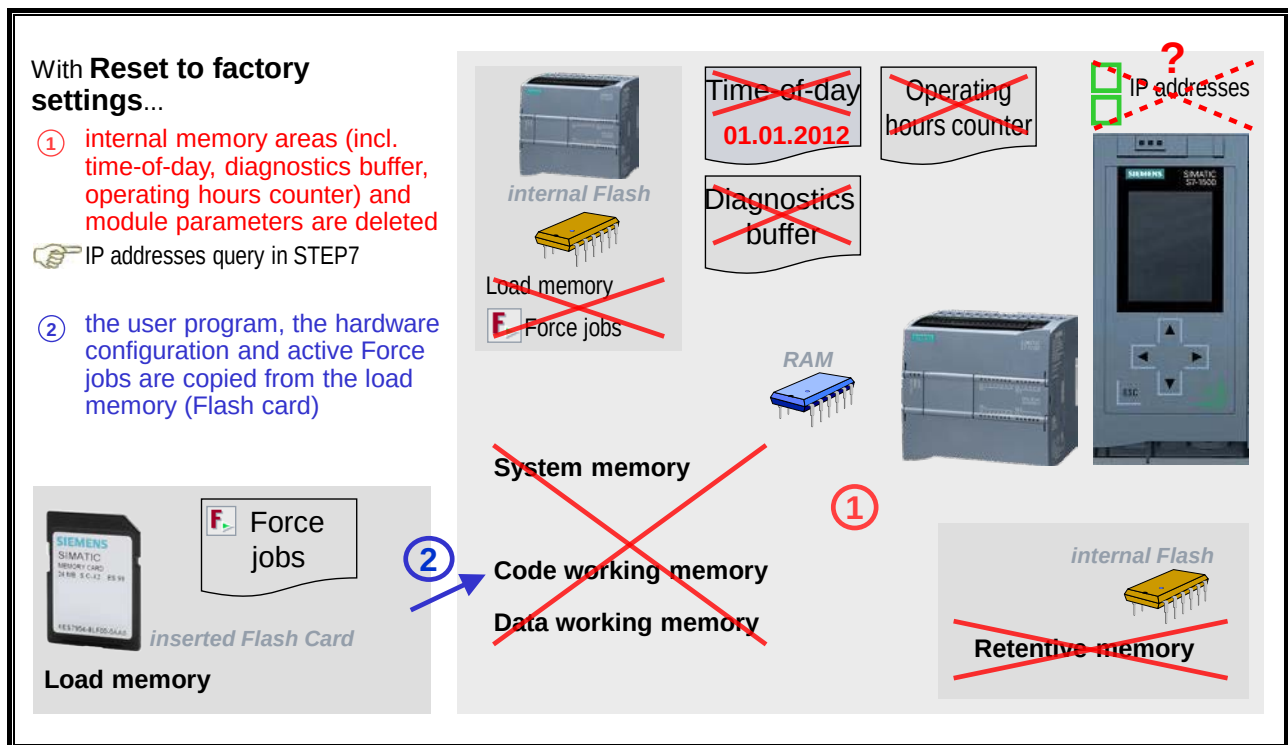
2.8.1. SIMATIC S7-1200/1500: Memory Concept for CPU Memory Reset



CPU Memory Reset

- **What to do:**
 - STEP7 online function → MRES in "CPU operator panel" of "Test" and "Online tools" Task Cards
 - Display (only S7-1500) → Main menu "Settings", submenu "Memory reset"
 - CPU mode selector switch (with inserted memory card)
- **Impact**
 - An existing online connection between the PG/PC and the CPU is terminated.
 - The entire RAM work memory is deleted, that is, all user data (process images, bit memories, counters, timers, all program/data blocks, even the retentive ones)
 - Retained are: IP addresses, diagnostics buffer, operating hours counter, CPU time-of-day.
 - After that, the CPU copies all data relevant for execution into the RAM work memory from the memory card. (Data relevant for execution: device configuration, program blocks, data blocks, current Force jobs).

2.8.2. SIMATIC S7-1200/1500: Memory Concept for CPU Reset to Factory Settings

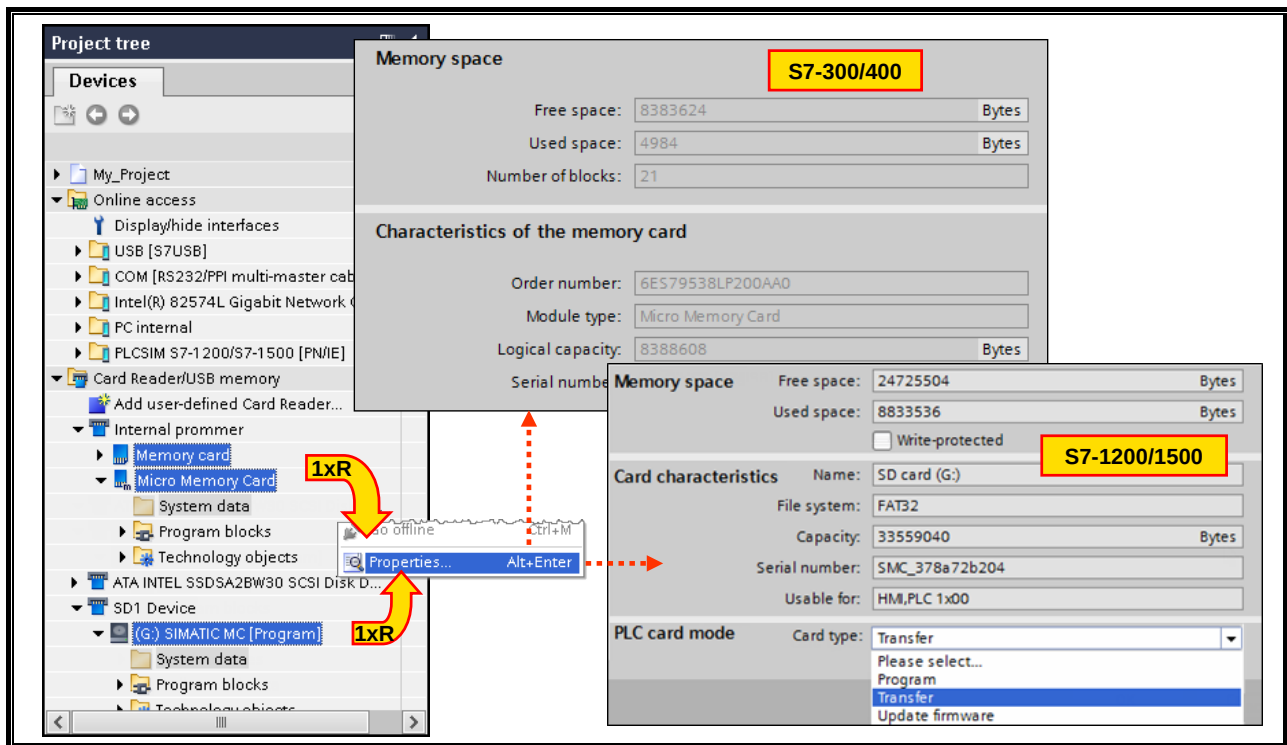


CPU Reset to Factory Settings

- **What to do:**
 - STEP7 online function → MRES in "CPU operator panel" of "Test" and "Online tools" Task Cards
 - Display (only S7-1500) → Main menu "Settings", submenu "Memory reset" → Factory Defaults
 - Mode selector switch (only without memory card)
- **Impact**
 - An existing online connection between the PG/PC and the CPU is terminated.
 - The entire RAM work memory is deleted, that is, all user data (process images, bit memories, counters, timers, all program/data blocks, even the retentive ones, diagnostics buffer), IP addresses are deleted if this is selected in STEP7.
 - Retained are all IP addresses if this was specified in STEP7.

If a memory card is inserted (or is already inserted), the CPU copies all data relevant for execution into the internal RAM work memory from the memory card. (Data relevant for execution: device configuration incl. IP addresses, program blocks, data blocks, current Force jobs).

2.9. SIMATIC Card Reader



SIMATIC Card

The SIMATIC Memory Card of an S7-1200 is an SD memory card pre-formatted by Siemens. It can be read and written with the Windows Explorer but under no circumstances can it be formatted with it!

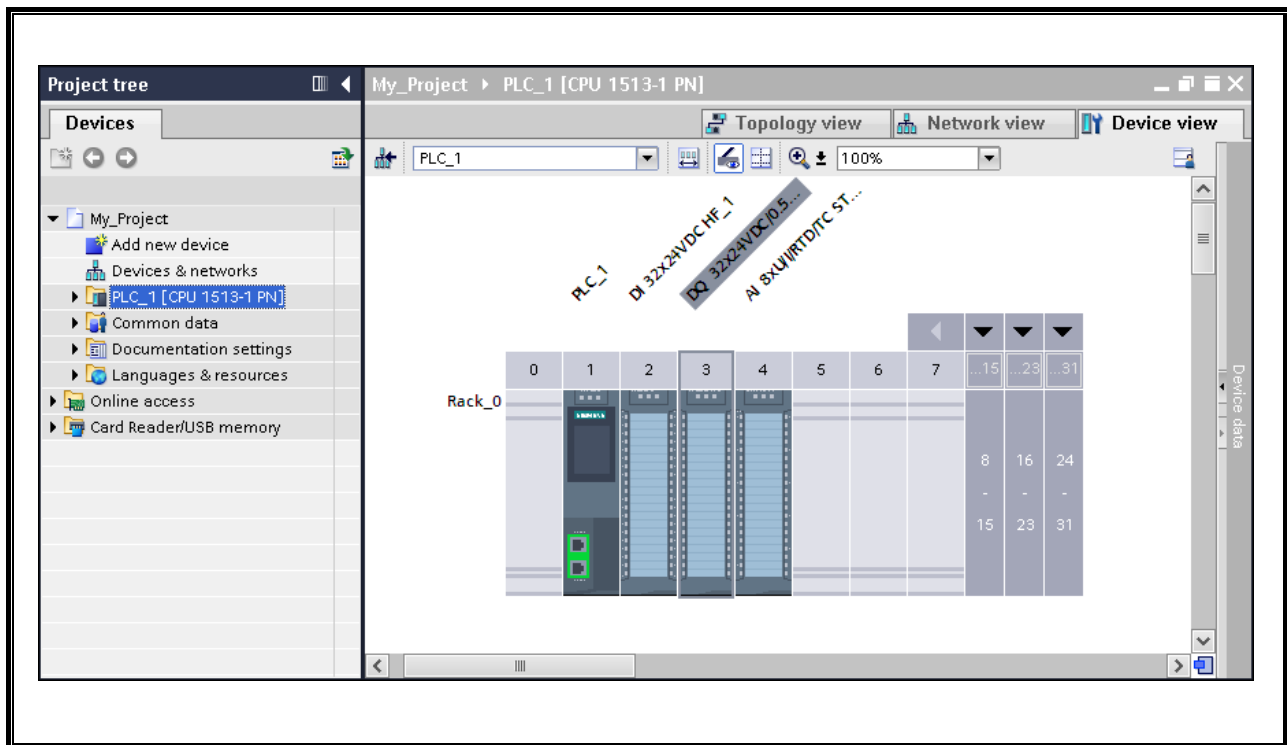
The SIMATIC Micro Memory Card (MMC) of an S7-300 is a memory card formatted by Siemens. It cannot be read with Windows and under no circumstances can it be formatted with Windows!

Card type of the SIMATIC Card for S7-1200/1500:

The SIMATIC card is used as a Program card or a Transfer card or for Firmware Updates. Before the relevant data is stored on the SD card, the card type must be selected as shown in the picture.

- SIMATIC card as Program card:**
 The card contains all configuration and parameterization data for the station as well as the entire user program with documentation. During operation, the card must remain inserted in the CPU because it is used as a replacement for the internal CPU load memory which remains unused.
- SIMATIC card as Transfer card (only for S7-1200):**
 The card contains the same data as a Program card but it doesn't have to remain inserted during operation. After inserting the card and subsequent Power ON, all data is copied into the internal load memory of the CPU. Then the card has to be removed and a restart has to take place.
- SIMATIC card to Update Firmware:**
 The SIMATIC card contains the files required for a firmware update. After execution (instructions are included as a Text file) the SIMATIC card must be removed.

2.10. Task Description: Creating a Project with an S7-1500 Station

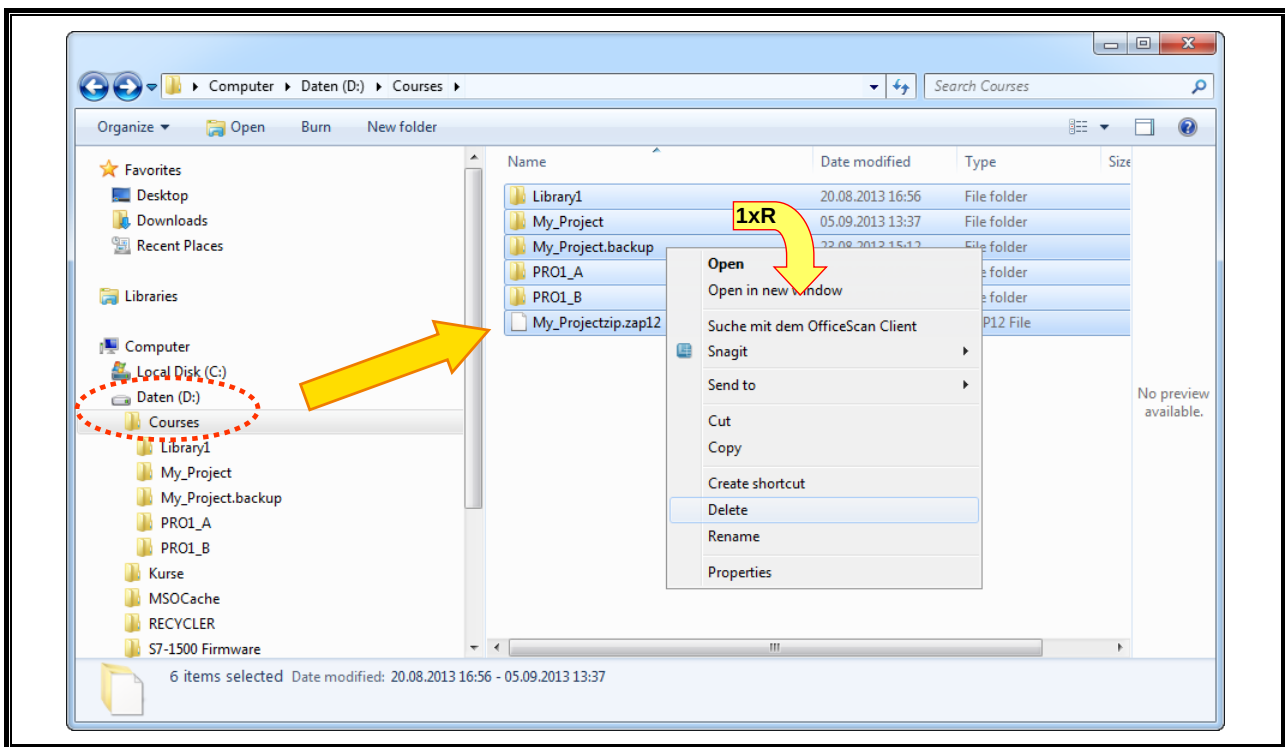


Task Description:

A project with the name "My_Project" is to be created. It is to contain an S7-1500 station whose configuration is to correspond exactly to that of your training unit.

Furthermore, the modules are to be assigned parameters in such a way that the input and output addresses match those shown in the chapter "Training Devices".

2.10.1. Exercise 1: Deleting Old Projects



Task

Delete the TIA Portal projects on the PG.

What to Do

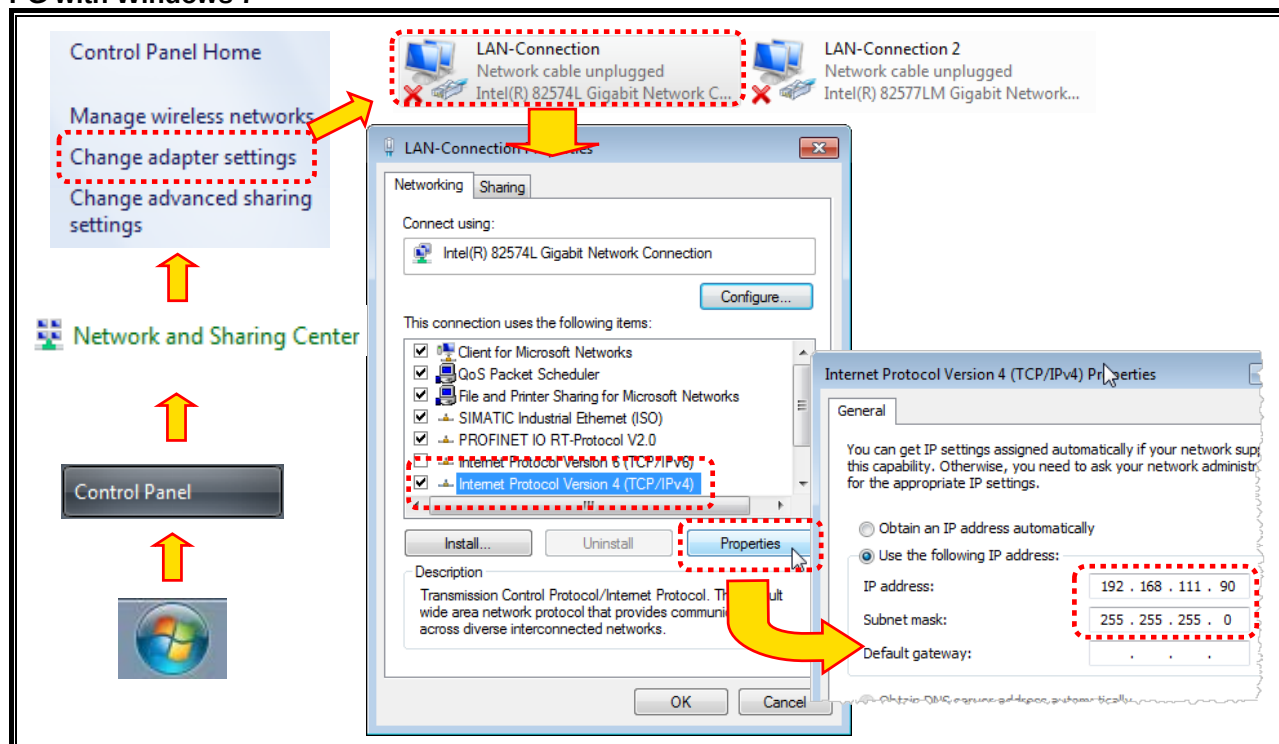
1. Start the Windows Explorer
2. In the directory *D:\Courses*, delete all projects.

Note

Projects that are to be deleted have to be closed!

2.10.2. Exercise 2: Connecting the PG and Setting the IP Address of the PG

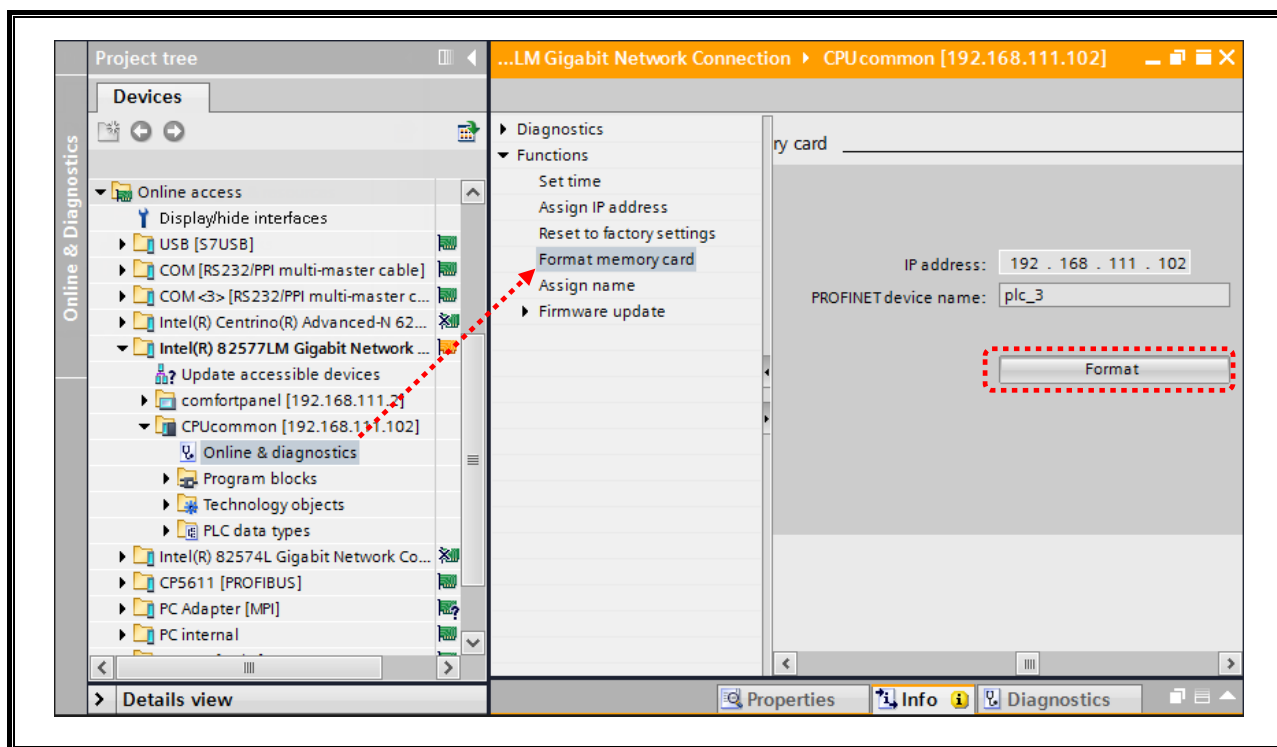
PG with Windows 7



Task and What to Do:

1. Connect the Ethernet interface "Intel(R) 82574L" of the PG to the "P1" connection on the training device using an Ethernet cable.
2. Assign the IP address 192.168.111.90 and the subnet mask 255.255.255.0 to this PG interface. Proceed as shown in the picture.

2.10.3. Exercise 3: Erasing the SIMATIC Memory Card of the CPU



Task

In order to completely erase the CPU, the SIMATIC Memory Card of the CPU must first be erased. This can be carried out as follows:

- with the Windows Explorer (SD card is inserted in the PG Card Reader)
- with the TIA Portal (SD card is inserted in the PG Card Reader)
- with the TIA Portal (SD card is inserted in the CPU)

What to Do:

1. Check whether the SD card is inserted in the CPU.
2. In the Project view under the "Intel(R) 82574L" interface, display all "Accessible devices"
3. Under the S7-1500 station, activate "Online & diagnostics" (see picture)
4. There under "Functions", activate "Format memory card" (see picture)

Note

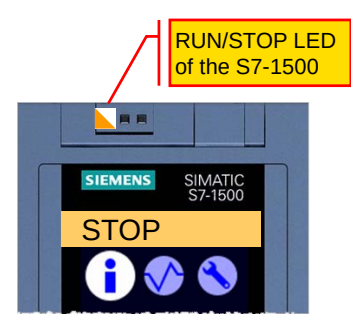
If a password is stored on the CPU that is unknown to you, it is only possible to erase the SD card if it is inserted in the PG Card Reader.

2.10.4. Exercise 4: Resetting the CPU using the Mode Selector Switch



1. Set the **mode selector switch to STOP**
2. Press and hold the mode selector switch in the MRES position until the RUN/STOP LED has flashed 2x slowly
then let go again
3. Press and hold the mode selector switch in the MRES position until the RUN/STOP LED begins to flash quickly
then let go again
4. Set the **mode selector switch to RUN**
A CPU restart is carried out

within 3 sec!!!



RUN/STOP LED
of the S7-1500

Task

In the last exercise you erased the SD of the CPU. Now, the work memory of the CPU is also to be erased. For this, a memory reset has to be carried out.

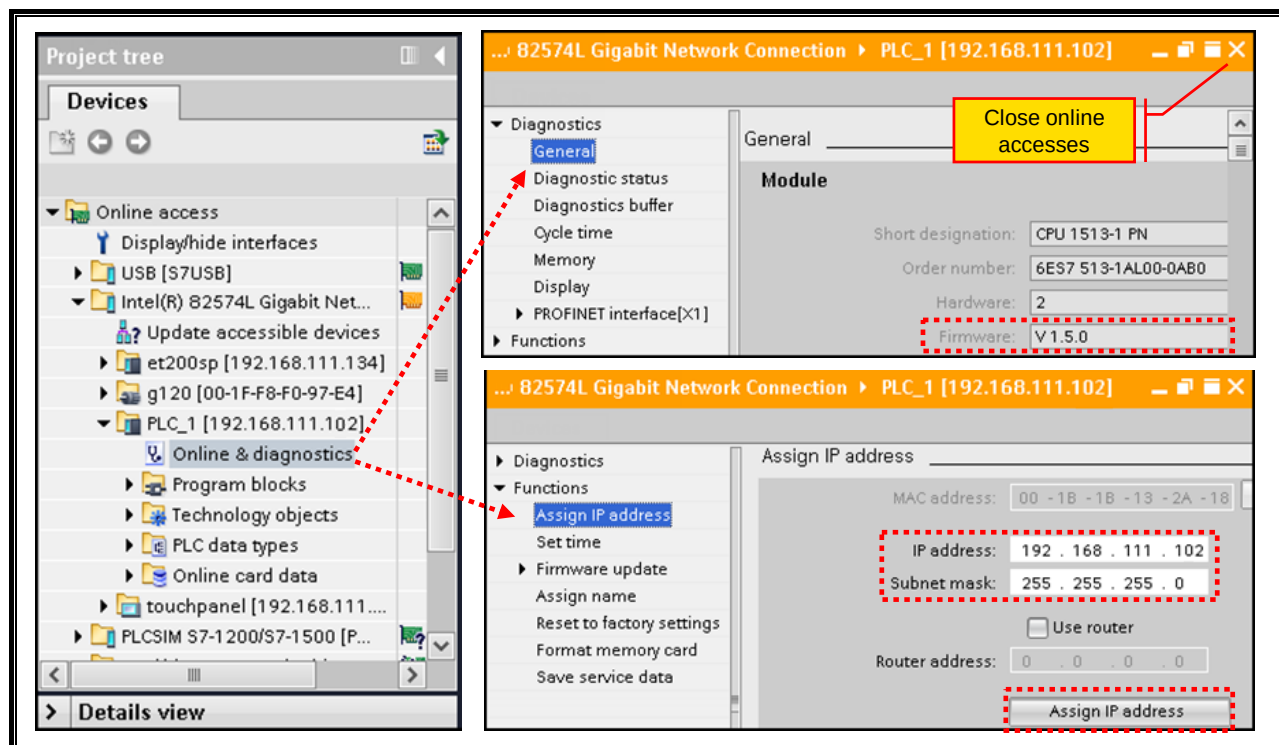
What to Do

1. Carry out the memory reset directly on the CPU following the steps shown in the picture.
2. Carry out a CPU restart by switching the mode selector switch from STOP to RUN.

Result:

- The CPU remains in the STOP state because no user program is loaded.
- The I/O modules indicate with a flashing green light that they are not parameterized.

2.10.5. Exercise 5: Determining the CPU Firmware Version and Assigning the IP Address (Node Initialization)



Task

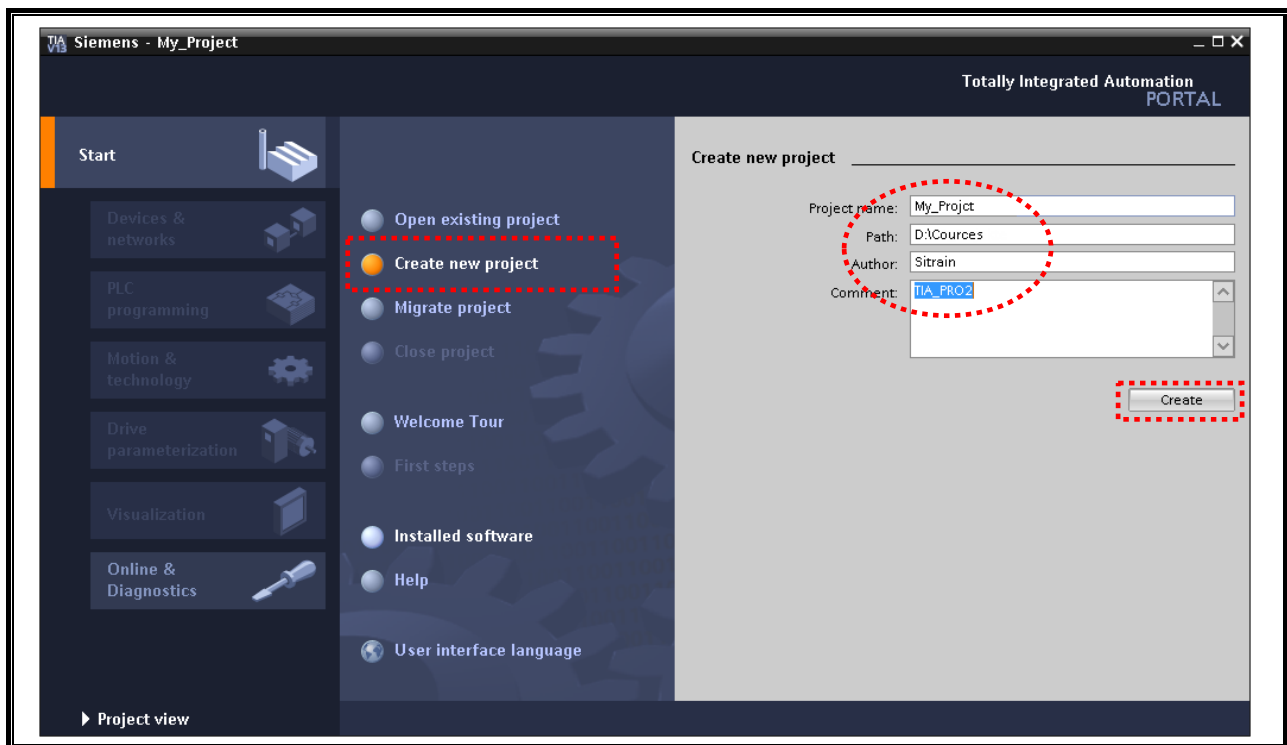
In order to later be able to create an S7-1500 station in the offline project which exactly matches your training controller, you are to determine online the Firmware version of the CPU and make note of it.

Furthermore, you are to check the IP address of the PROFINET interface of the CPU and, if necessary, set it.

What to Do:

1. In the Project tree under the interface "Intel(R) 82574L", display all "Accessible devices" (see picture)
2. Under the S7-1500 station, activate "Online & diagnostics" (see picture)
3. There, under "General" you can see the Firmware version and make note of it.
4. Under "Functions --> Assign IP address", check the IP address and correct it, if necessary.
5. In the working area, close the online accesses (see picture)

2.10.6. Exercise 6: Creating a New Project



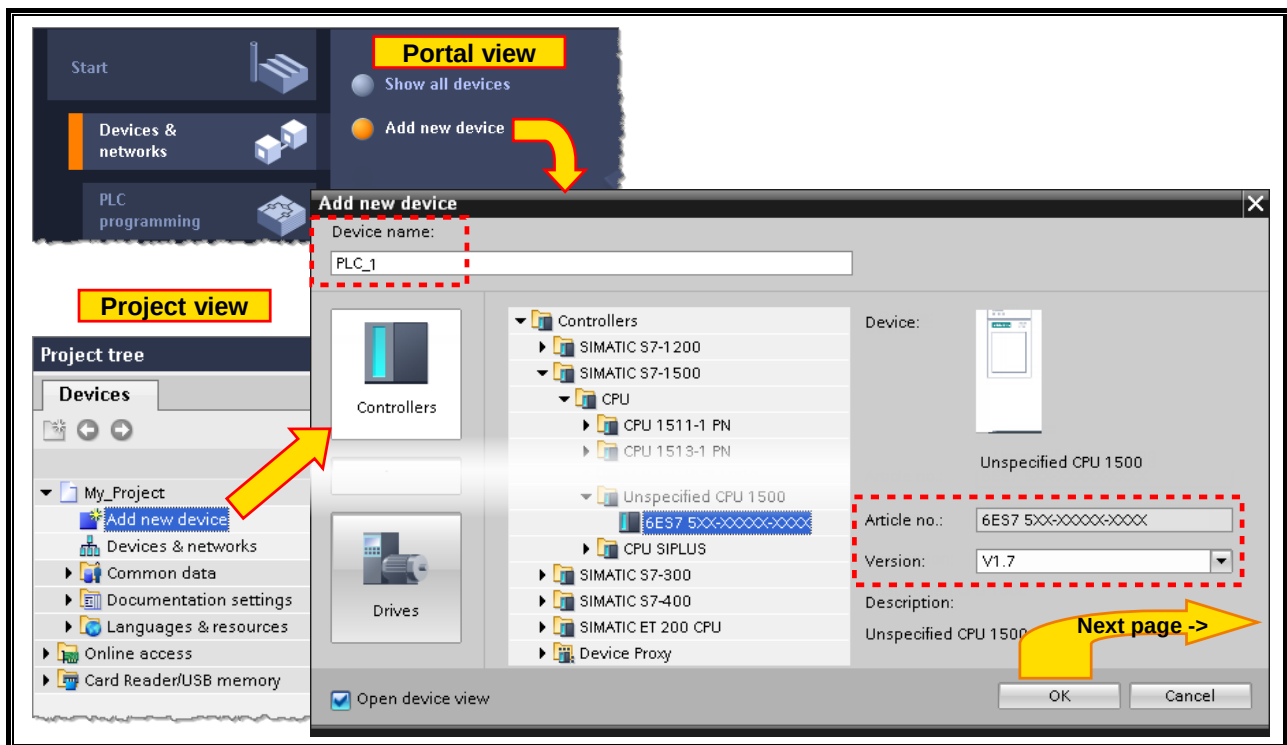
Task

A new project with the name "My_Project" is to be created.

What to Do

1. In the Portal view, activate "Create new project".
2. Enter the project name and the given path as shown in the picture and "Create" the project.

2.10.7. Exercise 7: Creating the S7-1500 Station



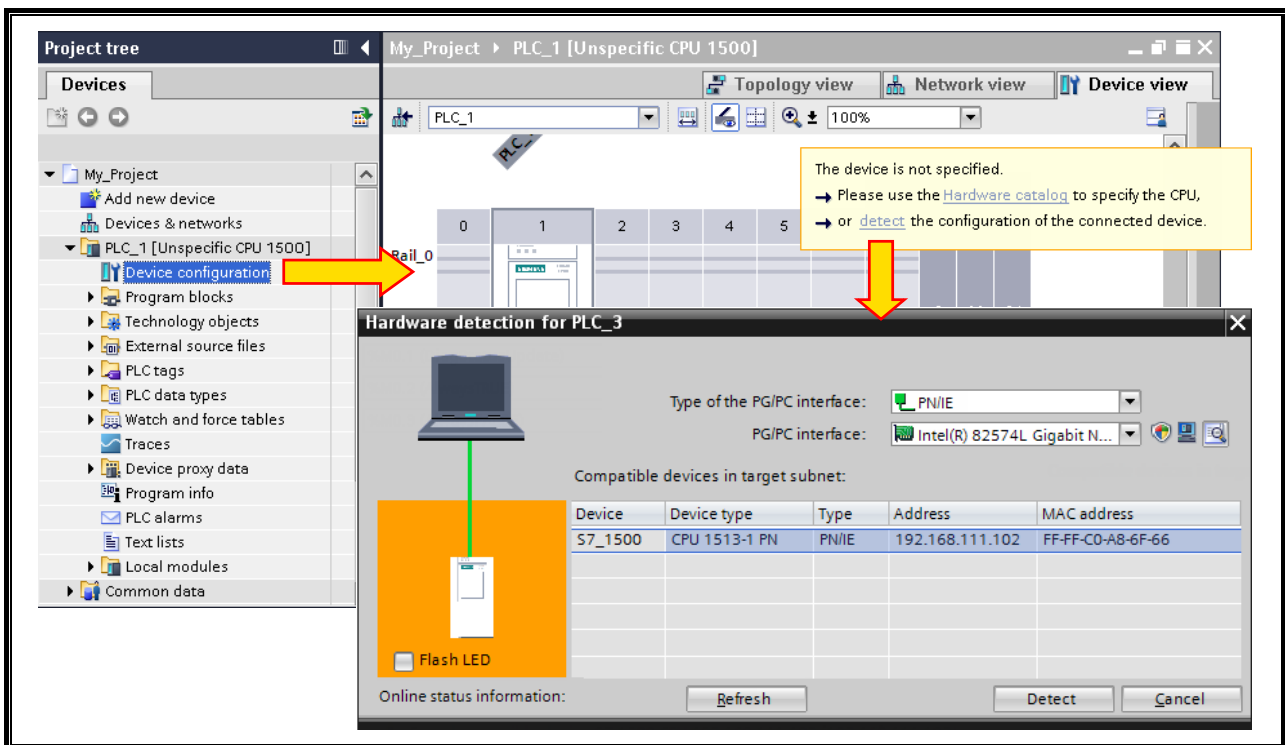
Task

As a "new device", create an S7-1500 station with an unspecified CPU.

What to Do

1. Activate the menu item: "Add new device".
2. As device, select an S7-1500 station with unspecified CPU with the appropriate Firmware version.
3. Give the device a device name.

2.10.8. Exercise 8: Reading-out the Actual Configuration



Task

You are to read out the actual configuration and parameter assignments of the modules from the CPU and save it in your project in the still "unspecific station".

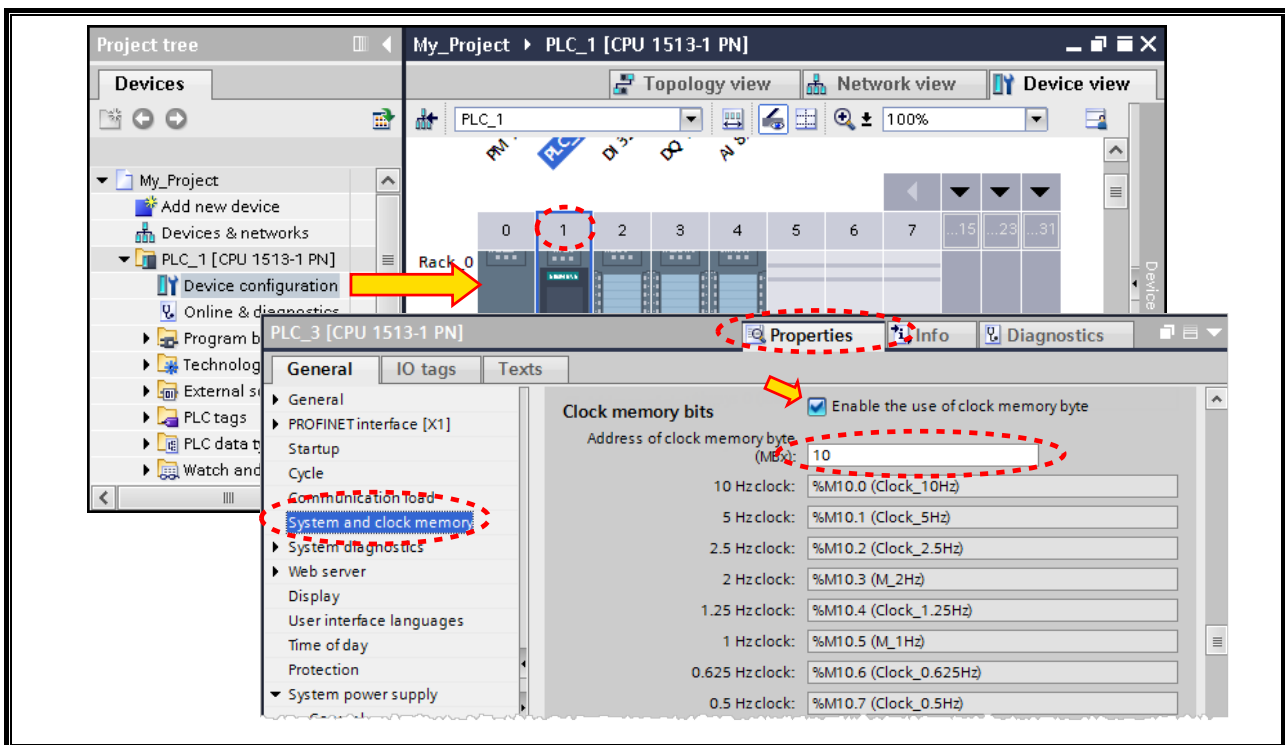
What to Do

1. Switch to the Project view.
2. Double-click on the "Device configuration" of the S7-1500 station with unspecific CPU. The Device view of the station opens with the message "The device is not specified" and the possibility of "detecting" the device.
3. Activate "detect" the configuration of the connected device".
4. In the dialog "Hardware detection", select the CPU of your station from the list "Compatible devices in target subnet" and load the configuration into the project using "Detect".
5. Save your project.

Result

The Device view shows the read-out configuration.

2.10.9. Exercise 9: CPU Properties: Parameterizing the Clock Memory Byte



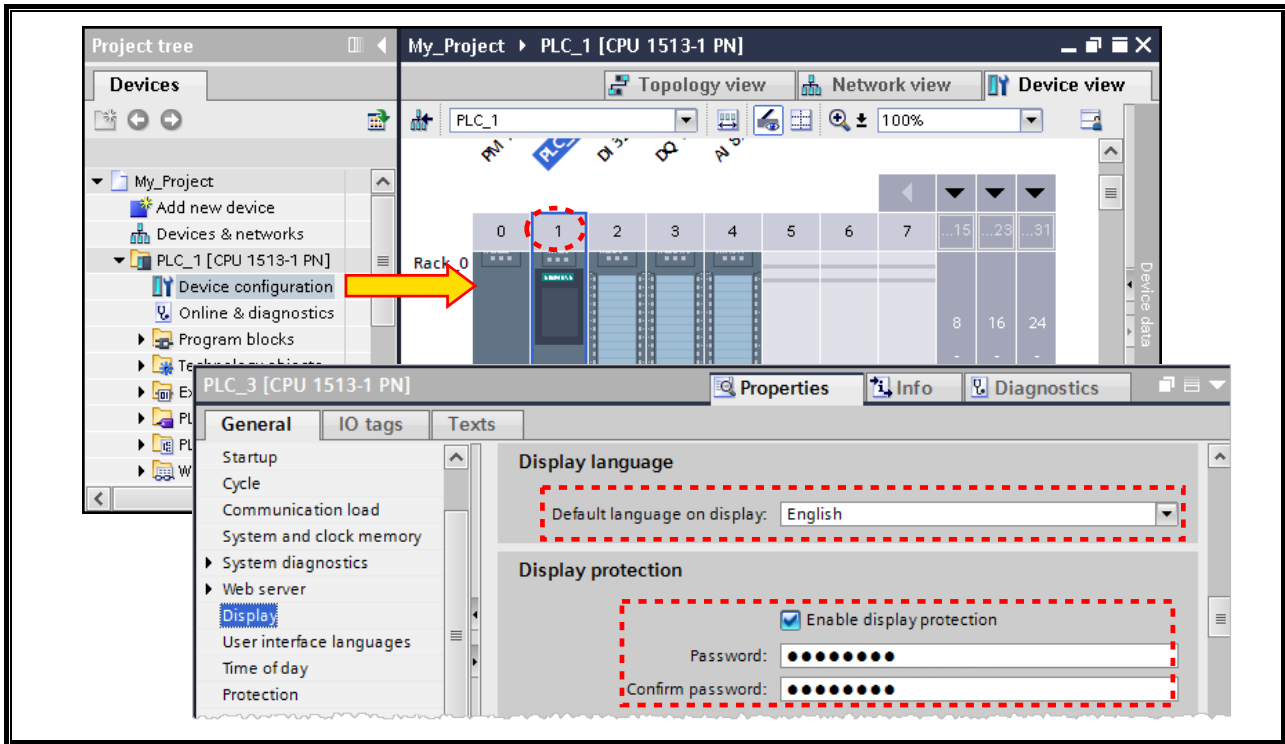
Task

In the CPU Properties, parameterize memory byte 10 as a clock memory byte.

What to Do

1. In the "Inspector window", select the "Properties" tab and there "System and clock memory".
2. Enable the "...use of clock memory byte" and parameterize MB10.

2.10.10. Exercise 10: CPU Properties: Parameterizing the Display Language and Display Protection



Task

In the CPU Properties, parameterize the display language of the CPU-Display and the display protection.

What to Do

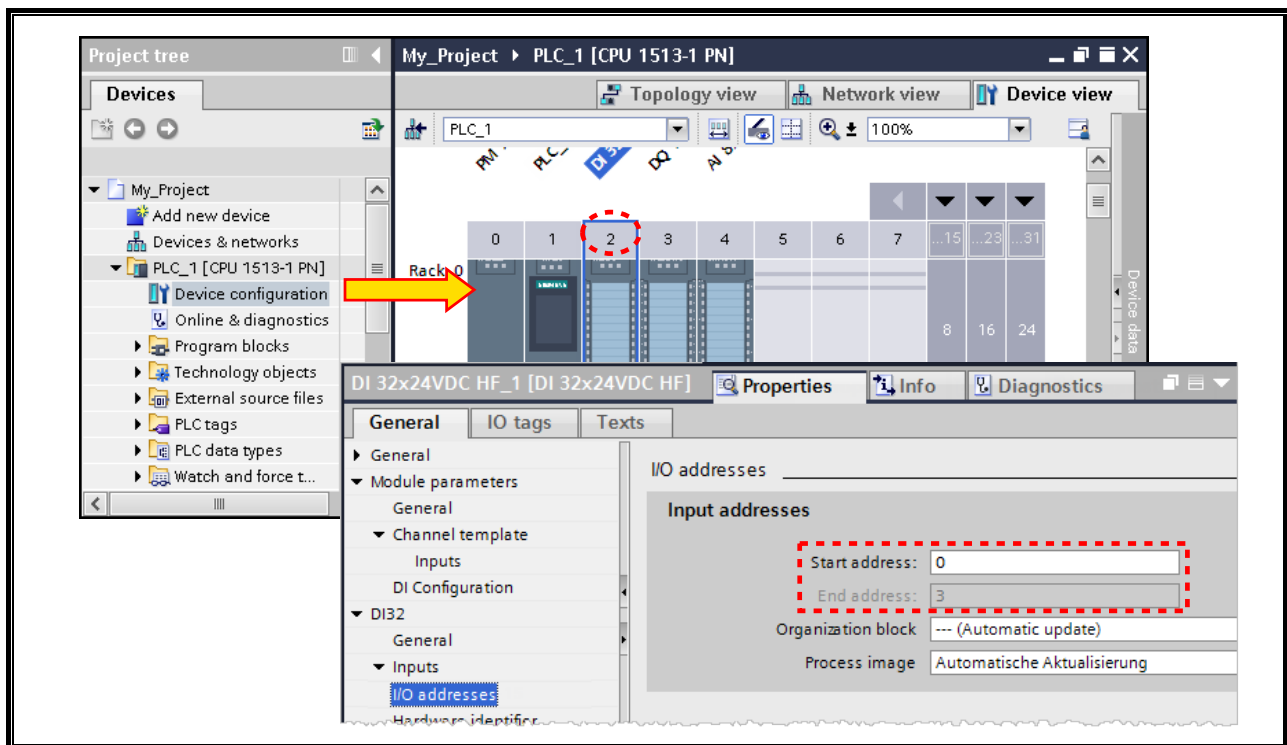
1. In the "Inspector window", select the "Properties" tab and there "Display".
2. Set the display language to "English".
3. Enable the display protection and enter a password

Note on Password Assignment:



Upper and lower case is not relevant, since only the letters A to Z and digits 0 to 9 can be selected when making entries on the Display. Since there is no Display keypad, it is recommended that you select a simple (possibly only numerical) password.

2.10.11. Exercise 11: Addresses of the DI Module



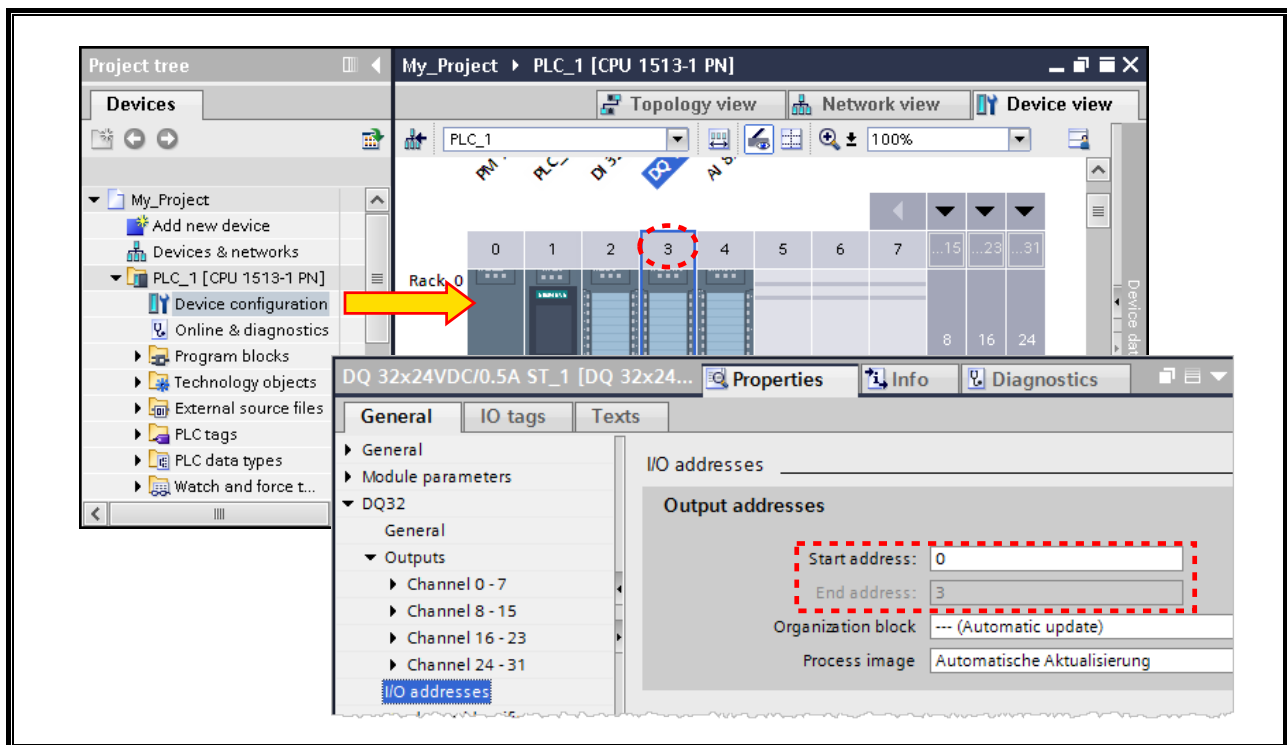
Task

Parameterize the I/O addresses of the DI module as shown in the picture.

What to Do

1. In the Device view, select the DI module (see picture).
2. In the "Inspector window", activate the "Properties" tab and there under "DI32" the "I/O addresses".
3. In the dialog box that appears, enter the I/O address 0 shown in the picture.

2.10.12. Exercise 12: Addresses of the DO Module



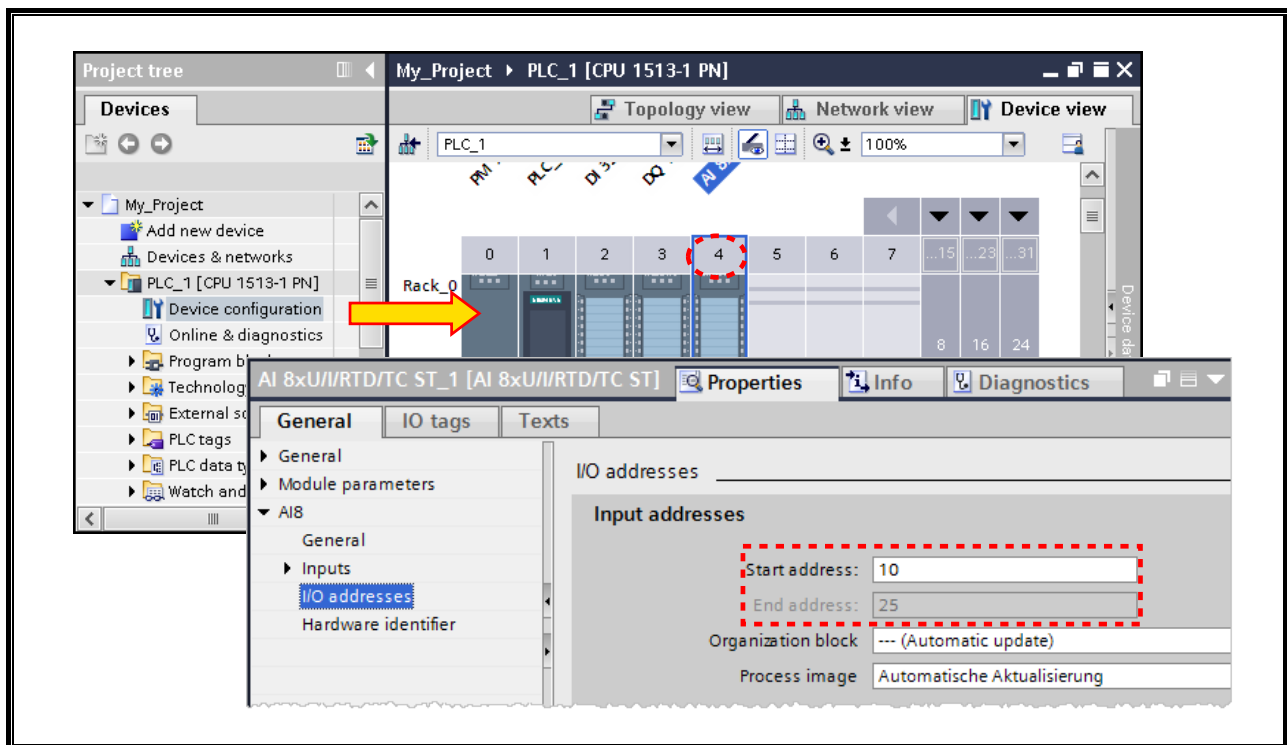
Task

Parameterize the I/O addresses of the DO module as shown in the picture.

What to Do

1. In the Device view, select the DO module (see picture).
2. In the "Inspector window", activate the "Properties" tab and there under "DO32" the "I/O addresses".
3. In the dialog box that appears, enter the I/O address 0 shown in the picture.

2.10.13. Exercise 13: Addresses of the AI Module



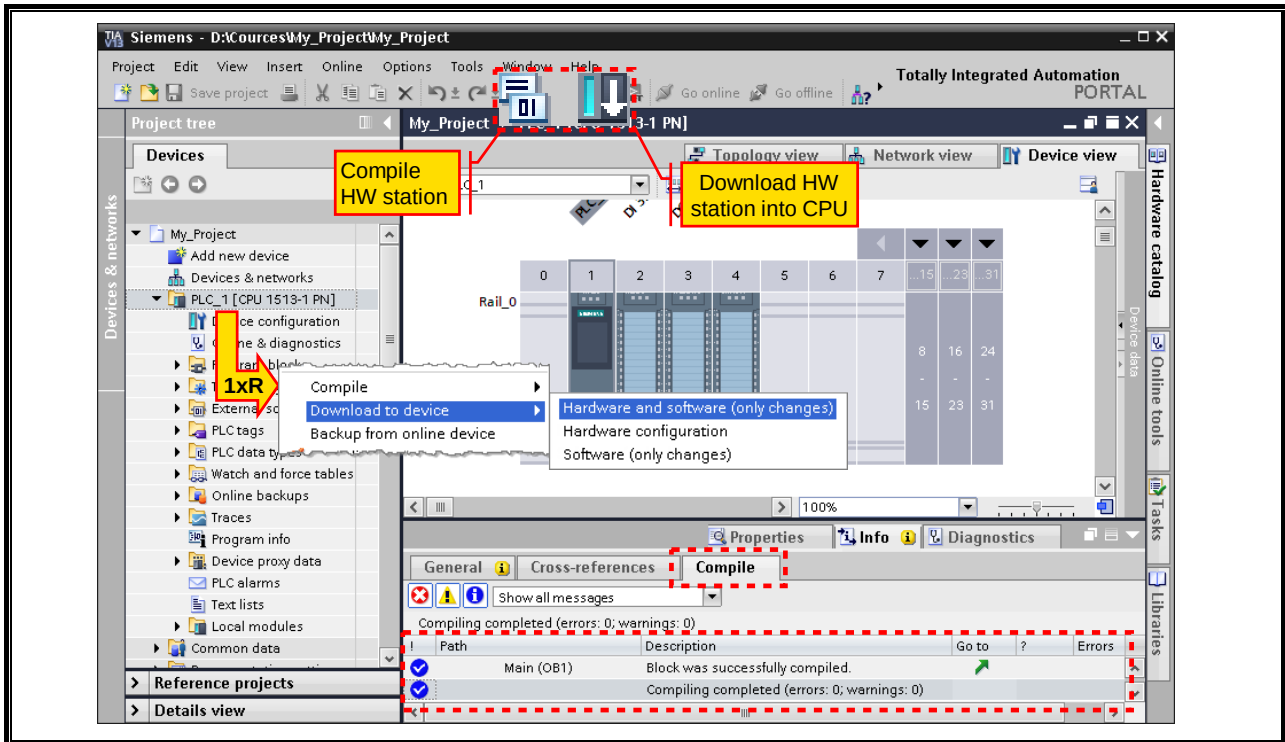
Task

Parameterize the I/O addresses of the AI module as shown in the picture.

What to Do

1. In the Device view, select the AI module (see picture).
2. In the "Inspector window", activate the "Properties" tab and there under "AI8" the "I/O addresses".
3. In the dialog box that appears, enter the I/O address 10 shown in the picture.

2.10.14. Exercise 14: Compiling the Device Configuration and Downloading It into the CPU



Task

The configuration and parameterization of the S7-1500 hardware station is to be compiled and then downloaded into the CPU. So that the CPU can switch to the RUN state after loading, the software or one CPU program must also be loaded.

At the moment, your CPU program only consists of the automatically created block "Main" (OB1), which doesn't contain any instructions yet but is sufficient so that the CPU switches to the RUN state during a restart.

What to Do

1. In the Project view, select your S7-1500 station.



2. Compile the HW-Station (either via the context menu [right-click on the station]) or via the button shown here on the left (see also picture) and in the Inspector window in the "Info" tab, check whether the compilation was completed without errors.

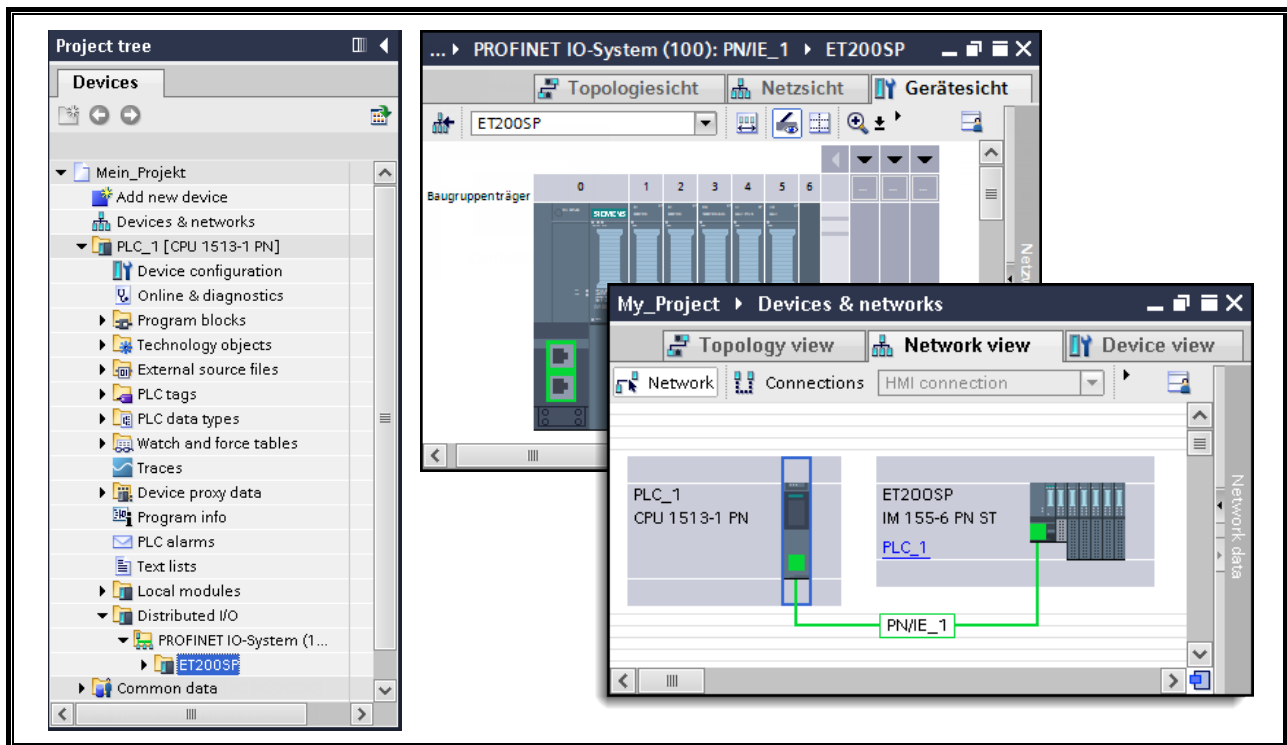


3. After an error-free compilation, download the HW-Station complete with hardware and software into the CPU (either via the context menu [right-click on the station]) or via the button shown here on the left (see also picture)
4. In the "Inspector window" under "Info -> General" check the result of the hardware configuration download:
5. Save your project.

Result:

All LEDs of the S7-1500 station show continuous green light.

2.11. Task Description: Commissioning the ET200SP



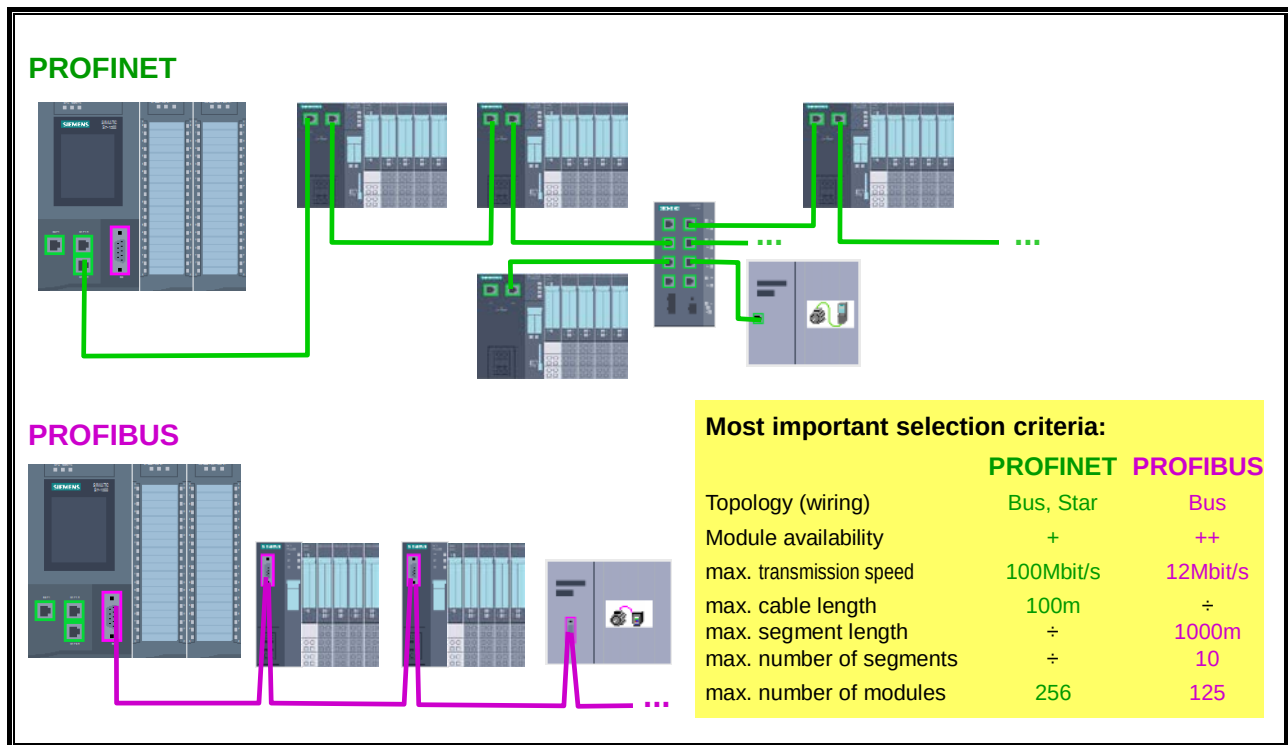
Task Description

The ET200SP distributed I/O station is to be commissioned since the conveyor model is later to be controlled via its input and output modules.

For this, the ET200SP station must be configured, assigned parameters and networked with the S7-1500 station in the offline project.

After compiling the new hardware configuration, it must be downloaded into the CPU. In the function of an I/O Controller, the CPU then automatically undertakes the parameterization of the ET200SP I/O-device.

2.12. Fieldbus Systems for SIMATIC S7



Fieldbus Systems for SIMATIC S7

To connect distributed I/O, there are different bus systems.
The most important for SIMATIC S7 are:

- PROFINET**
 as the standard for communication applications at the field level enables the connection of distributed field devices via Industrial Ethernet.
 The Industrial Ethernet network is a local area network (LAN) according to the international Standard IEEE 802.3 (Ethernet) and is designed for the industrial sector. It enables open and comprehensive network solutions with a high transmission performance.
- PROFIBUS**
 is the bus system for local area networks (LANs) with only a few participants. Through its fulfillment of requirements according to EN 50170, PROFIBUS ensures openness for the connection of standard-conforming components of all manufacturers.

Due to the physical and communication-related differences of the two bus systems, there are various criteria for the selection of the most suitable bus system.

Module Availability

Since PROFIBUS has already been established for a very long time in the area of fieldbus systems, a very wide spectrum of modules exists here.

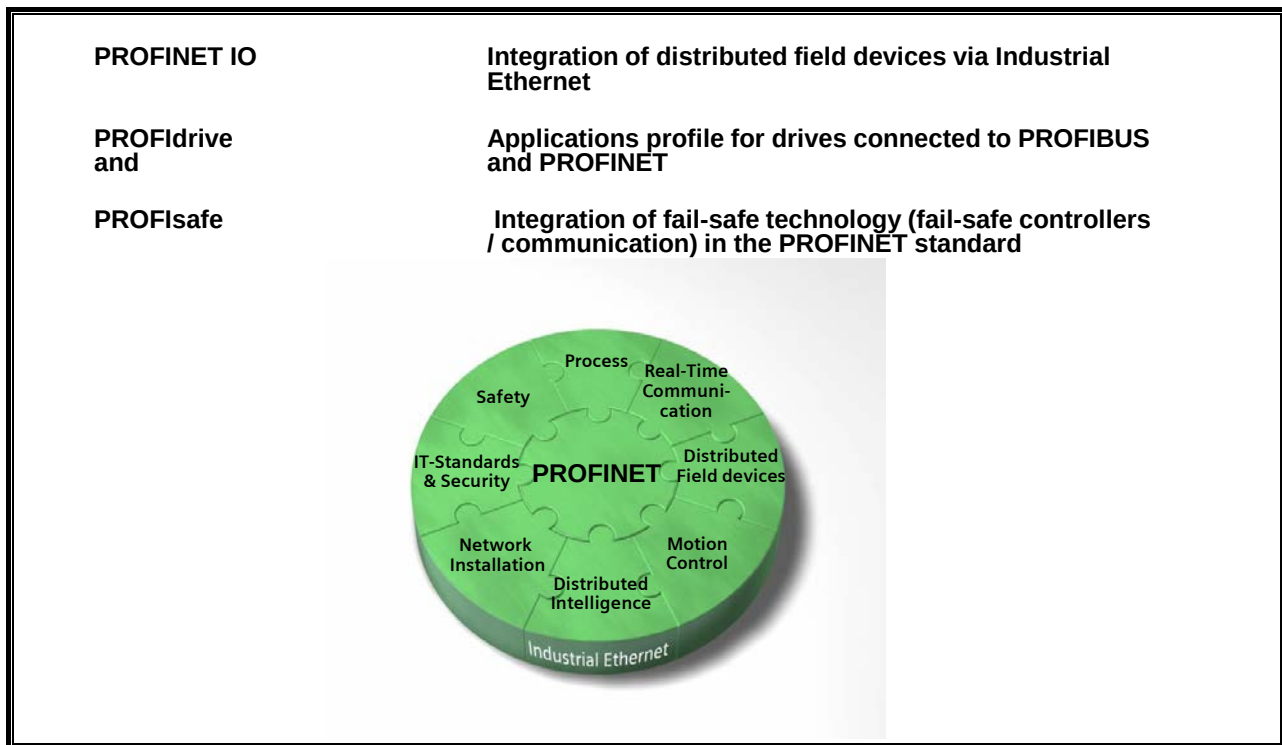
PROFINET is currently still a relatively "recent" fieldbus system where the diversity of modules is still evolving.

Cable Length, Segment Length

For PROFIBUS, a module line has to be reinforced after 100-1000m (depending on the transmission speed used); otherwise, the maximum bus length is reached.

For PROFINET, every connected component takes over this function. For that reason, only the cable length between two modules is relevant here.

2.13. Components of the PROFINET Standard



PROFINET

It completely covers the requirements of automation. PROFINET brings together the expertise of PROFIBUS and Industrial Ethernet. The utilization of the open standard, the easy handling and the integration of existing parts of a system (e.g. a plant) determined the definition of PROFINET right from the beginning. Today, PROFINET is integrated in IEC 61158.

PROFINET IO

With PROFINET IO, the integration of distributed field devices takes place directly on the Ethernet. For that, the Master-Slave procedure from PROFIBUS DP is carried over into a Provider-Consumer model. From the communication point of view, all devices on the Ethernet have equal rights. Through the configuration, however, the field devices are assigned to a central controller. As a result, the familiar user view in PROFIBUS is transferred to the PROFINET I/Os: The distributed I/O device reads in the I/O signals and transfers them to the controller. The controller processes them and transfers the outputs back to the distributed I/O device.

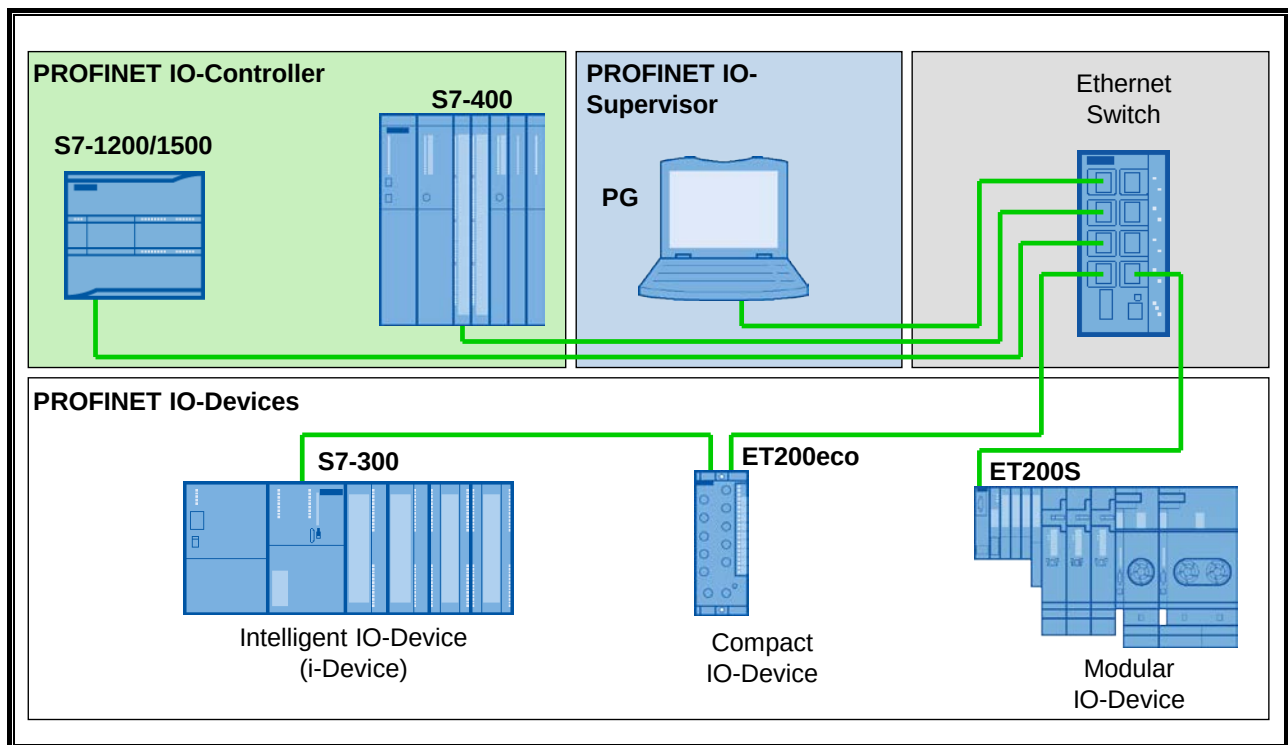
PROFIdrive

With PROFIdrive, very fast, clock-synchronous drive controls for high performance Motion Control applications will be implemented in the future.

PROFIsafe

With PROFIsafe, the network infrastructure already existing for standard communication can also be used at the same time for fail-safe communication. Fail-safe and standard data is transmitted through the same bus line. The existing bus protocols, such as, PROFIBUS and PROFINET (so-called "black channel") are used to transport fail-safe data as additional data (so-called PROFIsafe layer). With that, the fail-safe communication is independent of the bus system and the lower-level networks.

2.13.1. PROFINET IO Device Types



PROFINET IO-Controller

The IO-Controller (typically the PLC) establishes a logical connection to the connected IO-Devices after Power-On and subsequently parameterizes these (module parameters, address, etc.). (This corresponds to the function of a Class 1 Master in PROFIBUS).

PROFINET IO-Device

An IO-Device is a distributed IO device that is connected via PROFINET IO (this corresponds to the function of a slave in PROFIBUS).

Differentiation is made for the following IO-Device types:

- Compact IO-Device: Fixed degree of expansion.
- Modular IO-Device: Variable degree of expansion; can be expanded or reduced as required.
- Intelligent IO-Device: A PLC is configured not as an IO-Controller but as an IO-Device and provides a higher-level controller with I/O data.

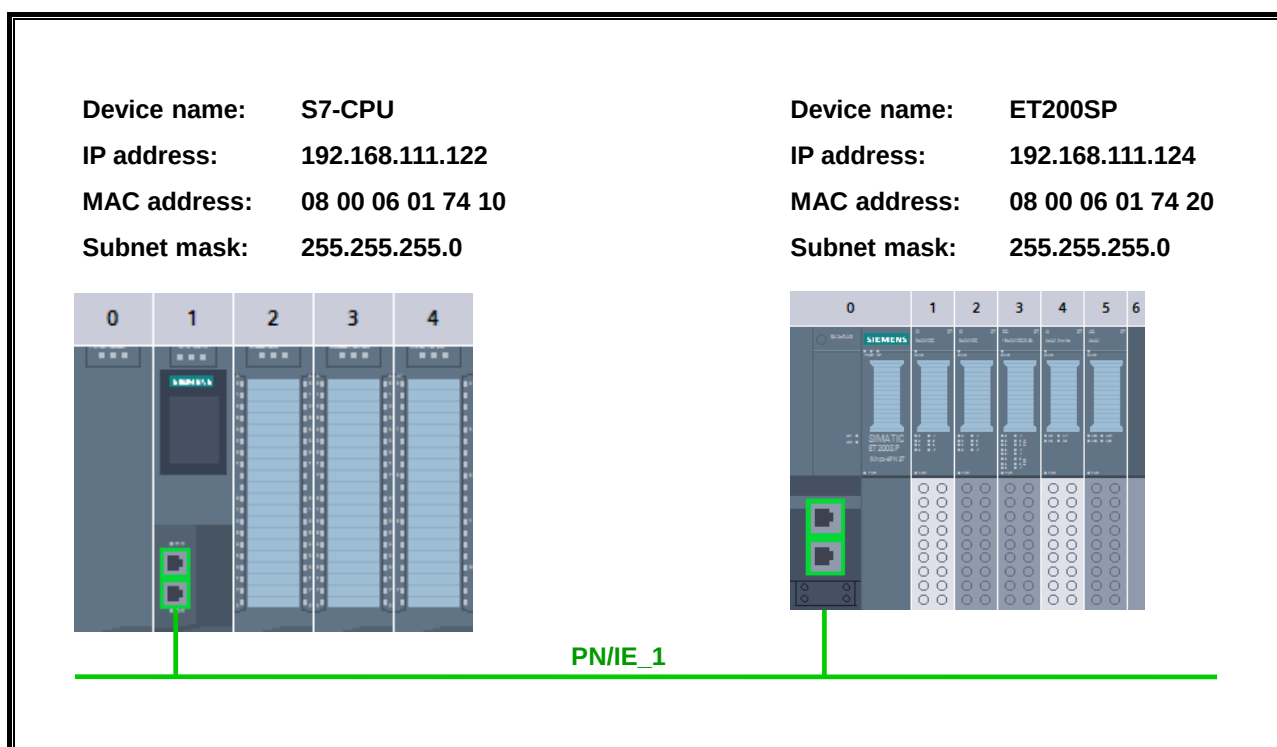
IO-Supervisor

This can be a programming device (PG), personal computer (PC) or Human Machine Interface (HMI) for commissioning or diagnostic purposes. (This corresponds to a Class 2 Master in PROFIBUS).

Ethernet Switch

PROFINET is based on Ethernet. For that reason, switches are always used as "network distributors". Every node is connected to a switch via a so-called "point-to-point" connection. This is also referred to as a **"Switched Ethernet"**. In most PROFINET devices, a 2 or multi-port switch is already integrated so that it is very easy to establish a line structure (comparable to PROFIBUS).

2.13.2. PROFINET Addresses



Internet Protocol

The Internet **P**rotocol (**IP**) is the basis for all TCP/IP networks. It creates the so-called datagrams (data packets specially tailored to the Internet protocol) and handles their transport within the local subnet or their "routing" (forwarding) to other subnets.

IP Addresses

IP addresses consist of 4 bytes. With the dot notation, each byte of the IP address is expressed by a decimal number between 0 and 255. The four decimal numbers are separated by dots (see picture).

PROFINET Device Name

In PROFINET, each RT / IRT device must be assigned a unique device name that is retentively stored in the device. A module exchange without PG/PC is made possible through the device names.

MAC Address

Every Ethernet interface is assigned a fixed address by the manufacturer that is unique worldwide. This address is referred to as the hardware or MAC address (**Media Access Control**). It is stored on the network card and uniquely identifies the Ethernet interface in a local network. Cooperation among the manufacturers ensures that the address is unique worldwide.

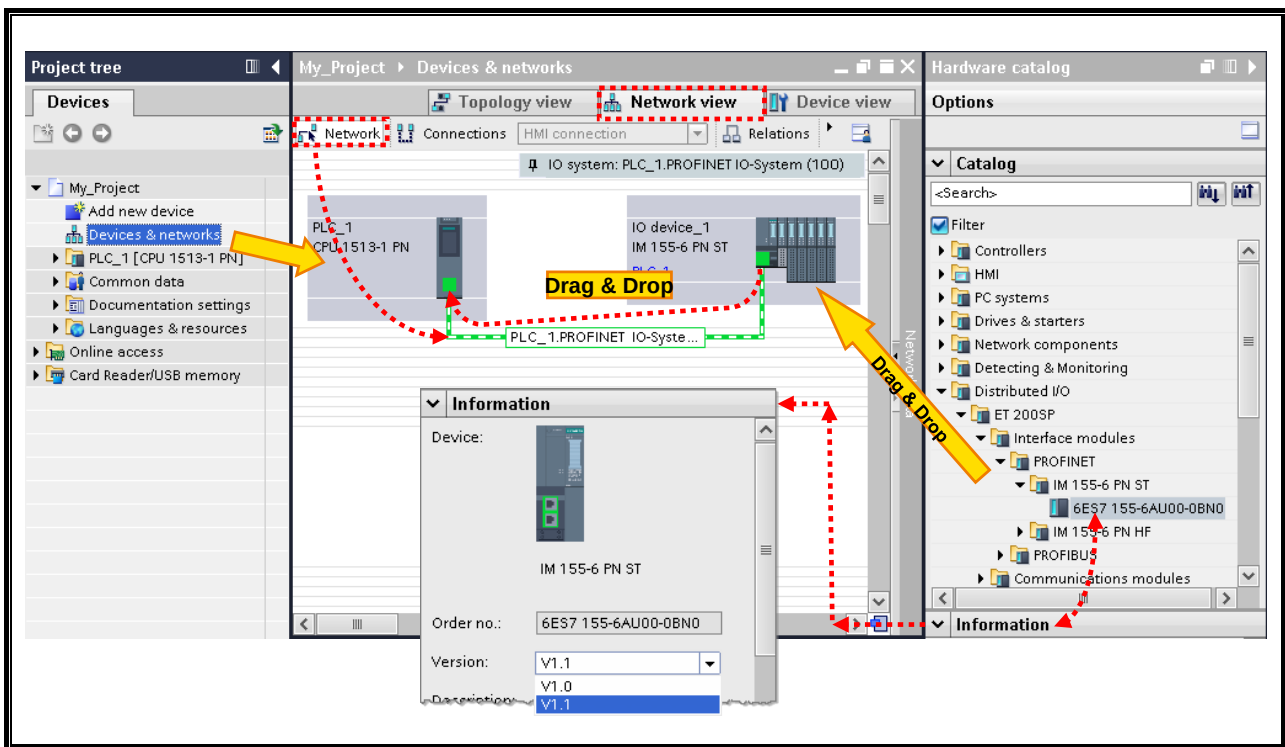
Subnet Mask

The subnet mask specifies which IP addresses in the local network can be accessed. It separates the IP address into the network and device part.

Only IP addresses whose network part is the same can be accessed.

e.g.: Subnet mask = **255.255.255.0** and IP address = **192.168.111.10**
 accessible IP addresses: **192.168.111.1** to **192.168.111.254**

2.13.3. Inserting and Networking Distributed I/O



Inserting Distributed I/O

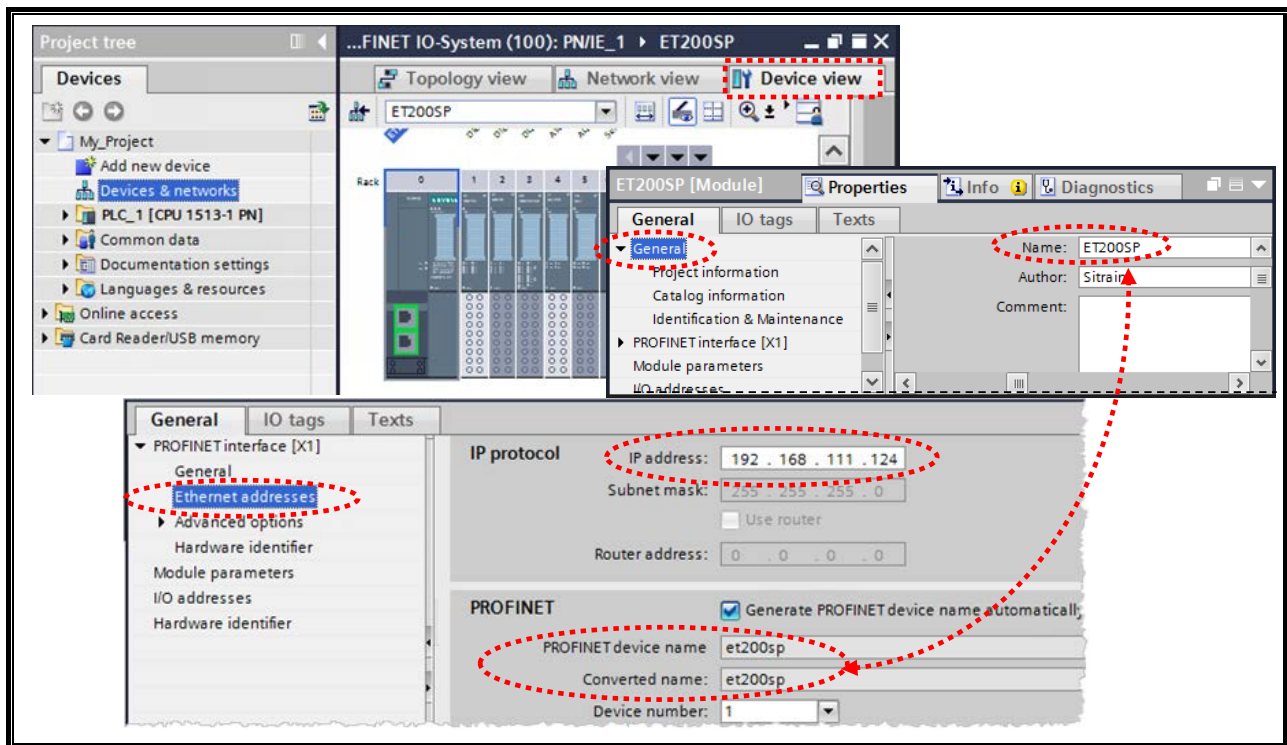
PROFINET IO-Devices are added in the Network view. Here, you can insert the relevant devices into the project by dragging & dropping them from the Hardware catalog.

In the beginning, the newly added ET200SP is not assigned to any controller and therefore appears in the Project tree as a "Not assigned device" in the same level as the PLCs and HMIs.

Networking Distributed I/O

After the ET200SP IO-Device is added, it must now be assigned to an IO-Controller or networked with a CPU. In case there are several CPUs in the network, a co-ordination or monitoring of the I/O addresses by the IO-Controller and IO-Device can only be done through this unique assignment.

2.13.4. PROFINET IO Device ET200SP: Assigning the IP Address and Device Name OFFLINE



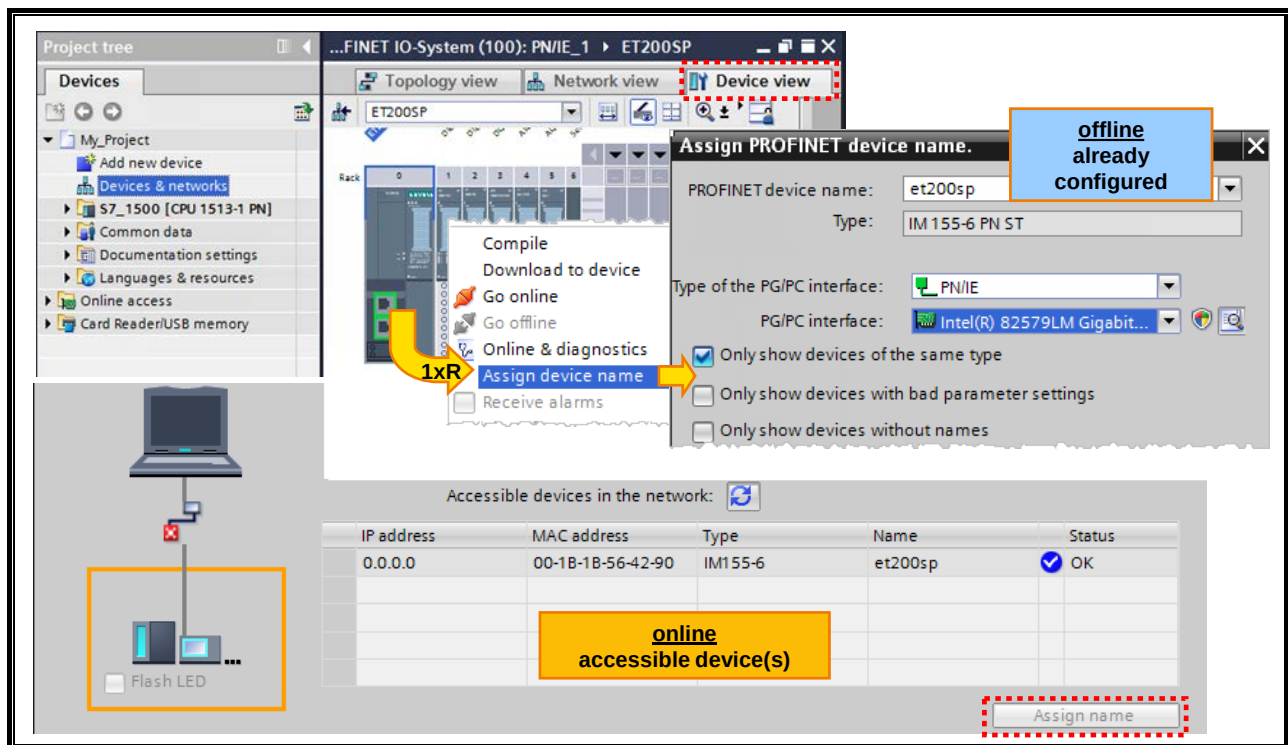
IP Address, Subnet Mask and PROFINET Device Name

For communication with the IO-Controller, a PROFINET device name must be assigned to the IO-Device (ET200SP) OFFLINE. The IO-Controller then assigns the IO-Device a valid IP address. If the IO-Device is assigned an IP address OFFLINE, this IP address is adopted. These parameters are downloaded to the IO-Controller (CPU) with the programming device. The IO-Controller (CPU) then transfers these and other parameters (such as, the I/O addresses) to the IO-Device (ET200SP).

Attention:

Only the PROFINET device name is relevant for the transmission of the offline configuration into the online device, not the IP address. The offline configured PROFINET device name and the online existing PROFINET device name must match. If the IO-Device has a different PROFINET device name or doesn't have a name at all, the IO-Controller cannot transfer the hardware configuration or the hardware parameter assignments to the IO-Device thus preventing a PROFINET system startup.

2.13.5. PROFINET IO Device ET200SP: Assigning the Device Name ONLINE



IP Address and PROFINET Device Name

The PROFINET device name of the IO-Device configured offline and the device name existing online must match since the IO-Controller first checks the device names of the connected IO-Devices and then assigns the configured IP addresses during system startup. If an IO-Device is not accessible under its configured device name, the IO-Controller cannot establish a connection to the IO-Device.

The IP address of the IO-Device configured offline and the address existing online do not have to match. The PROFINET name is relevant for the downloading of the hardware configuration. If it exists, the online existing IP address is overwritten with the offline configured IP address.

Ways of Assigning a Name Online

In principle, there are two ways of assigning a PROFINET device name to an IO-Device online:

Version 1 (safe, since there is no chance of typing errors)

The assignment of the device name is triggered from the device configuration of the IO-Device.

Device configuration of IO-Device → Right-click on the Interface module (Slot 0) → Online & diagnostics → Functions → Assign name (see picture)

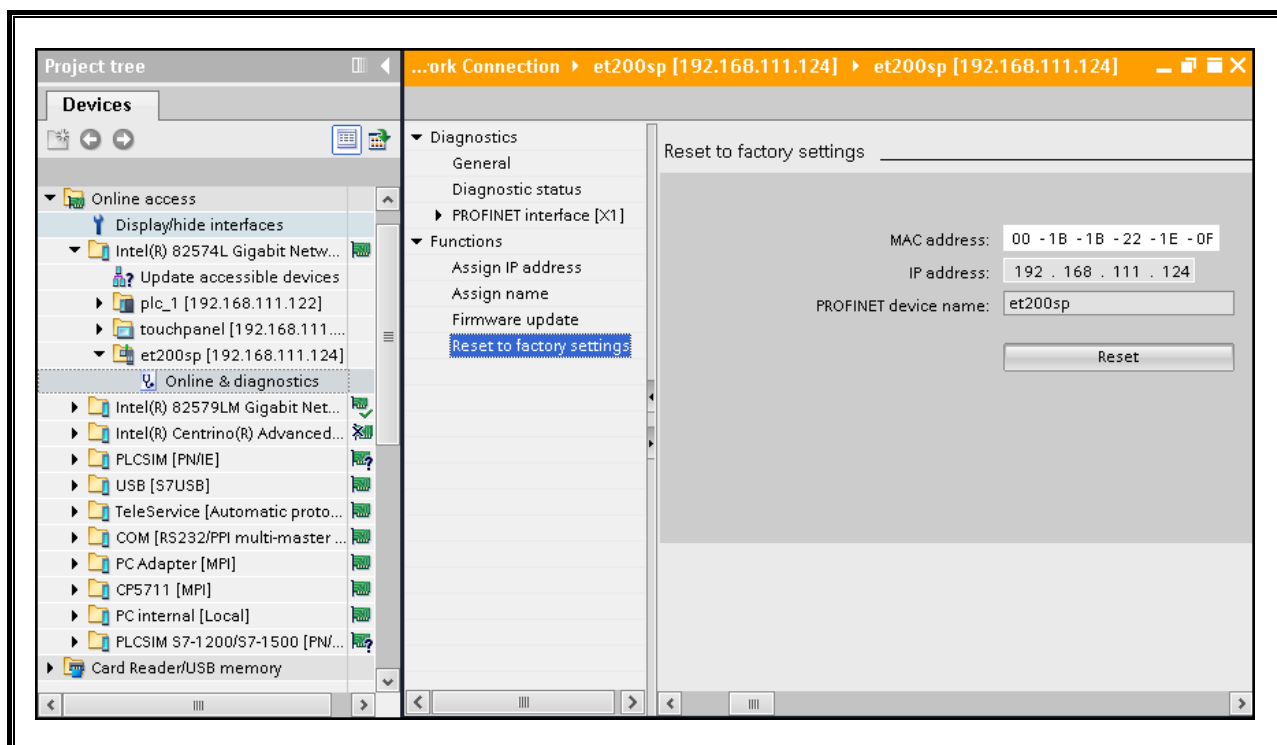
The advantage of this version is that the offline configured device name and the IP address are adopted 1:1 and so no typing errors can be made.

- Version 2 (typing errors possible)

The assignment of the device name is triggered via "Online accesses":

Project tree → Online accesses → Ethernet interface → IO-Device → Online & diagnostics → Functions → Assign name

2.13.6. Exercise 15: ET200SP: Reset to Factory Settings



Task

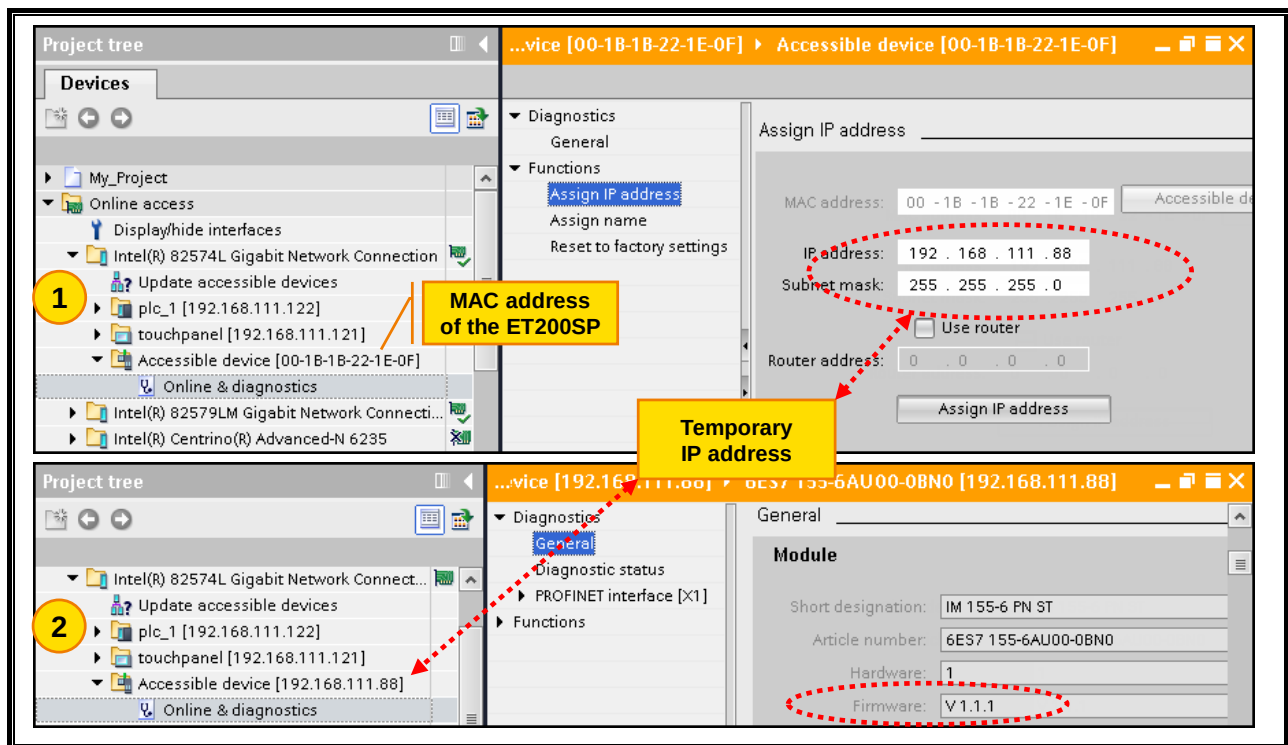
All settings so far (IP address, subnet mask and PROFINET name) of the Interface module and the memory card of the ET200SP station are to be deleted through a "Reset to factory settings". In the following exercises, you will then transfer your own settings onto the ET200SP station.

What to Do:

1. Open the Online accesses and there select the interface "Intel(R) 82574L Gigabit..."
2. There, activate "Update accessible devices" by double-clicking on it and wait until the list is completed.
3. Open the ET200SP and there activate the function "Online & diagnostics" by double-clicking on it.
4. In the "Online & diagnostics" window, open the "Functions" tab
5. There, activate "Reset to factory settings".
6. Close the "Online & diagnostics" window.
7. Check the success of the reset to factory settings by once again doing an "Update accessible devices" under
Online accesses -> Intel(R) 82574L Gigabit...". The reset ET200SP should now only be displayed as "Device" with MAC address.

Leave all windows open for the next exercise.

2.13.7. Exercise 16: Reading-out the Firmware Version of the ET200SP



Task

In the following exercises, in order to be able to configure an ET200SP in the offline project which corresponds exactly to that of the training unit, you now have to read out the Firmware version of the ET200SP online.

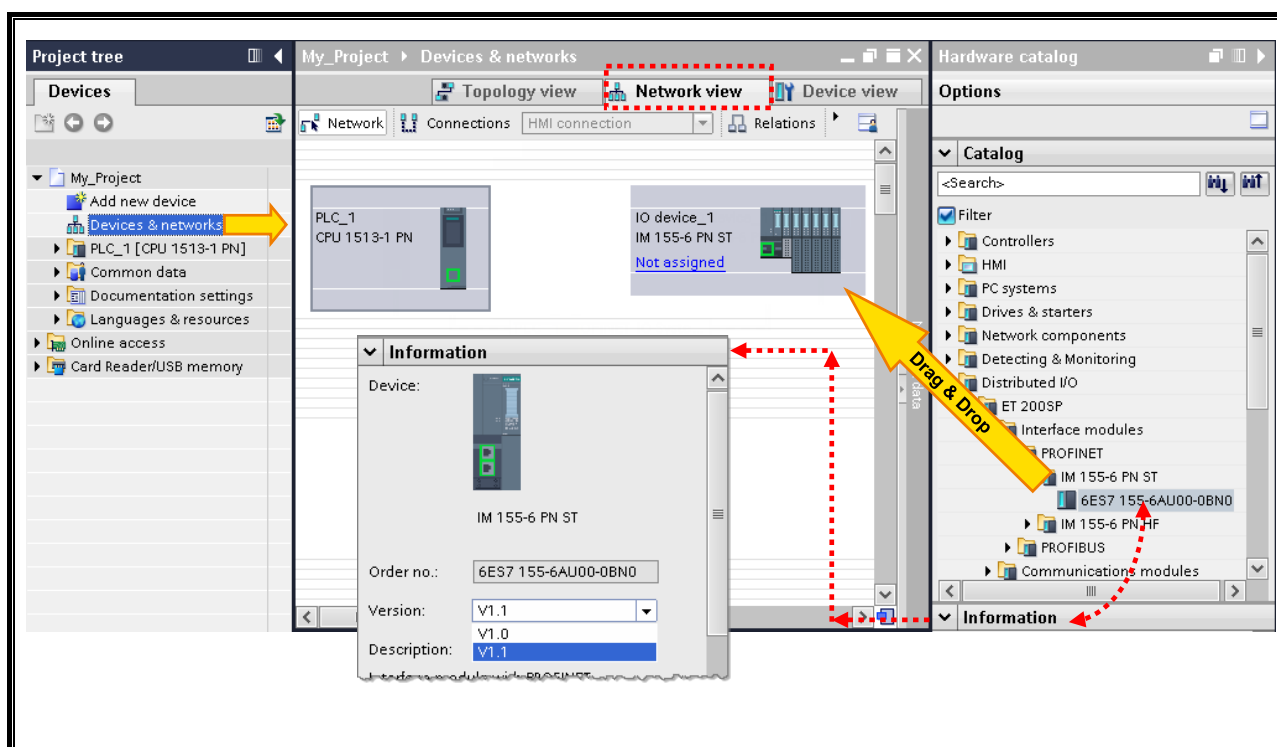
Problem

Due to the previous "Reset to factory settings", the ET200SP now no longer has a PROFINET device name nor an IP address, only a MAC address (see top picture). The Firmware version, however, cannot be read out via the MAC address, since an IP address is required for this diagnostic service.

What to Do

1. Open the ET200SP and, with a double-click, activate the "Online & diagnostics" function and there check whether the ET200SP Firmware version is displayed in the tab "Diagnostics -> General".
2. No Firmware version is displayed since the ET200SP doesn't have an IP address. To assign a temporary IP address, switch to the "Functions -> Assign IP address" tab. There enter the temporary IP address as well as the subnet mask shown in the picture and confirm via "Assign IP address" (see top picture).
3. In the Project tree, once again show the list of "Update accessible devices".
4. In the device list, the ET200SP is now displayed as a device with Article (Order) number and IP address. Once again activate "Online & diagnostics" (see bottom picture).
5. Make note of the Firmware version shown in the "Diagnostics -> General" tab.
6. Close the window and then the "Online accesses" in the Project tree.

2.13.8. Exercise 17: Offline Project: Adding the ET200SP



Task

An ET200SP is to be inserted into the project as a distributed I/O station.

PROFINET IO-Devices are added in the Network view. Here, you can insert the relevant devices into the project by dragging & dropping them from the Hardware catalog.

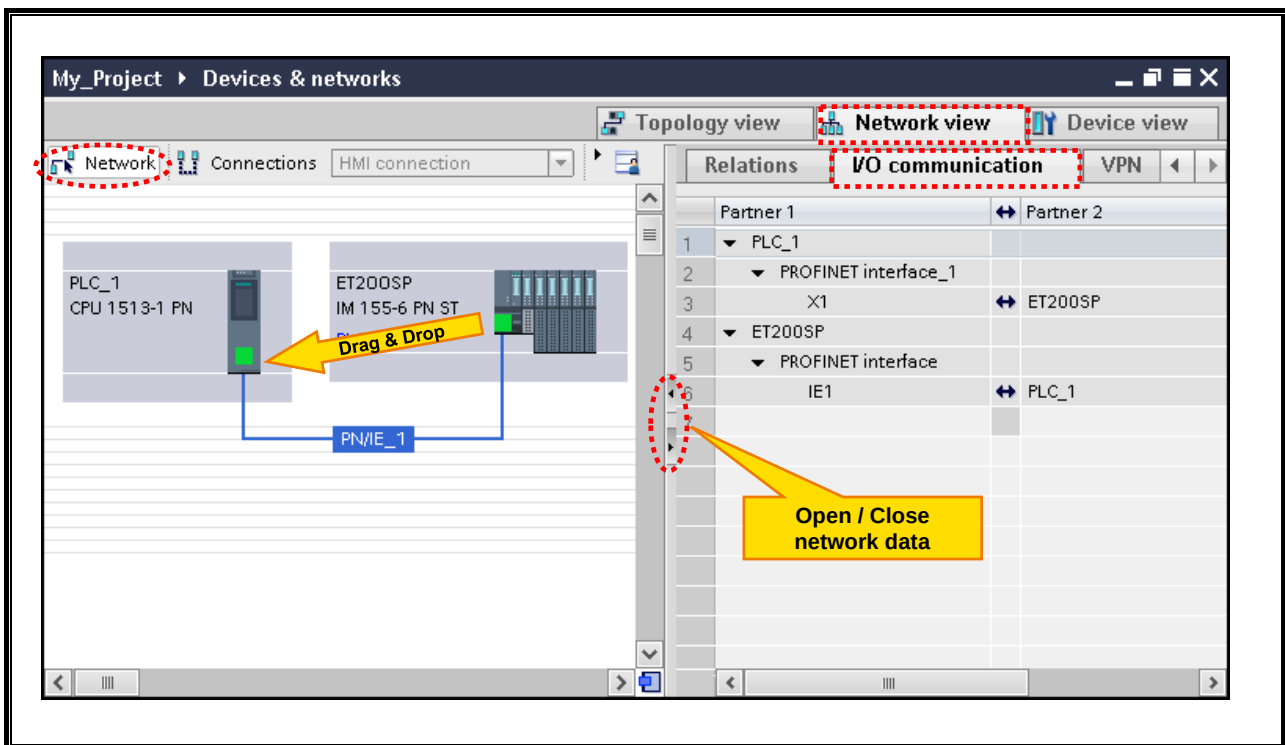
In the beginning, the newly added ET200SP is not assigned to any controller and therefore appears in the Project tree as a "Not assigned device" in the same level as the PLCs and HMIs.

What to Do

1. In the Project tree, open the "Devices & networks" editor by double-clicking on it.
2. Open the Hardware catalog Task Card and there
Distributed I/O -> ET200SP -> Interface modules -> PROFINET -> IM155-6PN ST
3. Select the IM module used in your training unit, open the Information window and there select the Firmware version of your IM module.
4. Using drag & drop, drag the IM module into the "Devices & networks" editor (see picture).

Leave all windows open because they are still needed for the next exercises!

2.13.9. Exercise 18: Networking the ET200SP



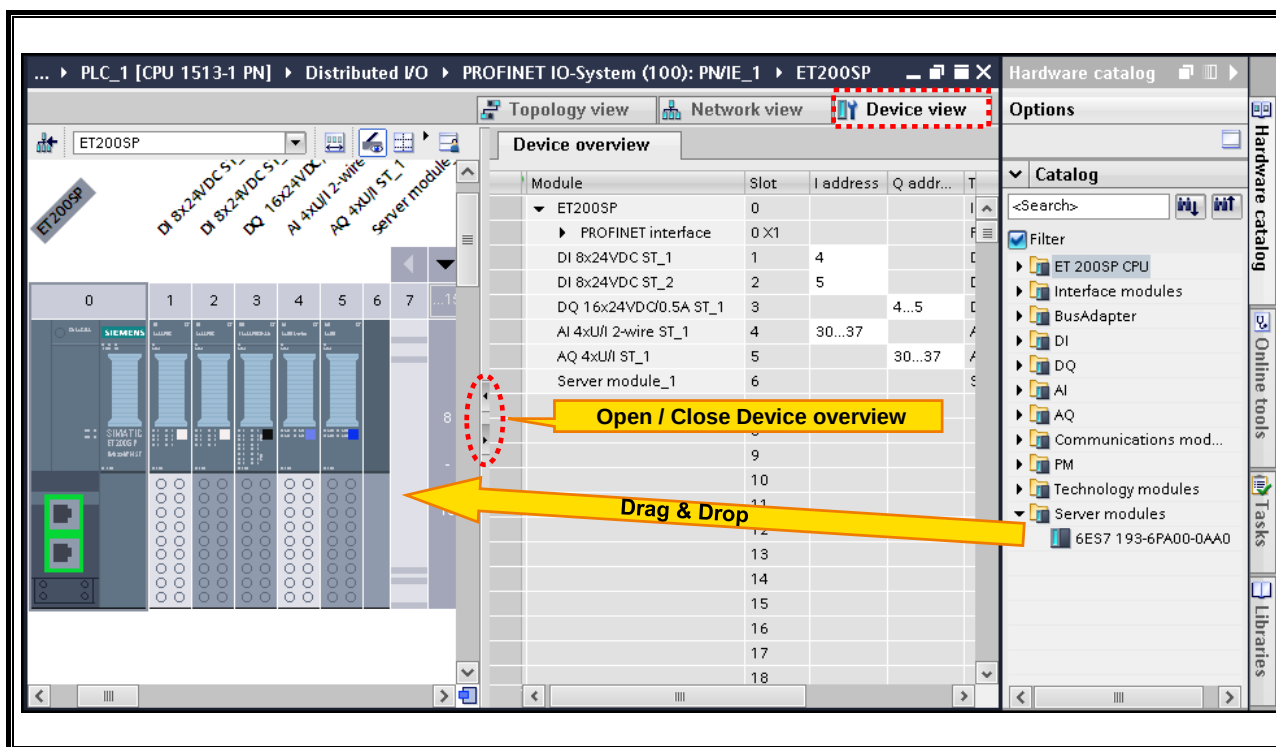
Task

After the ET200SP IO-Device is added, it must now be assigned to an IO-Controller or networked with a CPU. In case there are several CPUs in the network, a co-ordination or monitoring of the I/O addresses by the IO-Controller and IO-Device can only be done through this unique assignment.

What to Do

1. In the "Devices & networks" editor, select the Network view and there choose the "Network" tab.
2. Network the ET200SP with the CPU by connecting the Ethernet interface of the ET200SP with the Ethernet interface of the CPU using drag & drop.
3. Select the newly created PROFINET IO system and, in the Inspector window under "IO communication", check the generated communication partners.

2.13.10. Exercise 19: Configuring and Parameterizing the ET200SP



Task

The configuration of the ET200SP in the offline project must match exactly with the configuration of your training device. Attention should be given in particular to the article (order) numbers and versions of the modules.

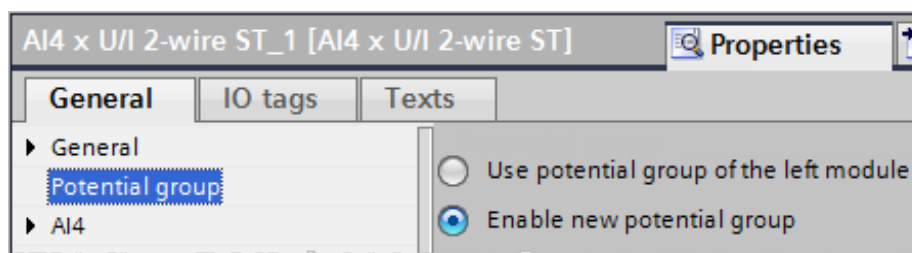
The ET200SP has digital and analog input and output modules to which the conveyor model is to be connected in the following. The I/O addresses used in the STEP 7 program must match the addresses of the DI/DO modules parameterized here.

The current address assignment is located in the lower section of the working area in the "Devices & networks" editor in the "Device view" tab of the module. The addresses can be changed in the table.

What to Do

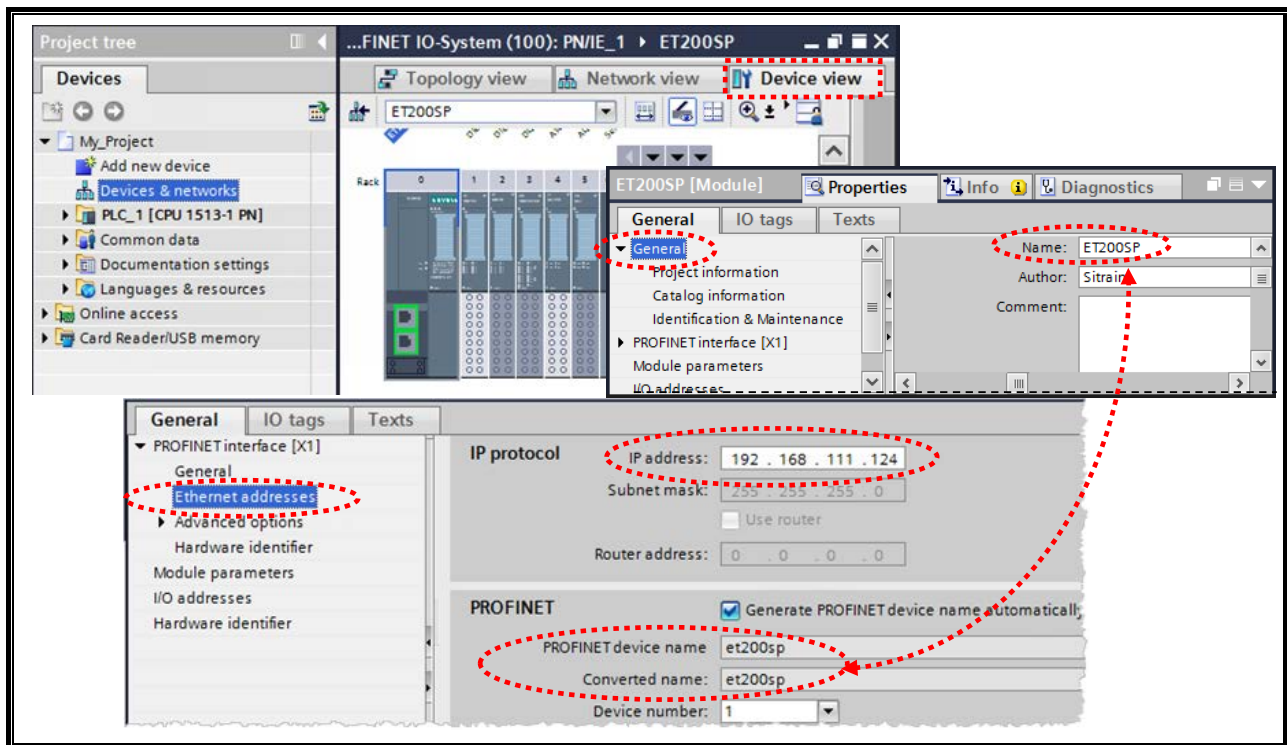
1. In the "Devices & networks" editor, select the "Device view" of the ET200SP.
2. In the Task Cards, open the "Hardware catalog".
3. Configure the ET200SP station according to your training unit.

Ensure that a new potential group is opened with the AI module on Slot 4:



4. Open the lower section of the working area of the "Devices & networks" editor (see picture) and, in the table, enter the I/O addresses shown in the picture.
5. Save your project.

2.13.11. Exercise 20: ET200SP: Assigning the IP Address / PROFINET Name OFFLINE



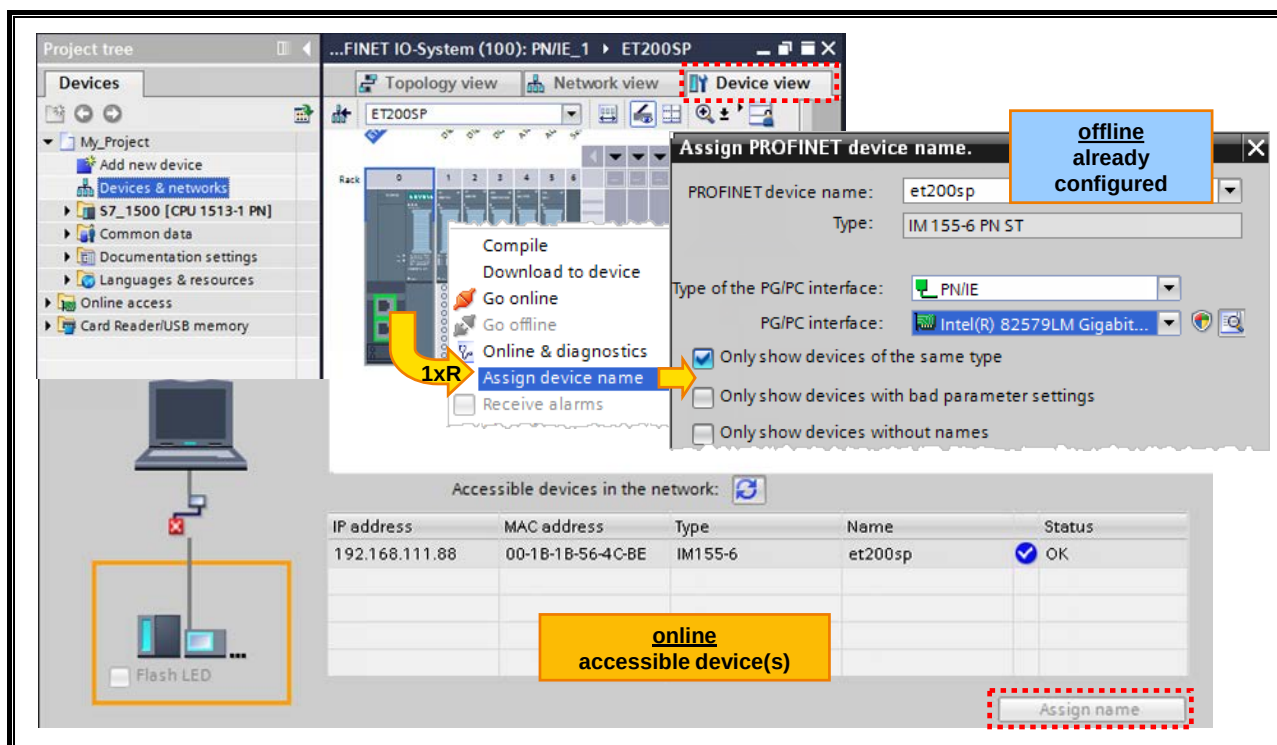
Task

The ET200SP station is later to work with the IP address, subnet mask and PROFINET device name shown in the pictures above.

What to Do

1. In the "Devices & networks" editor, select the "Device view" of the ET200SP.
2. Select the IM module on Slot 0 and open the "Properties" tab in the Inspector window.
3. There, select the "General" tab and under "Name" enter the PROFINET device name (see middle right picture).
4. Then select the "Ethernet addresses" tab and under "IP protocol" enter the IP address and subnet mask shown (see lower picture). In the same tab you will also find the PROFINET device name that you previously edited in the "General" tab.
5. Save your project.

2.13.12. Exercise 21: ET200SP: Assigning the PROFINET Name ONLINE



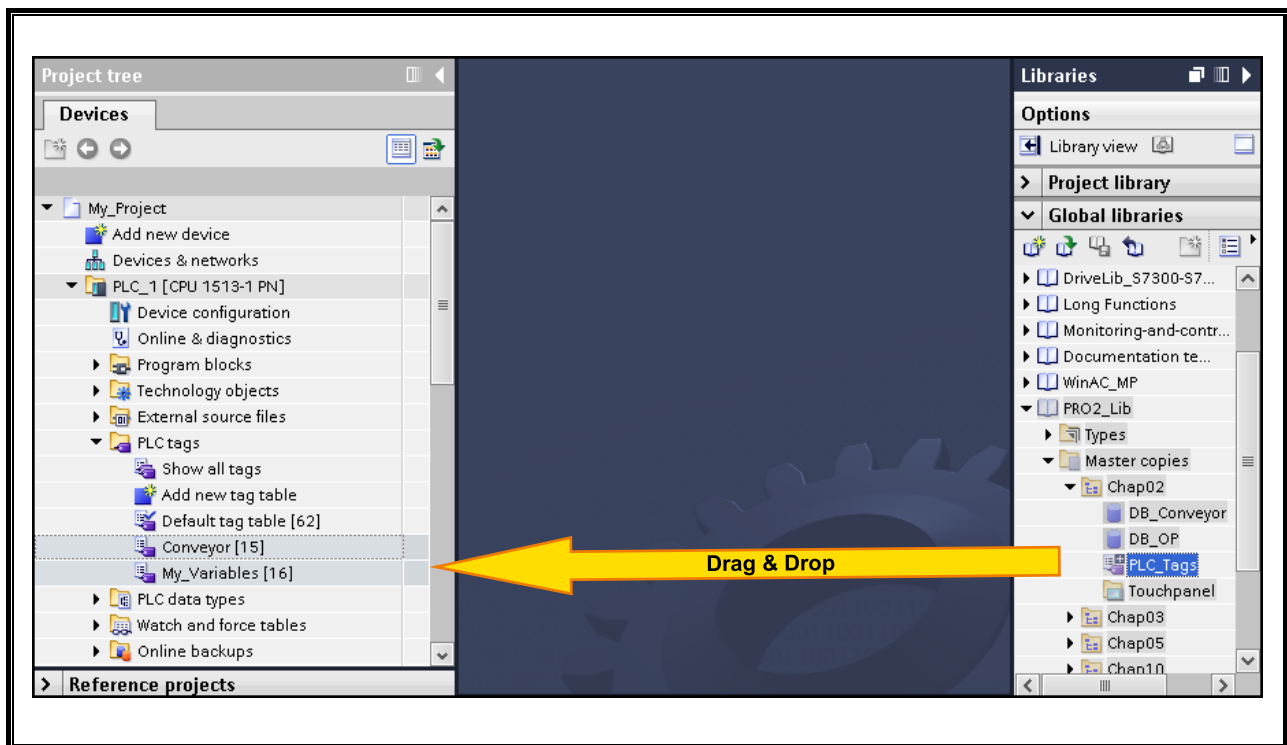
Task

The PROFINET device name previously assigned offline must now be assigned to the ET200SP online, so that the IO-Controller or the CPU can assign the offline-configured IP address during system startup of the ET200SP.

What to Do

1. In the "Devices & networks" editor, select the "Device view" of the ET200SP.
2. Right-click on the Interface module or the module on Slot 0 and in the menu that appears, activate the item "Assign device name".
3. In the dialog that appears, check the (offline) PROFINET device name.
4. Under "Type of the PG/PC interface", select the interface through which you are connected to the PROFINET (see picture).
5. In the lower part of the dialog, under the (online) "Accessible devices in the network", select the ET200SP or the Interface module IM156-6 and activate "Assign name".
6. Save your project.

2.13.13. Exercise 22: Copying a PLC Tag Table



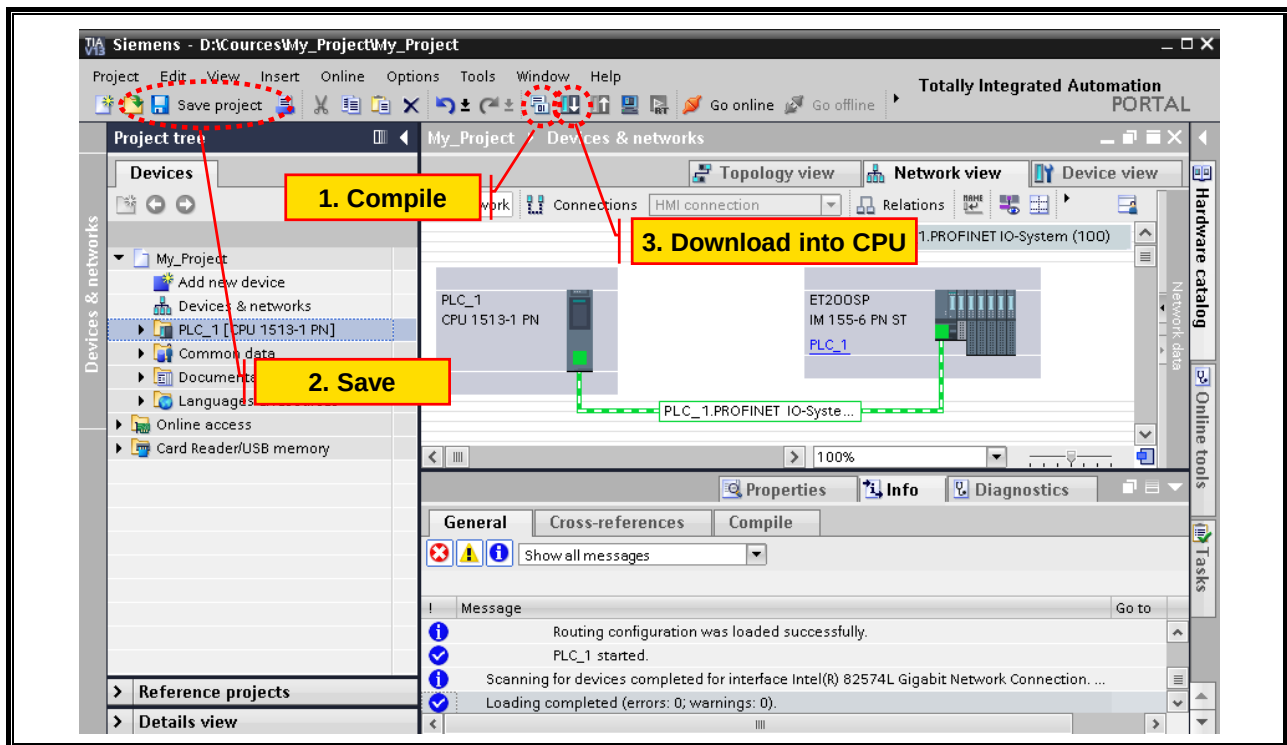
Task

A prepared table with PLC tags is to be copied from the "PRO2_Lib" global library into your own project.

What to Do

1. Switch to the Project view.
2. In the "Global libraries" open the library <Drive>:\Archives\TIA\PRO2_Lib
3. Using drag & drop, copy the object "PLC_Tags" from the "PRO2_Lib" global library into your own project in the container PLC tags.
4. Save your project.

2.13.14. Exercise 23: Compiling the HW Configuration and Downloading It into the CPU



Task

Now that the PROFINET I/O system is completely configured and parameterized, the project must be completely compiled, saved and downloaded into the CPU.

What to Do

1. Compile the hardware configuration by selecting the S7-1500 station in the Project tree and then clicking on the Compile button (see picture). In the Inspector window under "Info", check whether the compilation was successful. Should errors have occurred, correct them.
2. Save your project.
3. Download the entire station into the CPU by clicking on the Download button (see picture). In the Inspector window under "Info", check whether the loading was successful.
4. Check the module LEDs of your training unit: Only green LEDs should be lit and not flashing!!!
5. Save your project.

Result:

All LEDs show a green continuous light on the S7-1500 station as well as on all modules of the ET200SP.

2.13.15. Exercise 24: Testing the Wiring of the Conveyor Model's Sensors and Actuators

The screenshot shows the SIMATIC Manager interface. On the left, the 'Watch table_1' is displayed with the following data:

	Name	Address	Display format	Monitor value	Modify value
1	"B_LB"	%I4.0	Bool	TRUE	
2	"S_Bay1"	%I4.1	Bool	FALSE	
3	"S_Bay2"	%I4.2	Bool	FALSE	
4	"S_Bay3"	%I4.3	Bool	FALSE	
5	"S_BayLB"	%I4.4	Bool	FALSE	
6	"B_Bay1"	%I4.5	Bool	FALSE	
7	"B_Bay2"	%I4.6	Bool	FALSE	
8	"B_Bay3"	%I4.7	Bool	FALSE	
9		%QB4	Bin	2#0010_0000	2#0010_0000
10	<Add new>				

An arrow points from the 'Modify value' field of the 9th row to a 3D model of a conveyor system. The model shows a conveyor belt with several sensors and actuators. A cable connects the conveyor model to an ET200SP distributed I/O station, which is shown in a rack with slots 0 through 6. The ET200SP station is labeled 'ET200SP' and 'SIEMENS'.

Example: Q 4.5 (Conveyor to right)

Task

The wiring of the conveyor model's sensors and actuators to the ET200SP distributed IO station is to be checked.

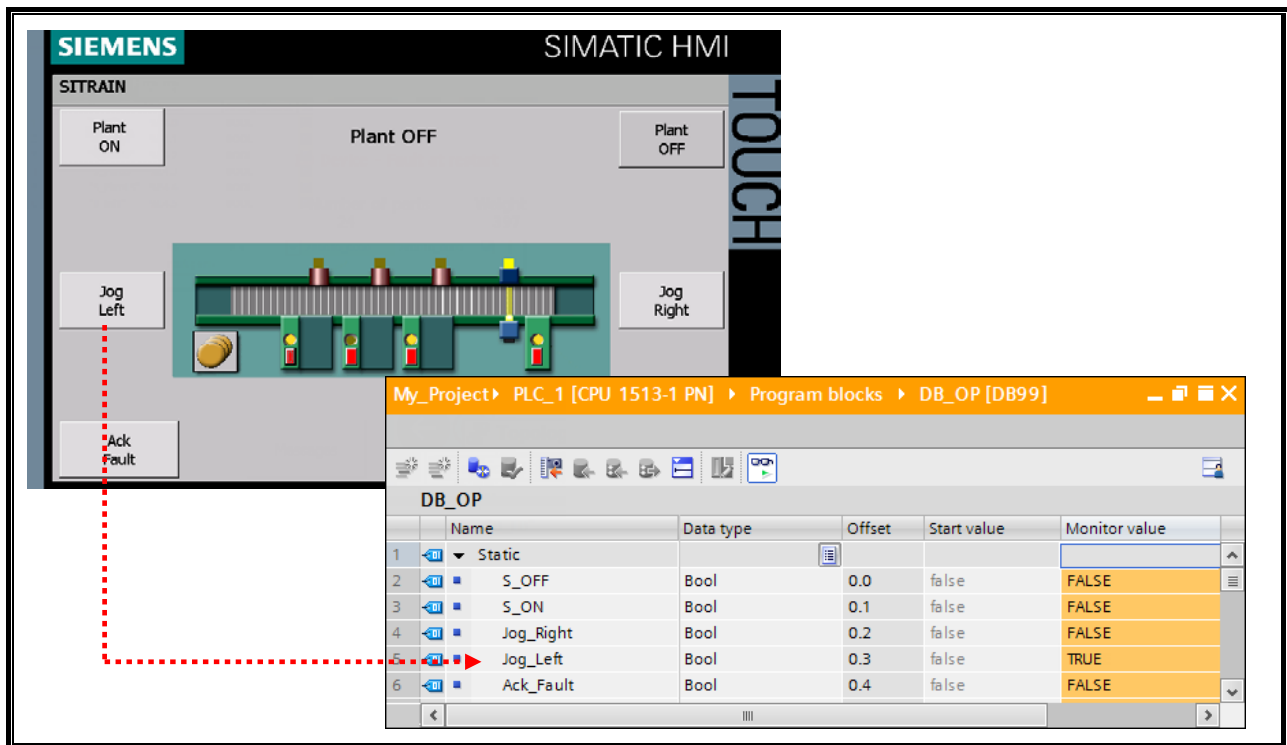
What to Do



1. On the back of the training case, insert the conveyor model cable or the Sub-D connector into the SUB-D socket with the labelling "ET200 DI/DO"
2. Switch the conveyor model on again ("ON" pushbutton on the conveyor model).
3. Edit a Watch and force table with the operands shown in the picture and give the Watch table the name "Watch table_1".
4. By specifically monitoring and modifying, check whether the sensors and actuators are correctly wired to the inputs and outputs shown in the picture.

Should you encounter any errors, please let your instructor know!

2.14. Task Description: Commissioning the Touchpanel

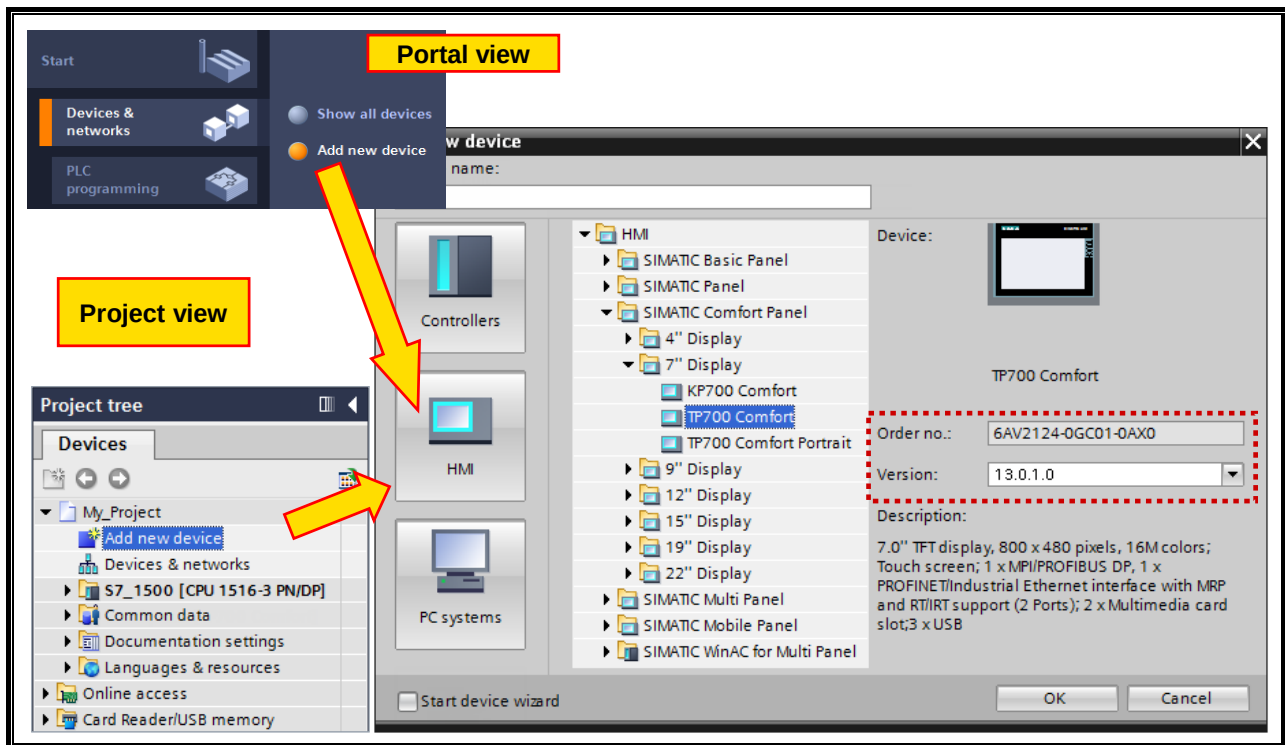


Task Description

The touchpanel project is to be commissioned. It will later be used to operate the system. As interface for the buttons in the "Conveyor" screen, the data block "DB_OP" (DB99) will be used.

After the touchpanel is commissioned, you are to check whether the relevant data block variables are set and reset when the buttons are pressed.

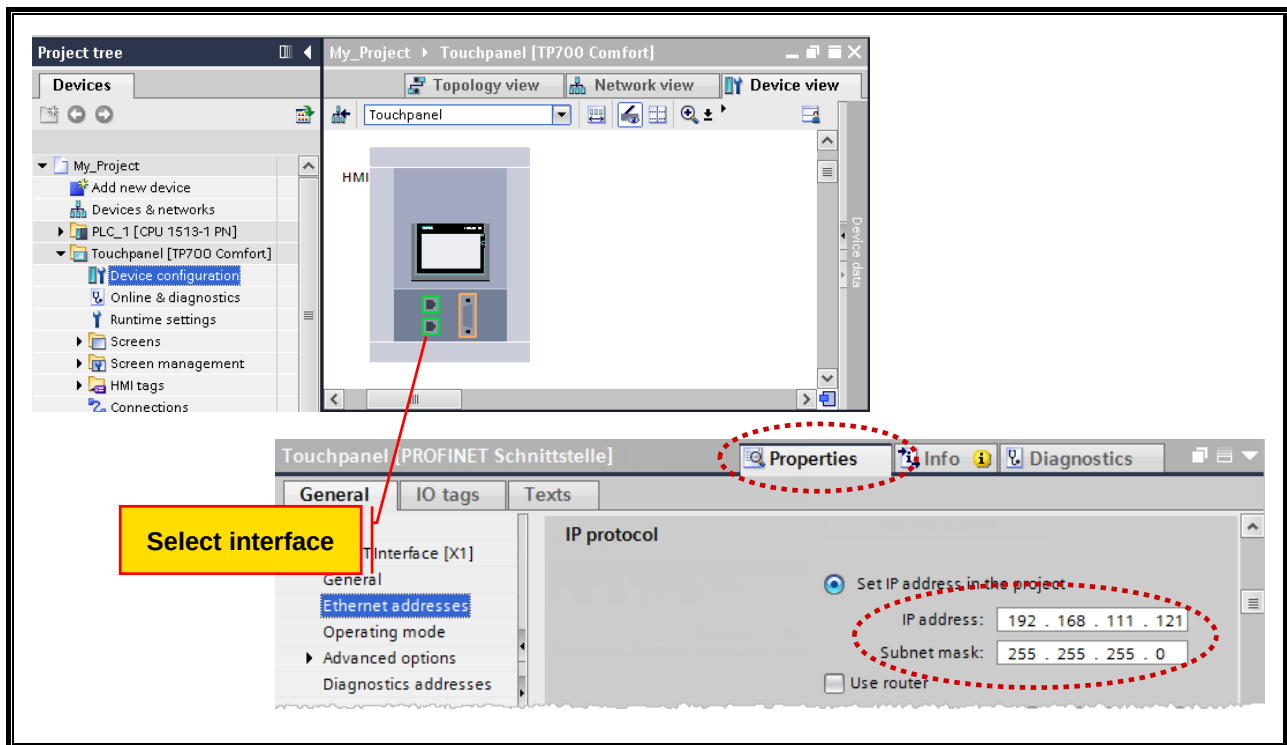
2.15. Adding an HMI Device



Adding an HMI Device

New HMI devices can be added from both the Portal view and the Project view. More than anything else, attention has to be paid to the device data such as article (order) number and version number.

2.16. Configuring the IP Address of a Touchpanel

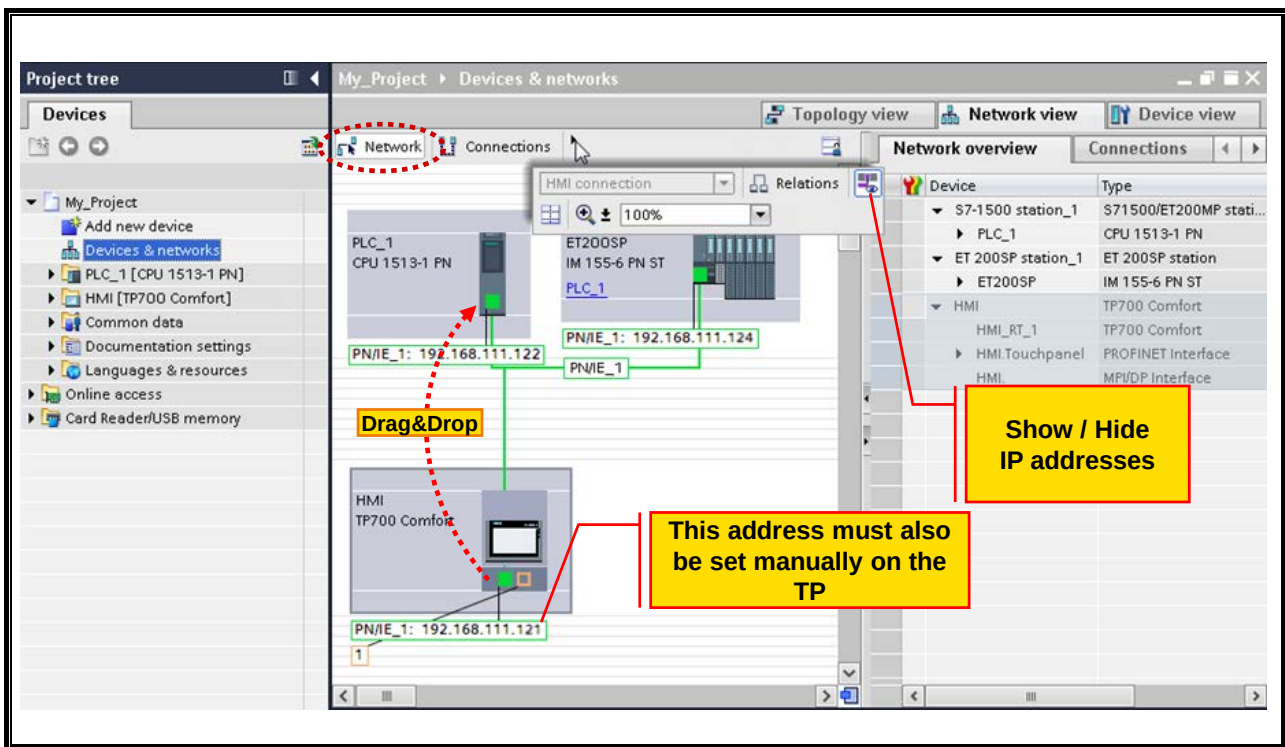


PROFINET Interface of the Touchpanel

Regardless of whether the Devices & networks editor is in the Devices view or the Network view, the settings of the PROFINET interface (IP address and subnet mask) can be made in the "Properties" tab in the Inspector window for a selected HMI device interface.

If an online connection between the HMI device and the CPU is to be established, both devices must be assigned the same subnet mask and IP addresses that are in the same subnet.

2.16.1. Networking a Touchpanel

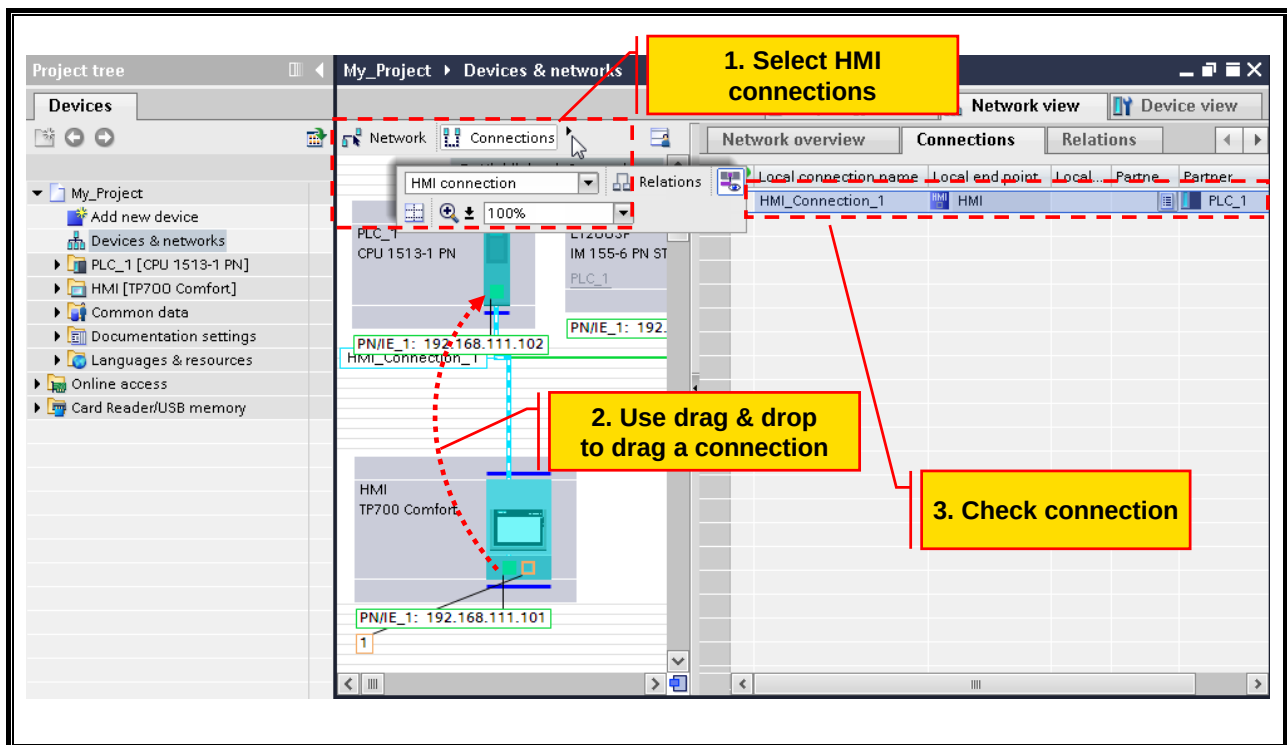


Networking a Touchpanel

During networking, devices are connected to a subnet. The device interface must be compatible with the type of network.

The devices are networked with the "Devices & networks" editor in the "Network" view by connecting the device interfaces using drag & drop.

2.16.2. Configuring an HMI Connection

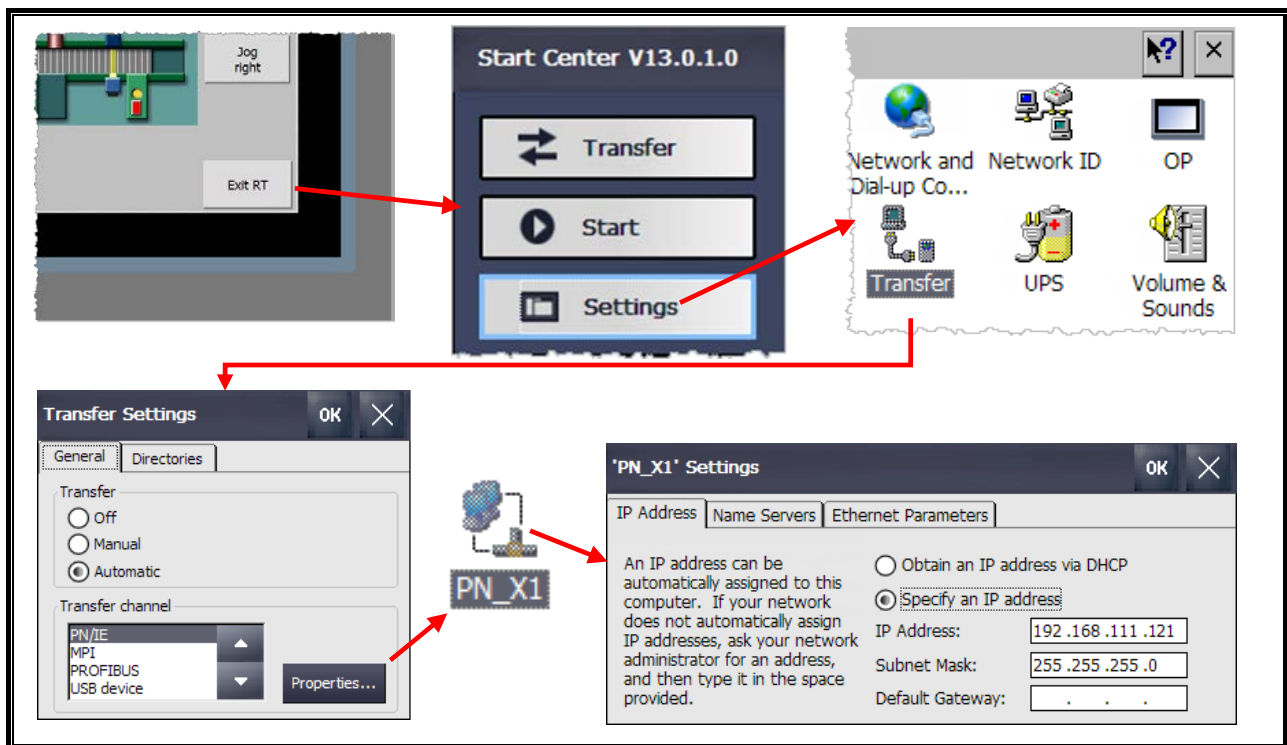


Configuring HMI Connections

In configuring HMI connection(s), the communications partners are defined with which the HMI device will later exchange data in the process control phase. The HMI device can also be connected to or exchange data with several controllers.

There can also be controllers in the same network with which the HMI device does not exchange data. Then, the HMI device is "networked" with these controllers but it is not "connected".

2.16.3. Setting the IP Address on the Touchpanel



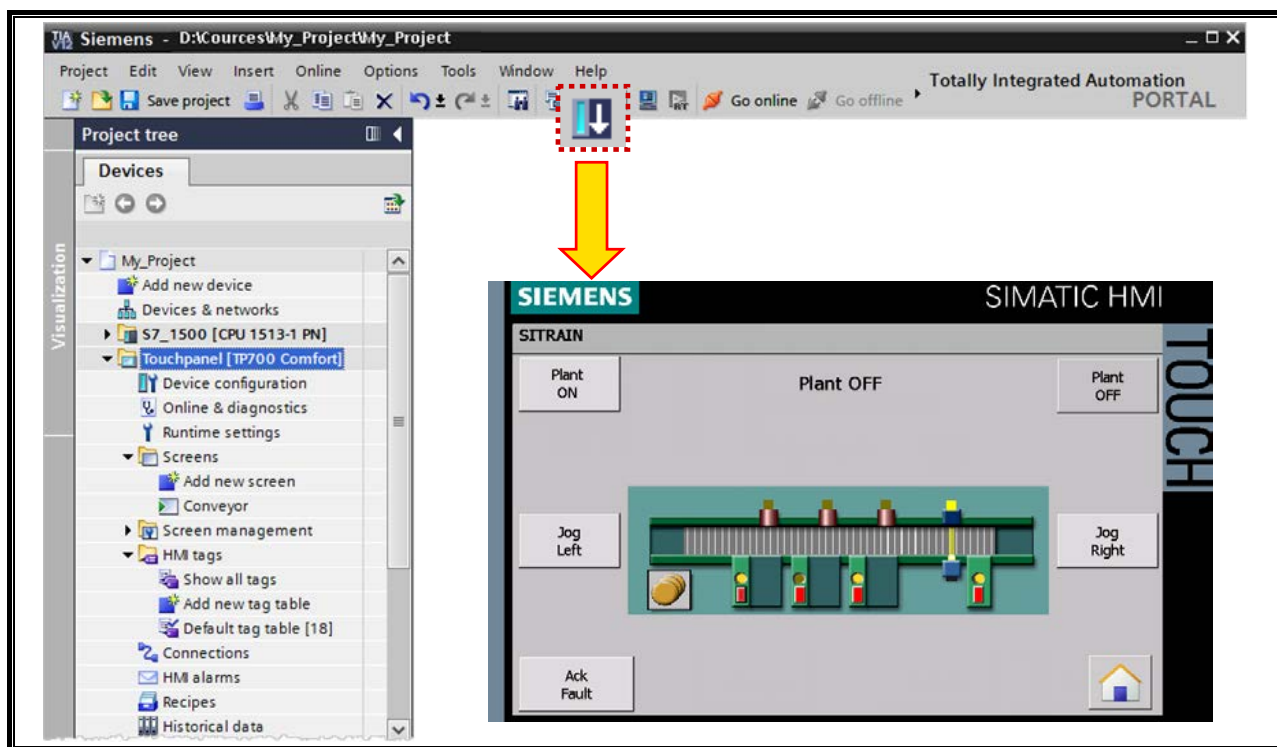
Setting the IP Address of the Touchpanel

The interface of the touchpanel must be set to the same IP address and subnet mask as it is also configured in the offline project.

Remote Control

You can initiate the loading of the WinCC flexible project without having to first manually end the Runtime of the HMI device. The Panel automatically ends the Runtime and switches to the Transfer Mode.

2.16.4. Downloading the HMI Project into the Touchpanel



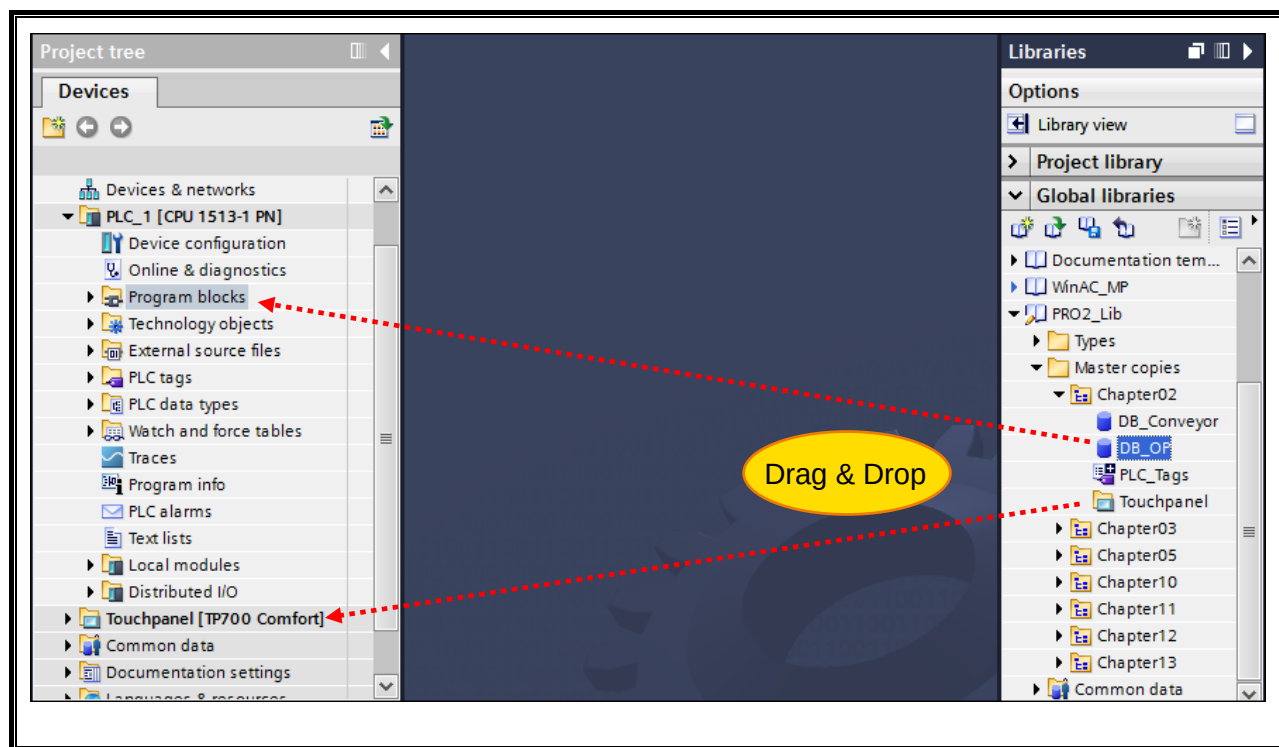
Downloading the HMI Project into the Touchpanel

When you transfer an HMI project to one or more operator panels, the part of the project that has been changed since the last transfer is automatically compiled before downloading. This ensures that the current project status is always transferred. Beyond that, it is also possible to activate the option "Overwrite all" before loading starts.

For commissioning, the project should be completely compiled using the command "Compile > Software (rebuild all blocks)" in the context menu of the operator panel. If HMI tags that are linked to PLC tags are also used in the project, all modified STEP 7 blocks should also be compiled using the command "Compile > Software" in the context menu and then be downloaded into the CPU.

It is also advisable to execute the "Compile > Software (rebuild all blocks)" command occasionally to reduce the time required for compiling delta data in current engineering sessions.

2.17. Exercise 25: Copying the Touchpanel Project and the Interface Data Block from the Library



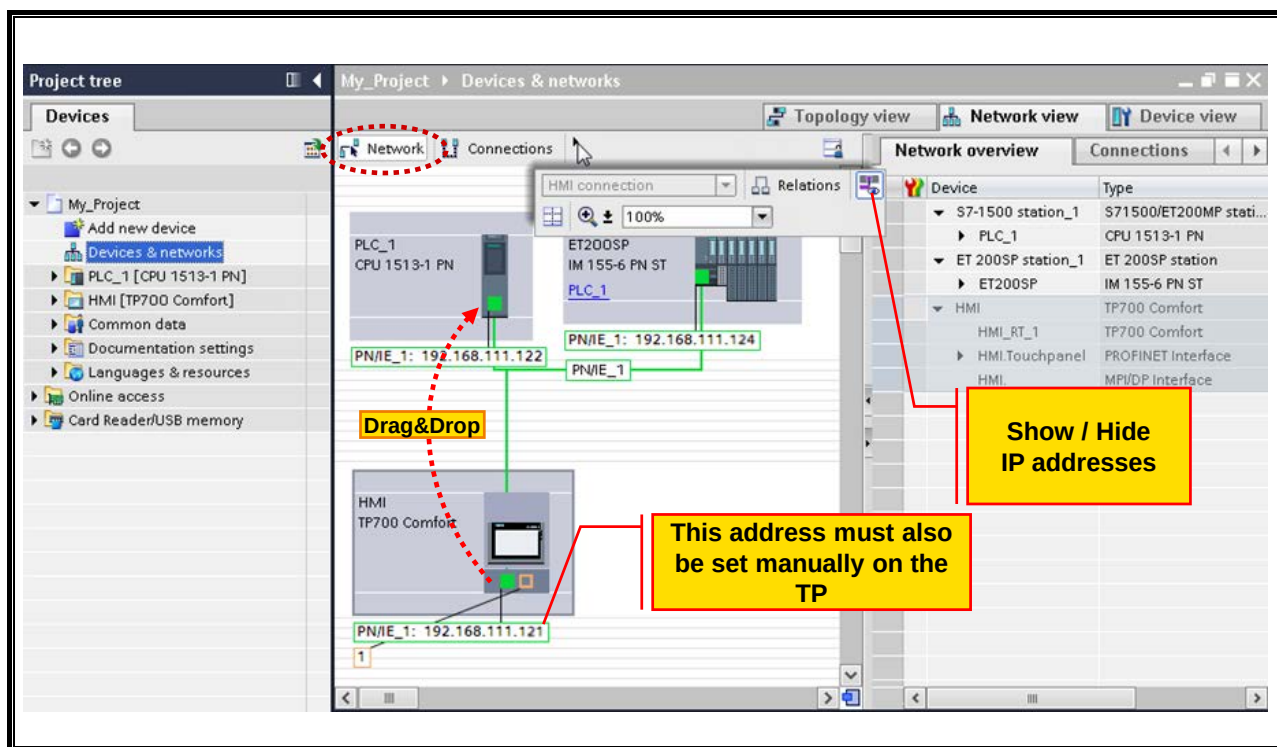
Task

Up until now, your project doesn't contain an HMI device. Instead of a completely new configuration, you are to copy a prepared panel project and the "DB_OP" (DB99) data block that is to serve as the interface between the controller and the touchpanel, from the global library "PRO2_Lib" into your project.

What to Do

1. Open the Global libraries <Drive>:\Archives \TIA \Pro2_Lib.
2. Using drag & drop, copy the library elements "Touchpanel and "DB_OP" (DB99) into your project.
3. Save your project.

2.17.1. Exercise 26: Networking the Touchpanel

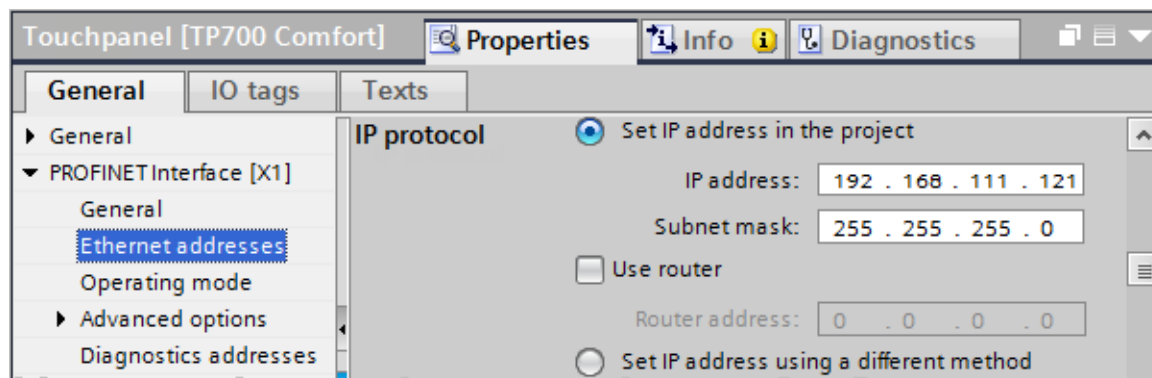


Task

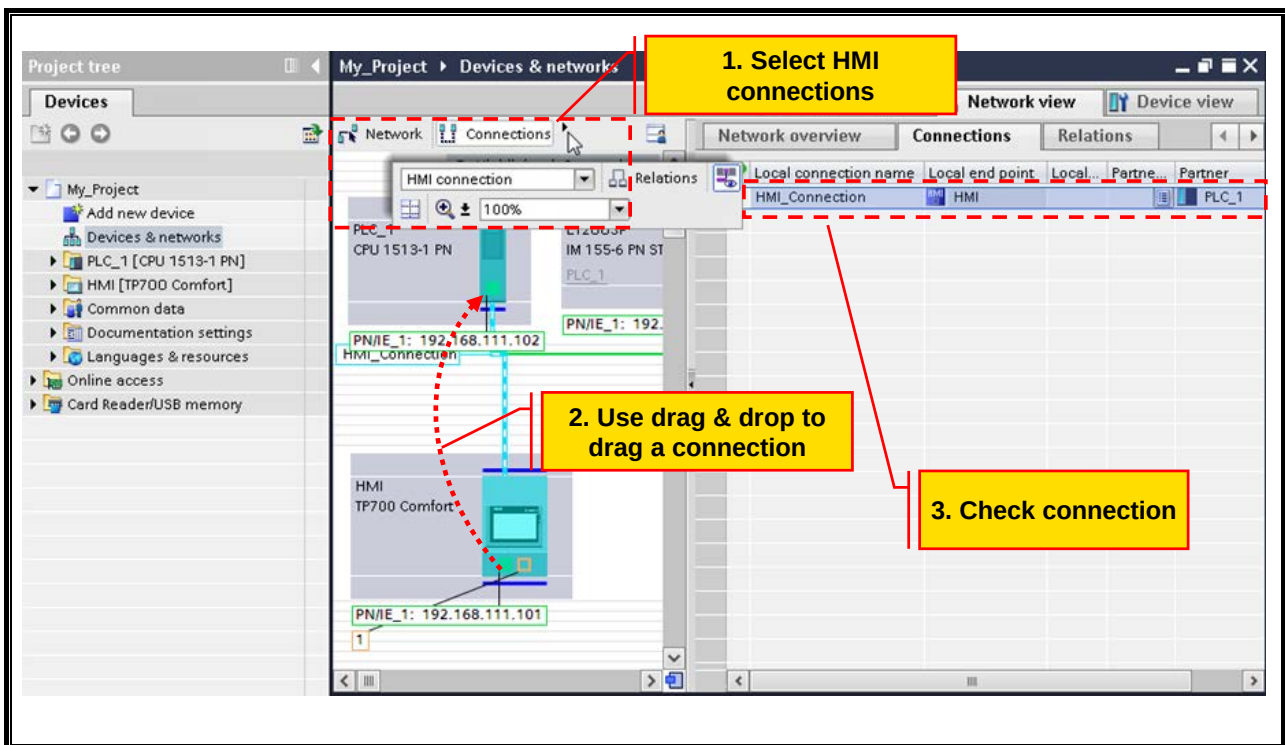
The added touchpanel is to be networked offline with the Ethernet network.

What to Do

1. In the Project tree, start the "Devices & networks" editor, switch to the "Network view" and there select "Network".
2. Position the mouse pointer on the small green square of the HMI device and, while keeping the left mouse button pressed down, drag a connection to the CPU. The network is created; the associated subnet and the parameters appropriate for the network (IP address and subnet mask) are automatically created.
3. Display the IP addresses of the devices via the button shown in the picture.
4. Select the touchpanel and switch to the "Device view".
5. In the Inspector window in the "Properties" of the touchpanel, check and correct the Ethernet address:



2.17.2. Exercise 27: Configuring the HMI Connection



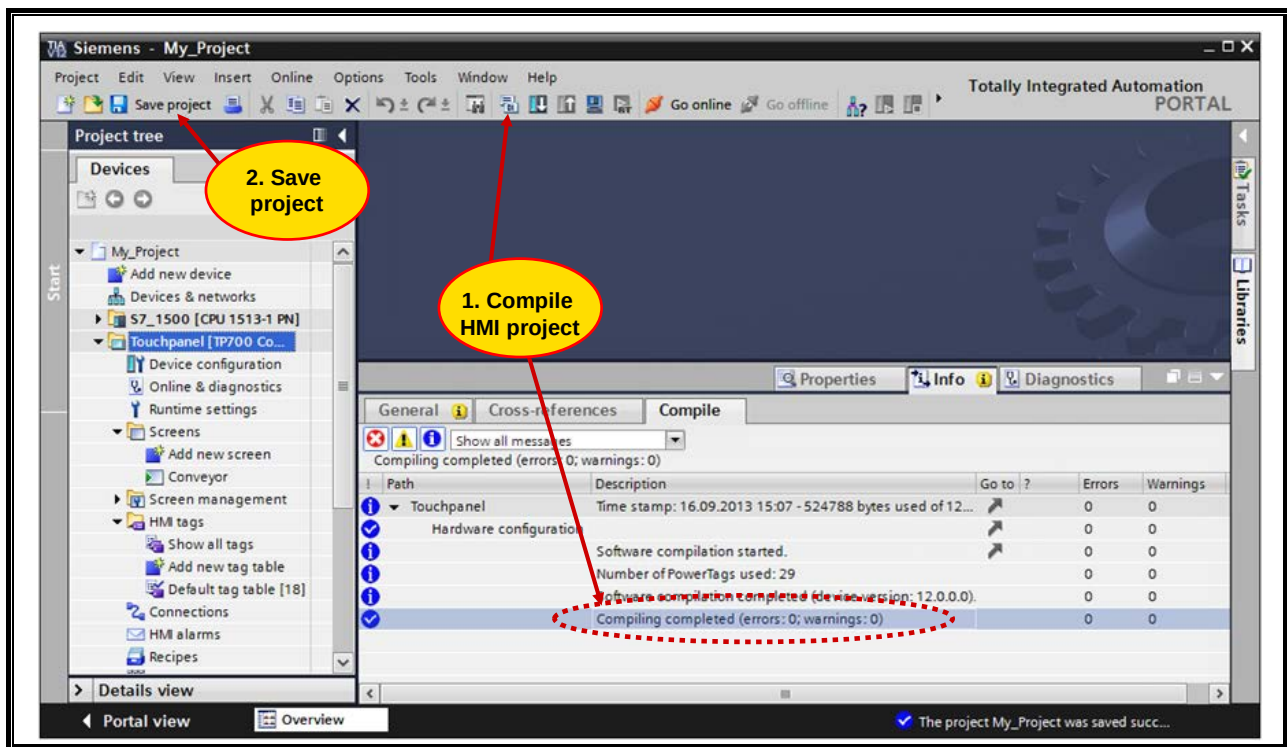
Task

Now that the TP is networked with the Ethernet network, an HMI connection between the TP and the CPU must be created. The automatically assigned name of the connection "HMI_Connection" can be adopted without changing it.

What to Do

1. In the Project tree, start the "Devices & networks" editor, switch to the "Network view" and there select "Connections" (see picture).
2. Position the mouse pointer on the small green square of the HMI device and, while keeping the left mouse button pressed down, drag a connection to the CPU. The connection is created and given the default name "HMI_Connection".
3. In the Details window, in the "Connections" tab, check whether the HMI connection was correctly created and, if necessary, change the name to "HMI_Connection". (see picture).
4. To display the type of connection between the S7-CPU and the HMI device, position the mouse pointer on the network and, in the dialog window that appears, select "HMI connection".

2.17.3. Exercise 28: Compiling and Saving the HMI Project



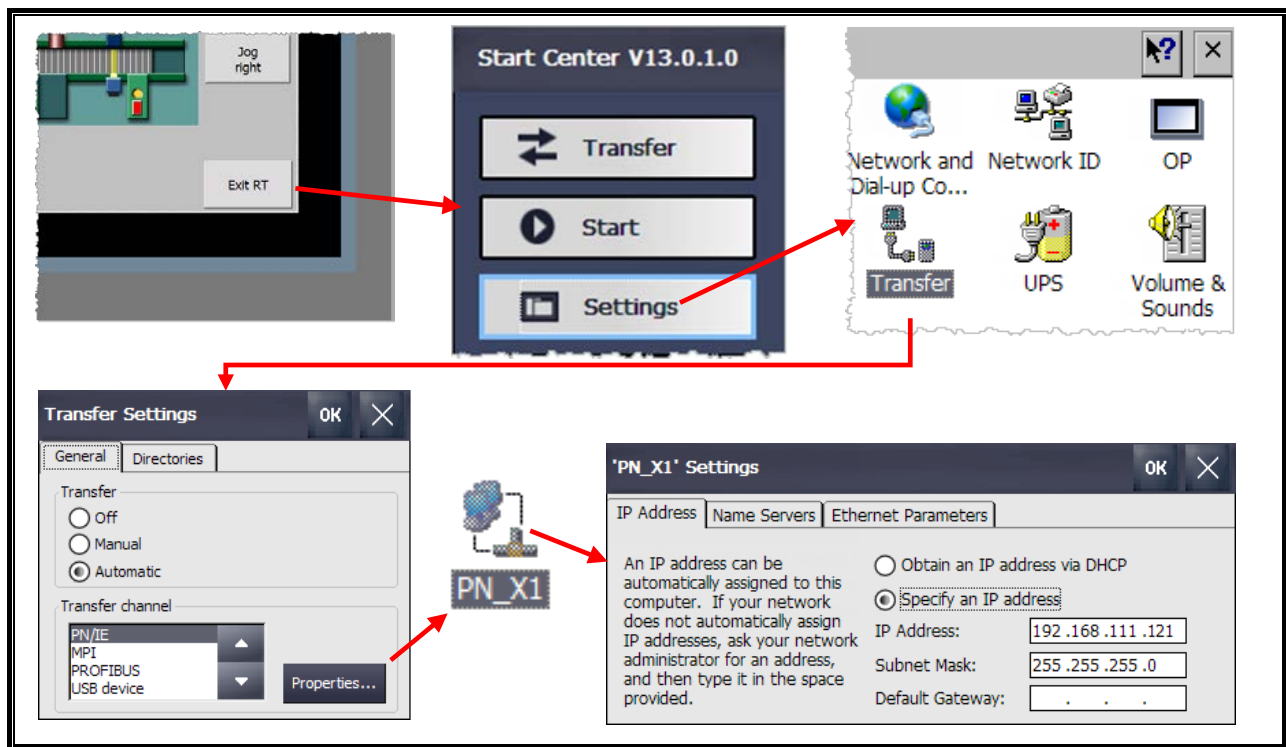
Task

The now complete HMI project is to be compiled and saved.

What to Do

1. Compile the HMI project by selecting the touchpanel in the Project tree and then clicking on the "Compile" button (see picture).
2. In the Inspector window under "Info", read the results of the compilation and eliminate any errors which may have occurred.
3. Save your project.

2.17.4. Exercise 29: Setting the IP Address on the TP



Task

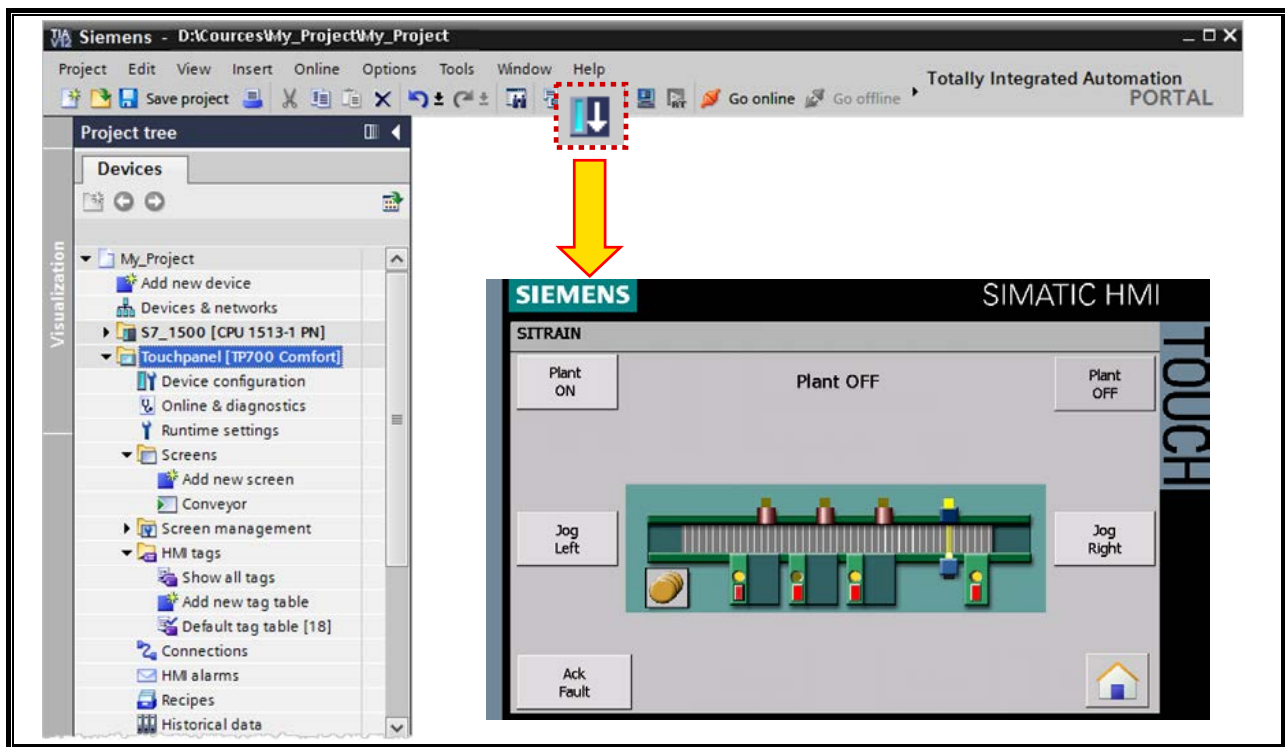
The interface of the touchpanel is to be set as shown in the picture so that the configuration can then be loaded into the panel.

Runtime must be exited before the interface can be assigned parameters. For this, a button for exiting Runtime is generally provided in the Start screen. When Runtime has been exited, the "Start Center" appears through which the "Settings" can be activated. The Start Center also appears every time power is restored.

What to Do

1. Exit the Runtime of the touchpanel.
2. In the Start Center, activate the "Settings".
3. Select "Transfer" by double-clicking on it.
4. Activate the **"Properties..."** of the **PN/IE** interface **PN/X1** (see picture)
5. Implement the settings shown in the picture.
6. Go back to the Start Center by closing the windows with "OK".
7. Activate the "Transfer" button so that the touchpanel waits for a connection to be established by the PG.

2.17.5. Exercise 30: Downloading the HMI Project into the Touchpanel



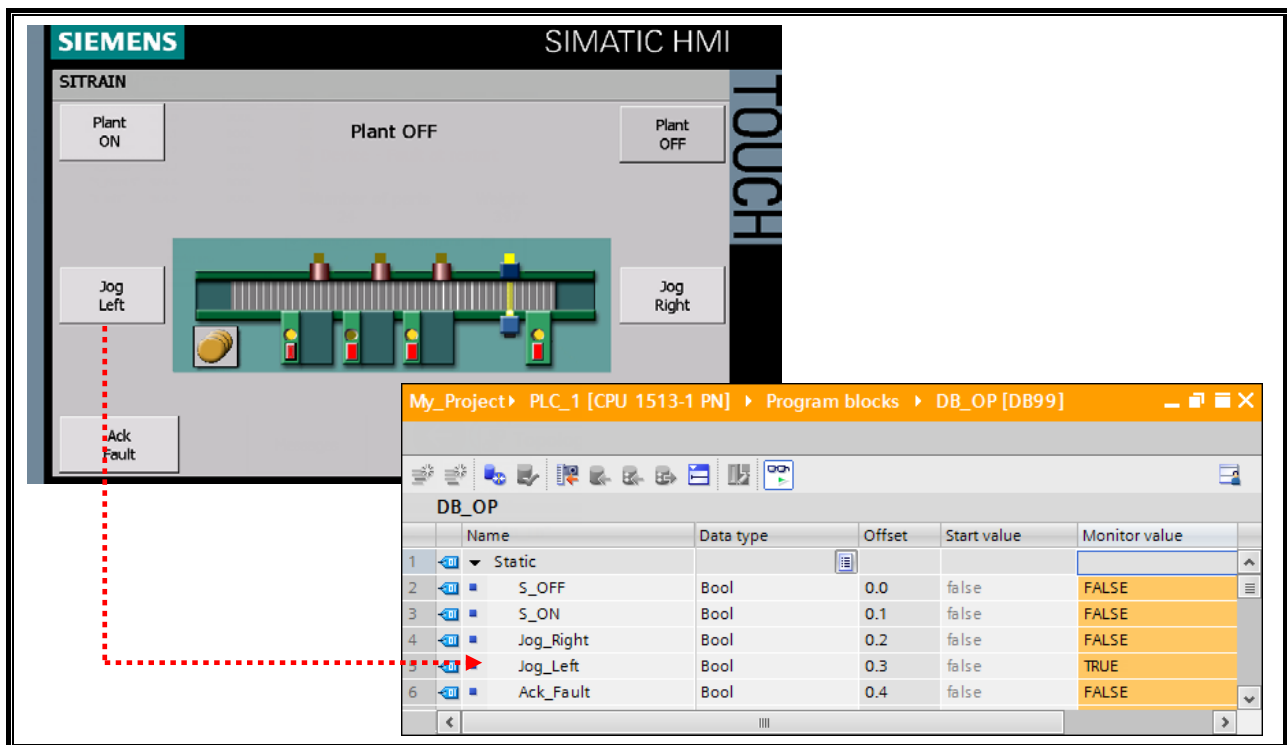
Task

From the now complete project, all S7 blocks are now to be downloaded once more into the CPU and the entire panel project is to be transferred into the touchpanel.

What to Do

1. Completely compile the entire CPU program and the entire panel project.
2. In the Inspector window, in the "Compile" tab, check whether the compilation was successful. If need be, make the necessary corrections.
3. Download the panel project into the touchpanel.
4. Save your project.

2.17.6. Exercise 31: Carrying Out a Function Test



Task

The connection of the touchpanel to the S7-1500 is to be checked by pressing the buttons of the panel and thereby monitoring the data block variables that are connected to the buttons.

What to Do

1. Open the data block "DB_OP" (DB99) and activate the function "Monitor".
2. One after the other, press the buttons on the touchpanel and observe how the touchpanel overwrites the states of the Boolean data block variables with '0' or '1'.
3. Save your project.

Contents

3.	Program Design Methods	3-2
3.1.	Plant Description: The Conveyor Model as Distribution Conveyor	3-3
3.2.	Types of Program Blocks	3-4
3.2.1.	Structured Programming	3-5
3.2.2.	Block Properties: Programming Language, Time Stamps	3-6
3.2.3.	Cyclic Program Execution	3-7
3.3.	Organization Blocks of the S7-1500	3-8
3.3.1.	Interrupting the Cyclic Program	3-9
3.3.2.	OB Start Information	3-10
3.4.	Block Programming	3-11
3.4.1.	Closing / Saving / Rejecting a Block	3-12
3.4.2.	Operand Edge Evaluation	3-13
3.4.3.	RLO Edge Evaluation	3-14
3.5.	Overview: Data Types in STEP 7	3-15
3.5.1.	Elementary Data Types	3-16
3.5.1.1.	Integer (INT, 16-Bit Integer) Data Type	3-17
3.5.1.2.	Double Integer (DINT, 32-Bit Integer) Data Type	3-18
3.5.1.3.	REAL (Floating-point Number, 32 Bit) Data Type	3-19
3.5.2.	IEC Counters: CTU, CTD, CTUD	3-20
3.5.2.1.	Counter Function: Inputs	3-21
3.5.2.2.	Counter Function: Outputs	3-22
3.5.3.	IEC Timer Function: TON	3-23
3.5.4.	Timer Function TON (ON Delay) Pulse Diagram	3-24
3.5.4.1.	IEC Timer / Counter Instance Data Blocks	3-25
3.6.	STEP 7 - Test Functions, Overview	3-26
3.7.	Procedure for Creating a Program	3-27
3.7.1.	Structure Elements of Structograms (1)	3-28
3.7.2.	Structure Elements of Structograms (2)	3-29
3.8.	Plant Description: The Conveyor Model as Distribution Conveyor	3-30
3.8.1.	Structure of the CPU Program and Interface to the Touchpanel	3-31
3.8.2.	DB Variables instead of Memory Bits	3-32
3.8.3.	Exercise 1: Copying "DB_Conveyor" from the Library	3-33
3.8.4.	Exercise 2: Startup Program "OB_Startup" (OB100) and Operating Mode Section "FC_Mode" (FC15)	3-34
3.8.5.	Exercise 3: Programming the Conveyor Motor Control "FC_ConvMotor" (FC16)	3-35
3.8.6.	Exercise 4: Programming the Time Monitoring of the Transport Sequences "FC_Fault" (FC17)	3-36
3.8.7.	Exercise 5: Programming the Indicator Lights "FC_Signal" (FC14)	3-37

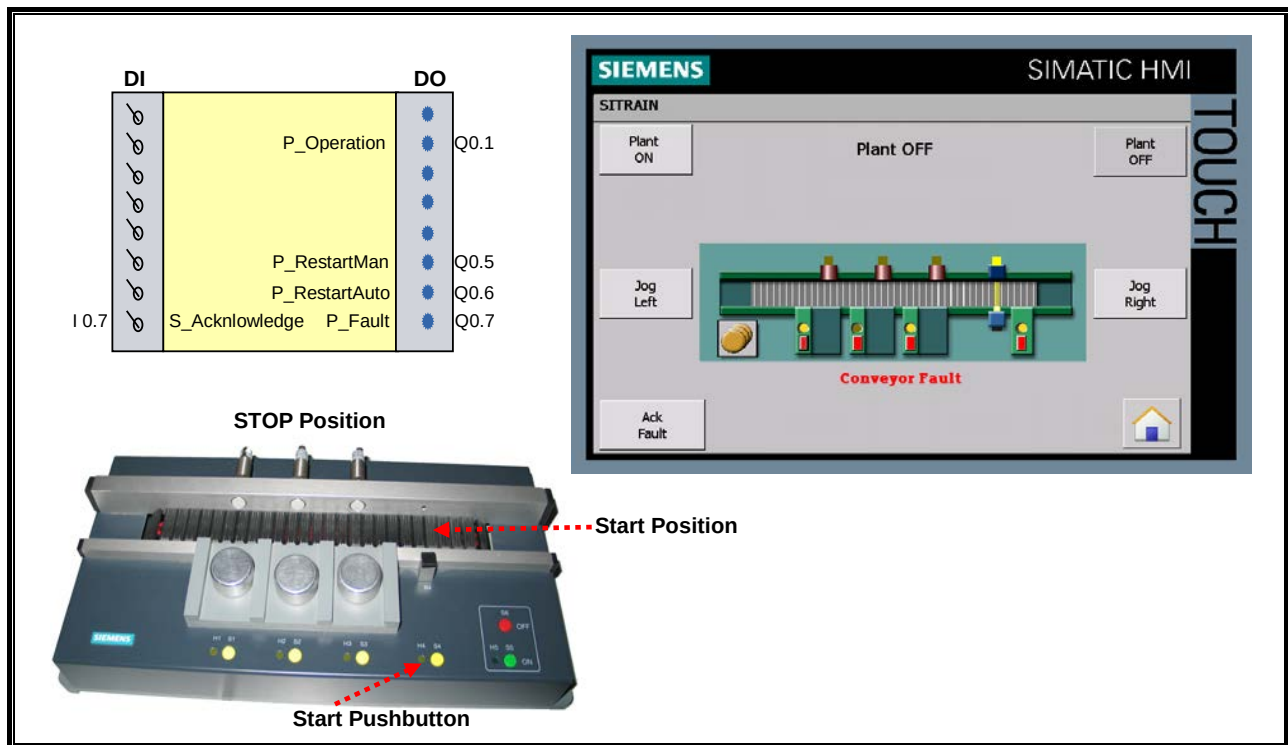
3. Program Design Methods

At the end of the chapter the participant will ...

- ... be familiar with elements of structured programming
- ... be able to create simple structograms
- ... be able to convert structograms into S7 program code
- ... reinforce and expand his SIMATIC knowledge



3.1. Plant Description: The Conveyor Model as Distribution Conveyor



Conveyor Model as Distribution Conveyor

- **Job Mode**

When "P_Operation" (Q0.1) is switched off, the conveyor can be jogged in the relevant direction using the buttons "Jog Right" and "Jog Left".

- **Automatic Mode**

When "P_Operation" (Q0.1) is switched on, the indicator light at the light barrier bay shows with a continuous light that a new part may be placed in the light barrier (Start Position), if ...

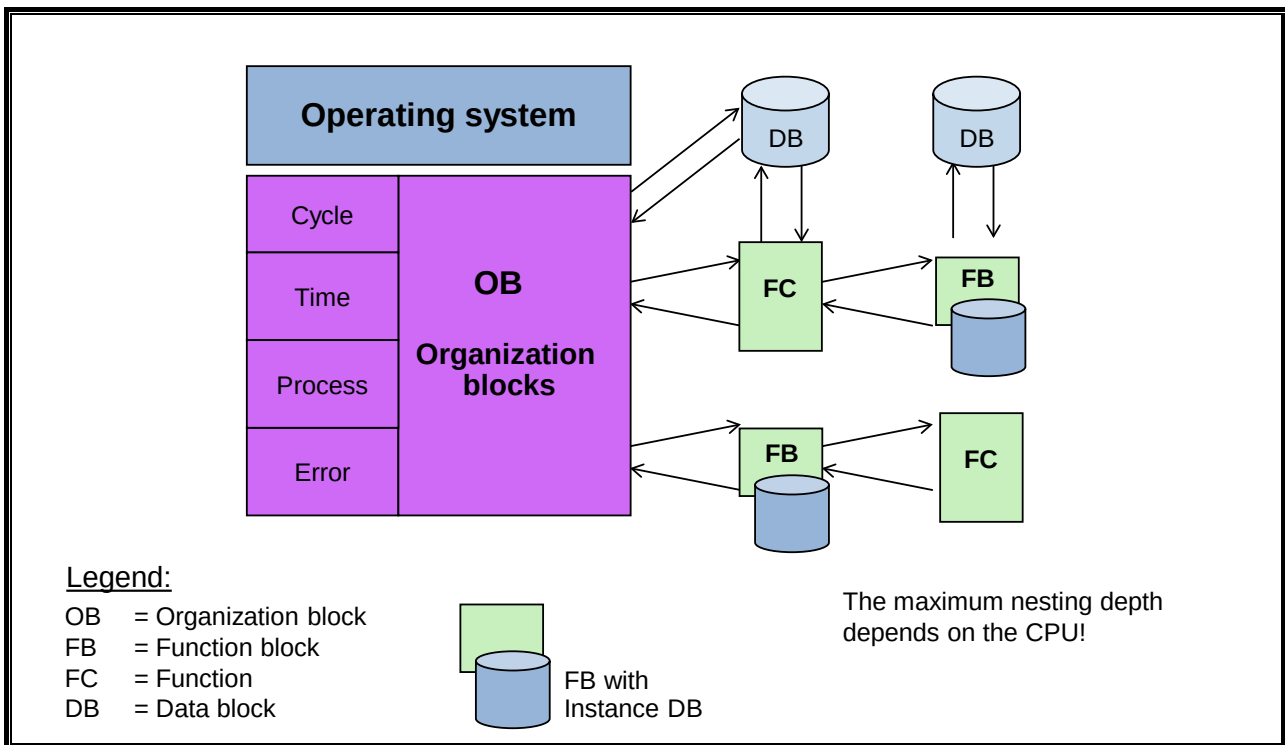
- ... at least one of the Bays 1 to 3 (STOP Positions) is free (= at least one of the bay indicator lights is dark)
- ... no conveyor fault exists (no indication at LED "P_Fault" or on the TP)

Transport is started by pressing the pushbutton at the light barrier bay. As soon as the part reaches a free bay, the conveyor motor is stopped.

A 2Hz flashing light from the bay indicator light indicates that the part can be removed from the conveyor. The bay is still considered to be occupied even though the part has been removed, which is indicated by a continuous light on the bay indicator light. Only after acknowledging with the bay pushbutton, is the bay "enabled" once more.

Every transport sequence is monitored for time. If it takes longer than 6 seconds, there is a fault and the conveyor motor is automatically switched off. Only after the fault is acknowledged via "S_Acknowledge" (I 0.7) or on the touchpanel, can a new transport sequence be started.

3.2. Types of Program Blocks



Blocks

The programmable logic controller provides various types of blocks in which the user program and the related data can be stored. Depending on the requirements of the process, the program can be structured in different blocks. You can use the entire operation set in all blocks (FB, FC and OB).

Organization Blocks (OBs)

Organization blocks (OBs) form the interface between the operating system and the user program. The entire program can be stored in OB1 that is cyclically called by the operating system (linear program) or the program can be divided and stored in several blocks (structured program).

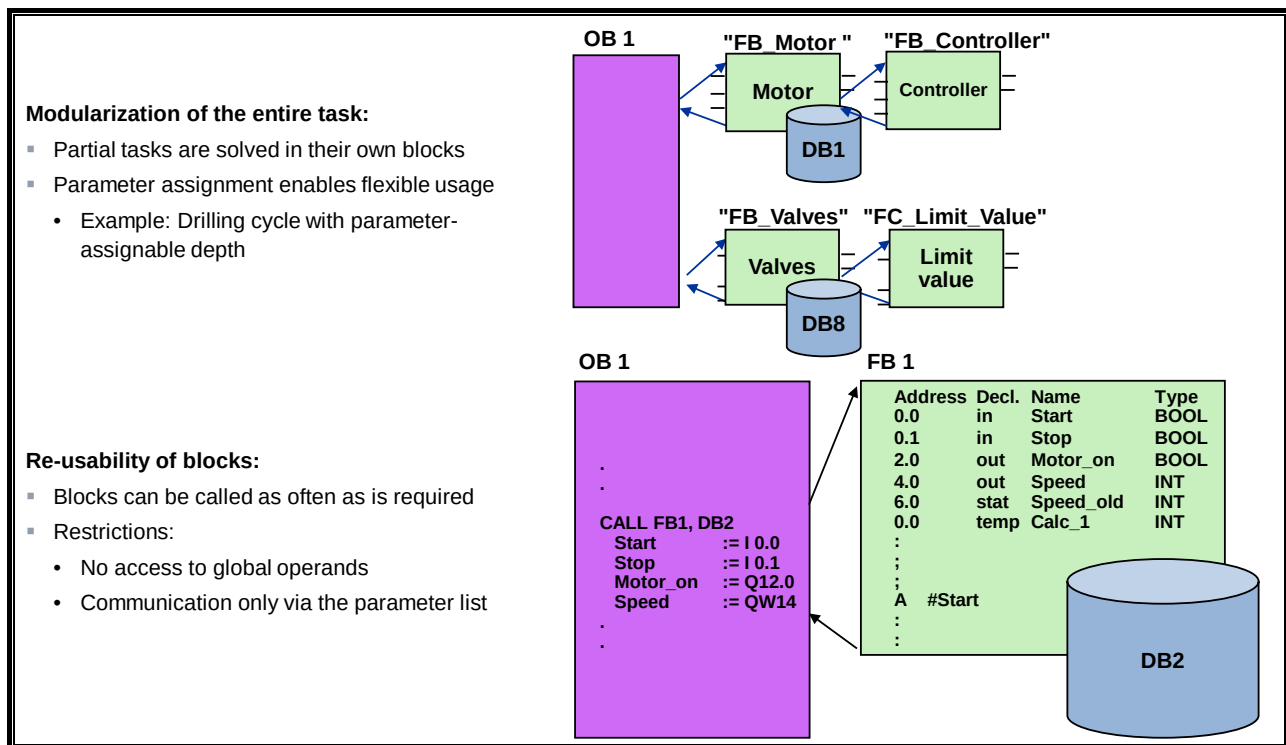
Functions (FCs)

A function (FC) contains a partial functionality of the program. It is possible to program functions as parameter-assignable so that when the function is called it can be assigned parameters. As a result, functions are also suited for programming frequently recurring, complex partial functionalities such as calculations.

Function Blocks (FBs)

Basically, function blocks offer the same possibilities as functions. In addition, function blocks have their own memory area in the form of instance data blocks. As a result, function blocks are suited for programming frequently recurring, complex functionalities such as closed-loop control tasks.

3.2.1. Structured Programming



Modularization of the Entire Task

Abstraction is the basis for solving complex problems, in which we concentrate on the fundamental aspects of a program in every abstraction level and ignore all the details that are not essential. Abstraction helps us to divide complex tasks into partial tasks which can then be solved on their own.

Structured Programming

STEP7 supports this concept of modularization with its block model. The partial tasks that result from the division of the entire task are assigned to blocks in which the necessary algorithms and data for solving the partial problems are stored. STEP7 blocks such as functions (FC) and function blocks (FB) can be assigned parameters so that the concepts of structured programming can be implemented with them. This means:

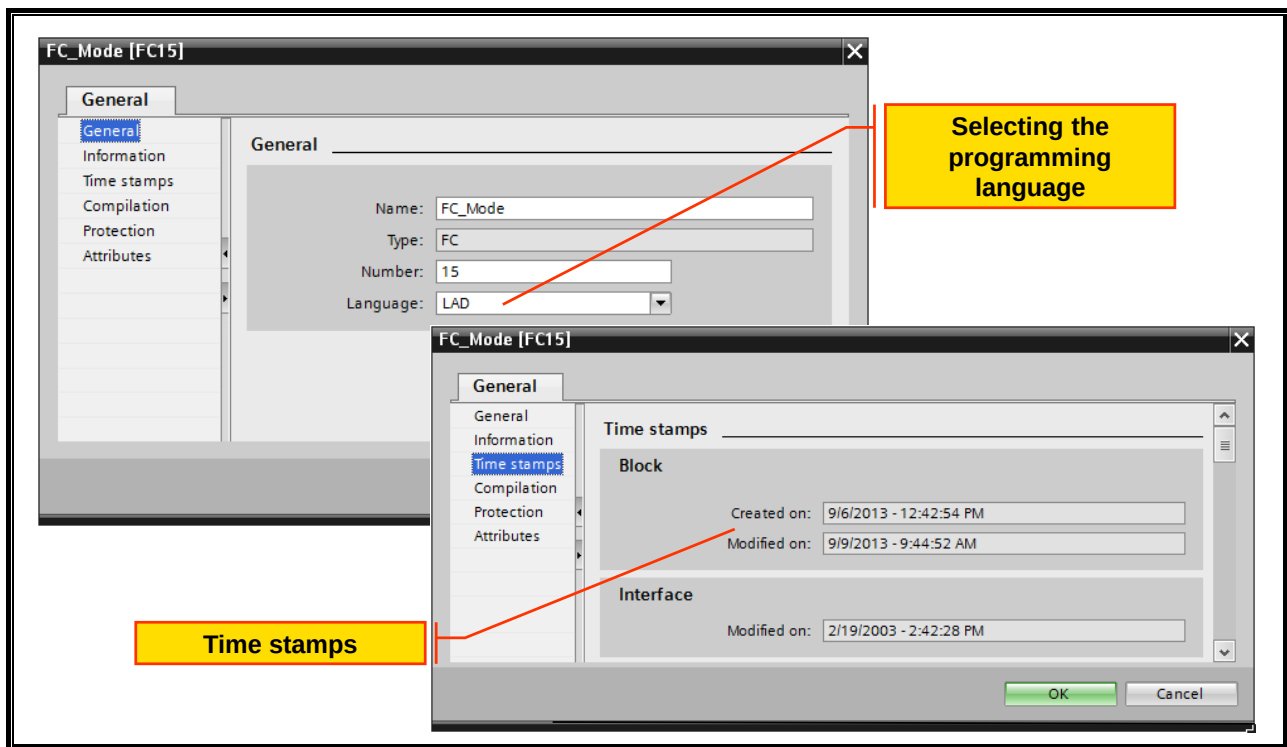
- Blocks for solving partial tasks implement their own data management with the help of local variables.
- Blocks communicate with the "outside world", that is, with the sensors and actuators of the process control or with other blocks of the user program, exclusively through their block parameters. No access to global operands such as inputs, outputs, bit memories or variables in DBs can be made from within the instruction section of blocks.

Advantages

Structured programming has the following advantages:

- The blocks for the partial tasks can be created and tested independent of one another.
- Blocks can be called as often as is required in different locations with different parameter data records, that is, they can be reused.
- "Re-usable" blocks for special tasks can be delivered in pre-designed libraries.

3.2.2. Block Properties: Programming Language, Time Stamps

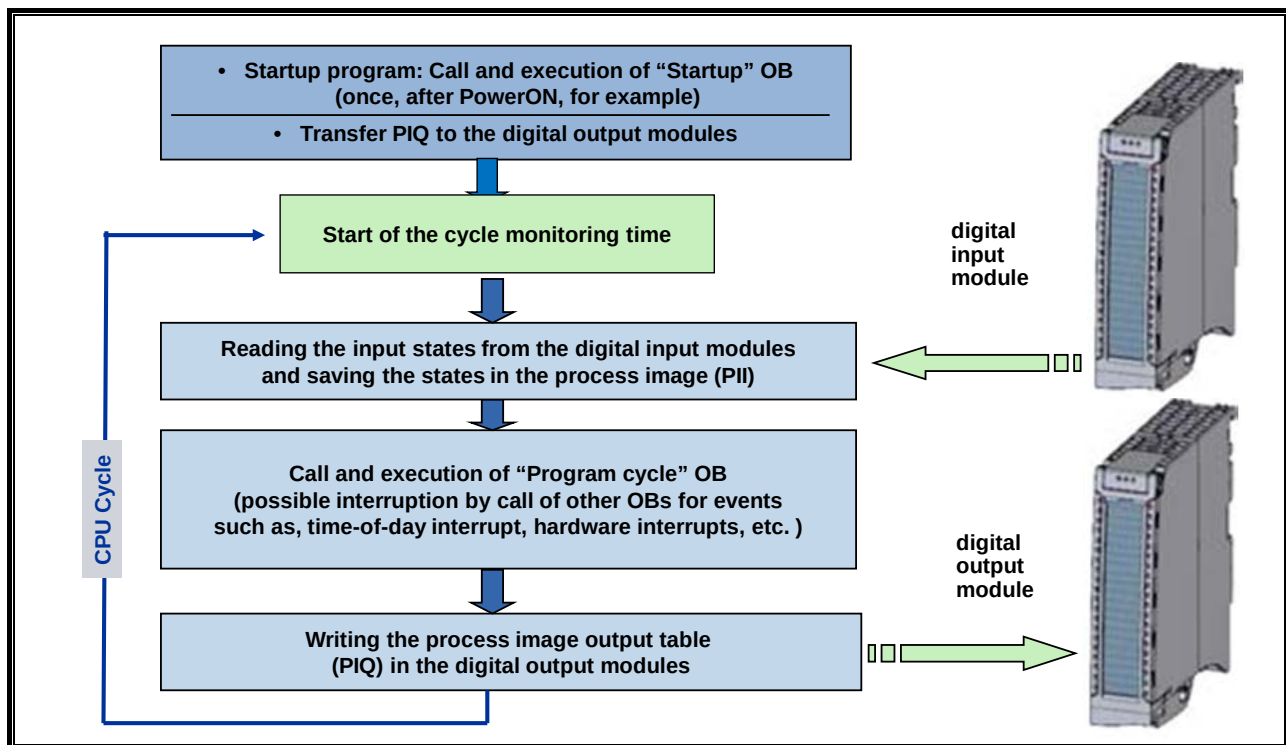


Properties

Each block has certain properties that you can display and edit. These properties are used to:

- Identify the block
- Display the memory requirements and the compilation status of the block
- Display the time stamp
- Display the reference information
- Specify the access protection
- Display and change the programming language
 - It is only possible to switch between LAD and FBD. For blocks that were created in STL, SCL or GRAPH, the programming language cannot be changed.

3.2.3. Cyclic Program Execution



Restart

When you switch on or switch from STOP --> RUN, the CPU carries out a complete restart (with OB "Startup"). During restart, the operating system deletes all non-retentive bit memories.

Cyclic Program Execution

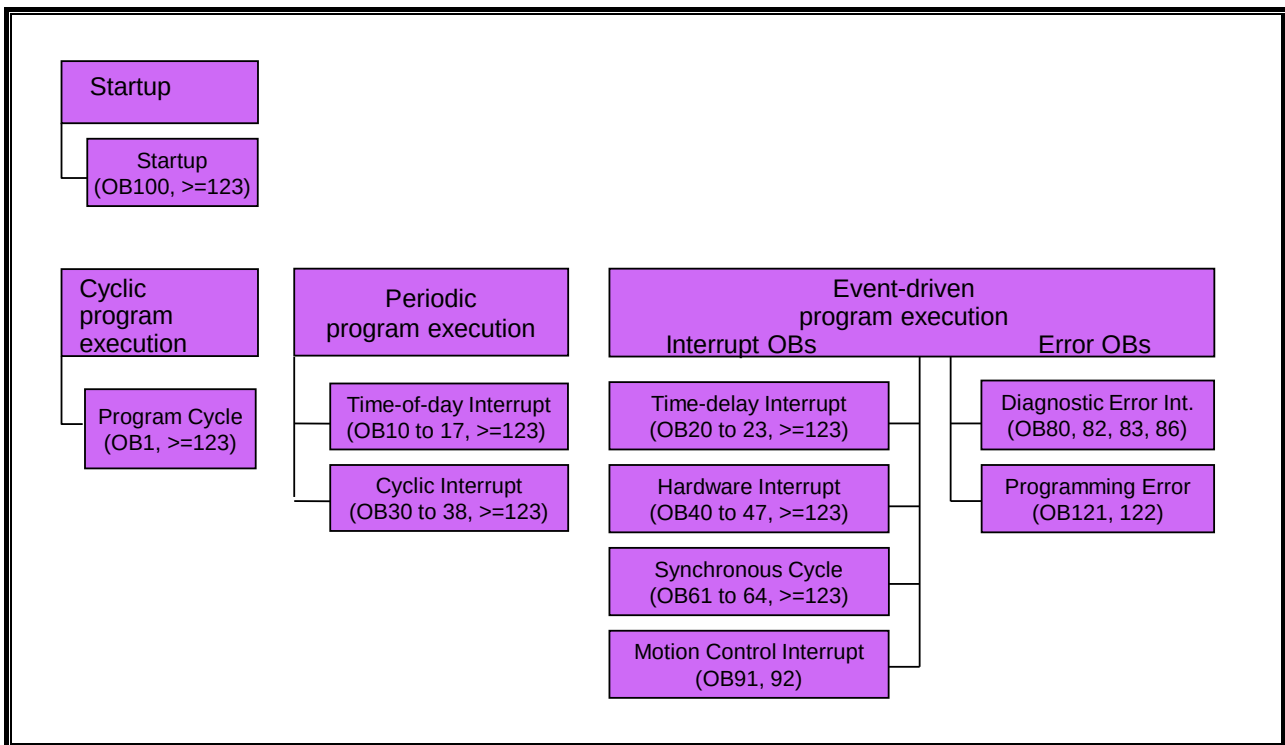
Cyclic program execution occurs in an endless loop. After the execution of a program cycle is completed, the execution of the next cycle occurs automatically. In every program cycle, the CPU carries out the following steps.

- The CPU starts the cycle monitoring time or resets it.
- The CPU scans the states of the input signals and updates the process image inputs.
- The CPU sequentially processes the instructions of the user program and so works directly with the process images, not with the inputs and outputs of the input / output modules.
- The CPU transfers the output states from the process image outputs to the output modules.

Cycle and Cycle Monitoring Time

The time that the CPU requires for the execution of the complete program cycle, is the cycle time which is monitored for time by the CPU operating system. If the cycle time exceeds the cycle monitoring time defined in the CPU properties by more than double, the CPU goes into the STOP state.

3.3. Organization Blocks of the S7-1500



Startup Program

After a power recovery, or a change of operating mode (through the CPU's mode selector or through PG operation), a startup program is carried out before the cyclic program execution. In the startup OBs you can, for example, carry out a pre-assignment of communication connections.

Cyclic Program Execution

The program stored in the Main OB (for example OB1) is executed cyclically, that is, after it is executed completely it is executed again. With this cyclic program execution, the reaction time results from the execution time for the CPU's operating system and the sum of the command runtimes of all executed instructions. The reaction time, that is, how fast an output can be switched in relation to an input signal, amounts to a minimum of one time and a maximum of two times the cycle time.

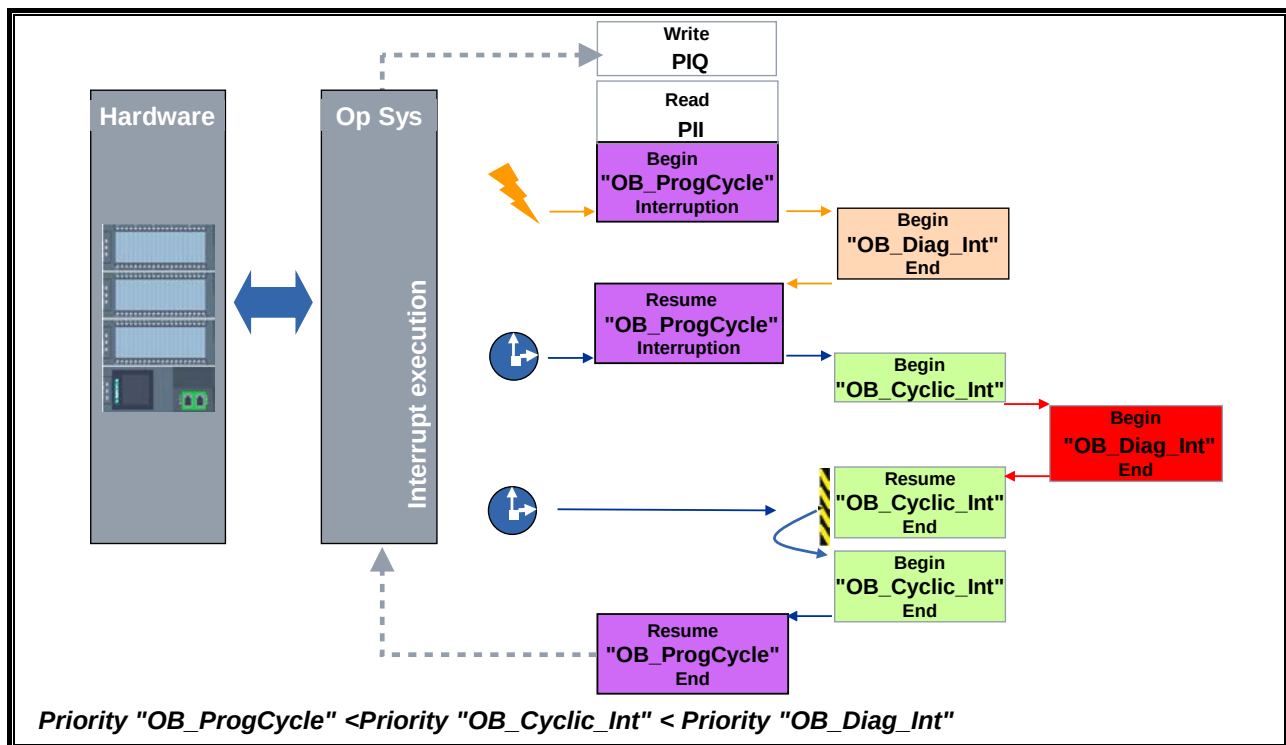
Periodic Program Execution

This makes it possible to interrupt the cyclic program execution at fixed intervals. With the cyclic interrupts, an organization block (for example OB35) is executed after an adjustable time base (for example, every 100ms) has expired. In these blocks, closed-loop control blocks with their sampling time, for example, are called. With the time-of-day interrupts, an OB which carries out a data backup, for example, is executed at a specific time, for example, every day at 17:00 hours (5:00 p.m.).

Event-driven Program Execution

In order to be able to react quickly to a process event, the hardware interrupt can be used. After an event occurs, the cycle is immediately interrupted and an interrupt program is executed. With time-delay interrupts, a freely definable event can be reacted to with a time-delay; with an error OB, the user can influence the behavior of the controller in case there is an error.

3.3.1. Interrupting the Cyclic Program



OB Calls

Organization blocks (OBs) form the interface between the CPU's operating system and the user program.

Organization blocks are called exclusively by the operating system. There are various start events (time-of-day interrupts, hardware interrupts - see picture) that each lead to the start of their associated organization block.

Interrupting the Cyclic Program

When the operating system calls another OB, it interrupts the cyclic program execution because the OB "Program cycle" has the lowest priority. Any other OB can therefore interrupt the main program and execute its own program. Afterwards, the OB "Program cycle" resumes execution at the point of interruption.

Priorities

The S7-1500 CPUs support the priorities 1 (lowest priority) to 26 (highest priority).

The OBs are executed on a purely priority-driven basis. This means that when several OB requests exist at the same time, the OB with the highest priority is executed first. When an event occurs that has a higher priority than the currently active OB, this OB is interrupted. Events of the same priority are executed in the order in which they occur.

The priority can be controlled in the OB's *Properties > Attributes* dialog.

Queue

If, during the execution of an interrupt OB, a further event with the same priority occurs, this event is placed in a queue according to its priority. The start events of a queue are processed at a later time in the order in which they occur. The number of pending events can be limited in order to keep temporary overload situations under control.

When, for example, the maximum number of start events for a cyclic interrupt OB is reached in the queue, every further start event is discarded.

The number of discarded start events is made available at the input parameter "Event_Count".

3.3.2. OB Start Information

Rack or station failure OB – Start information (Standard)

	Name	Data type	Offset	Default value	Comment
1	Temp				
2	EV_CLASS	Byte	...		Event class
3	FLT_ID	Byte	...		Fault identifier
4	PRIORITY	Byte	...		Priority of OB Execution
5	OB_NUMBR	Byte	...		OB number
6	RESERVED_1	Byte	...		Reserved for system
7	RESERVED_2	Byte	...		Reserved for system
8	MDL_ADDR	Word	...		Hardware identifier
9	RACKS_FLTD	Array[0..31] of Bool	...		Rack number
10	DATE_TIME	Date_And_Time	...		Date and time OB started
11	Constant				

Rack or station failure OB – Start information (Optimized)

	Name	Data type	Default value	Comment
1	Input			
2	LADDR	HW_IO		Hardware identifier
3	Event_Class	Byte		Event class
4	Fault_ID	Byte		Fault identifier
5	Temp			
6	<Add new>			
7	Constant			

It is possible to read out the entire start information with RD_SINFO

Instructions

Options

Favorites

Basic instructions

Extended instructions

Name

- Date and time-of-day
- String + Char
- Process image
- Distributed I/O
- PROFenergy
- Module parameter assignm...
- Interrupts
- Alarming
- Diagnostics
 - RD_SINFO
 - RT_INFO
 - LED
 - Get_IM_Data

Start Information

When the operating system calls organization blocks, the user is provided with a system start information in the local data stack.

For standard OBs, the start information has a length of 20 bytes and is available after the start of the OB execution.

For optimized OBs, the start information is reduced to the essentials for runtime reasons, it can however, also be completely read out with the instruction "RD_SINFO".

Variables

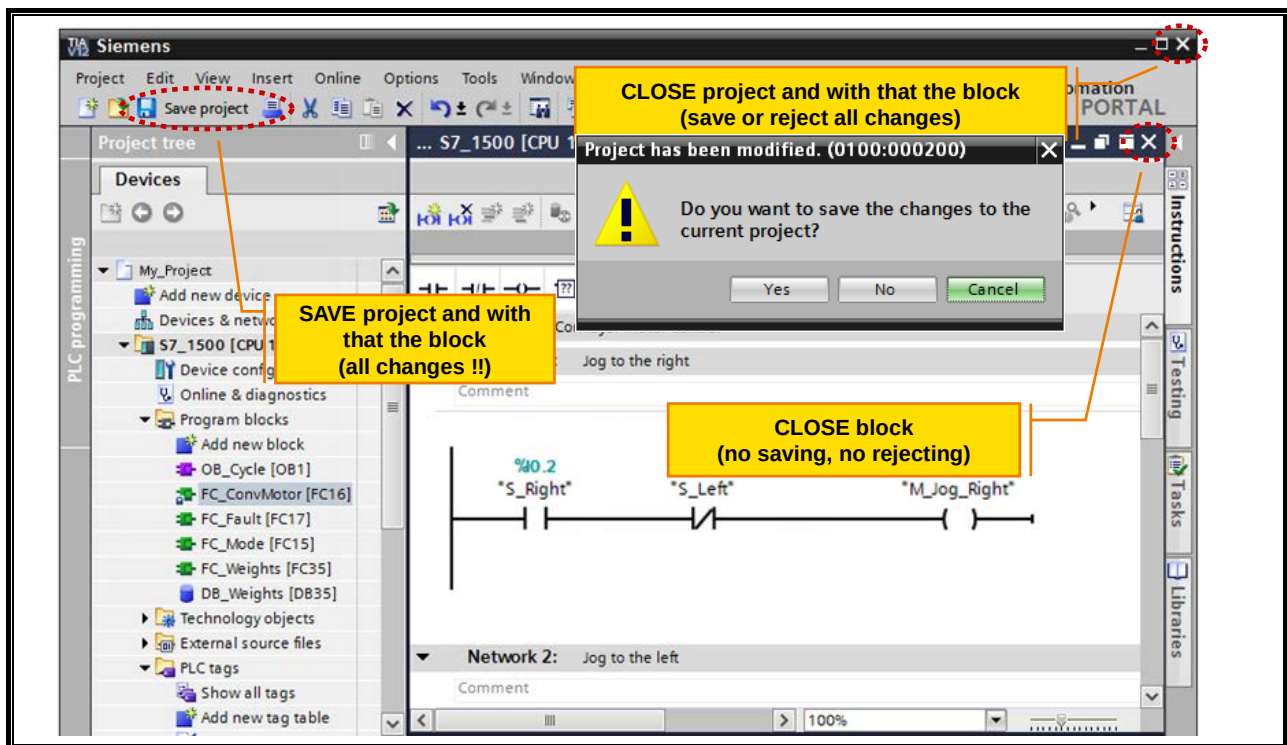
An explanation of the meaning of the variables can be found in the online help.

Note

In order to avoid errors, the structure of the standard declaration section must not be changed by the user. Following the standard declaration section, the user can declare further, additional temporary variables.

The switch-over from standard block to optimized block is done via Properties -> Attributes -> Optimized block access.

3.4.1. Closing / Saving / Rejecting a Block



Closing a Block

By clicking on the  symbol in the title bar, the block is merely closed. Changes are neither rejected nor are they saved on the hard drive!

Saving a Block

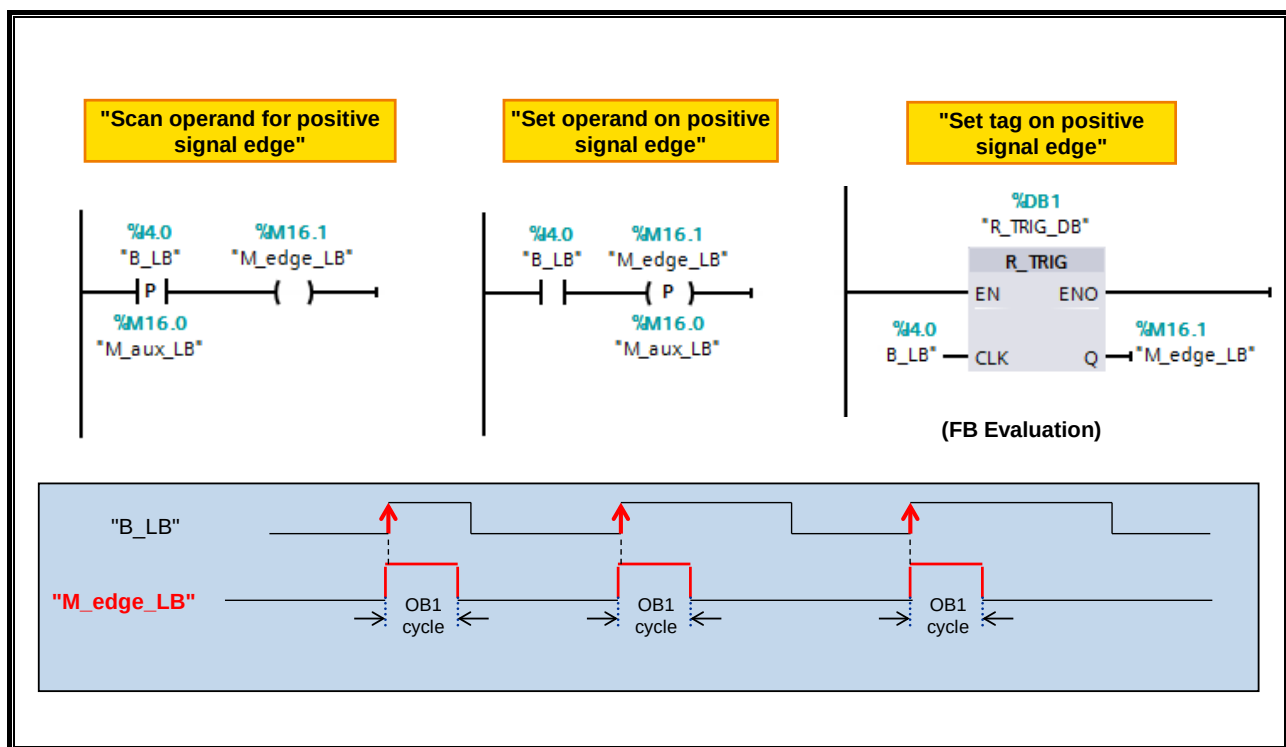


By using "Save project" the entire project, and with that also the block, is saved on the hard drive. All changes made to the project are saved.

Rejecting a Block

It is only possible to reject block changes by closing the entire project without saving. All changes made in the project are rejected.

3.4.2. Operand Edge Evaluation



Scan Operand for Positive Signal Edge

If the state of the operand at the input to the box (in the example "B_LB") changes from '0' to '1' ("rising edge"), RLO '1' is output at the output to the box for the duration of one program cycle.

Set Operand on Positive Signal Edge

On a positive edge of a logic operation or the signal state of an operand (as in the example "LB"), the operand is written with the value True for one cycle via the box (in the example "M_Edge_LB"). At the output of the box, the current input signal is output and thus is available for further logic operations.

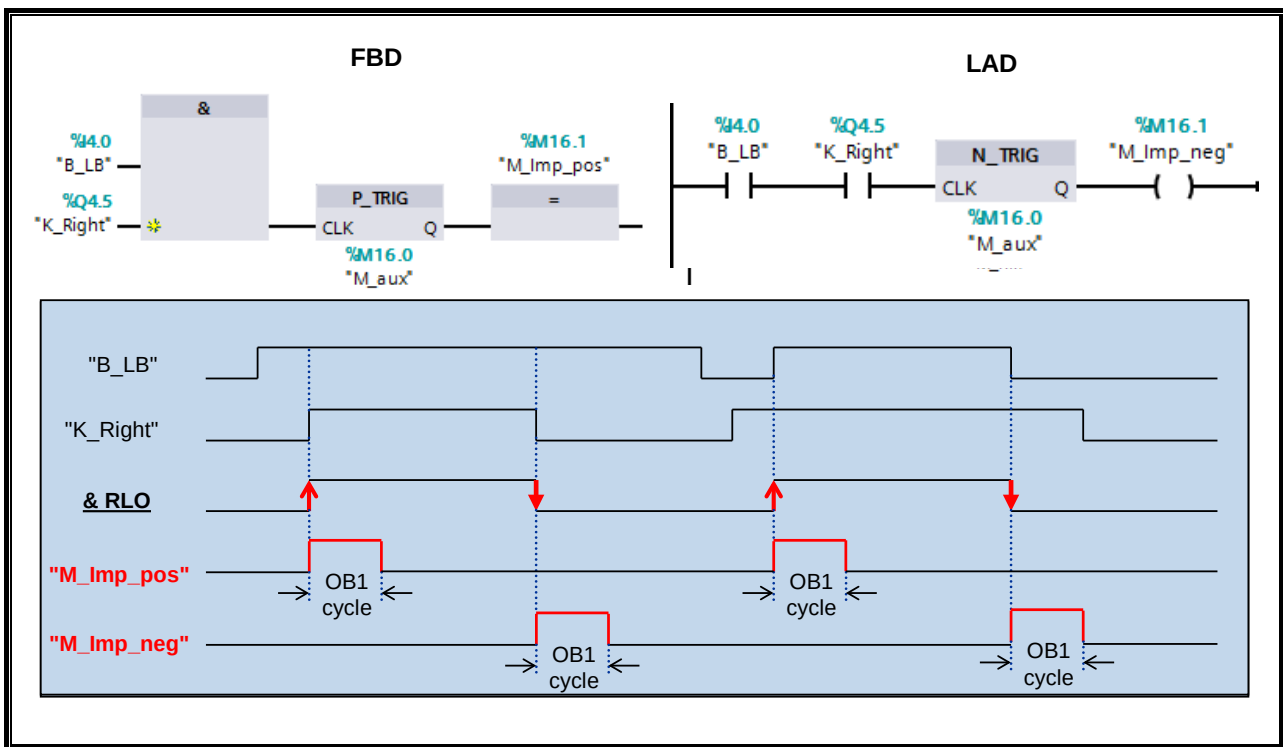
Set Tag on Positive Signal Edge

At output "Q", RLO '1' is output for the duration of one program cycle if a positive edge (RLO or signal state of an individual operand) is detected at input "CLK". The evaluation occurs per function (R_TRIG(ger)), which is to be passed an instance data block, or which can also be created as a multiple instance within another FB.

Note:

The instructions compare the current signal state of input "LB" with the signal state in the previous cycle which is stored in an edge memory bit "M_aux_LB" or in the instance DB "DB_R_TRIG". When the instruction detects a change from "0" to "1", a positive signal edge exists. Subsequently, the current signal state is stored in the edge memory bit or in the instance DB. The status of the edge memory bit or the contents of the IDB must not be overwritten once again at another location in the program.

3.4.3. RLO Edge Evaluation

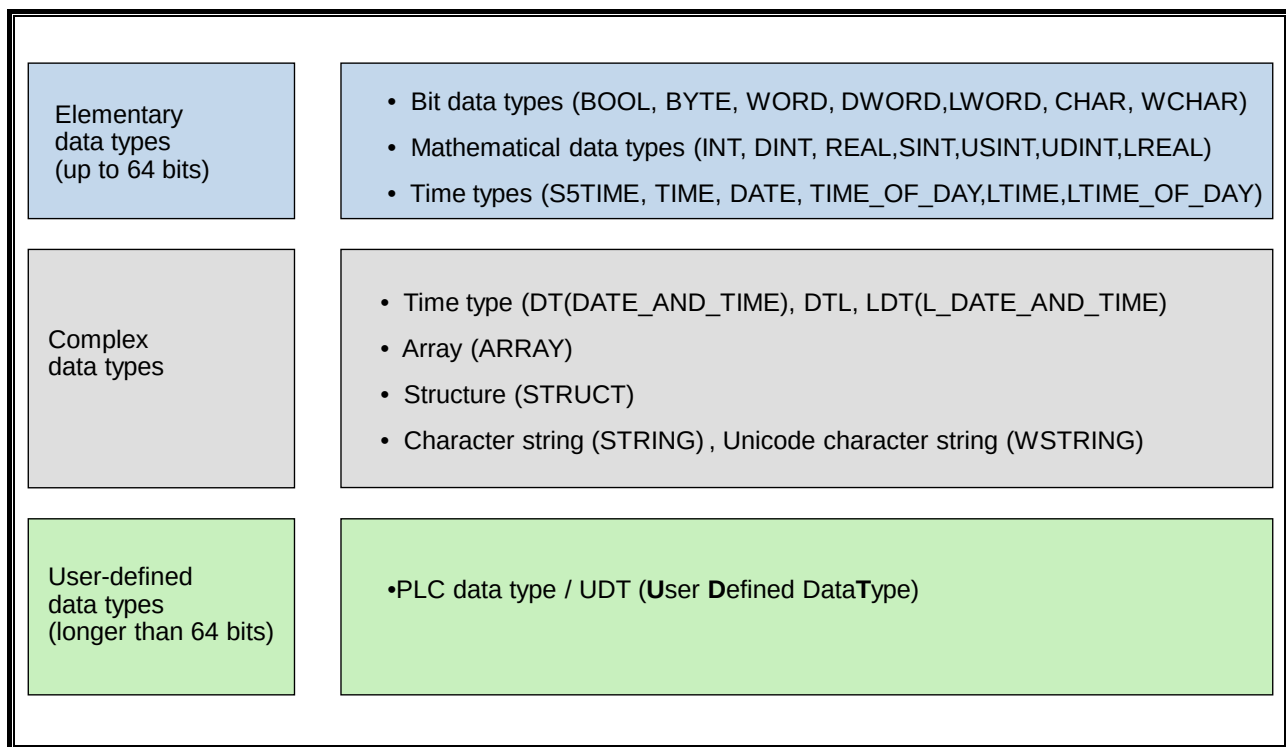


RLO Edge Evaluation (P_TRIG, N_TRIG)

With an RLO edge evaluation, it is possible to detect whether the status of a logic operation (in the example an AND RLO) has changed from '0' to '1' (rising or positive edge) or from '1' to '0' (falling or negative edge). If this is the case the instruction supplies, for the duration of one cycle, RLO '1' as the result, which can be further logically linked or can be assigned to another operand (in the example, the memory bit "M_Imp_pos" (M16.1)) as status. In the following cycle, the instruction then once again supplies '0' as the result even if the AND RLO still is status '1'.

The instruction compares the current result or the RLO of the logic operation with its RLO in the previous program cycle. This is stored in a so-called edge memory bit for this (in the example "M_aux"). It must be ensured that the status of this edge memory bit is not overwritten at another location in the program. For every RLO edge evaluation, a separate edge memory bit must be used accordingly, even then when the same operand (in the example, AND RLO) is evaluated once again, for example, in another block!

3.5. Overview: Data Types in STEP 7



Elementary Data Types

Elementary data types are predefined in accordance with IEC 61131-3. They always have a length less than or equal to 64 bits and can be further processed with elementary STEP 7 instructions.

Complex Data Types

Complex data types contain data structures that can be made up of elementary and/or complex data types. Complex data types can be used for the declaration of variables only in global data blocks and within blocks for the declaration of local variables (TEMP, STAT) as well as parameters (IN, OUT and INOUT). Variables of complex data types cannot be completely processed with elementary instructions (such as, A, O, L, T, +I) but only the individual components of the elementary data type.

PLC Data Types

PLC data types are data types defined by you that are used as templates for declaring parameters and variables of complex data types (e.g. structure variables). UDTs are created with the Data Block Editor and contain a data structure that is made up of elementary and/or complex data types. In the declaration of a variable according to data type UDTx, a structure variable is created whose inner data structure is defined by a UDT. UDTs can be used for the declaration of variables in global data blocks and within blocks for the declaration of local variables (TEMP, STAT) as well as parameters (IN, OUT and INOUT).

3.5.1. Elementary Data Types

Data type	Length (in bits)	Constants	Variables
BOOL	1	1 or 0	I 1.0
BYTE	8	B#16#A9	MB70
WORD	16	W#16#12AF	MW72
DWORD	32	DW#16#ADAC1EF5	QD40
LWORD	64	LW#16#5F52DE8B	
CHAR	8	' w '	DBB4
WCHAR	16	WCHAR#'a '	MW10
INT	16	123	#Value
DINT	32	L#65539	MD80
REAL	32	1.2 or 34.5E-12	DBD60
SINT	8	+/-50	MB24
USINT	8	50	MB24
UDINT	32	4875678	DBD64
LREAL	64	LREAL#1.0e-5	
LINT	64	LINT#+1543258759	

BOOL, BYTE, WORD, DWORD, CHAR, WCHAR

Variables of the data type BOOL consist of one bit. Variables of the data types BYTE, WORD, DWORD are bit sequences of 8, 16 or 32 bits. The individual bits are not evaluated in these data types. Special forms of these data types are the BCD numbers and the count value as it is used in conjunction with the count function as well as the data type CHAR which represents a character in the ASCII code and WCHAR for which a character is stored in Unicode.

INT, DINT, REAL

Variables of these data types represent numbers with which relevant arithmetical calculation operations can be carried out.

Extensions of INT, DINT, REAL, WORD

U - Unsigned

Variables with the extension U represent an integer without sign.

Data types: USINT, UINT, ULINT, UDINT

S - Short

Variables with the extension S represent an integer with a length of 8 bits.

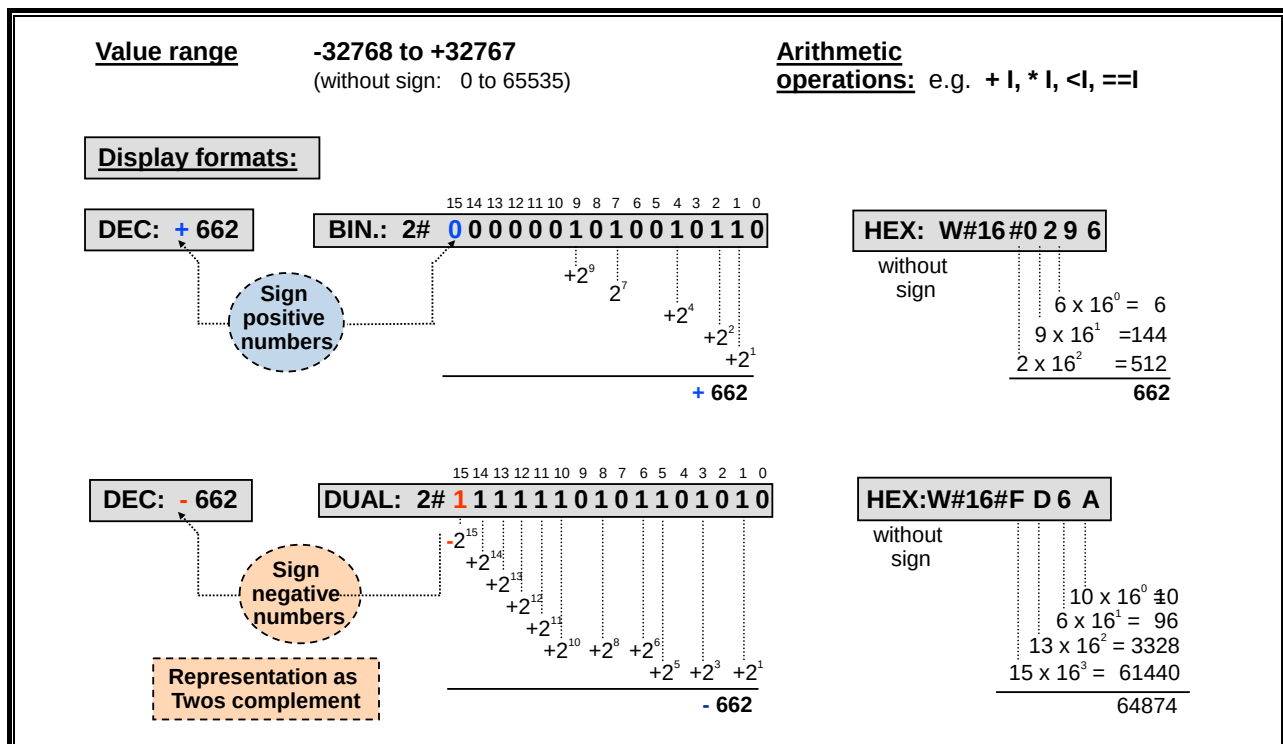
Data types: SINT, USINT

L - Long

Variables with the extension L represent a number with a length of 64 bits of the data type.

Data types: LWORD, LINT, ULINT, LREAL

3.5.1.1. Integer (INT, 16-Bit Integer) Data Type



Integer (16-Bit) Data Type

An *Integer* data type value is a whole number value, that is, a value without a decimal point. SIMATIC S7 stores *Integer* data type values with sign in 16 bit code. This results in the value range shown in the picture above. As well, SIMATIC S7 provides arithmetic operations for processing Integer values.

Decimal

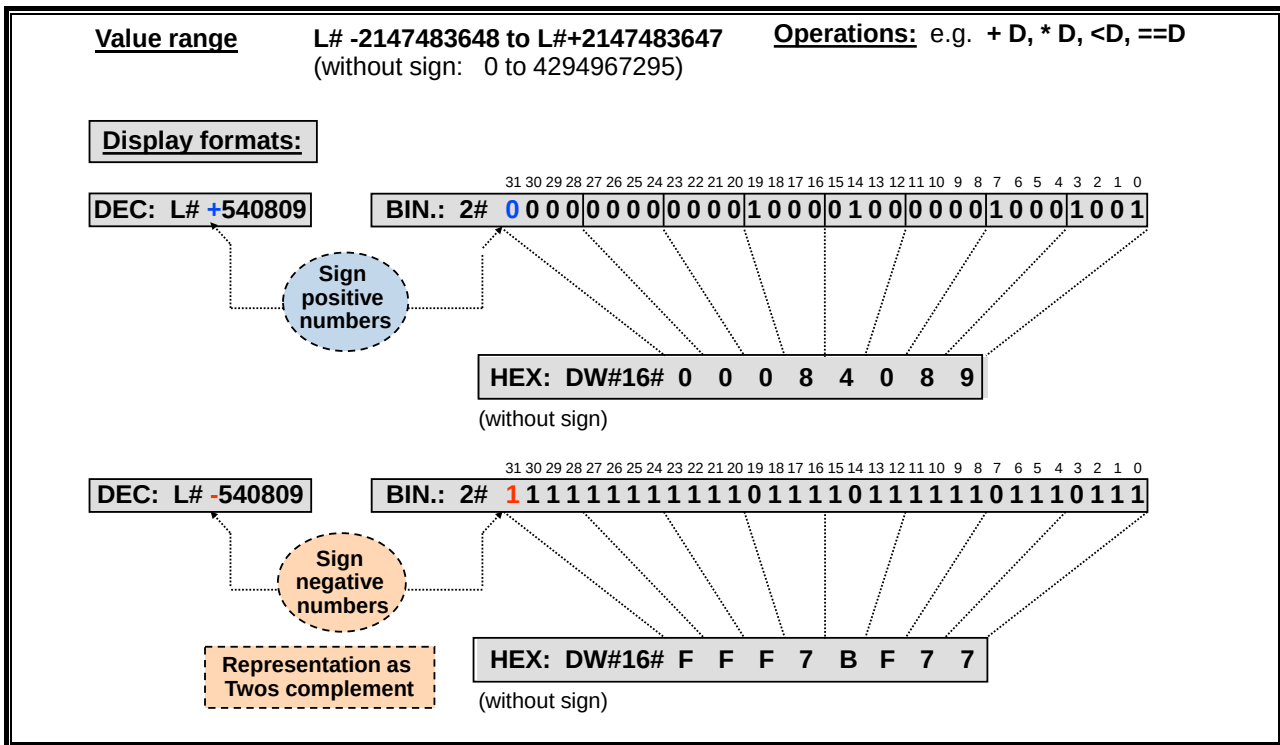
STEP7 uses the *Decimal* (not BCD!) display format to specify the constants of the *Integer* data type with sign and without explicit format description. The use of constant Integer values in the *Binary* and *Hexadecimal* display formats is possible in principle, but because of the poor legibility, they are more or less not suitable. For this reason, the syntax of STEP7 provides the specification of Integer values only in the decimal display format.

Binary

In a digital computer system, all values are stored in a binary-coded form. Only the digits 0 and 1 are available in the binary number system. Base 2 of this number system results from the number of available digits. Accordingly, the value of every position of a binary number results from a power of Base 2. This is also expressed in the format specification **2#....**.

Negative values are represented as binary numbers in two's complement. In this representation, the most significant bit (bit no. 15 for the Integer data type) has the value -2^{15} . Since this value is greater than the sum of all residual values, this bit also has the sign information. That is, if this bit = 0, then the value is positive; if the bit = 1, then the value is negative. The conversion of a binary number into a decimal number is made by adding the values of the positions that have a 1 (see picture).

3.5.1.2. Double Integer (DINT, 32-Bit Integer) Data Type



Double Integer (32-Bit Integer)

SIMATIC S7 stores *Double Integer* data type values with sign as 32 bit code. This results in the value range shown in the picture above. As well, SIMATIC S7 provides arithmetic operations for processing DINT values.

Decimal

STEP7 uses a decimal number (not BCD!) to specify a constant of the *Double Integer* data type with sign and the format **L#** for "long" (double word, 32 bit). When a value smaller than -32768 or greater than 32767 is specified, the format **L#** is automatically added. For negative numbers smaller than -32768, the user must specify the format as **L# -** (for example: **L# -32769**). This is imperative if the value is to be further processed arithmetically as a double integer since you would otherwise work with false values (value + sign)!

Hexadecimal

The hexadecimal number system provides 16 different digits (0 to 9 and A to F). This results in Base 16 of this numbers system. Accordingly, the value of every position of a hexadecimal number results from a power of Base 16.

Hexadecimal numbers are specified with the format **W#** for the dimension (W = word = 16 bit) or **DW#** (DW = double word = 32 bit) and **16#** for identifying the basic numbering system. The number of specifiable bits is variable from 1 to 8 whereby missing (unspecified) bits are filled with leading zero digits.

The digits A to F correspond to the decimal values 10 to 15. The value 15 is the last value that can be binary-coded - without sign - with 4 bits. Out of this correlation, the simple conversion of a binary number into a hexadecimal number and vice versa can be obtained. In this way, four binary bits each can easily make up one digit of a hexadecimal number.

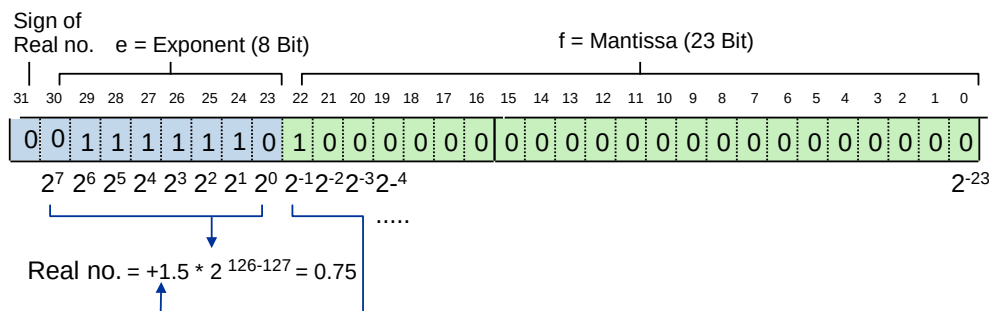
3.5.1.3. REAL (Floating-point Number, 32 Bit) Data Type

Value range $-3.402823 \cdot 10^{+38}$ to $-1.175495 \cdot 10^{-38}$, 0.0, $+1.175495 \cdot 10^{-38}$ to $+3.402823 \cdot 10^{+38}$

Operations: e.g. + R, * R, <R, ==R
sin, acos, ln, exp, SQR

General format of a Real number = (Sign) • (1.f) • (2^{e-127})

Example: 0.75



Real

The previously described INT and DINT data types are used to store whole number values with sign. Accordingly, only operations that supply a whole number value as the result can be performed with these values. In cases where analog process variables such as voltage, current, and temperature have to be processed, it becomes necessary to use *Real* values (real numbers, "decimal numbers"). In order to be able to represent such values, binary digits have to be defined whose value is less than 1 (power of base 2 with negative exponent).

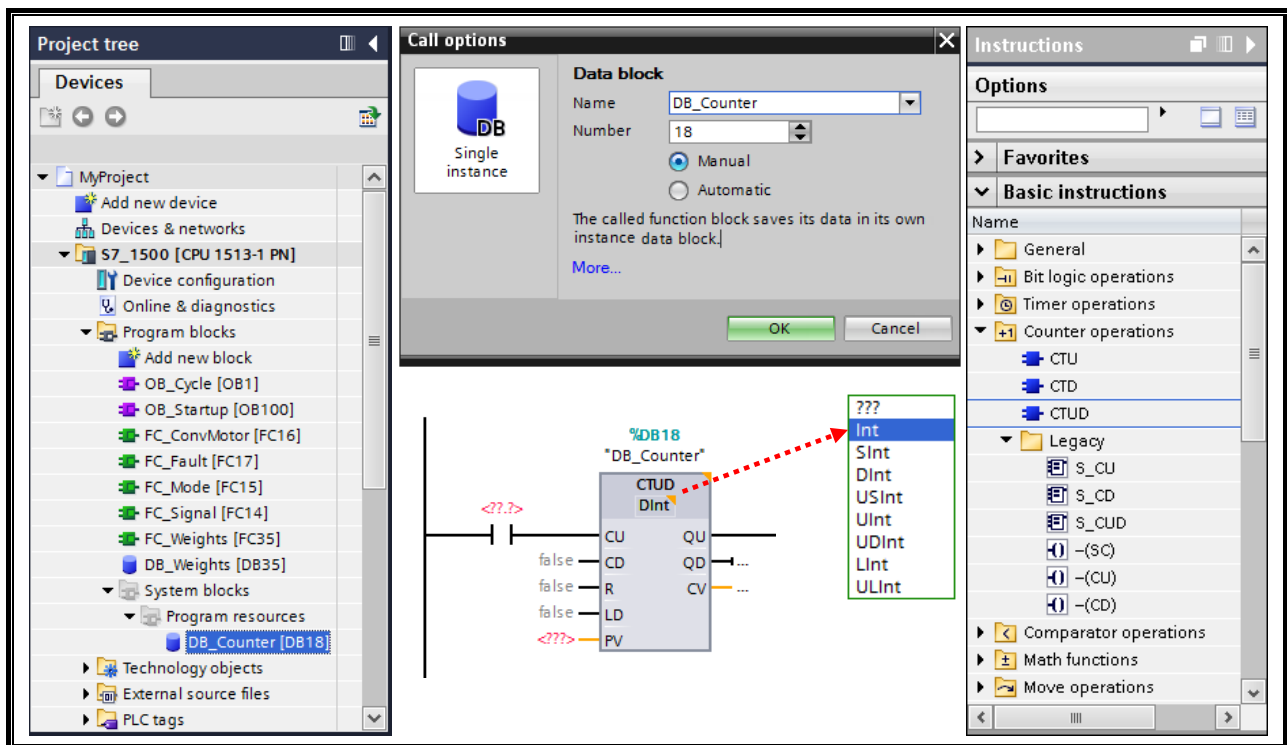
Real Format

In order to be able to form the greatest possible value range within a defined memory capacity (for SIMATIC S7: double word, 32 bit) (see picture), you must be able to select the decimal point position as required. Early on, IEEE defined a format for floating-point numbers. This format was laid down in IEC 61131 and was included in STEP 7. This format makes it easy to process a variable decimal point position. In a binary coded floating-point number, a portion of the binary digits contain the mantissa (23 Bit) and the rest contain the exponent (8 Bit) and the sign of the floating-point number. When you specify real values, you do so without specifying the format. After you enter a constant real value (for example: 0.75), the Editor automatically makes a conversion to scientific notation (for example: 7.5000e-001).

Application

Floating-point numbers are used for "analog value processing", among other things. A great advantage of floating-point numbers is in the number of operations possible with such numbers. These include, in addition to the standard operations such as +, -, *, / also instructions such as sin, cos, exp, ln, etc, that are used mainly in closed-loop control algorithms.

3.5.2. IEC Counters: CTU, CTD, CTUD



Counters

Counters are used to count events, record quantities, etc. There are up counters and down counters as well as counters that can count in both directions.

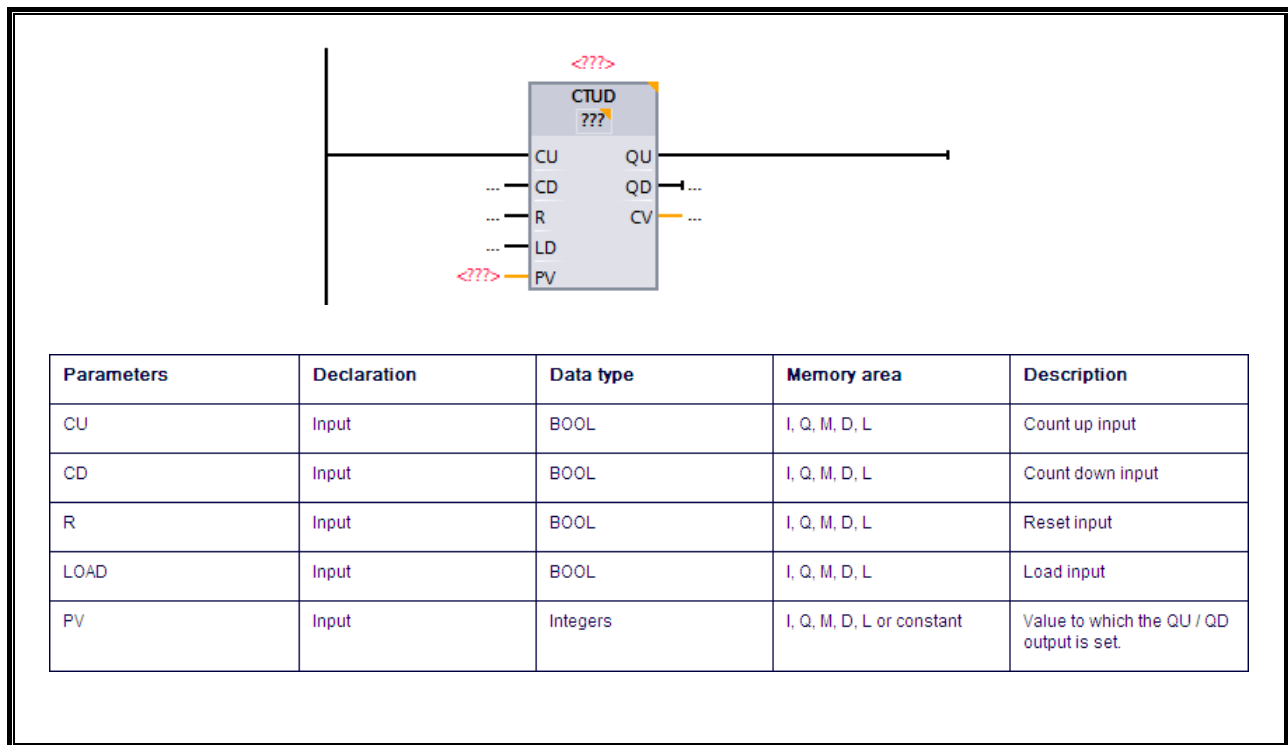
Value Range of a Counter

The count or value range of a counter depends on its data type (see picture) which is always an integer. The various selectable Integer data types merely differentiate themselves in their value range and thus determine the count range of the counter.

Instance Data Block

In addition to internally required variables, the counter also stores its current counter value in a so-called instance data block which must be specified when programming a counter. The specified instance data block is automatically generated by the Editor with exactly the internal structure that the counter requires. The user has no further programming effort with this data block other than having to download it into the CPU.

3.5.2.1. Counter Function: Inputs



Input CU and CD

With a positive edge at input CU, the current count is increased by one; with a positive edge at input CD, the current count is decreased by 1. If a positive edge is detected at both inputs simultaneously or in the same cycle, the current count remains unchanged. If the upper or lower limit of the specified data type is reached, the count is no longer increased or decreased for a positive edge at CU or CD.

Input R

The input R acts statically, that is, as long as RLO '1' is at input R, the count is set to 0 and rising edges or RLO '1' at the inputs CU, CD and LOAD have no effect on the current count.

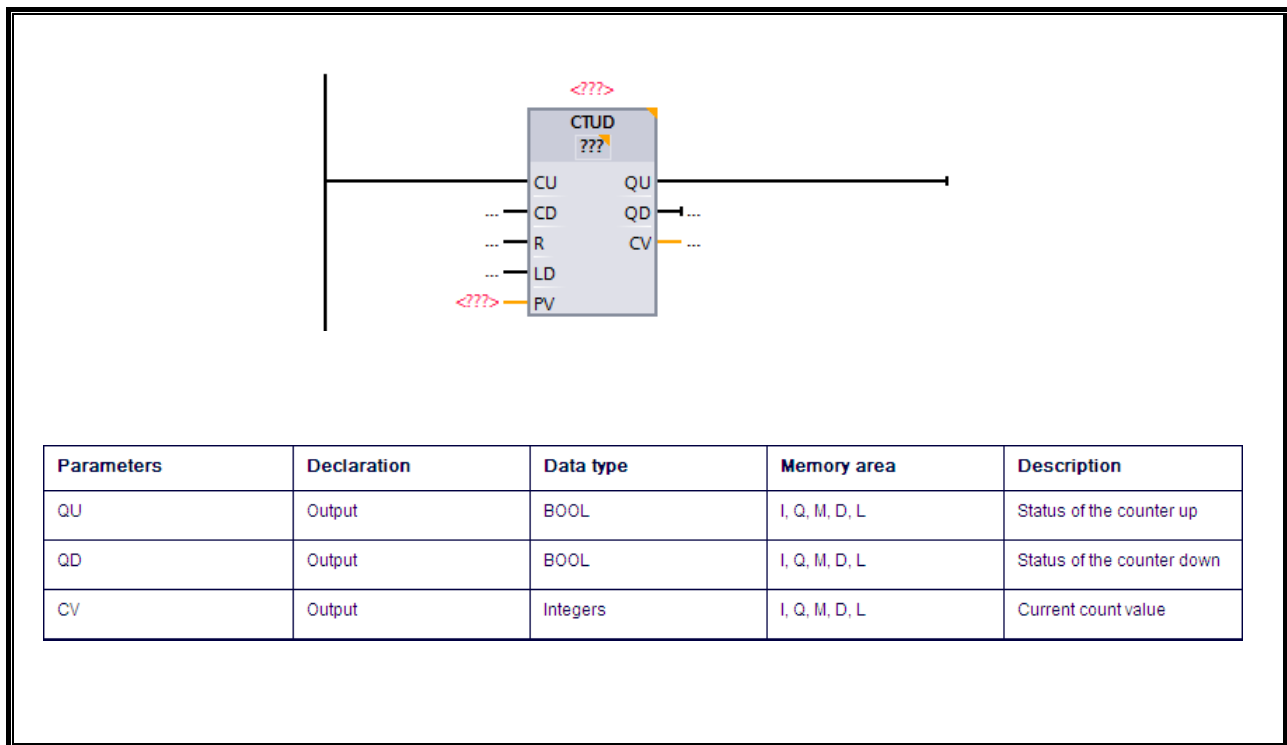
Input LD (Load, only for down counters)

The input LD acts statically, that is, as long as RLO '1' is at input LOAD, the current count is set to the value that is passed to the input PV, and positive edges at the inputs CU and CD have no effect on the count.

Input PV

The value to which the count is to be set must be passed to the input PV as long as RLO '1' is at input LD. The variable or constant passed to the input must be compatible with the data type of the counter.

3.5.2.2. Counter Function: Outputs



Output QD (Only for down counters)

The current status of the down counter can be checked at the output QD. As long as the current count is less than or equal to zero, the output QD has Status "1", otherwise, Status '0'.

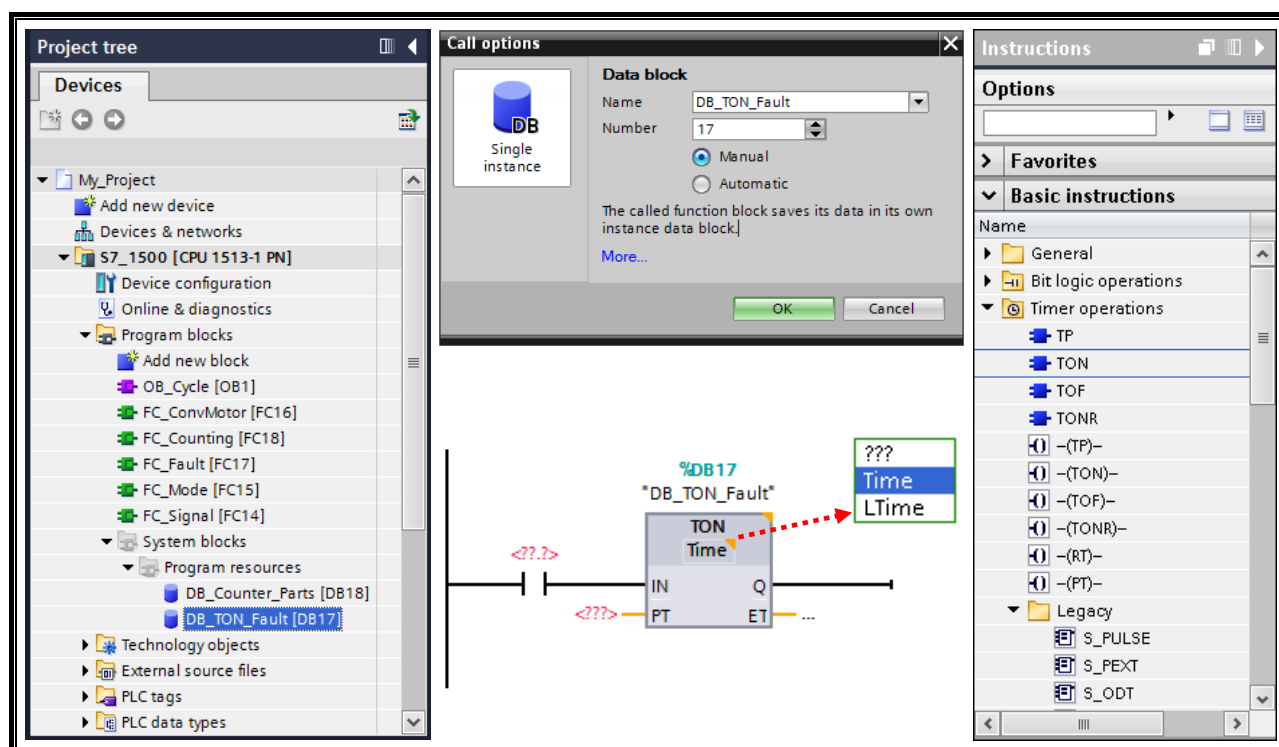
Output CV

The current count is output at output CV. The variable passed to the output must be compatible with the data type of the counter.

Output QU

The current status of the up counter can be checked at the output QU. As long as the current count is greater than or equal to the value of the parameter PV, the output QU has Status '1', otherwise, Status '0'.

3.5.3. IEC Timer Function: TON



Data Type TIME

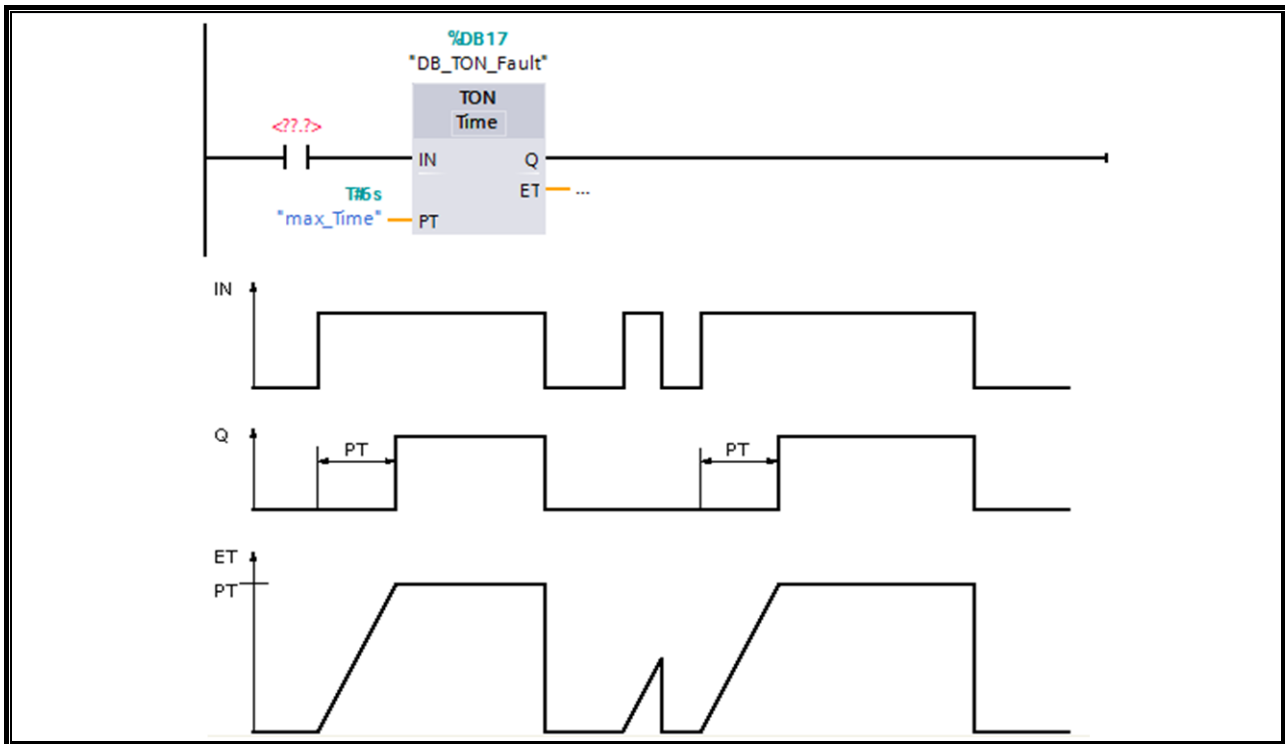
The contents of a variable or constant of the data type TIME is interpreted as an integer number in milliseconds and stored in the memory as a 32-bit integer with sign. The representation contains information for days (d), hours (h), minutes (m), seconds (s) and milliseconds (ms). Examples:

Length (bits)	Format	Value range	Examples of value input
32	Time period with sign:	T# -24d20h31m23s648ms to T#+24d20h31m23s647ms	T#10d20h30m20s630ms, TIME#10d20h30m20s630ms, 10d20h30m20s630ms

Data Block

In addition to internally required variables, the timer function also stores the current already expired time in a data block which must be specified when programming the timer function. The specified data block is automatically generated by the Editor with exactly the internal structure that the timer function requires. The user has no further programming effort with this data block other than having to download it into the CPU.

3.5.4. Timer Function TON (ON Delay) Pulse Diagram

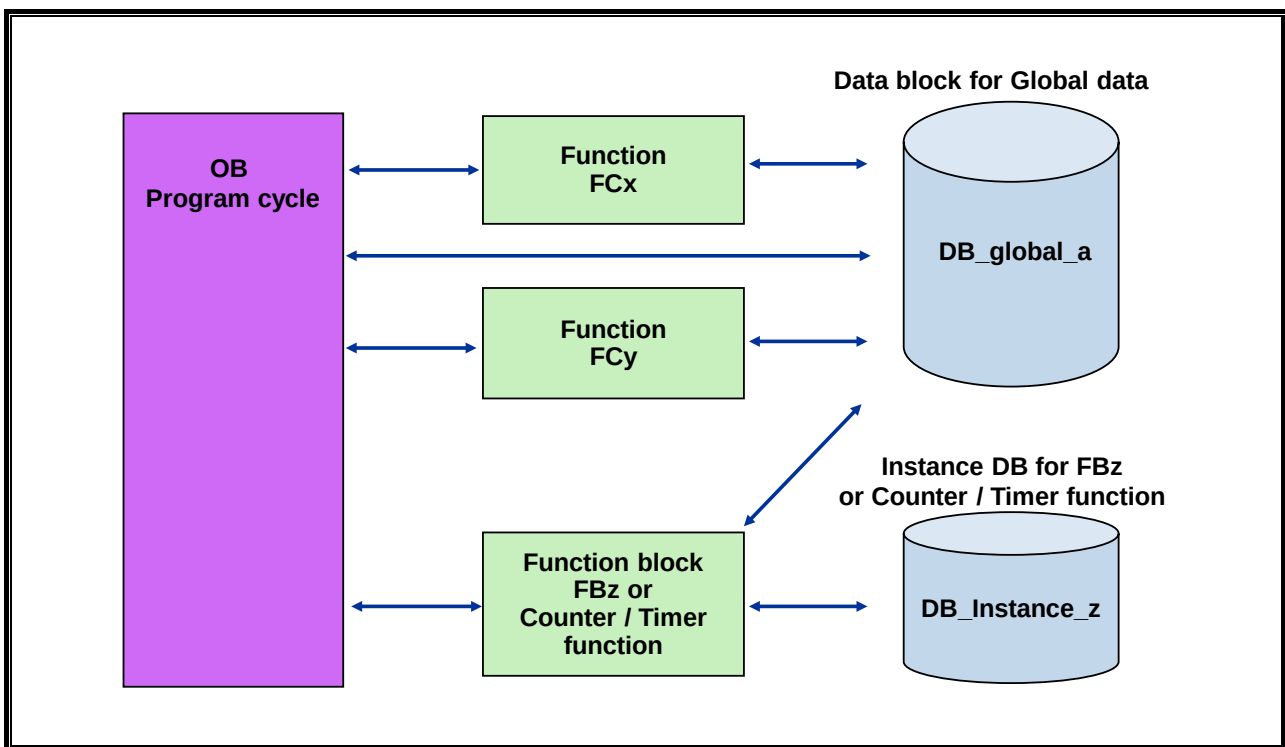


TON

The timer function TON (Timer on delay, "ON Delay") is started with a rising edge at input IN. So that the time expires, RLO must continue to be '1'. The timer function supplies a '1' signal at output Q, as soon as the specified time (variable or constant of data type TIME) at input PT has expired and as long as the start signal at input IN still exists. The already expired time can be queried at output ET by passing a variable of data type TIME.

Parameters	Declaration	Data type	Memory area	Description
IN	Input	BOOL	I, Q, M, D, L	Start input
PT	Input	TIME	I, Q, M, D, L or constant	Duration of the on delay. The value of the PT parameter must be positive.
Q	Output	BOOL	I, Q, M, D, L	Output that is set when the time PT expires.
ET	Output	TIME	I, Q, M, D, L	Current time value

3.5.4.1. IEC Timer / Counter Instance Data Blocks



Data Blocks

Data blocks are used for storing user data. They occupy memory space in the user memory of the CPU. Data blocks contain variable data (such as numeric values) with which the user program works. The user program can access the data in a data block with bit, byte, word or double-word operations. Access can be made symbolically or absolutely.

Area of Application

Data blocks are used with two different purposes:

- Global data blocks: These contain information that all the logic blocks in the user program can access.
- Instance data blocks: These are always assigned to a particular FB or functions such as counters and timers. The data of these instance DBs should only be processed by the associated FB or counter / timer function.

Creation of DBs

Global DBs are created either with the Program Editor or according to a previously created "user-defined data type". Instance data blocks are generated when a function block or a counter / timer function is called.

3.6. STEP 7 - Test Functions, Overview

Errors Detected by the System		Functional Errors
• Programming error (w/o OB121): • Exceeding the cycle time • Access error • Diagnostics interrupts	STOP STOP RUN RUN	• Process fault (e.g. wire break) • Logical programming error (e.g. double assignment)
• Online & Diagnostics - Diagnostics buffer • Task Card "Testing" - Call hierarchy - Local data stack (in planning stage) • Diagnose Modules - Diagnostic status (all modules)		• Monitor / Modify Variables → Watch and Force tables • Monitor Blocks (Block Status) → Monitor in the Blocks editor - Additionally with call environment • Tools - Cross references - Assignment list (I/Q/M/T/C) - Call structure
	• "Trace" analyzer function • Program comparison • Set breakpoints	in planning for S7-1200/1500

Test Functions

There are various STEP 7 test functions for troubleshooting, depending on the type of error caused:

- **...when CPU in STOP**

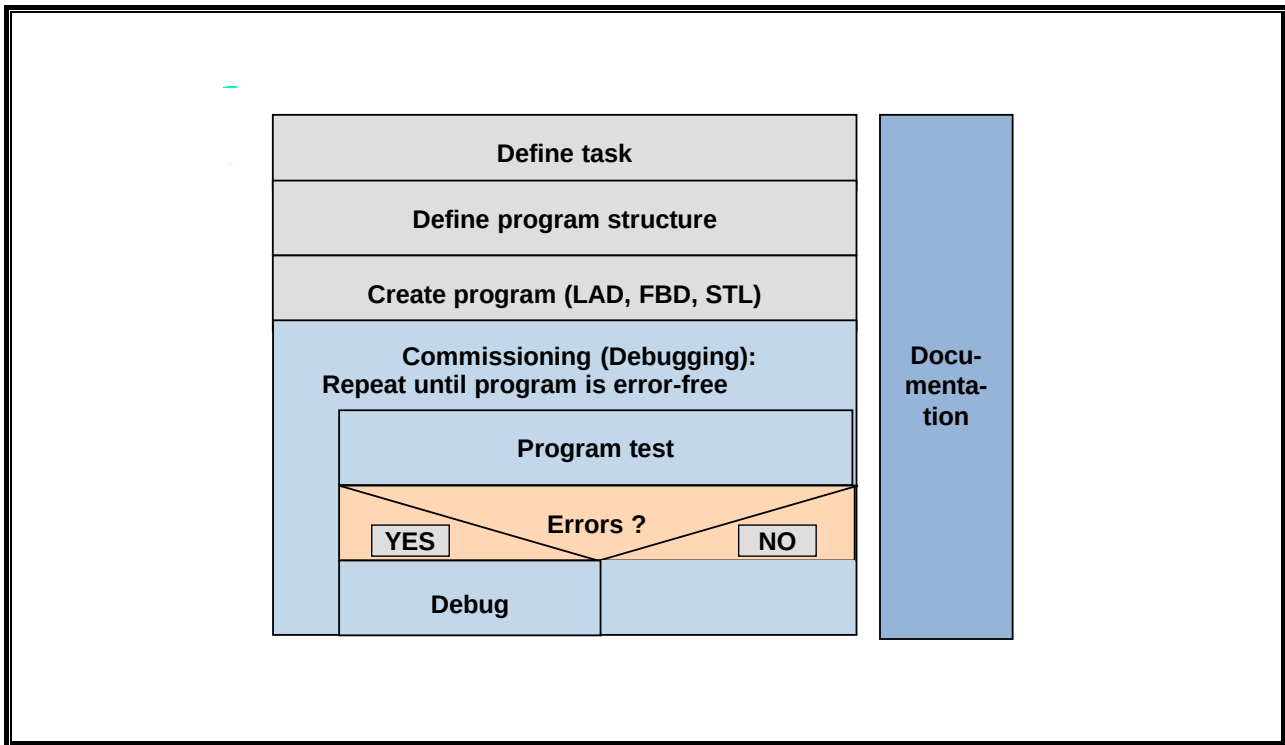
For errors that are detected by the system, the test functions Diagnostics buffer, Call hierarchy, Local data stack and Hardware diagnostics give detailed information on the cause of the error and the location of the interruption. By programming Error OBs, information on the error that occurred can be evaluated by program and the transition of the CPU into the STOP state can be prevented. If the CPU has stopped, the use of the test functions Monitor / Modify Variable and Monitor Blocks makes little sense since the CPU neither reads nor outputs process images while in the STOP state, and also no longer executes the program.

- **...when CPU in RUN**

Vice versa, it makes little sense, as a rule, to use test functions such as Local data stack for troubleshooting when the CPU is in RUN, since program execution has not been interrupted and the system does not provide any information on the error that occurred. The Module Information test function merely provides general information on the CPU's operating status or on errors that occurred in the past. Functional errors can be diagnosed as follows:

- **Process Fault** (such as a wiring error)
 - wiring test of the inputs: *Monitor Variable*
 - wiring test of the outputs: *Enable Peripheral Outputs* (only for CPU STOP)
- **Logical Programming Errors** (such as a double assignment)
 - All test functions listed, with the exception of *Enable Peripheral Outputs*, can be used for searching for logical program errors.
- **Force:** Forced control of operands regardless of the program logic
- **Breakpoints:** Program execution in single steps

3.7. Procedure for Creating a Program



Procedure

When complex correlations are to be converted into functions and function blocks, the following steps are necessary:

1. Task definition:

For tasks such as data management, measuring functions etc., you have to define which information has to be processed and which results are to be supplied.

2. Program structure:

Here you define which steps are necessary to achieve the goal (result) and in which sequence these individual steps have to be processed. The structogram can help you here.

3. Program creation:

Creation of a symbol table. Decision, whether the block is to be parameter assignable or not. Possibly: creation of the declaration table for the block parameters and the local variables.

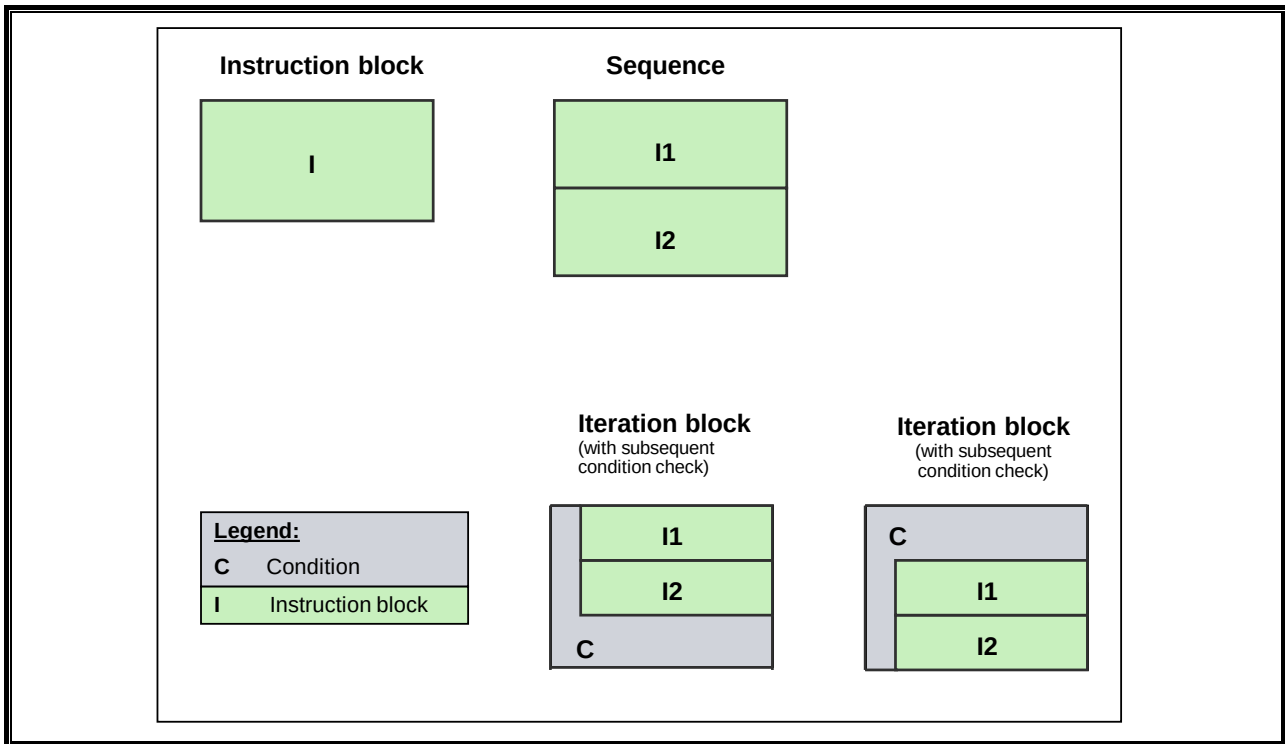
4. Commissioning, Program test and Debugging:

You can check the function of a block using "Program status". With "Monitor/Modify Variable", you can monitor or modify variables. The test function "Breakpoint" is suitable for blocks with jumps and loops so that you can check the program step by step. You can eliminate formal errors by checking the cross reference list for double assignment or the program structure for program loops, for example.

5. Documentation:

An accompanying document is necessary for every phase of the program creation.

3.7.1. Structure Elements of Structograms (1)



Area of Use

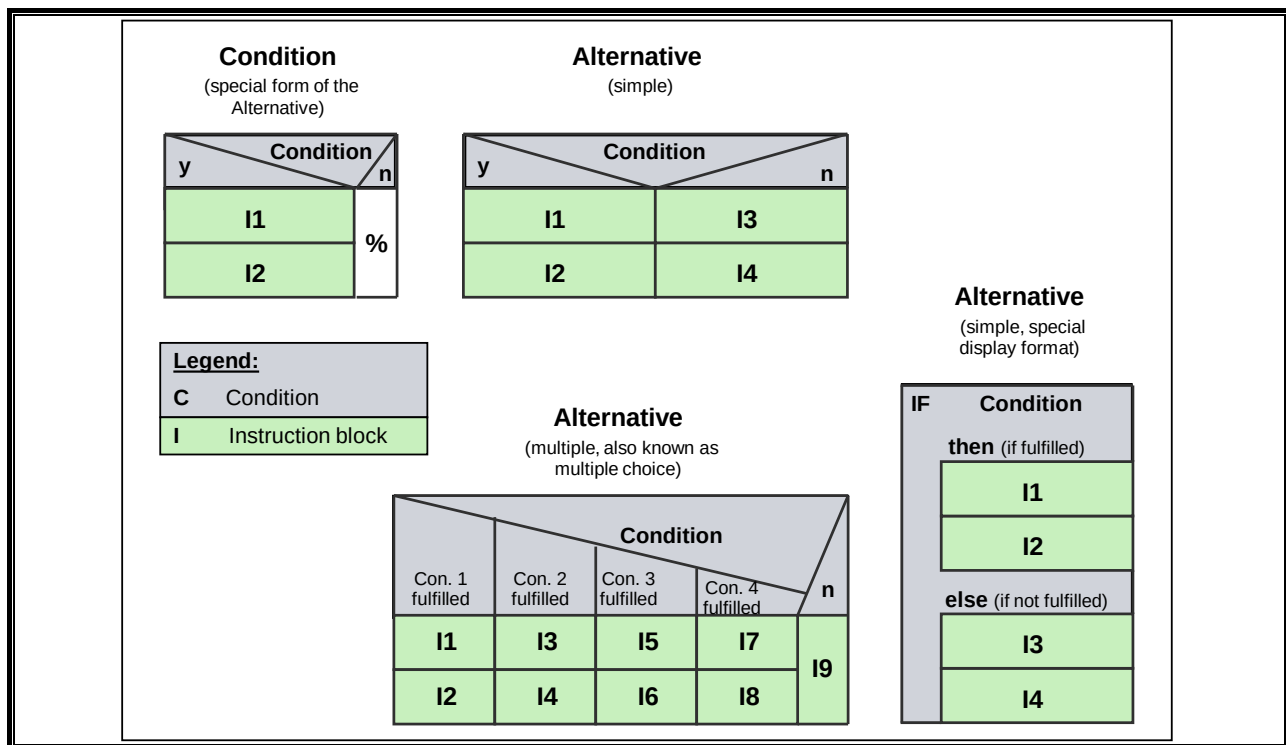
When configuring programs or complex blocks without graphic help, the difficulty exists in simultaneously developing the solution algorithm in the mind and converting this directly into program code at one go.

Structograms according to Nassi - Schneiderman

Structograms provide a sensible alternative here. The programming person first of all plans the procedure (algorithm) or the function using graphic means and in a second step, codes the structogram, that is, converts it into a program. The diagrams are standardized according to DIN 66261. They are well known and wide-spread both nationally and internationally and can therefore also be used as a meaningful documentation means.

- **Structure element**
Structograms consist of structure elements that are used by the user in any element sequence he likes and that are given task-specific texts. Instead of an identifier, such as 'C' as in Condition, the program designer should use plain text, such as '*Recipe No. 10 selected?*'.
- **Instruction block**
The instruction block is the simplest structure element. It contains a freely definable action or action group that proceeds linearly and doesn't contain any branches or similar.
- **Sequence**
The sequence is a linear sequence of instructions that are executed one after the other. The direction of execution is always from top to bottom.

3.7.2. Structure Elements of Structograms (2)



Iterations

With iterations, individual instructions or whole sequences, for example, are repeated until the repetition condition is no longer fulfilled. Iterations are also referred to as loops. There are two types of iterations: start-controlled and end-controlled iterations.

- ...start-controlled: With start-controlled iterations, the loop is only executed when the repetition condition is already fulfilled at the beginning.
- ... end-controlled: With end-controlled iterations, the loop is executed at least once. Repetition then depends on the fulfillment of the repetition condition.

Condition

The condition is a special form of alternative. The sequence or instruction block is only executed if the condition is fulfilled. It has no alternative.

Alternative

The simple alternation gives only one condition. If the condition is fulfilled, the "yes" branch is executed, otherwise the alternative "no" branch. An alternative can, of course, contain an underlying alternative.

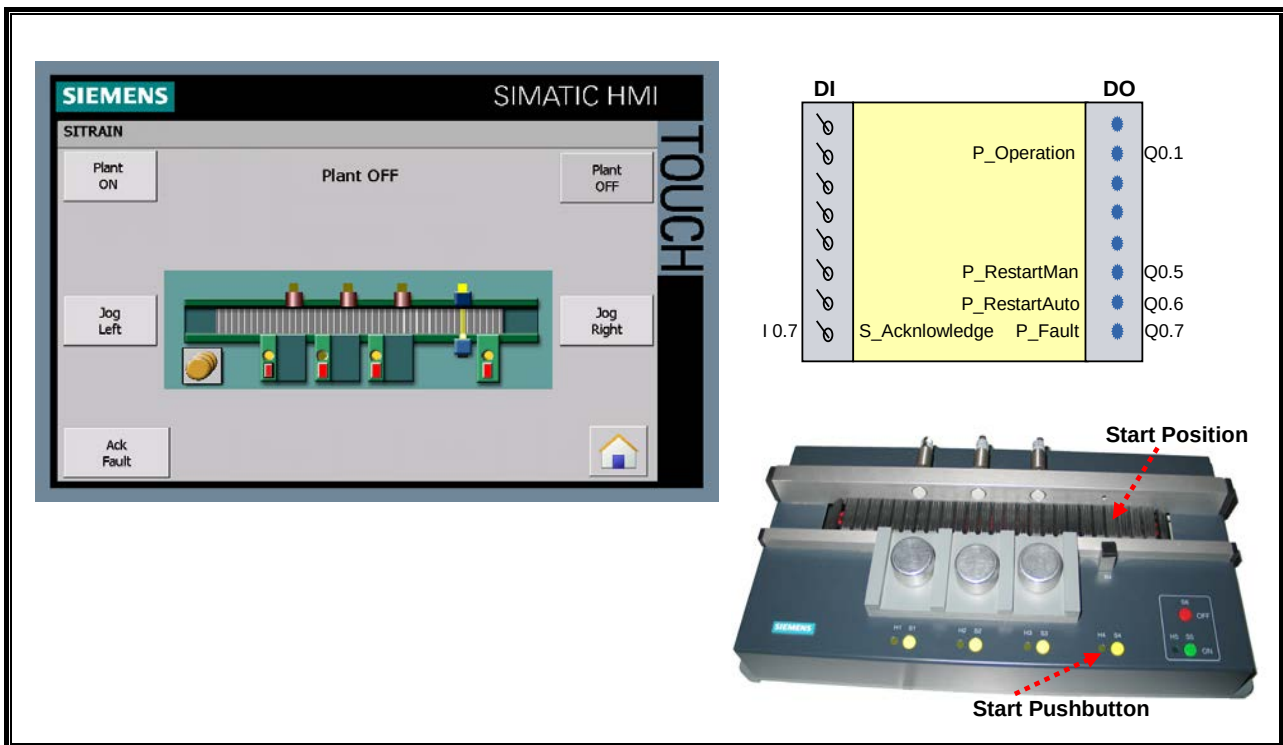
Alternative

The multiple alternative is also known as the multiple choice block. For the given condition, there is not only one but several "yes" branches. An example would be the query of a selected recipe number, whereby several different numbers are allowed. The possible values are each displayed in the header over the branch that they represent.

Note

Structograms have a strict processing sequence, from top to bottom. This is to prevent the generation of so-called "spaghetti code" as it frequently occurred in earlier basic interpreters.

3.8. Plant Description: The Conveyor Model as Distribution Conveyor



Conveyor Model as Distribution Conveyor

- **Job Mode**

When "P_Operation" (Q0.1) is switched off, the conveyor can be jogged in the relevant direction using the buttons "Jog Right" and "Jog Left".

- **Automatic Mode**

When "P_Operation" (Q0.1) is switched on, the indicator light at the light barrier bay shows with a continuous light that a new part may be placed in the light barrier (Start Position), if ...

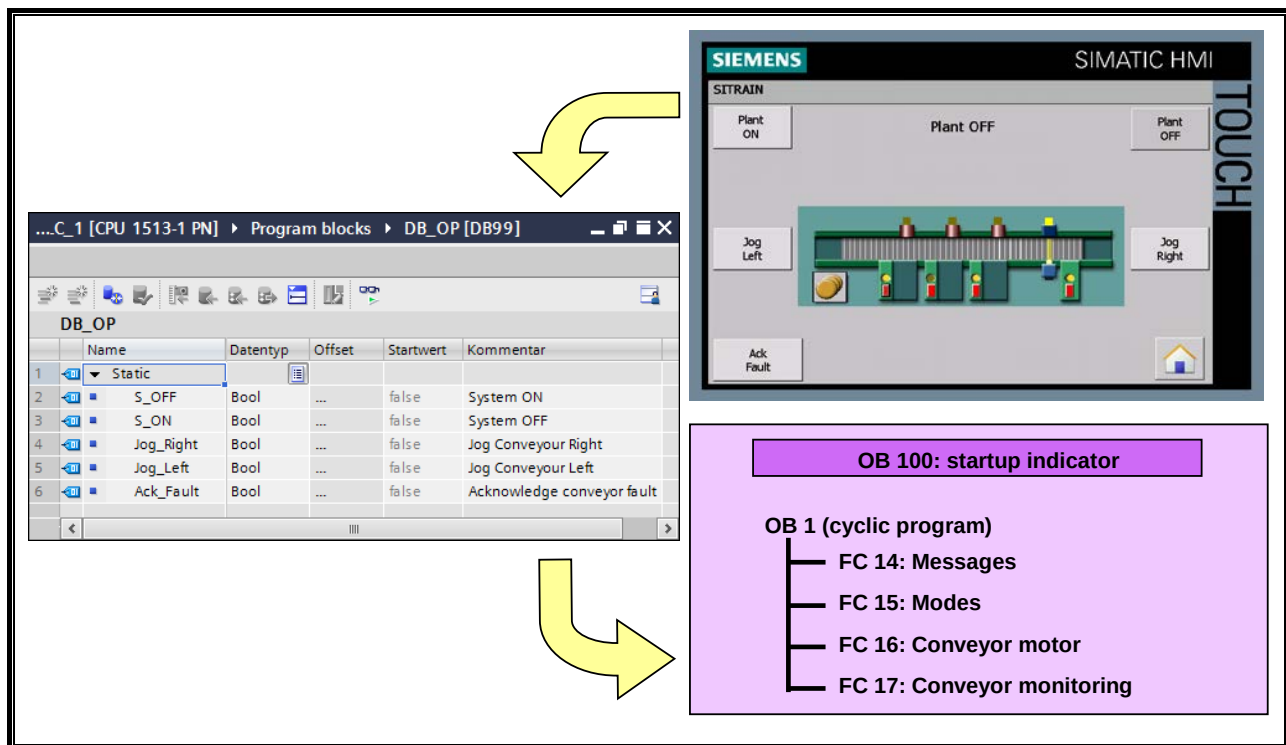
- ... at least one of the Bays 1 to 3 (STOP Positions) is free (= at least one of the bay indicator lights is dark)
- ... no conveyor fault exists (no indication at LED "P_Fault" or on the TP)

Transport is started by pressing the pushbutton at the light barrier bay. As soon as the part reaches a free bay, the conveyor motor is stopped.

A 2Hz flashing light from the bay indicator light indicates that the part can be removed from the conveyor. The bay is still considered to be occupied even though the part has been removed, which is indicated by a continuous light on the bay indicator light. Only after acknowledging with the bay pushbutton, is the bay "enabled" once more.

Every transport sequence is monitored for time. If it takes longer than 6 seconds, there is a fault and the conveyor motor is automatically switched off. Only after the fault is acknowledged via "S_Acknowledge" (I 0.7) or on the touchpanel, can a new transport sequence be started.

3.8.1. Structure of the CPU Program and Interface to the Touchpanel



Top-Down Principle

The previously described entire task is broken down into partial tasks. In the picture you can see a possible subdivision of the entire task into partial tasks. In this rough structure, every partial task can - depending on the complexity - be divided into further subordinate partial tasks or subfunctions.

Black-Box Principle

The individual partial functions are then programmed in separate blocks that implement their partial tasks as independently as possible from other blocks.

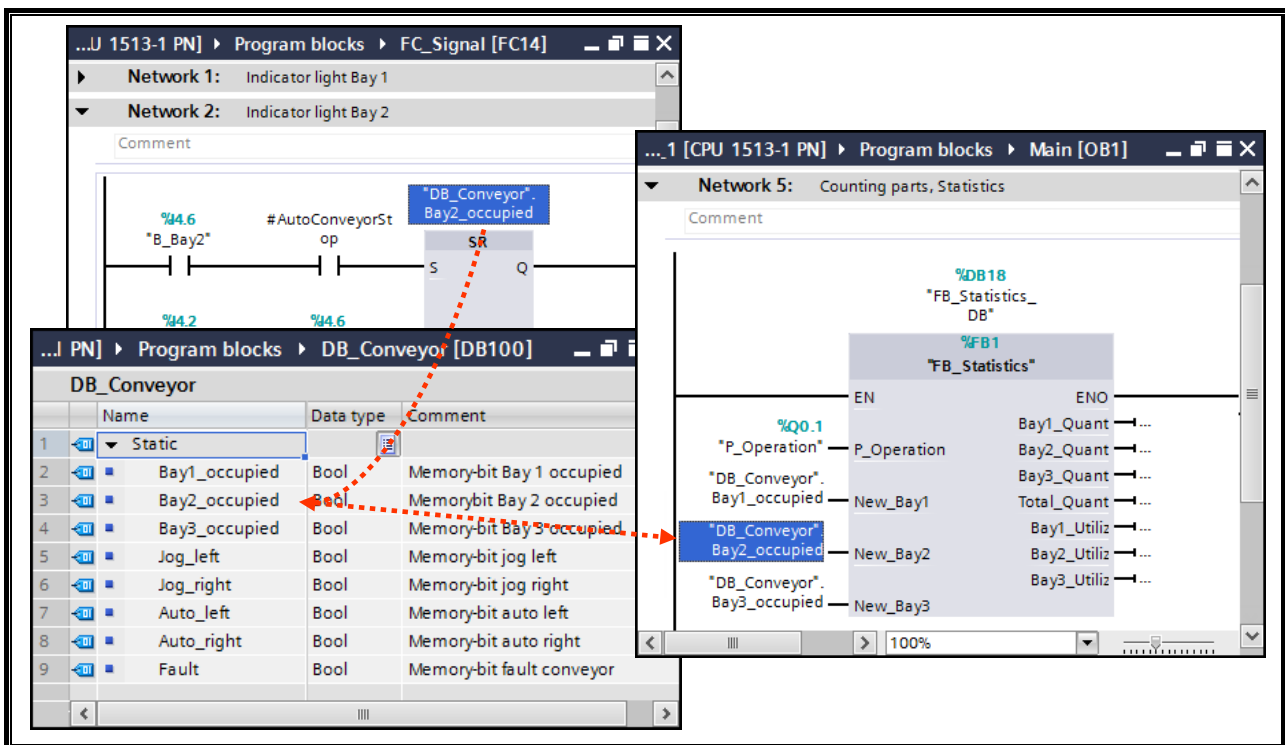
Touchpanel Program

The program for the touchpanel is already created and only has to be commissioned by you. In the creation of the CPU program, you must, however, take into consideration the interface, that is, the interface in the "DB_OP" (DB 99) data block, between the touchpanel and the CPU (see picture).

CPU Program

- **"OB_Startup" (OB100):** Display of the type of the last restart:
"P_RestartMan" (Q0.5) = manual restart, "P_RestartAuto" (Q0.6) = automatic restart
- **"FC_Signal" (FC14):** Control of the conveyor model indicator lights
- **"FC_Mode" (FC15):** "P_Operation" (Q0.1), Acknowledgement of the restart indicator
- **"FC_ConvMotor" (FC 16):** Control of the conveyor motor:
Jog mode: Jog right / left
AUTO: Transports from light barrier bay to Bay 1/2/3
- **"FC_Fault" (FC 17):** Monitoring the automatic transport sequences for time when "P_Operation" (Q0.1) is switched on.

3.8.2. DB Variables instead of Memory Bits



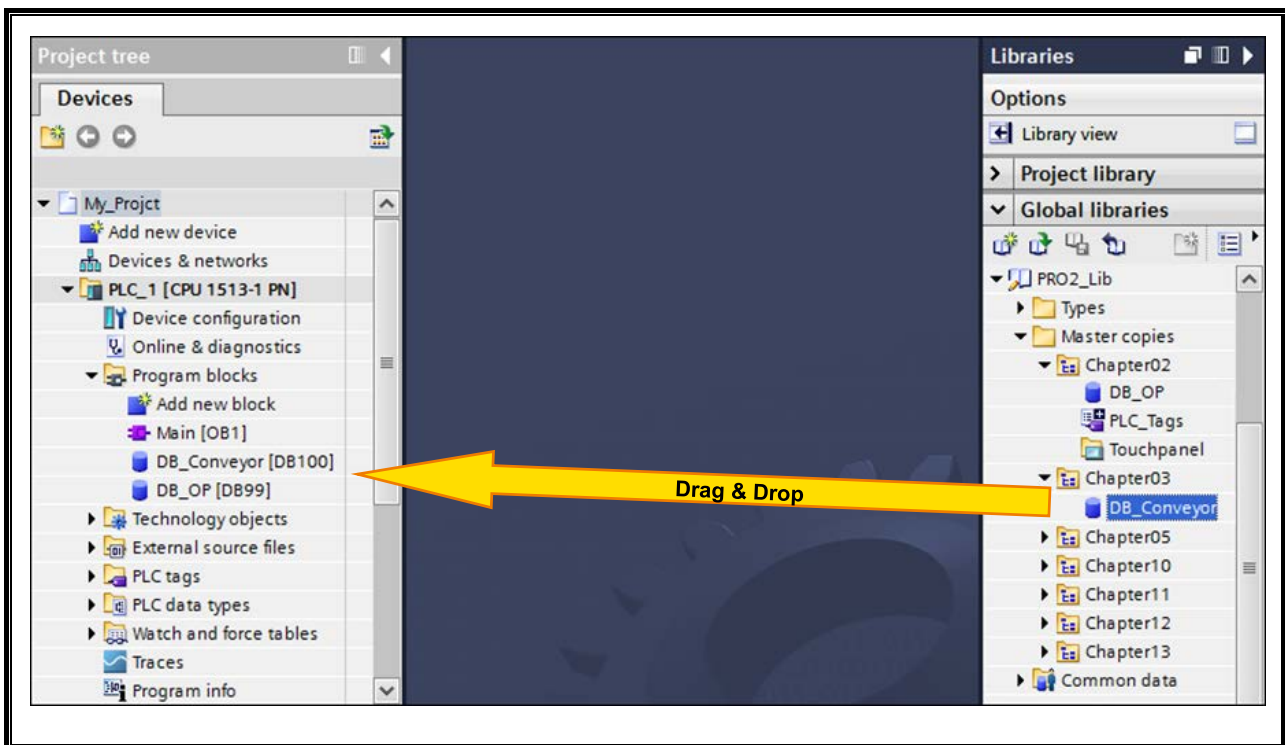
DB Variables instead of Memory Bits

In the traditional programming style memory bits, among other things, were used to 'remember' information in blocks or to pass information from block to block (see picture). The disadvantage of using memory bits is that the user has to organize the management of the memory bits himself and the overlapping of bit, byte, and word accesses frequently leads to errors. As well, making changes to the memory bit structure later on (e.g. by adding additionally required memory bit variables later on) frequently leads to extensive program adjustments.

If, on the other hand, variables in optimized data blocks are used instead of memory bits, this has the following advantages:

- The access to variables in optimized data blocks is more efficient than that to memory bits
- Through consequent symbolic programming, overlapping accesses are avoided
- Adding variables later on (see picture) is not a problem since the entire program is automatically adjusted during compiling.

3.8.3. Exercise 1: Copying "DB_Conveyor" from the Library



Task

"DB_Conveyor" (DB100)

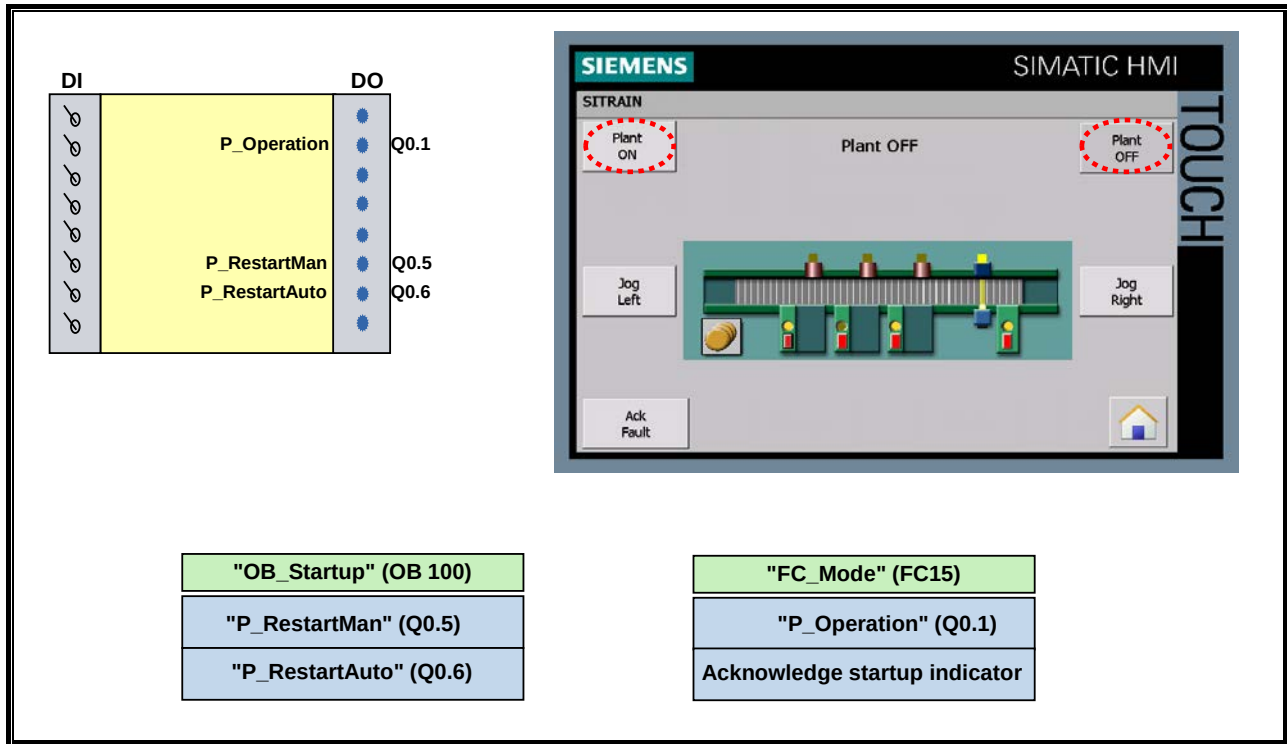
For the reasons described previously, only data block variables are to be used in the S7 program instead of memory bits. If it should come to be, during the course of programming, that further "memory bits" or data block variables than those shown in the picture are required, the data block can be expanded at any time.

The "DB_Conveyor" (DB100) data block has already been created and can be copied from the global library "PRO2_Lib" into your own project.

What to Do

1. Switch to the Project view.
2. In the "Global libraries", open the library <Drive>:\Archives\TIA\PRO2_Lib.
3. Using drag & drop, copy the object "DB_Conveyor" from the global library "PRO2_Lib" into your own project in the container "Program blocks".
4. Save your project.

3.8.4. Exercise 2: Startup Program "OB_Startup" (OB100) and Operating Mode Section "FC_Mode" (FC15)



Task

Function (OB100) Startup Program

A manual restart (CPU STOP->RUN) is indicated on the simulator LED "P_RestartMan" (Q0.5); an automatic restart (power recovery) on the simulator LED "P_RestartAuto" (Q0.6).

Function (FC15)

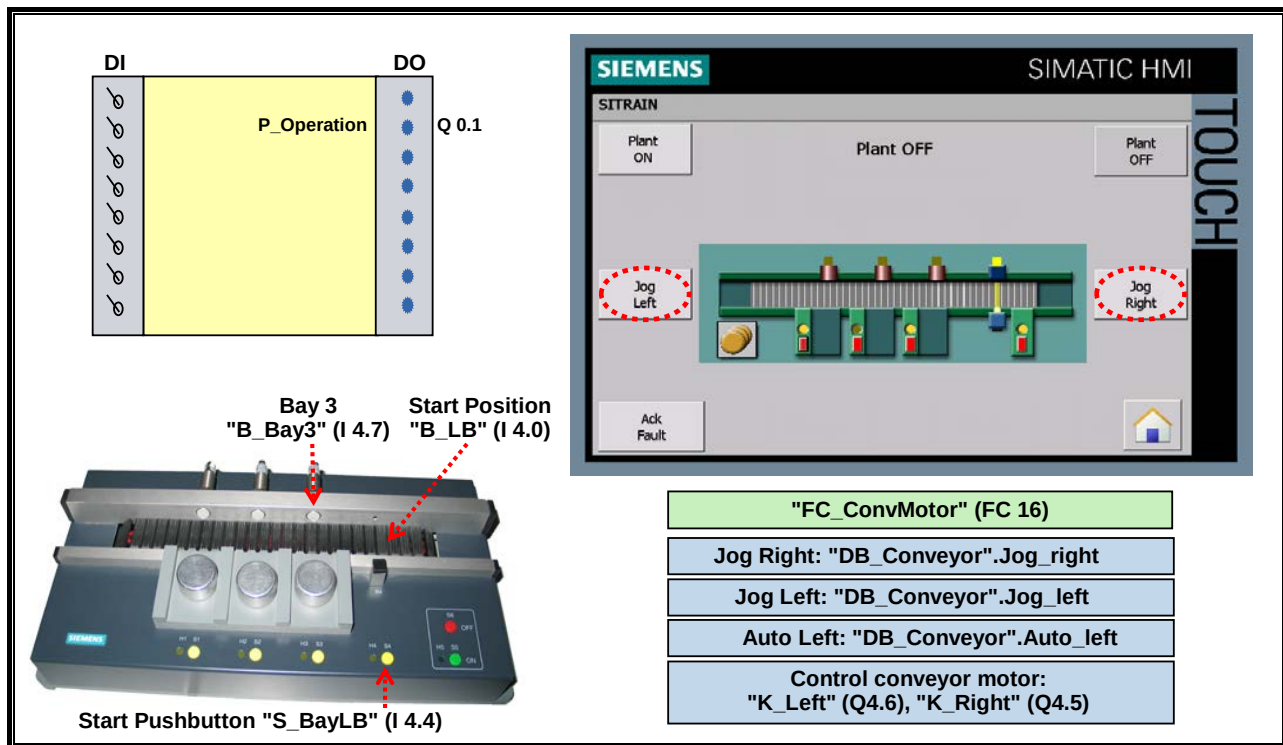
"P_Operation" (Q0.1) is switched on via the touchpanel button "Plant ON" (DB_OP.Operation_ON) and is switched off via the button "Plant OFF" (DB_OP.Operation_OFF).

When "P_Operation" (Q0.1) is switched on, the startup indicators "P_RestartMan" (Q0.5) and "P_RestartAuto" (Q0.6) are automatically acknowledged or switched off.

What to Do

1. Write a program for the "OB_Startup" (OB100) block with the required functions.
2. Write a program for the "FC_Mode" (FC15) block with the required functions.
3. Check whether the operating modes can be activated via the touchpanel as required and whether they are displayed on the touchpanel as well as indicated on the simulator.
4. Save your project.

3.8.5. Exercise 3: Programming the Conveyor Motor Control "FC_ConvMotor" (FC16)



Task: "FC_ConvMotor" (FC16)

The control of the conveyor motor is to be programmed in "FC_ConvMotor" (FC16) and is to function as follows:

- "P_Operation" (Q0.1) switched off:
When "P_Operation" (Q0.1) is switched off, the conveyor motor can be moved in the relevant direction using the buttons "Jog Right" (DB_OP.Jog_Right) and "Jog Left" (DB_OP.Jog_Left).
- "P_Operation" (Q0.1) switched on:
An automatic transport sequence works as follows:
A transport sequence is started by placing a part in the light barrier and pressing the Start pushbutton "S_BayLB" (I 4.4). The transport sequence is stopped as soon as the part reaches the first free bay (in the current program status, always Bay 3, see Note).

What to Do

Write a program for the "FC_ConvMotor" (FC 16) block with the required functions and check

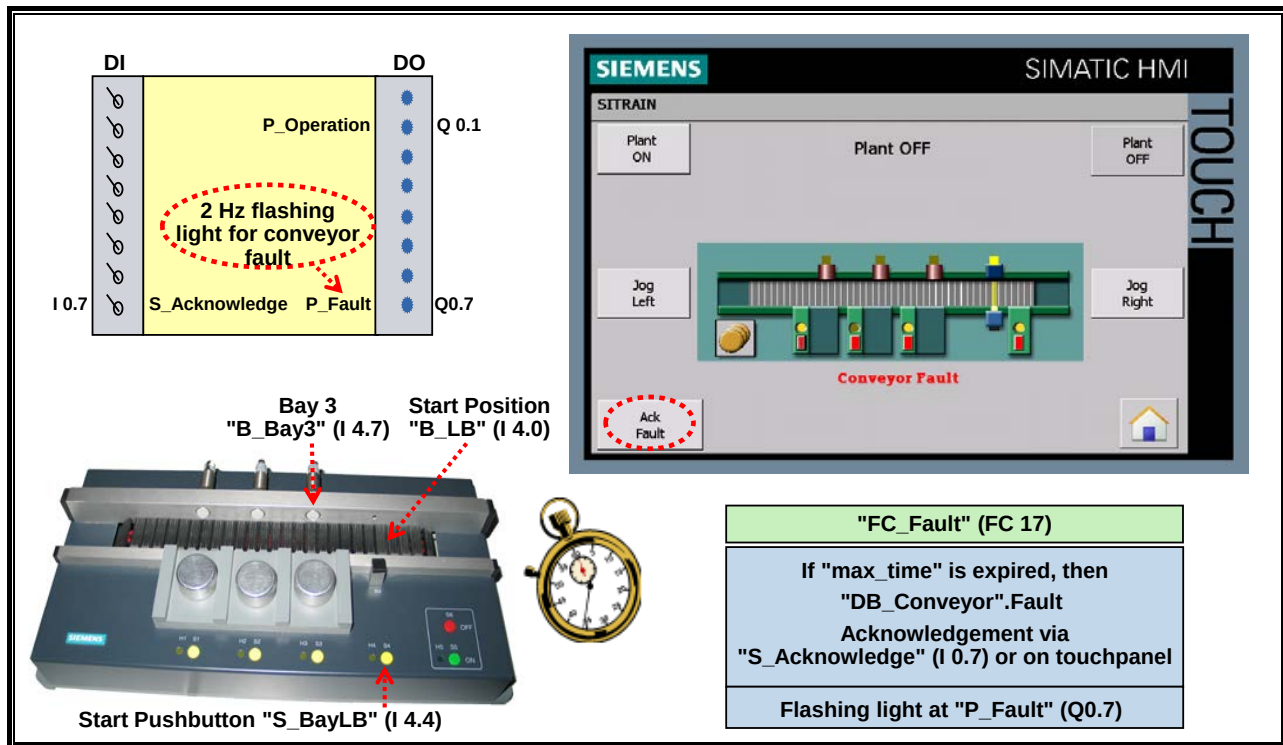
- whether the conveyor motor can be jogged to the right and the left using the touchpanel when "P_Operation" (Q0.1) is switched off.
- and whether parts are automatically transported from the light barrier bay to Bay 3 when "P_Operation" (Q0.1) is switched on.

Note

An automatic transport sequence ends when a part reaches the first "free" bay. The conveyor is stopped, the bay's indicator light is switched on and the bay is marked as "occupied". Only after the part has been removed and it is acknowledged by pressing the associated bay pushbutton, is the indicator light switched off and the bay is "enabled" once again.

Since the "FC_Signal" (FC 14) block for controlling the indicator lights and for "occupying" and "enabling" the bays has not yet been programmed, the first free bay is currently always Bay 3, so that every transport sequence ends at this Bay 3.

3.8.6. Exercise 4: Programming the Time Monitoring of the Transport Sequences "FC_Fault" (FC17)



Task

Time Monitoring of the Transport Sequences "FC_Fault" (FC17)

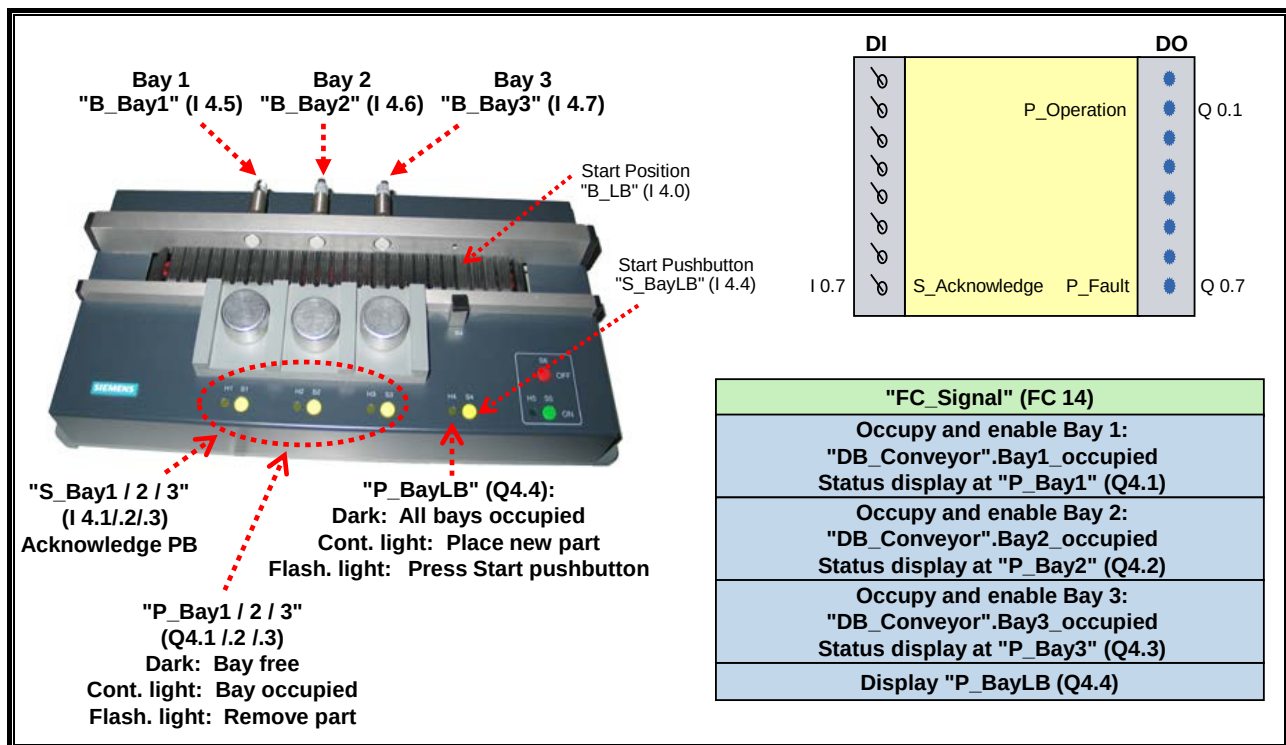
When "P_Operation" (Q0.1) is switched on, the transport sequences are monitored. If a transport sequence takes longer than 6 seconds, the conveyor motor is switched off and the fault is indicated on the simulator LED "P_Fault" (Q0.7) with a flashing light and on the touchpanel in the screen "Conveyor" with a text message.

Only after the fault has been acknowledged via the touchpanel button "Ack Fault" ("DB_OP".Ack_Fault) or on the simulator via the switch "S_Acknowledge" (I 0.7) can a new transport sequence be started.

What to Do

1. In the PLC tag table "My_Tags", declare the user constant max_time := 6s [data type TIME] and use this for the subsequent programming of the "FC_Fault" (FC17) block.
2. Write a program for the "FC_Fault" (FC 17) block with the required functions. In case of a conveyor fault, the block sets the DB variable "DB_Conveyor".Fault = '1' and controls the simulator LED "P_Fault" (Q0.7) with 2Hz flashing light.
3. Expand the "FC_ConvMotor" (FC 16) block so that the conveyor motor is switched off when there is a fault ("DB_Conveyor".Fault = '1').
4. Check whether a conveyor fault is displayed on the simulator as well as on the touchpanel and that the fault can be acknowledged.
5. Save your project.

3.8.7. Exercise 5: Programming the Indicator Lights "FC_Signal" (FC14)



Task

Controlling the Indicator Lights "FC_Signal" (FC14)

When "P_Operation" (Q0.1) is switched on, the parts are transported from the light barrier bay to the first free bay. The bay indicator lights are controlled as follows:

- "P_Bay1 / 2 / 3" (Q4.1 / .2 / .3)
When the part reaches the first free bay, the conveyor stops and the indicator light of the associated bay indicates with a 2Hz flashing light that the part can be removed from the conveyor. After the part has been removed, the bay is still considered to be "occupied" which is indicated by a continuous light on the bay indicator light. Only after acknowledging with the associated bay pushbutton, is the bay "enabled" once again. ("DB_Conveyor".BayX set to 0).
- "P_BayLB" (Q4.4)
A continuous light on the indicator light "P_BayLB" indicates that a new part can be placed in the light barrier if there is no longer a part on the conveyor and at least one of the Bays 1 to 3 is free ("DB_Conveyor".Bay1 / 2 / 3 = 0). If a part is placed in the light barrier and the conveyor has not yet been started, the indicator light flashes indicating that the conveyor can be started. If all bays are occupied, the indicator light "P_BayLB" (Q4.4) remains dark and no further transport sequence can be started.

Solution Hints

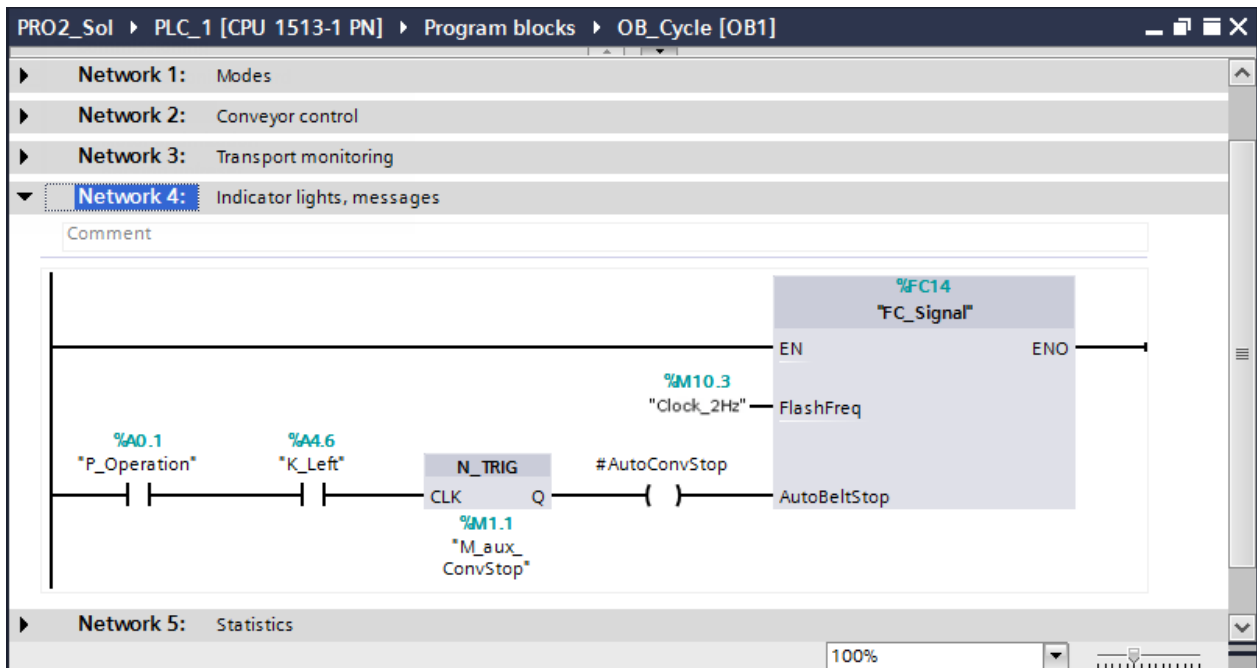
On the conveyor model as well as on the touchpanel, occupied bays are indicated by a switched-on bay indicator light. Within the program, each bay is assigned a data block variable "DB_Conveyor".Bay1 / 2 / 3 (see picture) that is assigned the status '1' when the bay is occupied. These tags (variables) must be managed by the "FC_Signal" (FC 14) block now to be programmed (set at conveyor stop and reset when acknowledged) and accordingly interlocked in the "FC_ConvMotor" (FC 16) block.

Continued on the next page

What to Do

1. Write a program for the "FC_Signal" (FC 14) block with the required functions. To avoid the use of memory bits within the block, provide the block with the IN parameters #FlashFreq and #AutoBeltStop which you will later assign as follows when "FC_Signal" (FC14) is called in OB1 (see picture below):
 - For the required flashing of the simulator indicator light "P_Fault" (Q0.7), assign the flashing frequency memory bit "Clock_2Hz" (M10.3) to the IN parameter #FlashFreq.
 - So that no edge evaluation has to be programmed within the block for the Belt-Stop-edge required for the program, program this in OB1 and pass its result to the temporary variable #AutoConvStop (must be declared and will be required later on) and to the IN parameter #AutoBeltStop.
2. Expand the "FC_ConvMotor" (FC 16) block so that the conveyor can only be started when at least one of the Bays 1 to 3 is "free".
3. Check the required functions.
4. Save your project.

Call of "FC_Signal" (FC14) in "OB_Cycle" (OB1)



Contents

4.	Jump and Accumulator Functions	4-2
4.1.	Task Description	4-3
4.2.	Overview of the Accumulator Functions	4-4
4.2.1.	Arithmetic Instructions.....	4-5
4.2.2.	Swapping the Order of the Bytes in ACCU1.....	4-6
4.2.3.	Shift Instructions (ACCU 1).....	4-7
4.2.4.	Word Logic Operations	4-8
4.2.4.1.	Word Logic Operation: Example: Separating Code_Color by Masking.....	4-9
4.3.	Jump Functions.....	4-10
4.3.1.	Jump List.....	4-11
4.3.2.	Jump Distributor	4-12
4.4.	Program Loop in STL.....	4-13
4.4.1.	Program Loop in FBD (LAD).....	4-14
4.5.	Exercise 1: Running Light when Operation ("FB_Running_Light") is Switched Off	4-15

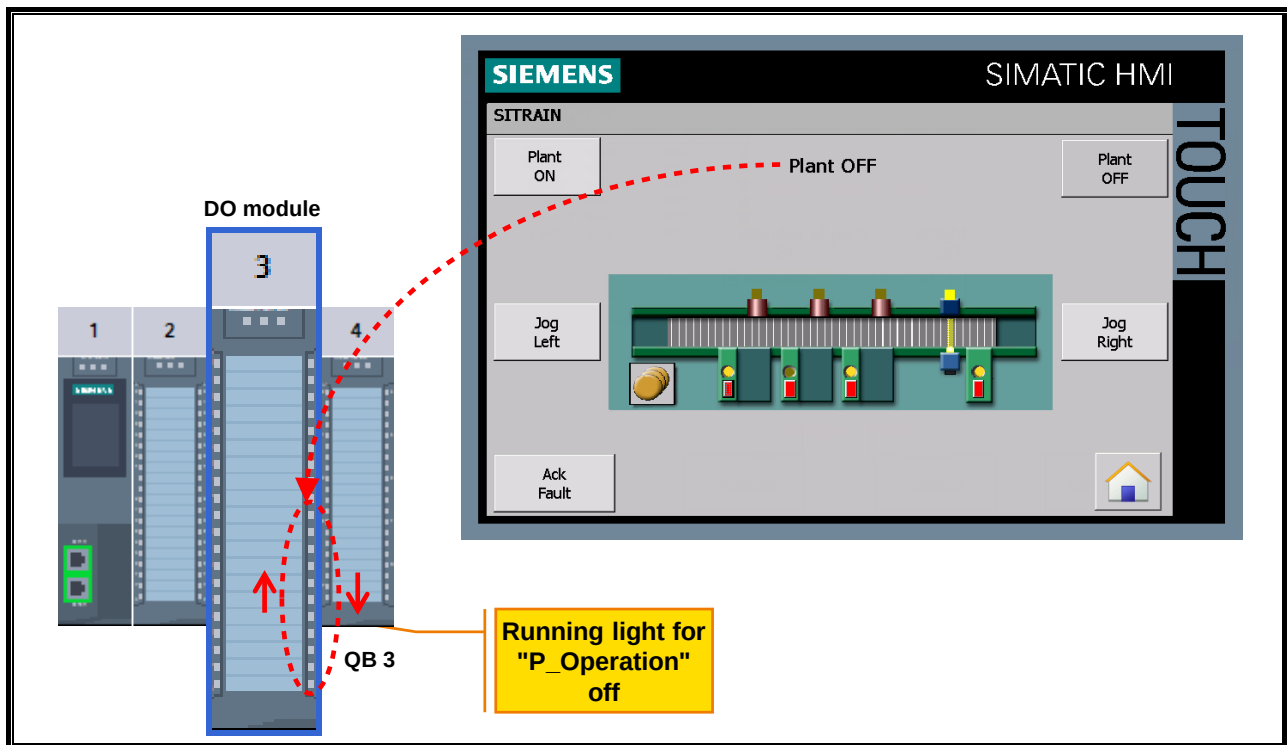
4. Jump and Accumulator Functions

At the end of the chapter the participant will ...

- ... be familiar with selected accumulator functions and will be able to use them
- ... be familiar with selected jump functions and will be able to use them
- ... be familiar with selected word operations and will be able to use them



4.1. Task Description



Task Description

Running light, as long as "P_Operation" (Q0.1) is switched off

As long as "P_Operation" (Q0.1) is switched off, the output byte QB3 of the S7-1500 DO module in the central rack is to show a running light that is pulsed with 2Hz (flashing clock memory bit M 10.3).

4.2. Overview of the Accumulator Functions

Instructions that affect several Accumulators

- TAK: Toggle (swap) the contents of ACCU1 and ACCU2
- Arithmetic instructions and word logic operations

Instructions that only affect ACCU1

- INC: Increment the contents of ACCU1-LL
- DEC: Decrement the contents of ACCU1-LL
- CAW: Reverse the order of the bytes in ACCU1-L
- CAD: Reverse the order of the bytes in ACCU1
- INVI, INVD: Form the ones complement
- NEGI, NEGD, NEGR: Form the twos complement (negation)
- SLW, SLD, SRW, SRD: Shift the contents of ACCU1 to the left or to the right
- RLD, RRD: Rotate the contents of ACCU1 to the left or to the right
- RLDA, RRDA: Rotate the contents of ACCU1 to the left or to the right using CC1

Accumulator Functions

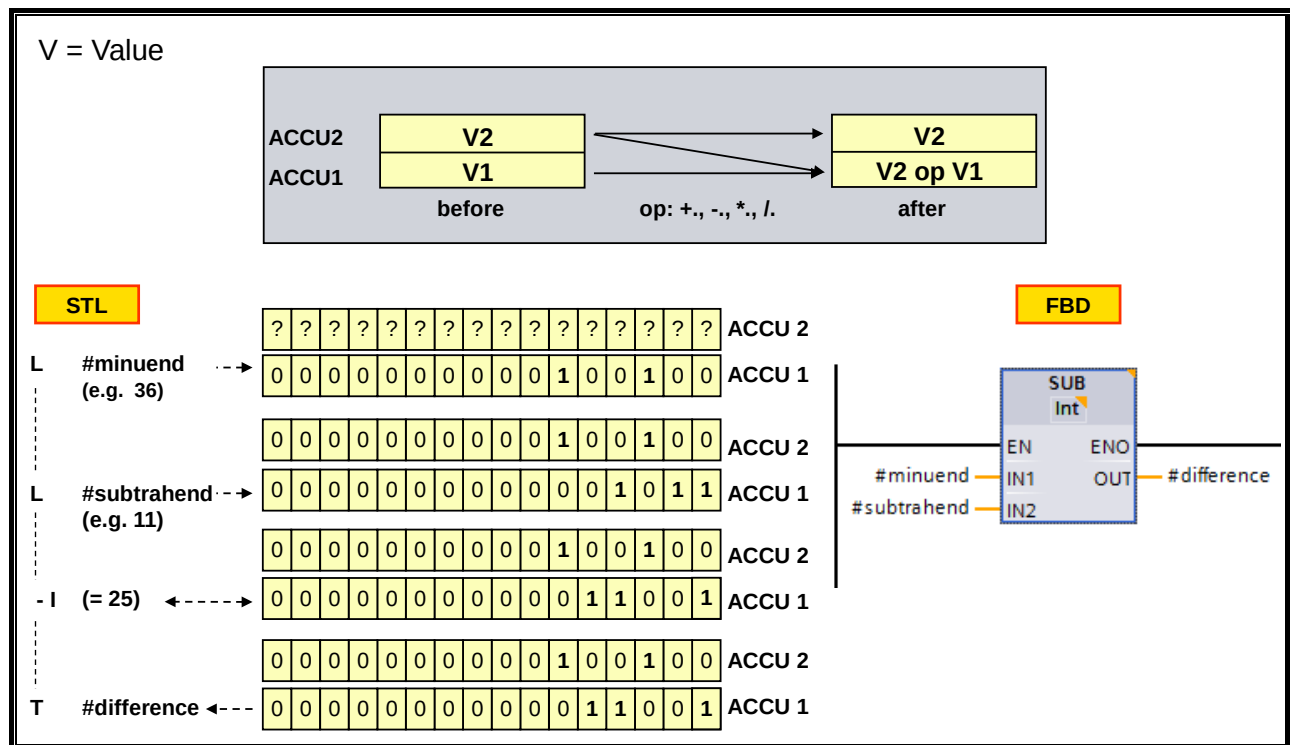
The accumulator functions transmit values between the accumulators or exchange the order of the bytes in ACCU1. The execution of pure accumulator functions is independent of the result of logic operation or of the status bits.

In the STL programming language, the accumulators in the S7-1500 are emulated so that the accumulator functions can be used.

Note

The following pages describe some important instructions. Make use of the Instruction List or the online Help for a complete overview and more information.

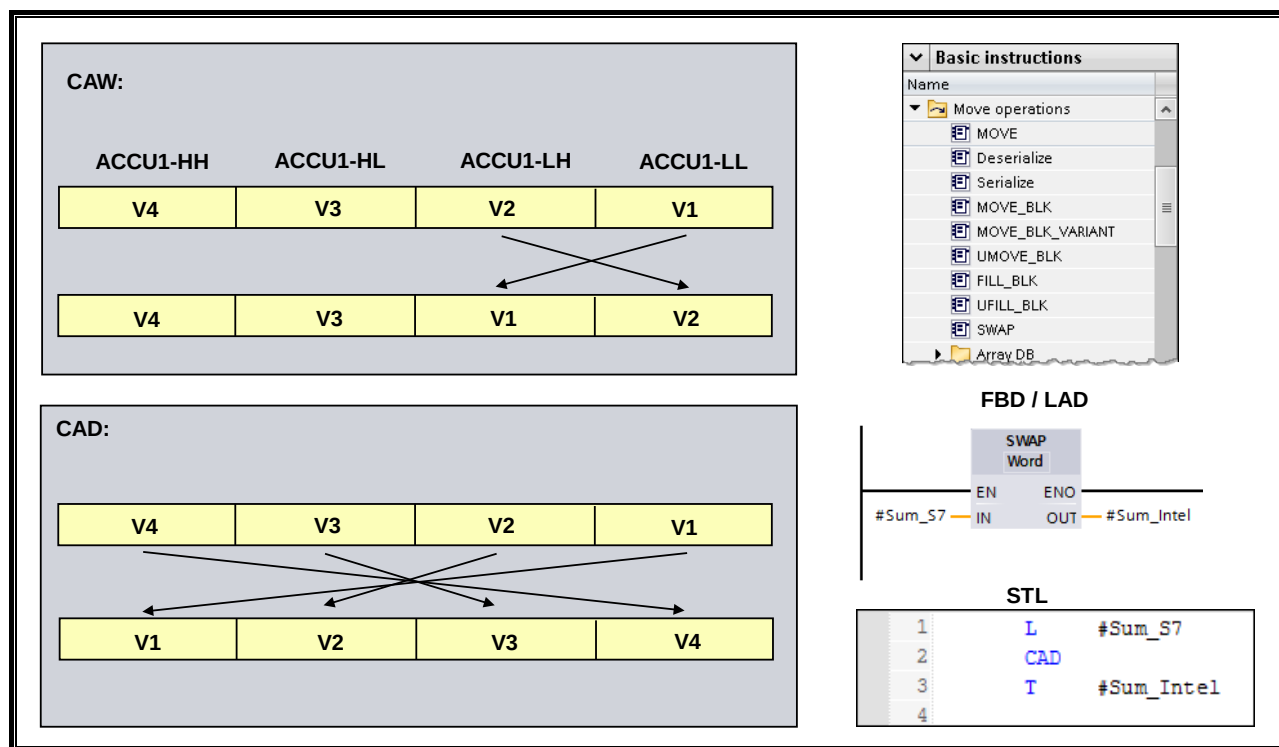
4.2.1. Arithmetic Instructions



Arithmetic Instructions

The arithmetic instructions combine two digital values found in accumulators 1 and 2, according to the fundamental operations of arithmetic. The result of the calculation is found in ACCU1.

4.2.2. Swapping the Order of the Bytes in ACCU1



CAW

With the CAW instruction, the bytes in the right data word of ACCU1 are exchanged. That is, the contents of ACCU1-LH are transferred to ACCU1-LL and vice versa.

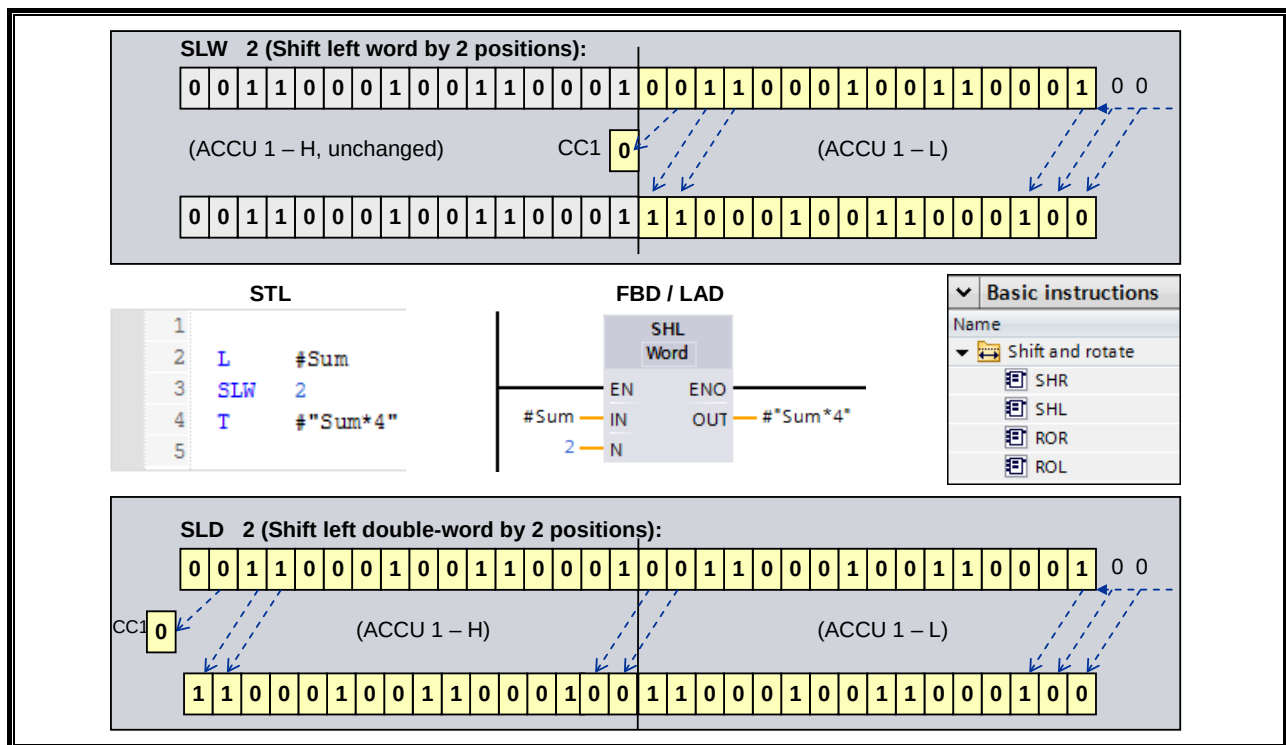
With this instruction it is possible to convert a 16-bit numbers format (INT and WORD) in the SIMATIC programming language to the numbers format of the INTEL programming language (data transfer to PCs).

CAD

With the CAD instruction, the bytes in ACCU1 are exchanged. That is, the contents of ACCU1-HH are transferred to ACCU1-LL and vice versa or the contents of ACCU1-HL to ACCU1-LH and vice versa.

With this instruction it is possible to convert a 32-bit numbers format (DINT, DWORD and REAL) in the SIMATIC programming language to the numbers format of the INTEL programming language (data transfer to PCs).

4.2.3. Shift Instructions (ACCU 1)



SLW <const>, SRW <const>

The instructions **Shift Left Word** and **Shift Right Word** shift the contents of ACCU1-L bit-by-bit to the left or to the right, whereby ACCU1_H remains unchanged. The bit positions that become empty are filled with zeroes. The last bit to be shifted out of ACCU1_L to the left or to the right is loaded into status bit CC1.

The number of bit positions that are to be shifted is determined either as a constant <const> or when no constant is specified by the value in ACCU2-L-L. Values from 0 to 15 are good policy since larger values always result in ACCU1_L = 0.

The status bits CC0 and OV are reset to "0" if <const> is greater than zero. When <const> is equal to "0", the shift instructions have no effect (compare NOP 0).

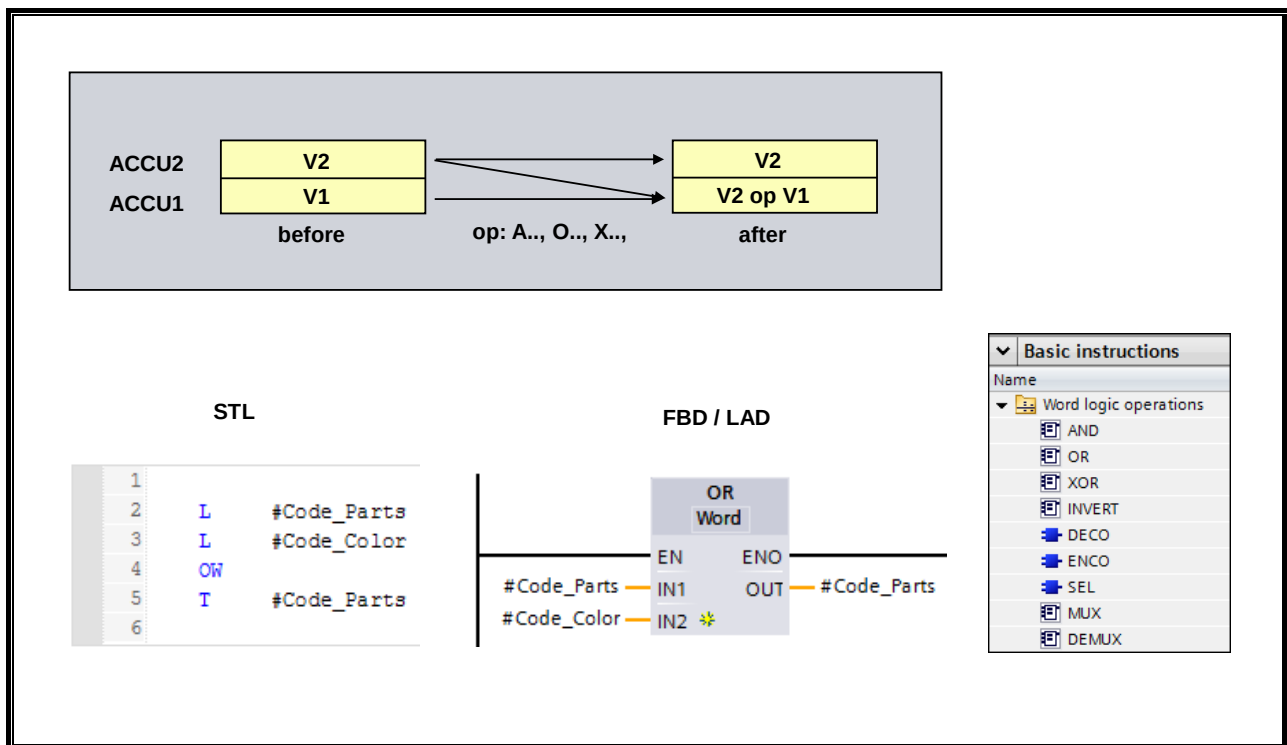
SLD <const>, SRD <const>

The double-word shift instructions function like the comparable word instructions with the difference that the entire ACCU1 contents are shifted. Furthermore, the number of bit positions that are to be shifted assume values from 0 to 31.

Note on the example shown:

The variable #Sum loaded in ACCU 1 is of the data type INT. Since the value of the variable is positive (bit no.15 of the variable is =0), the shifting of the ACCU 1 contents by 2 positions to the left represents a quadruplication. In the example, this quadrupled value is assigned to the variable #Sum*4.

4.2.4. Word Logic Operations



Word Logic Operations

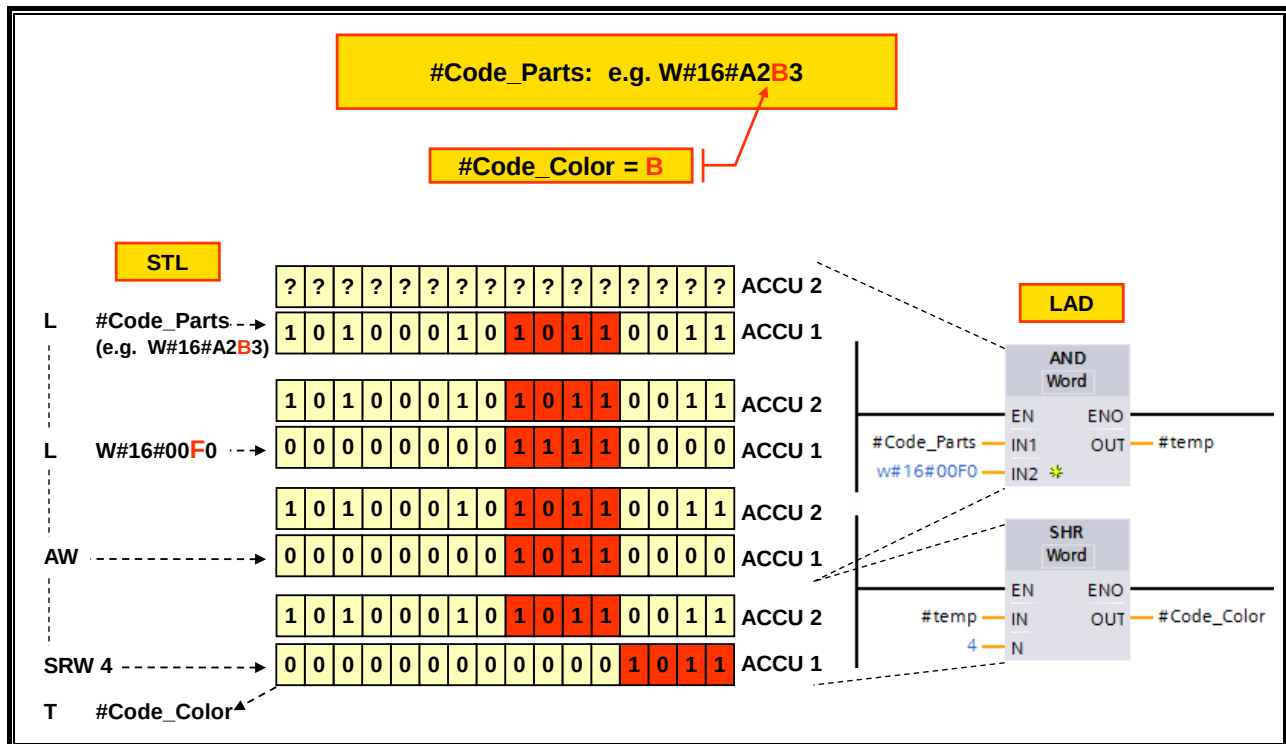
The word logic operations combine bit by bit the values found in ACCU1 with a constant or with the contents of ACCU2 and store the result in ACCU1.

The contents of the remaining ACCUs (ACCU2 for S7-300, or, ACCU2, ACCU3 and ACCU4 for S7-400) remain unchanged. The logic operation can be performed word-by-word or double-word-by-double-word.

The operations AND, OR or Exclusive OR (XOR) are available as word logic operations.

4.2.4.1. Word Logic Operation:

Example: Separating Code_Color by Masking



Problem

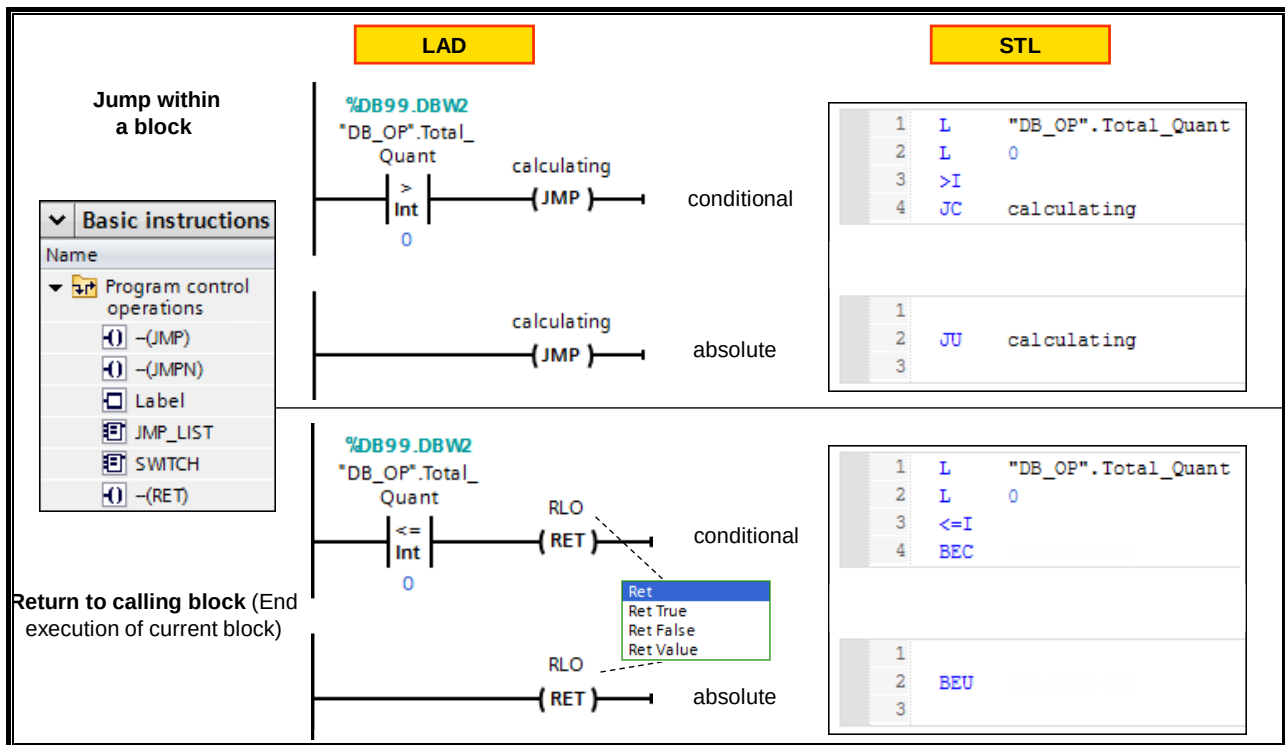
In a hexadecimal 4-digit #Code_Parts, one digit is the color code of the part (in the example =B). The task is now to separate this color code and to assign it to the variable #Code_Color which then, for example, is to be compared to color code constants.

The problem here is that the color code contained in #Code_Parts as well as every other digit of the 4-digit #Code_Parts has a dimension of 4 bits which can be addressed neither symbolically nor absolutely.

Solution:

The example shows how, in the first step, the content of #Code_Color is combined word-by-word according to AND with the constant W#16#00F0, which serves as a so-called "mask" here. Through the structure of the "mask", all bit positions except those of the color code =0 are in the result of this AW operation. The resulting 'lonely' color code must then only be shifted 4 bit positions to the right before it is assigned to the variable #Code_Color.

4.3. Jump Functions



Jump Instructions JMP and JMPN

With the jump instructions JMP (JU or JC) and JMPN (JCN) the linear execution of the program can be interrupted within a block and continued in another network.

With the jump instruction, a Label is specified which also identifies the destination network (destination instruction in STL). The specified label must be located in the same block and be unique.

Each label can be jumped to from several locations. The jump can take place in networks with higher (forwards) or lower numbers (backwards).

Return (End Block Execution) RET

With the instruction RET, the program execution of the entire block is ended. The program execution is then continued in the calling block with the instruction that follows the call of this block.

With the instruction it is specified which status is to be returned to the calling block via its own ENO output parameter:

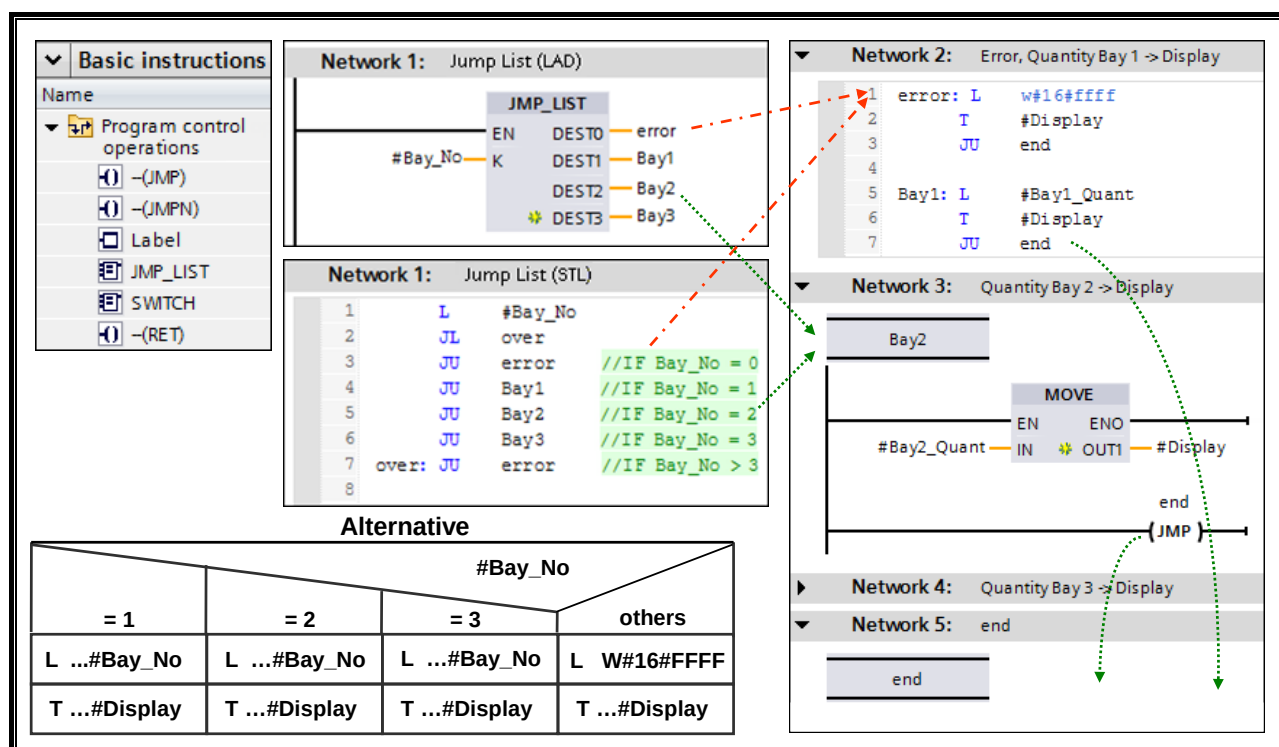
- Ret: the current RLO (in the example the comparison result)
- Ret True: always Status '1'
- Ret False: always Status '0'

Absolute or Conditional Execution of the Instructions:

Absolute means that the instruction (JU, BEU) is executed regardless of the current RLO, which in LAD/FBD is also reflected in the fact that no condition is formulated at the input of the instruction.

Conditional means that the instruction (JC, BEC) is executed only when RLO = '1' (JCN for RLO = '0'). In LAD/FBD, the RLO is formed through the logic operation programmed at the input of the instruction.

4.3.1. Jump List



Jump List

The JL jump list permits the targeted jumping to a program section in the block dependent on a jump number. The JL instruction works with a list of JU jump functions. This list follows immediately after JL and can include a maximum of 256 entries. For JL there is a Label which points to the end of the list, that is, to the first instruction after the list.

Only JU instructions can be placed between the JL <Jump label> and the <Label>: <Instruction>. If "0" is located in the ACCU1-L-L, the first jump instruction is executed, with "1" the second, etc. If the number is larger than the length of the list, JL branches to the end of the list.

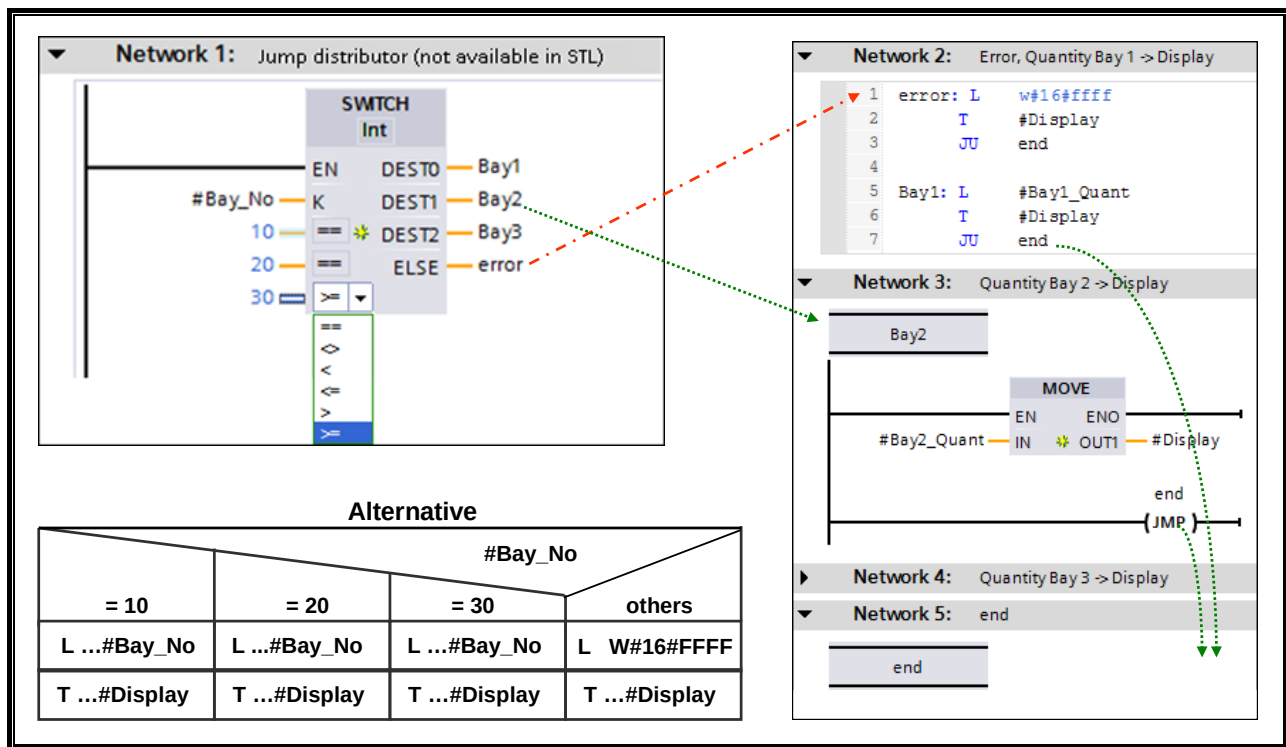
The JL instruction is executed regardless of any conditions.

Note

Jumps can be made forwards as well as backwards. Jumps can only be executed within one block. That is, the jump instruction and the jump destination must lie in the same block.

The jump destination ("Label") must only exist once within this block.

4.3.2. Jump Distributor



Jump Distributor

Just as the JL jump list, the SWITCH jump distributor permits the targeted jumping to a program section in the block dependent on a jump number. For the "jump list", these jump numbers are defined, consecutively beginning with 0. For the jump distributor, the user can define any jump numbers.

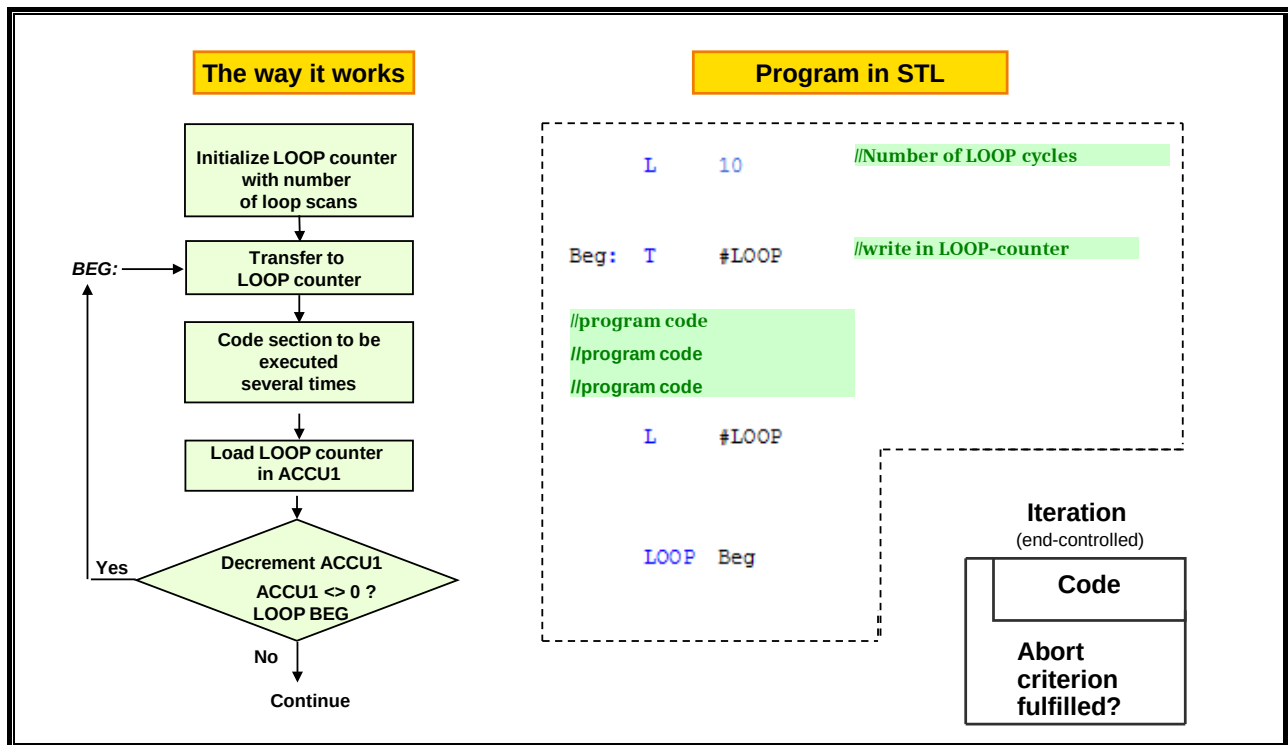
The SWITCH LAD instruction, as such, is not available in STL and must be programmed by the user with discrete instructions.

Note

Jumps can be made forwards as well as backwards. Jumps can only be executed within one block. That is, the jump instruction and the jump destination must lie in the same block.

The jump destination ("Label") must only exist once within this block.

4.4. Program Loop in STL



Loop Instruction

The loop instruction LOOP simplifies the programming of program loops.

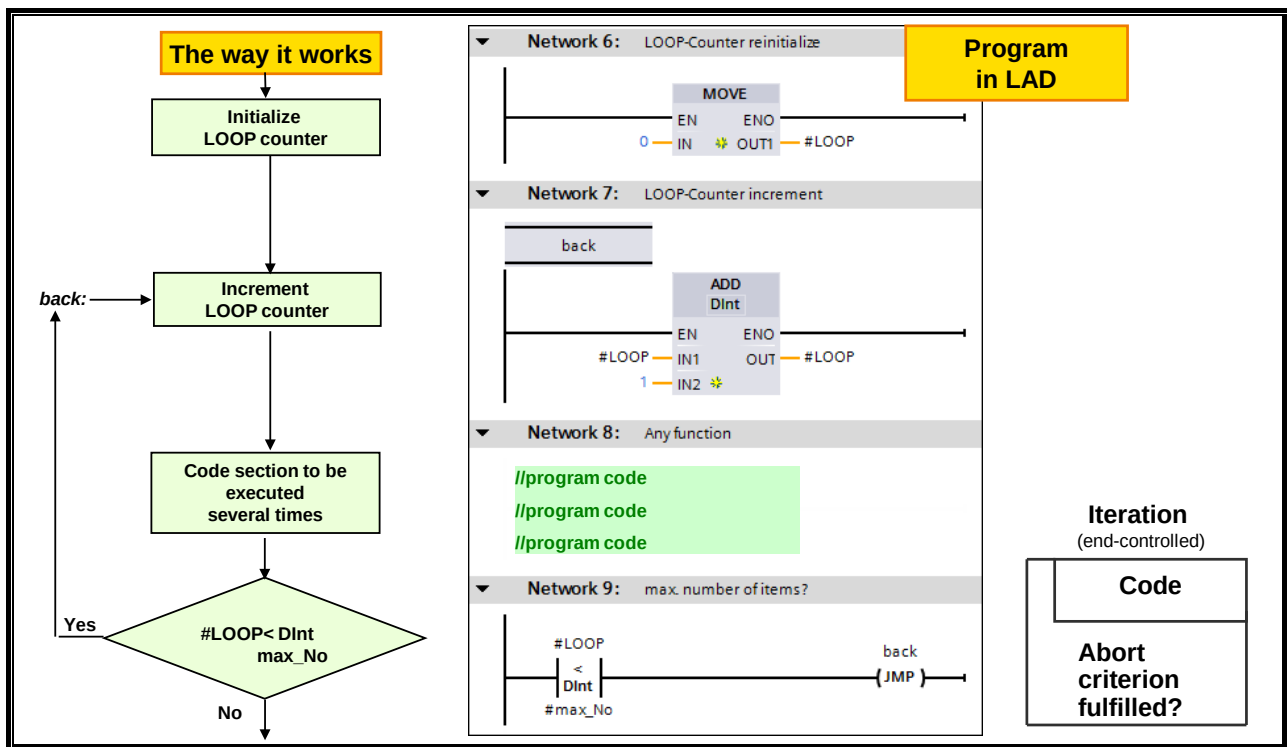
For programming a loop instruction, the desired number of loop scans to be executed is loaded into ACCU1-L. LOOP interprets the right word of Accumulator 1 as unsigned 16-bit number in the range from 0 to 65535.

With every execution of the LOOP instruction, the value in ACCU1-L is decremented by one. Subsequently, the value is compared to zero. If the value is unequal to zero, a jump takes place to the jump label designated in the LOOP instruction. If the value is equal to zero no jump takes place, and instead, the immediately following instruction is executed.

Note

The loop counter must not be initialized with 0, because this would cause the loop to be executed 65535 times.

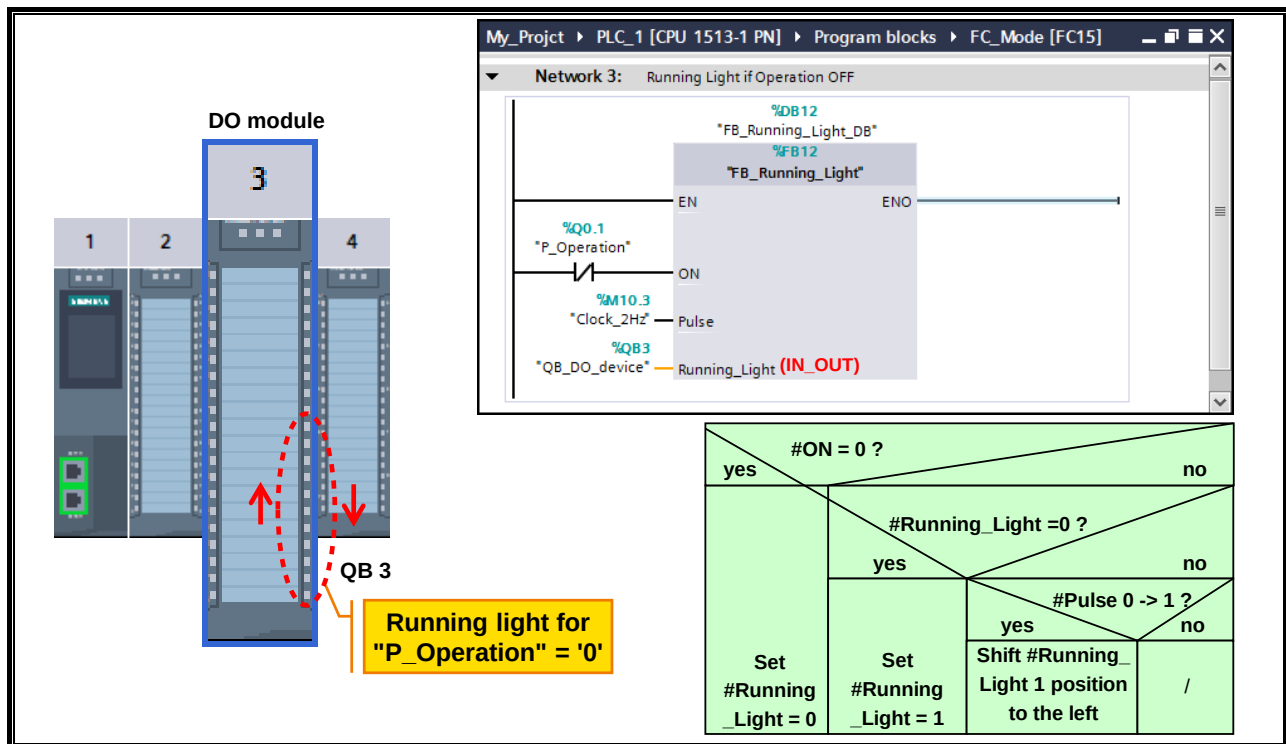
4.4.1. Program Loop in FBD (LAD)



Loop Instruction

The previously shown STL instruction LOOP is explicitly not available in FBD (LAD). However, a program loop is also easily programmable in FBD (LAD) with discrete instructions.

4.5. Exercise 1: Running Light when Operation ("FB_Running_Light") is Switched Off



Function Up Until Now

The type of startup (manual or automatic restart) is indicated by an LED on the simulator. When the system ("P_Operation", Q 0.1) is switched on the display is acknowledged.

Task

As long "P_Operation" (Q0.1) is not switched on, the output byte "QB_DO_Device" (QB3) of the central DO module of the S7-1500 is to show a running light that is pulsed with 2Hz (flashing clock memory bit "Clock_2Hz" (M10.3)).

The running light is to fulfill the following conditions:

- During switch on (input parameter #ON = '1') the running light always starts from Bit No. 0 (Q3.0).
- During switch off (input parameter #ON = '0') all bits are set to 0.

What to Do

1. Insert the new block "FB_Running_Light" (FB 12) into your S7 program and in FB 12 declare the IN-Parameters #ON (data type BOOL) and #Pulse (data type BOOL) as well as the IN_OUT-Parameter #Running_Light (data type BYTE).
2. In "FB_Running_Light" (FB12), program the above-described function using the Shift Left Word instruction (SLW x).
3. Program the call of "FB_Running_Light" (FB12) in a new network of "FC_Mode" (FC15) (see picture).
4. Download all modified blocks into the CPU and check the functioning of the running light.

Solution Hint

An edge evaluation is required (block) internally for the signals passed at the IN-Parameters #ON and #Pulse. Use a local static variable for this.

Contents

5.	Analog Value Processing & Arithmetic	5-2
5.1.	Task Description: Checking the Weight and Statistic Values	5-3
5.2.	Principle of Analog Value Processing	5-4
5.3.	Analog Input and Output Modules	5-5
5.3.1.	Analog Input Modules	5-6
5.3.2.	Analog Output Modules	5-7
5.4.	Analog Value Representation and Measured Value Resolution.....	5-8
5.4.1.	Analog Value Representation of Different Measuring Ranges	5-9
5.4.2.	Analog Value Representation for the Analog Outputs	5-10
5.5.	Analog Value Processing: Direct I/O Access with ":P"	5-11
5.5.1.	Scaling Analog Input Values with SCALE.....	5-12
5.5.2.	Unscaling Analog Output Values with UNSCALE.....	5-13
5.5.3.	Scaling with NORM_X and SCALE_X	5-14
5.5.4.	Math Functions: MIN, MAX, LIMIT	5-15
5.5.5.	Verification with IN_RANGE, OUT_RANGE, OK.....	5-16
5.6.	Task Description: Checking the Weight and Displaying It	5-17
5.6.1.	Exercise 1: Setting the Channel Parameters of the AI Modules (ET200SP and Central AI).....	5-18
5.6.2.	Exercise 2: Checking the Weight with Cyclic Interrupt "OB_Cyclic interrupt" (OB35)	5-19
5.6.3.	Exercise 3: Displaying the Part Weight on the Touchpanel.....	5-20
5.7.	Task Description: Calculating and Displaying the Statistic Values.....	5-21
5.8.	Data Storage in Accumulator 1	5-22
5.8.1.	Loading and Transferring Data	5-23
5.8.2.	Basic Mathematical Functions: Addition	5-24
5.8.3.	Meaning of EN, ENO and BR Bit	5-25
5.8.4.	Basic Mathematical Functions: Comparator Operations	5-26
5.8.5.	Conversion Operations: Data Type Conversion	5-27
5.8.6.	Implicit Data Type Conversion	5-28
5.8.7.	LAD/FBD: CALCULATE Box	5-29
5.8.8.	Sample Calculation: Utilization Bay 1	5-30
5.8.9.	Exercise 4: Copying the TP Screen "Statistic" from the Library	5-31
5.8.10.	Exercise 5: Statistics Data "FB_Statistics" (FB18)	5-32

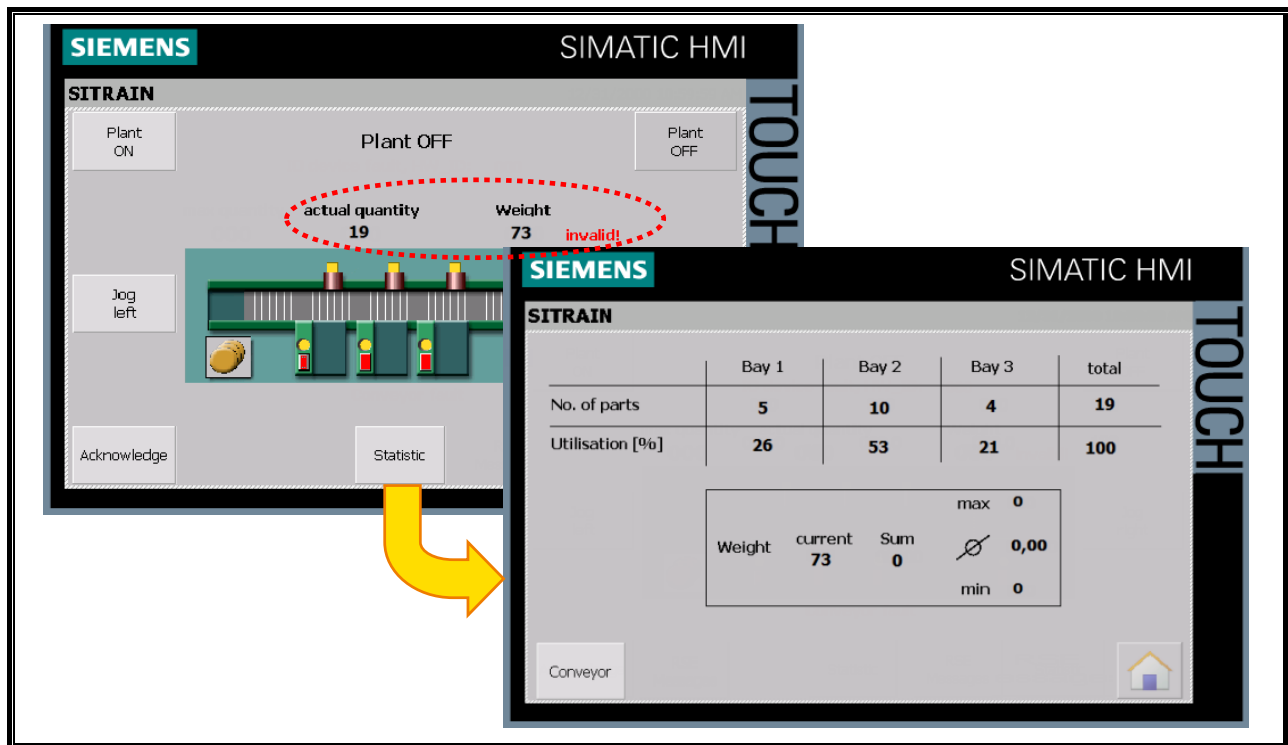
5. Analog Value Processing & Arithmetic

At the end of the chapter the participant will ...

- ... be familiar with the principle of analog value processing
- ... be able to assign parameters to an analog module
- ... be able to address an analog module
- ... be able to interpret the resolution of a module
- ... be familiar with arithmetic functions and be able to use them



5.1. Task Description: Checking the Weight and Statistic Values



Task Description

Checking the Weight:

The parts to be transported are weighed in the Start Position (light barrier) before the conveyor is started. The part weight (0 to 500kg) is simulated via the simulator potentiometer "S_SliderPot1" (IW10) and displayed on the touchpanel screen "Conveyor" at the output field "Weight".

The permitted range of the part weight is **100kg ≤ Weight ≤ 400kg**

As long as an invalid weight (weight < 100kg or weight > 400kg) is set ...

- ... the simulator indicator light "P_WeightFault" (Q 0.0) shows a 2Hz flashing light.
- ... the indicator light on the light barrier bay is dark because no part with an invalid weight may be transported
- ... no transport sequence can be started

Statistic Values:

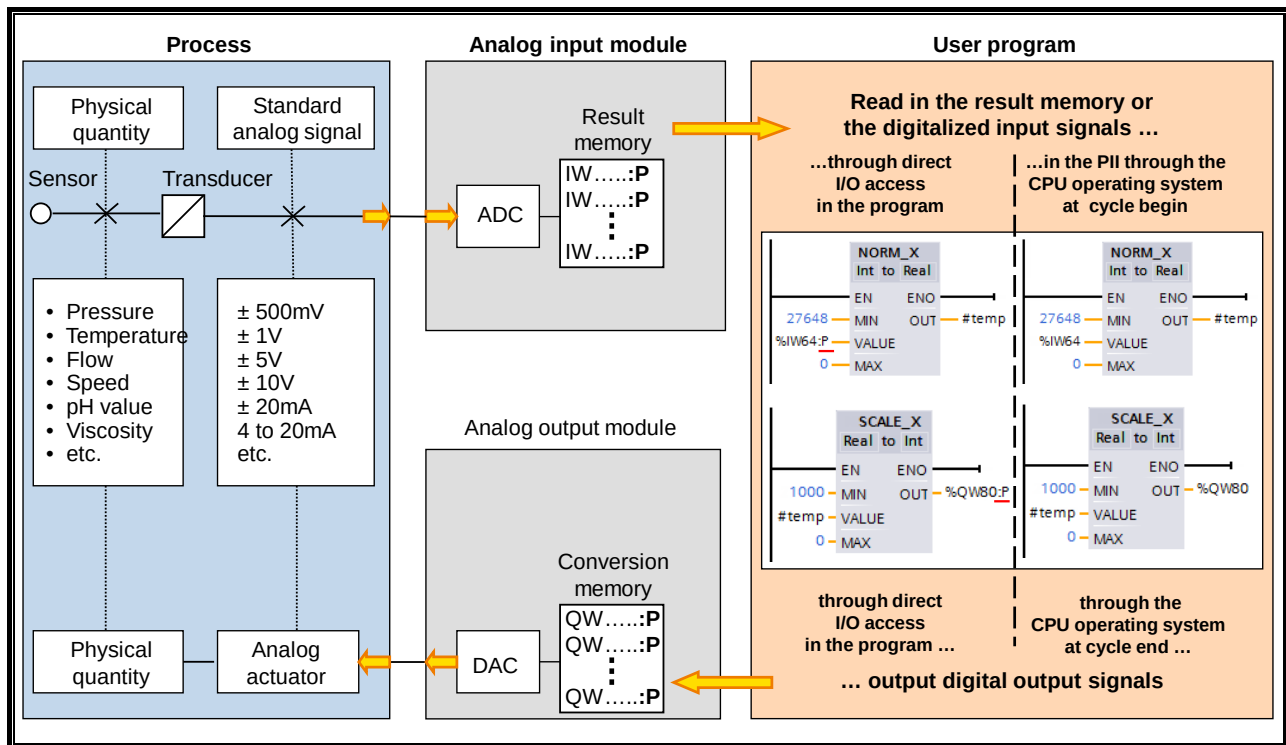
All transported parts are counted when they reach the next free bay. Both the number of parts of the individual bays and the total number of parts are recorded. In addition, the utilization of the bays is determined, that is, which percentage of transported parts has been transported to Bay 1, 2, and 3 is calculated.

All statistic values are displayed on the touchpanel in the "Statistic" screen.

Note:

The weight statistic values shown in the picture are calculated in a later chapter.

5.2. Principle of Analog Value Processing



Principle of Analog Value Processing

In a production process, there are a variety of physical quantities (such as pressure, temperature, speed, rotational speed, pH value, and viscosity etc.) that need to be processed in the PLC for automation purposes.

Sensor

Measuring sensors respond to changes in the quantity to be measured by such things as linear expansion, angular ductability, and alteration of electrical conductivity.

Transducer

Measuring transducers convert these above-mentioned changes into standard analog signals, such as: $\pm 500\text{mV}$, $\pm 10\text{V}$, $\pm 20\text{mA}$, 4 to 20mA. These signals are supplied to the analog input modules.

ADC

Before these analog values can be processed in the CPU, they must be converted to digital form. The ADC (Analog-to-Digital Converter) on the analog input module handles this conversion. The analog-to-digital conversion is performed sequentially. This means the signals are converted for each analog input channel in turn.

Result Memory

The result of the conversion is stored in the result memory and remains there until it is overwritten by a new value. You can use the "L IW:P..." load instruction to read the converted analog value directly from the module.

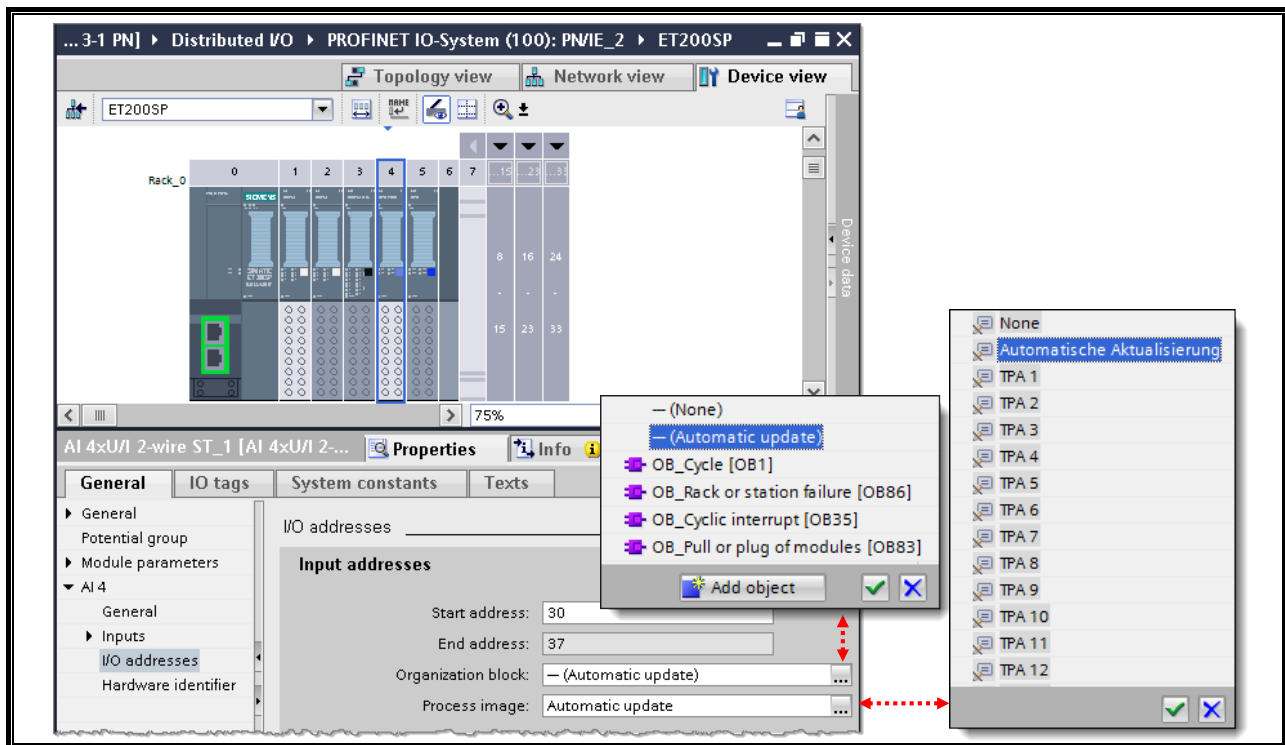
Analog Output

The (MOVE) transfer instruction is used to write the analog values the user program calculated to an analog output module, where a DAC (Digital-to-Analog Converter) converts them to standard analog signals.

Analog Actuators

You can connect standard actuators directly to the analog output modules.

5.3. Analog Input and Output Modules



Module Parameter Assignment

The tool for assigning parameters to analog input and output modules in SIMATIC S7 is "Devices & networks". The settings or parameters of all modules are downloaded into the CPU. The CPU must be in the STOP state to do this. In a subsequent CPU warm restart, the CPU transfers these parameters to the relevant modules via the backplane bus.

Organization Block and Process Image

The converted channel values can be read into the general process image input table (PII) or into a process image partition which in turn is assigned to an OB.

The general PII (called "Automatic update" in the picture) is not assigned to any OB as a process image partition and is cyclically read in at the cycle control point.

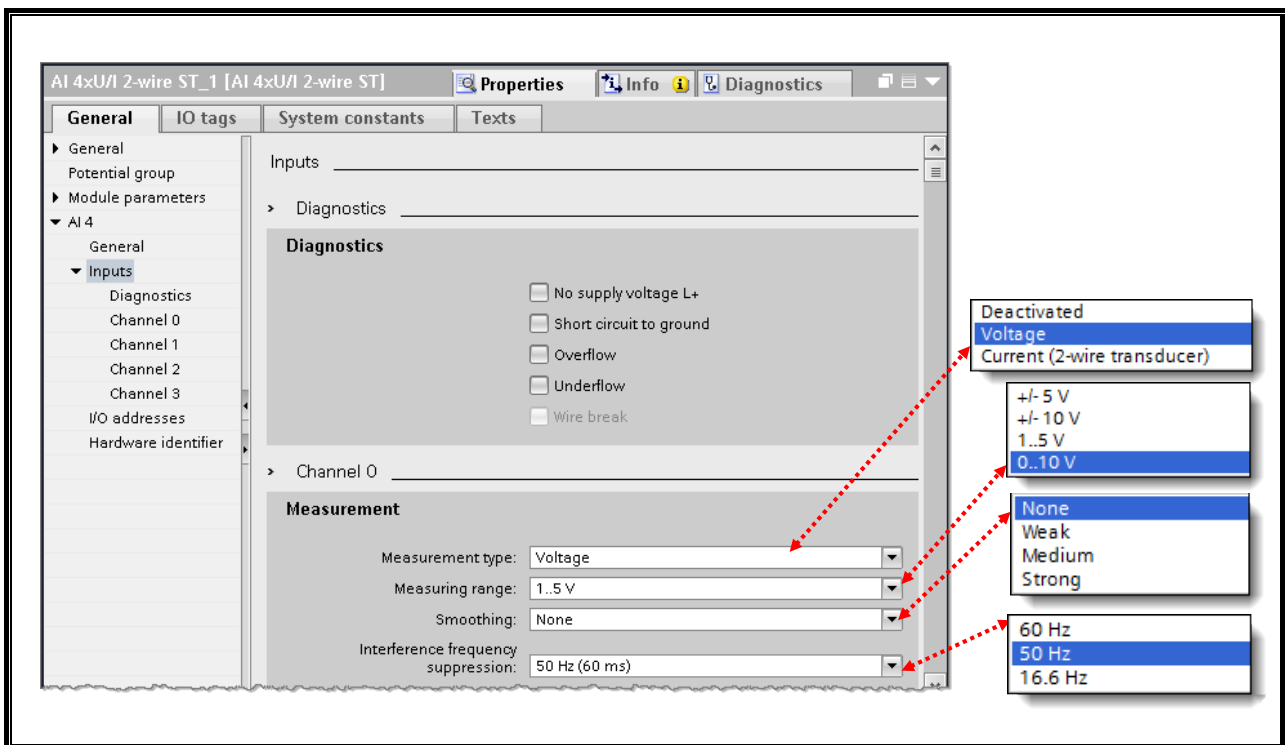
Process image partitions are updated during the call of the OB to which they are assigned as a process image partition.

Integration Time and Interference Frequency Suppression (see picture on the next page)

_ Are interdependent quantities:

- Integration time
When you use the "Device configuration" tool, the resolution is specified indirectly through the selection of an integration time. The following table for SM331 illustrates the relationship between integration time, resolution and interference frequency suppression:
 - Integration time [ms] Resolution [Bit] Interference frequency suppression (Hz)
- | | | |
|------|---------------|-----|
| 2.5 | 9 + sign bit | 400 |
| 16.6 | 12 + sign bit | 60 |
| 20 | 12 + sign bit | 50 |
| 100 | 14 + sign bit | 10 |

5.3.1. Analog Input Modules



Measurement Type:

- The type of measurement, such as voltage, is set with this parameter. An unused channel must then be deactivated since it is otherwise also converted which would result in a longer total conversion time of the module.

Measuring Range:

Click the field to display the available measuring ranges for the corresponding measurement type.

The module triggers a diagnostic interrupt on the CPU when an error occurs. Then, error information is entered in the diagnostic buffer and the CPU immediately processes the program in the "Diagnostic error interrupt"-OB. In this OB, the user can program the necessary response to the error that occurred. Which errors an analog module can recognize depends on the module type.

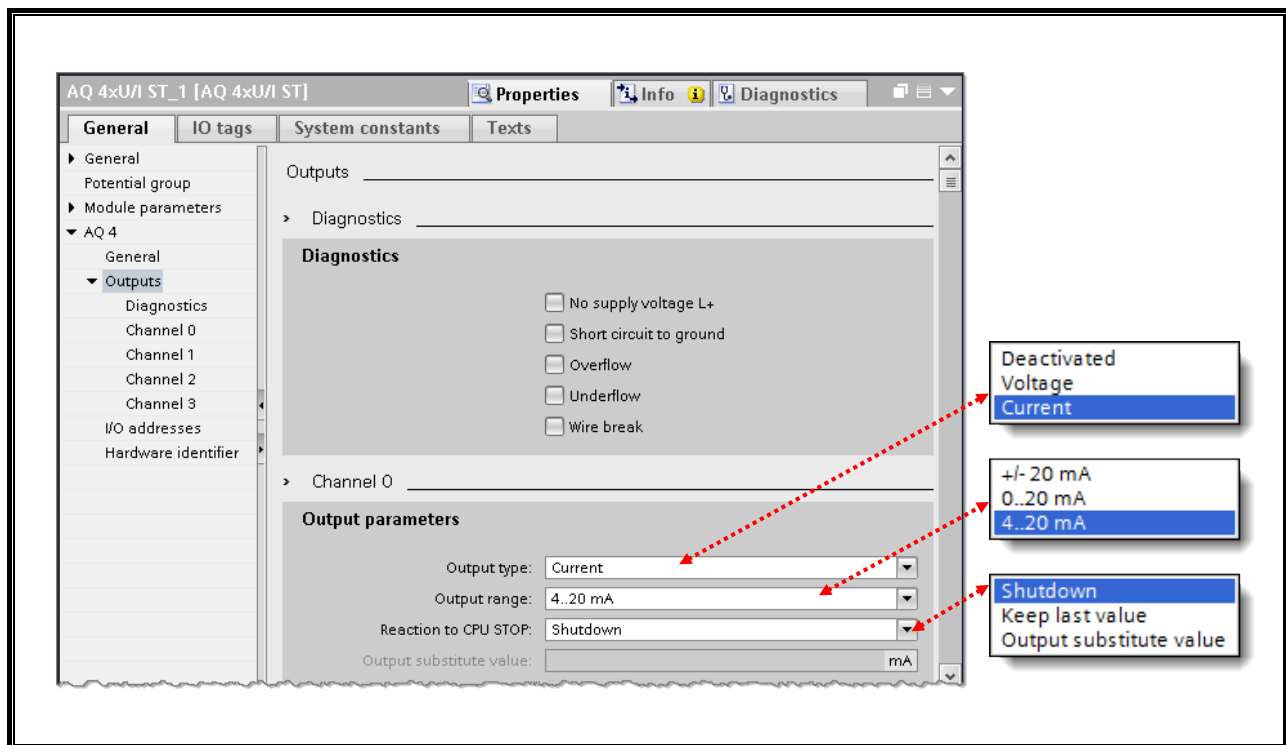
Error examples:

- Incorrect parameter assignment data
- Short circuit to ground (only for analog output channels)
- Wire break
- Missing load voltage L+

Hardware Interrupt

The module triggers a hardware interrupt when a particular event occurs (such as exceeding a voltage limit on a channel of an analog input module). Then the CPU immediately processes an interrupt program that the user stores in one of the "Hardware interrupt"-OBs, to determine the response to the event.

5.3.2. Analog Output Modules



General

Depending on the type of analog output module, voltages and/or currents can be converted or output (see picture).

Reaction to CPU STOP

The behavior of analog output modules in the CPU STOP state can be parameterized:

- Shutdown (Output has no current or voltage)
- Keep last value
- Output substitute value (substitute a value)

Channel Parameters

- Output type
The type of output, such as voltage, is set with this parameter. Unused outputs must be deactivated since these are otherwise also converted which would result in a longer total conversion time of the module.
- Output range
The output range of the selected type of output is set with this parameter.

Diagnostic Interrupt

The module triggers a diagnostic interrupt on the CPU when an error occurs. Then, error information is entered in the diagnostic buffer and the CPU immediately processes the program in the "Diagnostic error interrupt"-OB. In this OB, the user can program the necessary response to the error that occurred. Which errors an analog module can recognize depends on the module type.

Note

Unused outputs should be deactivated since this shortens the conversion time.

5.4. Analog Value Representation and Measured Value Resolution

Bit no.		min. units		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit value		Dec.	Hex.	VZ	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
Resolution in bits + sign	8	128	80	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0
	9	64	40	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0
	10	32	20	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0
	11	16	10	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0
	12	8	8	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0
	13	4	4	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0
	14	2	2	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0
	15	1	1	*	*	*	*	*	*	*	*	*	0	0	0	0	0	0	0

*

 = 0 or 1

Representation

Negative analog values are represented as the two's complement.
The value is positive if bit No. 15=0 and negative if bit No.15=1.

Resolution

If the resolution of an analog module is less than 16 bits, the analog value is written into the accumulator (module result memory) left-justified. The unused less significant bit positions are filled with "0"s.

Accuracy

Resolutions of between 8 and 16 bits are possible, depending on the type of module (S7-300/400).

S7-1500 modules all have a resolution of 16 bits.

5.4.1. Analog Value Representation of Different Measuring Ranges

Range	Voltage such as:		Current such as:		Resistance such as:		Temperature e.g. Pt100 (Standard)	
	Meas.range $\pm 10V$	Units	Meas.range 4 to 20mA	Units	Meas.range 0to300Ohm	Units	Meas.range -200to+850°C	Units
Overflow	≥ 11.759	32767	≥ 22.815	32767	≥ 352.778	32767	≥ 1000.1	32767
Over range	11.7589 : 10.0004	32511 : 27649	22.810 : 20.0005	32511 : 27649	352.767 : 300.011	32511 : 27649	1000.0 : 850.1	10000 : 8501
Rated range	10.00 7.50 : -7.5 -10.00	27648 20736 : -20736 -27648	20.000 16.000 : : 4.000	27648 20736 : : 0	300.000 225.000 : : 0.000	27648 20736 : : 0	850.0 : : -200.0	8500 : : -2000
Under range	- 10.0004 : - 11.759	- 27649 : - 32512	3.9995 : 1.1852	- 1 : - 4864	negative values not possible	- 1 : - 4864	- 200.1 : - 243.0	- 2001 : - 2430
Underflow	$\leq - 11.76$	- 32768	≤ 1.1845	- 32768		- 32768	$\leq - 243.1$	- 32768

Voltage, Current (Symmetrical)

Converting the symmetrical voltage or current ranges

- $\pm 80mV$
- $\pm 250 mV$
- $\pm 500 mV$
- $\pm 1 V$
- $\pm 2.5 V$
- $\pm 5V$
- $\pm 10V$
- $\pm 3.2 mA$
- $\pm 10 mA$
- $\pm 20 mA$

results in a rated range of -27648 to +27648.

Voltage, Current (Asymmetrical)

Converting the asymmetrical voltage or current ranges

- 0 to 2 V
- 1 to 5 V
- 0 to 20 mA
- 4 to 20 mA

results in a rated range of 0 to +27648.

Resistance

Converting the resistance ranges

- 0 to 150 Ohm
- 0 to 300 Ohm
- 0 to 600 Ohm

results in a rated range of 0 to +27648.

Temperature

Temperatures are measured with resistance thermometers or thermocouples. Converting results in a rated range of ten times the temperature range.

5.4.2. Analog Value Representation for the Analog Outputs

Range	Units	Voltage			Current			
		Output ranges:			Output ranges:			
		0 to 10V	1 to 5V	± 10V	0 to 20mA	4 to 20mA	± 20mA	
Overflow	>=32767	0	0	0	0	0	0	
Over range	32511 ⋮ 27649	11.7589 ⋮ 10.0004	5.8794 ⋮ 5.0002	11.7589 ⋮ 10.0004	23.515 ⋮ 20.0007	22.81 ⋮ 20.005	23.515 ⋮ 20.0007	
	Rated range	27648 ⋮ 0 ⋮ - 6912 - 6913 ⋮ ⋮ ⋮ - 27648	10.0000 ⋮ 0	5.0000 ⋮ 1.0000	10.0000 ⋮ 0	20.000 ⋮ 0	20.000 ⋮ 4.000	20.000 ⋮ 0
Under range		- 27649 ⋮ - 32512	0	0.9999	0	0	3.9995	- 20.000
			0	0		0		
			0	0		0		
			0	0		0		
Underflow		<=- 32513			- 10.0004 ⋮ - 11.7589		- 20.007 ⋮ - 23.515	
	0				0			

Voltage, Current (Symmetrical)

For symmetrical voltage or current ranges, a rated range of -27648 to +27648 is converted to:

- $\pm 10V$
- $\pm 20mA$.

Voltage, Current (Asymmetrical)

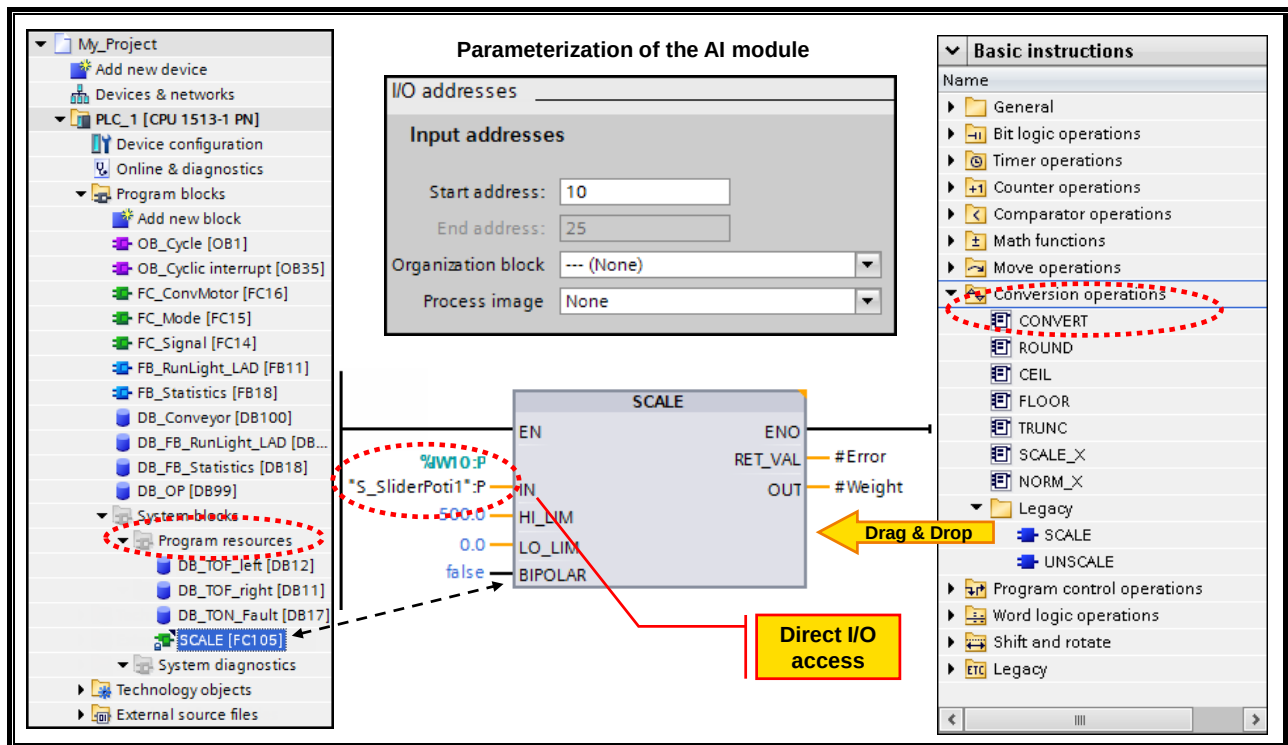
For asymmetrical voltage or current ranges, a rated range of 0 to +27648 is converted to:

- 0 to 10V
- 1 to 5V
- 0 to 20mA
- 4 to 20mA.

Overflow

If the value to be converted reaches the overflow range, the analog output module is disabled (0V, 0mA).

5.5. Analog Value Processing: Direct I/O Access with ":P"



Scaling Analog Values

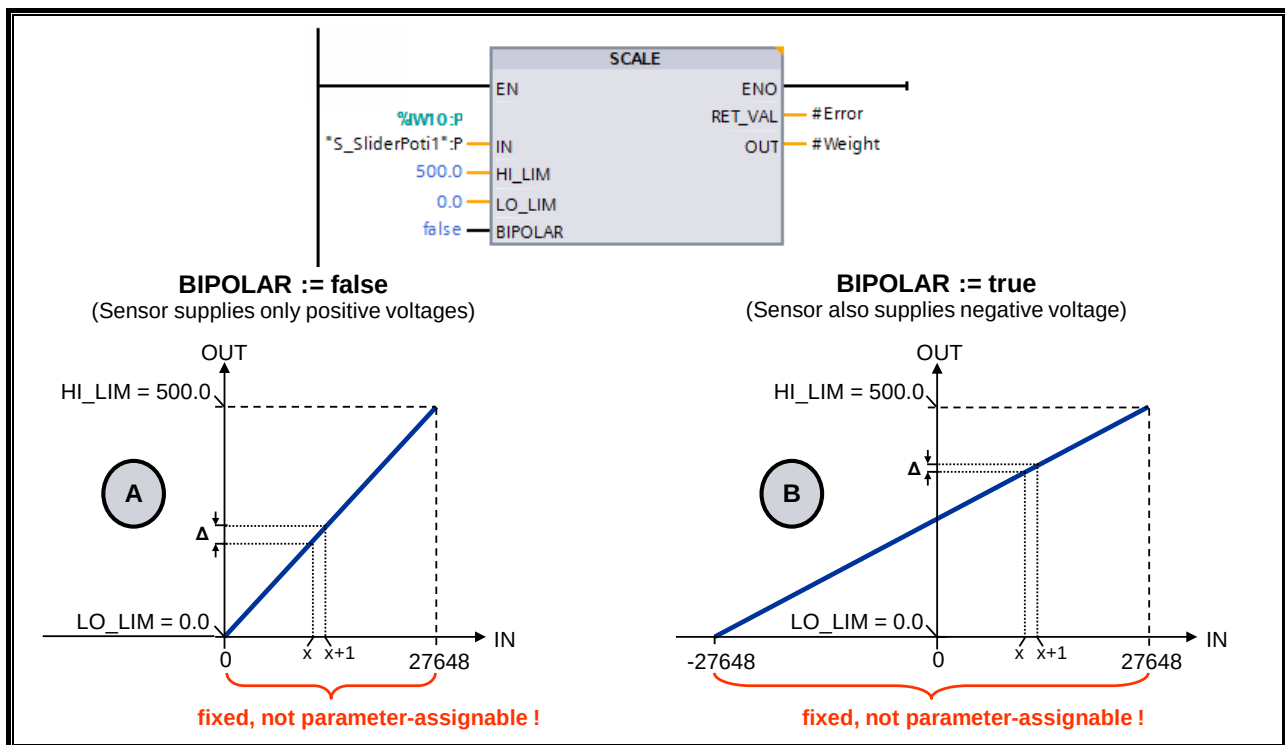
For reading in and scaling analog values, the "Basic Instruction" SCALE is available, among others, which corresponds to the Library Block "SCALE" (FC105) in STEP 7 Classic. When programming this "Basic Instruction", this FC is stored in the directory "System blocks -> Program resources".

Direct I/O Access

Direct I/O accesses are identified with the addition ":P". This addition can be programmed in conjunction with the absolute address or the symbolic name of the analog channel.

Direct I/O accesses are particularly necessary where the modules (as shown in the picture) are not assigned to any process image partition and the converted channel values are therefore read out from the module directly.

5.5.1. Scaling Analog Input Values with SCALE



Examples

The level of a tank, whose volume is 500 liters, is to be measured in liters.

Example **A** shows the scaling when a sensor is used that supplies a measured voltage of 0V when the tank is empty and a measured voltage of +10V when the tank is full. Example **B** shows the scaling when a sensor is used that supplies a measured voltage of -10V when the tank is empty and a measured voltage of +10V when the tank is full.

Resolution

In example **B**, the level is measured with twice the resolution or with half as much measuring tolerance Δ , since the volume of the tank is scaled to the greater unit range of -27648 to +27648.

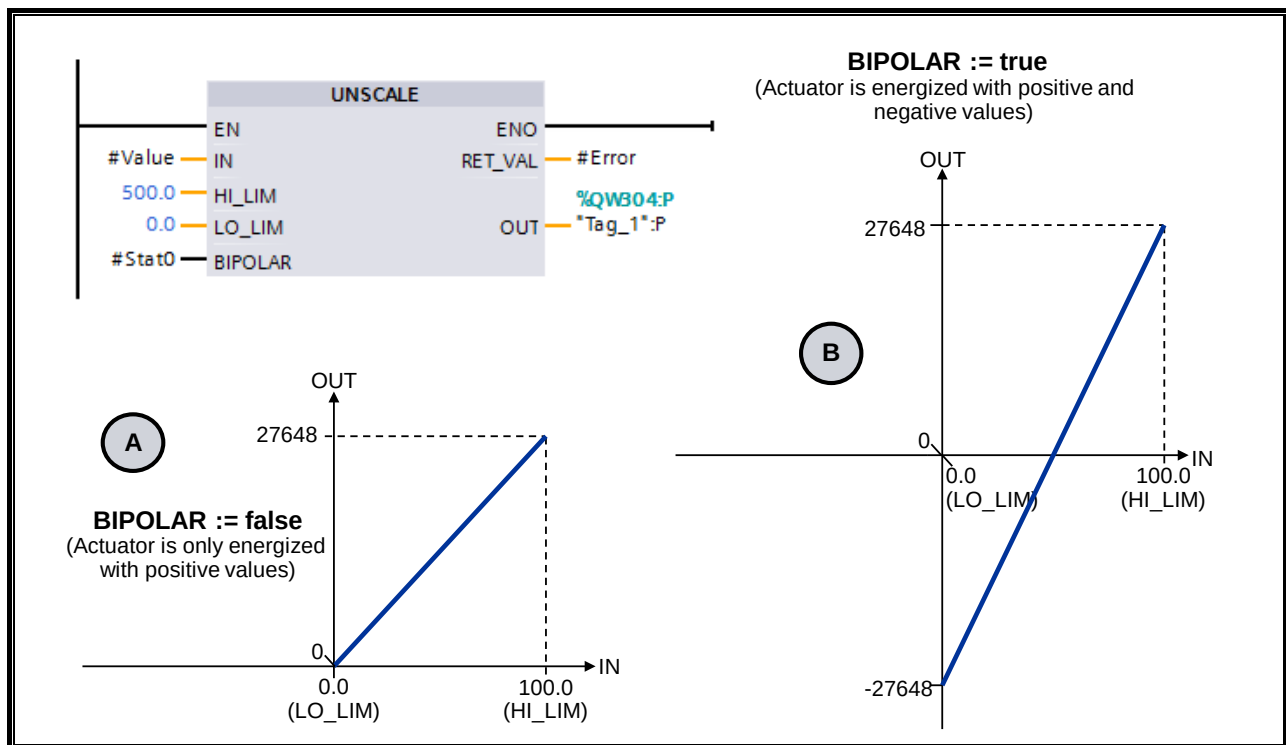
Scaling

The analog module converts the voltage range of -10V to +10V into the value range of -27648 to +27648. The conversion of this value range to the original physical quantity (such as 0 l to 500 l) is called scaling.

The instruction SCALE is used for scaling the analog value:

- **IN:** The analog value at input IN can be read in from the module directly or can be passed from a data interface in INT format.
- **LO_LIM, HI_LIM:** Inputs LO_LIM (low limit) and HI_LIM (high limit) are used for specifying the limits of the basic physical quantity. In the example, a conversion to the range 0 to 500 liters is made.
- **OUT:** The scaled value (physical quantity) is stored as a real number at output OUT ($LO_LIM \leq OUT \leq HI_LIM$).
- **BIPOLAR:** At input BIPOLAR you can specify (parameterize) whether only positive or also negative values are to be converted. If an operand with the state '0' (unipolar) is passed to the parameter, scaling is made for the range 0 to +27648. If the operand state is '1' (bipolar), scaling is made for the range -27648 to +27648.
- **RET_VAL:** The output RET_VAL supplies the value 0 when execution is error-free.

5.5.2. Unscaling Analog Output Values with UNSCALE



Examples

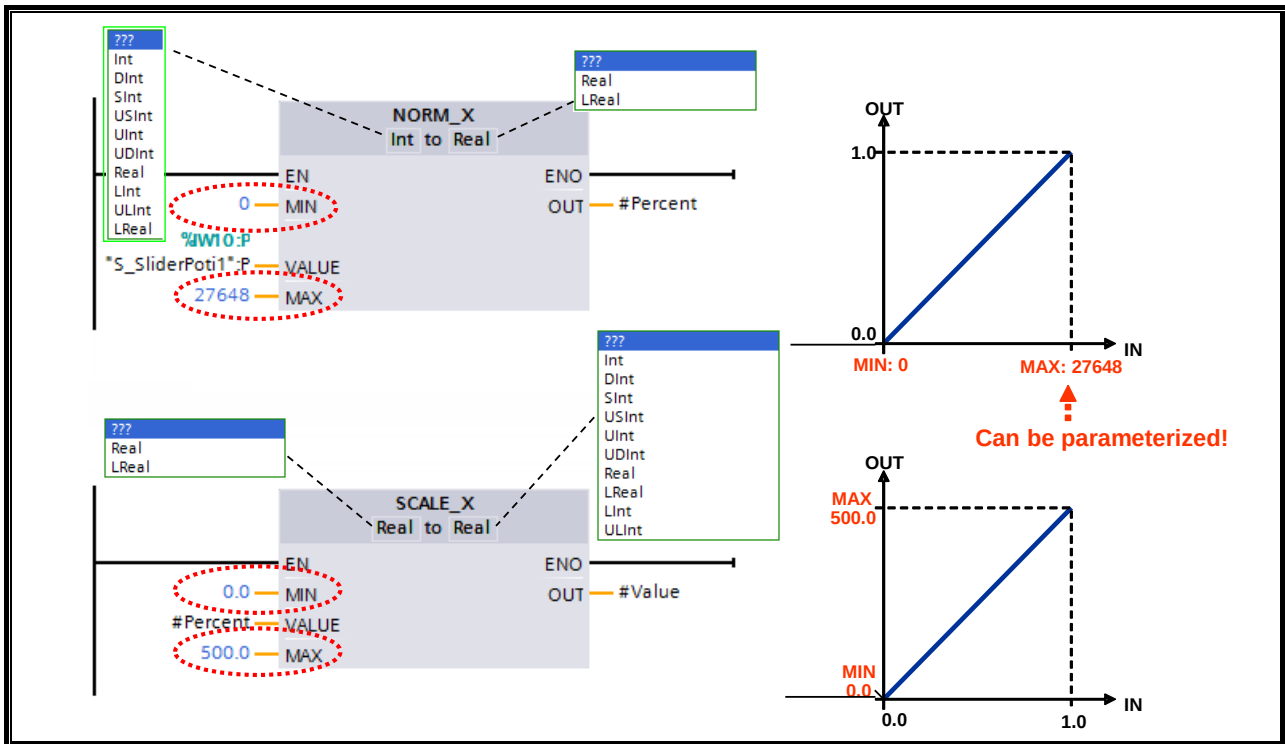
An analog value in the range 0.0 to 100.0% that is calculated by the user program is converted (unscaled) with the instruction UNSCALE to the range 0 to +27648 (unipolar) or -27648 to +27648 (bipolar). When the unscaled value is output to an analog output module, this module will energize the analog actuator (such as a servo valve) with a value such as 0V to +10V (unipolar) or -10V to +10V (bipolar).

Example A shows the scaling when an actuator is used that is to be energized with the value 0 (0V or 0mA) when the program value is 0% and is to be energized with maximum value (such as +10V or 20mA) when the program value is 100%.

Example B shows the scaling when an actuator is used that is to be energized with minimum value (-10V or -20mA) when the program value is 0% and is to be energized with maximum value (such as +10V or 20mA) when the program value is 100%.

- **IN:** The value calculated by the program must be passed in the REAL format.
- **LO_LIM, HI_LIM:** Inputs LO_LIM (low limit) and HI_LIM (high limit) specify the limits for the program value. In the example, this is the range 0.0% to 100.0%.
- **OUT:** The unscaled value is output in the INT format at the output OUT.
- **BIPOLAR:** At input BIPOLAR you can specify (parameterize) whether only positive or also negative values are to be converted. If an operand with the state '0' (unipolar) is passed to the parameter, unscaling is made for the range 0 to +27648. If the operand state is '1' (bipolar), unscaling is made for the range -27648 to +27648.
- **RET_VAL:** The output RET_VAL supplies the value 0 when execution is error-free.

5.5.3. Scaling with NORM_X and SCALE_X



NORM_X

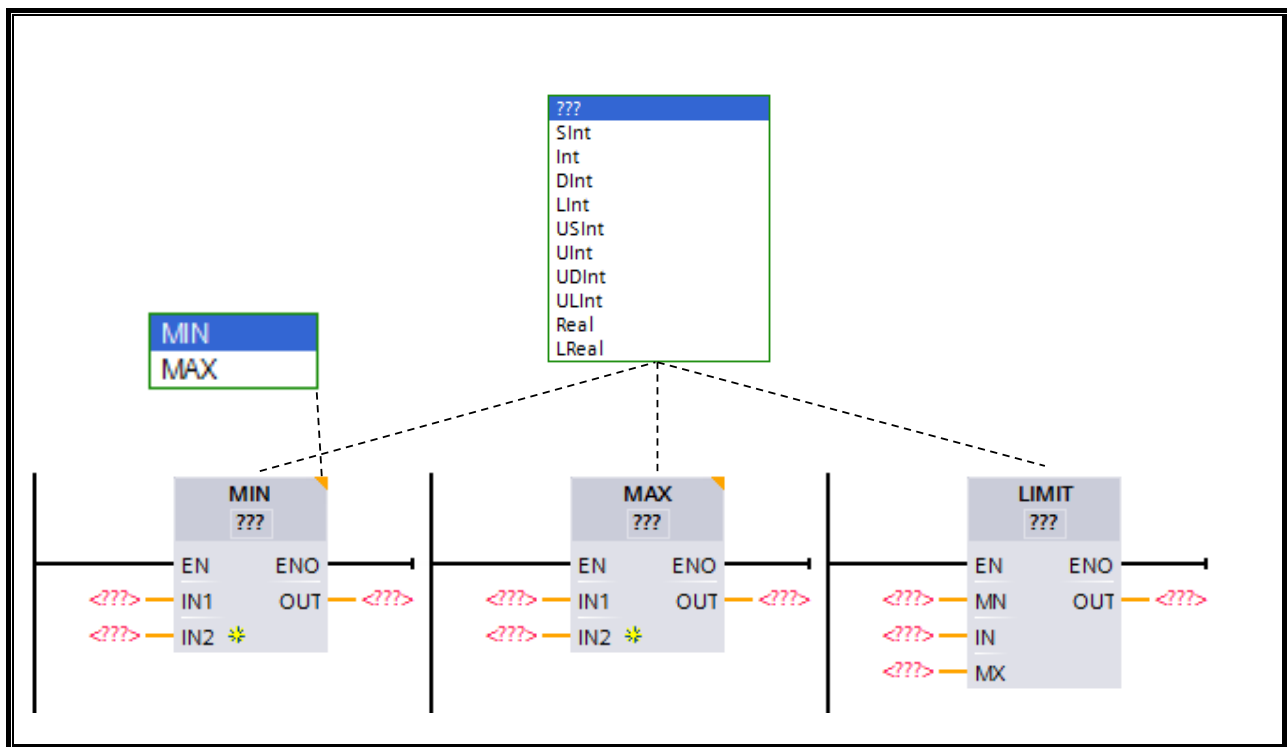
NORM_X scales the input signal "Value" with parameter-assignable data type and value range – parameterizable through the input parameters "MIN" and "MAX" - to the norm signal 0.0 to 1.0 output at output OUT.

SCALE_X

SCALE_X maps the norm signal with value range 0.0 to 1.0 at input "Value" to the value range parameterizable through the inputs MIN and MAX. The value scaled in such a way is output at output OUT (MIN ≤ OUT ≤ MAX).

Through the combination of these instructions, any value ranges can be mapped or scaled on other value ranges.

5.5.4. Math Functions: MIN, MAX, LIMIT



MIN / MAX

The "Get minimum" ("Get maximum") instruction compares the values of the inputs IN1, IN2 and IN3 and outputs the smallest (largest) input value at output OUT. The error-free processing of the instruction (ENO = 1) presumes that the values at all inputs are valid and are of the same data type.

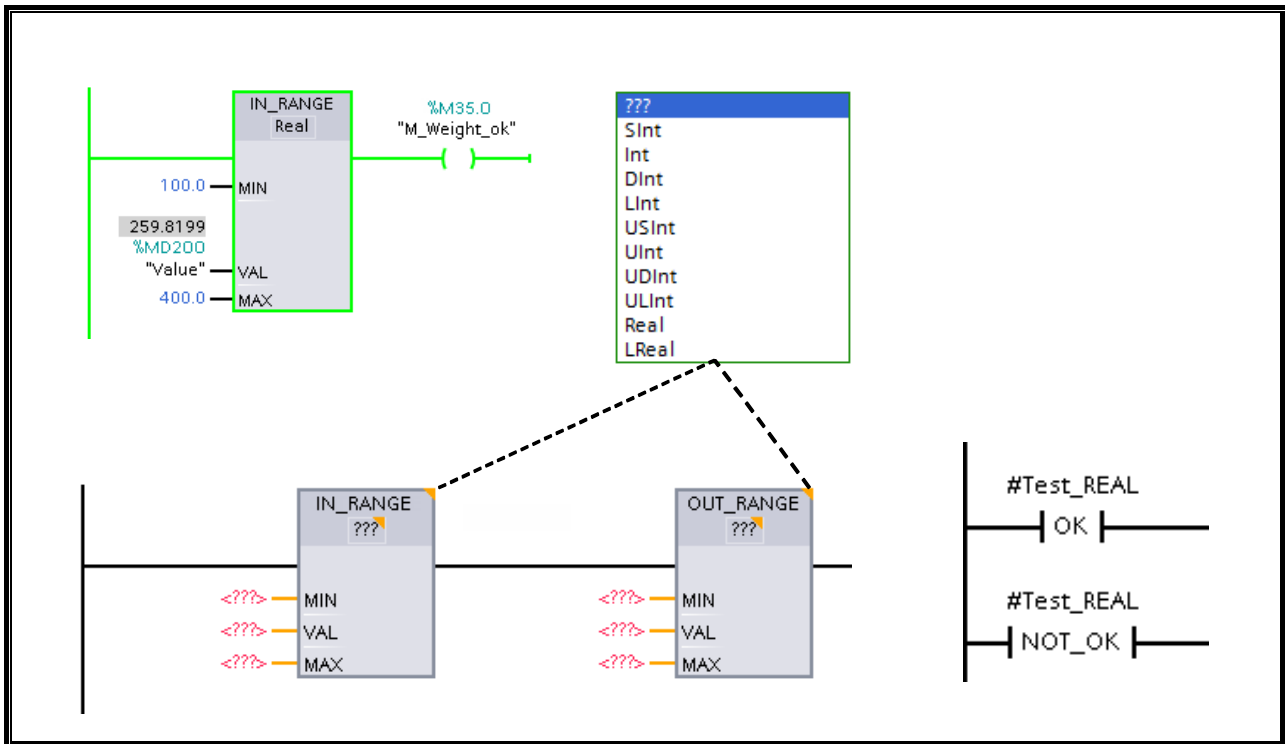
LIMIT

The "Limit" instruction limits the value at input IN to the values at the inputs MN and MX. When the value at input IN satisfies the condition $MN \leq IN \leq MX$, it is output at the output OUT.

- If the input value IN falls below the lower limit MN, the value of MN is output at the output OUT.
- If the input value IN exceeds the upper limit MX, the value of MX is output at the output OUT.
- If the value at input MN is greater than the value at input MX, the result is undefined and the enable output ENO supplies RLO '0'.

The error-free processing of the instruction (ENO = 1) presumes that the values at all inputs are valid and are of the same data type, and that $MX > MN$.

5.5.5. Verification with IN_RANGE, OUT_RANGE, OK



IN_RANGE / OUT_RANGE

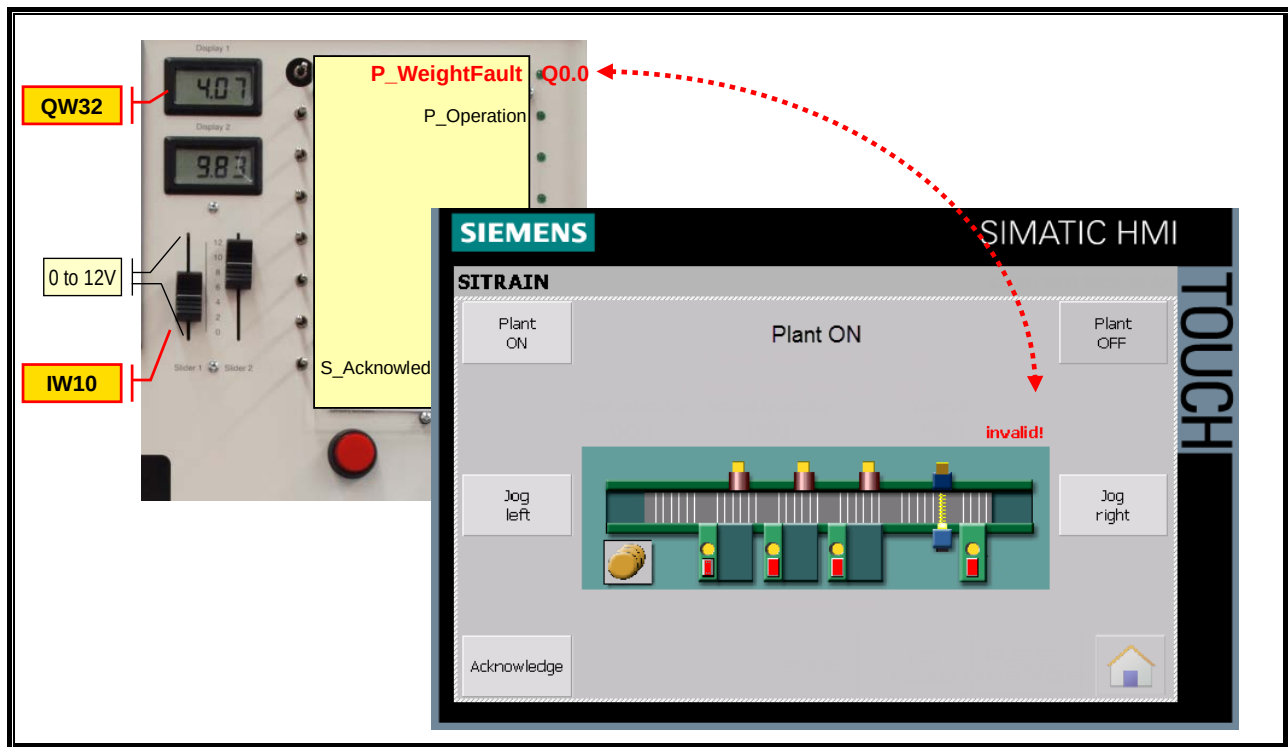
The "IN_RANGE" ("OUT_RANGE") instruction checks whether the value at input VAL lies within (outside of) the value range defined through the inputs MIN and MAX. If this condition is satisfied, the box output supplies RLO "1".

The error-free processing of the instruction presumes that the values at both inputs are valid and are of the same (parameter-assignable) data type.

OK / NOT_OK

The OK (NOT_OK) instruction checks whether the value of the variable specified through the box corresponds to a valid REAL or LREAL. If this is the case, the box supplies RLO '1' at its output.

5.6. Task Description: Checking the Weight and Displaying It



Task Description: Checking the Weight:

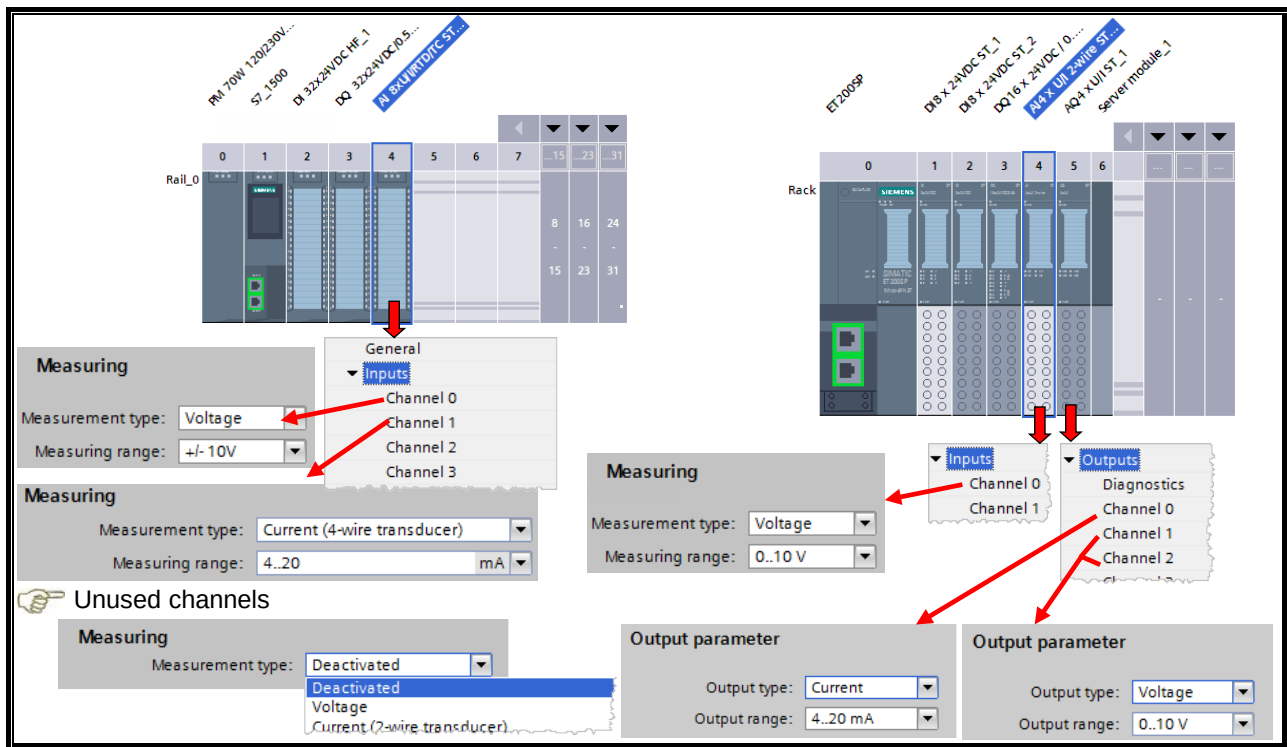
The parts to be transported are weighed in the Start Position (light barrier) before the conveyor is started. The part weight (0 to 500kg) is simulated via the simulator potentiometer "S_SliderPoti1" (IW10) and displayed on the touchpanel screen "Conveyor" at the output field "Weight".

The permitted range of the part weight is **100kg <= Weight <= 400kg**

As long as an invalid weight (weight < 100kg or weight > 400kg) is set ...

- ... the simulator indicator light "P_WeightFault" (Q 0.0) shows a 2Hz flashing light.
- ... the indicator light on the light barrier bay is dark because no part with an invalid weight may be transported
- ... no transport sequence can be started

5.6.1. Exercise 1: Setting the Channel Parameters of the AI Modules (ET200SP and Central AI)

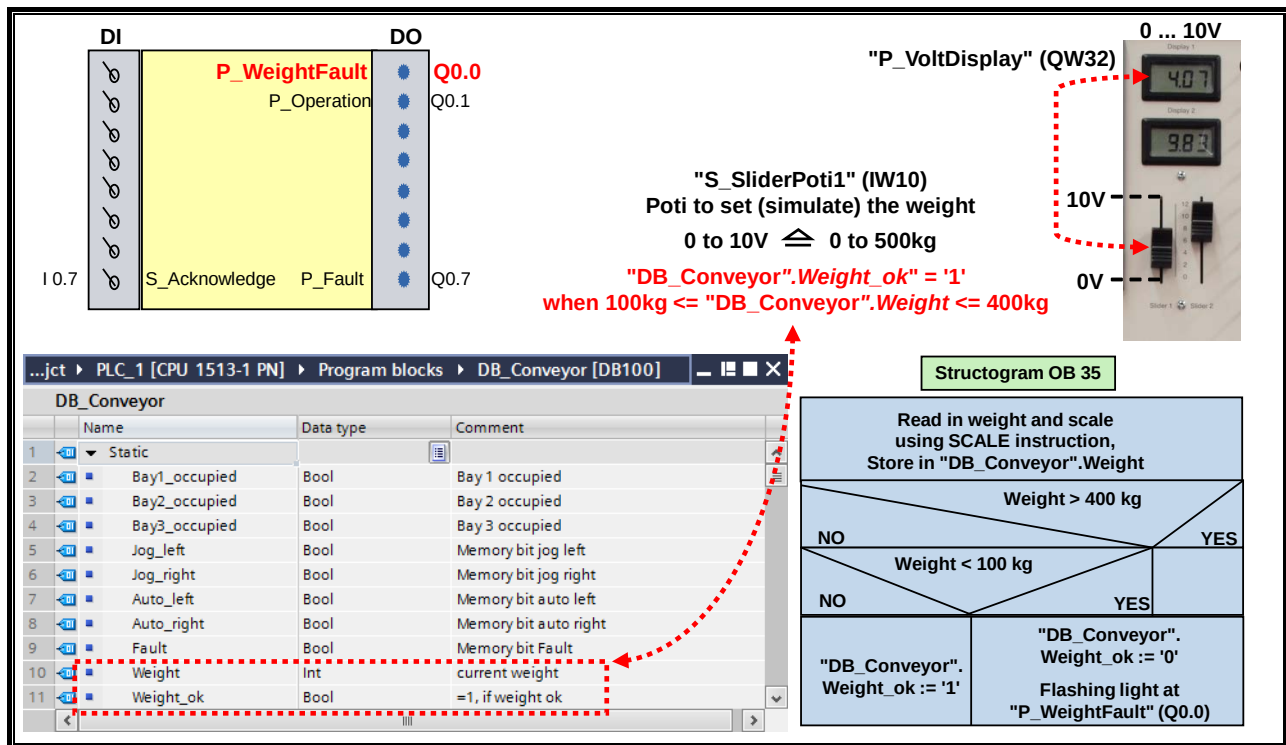


Task and What to Do:

Set the analog channels to the relevant parameters:

1. Inputs central I/O
 - Channel 0 → Voltage +/-10V
 - Channel 1 → Current (4-wire measuring transducer) 4 to 20mA
2. Inputs distributed I/O (ET200SP)
 - Channel 0 → Voltage 0 to 10V
3. Outputs distributed I/O (ET200SP)
 - Channel 0 → Current 4 to 20mA
 - Channel 1 → Voltage 0 to 10V
 - Channel 2 → Voltage 0 to 10V
4. Deactivate all unused analog inputs and outputs.
5. Download the modified parameterization data into the CPU.
6. Save your project.

5.6.2. Exercise 2: Checking the Weight with Cyclic Interrupt "OB_Cyclic interrupt" (OB35)



Checking the Weight

The part weight (0 to 500kg) is simulated via the simulator potentiometer "S_SliderPoti1" (IW10) and displayed on the touchpanel screen "Conveyor" at the output field "Weight".

As long as an invalid weight (weight < 100kg or weight > 400kg) is set ...

- ... the simulator indicator light "P_WeightFault" (Q 0.0) shows a 2Hz flashing light.
- ... the conveyor model indicator light "P_BayLB" (Q4.4) is dark because no part with an invalid weight may be transported
- ... no transport sequence can be started

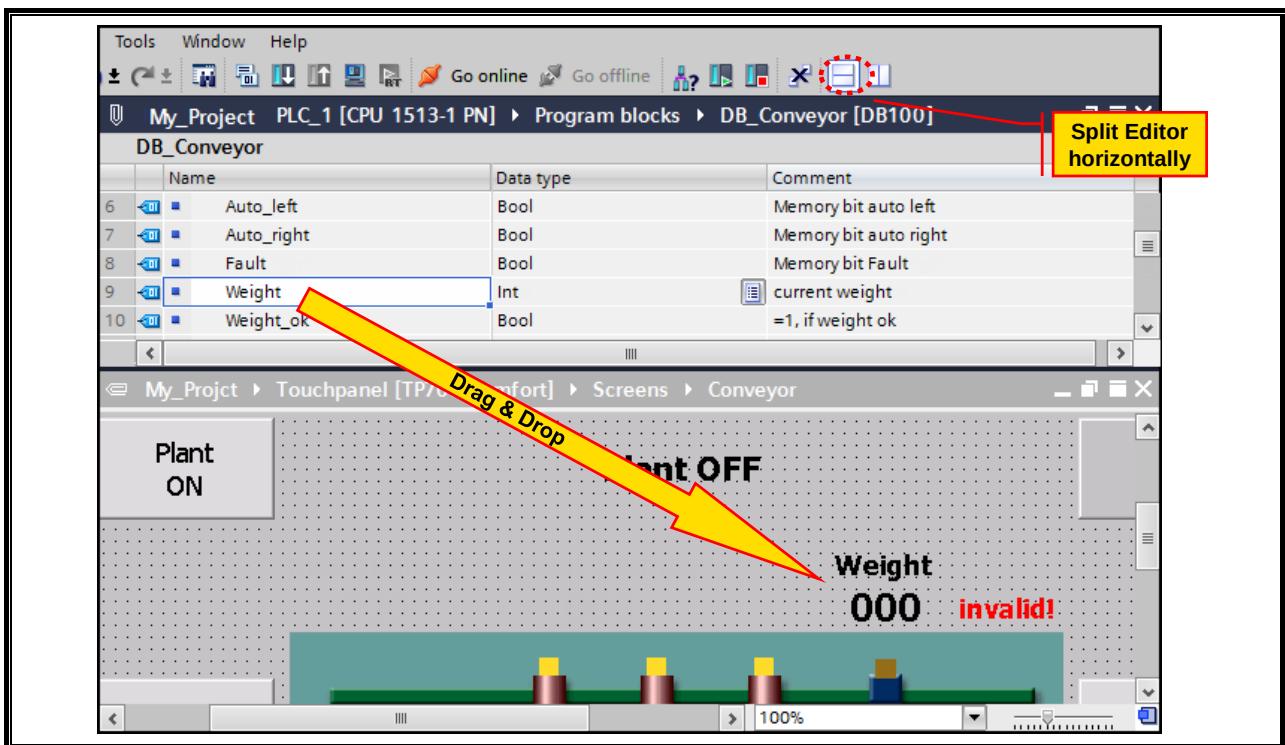
What to Do

- Expand "DB_Conveyor" (DB100) by adding the variables *Weight* (data type INT) and *Weight_ok* (data type BOOL) as shown in the picture.
- Insert a new "Cyclic Interrupt" OB, rename it "OB_Cyclic interrupt" (OB35) and set the call interval to 250ms in its Block Properties.
- In this OB, program that the voltage set at "S_SliderPoti1" (IW10) is displayed at the upper display "P_VoltDisplay" (QW32).
- Furthermore, program the required functions and use the "Basic Instruction" SCALE.
- In "FC_Signal" (FC14) and "FC_ConvMotor" (FC16), program the above-mentioned interlocks.
- Check all functions and save your project.

Note

With the programming of the instruction SCALE, the FC105 is copied into the project in the container Program blocks -> System blocks -> Program resources. The block, however, only becomes visible there after the program is compiled.

5.6.3. Exercise 3: Displaying the Part Weight on the Touchpanel

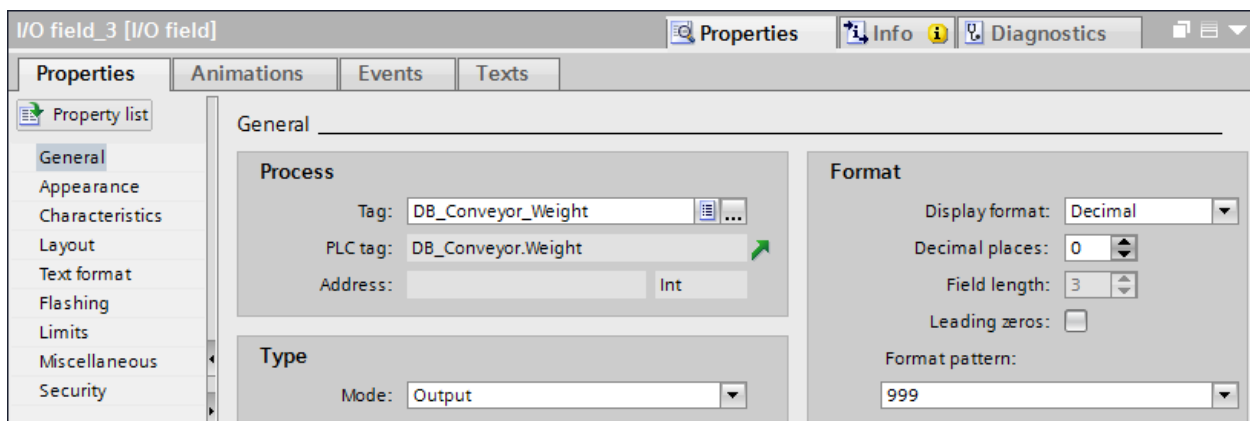


Task

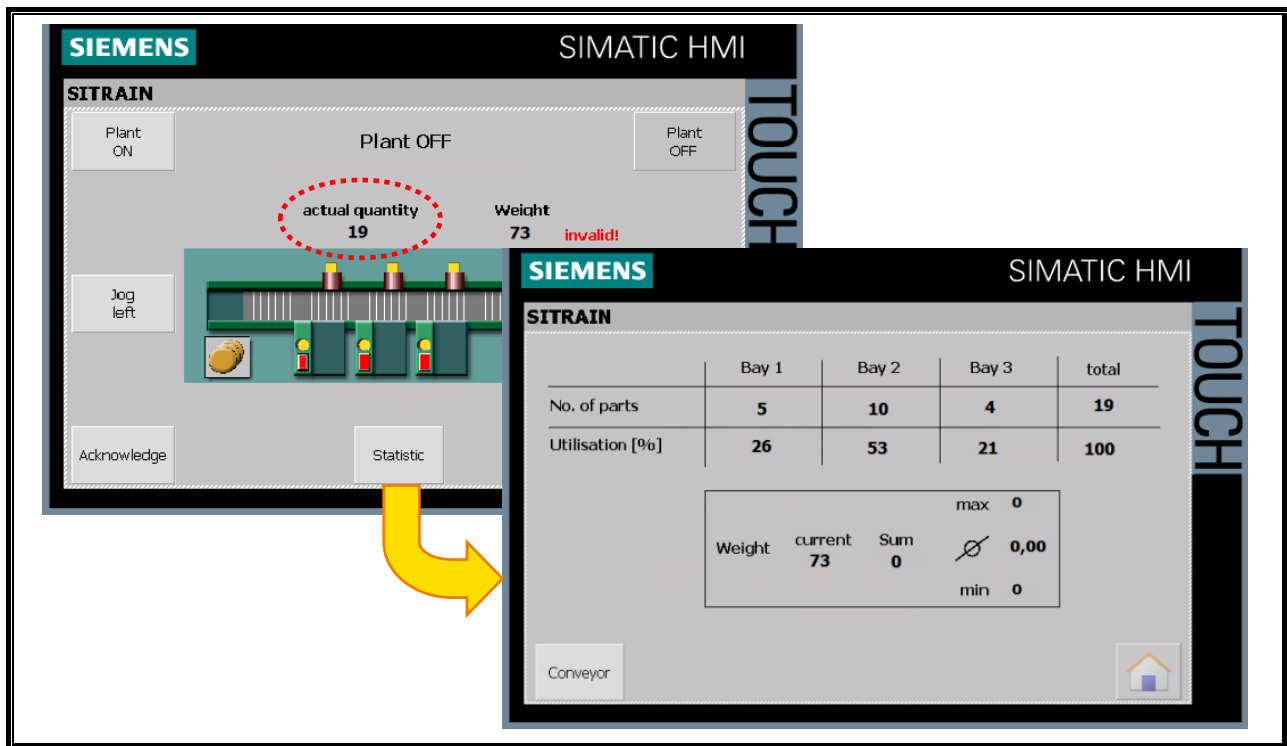
The current part weight set on the simulator and saved in the variable "DB_Conveyor".Weight is to be displayed on the touchpanel. For this, an output field still has to be created in the screen "Conveyor".

What to Do

1. Close all objects that are open in the Editor.
2. Open the "DP_Conveyor" (DB100) data block in which the weight is saved.
3. In addition, open the touchpanel screen "Conveyor".
4. Split the editor area into two areas (see picture).
5. Configure the output field for displaying the weight by dragging the variable *Weight* from the data block into the touchpanel screen using drag & drop.
6. Select the newly created input/output field and, in the Inspector window under "Properties->General", set "Output" as the Mode for "Type".



5.7. Task Description: Calculating and Displaying the Statistic Values



Task Description: Statistic Values:

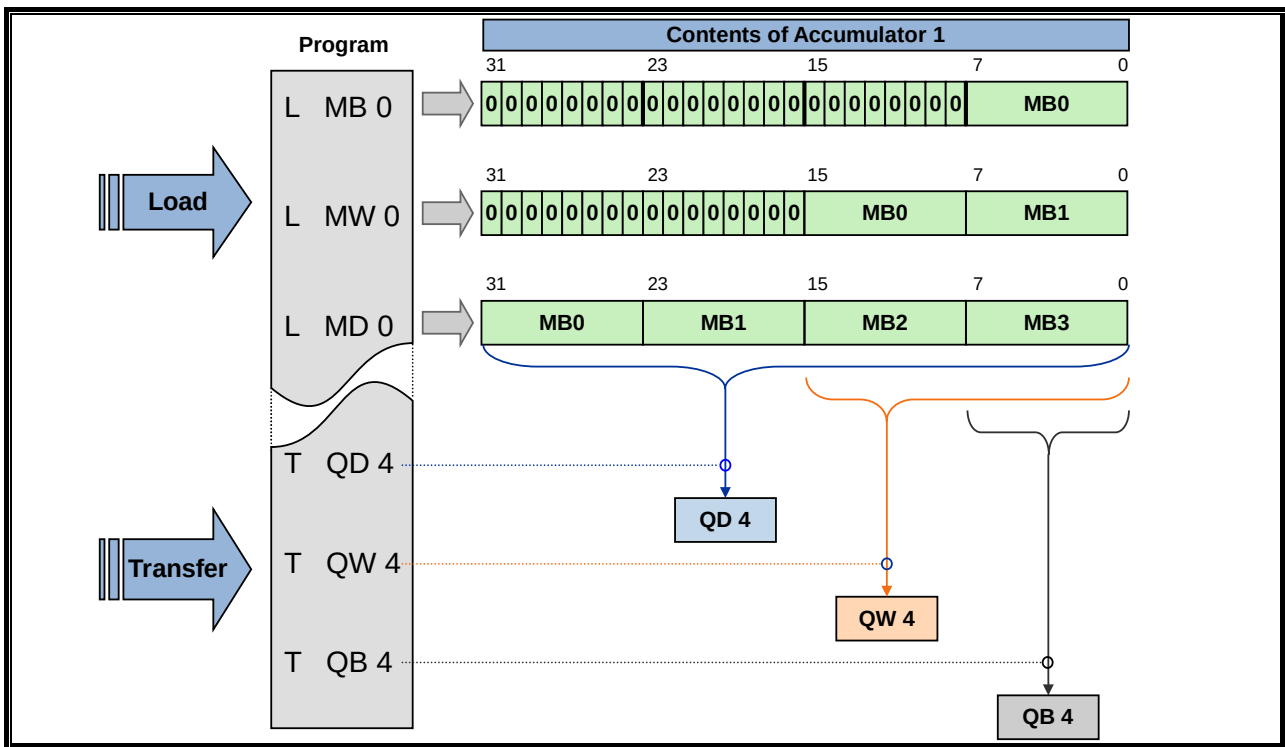
All transported parts are counted when they reach the next free bay. Both the number of parts of the individual bays and the total number of parts are recorded. In addition, the utilization of the bays is determined, that is, which percentage of transported parts has been transported to Bay 1, 2, and 3 is calculated.

In the touchpanel screen "Statistic", all statistic values are displayed; in the screen "Conveyor", only the total number of parts.

Note:

The weight statistic values shown in the picture are calculated in a later chapter.

5.8. Data Storage in Accumulator 1



General

Accumulators are registers of the S7-300/400 CPUs, of which the S7-300 usually has 2 with 32 bits each. Powerful CPUs such as the CPU 318-2 and S7-400 each have 4 accumulators also with 32 bits each.

For S7-1500, Accumulators 1 and 2 are emulated and are thus also available for the exchange of data between variables and for mathematical calculations in STL.

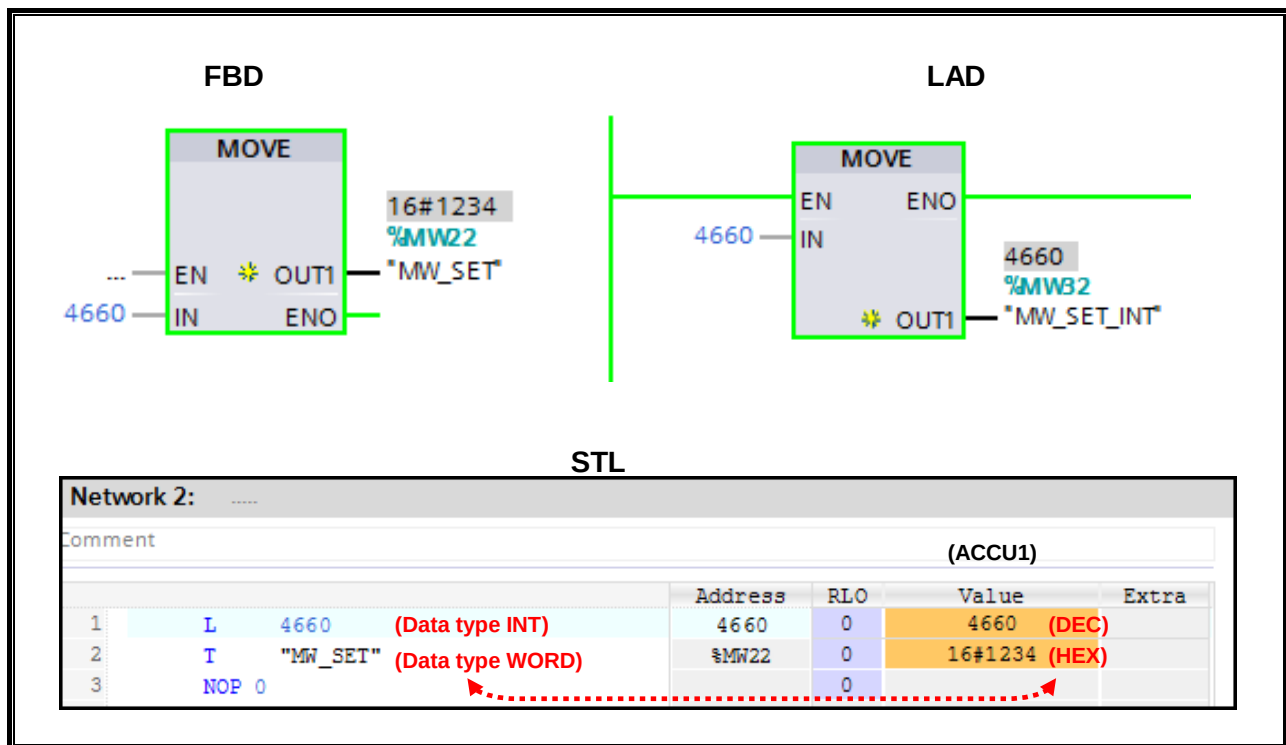
Load

The load instruction loads the contents of the specified byte, word or double word into accumulator (ACCU) 1.

Transfer

When a transfer instruction is executed, the contents of ACCU 1 are retained. Consequently, the same information can be transferred to different destinations. If only one byte is transferred, the eight bits farthest to the right are used (see picture).

5.8.1. Loading and Transferring Data



MOVE (LAD/FBD)

When RLO=1 is at input EN, the value at input "IN" is copied to the variable specified at output "OUT". "ENO" has the same signal state as "EN".

L and T (STL)

Load and transfer instructions are executed regardless of the result of logic operation (RLO). Data is exchanged through the accumulator ACCU1.

Standard / ACCU1

Accumulator 1 (ACCU1) is the central register in the CPU. The load instruction writes the value from the source address right-justified into the (standard) Accumulator 1 and pads the remaining bits (32 bits in all) with "0"s. The transfer instruction copies some or all of the contents of Accumulator 1 to the specified memory address without clearing (deleting) the contents of Accumulator 1 (see next page). All arithmetic and digital calculation instructions store their result in Accumulator 1.

ACCU2

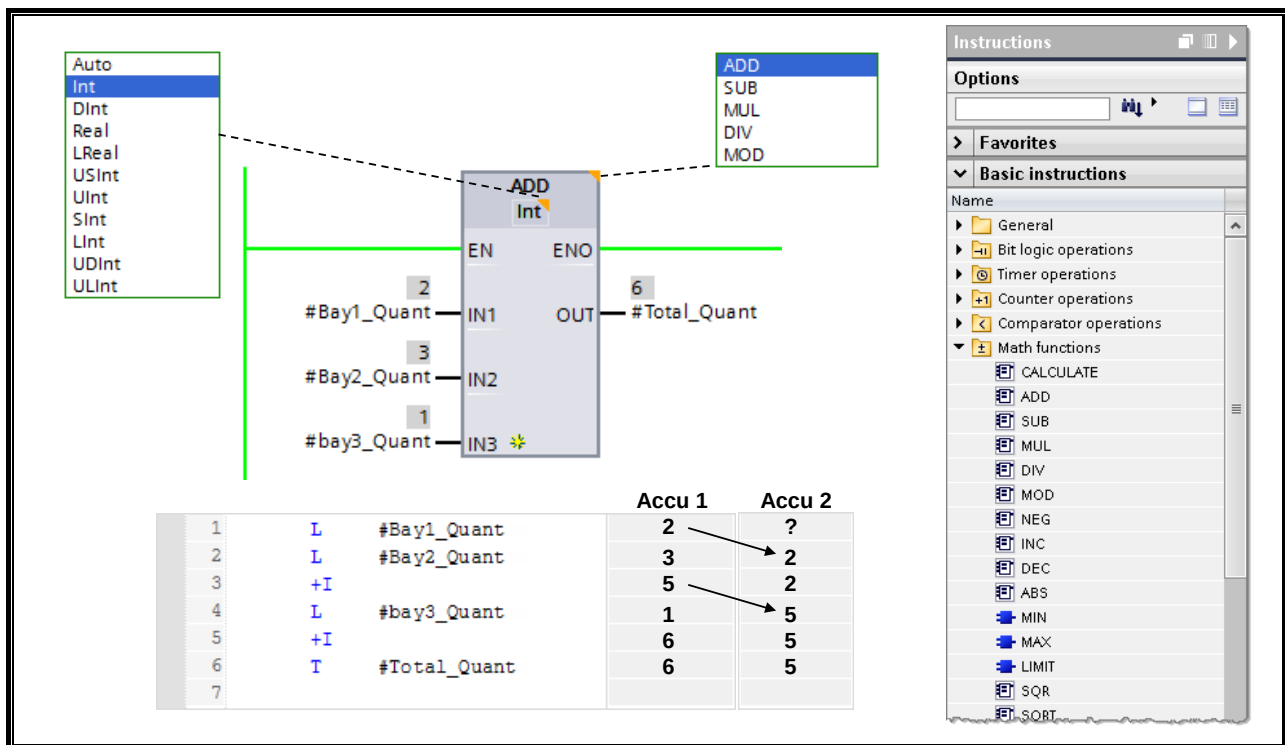
When a load instruction is executed, the old contents of accumulator 1 are first shifted to Accumulator 2 and Accumulator 1 is cleared (reset to "0") before the new value is written into Accumulator 1.

EN

The execution of the operation can be controlled as follows through the EN input:

- EN is not connected: The operation is always executed (regardless of the RLO)
- Logic operation at EN is fulfilled (RLO = 1): The operation is executed
- Logic operation at EN is not fulfilled (RLO = 0): The operation is not executed

5.8.2. Basic Mathematical Functions: Addition



Arithmetic Instructions

For S7-1500, there are a number of arithmetic (math) instructions for processing variables of the arithmetic data types, such as, Integer (I), Double-Integer (DI) and Real (R).

Inputs and Outputs of the LAD/FBD Elements:

- **IN1 / IN2 / IN3.....**

A mathematical function is performed with the value passed at IN1 and the value passed at IN2 according to the arithmetic instruction and the result is output at the output OUT.

ENO / OUT

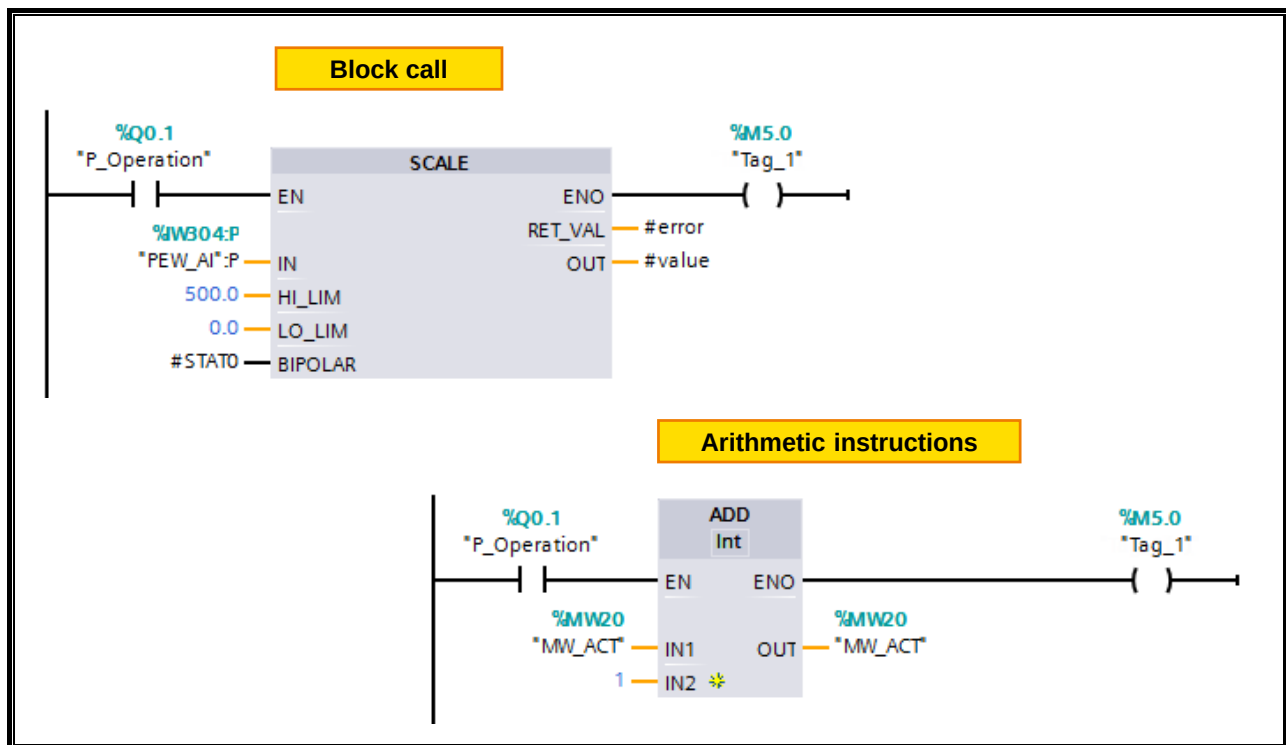
- **OUT:** Result of the addition, sum
- **ENO**
 - ENO = 0 because RLO=0 at EN input → OUT not written
The variable passed at the output OUT is not written, that is, it retains its original value
 - ENO = 0 because an error occurred → OUT contains an invalid value
The variable passed at the output OUT is overwritten with an invalid value
 - ENO = 1 (instruction was executed without error) → OUT contains result
The variable passed at the output OUT is overwritten with the result

Addition in STL

With the first-time loading, Accu1 is given the value of **#Bay1_Quant** and the value of Accu1 up until now (in the picture, represented with ?) is shifted to Accu2. Then, the value of **#Bay2_Quant** is loaded into Accu1 and Accu2 is overwritten with the old Accu1-content (**#Bay1_Quant**).

Since only 2 values can be loaded and added in the Accus at a time, after loading **#Bay1_Quant** and **#Bay2_Quant**, a subtotal must be formed through the instruction **+I**. After that, **#Bay3_Quant** is loaded in Accu1 and added to the subtotal (Accu2) → Result in Accu1.

5.8.3. Meaning of EN, ENO and BR Bit



EN = Enable Input

You can modify a call with the help of the enable input EN. It exists at every box for the call of a block or a complex function in Ladder Diagram.

- If EN is not activated, the function in the box is always executed
- If EN is activated or if a condition is formulated at EN, the function in the box is only executed if the condition is fulfilled (signal state "1"). If the condition is not fulfilled (signal state '0'), the function in the box is not executed.

ENO = Enable Output

With the help of the enable output ENO, the block that is called or the function in the box signals whether the execution took place and whether it could be executed error-free.

You can use the BR-bit of the status word to save errors. The BR-bit is only changed by the user program, not by the system.

If an error occurs during execution, you can "save" this error state by resetting the BR-bit. After executing a box in LAD/FBD, the state of the BR-bit is then copied into the "output parameter" ENO.

A uniform mechanism for passing the error status is thus available in STEP 7. In this way, for example, a block that is called can inform the calling block if the processing was executed error-free or not.

Note

The EN parameter is not a true input parameter. If it is activated, then two instructions with a conditional jump to a label behind the box execution are automatically generated by the LAD/FBD editor.

By the same token, ENO is not a true output parameter. If ENO is activated, then two instructions for copying the BR-bit into the current output parameter are automatically generated by the LAD/FBD editor.

5.8.4. Basic Mathematical Functions: Comparator Operations

The screenshot shows the SIMATIC TIA Portal interface. On the left, a ladder logic diagram features a network with a variable `#Total_Quant` (value 6) connected to a `<=` (less than or equal to) instruction block. The block is labeled `Int` and `0`. The output of this instruction is connected to a `#RLO` (Reset Logic Output) variable. To the right, a list of comparison operators is shown: `>`, `==`, `<>`, `<`, `>=`, and `<=`. Below the diagram, a table displays the state of the RLO and accumulator variables:

	RLO	Accu 1	Accu 2
1 L #Total_Quant	0	6	?
2 L 0	0	0	6
3 <=I	0	0	6
4 BEC		0	6

On the right side of the interface, the 'Instructions' palette is visible, showing a tree structure with categories like General, Bit logic operations, Timer operations, Counter operations, and Comparator operations. The 'Comparator operations' category is expanded, listing various comparison instructions such as `CMP ==`, `CMP <>`, `CMP >=`, `CMP <=`, `CMP >`, `CMP <`, `IN_Range`, `OUT_Range`, `-[OK]-`, and `-[NOT_OK]-`.

Comparator Function

With the comparator operations, you can compare the following pairs of numerical values:

- All variations of Integer
- All variations of floating-point numbers (Real = IEEE floating-point numbers)
- All variations of time data types

If the result of the comparison is "true", then the RLO of the operation is "1", otherwise it is "0". Input IN1 is compared with IN2 according to the selected type of comparison:

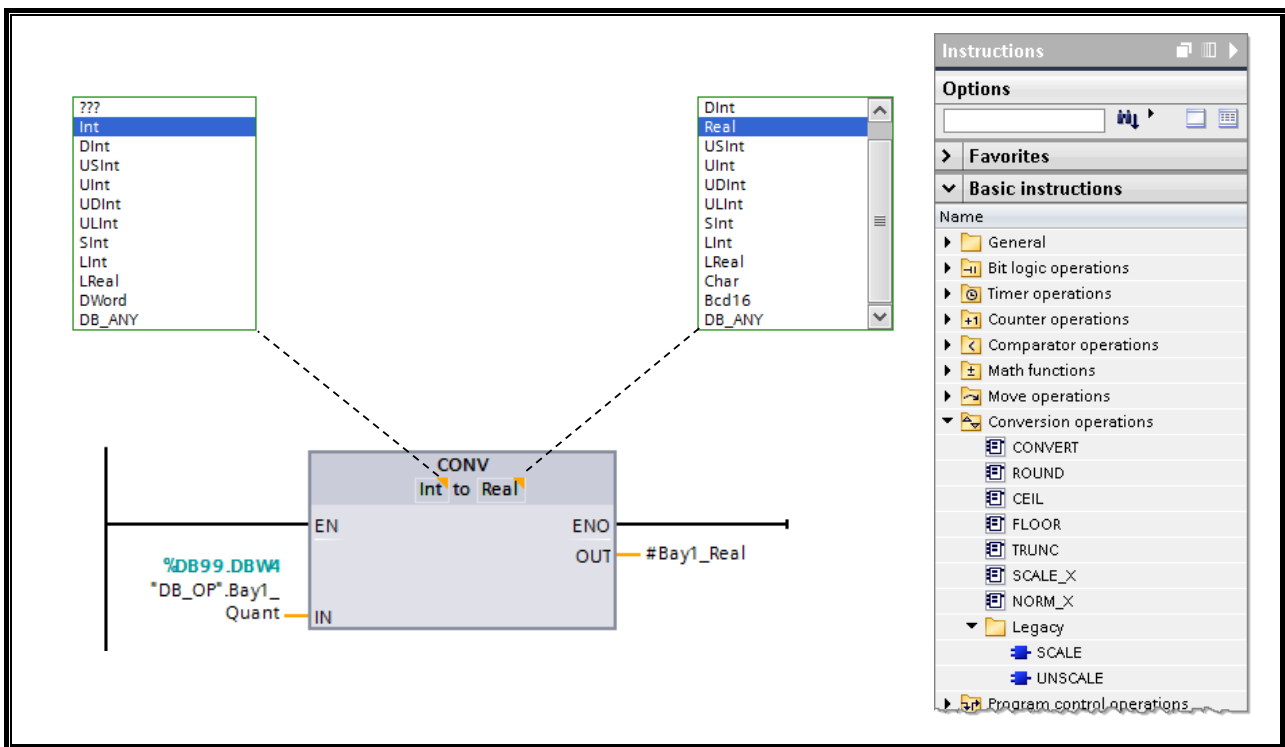
• ==	IN1	is equal to	IN2
• <>	IN1	is not equal to	IN2
• >	IN1	is greater than	IN2
• <	IN1	is less than	IN2
• >=	IN1	is greater than or equal to	IN2
• <=	IN1	is less than or equal to	IN2.

Comparison in STL

Accu 2 is always compared with Accu 1.

Compared in the example shown, whether `#Total_Quant` is `<= 0`.

5.8.5. Conversion Operations: Data Type Conversion



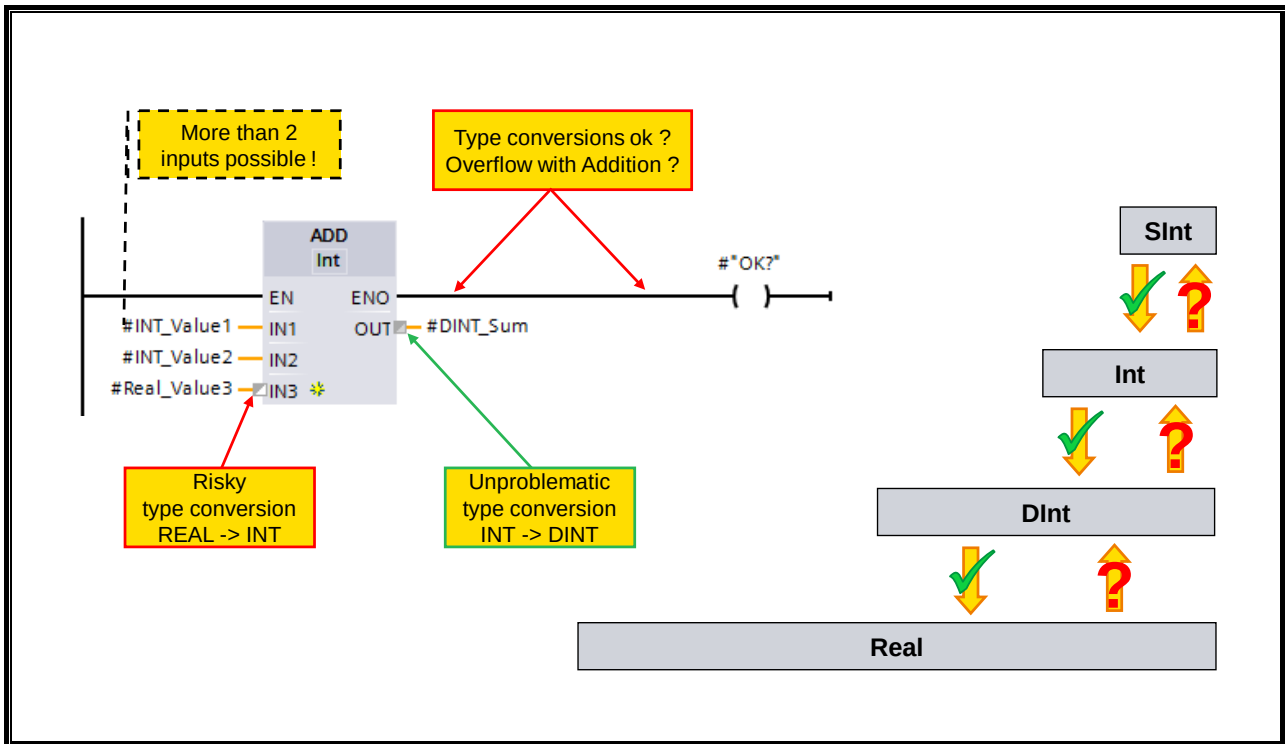
Conversion Operations

Conversion operations are used to convert variables of different arithmetic data types to the same type since only variables of the same data type can be calculated.

Unproblematic are data type conversions into another data type with a larger value range (for example, from INT to REAL). Here, no overflow can result and, with that, no error.

Problematic, on the other hand, can be data type conversions into another data type with a smaller value range (for example, from REAL to INT). An overflow can result (when the current value of the REAL variables is 35000, for example), so that an incorrect value is then output at the output OUT. Through the output ENO (= '1' when error-free) you can however query whether the data type conversion occurred without an error.

5.8.6. Implicit Data Type Conversion



Implicit Type Conversion

Basically, math operations are only possible with operands of the same data type. If, for example, two variables of different data types are to be processed mathematically (e.g. addition, multiplication...), the data types of the two variables must be adapted to one another with explicit data type conversions.

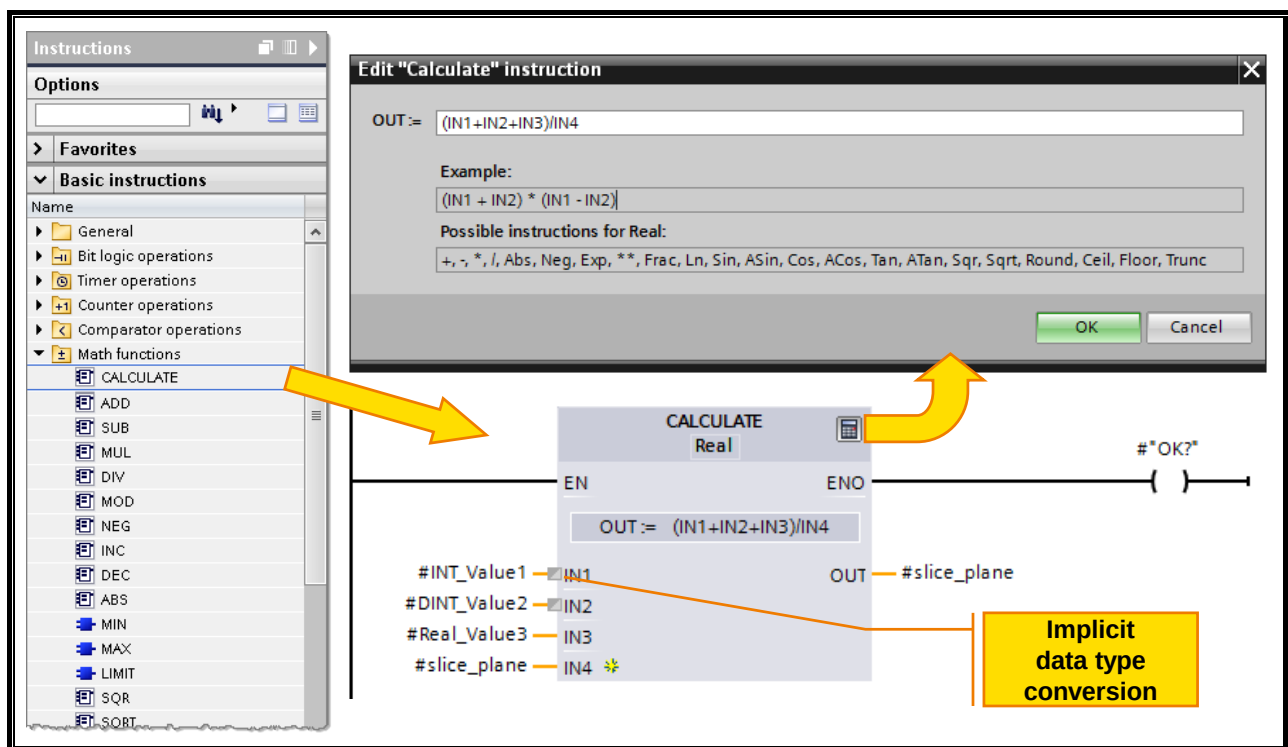
In S7-1200/1500, with the math instructions in LAD and FBP, data type conversions are implicit, that is, possibly necessary data type conversions are integrated in the math instruction.

The example shows an integer addition of "INT_Value" and "REAL_Value". For this, "REAL_Value" is implicitly converted from REAL to INT and then added to "INT_Value". The integer result of the addition is converted in an implicit type conversion from INT to DINT and assigned to "DINT_Sum".

Attention

For data type conversions into a data type with a smaller value range – in the example, for example, the conversion of the value of the variable "REAL_Value" from REAL to INT – is in this respect risky, in that an overflow leads to an invalid conversion result. Accordingly, the result of the integer addition would then also be invalid. By evaluating the ENO output (= '1' when error-free), errors that result in this way can be discovered.

5.8.7. LAD/FBD: CALCULATE Box

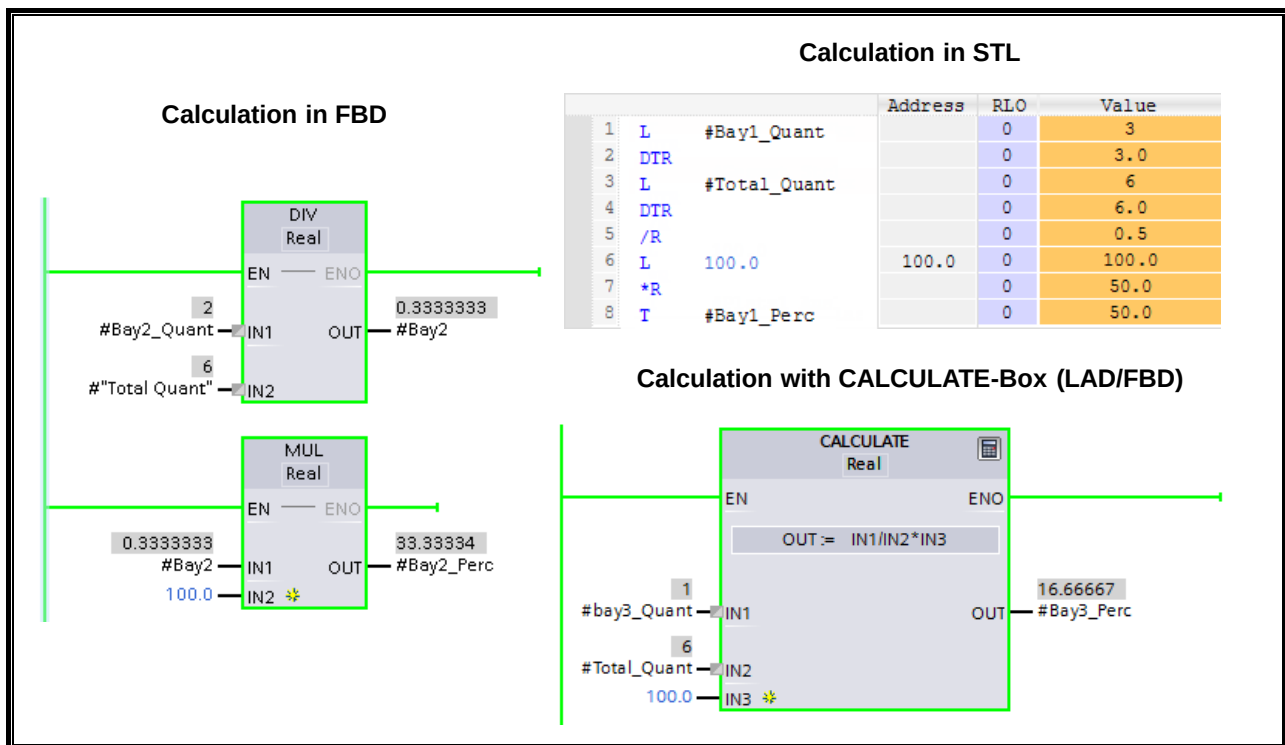


Calculate Box

With the Calculate Box, calculations can be combined which contain several different math operations.

The implicit data type conversion is available at the inputs and outputs of the box and a check can be made through the ENO output as to whether errors occurred in the calculation, such as, overflows in the type conversions or the math operations, or, that the result of the calculate box is error-free.

5.8.8. Sample Calculation: Utilization Bay 1



Calculating the Utilization of Bay 1

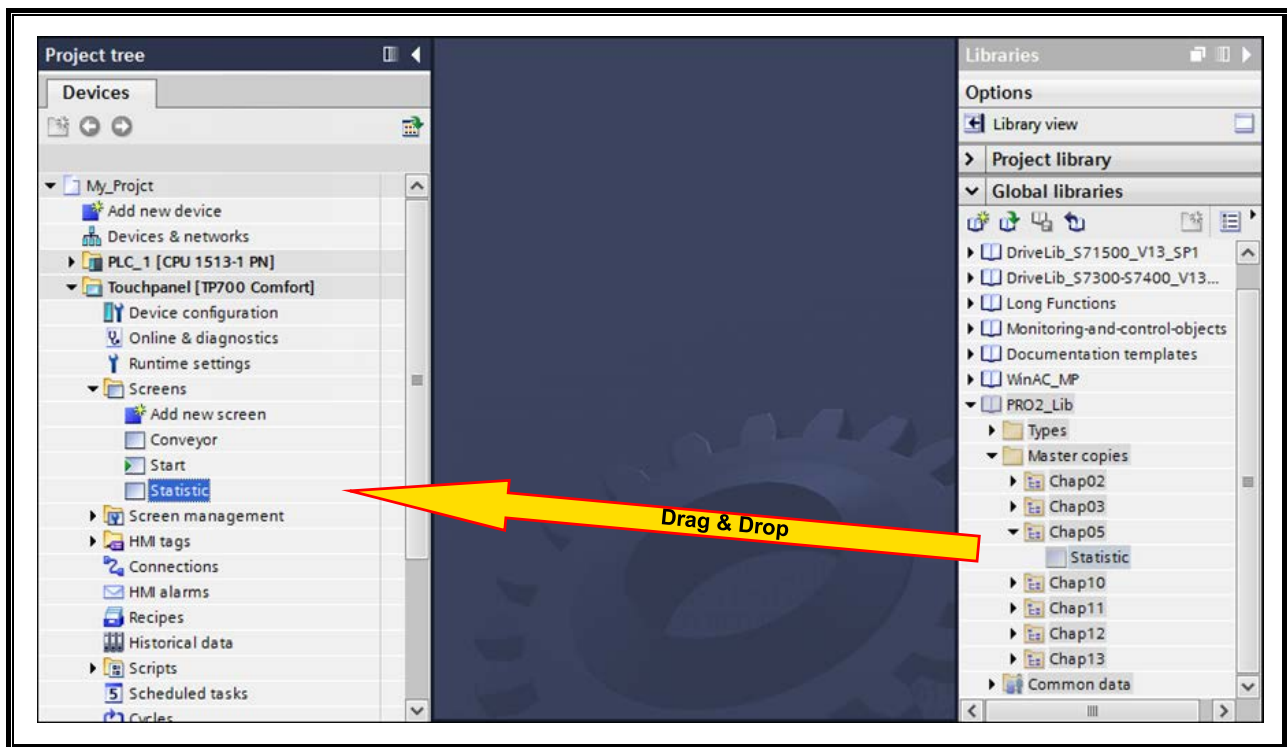
The utilization of Bays 1, 2 and 3 is to be determined, that is, what percentage of the transported parts was transported to each of the bays has to be calculated. For this, the bay quantities (number of parts) have to be divided by the total quantities and the result (quotient) has to be multiplied by 100.0 in order to get "Percent" as the unit.

Data type conversion necessary:

Both the bay quantities and the total quantities are stored in variables of the data type INT. The utilization of the bays in percent is, however, to be calculated as a REAL value, whereby data type conversions are necessary.

For the programming in STL, the necessary data type conversions have to be explicitly programmed. In the graphic programming languages, implicit data type conversions can also be used.

5.8.9. Exercise 4: Copying the TP Screen "Statistic" from the Library



Task

For the subsequent commissioning, the prepared touchpanel screen "Statistic" is to be copied from the Global Library "PRO2_Lib" into your project.

What to Do

1. In the "Global libraries", open the library <Drive>:\Archives\TIA\PRO2_Lib
2. Using drag & drop, copy the TP screen "Statistic" into the touchpanel configuration of your project.
3. Save your project.

5.8.10. Exercise 5: Statistics Data "FB_Statistics" (FB18)

The screenshot displays the SIMATIC Manager interface. On the left, the 'Network 5: Statistics' ladder logic shows the 'FB_Statistics' (FB18) block. The block's inputs are '%Q0.1' (Operation), 'DB_Conveyor', 'Bay1_occupied', 'DB_Conveyor', 'Bay2_occupied', 'DB_Conveyor', and 'Bay3_occupied'. The outputs are 'Bay1_Quant', 'Bay2_Quant', 'Bay3_Quant', 'Total_Quant', 'Bay1_Utiliz', 'Bay2_Utiliz', and 'Bay3_Utiliz'. A red dashed box highlights the 'DB18' data block, and a red arrow points from it to the 'SITRAIN' table.

The 'SITRAIN' table shows the following data:

	Bay 1	Bay 2	Bay 3	total
No. of parts	5	10	4	19
Utilisation [%]	26	53	21	100

Below the table, a 'Weight' section shows 'current' as 73, 'Sum' as 0, 'max' as 0, and 'min' as 0. A red note states 'Not yet implemented !'.

The 'OBJECT_47 [I/O field]' properties window is open, showing the 'General' tab. The 'Tag' is 'Bay1_Quant' and the 'PLC tag' is 'DB_FB_Statistics.Bay1_Quant'. The 'Address' is 'Int'.

The 'FB_Statistics' (FB18) block is configured with the following steps:

- Initialize utilizations Bay 1, 2, 3 when #Operation ON
- Acquire no. of parts Bay 1, 2, 3 with IEC counters
- Calculate total quantity
- Total quantity <= 0 (Yes/No)
- Calculate utilization Bay 1, 2, 3

Acquiring Statistical Data "FB_Statistics" (FB18)

When "P_Operation" (Q0.1) is switched on, the transported parts are counted for each bay. The total quantity and the number of parts of the individual bays as well as their utilization in percent are displayed on the touchpanel in the screen "Statistic".

All values mentioned are initialized or overwritten with 0 every time "P_Operation" (Q0.1) is switched on.

What to Do

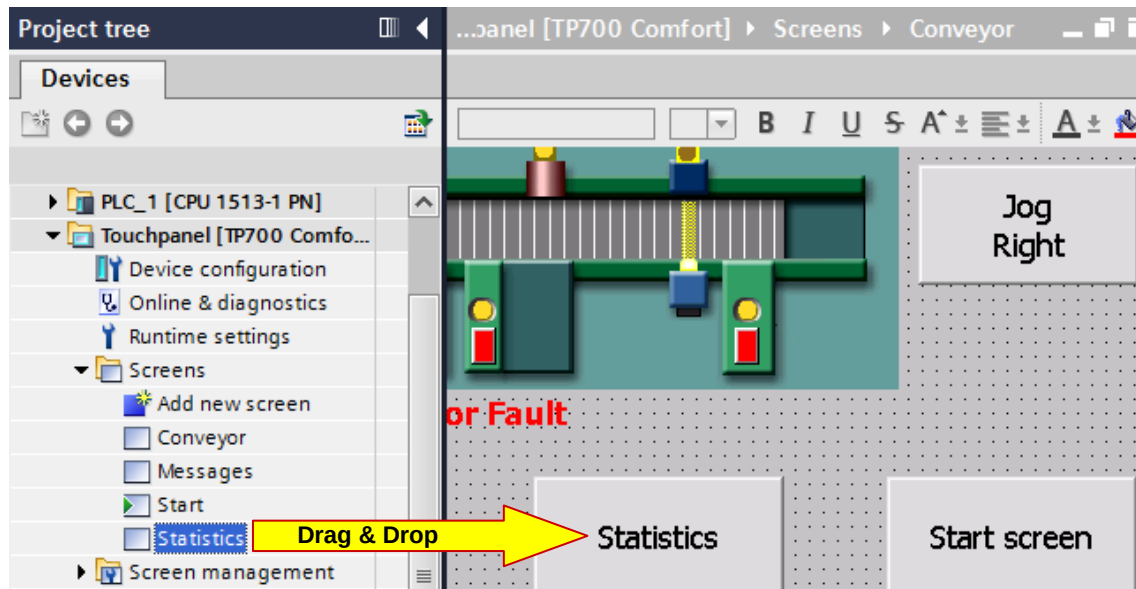
1. Insert the new block "FB_Statistics" (FB18) and provide it with the interface shown in the picture.
2. In the block, program the counting of the arrived parts for every bay. For this, use IEC counters that are reset when #Operation is switched on, and assign the data blocks DB1, 2, and 3 as instances to these.
3. Calculate the total quantity as well as the utilizations of the individual bays (see previous page) and assign the values to the appropriate output parameters (see picture).
4. Program the initialization of all values (also the resetting of the IEC counters) when #Operation is switched on.
5. In OB1, program the call of "FB_Statistics" (FB18) and parameterize its interface with the variables shown in the picture.

Continued on the next page

6. Connect the output fields of the touchpanel screen "Statistic" with the output parameters of the FB or with the relevant variables in the instance-DB "DB_FB_Statistics" (DB18) of "FB_Statistics" (FB18).
7. Connect the output field "Weight current" with the variable "DB_Conveyor".Weight.
8. In the screens "Start" and "Conveyor", program the screen navigation, that is, a button for selecting the screen "Statistic".

Proceed as follows:

Open the "Conveyor" or "Statistic" screen and then, from the "Screens" touchpanel container, drag the screen "Statistic" into the screen open in the Editor. A button or the navigation to the screen "Statistic" is then automatically created.



9. Expand the TP screen "Conveyor" by adding an output field so that, here as well, the total quantity is displayed (as shown in the picture of the next exercise).
10. Download the modified programs into the CPU or the touchpanel and check the functions.
11. Save your project.

Contents

6.	Functions, Function Blocks and Multiple Instances	6-2
6.1.	Task Description: Programming Re-usable Blocks	6-3
6.2.	Blocks for Structured Programming	6-4
6.2.1.	Parameter Declaration in Functions and Function Blocks	6-5
6.2.2.	Properties of Functions	6-6
6.3.	Properties of Function Blocks	6-7
6.4.	Local and Global Operands	6-8
6.4.1.	Management of the Local Data Stack	6-9
6.4.2.	The Use of Local, Temporary Variables	6-10
6.5.	Instance and Multiple Instance Data	6-11
6.5.1.	Example: Storing IEC Counter/Timer Instances in a Global DB	6-12
6.5.2.	Exercise 1: "FB_Statistics" (FB18): Creating IEC Counter Instances in a Global DB	6-13
6.6.	Task Description: Programming "FB_Fault" (FB17) as IEC-conform and with that Re-usable	6-14
6.6.1.	Instance Formation of Function Blocks	6-15
6.6.2.	Structure of the Multiple Instance Model	6-16
6.6.3.	Exercise 2: Programming "FB_Fault" (FB17) as IEC-conform and with that Re-usable	6-17
6.6.4.	Exercise 3: Programming "FB_Statistics" (FB18) as Re-usable	6-18

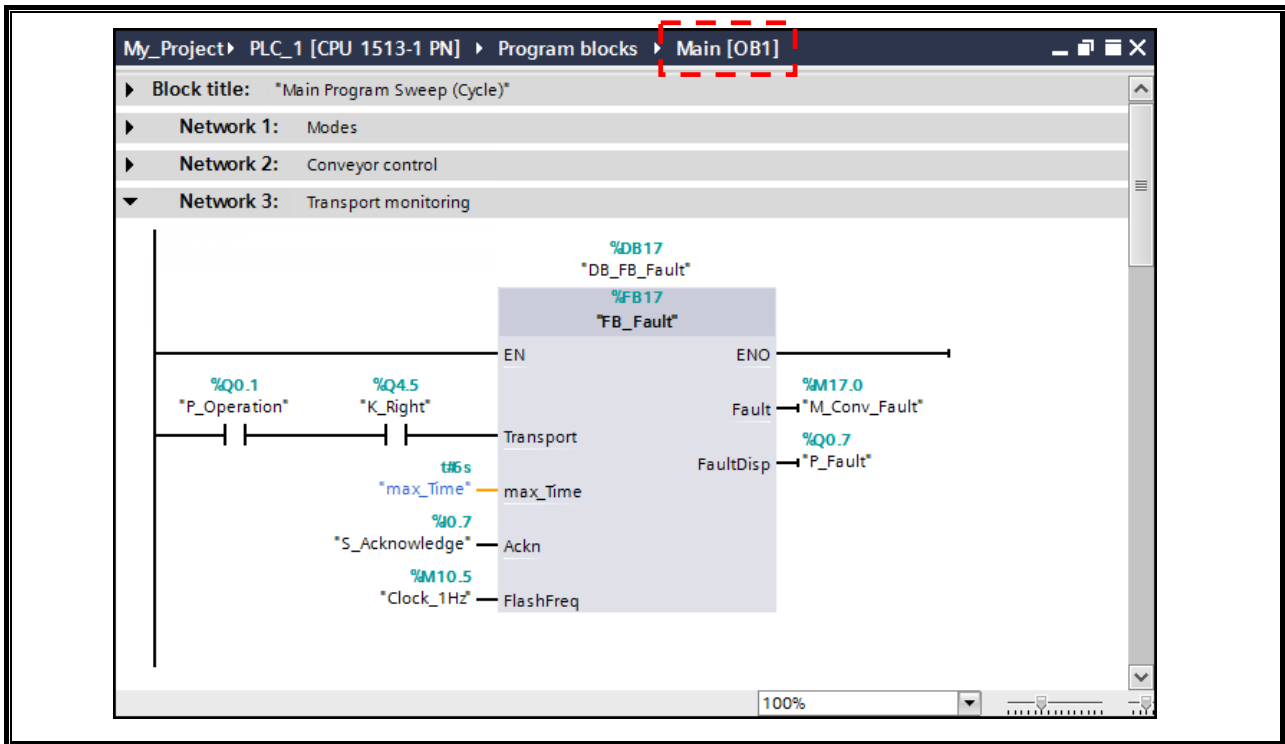
6. Functions, Function Blocks and Multiple Instances

At the end of the chapter the participant will ...

- ... understand the properties of functions and function blocks and be able to apply them
- ... understand the term “re-usability” and be able to implement it
- ... understand the concept of multiple instances and be able to use them
- ... be able to program re-usable blocks



6.1. Task Description: Programming Re-usable Blocks



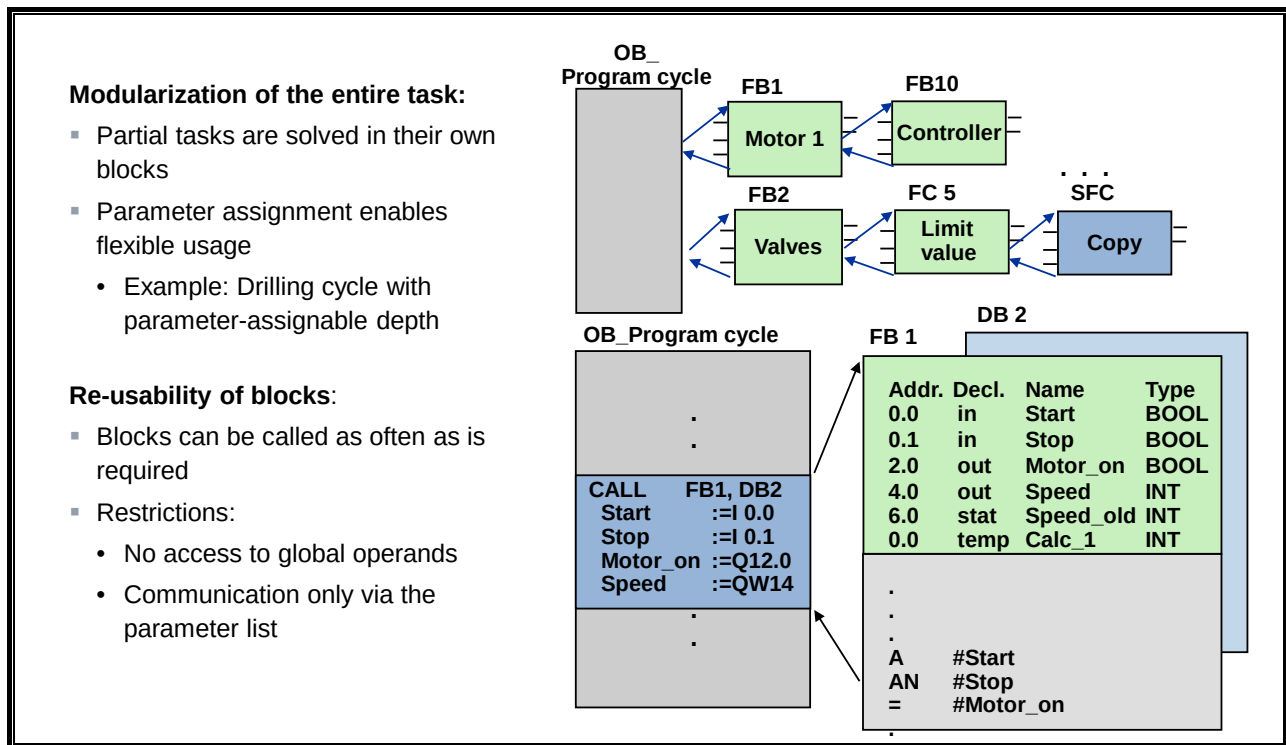
Task Description

According to the IEC61131-3 standard, global operands must not be used within FCs and FBs, that is, no inputs, outputs, memory bits, data block variables, SIMATIC timers / counters. Allowed are only local operands, that is, only parameters and local variables (tags).

The communication of the block with its "outside world" occurs exclusively through the interface of the block or only through the input, output and in/out parameters. This also means that the block works with no other operands than with those that are passed formal operands during the block call.

This ensures that a block is "re-usable", that is, can also be integrated in other programs unchanged.

6.2. Blocks for Structured Programming



Modularization of the Entire Task

Abstraction is the basis for solving complex problems, in which we concentrate on the fundamental aspects of a program in every abstraction level and ignore all the details that are not essential. Abstraction helps us to divide complex tasks into partial tasks which can then be solved on their own.

Structured Programming

STEP7 supports this concept of modularization with its block model. The partial tasks that result from the division of the entire task are assigned to blocks in which the necessary algorithms and data for solving the partial problems are stored.

STEP7 blocks such as functions (FC) and function blocks (FB) can be assigned parameters so that the concepts of structured programming can be implemented with them. This means:

- Blocks for solving partial tasks implement their own data management with the help of local variables.
- Blocks communicate with the "outside world", that is, with the sensors and actuators of the process control or with other blocks of the user program, exclusively through their block parameters. No access to global operands such as inputs, outputs, bit memories or variables in DBs can be made from within the instruction section of blocks.

Advantages

Structured programming has the following advantages:

- The blocks for the partial tasks can be created and tested independent of one another.
- With the help of parameters, blocks can be designed so that they are flexible. That way, for example, a drilling cycle can be created that has the coordinates and the depth of the drilling hole passed on to it by means of parameters.
- Blocks can be called as often as is required in different locations with different parameter data records, that is, they can be reused.
- "Re-usable" blocks for special tasks can be delivered in pre-designed libraries.

6.2.1. Parameter Declaration in Functions and Function Blocks

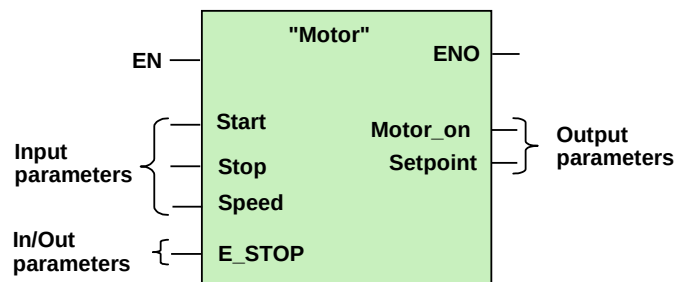
Parameters are the channels for transferring information:

- Input parameters: Caller -> Block
- Output parameters: Block -> Caller
- In/Out parameters: Caller <-> Block

Parameters form the "shell" of a block:

- Parameters are in the code section like "local" variables
- Parameters can have any data type
- Data type check during call (exception: POINTER, ANY)
- Call interface is language-independent
- PLC languages can be mixed

Addr.	Decl.	Name	Type	Initial value	
0.0	in	Start	BOOL	FALSE	
0.1	in	Stop	BOOL	TRUE	
2.0	in	Speed	INT	0	
4.0	out	Motor_on	BOOL	FALSE	
8.0	out	Setpoint	INT	0	
10.0	inout	E_STOP	BOOL	FALSE	
...	stat	...	...	...	
...	temp	...	...	...	
	constant	...	...	...	



Overview

Parameters are used as channels for transferring information between the calling block and the called block. The symbolic names, data types and, if necessary, initial values of parameters are established in the declaration table.

Declaration Types

The type of parameter indicates the direction of the data transfer:

- IN Input parameters (FB, FC): They are used to pass information from the calling block to the called block. Only a read-only access to the input parameters is possible within the called block.
- OUT Output parameters (FB, FC): They are used to pass information (results) from the called block back to the calling block.
- INOUT In/Out parameters (FB, FC): In/Out parameters are used to transfer information in both directions. A read and write access is possible to in/out parameters.

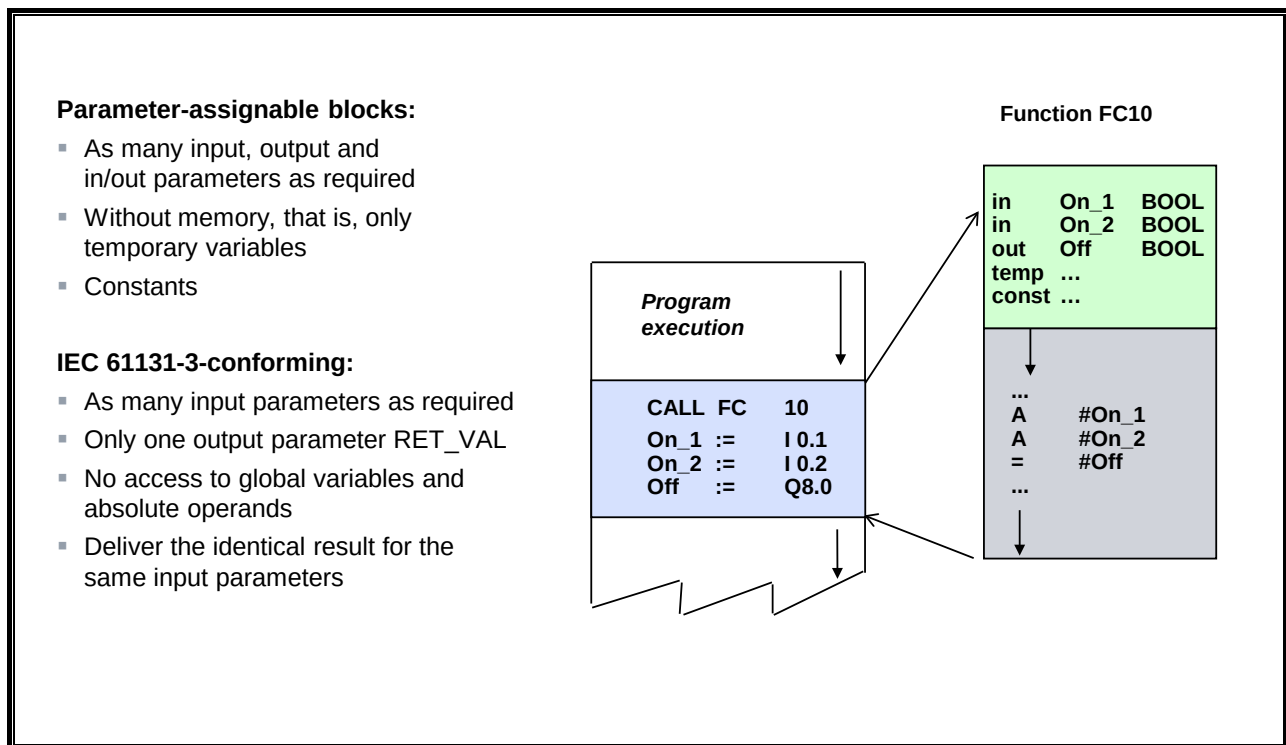
Name and Type

Just like local variables, parameters have a symbolic name and a type (data or parameter type). Parameters can be used in the code section of a block in the same way as local variables of the same data type. For this reason, parameters are also called formal parameters within a block.

Block Calls

When a block is called, relevant actual parameters must (FC) or can (FB) be assigned to the formal parameters of a block. In order to prevent a misinterpretation (regarding the data type) or erroneous use of the actual operands that are passed, the PLC Editor checks that the actual operand created is exactly the same type as the formal parameter when the block is called (Exception: POINTER and ANY). The type check and the parameter passing mechanism is language-independent. This guarantees that blocks that were created with different PLC Editors (STL, LAD, FBD, SCL, etc.) can call each other.

6.2.2. Properties of Functions



Overview

Functions represent parameter-assignable blocks without memory. In STEP 7 they can have as many input parameters, output parameters and in/out parameters as are required. Additionally, constants that are local in the block can be created for S7-1200/1500.

Functions have no memory; no separate, permanent data area for storing results exists. Temporary results that occur during function execution can only be stored in the temporary variables of the respective local data stack.

Application

Functions are primarily used when function values are to be returned to the calling blocks. (for example, mathematical functions, single control with binary logic operation).

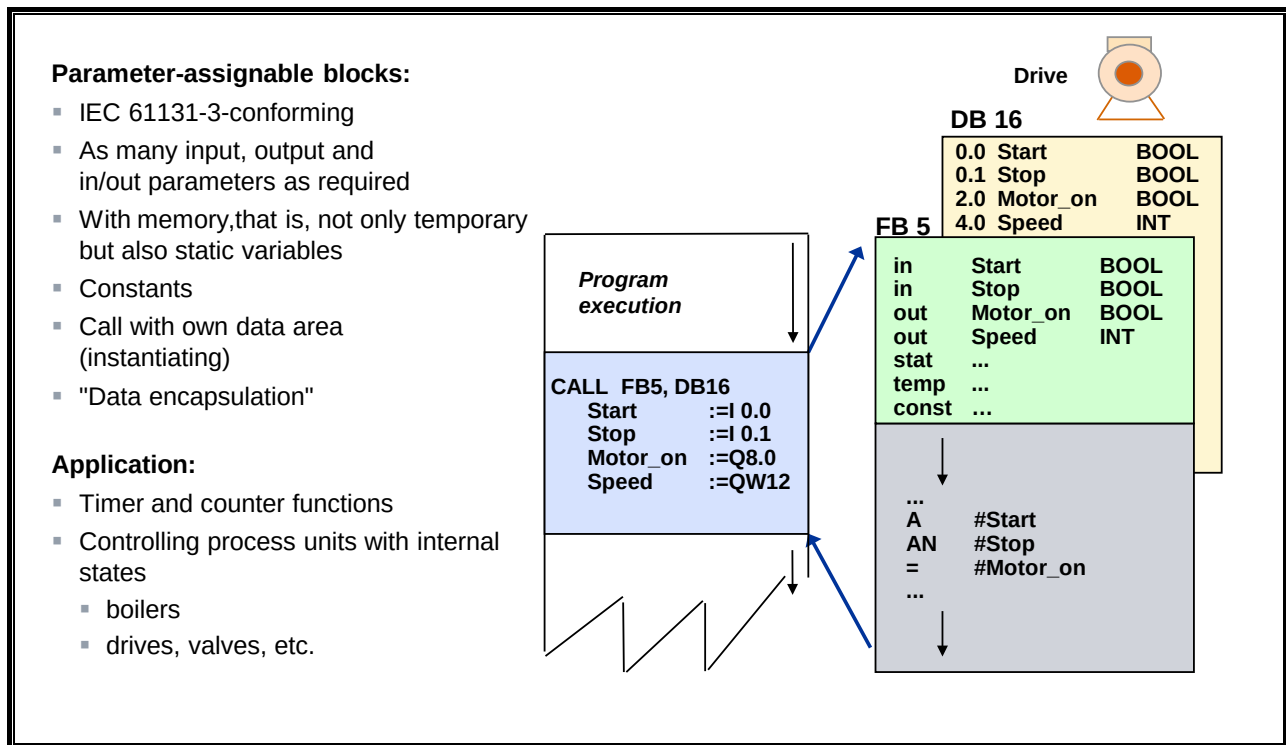
IEC-61131-Conforming Functions

If IEC 61131-3-conforming functions are to be created, then the following rules must be observed:

- Functions can have as many input parameters as is required. They can, however, only return one result to the output parameter RET_VAL.
- Global variables can neither be read nor written within functions.
- No instances of function blocks can be called within functions.
- Because of the missing "memory", the returned result of a norm-conforming function is solely dependent on the values of the input parameter. For identical values of the input parameter, a function also returns the identical result.

It is therefore up to the programming person to create norm-conforming functions or to do the block programming and structuring in STEP 7 as it is in STEP 5.

6.3. Properties of Function Blocks



Overview

Function blocks (FB) are blocks of the user program and represent logic blocks with memory according to the IEC Standard 61131-3. They can be called by OBs, FBs and FCs.

In function blocks, as many input, output and in/out parameters, static and temporary variables as are required can be declared, as well as constants that are local in the block for S7-1200/1500.

Unlike FCs, FBs are instantiated, that is, an FB is assigned its own private data area in which the FB can "remember" process states from call to call, for example. In the simplest form, this private data area is its own DB, the so-called instance DB.

"Memory"

The programming person has the opportunity to declare static variables in the declaration section of a function block. The function block can "remember" information from call to call in these variables.

The ability of a function block to "remember" information over several calls is the essential difference to functions.

Application

With the help of this "memory", a function block can implement counter and timer functions or control process units, such as processing stations, drives, boilers etc., for example.

In particular, function blocks are well suited for controlling all those process units whose performance depends not only on outside influences but also on internal states, such as processing step, speed, temperature etc.

When controlling such units, the internal status data of the process unit are then copied to the static variables of the function block.

6.4. Local and Global Operands

Global Operands (valid in the entire program)	Local Operands (only valid in one block)
• PII / PIQ • I / O peripherals • Bit memories • Variables in DBs • S5-Timers *) • S5-Counters *) • Constants *) only for S7-300/400/1500/WinAC	Formal Parameters • interface for data exchange between calling and called block • temporary storage in the L-stack for FCs or storage in the IDB for FBs • can be used in FCs / FBs
	Temporary Variables • are overwritten after the block is executed • temporary storage in the L-stack • can be used in OBs / FCs / FBs
	Static Variables • retain their value after the block is executed • permanent storage in IDBs • can <u>only</u> be declared in FBs
	Constants • read-only and symbolic access • no memory usage • can be used in OBs / FCs / FBs

Global Operands

Global operands are valid throughout the entire S7 program. Accordingly, every logic block (OB, FC, FB) can access these operands.

Global operands include the traditional PLC operands - inputs, outputs, bit memories, SIMATIC timers, SIMATIC counters, constants and variables - which are declared in global data blocks.

Local Operands

Local operands are only valid in the block in which they were declared in the declaration part. Accordingly, only this block can access them.

- Temporary Variables

Temporary variables can be declared in every logic block (OB, FC, FB) and are managed in the local data stack of the CPU. Accordingly, they only retain their values while the block is being executed. For that reason, it is important that in the current cycle, a write access must have taken place on the temporary variable in the block before a read access can take place. They are unsuitable, for example, as auxiliary variables for edge evaluations or to store quantities. They are, in fact, used to store intermediate results, such as, for complex calculations or format conversions.

- Static Variables

Static variables can only be declared in function blocks (FBs) and are stored in the associated instance data block (IDB). Accordingly, these variables retain their value even after the FB is executed.

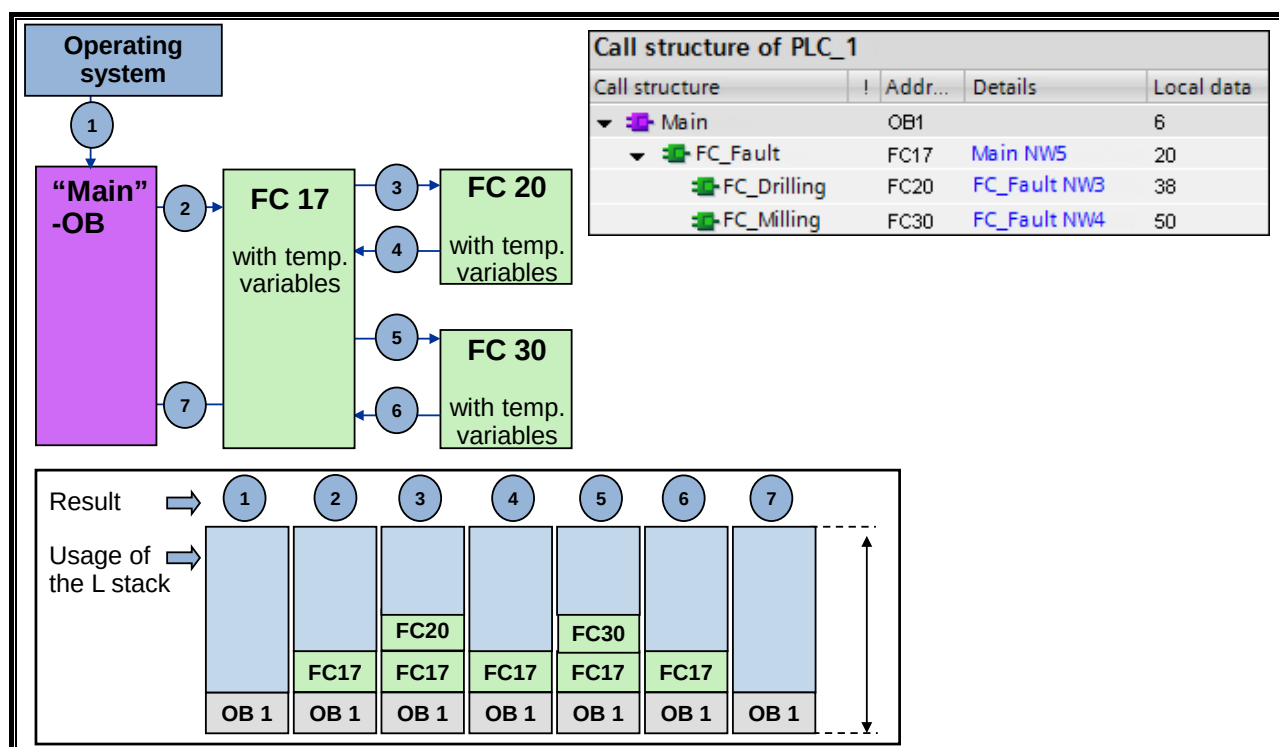
- Formal Parameters

Formal parameters form the interface between calling and called block. They are used to realize a data exchange between the calling and the called block.

- Constants

Constants are fixed values which have a read-only access and which do not take up any memory space.

6.4.1. Management of the Local Data Stack



Total Usage of the Local Data Stack (L-Stack)

For every program execution level or priority class (such as, "Main" with all blocks that are called in it), a separate local data stack is reserved. That is, a segment of defined size is reserved on the L stack of the CPU (allocation or reservation of memory space).

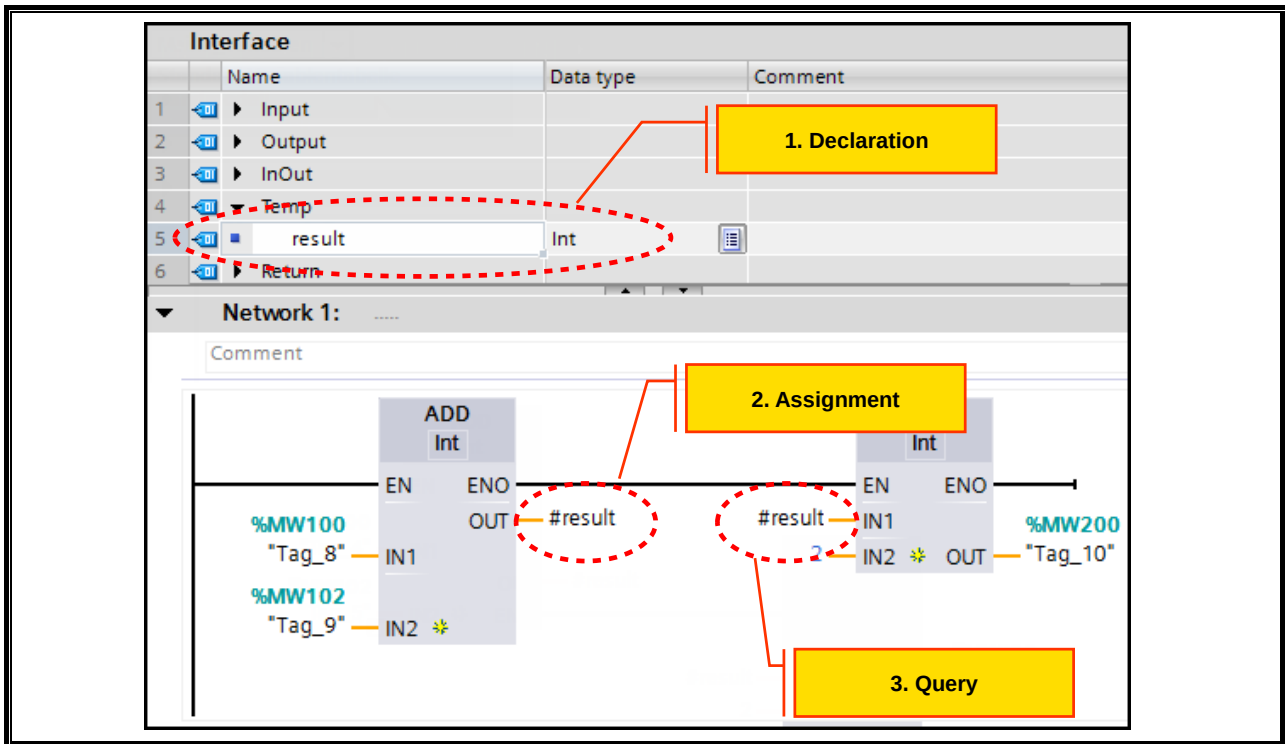
The local operands of "Main"-OB as well as the local, temporary variables of the blocks (FCs and FBs) called in or by OB 1 are stored in this local data stack.

You can use the "Reference Data" tool to display the "Program Structure" to see to what extent an S7 program puts a burden on the local data stack. The reference data are dealt with in more detail in another chapter.

Note

If the (CPU-specific) maximum number of local data is exceeded (overflow of the L stack) during program execution in the CPU, the CPU goes into the STOP state. "STOP caused by error when allocating local data" is entered as the cause of error in the diagnostics buffer.

6.4.2. The Use of Local, Temporary Variables



Declaration

The variables are defined in the declaration part of the block. The name of the variable and the data type must be specified.

You cannot predefine the variable with an initial value. After you have saved the block, the relative address (storage location) of the variable in the L stack is displayed in the "Address" column.

Access

At the beginning of block execution, all temporary variables have an indefinite (undefined) value. When working with temporary variables, you must therefore make sure that the variable is first of all assigned a defined value before it is queried.

In the example, the result of Add(ition) is assigned to the temporary variable #result before it is then queried at the Mul(tiplication).

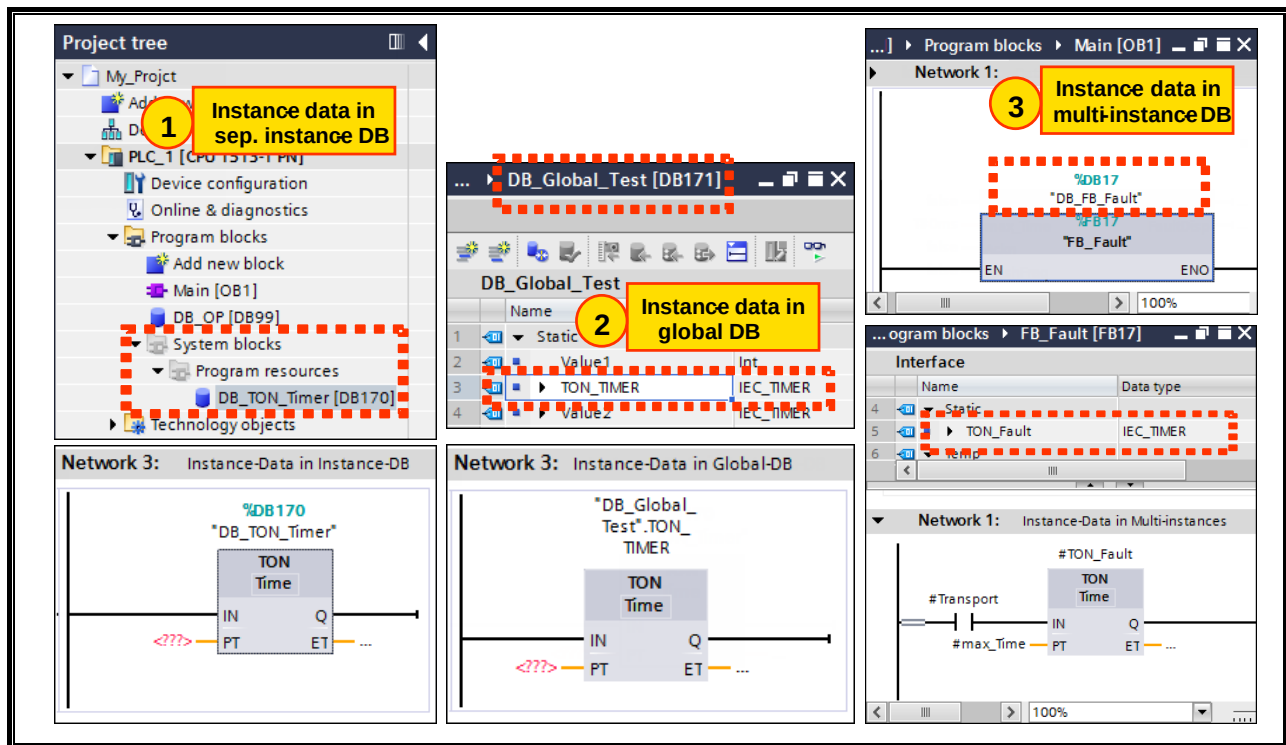
You can also access temporary variables absolutely (such as T LWx). You should, however, try to avoid this since the program is more difficult to read.

Note

Operands that begin with the # special character are local operands (parameters or local variables) that must be declared in the declaration part of the block. Local operands are only valid and usable in the block in which they were declared.

The Program Editor automatically inserts the # character.

6.5. Instance and Multiple Instance Data



Instantiation

The concept of the instantiation of function blocks has great meaning and forms the fundamental difference to functions. The creation of tags within a high level language, such as, "C" specifying the tag name and the data type is called "instantiation" or "instance formation".

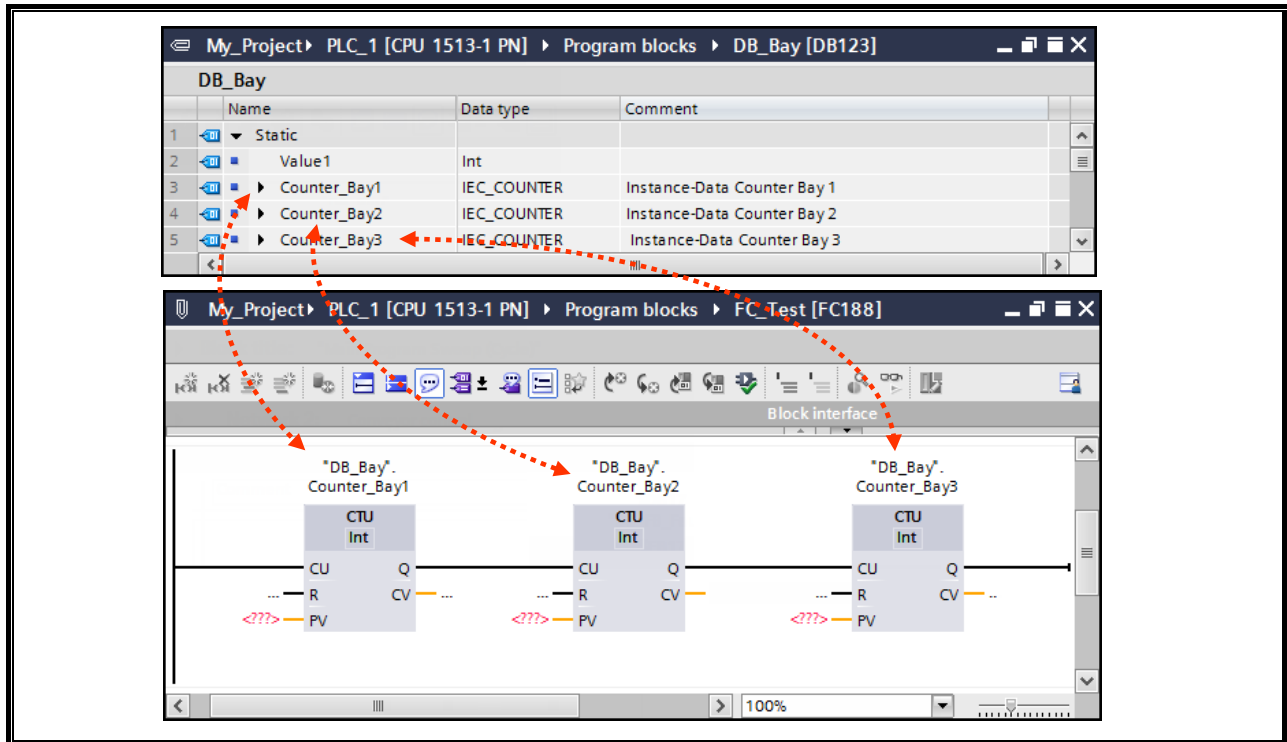
Just as tags, function blocks are also "instantiated". Only through the assignment of its own data area in which the values of the block parameter as well as the static variables are stored does a function block become an executable unit (FB instance).

The control of a physical process unit, such as, a drive or a boiler then takes place with the help of an FB instance, that is, a function block with associated data area. The relevant data for this process unit is then stored in this data area.

The formation of an FB instance, that is, the assignment of its own memory area during an FB call can occur in two ways in STEP 7:

- 1 by explicitly specifying a so-called instance data block during the call of the function block.
- 2 by explicitly declaring instances of an IEC counter or an IEC timer function within a global data block.
- 3 by explicitly declaring instances of a function block within a higher-level function block (also see the following pages, multiple instance model).

6.5.1. Example: Storing IEC Counter/Timer Instances in a Global DB



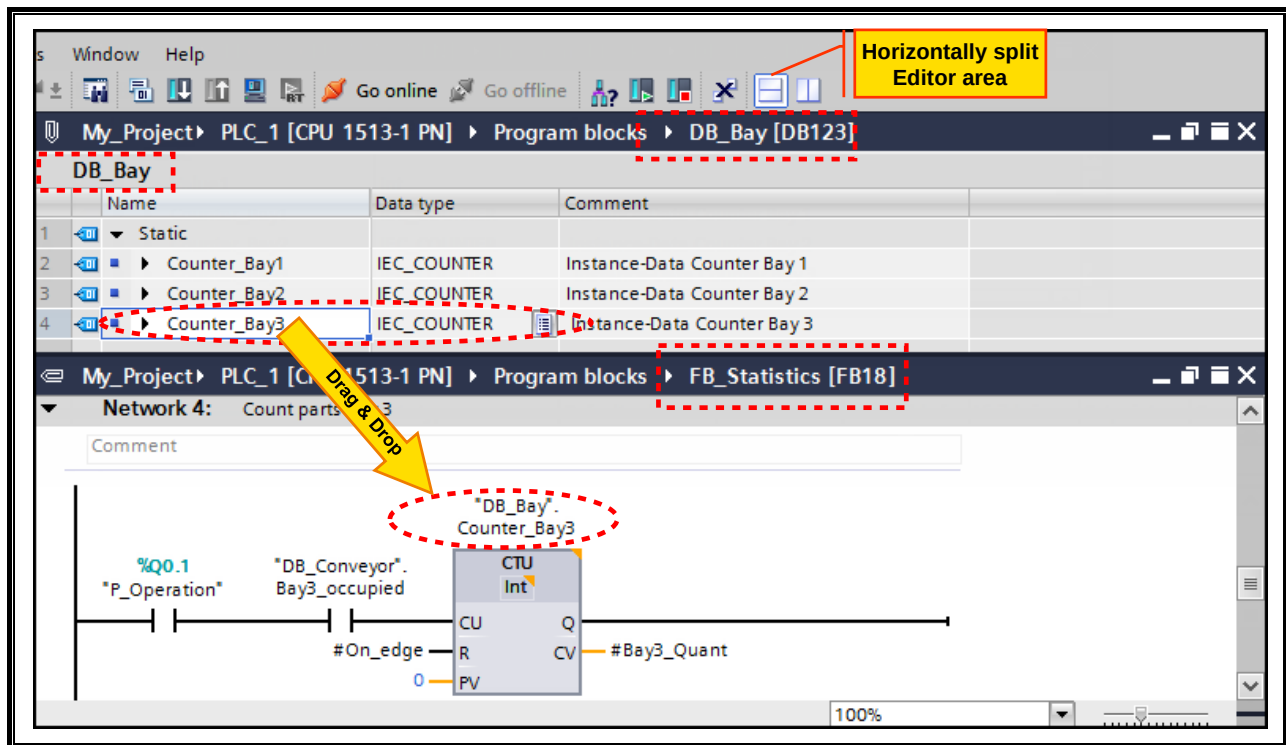
Application

STEP7 provides the possibility of creating instances of IEC counters and timers in a global DB.

With that, the instance data of IEC counters and timers used within a global data block can also be gathered within a function (FC) (see picture).

However, it must be noted that because of this the calling FC (in the example "FC_Test") is not IEC-conform and with that is not re-usable, since even the passing of IEC counter instances, represents the use of global operands.

6.5.2. Exercise 1: "FB_Statistics" (FB18): Creating IEC Counter Instances in a Global DB



Task

In "FB_Statistics" (FB18), using drag & drop, the separate single instance DBs are to be replaced with instances that are declared in one global "DB Bay" (DB123).

Note:

In the example shown, the instances of the IEC counters are stored in one global data block as an example, even though the multiple instance concept could also be used as an alternative within the FB.

In the course of the next exercise, you will then become familiar with this more sensible approach.

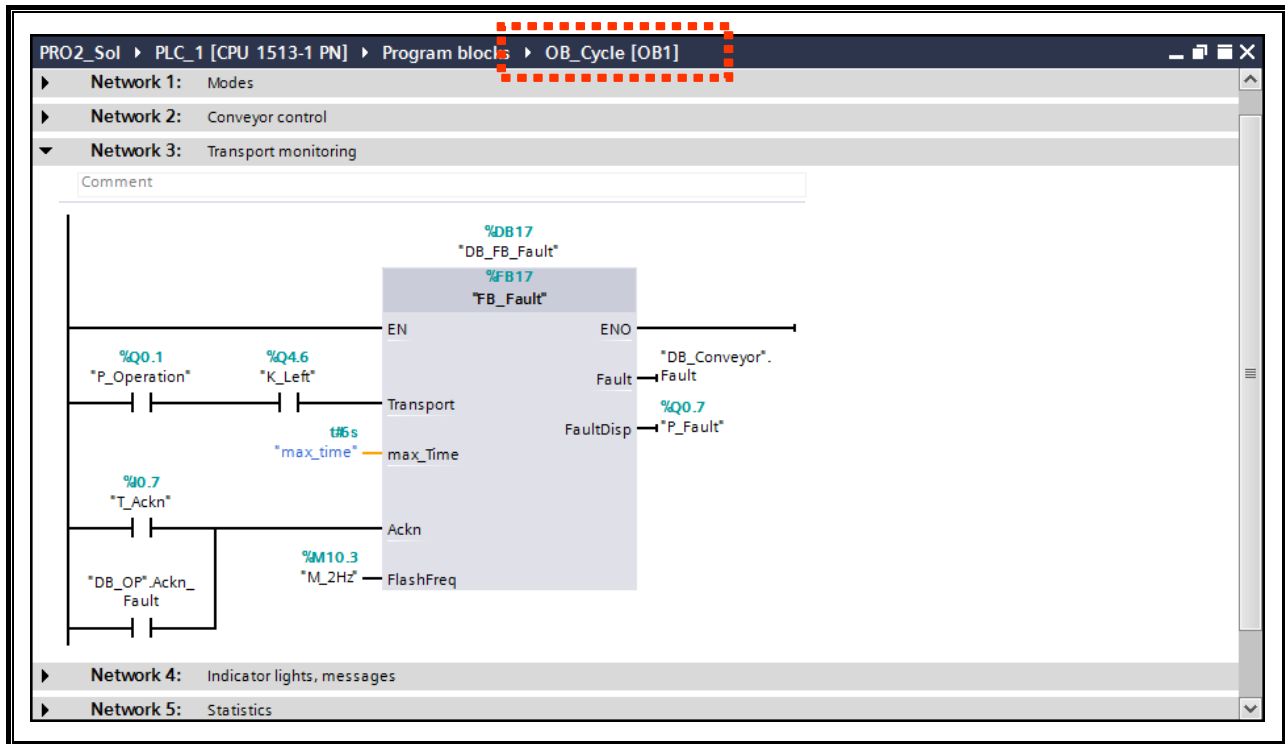
What to Do:

1. Create the new "DB_Bay" (DB123) with the variables shown in the picture.
2. In "FB_Statistics" (FB18), replace the separate, single instance DBs with the instances that you previously declared in "DB_Bay" (DB123).

To do so, horizontally split the Editor area (see picture) so that the two blocks "DB_Bay" (DB123) and "FB_Statistics" (FB18) can both be displayed at the same time. Using drag & drop, replace the single instance DBs with the instances that you declared in "DB_Bay" (DB123).

3. Download all modified / newly generated blocks into the CPU and test the program.
4. Save your project.

6.6. Task Description: Programming "FB_Fault" (FB17) as IEC-conform and with that Re-usable



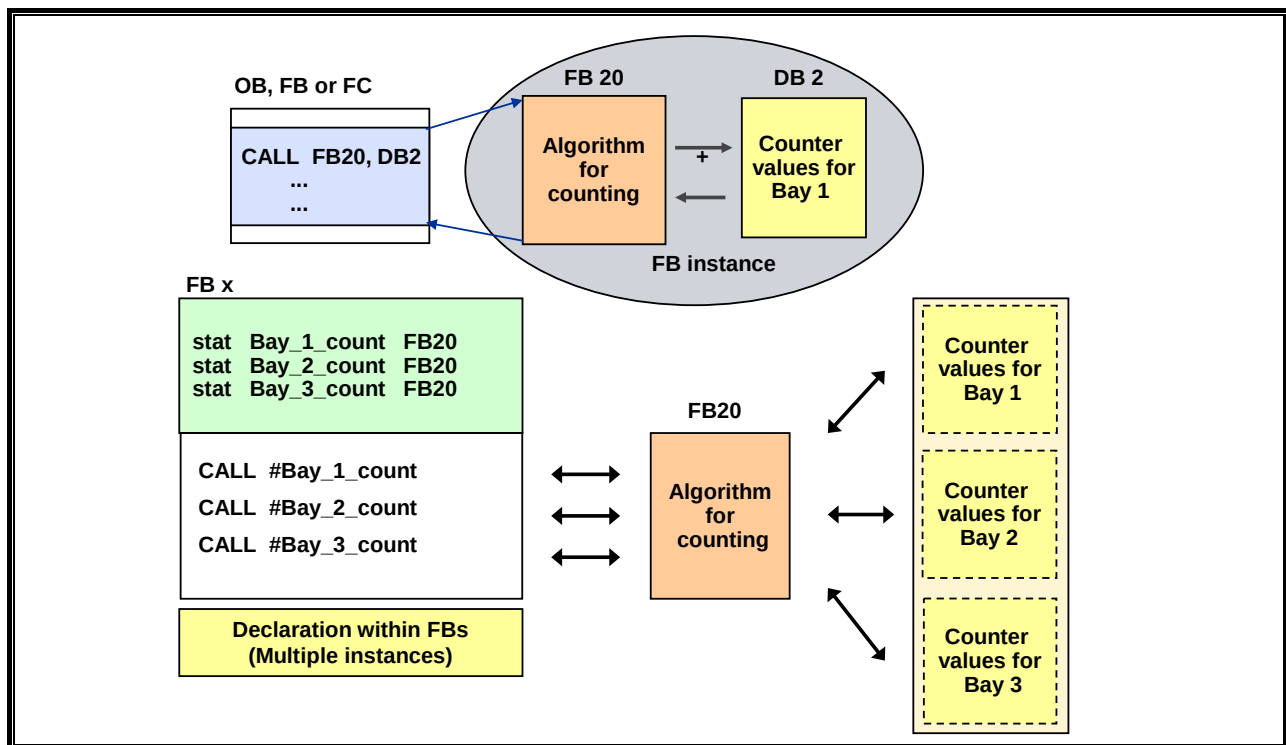
Task Description

According to the IEC61131-3 standard, no global operands can be used within FCs and FBs, that is, no inputs, outputs, bit memories, data block variables, SIMATIC timers or SIMATIC counters - only local operands are allowed, that is, only parameters and local variables.

The block's communication with its "outside world" takes place exclusively via the block's interface or only via the input, output and in/out parameters.

The "FB_Fault" (FB17) block is to replace the "FC_Fault" (FC17) block and fulfill exactly the same functions and thereby completely fulfill the named requirements of IEC61131-3.

6.6.1. Instance Formation of Function Blocks



What is an Instance?

The concept of instantiating function blocks has great importance and makes up the essential distinctive criterion to the functions. The setting up of variables within a high level language such as "C" under declaration of variable name and data type in the declaration is called "instantiating" or "instance forming".

Just like variables, function blocks are also "instantiated". Only through this 'own' data area, in which the block parameter values as well as the static variables are stored, does an FB become an executable unit (FB-instance). The control of a physical process unit, such as a drive or a boiler then takes place with the help of an FB instance, that is, a function block with an assigned data area. The relevant data for this process unit are then stored in this data area.

Instantiating

You can create an FB instance, that is, the assignment of its own memory area in an FB call, in two ways in STEP 7:

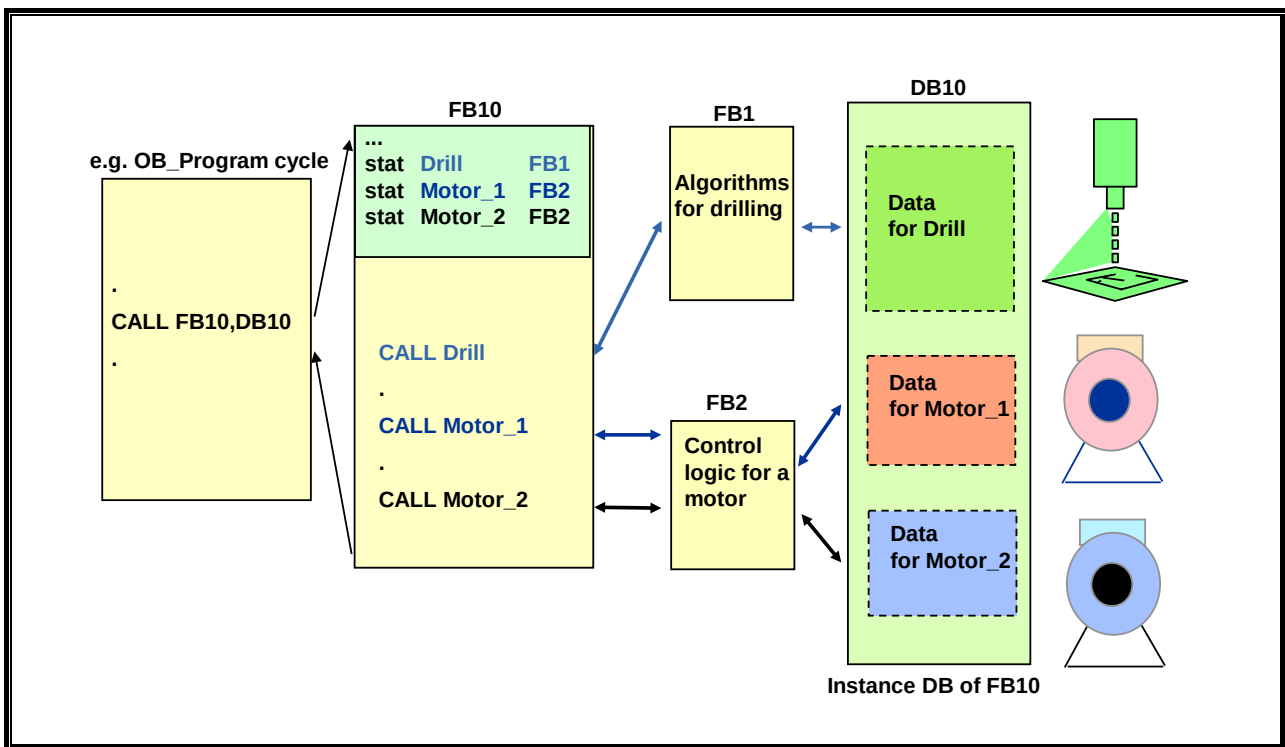
- by explicitly specifying an instance data block when a function block is called.
- by explicitly declaring instances of a function block within a higher-level function block (multiple instance model). STEP 7 then makes sure that the data area required for the instance is set up within the data area of the higher-level FB.

Benefits

The instance concept of STEP 7 has the following benefits:

- In the call of FBs, no measures for saving and administrating local data are necessary except for the assignment of instance DBs.
- A function block can be used several times due to the instance concept. If, for example, several drives of the same type are to be controlled, then this takes place using several instances of an FB. The status data of the individual drives are stored in the static variables of the FB.

6.6.2. Structure of the Multiple Instance Model



Multiple Instance Model

In addition to instantiation of function blocks when you specify an instance DB in an FB call, STEP 7 also supports the explicit declaration of FB instances within a higher-level function block. For this purpose, instances of the called function blocks are declared with data type FB1 or FB2 using symbolic identifiers (*Drill*, *Motor_1* and *Motor_2*). This takes place in the declaration section of the calling FB 10 function block in the section "*static variable*". Within the higher-level function block, the individual instances are then called using their symbolic identifier. The higher-level FB10 function block must however be called with its own instance DB (DB10).

In the creation of the higher-level instance DB, STEP 7 makes sure that the data areas required for the individual instances are set up in the data area of the higher-level FB10.

In the call of the individual instances using the symbolic names, the CALL macro makes sure that the AR2 register is set to the beginning of the data area assigned to the instance so that the parameters and local variables of the instance are also accessed during the processing of the called function block.

Benefits

The use of the multiple instance model has the following benefits:

- The individual instances do not require their own data block every time. Within a call hierarchy of function blocks, only one instance DB is used in the call of the "outer" function block.
- The multiple instance model "welds" a function block and an instance data area into one object (FB instance), that can also be handled as one unit. The programming person does not have to take care of the management (creation, addressing) of the individual instance data areas. He must simply provide an instance DB for the "outer" FB.
- The multiple instance model supports an object-oriented programming style.

6.6.3. Exercise 2: Programming "FB_Fault" (FB17) as IEC-conform and with that Re-usable

PRO2_Sol ▸ PLC_1 [CPU 1513-1 PN] ▸ Program blocks ▸ **FB_Fault [FB17]**

Interface				
	Name	Data type	Default value	Comment
1	▼ Input			
2	Transport	Bool	false	
3	max_Time	Time	T#0ms	
4	Ackn	Bool	false	
5	FlashFreq	Bool	false	
6	▼ Output			
7	Fault	Bool	false	
8	FaultDisp	Bool	false	
9	▶ InOut			
10	▼ Static			
11	▶ TON_Fault	IEC_TIMER		
12	▶ Aux_Fault	Bool	false	
13	▶ Temp			

▼ **Network 1:** Monitor the transportation time

▼ **Network 2:** Indicator lights Conveyour fault

100%

Task

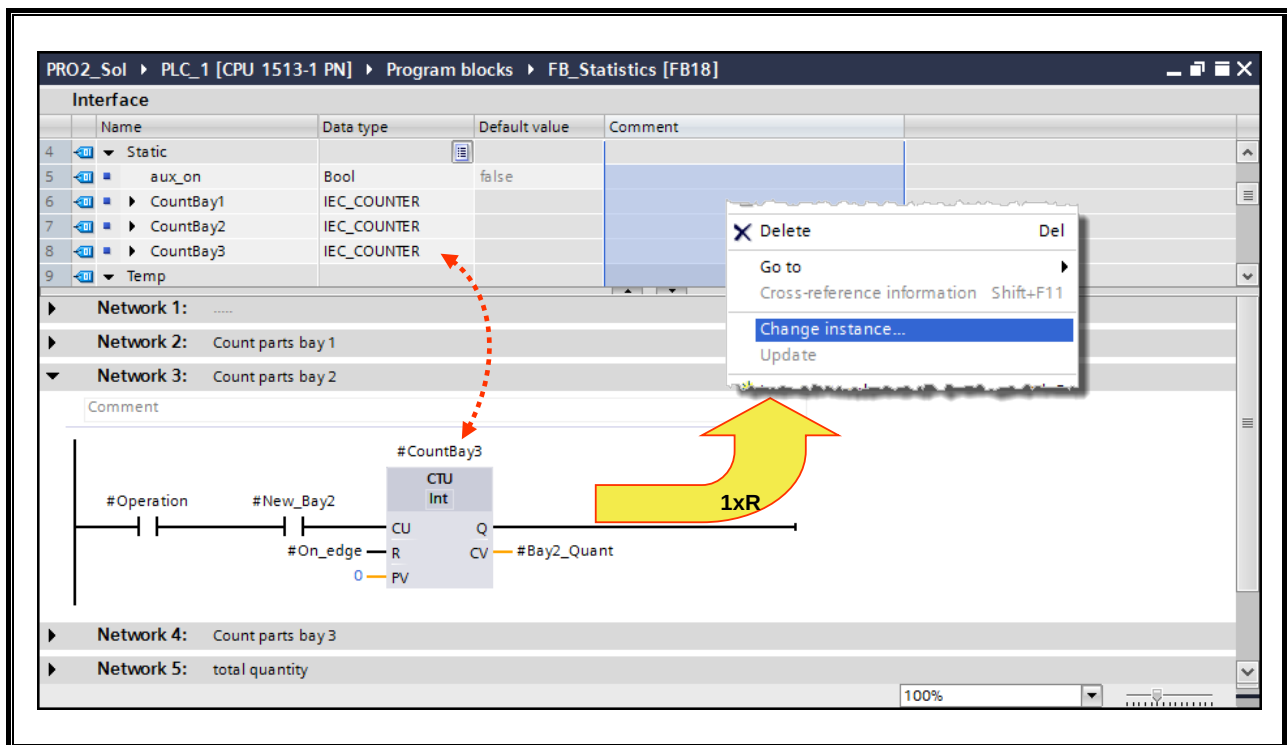
Up until now, the monitoring for time of the transport sequences from the light barrier to Bay 1, 2 or 3 is implemented in "FC_Fault" (FC17). The IEC timer function "TON" used for this is thereby passed to data block DB 104 as an instance DB.

The new "FB_Fault" (FB17) block is to replace the existing "FC_Fault" (FC17) and it is to be created in such a way that it fulfills the requirements of the IEC61131-3 and with that is given the block attribute "Block can be used as know-how protected library element".

What to Do

1. Insert the new "FB_Fault" (FB17) into your S7 program and call the "Basic instruction TON" as a multiple instance (see picture).
2. In "FB_Fault" (FB 17), declare the IN and OUT parameters shown in the picture as well as the required local variables (TEMP and/or STAT) and only use these local operands in the entire FB program.
3. Compile the block and then check whether the block has been given the attribute "Block can be used as know-how protected library element".
(Project tree, right-click on the block -> Properties -> General -> Attributes)
4. In OB1, replace the call of "FC_Fault" (FC17) with the call of "FB_Fault" (FB 17) and at the same time pass the data block "DB_FB17_Fault" (DB 17) as the instance data block.
5. Download all modified / newly generated blocks into the CPU and check how the program functions.
6. Save your project.

6.6.4. Exercise 3: Programming "FB_Statistics" (FB18) as Re-usable



Task

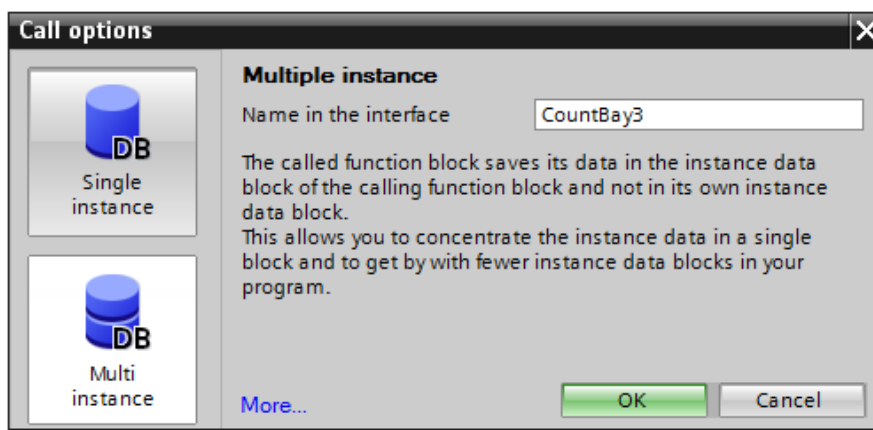
In "FB_Statistics" (FB18), the counting of the parts transported to Bay 1, 2 or 3 is implemented with IEC counters whose instance data is stored in a global data block.

As previously with the "FB_Fault" (FB17) block, "FB_Statistics" (FB18) is now also to be modified in such a way that it fulfills the requirements of the IEC61131-3 and with that is given the block attribute "Block can be used as know-how protected library element".

For this, the global variables "DB_Bay".Counter_Bay1 (2) (3) used so far in the program, that were used to store the instance data of the IEC counters, have to be replaced with multiple instances.

What to Do

1. Open "FB_Statistics" (FB18), right-click on one of the IEC counters and select the function "Change instance..." (see picture).
2. In the dialog that appears, select "Multi instance" and give it a name.



Continued on the next page

3. After you have created multiple instances for all IEC counters, compile the entire S7 program, because the call of "FB_Statistics" (FB18) in OB1 and the multiple instance DB passed there to the FB are thereby also updated.
4. Check whether "FB_Statistics" (FB18) was given the block attribute "Block can be used as know-how protected library element".
(Project tree, right-click on the block -> Properties -> General -> Attributes)
5. Delete the data block "DB_Bay" (DB123) as it is no longer required.
6. Download the entire S7 program into the CPU and check all program functions.
7. Save your project.

Contents

7.	Complex Data and Addressing Possibilities.....	7-2
7.1.	Task Description: Managing Weight Values in WeightStore and Displaying Statistic Values	7-3
7.2.	Meaning of Variables and Data Types.....	7-4
7.3.	Overview: Data Types in STEP 7	7-5
7.3.1.	Elementary Data Types	7-6
7.3.2.	Data Types for Timers, Date and Time-of-day	7-7
7.3.3.	Complex Data Types	7-9
7.3.4.	Meaning of Complex Data Types.....	7-10
7.3.5.	Time Stamp: DATE_AND_TIME (DT)	7-11
7.3.5.1.	Access to DT by means of AT-View	7-12
7.3.6.	Time Stamp: DTL.....	7-13
7.3.7.	Character String: STRING	7-14
7.3.7.1.	Access to STRING by means of AT-View	7-15
7.3.8.	Slice Access (All Languages)	7-16
7.3.8.1.	Example of a Slice Access: 'Sign' Scan	7-17
7.3.9.	ARRAY	7-18
7.3.10.	Indexed Array Accesses	7-19
7.3.10.1.	Example of Indexed Array Accesses: Saving Part Weights in WeightStore	7-20
7.3.11.	Structure: STRUCT	7-21
7.3.12.	PLC Data Types: UDT (User defined Data Type).....	7-22
7.3.12.1.	Using UDTs.....	7-23
7.4.	"Shifting" (Copying) Variables of a Complex Data Type.....	7-24
7.5.	Simulating the PLC Controller (S7-PLCSIM S7-1200/1500)	7-25
7.6.	Exercise 1: Creating "UDT_WeightStore" and Weight Database "DB_Weights"	7-26
7.6.1.	Exercise 2: Expanding "DB_OP" (DB99) and Connecting the TP Screen "Statistic"	7-27
7.6.2.	Exercise 3: Programming "FB_Weights" (FB35)	7-29
7.6.3.	Exercise 4: Testing "FB_Weights" (FB35) using PLCSIM	7-31
7.6.4.	Exercise 5: Commissioning "FB_Weights" (FB35)	7-32
7.7.	Additional Information	7-33
7.7.1.	Settings for Simulating an HMI Device	7-34
7.7.2.	Simulating an HMI Device.....	7-35
7.7.3.	Additional Exercise: Simulating the Touchpanel.....	7-36
7.7.4.	Extended Instructions: STRING, DT	7-37

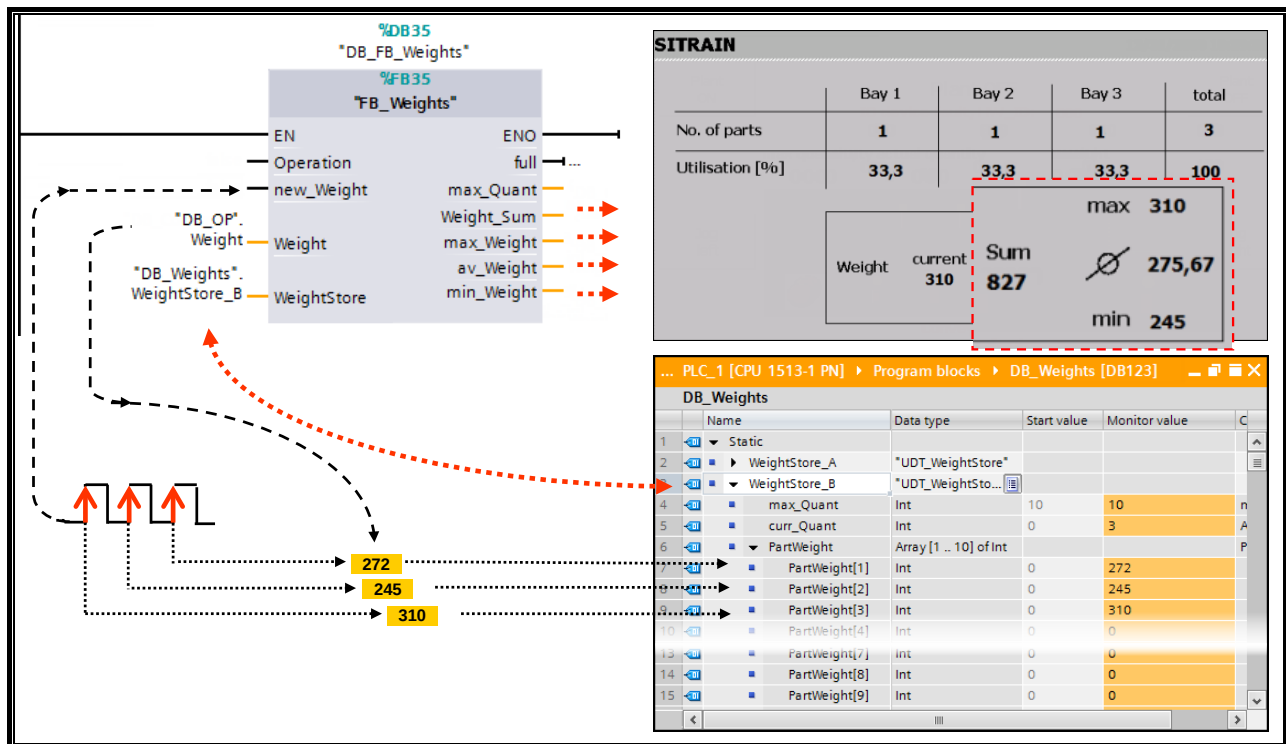
7. Complex Data and Addressing Possibilities

At the end of the chapter the participant will...

- ... be familiar with the use of complex data types and be able to explain it
- ... be able to declare variables of complex data types and be able to use them
- ... be familiar with various direct and indirect accesses to complex variables and be able to use them



7.1. Task Description: Managing Weight Values in WeightStore and Displaying Statistic Values



Program Function Up Until Now

When "P_Operation" (Q0.1) is switched on, parts are placed on the conveyor at the light barrier bay, weighed (weight simulation via "S_SliderPot1" (IW10) on the simulator) and, after pressing the bay pushbutton, are transported to the next free Bay 1, 2 or 3. When a part reaches the bay the indicator light signals, with a flashing light, that the part can be removed. After that, it has a continuous light indicating that the bay is still occupied. After acknowledging using the associated bay pushbutton, the bay indicator light goes dark and the bay is once again free.

The transported parts are counted for each bay and the statistical data - total quantity as well as the utilization of the 3 bays - is determined. After a part has reached a bay, the bay can immediately be enabled again by pressing the associated bay pushbutton, so that any number of parts can be transported to every bay. Only when "P_Operation" (Q0.1) is switched off and switched on again are the IEC counters used for counting as well as the statistic values reset to 0.

Expanding the Program

The program is now to be expanded so that the weights of the transported parts are saved. For this, a weight storage (WeightStore) is created in which the (simulated) weight of the transported part is saved as soon as it reaches a free bay.

In the touchpanel screen "Statistic", in addition to the statistic values so far, the minimum, maximum and average weight up until now is also displayed as well as the sum of all weights.

Since the capacity of the weight storage is limited (*max_Quant*), a maximum number of parts can also be transported. If this maximum number of parts is reached, no further transportation can be started at the light barrier bay (indicator light dark), even if one or more of the Bays 1, 2, or 3 is enabled (free).

Only after "P_Operation" (Q0.1) is switched off and then switched on again, are all statistic and weight values overwritten with 0 and new part transport sequences can be started.

7.2. Meaning of Variables and Data Types

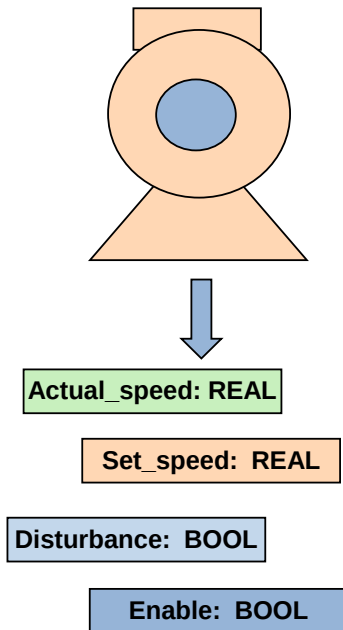
Variables represent the abstraction of reality and permit you to save and later continue to process values.

By declaring a variable, the following properties are defined:

- Symbolic name
- Data type
- Validity range
- Memory area

The data type establishes:

- The possible value range (e.g. INT: -32 768 to +32 767)
- The permitted instructions (e.g. math instructions: +I, -I)
- Data types abstract from the underlying representation of the bits in the memory



"Traditional" PLC Operands

In traditional PLC programming, PLC memory addresses were accessed directly by specifying the operand area (for example: M= bit memory, I=inputs, etc.), the access length (for example: B=byte, W=word, etc.) and by specifying the byte/(bit) address. These memory areas addressed via absolute addresses could be used within the program for different purposes, for example, as integer (e.g. DINT), as floating-point number (e.g. REAL) or simply as a collection of individual signals (e.g. WORD). Up until now, it was up to the programming person to make a mental note of the format and the purpose of the individual memory locations. Accordingly the programming was prone to errors.

The Meaning of Variables

Next to commands, variables are the most important elements of a programming system. Their task is to save values in a program so that they can be further processed at a later time. The value of a variable can be saved "anywhere" in the PLC memory.

The data represents an abstraction of reality in which irrelevant properties of objects are ignored.

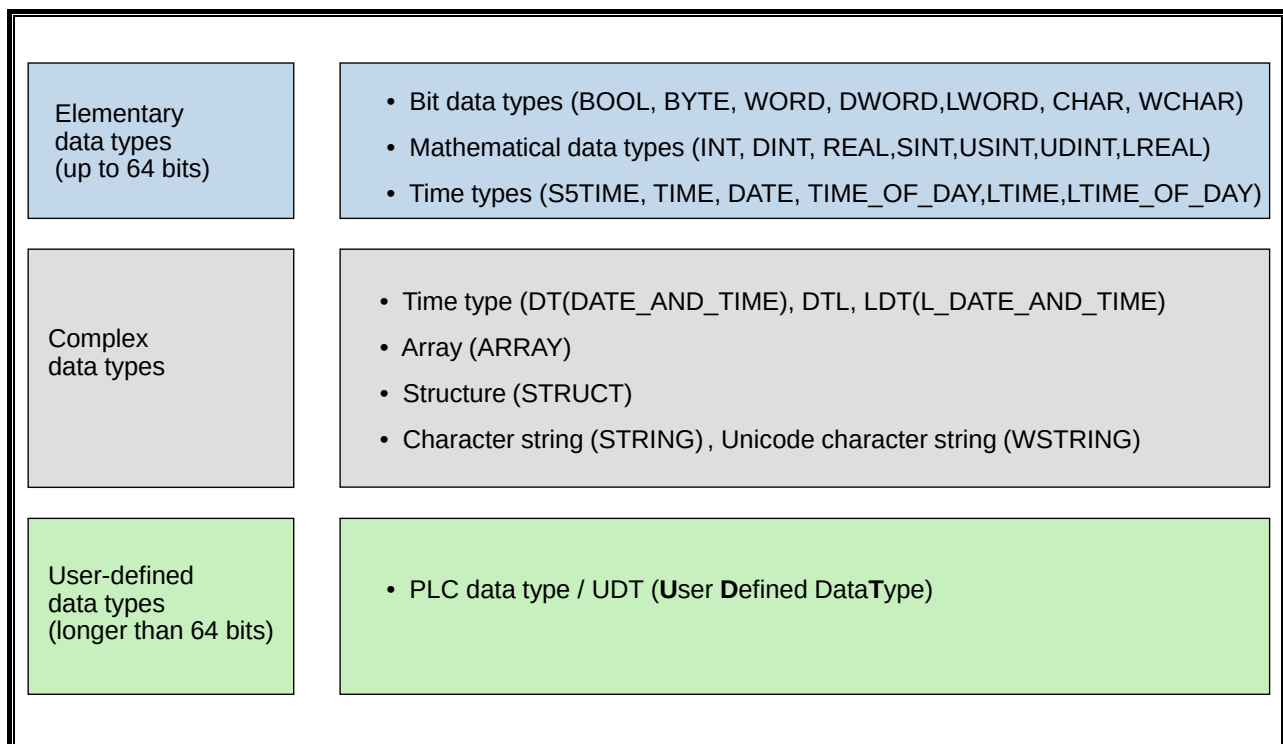
Data Types

It is often quite difficult to decide how data is to be represented and the available possibilities quite often restrict the choice. On the one hand, the object properties the data describe must be correctly reflected. On the other hand, it must also be possible to carry out the instructions necessary for process with the data.

The data type determines which values are accepted by data and which instructions can be carried out with these values. The data type uniquely defines

- the possible value range
- the permitted instructions

7.3. Overview: Data Types in STEP 7



Elementary Data Types

Elementary data types are predefined in accordance with IEC 61131-3. They always have a length less than or equal to 64 bits and can be further processed with elementary STEP 7 instructions.

Complex Data Types

Complex data types contain data structures that can be made up of elementary and/or complex data types. Complex data types can be used for the declaration of variables only in global data blocks and within blocks for the declaration of local variables (TEMP, STAT) as well as parameters (IN, OUT and INOUT). Variables of complex data types cannot be completely processed with elementary instructions (such as, A, O, L, T, +I) but only the individual components of the elementary data type.

PLC Data Types

PLC data types are data types defined by you that are used as templates for declaring parameters and variables of complex data types (e.g. structure variables). UDTs are created with the Data Block Editor and contain a data structure that is made up of elementary and/or complex data types. In the declaration of a variable according to data type UDTx, a structure variable is created whose inner data structure is defined by a UDT. UDTs can be used for the declaration of variables in global data blocks and within blocks for the declaration of local variables (TEMP, STAT) as well as parameters (IN, OUT and INOUT).

7.3.1. Elementary Data Types

Data type	Length (in bits)	Constants	Variables
BOOL	1	1 or 0	I 1.0
BYTE	8	B#16#A9	MB70
WORD	16	W#16#12AF	MW72
DWORD	32	DW#16#ADAC1EF5	QD40
LWORD	64	LW#16#5F52DE8B	
CHAR	8	'w '	DBB4
WCHAR	16	WCHAR#'a '	MW10
INT	16	123	#Value
DINT	32	L#65539	MD80
REAL	32	1.2 or 34.5E-12	DBD60
SINT	8	+/-50	MB24
USINT	8	50	MB24
UDINT	32	4875678	DBD64
LREAL	64	LREAL#1.0e-5	
LINT	64	LINT#+1543258759	

BOOL, BYTE, WORD, DWORD, CHAR, WCHAR

Variables of the data type BOOL consist of one bit. Variables of the data types BYTE, WORD, DWORD are bit sequences of 8, 16 or 32 bits. The individual bits are not evaluated in these data types. Special forms of these data types are the BCD numbers and the count value as it is used in conjunction with the count function as well as the data type CHAR which represents a character in the ASCII code and WCHAR for which a character is stored in Unicode.

INT, DINT, REAL

Variables of these data types represent numbers with which relevant arithmetical calculation operations can be carried out.

Extensions of INT, DINT, REAL, WORD

U - Unsigned

Variables with the extension U represent an integer without sign.

Data types: USINT, UINT, ULINT, UDINT

S - Short

Variables with the extension S represent an integer with a length of 8 bits.

Data types: SINT, USINT

L - Long

Variables with the extension L represent a number with a length of 64 bits of the data type.

Data types: LWORD, LINT, ULINT, LREAL

7.3.2. Data Types for Timers, Date and Time-of-day

Data type	Length (in bits)	Example
Timers		
S5TIME	16	S5T#5s
TIME	32	T#2h46m30s630ms
LTIME	64	LT#11350d20h25m14s830ms652μs315ns
Date and Time-of-day		
DATE	32	D#1984-01-01
TIME_OF_DAY (TOD)	32	TOD#18:15:18:999
LTIME_OF_DAY (LTOD)	64	LTOD#10:20:30.400_365_215
DATE_AND_TIME (DT)	64	DT#1984-01-21-13-45-56-567-3
DATE_AND_LTIME (LDT)	64	LDT#2008-01-01-18:00:30:25
DTL	96	DTL#1984-01-01-18:00:30:250

S5TIME

Variables of the data type S5TIME are required for specifying time values in timer functions (S5-timing elements). You specify the time in hours, minutes, seconds or milliseconds. You can enter the timer values with an underline (1h_4m) or without an underline (1h4m).

TIME

A variable of the data type TIME (duration in [ms]) occupies a double-word. This variable is used, for example, for specifying time values in IEC timer functions. The contents of the variable are interpreted as a DINT number in milliseconds and can be either positive or negative (for example: T#1s=L#1 000, T#24d20h31m23s647ms = L#2147486470).

LTIME

Just like TIME, LTIME represents a duration whereby the duration is saved with nanosecond resolution in 64 bits. This means that in variables of the data type LTIME, longer durations with greater resolution can be saved.

DATE

A variable of the data type DATE is stored in a word in the form of an unsigned integer. The contents of the variable represent the number of days since 01.01.1990 (for example: D#2003-10-31 = W#16#13BB).

TOD

The data type TOD (TIME_OF_DAY) occupies a double-word and stores the number of milliseconds since the beginning of the day (0:00 o'clock) as an unsigned integer.

LTOD

Variables of the data type LTOD occupy two double-words and store the number of nanoseconds since the beginning of the day.

DT

The data type DATE_AND_TIME represents a point in time consisting of the date and the time-of-day. Instead of DATE_AND_TIME, the abbreviation DT can also be used.

Byte	Contents	Range of values
0	Year	0 to 99 (Years 1990 to 2089) BCD#90 = 1990 ... BCD#0 = 2000 ... BCD#89 = 2089
1	Month	BCD#0 to BCD#12
2	Day	BCD#1 to BCD# 31
3	Hour	BCD#0 to BCD#23
4	Minute	BCD#0 to BCD#59
5	Second	BCD#0 to BCD#59
6	The two most significant digits of MSEC	BCD#0 to BCD#999
7 (4MSB) ¹⁾	The least significant digit of MSEC	BCD#0 to BCD#9
7 (4LSB) ²⁾	Weekday	BCD#1 to BCD#7 BCD#1 = Sunday ... BCD#7 = Saturday

LDT

The data type LDT consists of the data types DATE and LONGTIME and stores the date and time in nanoseconds since 1.1.1970.

DTL

The data type DTL has a length of 12 bytes and, like LDT, stores information on date and time-of-day precise to the nanosecond since 1.1.1970, only in a pre-defined structure.

Advantage: the individual values (day, hour, etc.) are easier to read out.

Byte	Component	Data type	Range of values
0 – 1	Year	UINT	1970 to 2554
2	Month	USINT	1 to 12
3	Day	USINT	1 to 31
4	Weekday	USINT	1(Sunday) to 7(Saturday) The weekday is not considered in the value entry.
5	Hour	USINT	0 to 23
6	Minute	USINT	0 to 59
7	Second	USINT	0 to 59
8 – 11	Nanosecond	UDINT	0 to 999999999

7.3.3. Complex Data Types

Data type	Length (in bits)	Example	
STRING (character string with max. 254 characters)	8 * (number of characters + 2)	`This is a string` `SIEMENS`	
WSTRING (character string with max. 16382 characters in unicode)	16 * (number of characters + 2)	WSTRING#`Hello SITRAIN`	
ARRAY (Field, Group of components of the same data type)	user-defined	Measured values: ARRAY[1..20] of INT	
STRUCT (Structure, Group of components of different data types)	user-defined	Motor: STRUCT Speed : INT Current : REAL END_STRUCT	
PLC Data types (UDT) (User Defined Data Type = "Template " consisting of elementary or complex data types)	user-defined	UDT as block	UDT as array element
		Static Speed : INT Current : REAL	Drive: ARRAY[1...4] of UDT1

Complex Data Types

Complex data types (arrays and structures) consist of groups of elementary or complex data types.

They enable you to create data types suitable for your problem with which you can structure large quantities of data and process it symbolically.

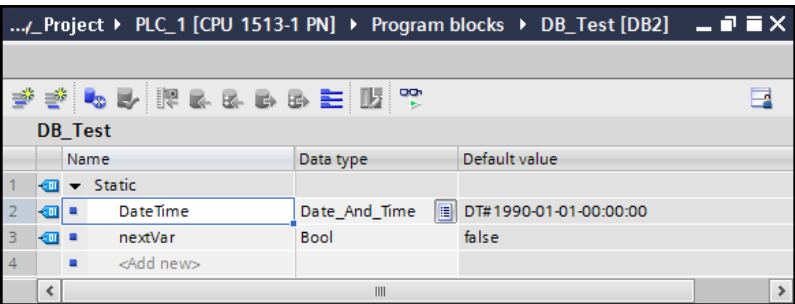
Complex data types are predefined, whereby the lengths of the data types ARRAY (field), STRUCT (structure), STRING (character string) and WSTRING (character string in unicode) are defined by the user.

Variables with complex data types can only be declared within global data blocks and as parameters or local variables of logic blocks.

User-defined Data Type (UDT)

User-defined data types represent self-defined structures. This structure is stored in UDT blocks (UDT1 to UDT65535) and can be used as a "template" in another variable's data type. You can save typing time when you input a data block if you need the same structure several times.

7.3.5. Time Stamp: DATE_AND_TIME (DT)



Range of values

Min.: DT#1990-01-01-0:0:0

Max.: DT#2089-12-31-23:59:59.999

Structure:

Byte n ¹⁾	Year (90 ... 89)		Month (01 ... 12)	Byte n+1
Byte n+2	Day (01 ... 31)		Hour (00 ... 23)	Byte n+3
Byte n+4	Minute (00 ... 59)		Second (00 ... 59)	Byte n+5
Byte n+6	Milliseconds (000 ... 999)		Week- day (1..7)	Byte n+7

(all values BCD-coded)

¹⁾ n = even

Weekday identifier:

1 = Sunday

2 = Monday

3 = Tuesday

4 = Wednesday

5 = Thursday

6 = Friday

7 = Saturday

Overview

The DATE_AND_TIME data type represents a point in time, consisting of the date and the time of day. Instead of DATE_AND_TIME you can also use the abbreviation DT.

DATE_AND_TIME or DT are keywords and can therefore also be written in lower case.

Pre-assignment

In the declaration, you can pre-assign an initial value to a variable (not as block parameter in an FC, as in/out parameter in an FB or as temporary variable).

The pre-assignment must be of the type:

- DT#year-month-day-hours:minutes:seconds.milliseconds

You don't have to specify the milliseconds.

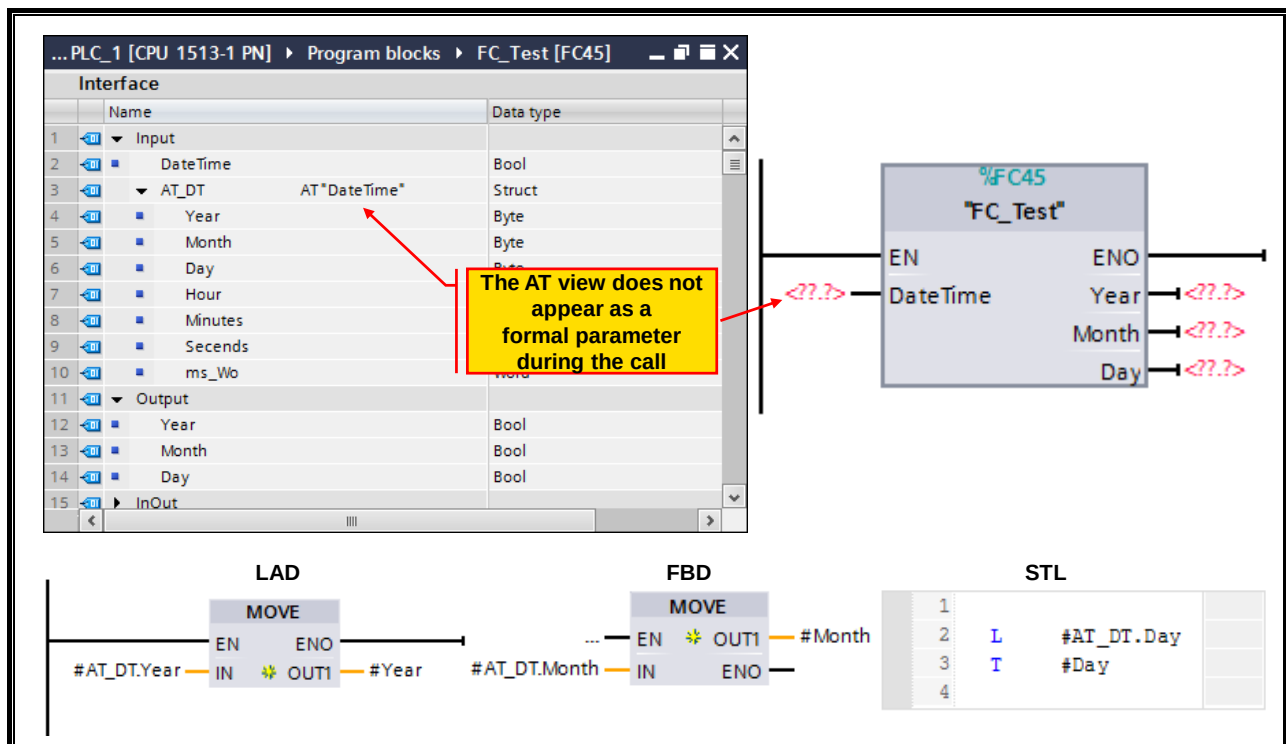
Processing

Variables of the DATE_AND_TIME data type can either be processed with the help of absolute accesses to the individual components or with the relevant functions of the IEC library.

Note

The current time of the CPU real-time clock can be read out with the "Extended instruction" RD_SYS_T. The instruction returns the current time stamp via an output parameter of the type DATE_AND_TIME.

7.3.5.1. Access to DT by means of AT-View



Using an AT View to Read-out a Time Stamp of the Data Type DATE AND TIME (DT)

The data structure of a time stamp of the data type DATE AND TIME consists of 8 bytes. The values for year, month, day, hours, minutes, seconds, milliseconds and weekday identifier are stored in the BCD format.

Since the components of the time stamp cannot be addressed symbolically, these components can only be read out by means of indirect addressing or by creating an AT view. If the structure of the AT view is created exactly according to the time stamp, the data areas of the time stamp can be accessed symbolically using the structure elements of the AT view.

The AT view is neither an additional parameter nor an additional variable. Accordingly, an AT view to a parameter also does not appear in its interface during the call of the block. An AT view to a temporary or static variable also does not occupy any memory space on the local data stack or in the instance data block.

Declaring the AT View

1. Create variables that are to be superimposed,
2. Create AT view variable of the data type AT (confirm with Return key),
3. Change the data type of the AT view variable,
4. If the AT view variable is of the type STRUCT, the structure still has to be created.

Rules:

Variable can be superimposed in logic blocks with standard access or in logic blocks with optimized access for variables using the retentive setting "Set in IDB".

The data length of the superimposing variable must be equal to or less than that of the superimposed variable.

The data types VARIANT and INSTANCE cannot be superimposed.

Blocks from libraries that are declared as parameters in the interface cannot be superimposed. Structured PLC tags that are declared as parameters in the interface cannot be superimposed.

7.3.6. Time Stamp: DTL

The screenshot displays the SIMATIC Manager interface for a project named 'My_Project' under 'PLC_1 [CPU 1513-1 PN]' in the 'Program blocks' section, specifically for 'FC_Test [FC45]'. The 'Interface' table lists the DTL data type structure:

Name	Data type	Offset	Comment
Input			
DateTime	DTL		
YEAR	UInt		
MONTH	USInt		
DAY	USInt		
WEEKDAY	USInt		
HOUR	USInt		
MINUTE	USInt		
SECOND	USInt		
NANOSECOND	UDInt		
Output			
Year	Byte		
Month	Byte		
Day	Byte		
InOut			

A tooltip titled 'Range of values' is shown, indicating the minimum and maximum values for the DTL data type:

```
Min.: DTL#1970-01-01-00:00:00.0
Max.: DTL#2554-12-31-23:59:59.999999999
```

Below the table, the LAD, FBD, and STL representations are shown:

- LAD:** A 'MOVE' block with 'EN' and 'ENO' inputs. The 'IN' input is connected to '#DateTime.YEAR' and the 'OUT1' output is connected to '#Month'.
- FBD:** A 'MOVE' block with 'EN' and 'ENO' inputs. The 'IN' input is connected to '#DateTime.MONTH' and the 'OUT1' output is connected to '#Month'.
- STL:** A ladder logic network with four rungs:


```

1
2 L #DateTime.DAY
3 T #Day
4
      
```

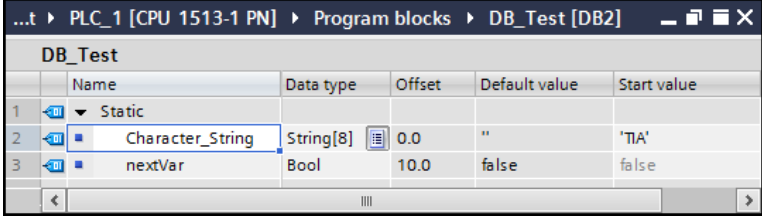
Overview

The DTL (Date and Time) data type represents a point in time, consisting of the date, the time of day and the weekday identifier.

Unlike the time stamp of the data type DT, the time stamp of the data type DTL is created as a structure for which the individual components can be addressed symbolically.

Furthermore, the values of the individual structure elements are coded in arithmetic data types so that these can be further processed by the program with the available "Mathematical Functions".

7.3.7. Character String: STRING



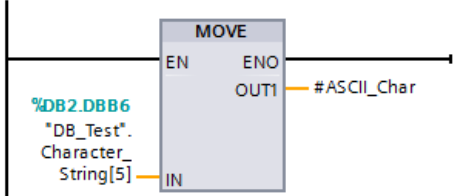
Byte	Value
Byte n ¹⁾	max length= 8
Byte n+1	actual length= 3
Byte n+2	1. character= 'T'
Byte n+3	2. character= 'I'
Byte n+4	3. character= 'A'
Byte n+5	B#16#00
Byte n+6	B#16#00
Byte n+7	B#16#00
Byte n+8	B#16#00
Byte n+9	B#16#00

STRING:
Group of ASCII characters with 2 preceding length bytes

→ Specifies the max. number of storable characters

→ Specifies the actual number of stored valid characters

String access with direct addressing



Overview

The STRING data type is used for storing character strings (such as message texts). In this way, a simple “(message) word processing system” can be implemented in an S7-CPU. The STRING data type represents a character string of up to 254 characters.

The number specified in square brackets in the declaration (1 to 254) gives the maximum number of characters that can be saved in the STRING variable.

Storing Variables

In setting up the variable, you enter its maximum length into the first byte of the variable according to the variable declaration. Likewise, the actual number of stored ASCII characters according to the start value, is entered in the second byte.

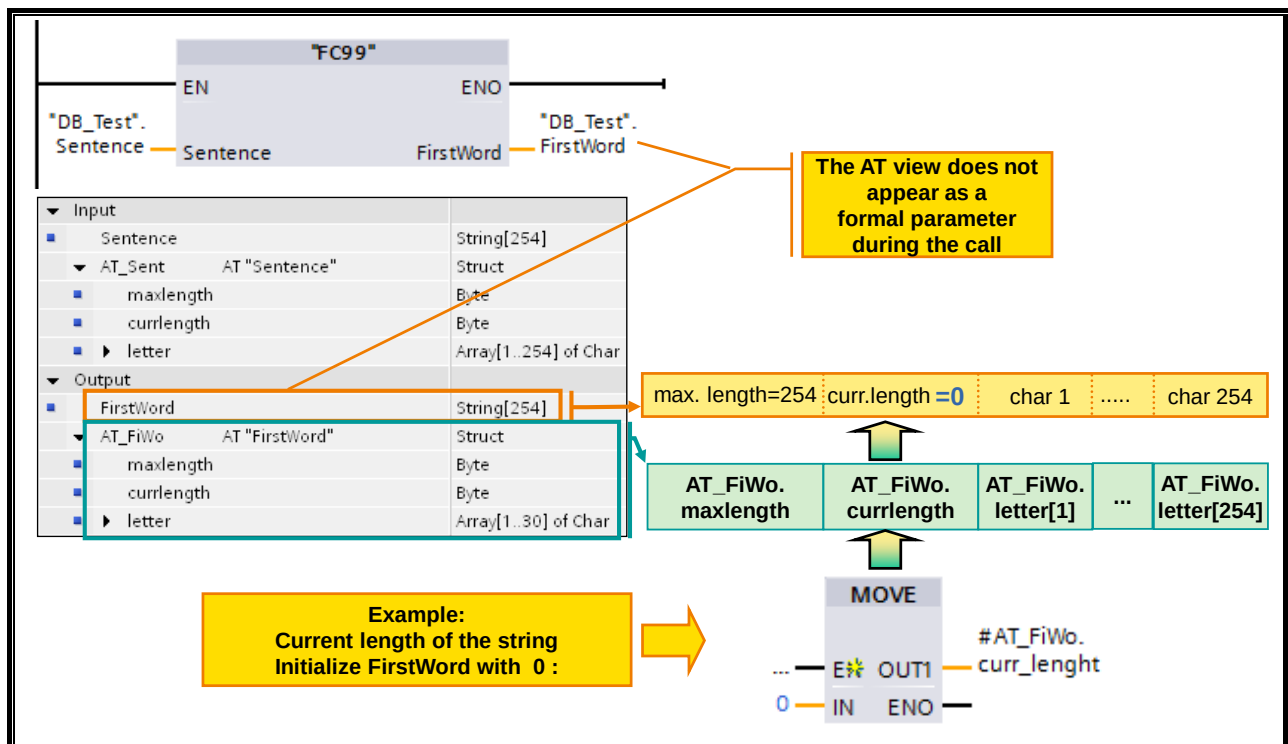
The standard instructions require these two pieces of information to process STRING variables.

Subsequently, the characters follow in ASCII-Format. Unoccupied characters in the STRING variable are initialized with B#16#0 during initialization.

Parameter Passing

Just like ARRAY or STRUCT variables, variables of the STRING data type can be passed to block parameters with the same data type, that is, same STRING length.

7.3.7.1. Access to STRING by means of AT-View

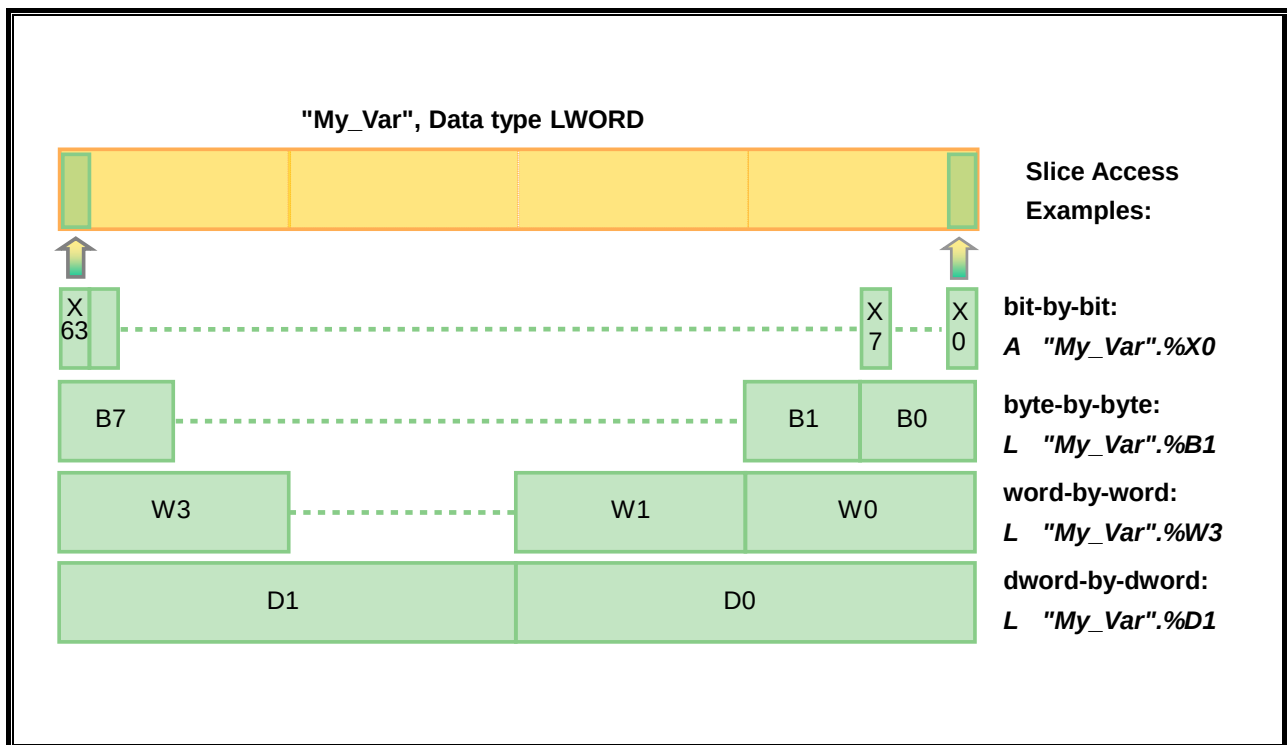


AT View to Parameters and Variables

With parameters or variables, AT views are used to be able to symbolically access individual data areas which otherwise would only be possible by means of indirect addressing.

Thereby the AT view fulfills the function of a template with which the variable or the parameter is superimposed in order to then be able to access the relevant data areas through the symbolic structure elements of the template.

7.3.8. Slice Access (All Languages)



Slice Access

The slice access enables a bit, byte, word and double-word access to variables of greater dimensions respectively.

With a bit access, for example, the individual bits of an integer variable can be scanned or low and high byte can be loaded even though no symbolic names exist for these variable elements.

7.3.8.1. Example of a Slice Access: 'Sign' Scan

The screenshot displays the SIMATIC Manager interface for a project named 'My_Project'. The path is 'PLC_1 [CPU 1513-1 PN] > Program blocks > DB_OP [DB99]'. The 'DB_OP' data block is shown with the following variables:

	Name	Data type	Offset	Start value	Comment
1	Static				
2	Operation_OFF	Bool	0.0	false	Operation OFF (HMI)
3	Operation_ON	Bool	0.1	false	Operation ON (HMI)
4	Jog_right	Bool	0.2	false	Jog right
5	Jog_left	Bool	0.3	false	Jog left
6	Ackn_Fault	Bool	0.4	false	Acknowledge fault
7	Weight	Int	2.0	0	current weight

Below the table, three representations of a sign scan are shown:

- LAD:** A normally open contact labeled `"DB_OP".Weight.x15` is connected to a coil labeled `#Weight_plaus`.
- FBD:** A function block with an input `"DB_OP".Weight.x15` and an output `#Weight_plaus`.
- STL:**

```

1
2  AN  "DB_OP".Weight.x15
3  =   #Weight_plaus
4

```

Example: Scanning the Sign Bit of an Integer Variable

An individual bit of, for example, an Integer variable cannot be addressed symbolically and can only be scanned by means of indirect addressing or by slice access.

In the example shown, the sign bit (bit no. 15) of the data block variable *Weight* is scanned in order to check whether the value of the variable is positive and with that plausible (a negative weight value cannot exist and so would not be plausible).

7.3.9. ARRAY

ARRAY:
Group of components of the same data type

Array access with direct addressing

```

graph LR
    IN["#Value"] --> MOVE["MOVE"]
    MOVE --> ENO["ENO"]
    MOVE --> OUT1["OUT1"]
    OUT1 --> DB[""DB_Test".MeasuringPoint[5]"]
  
```

Name	Data type	Offset	Default value	Start value
Static				
MeasuringPoint	Array[1..10] of Int	0.0		
MeasuringPoint[1]	Int	0.0	0	0
MeasuringPoint[2]	Int	2.0	0	0
MeasuringPoint[3]	Int	4.0	0	0
MeasuringPoint[4]	Int	6.0	0	0
MeasuringPoint[5]	Int	8.0	0	0
MeasuringPoint[6]	Int	10.0	0	0
MeasuringPoint[7]	Int	12.0	0	0
MeasuringPoint[8]	Int			
MeasuringPoint[9]	Int			
MeasuringPoint[10]	Int			

Name	Data type	Offset	Default value	Start value
Static				
ArrayNo	Array[1..3,1..3] of Byte	0.0		
ArrayNo[1,1]	Byte	0.0	16#0	16#0
ArrayNo[1,2]	Byte	1.0	16#0	16#0
ArrayNo[1,3]	Byte	2.0	16#0	16#0
ArrayNo[2,1]	Byte	3.0	16#0	16#0
ArrayNo[2,2]	Byte	4.0	16#0	16#0
ArrayNo[2,3]	Byte	5.0	16#0	16#0
ArrayNo[3,1]	Byte	6.0	16#0	16#0
ArrayNo[3,2]	Byte	7.0	16#0	16#0
ArrayNo[3,3]	Byte	8.0	16#0	16#0

Overview

The ARRAY data type represents a field with a fixed number of components (elements) of the same data type. A field (=ARRAY) can have up to six dimensions (number of indexes).

The data types of the components (within an array) can be bit sequences, integers, floating-point numbers, timers, character strings, structures or PLC-data types (UDT). Parameter types or FBs (multiple instance model) are not permitted.

ARRAYs cannot be nested. The limits of the minimum or maximum index range are defined for S7-300/400 by the value range of INT, that is, from -32768 to 32767 and for S7-1500 from -2147483648 to 2147483647.

Maximum Array Limits

The maximum Array limits depend on the following factors:

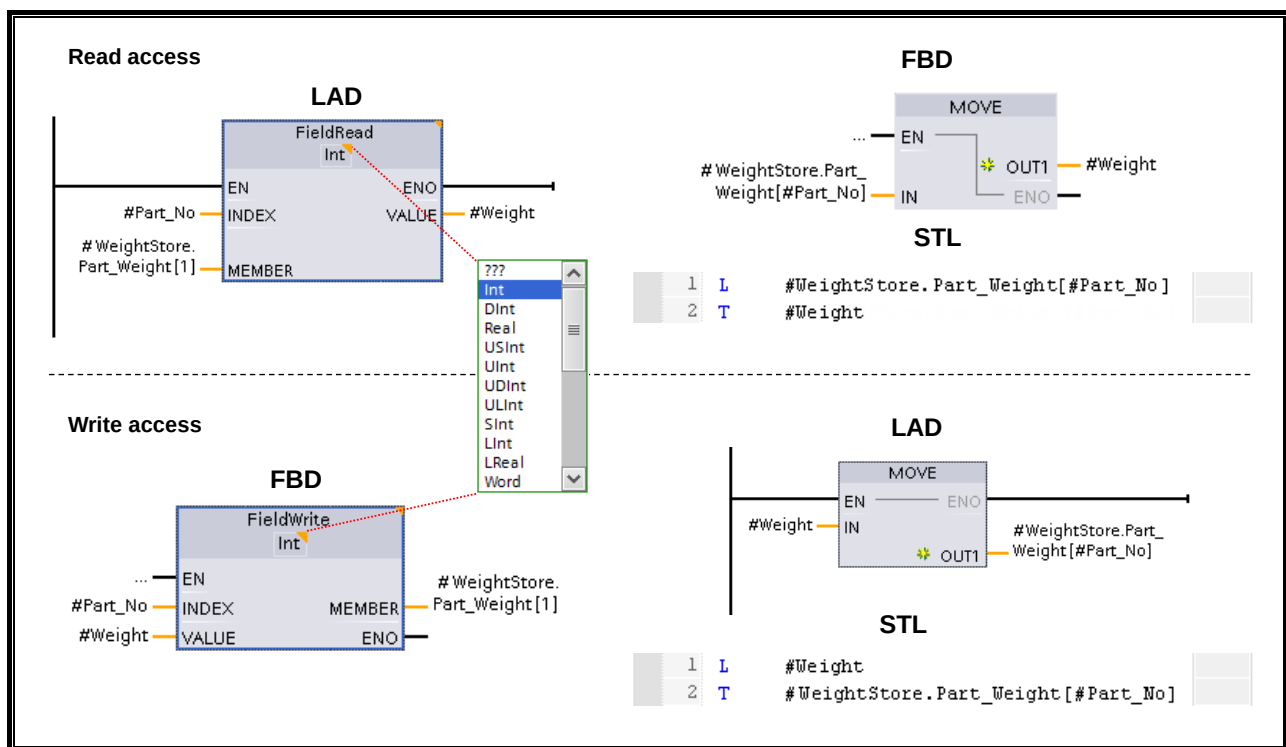
- Data type of the Array elements
- Memory reserve (only in blocks with optimized access).
- Maximum size of a data block of a CPU (you will find further information in the respective manual)
- The entire length of the Array is available within a data block. Within a program block (OB, FB or FC), the possible length is reduced according to the memory capacity which the program code requires.

Note

Valid for CPUs of the S7-1500 series

For a block with the block property "Optimized block access", one bit requires a memory space of 1 byte. This is also true if you use an ARRAY of <Datatype>. Thus, for example, an ARRAY [0..1] of BOOL requires 2 bytes in an optimized block.

7.3.10. Indexed Array Accesses



"FieldWrite"

The instruction "FieldWrite" writes the value of the operand passed at input VALUE into the array passed at output MEMBER. The value is written in the array element whose number is specified via the input INDEX (even though the block editor always displays the first array element `#WeightStore.Part_Weight[1]` at the output MEMBER).

The data types of the array specified at output MEMBER and of the operand at input VALUE must match the data type of the instruction "FieldWrite".

The instruction signals an error with Status '0' at output ENO when the array element specified via input INDEX does not exist in the array specified at output MEMBER.

"FieldRead"

The instruction "FieldRead" reads the value of an array element of the array passed at input MEMBER and writes this in the variable specified at output VALUE. The value of the array element is read whose number is specified via the input INDEX.

The data types of the array specified at input MEMBER and the variable specified at output VALUE must match the data type of the instruction "FieldRead".

The instruction signals an error with Status '0' at output ENO when the array element specified via input INDEX does not exist in the array specified at input MEMBER.

Indexed Array Accesses with the FBD Instruction "MOVE" and the STL Instructions "L, T"

Just as with the instructions "FieldRead" and "FieldWrite", indexed, read and write Array accesses can be programmed in FBD with the MOVE instruction as well as in STL with the L and T instructions (Load and Transfer via the emulated accumulator ACCU1).

7.3.10.1. Example of Indexed Array Accesses: Saving Part Weights in WeightStore

The screenshot displays two side-by-side windows from the SIMATIC Manager, showing the interfaces for two functions: **FC_Weight_STL [FC35]** and **FC_Weight_LAD [FC36]**. A yellow box with the text "Interfaces identical" is placed over the variable lists of both functions, indicating they share the same parameters.

FC_Weight_STL [FC35] Interface:

Name	Data type	Comment
Input		
Operation	Bool	
new_Weight	Bool	
Weight	Int	
Output		
full	Bool	
max_Quant	Int	
Weight_Sum	Int	
max_Weight	Int	
av_Weight	Real	
min_Weight	Int	
InOut		
WeightStore	*UDT_WeightSto...	
max_Quant	Int	max no
curr_Quant	Int	Actual q
PartWeight	Array [1 .. 10] of Int	Part wei
PartWeight[1]	Int	
PartWeight[2]	Int	
PartWeight[3]	Int	

FC_Weight_LAD [FC36] Interface:

Name	Data type	Comment
Input		
Operation	Bool	
new_Weight	Bool	
Weight	Int	
Output		
full	Bool	
max_Quant	Int	
Weight_Sum	Int	
max_Weight	Int	
av_Weight	Real	
min_Weight	Int	
InOut		
WeightStore	*UDT_WeightSto...	
Temp	Temp	
part_no	Dint	

The LAD view shows two networks. Network 1 contains an **ADD Auto (Int)** block that increments **#WeightStore.curr_Quant** by 1, followed by a **MOVE** block that moves the result to **#part_no**. Network 2 contains a **FieldWrite** block that writes the value of **#Weight** to the array element **#WeightStore.PartWeight[#part_no]**.

Below the variable lists, the STL code for FC_Weight_STL is shown:

```

1 L #WeightStore.curr_Quant
2 + 1
3 T #WeightStore.curr_Quant
4 T #part_no
5
6 L #Weight
7 T #WeightStore.PartWeight[#part_no]

```

A yellow circle labeled "STL" is placed next to the code. A yellow circle labeled "LAD" is placed next to the Ladder Logic diagram.

Example: Saving Part Weights in Weight Store

In the example shown, the value of the variable **#WeightStore.curr_Quant** is increased by 1 every time a part arrives, that is, with every execution of the block. With that, the value of this variable corresponds exactly to the number of the element of the array variable **#WeightStore.PartWeight[i]**, in which the current value of the variable **#Weight** is to be saved.

The instruction "FieldWrite" writes the value **#Weight** in the array variable **#WeightStore.PartWeight[i]** in exactly the array element whose number is specified via **#part_no** (even though the block editor always displays the first array element **#WeightStore.PartWeight[1]** at output MEMBER).

7.3.11. Structure: STRUCT

	Name	Data type	Offset	Default value	Start value
1	Static				
2	Motor	*UDT_Motor*	0.0		
3	SET_Speed	Int	0.0	0	0
4	ACT_Speed	Int	2.0	0	0
5	enable	Bool	4.0	false	false
6	fault	Bool	4.1	false	false
7	CosPi	Real	6.0	0.0	0.0
8	nextVar	Int	10.0	0	0

STRUCT:
Group of components of different data types

Structure access with direct addressing

Overview

The STRUCT (Structure) data type represents a specific number of components that can each have different data types. A structure can have up to eight nesting levels.

A structure can be declared in the declaration part of a logic block, in a global DB or in a PLC data type (UDT).

The data types of the components within a structure can be bit sequences, integers, floating-point numbers, timers, character strings, structures or PLC-data types (UDT). Parameter types or FBs (multiple instance model) are not permitted.

Initialization of STRUCT

The individual structure components can be pre-assigned values in the declaration (column: Start value). The following parameters or variables cannot be initialized:

- input, output and in/out parameters for FCs
- in/out parameters in FBs
- local data in OBs, FBs and FCs

The initialization value's data type must be compatible with the component's data type.

7.3.12. PLC Data Types: UDT (User defined Data Type)

The screenshot illustrates the creation and use of a User Defined Data Type (UDT) in SIMATIC Manager. It is divided into three main sections:

- UDT_Motor Definition:** A table defining the structure of the UDT_Motor.

	Name	Data type	Default value	Comment
1	SET_Speed	Int	0	
2	ACT_Speed	Int	0	
3	enable	Bool	false	
4	fault	Bool	false	
5	CosPi	Real	0.0	
- Structure access with direct addressing:** A ladder logic network showing the use of the UDT. It includes a normally open contact labeled `%Q0.1 "P_Operation"`, a normally closed contact labeled `%M17.0 "M_Conv_Fault"`, and a coil labeled `%DB2.DBX14.0 "DB_Test". Motor2.enable`. Red dashed arrows indicate the mapping from the UDT_Motor definition to the variable access in the program.
- DB_Test [DB2] Structure:** A table showing the declaration of the UDT_Motor within a Data Block.

	Name	Data type	Offset	Start value
1	Static			
2	Motor1	"UDT_Motor"	...	
3	Motor2	"UDT_Motor"	...	
4	SET_Speed	Int	...	0
5	ACT_Speed	Int	...	0
6	enable	Bool	...	false
7	fault	Bool	...	false
8	CosPi	Real	...	0.0
9	Motor3	"UDT_Motor"	...	

Additional details include a **MOVE** instruction block with `#Q_Speed` as the input and `%DB2.DBW10 "DB_Test". Motor2.SET_Speed` as the output, and a red dashed arrow pointing from the UDT_Motor definition to the Motor2 entry in the DB_Test table.

Overview

When a data structure repeats itself frequently in a user program for the declaration of variables and parameters or when a data structure is to be identified with its own name, then STEP 7 permits user-defined, so-called PLC data types (user-defined data types or User Defined Data Type, UDT) to be created (like *typedef* in the high level language "C").

By using application-oriented data types, a task to be solved can be organized more clearly. Users as well as manufacturers can then design and implement data types adapted for their problems.

Creating PLC Data Types (UDTs)

By creating a PLC data type (UDTs), a globally valid "template" is created which then can be used as often as you like for the declaration of new variables and parameters or for the creation of global DBs.

7.3.12.1. Using UDTs

UDT_Motor

Name	Data type	Default value
1 SET_Speed	Int	0
2 ACT_Speed	Int	0
3 enable	Bool	false
4 fault	Bool	false
5 CosPi	Real	0.0

DB_Test

Name	Data type	Default value
1 Static		
2 Motor	Array[1..3] of "UDT_Motor"	0.0
3 Motor[1]	"UDT_Motor"	0.0
4 Motor[2]	"UDT_Motor"	10.0
5 SET_Speed	Int	0.0
6 ACT_Speed	Int	2.0
7 enable	Bool	4.0
8 fault	Bool	4.1
9 CosPi	Real	6.0
10 Motor[3]	"UDT_Motor"	20.0

FC_Drive [FC120]

Name	Data type	Comment
1 Input		
2 Output		
3 InOut		
4 Motor	"UDT_Motor"	
5 SET_Speed	Int	
6 ACT_Speed	Int	
7 enable	Bool	
8 fault	Bool	
9 CosPi	Real	
10 Temp		

Network 1:

```

#condition1 -- #condition2 -- #Motor.enable
  
```

Annotations:

- Parameter declaration according to UDT:** Points to the UDT_Motor table and the Motor parameter in the FC_Drive interface.
- Variable declaration according to UDT:** Points to the Motor[1] and Motor[2] variables in the DB_Test table.
- Access to parameter components:** Points to the #Motor.enable variable in the ladder logic network.

Overview

In the example shown, the PLC data type "UDT_Motor" is composed of the components SET_Speed, ACT_Speed, enable, fault and CosPi.

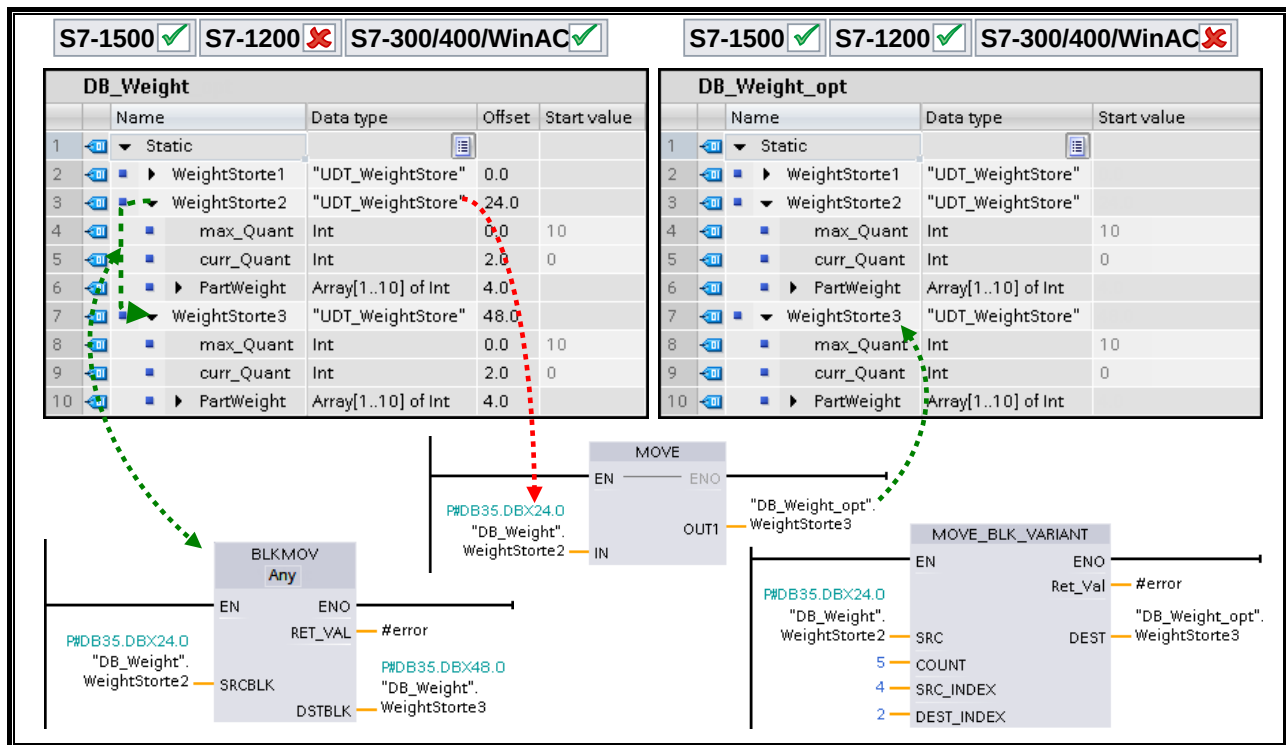
In the "DB_Test" (DB1) data block, the UDT is used to declare the data block variable *Motor*; in "FC_Drive" (FC120) to declare the INOUT parameter *Motor*.

Initial (Start) Values for UDTs

PLC data types are pre-assigned just like Structures and then used in the user program. The structure of a PLC data type is the same as that for a Structure. With the creation of a PLC data type, the declaration of variables that can be processed by the user program has not yet taken place. The PLC data type is a "template" that you can use as often as you like for the declaration of new variables.

Just as with a Structure, you can also define default values in the PLC data type. If the PLC data type is then used for the declaration of a variable, these default values are adopted as the start values of this variable (not for parameters in FCs, not for in/out parameters of FBs and not for temporary variables).

7.4. "Shifting" (Copying) Variables of a Complex Data Type



Copying Variables of a Complex Data Type

- **BLKMOV**
The BLKMOV instruction works with ANY pointers which have a 32-bit pointer with absolute byte address and bit address. Since absolute addresses are only available in data areas (DBs and L-stack of logic blocks) created as not optimized, the BLKMOV instruction can also only copy data which originates from such data areas or are to be copied there.

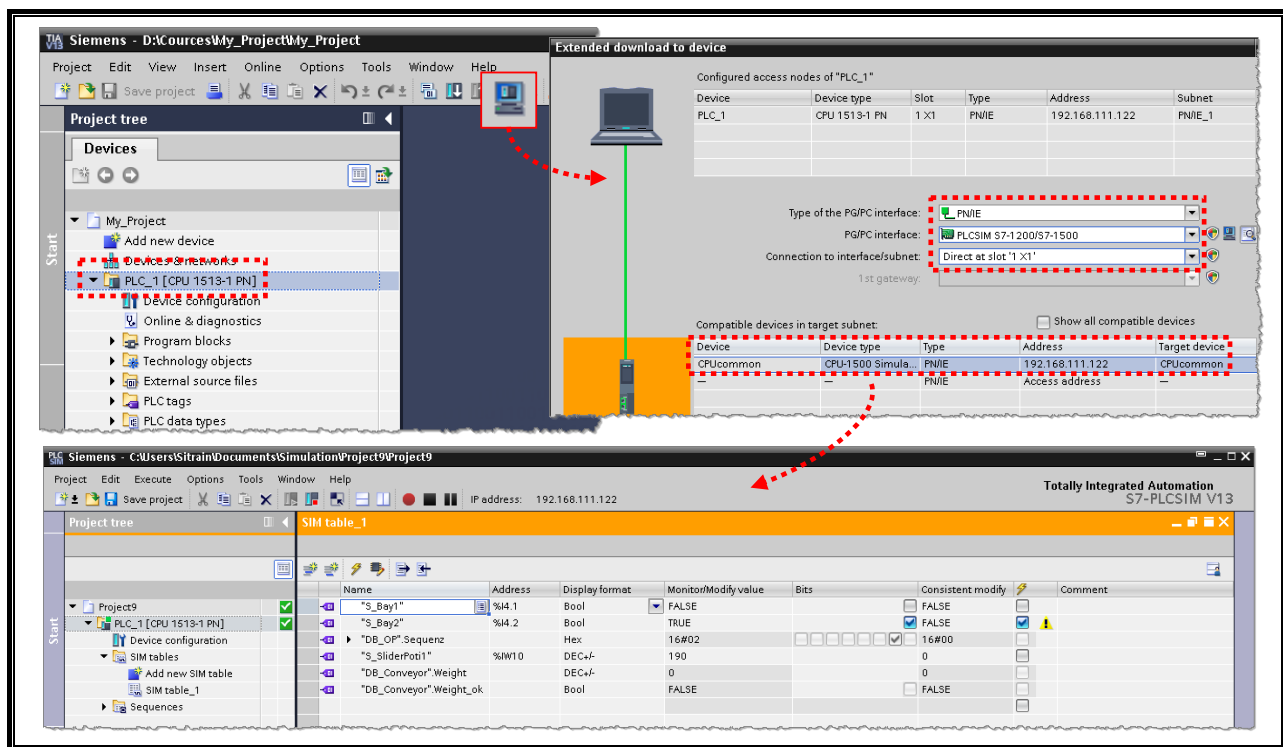
In the example shown, the variable WeightStorte2 is copied from "DB_Weight" (created as not optimized, can be seen in the column Offset) onto the variable WeightStorte3.

- **MOVE**
The MOVE instruction copies variables of any data type (even complex variables declared according to PLC data type). Source (IN) and destination (OUT) must be of the same data type, can however be located in optimized as well as in not optimized data areas.
- **MOVE_BLK_VARIANT**
From the Array variable at <SRC>, Array elements as of the <SRC_INDEX>-th element <COUNT> are copied into the Array variable at <DEST> as of the <DEST_INDEX>-th element. The data types (elementary or declared according to PLC data type) of the Array elements of the Array variables at <SRC> and <DEST> must match. Both source and destination can be located in optimized or not optimized data areas.

In the example shown, 5 elements of the source Array "DB_Weight".WeightStorte2, beginning as of the 4th element, are copied into the destination Array "DB_Weight_opt".WeightStorte3, beginning as of the 2nd element.

- **MOVE_BLK** (not shown in the picture)
in principle, fulfills the same function as MOVE_BLK_VARIANT, however, with the restrictions that only Array elements (no complete variables!) can be copied which must not be declared according to PLC data type.

7.5. Simulating the PLC Controller (S7-PLCSIM S7-1200/1500)



PLC controllers can be simulated without real hardware using PLCSIM.

How to Proceed:

1. Select (highlight) PLC controller,
2. Start simulation using the icon ,
3. In the dialog "Extended download to device" set the interface (PG/PC interface: PLCSIM S7-1200/1500)
4. Start search, select device type "CPU-1500 Simulation", download the project using the button "Load",

In the PLCSIM software, the loaded program is processed whereby you can monitor and change individual variables.

Since no real inputs exist, they must be controlled in a so-called SIM table. There are three possibilities for this:

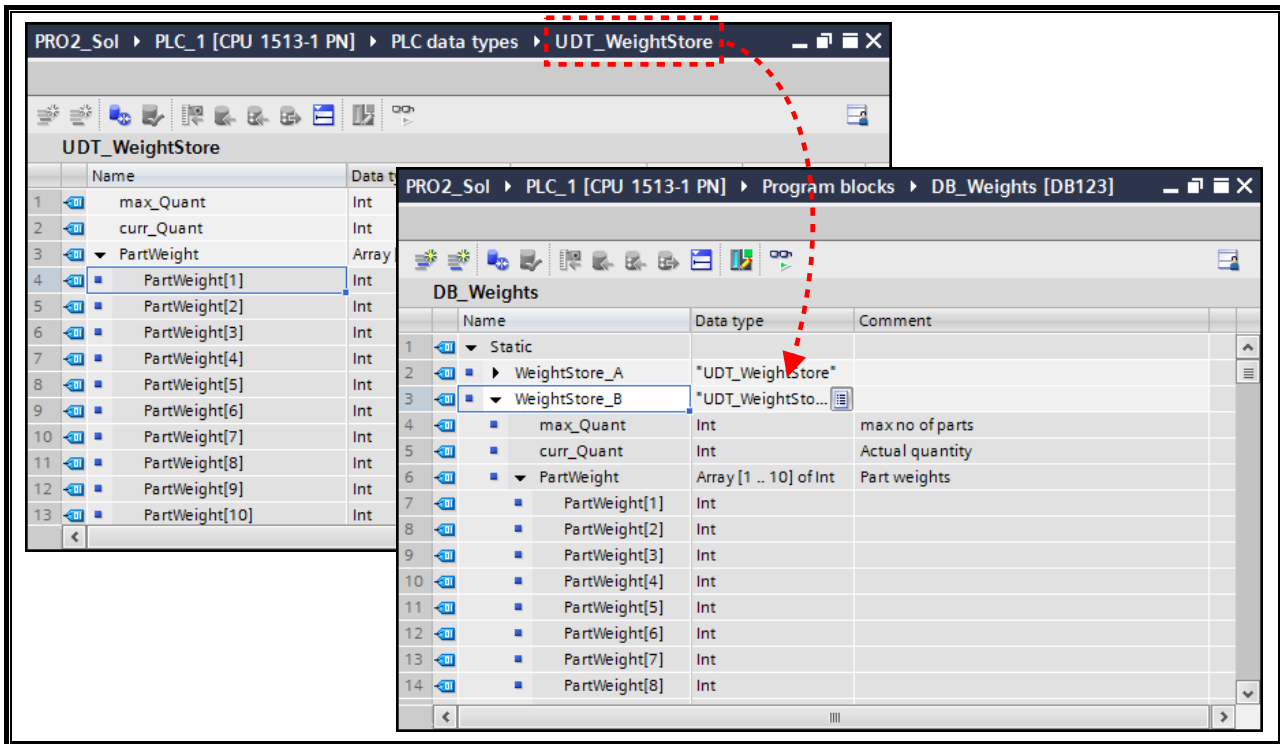
1. With the help of the button  consistently in groups whereby all variables that are activated in the column  are written with the values from the column "Consistent modify",
2. In the column "Bits", in order to switch the values of the individual bits of the associated variables directly between TRUE and FALSE.

In order to be able to change outputs, memory bits or variables in data blocks, the function

"Modification of non-inputs" must be activated (enabled) with the help of the button .

In addition, you can monitor the actual values of the variables in the column "Monitor/Modify value".

7.6. Exercise 1: Creating "UDT_WeightStore" and Weight Database "DB_Weights"



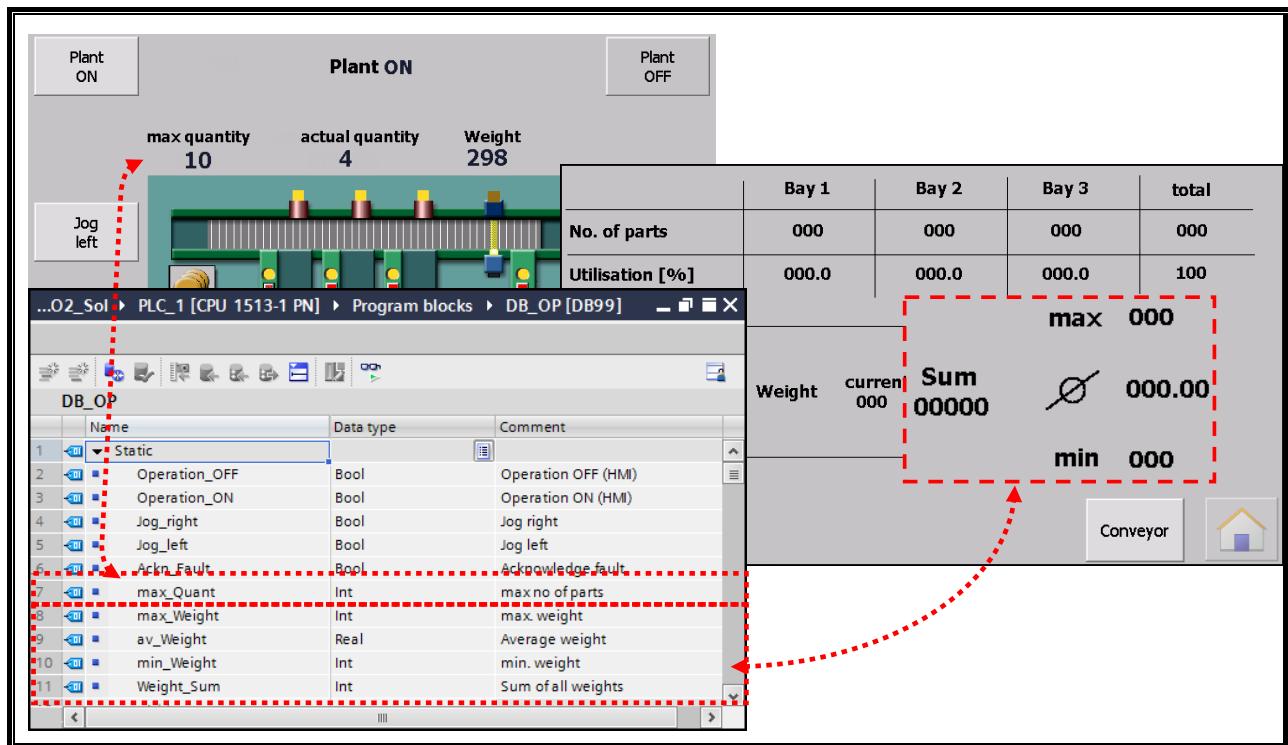
Task

The PLC data type "UDT_WeightStore" is to be created as shown in the picture, according to which the weight storage or structure variables *WeightStore_A*, *WeightStore_B* and *WeightStore_C* are then to be declared within "DB_Weights" (DB123). These variables are later to be used to save the weights of the transported parts.

What to Do

1. Create the new PLC data type "UDT_WeightStore" as shown in the picture. Make sure that you create the component *max_Quant* with a default value **10**, which serves as the maximum number of storable part weights.
2. Create the new "DB_Weights" (DB123) and in it declare the weight storage or structure variables *WeightStore_A*, *WeightStore_B* and *WeightStore_C* according to "UDT_WeightStore" (see picture).
3. Save your project.

7.6.1. Exercise 2: Expanding "DB_OP" (DB99) and Connecting the TP Screen "Statistic"



Task

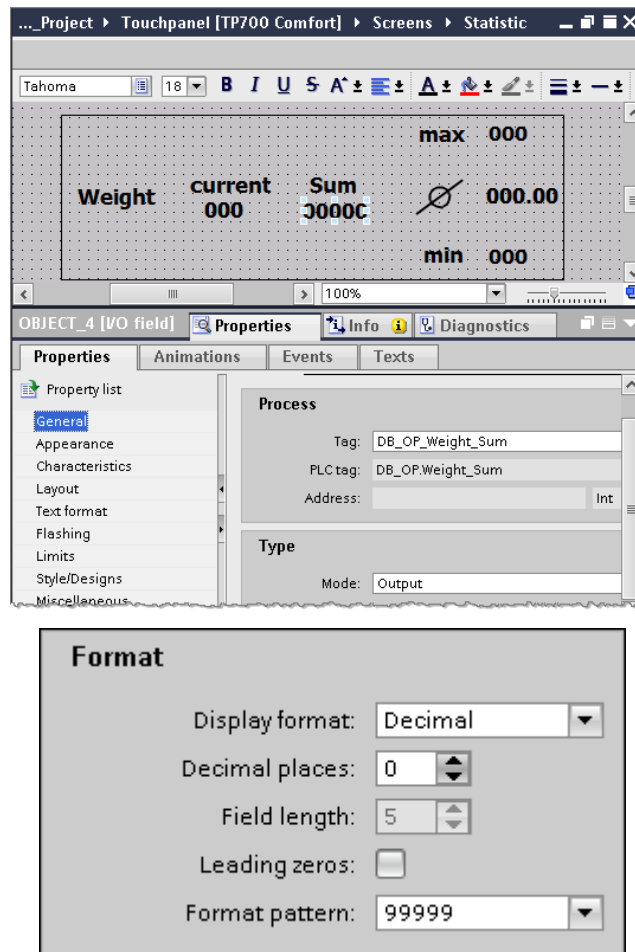
In addition to the already displayed statistic values with regards to part quantities and current weight, the maximum, minimum and average value of saved weights so far as well as their sum are now also to be displayed in the touchpanel screen "Statistic".

What to Do

1. Expand "DB_OP" (DB99) by adding the variables *max_Weight*, *av_Weight*, *min_Weight* and **Weight_Sum** with the relevant data types as shown in the picture.

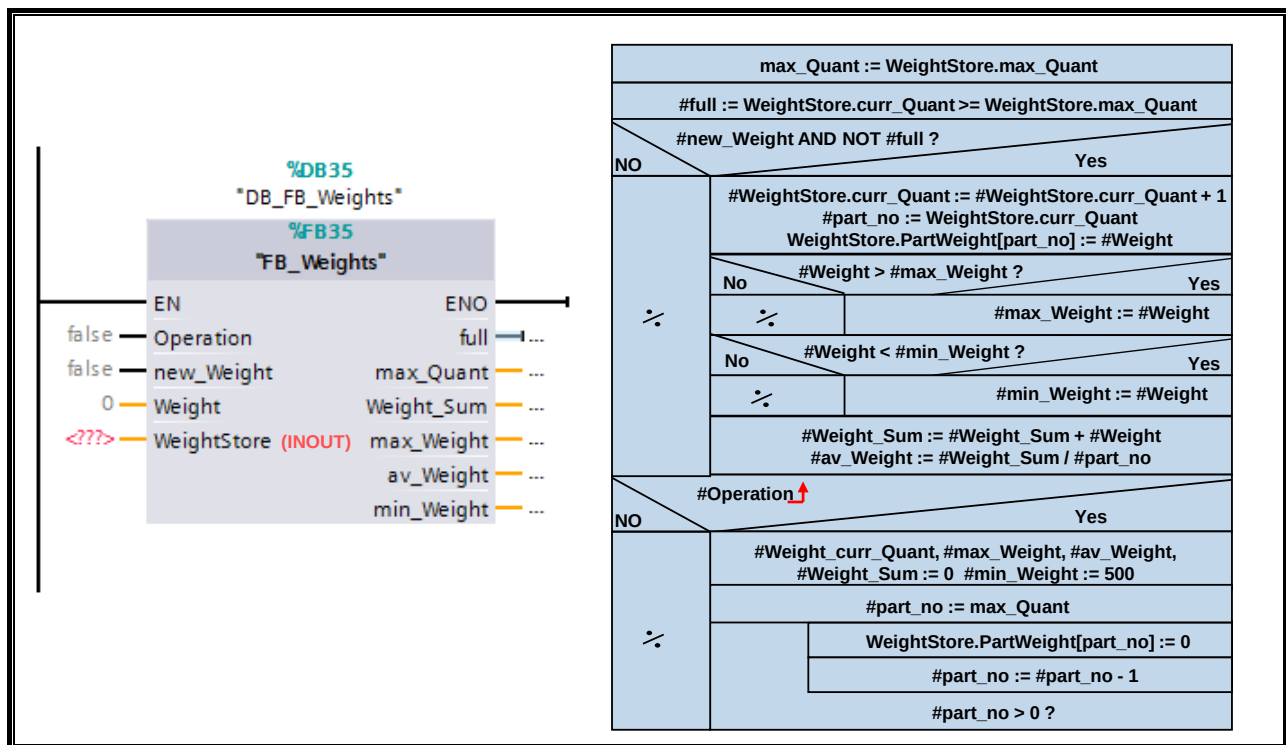
Continued on the next page

- Open the touchpanel screen "Statistic" and, in the Properties of the output fields **Sum**, **max**, **av** and **min**, define the relevant PLC tags from the previously created "DB_OP" (DB99) as variables.



- Load the modified configuration into the touchpanel and in the Inspector window check whether the compilation and loading could be carried out without error.
- Save your project.

7.6.2. Exercise 3: Programming "FB_Weights" (FB35)



Task

The new block "FB_Weights" (FB35) is to be programmed. It is to save the weights of the transported parts in a weight-storage and determine the maximum, average and minimum value of the saved weights as well as their sum.

What to Do

1. Insert the new "FB_Weights" (FB35) block and provide it with the following parameters (also see picture):

- **Input Parameters (IN)**

#Operation (BOOL)

With a rising edge at this input, the block overwrites all values of #WeightStore with 0 (exception *max_Quant*)

#new_Weight (BOOL)

With Signal '1' at this input, the block increases the *WeightStore.curr_Quant* by 1 and writes this at parameter *#Weight* in the *WeightStore.PartWeight[part_no]*. Since this is only to occur once per part, an edge must be passed here.

#Weight (INT)

The weight currently set on the simulator.

Continued on the next page

- **Output Parameters (OUT)**

#full (BOOL):

Signal '1', when `WeightStore.curr_Quant >= WeightStore.max_Quant`

#max_Quant (INT):

Output of value from `WeightStore.max_Quant`

#max_Weight (INT):

Largest part weight stored

#av_Weight (REAL):

Average value of all part weights

#min_Weight (INT)

Smallest part weight stored

#Weight_Sum (INT):

Sum of all stored part weights

- **In/Out Parameters (INOUT)**

WeightStore ("UDT_WeightStore")

Weight-storage in which the part weights are saved

- **Static Variables (STATIC)**

#aux_operation_on (BOOL)

Evaluation of the positive edge at input `#Operation` for initializing the weight storage

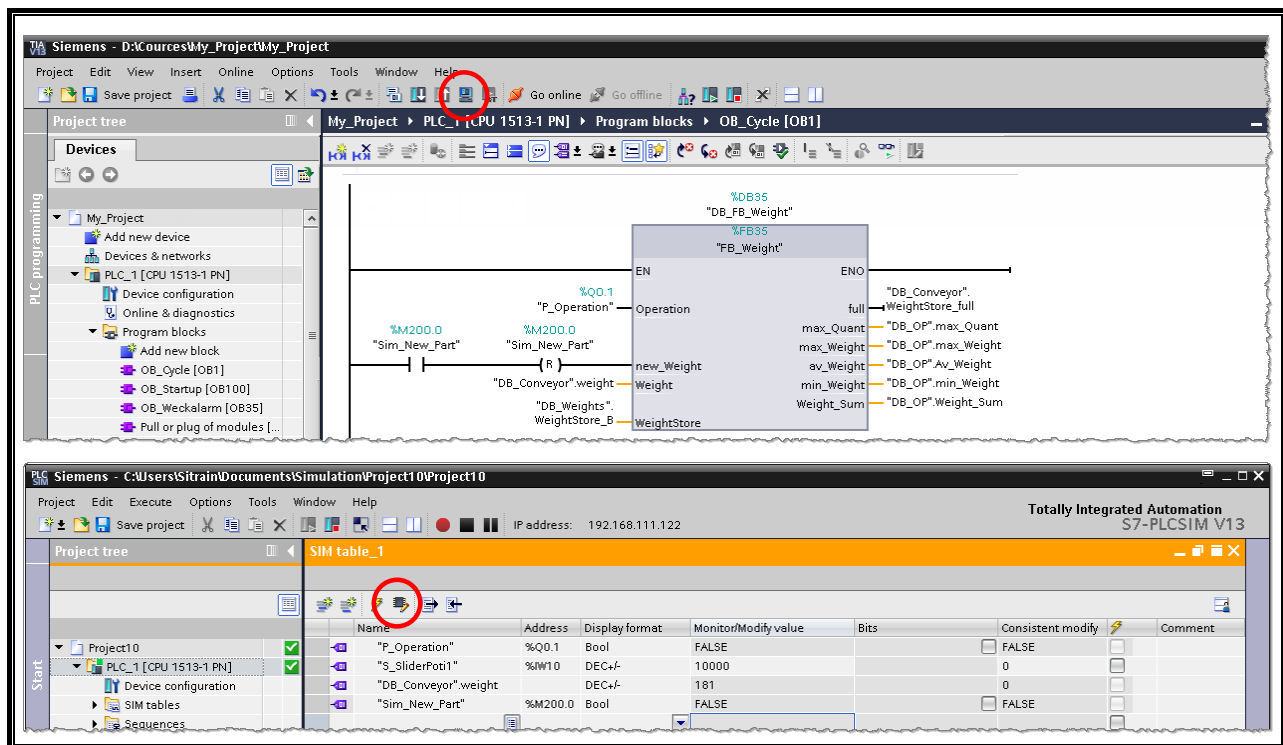
- **Temporary Variables:**

#part_no (DINT)

Number of the current part weight = `WeightStore.curr_Quant`

2. Write a program for "FB_Weights" (FB35) and proceed as shown in the structogram.

7.6.3. Exercise 4: Testing "FB_Weights" (FB35) using PLCSIM

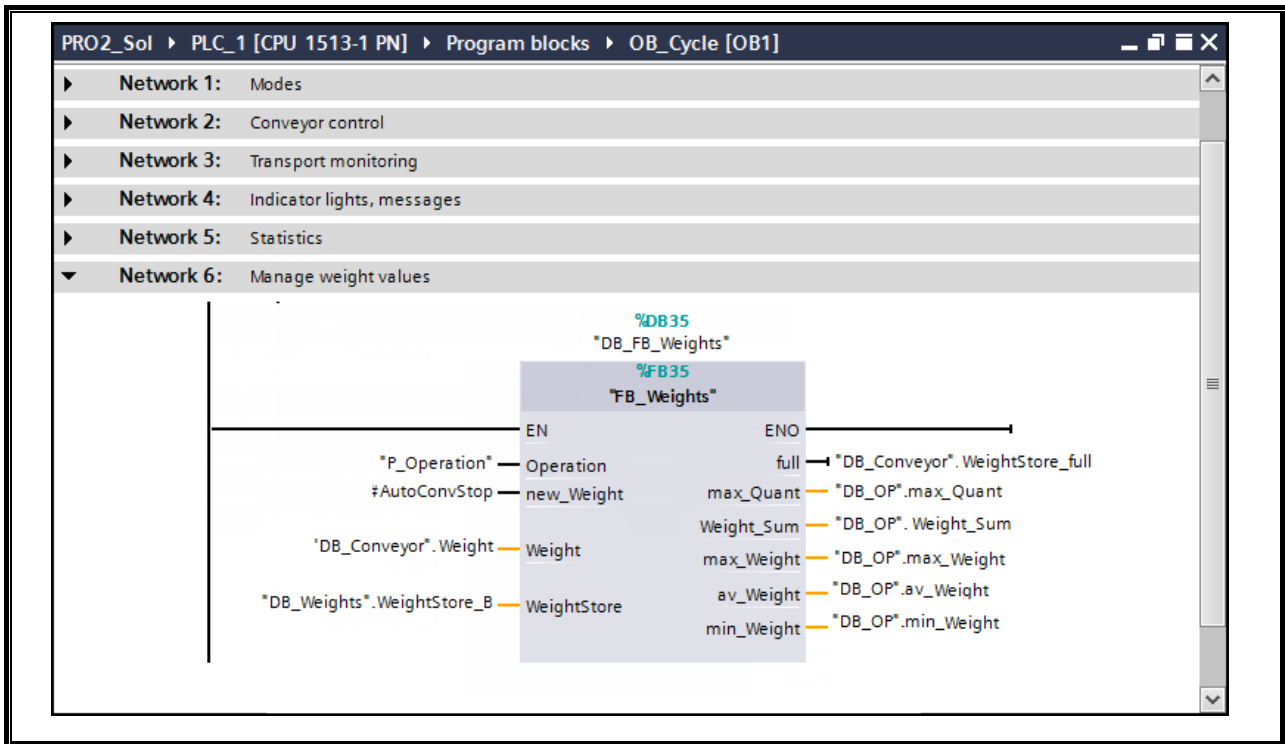


Checking the Functioning of the Block "FB_Weight(s)" with PLCSIM

What to Do:

1. Expand "DB_Conveyor" (DB100) by adding the new variable *WeightStore_full* (BOOL)
2. In OB1, program the call of "FB_Weight(s)" (FB35) and assign parameters to it using the actual operands shown in the picture.
3. Start the simulation of the CPU and specify "PLCSIM S71200/S71500" for the PG/PC interface in the dialog.
4. In PLCSIM, open the SIM table or create a new table.
5. In the table, enter the variables "P_Operation" (for switching the operating modes), "S_SliderPot1" (for setting the weight), "DB_Conveyor".weight (for checking the weight) and M200.0 ("Sim_New_Part" as simulation bit for "New part was transported").
6. Open the data blocks "DB_Weights" and "DB_OP" and monitor (observe) the WeightStore_B of the data block "DB_Weights" as well as the calculated values in data block "DB_OP".
7. In PLCSIM, give the variable S_SliderPot1 a value between 0 and 27648 decimal (the values correspond to the values of the variable "DB_Conveyor".weight 0 to 500 decimal),
8. Activate (Enable) the function "Modification of non-inputs" using the button , in order to change outputs, memory bits and variables of data blocks.
9. In the column "Bits", activate the bit "P_Operation" for Operation = ON and, for simulating a newly transported part the bit M200.0 "Sim_New_Part" (the memory bit is automatically reset to 0 in the program and so is TRUE for only one cycle).
10. Check whether the values of "DB_Weights".WeightStore_B are correct.

7.6.4. Exercise 5: Commissioning "FB_Weights" (FB35)



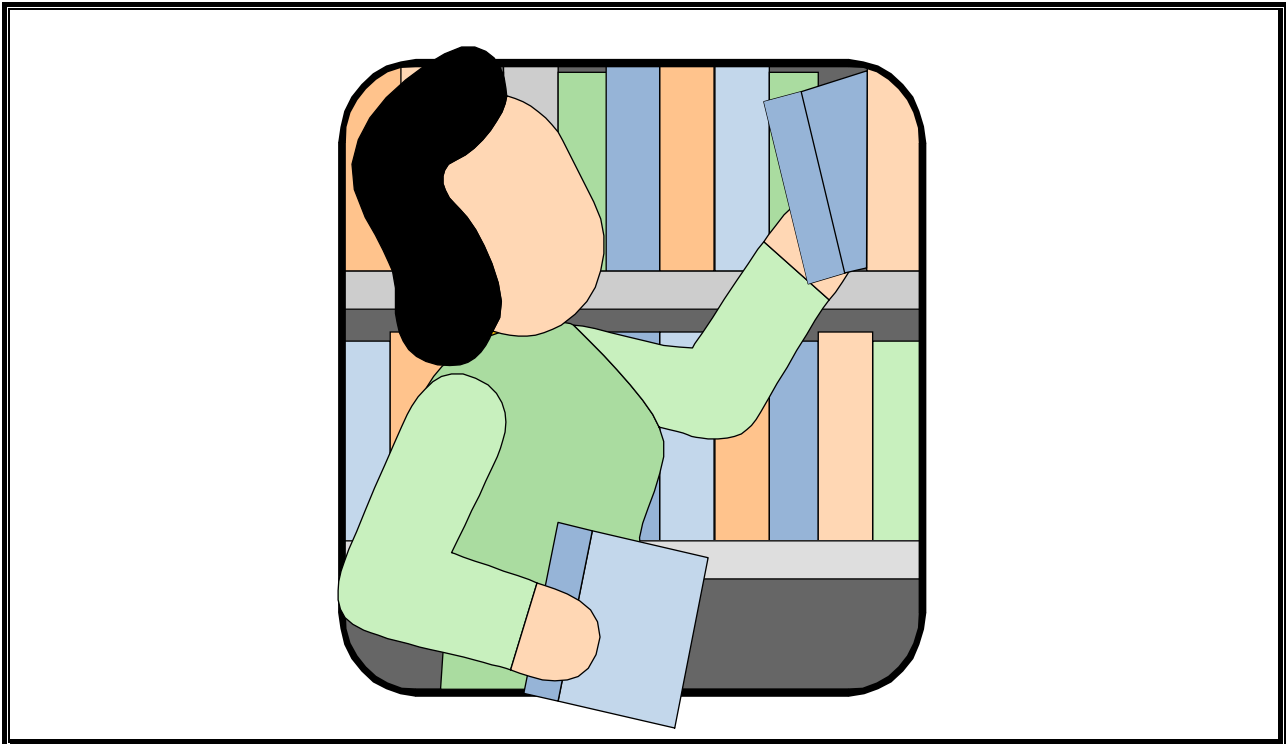
Task

- To integrate "FB_Weights" (FB35) in the program, it is to be called in OB1 and parameterized with the actual operands shown in the picture.

What to Do

- Logically combine the new variable "DB_Conveyor".WeightStore_full in "FC_Signal" (FC14) in such a way that the conveyor model indicator light "P_BayLB" (Q4.4) remains dark when the WeightStore is full.
Furthermore, logically combine this new variable in "FC_ConvMotor" (FC16) in such a way that no part transportation can be started when the WeightStore is full.
- In OB1, change the parameterization of "FB_Weights" (FB35) with the actual operands shown in the picture.
- Download all S7 blocks into the CPU and check the new program functions by interpreting the values shown in the touchpanel screen "Statistic".
- Save your project.

7.7. Additional Information



7.7.1. Settings for Simulating an HMI Device

The screenshot displays the Siemens TIA Portal interface. On the left, the 'Project tree' shows the hierarchy: My_Project > Touchpanel [TP700 Comfort] > Connections. The 'Connections' table lists 'HMI_Connection' with 'SIMATIC S7 1500' as the communication driver. The 'Parameter' tab for 'TP700 Comfort' shows the 'Interface' set to 'ETHERNET'. The 'HMI device' section shows the 'Address' as '192.168.1.111' and the 'Access point' as 'S7ONLINE'. A red dashed box highlights the 'Access point' field. To the right, the 'Set PG/PC Interface (32-bit)' dialog box is open, showing the 'Access Point of the Application' set to 'S7ONLINE (STEP 7)' and the 'Interface Parameter Assignment Used' set to 'Intel(R) 82574L Gigabit Network Connection'. A red dashed box highlights the 'Access Point of the Application' field. Above the dialog box, a flowchart shows the sequence: Windows logo icon → Control Panel → Set PG/PC Interface (32-bit) → Set PG/PC Interface dialog box.

Example of Interface Parameter Assignment Used:

- Intel(R) 82574L Gigabit Network Connection
- PLCSIM S7-1200/S7-1500.TCPIP.1

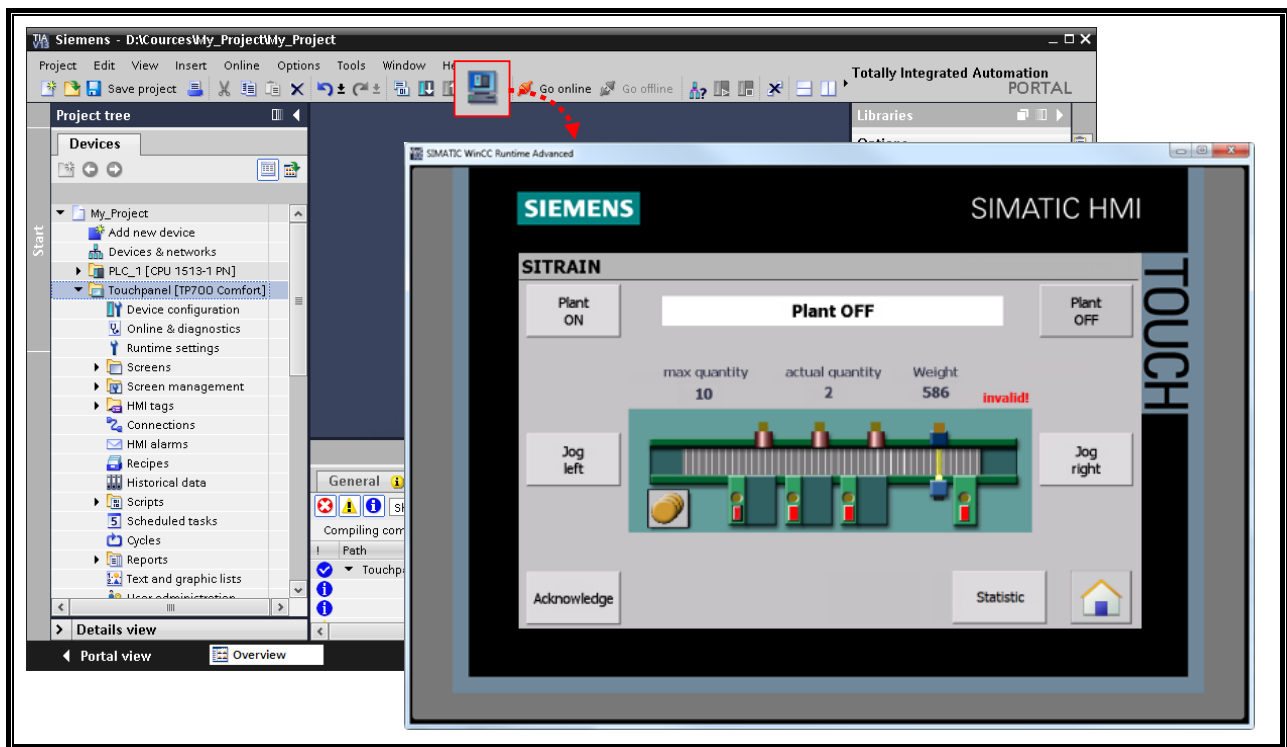
Access via given network card (on real CPU)
Access to simulated CPU in PLCSIM S7-1200/1500

HMI devices (the touchpanel) can also be simulated. For this, however, you must first set through which interface the simulation is to access the PLC controller.

In the Connections of the HMI device, the correct Access point must be specified (default setting is "S7ONLINE") and this, in turn, must be assigned the correct PG/PC interface (see picture "Set PG/PC Interface").

You can select a real network card when the communication is to be established to a controller connected via this network card (select the relevant network card). Furthermore, you can specify that the communication is to take place to a simulated CPU which is simulated with the PLCSIM S7-1200/1500 software. (select PLCSIM S7-1200/1500.xxx, whereby xxx corresponds to the type of interface).

7.7.2. Simulating an HMI Device



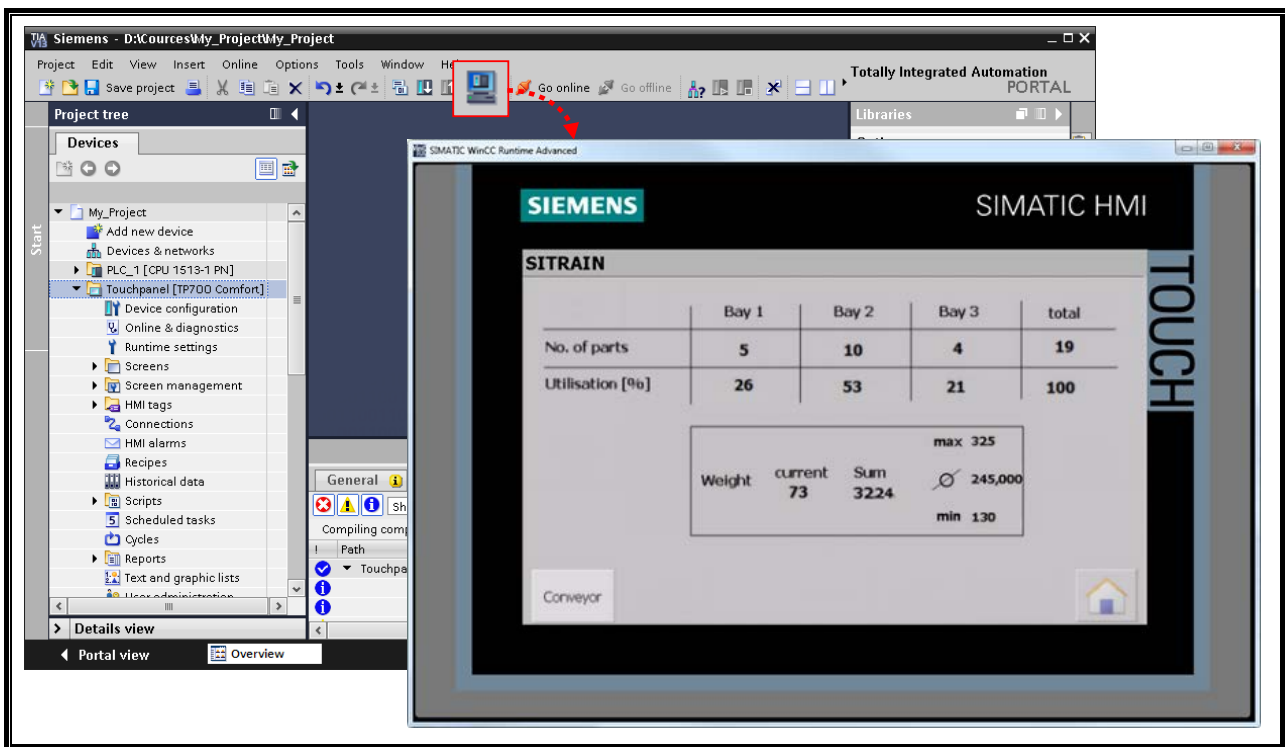
Starting the Simulation:

The simulation of an HMI device is started with the button  "Start simulation".

Note:

The simulation can be active parallel to a real HMI device.

7.7.3. Additional Exercise: Simulating the Touchpanel



Start the simulation of the touchpanel:

What to Do:

1. Check the configured Access point in the connections.
2. In the window "Set PG/PC Interface", assign the appropriate interface to the Access point "S7ONLINE".
3. Start the simulation of the touchpanel using the button  "Start simulation".
4. Check the displayed values and the functioning of the individual buttons.

7.7.4. Extended Instructions: STRING, DT

Extended instructions		Extended instructions	
Name	Description	Name	Description
▶ Date and time-of-day		▶ Date and time-of-day	
▶ String + Char		▶ String + Char	
▢ S_MOVE	Move character string	▢ T_COMP	Compare time tags
▢ S_COMP	Compare character strings	▢ T_CONV	Convert times and extract
▢ S_CONV	Convert character string	▢ T_ADD	Add times
▢ STRG_VAL	Convert character string to numerical value	▢ T_SUB	Subtract times
▢ VAL_STRG	Convert numerical value to character string	▢ T_DIFF	Time difference
▢ Strg_TO_Chars	Convert character string to Array of CHAR	▢ T_COMBINE	Combine times
▢ Chars_TO_Strg	Convert Array of CHAR to character string	Clock functions	
▢ MAX_LEN	Determine the length of a character string	▢ WR_SYS_T	Set time-of-day
▢ ATH	Convert ASCII string to hexadecimal number	▢ RD_SYS_T	Read time-of-day
▢ HTA	Convert hexadecimal number to ASCII string	▢ RD_LOC_T	Read local time
Instructions		▢ WR_LOC_T	Write local time
▢ LEN	Determine the length of a character string	▢ SNC_RTCB	
▢ CONCAT	Combine character strings	▢ TIME_TCK	Read system time
▢ LEFT	Read the left characters of a character string	▢ RTM	Runtime meters
▢ RIGHT	Read the right characters of a character string		
▢ MID	Read the middle characters of a character string		
▢ DELETE	Delete characters in a character string		
▢ INSERT	Insert characters in a character string		
▢ REPLACE	Replace characters in a character string		
▢ FIND	Find characters in a character string		
▶ Process image			

Contents

8.	Blocks with "Optimized Block Access"	8-2
8.1.	Task Description: Downloading Structurally Changed Data Blocks without Re-initialization	8-3
8.2.	Programming Languages: Overview	8-4
8.3.	System Architecture of S7-1500	8-5
8.4.	Block Attribute: Optimized Block Access	8-6
8.4.1.	Comparison: Standard / Optimized Block Accesses	8-7
8.4.2.	"Standard" and "Optimized" Block Access	8-8
8.4.3.	Optimized Blocks (All): Automatic Initialization of Temporary Variables	8-9
8.4.4.	Principle: "Download without Re-initialization" Structurally Changed Data Blocks	8-10
8.4.5.	Memory Reserve of Data Blocks	8-11
8.4.6.	Data Block Initialization after Structural Changes	8-12
8.4.7.	Data Blocks "Download without Re-initialization"	8-13
8.4.8.	Resetting the Data Block Memory Reserve	8-14
8.4.9.	Downloading without Re-initialization for Structurally Changed Function Blocks	8-15
8.4.10.	Exercise 1: Expanding a Data Block and Downloading it into the CPU without Re-initialization	8-16
8.4.11.	Exercise 2: Resetting the Data Block Memory Reserve	8-17
8.5.	Initializing Setpoints in the Online Program	8-18
8.6.	Additional Information	8-19
8.6.1.	Data Block Control	8-20

8. Blocks with "Optimized Block Access"

At the end of the chapter the participant will...



- ... be able to explain the difference between blocks with standard access and with optimized access
- ... be able to explain under which conditions DBs can be downloaded into the CPU with and without re-initialization
- ... be able to download an expanded data block into the CPU without re-initialization

8.1. Task Description: Downloading Structurally Changed Data Blocks without Re-initialization

1
Add variable (tag) *new_var* and download DB into CPU via

2
Variables (tags) were not re-initialized

Name	Data type	Default value	Start value	Monitor value	Retain
Static					
WeightStore_A	*UDT_WeightStore*				
WeightStore_B	*UDT_WeightStore*				
max_Quant	Int	10	10	10	
curr_Quant	Int	0	0	3	
PartWeight	Array [1..10] of Int				
PartWeight[1]	Int	0	0	272	
PartWeight[2]	Int	0	0	245	
PartWeight[3]	Int	0	0	310	
PartWeight[4]	Int				
PartWeight[5]	Int				
PartWeight[6]	Int				
PartWeight[7]	Int				
PartWeight[8]	Int				
PartWeight[9]	Int				
PartWeight[10]	Int				
WeightStore_C	*UDT_WeightStore*				

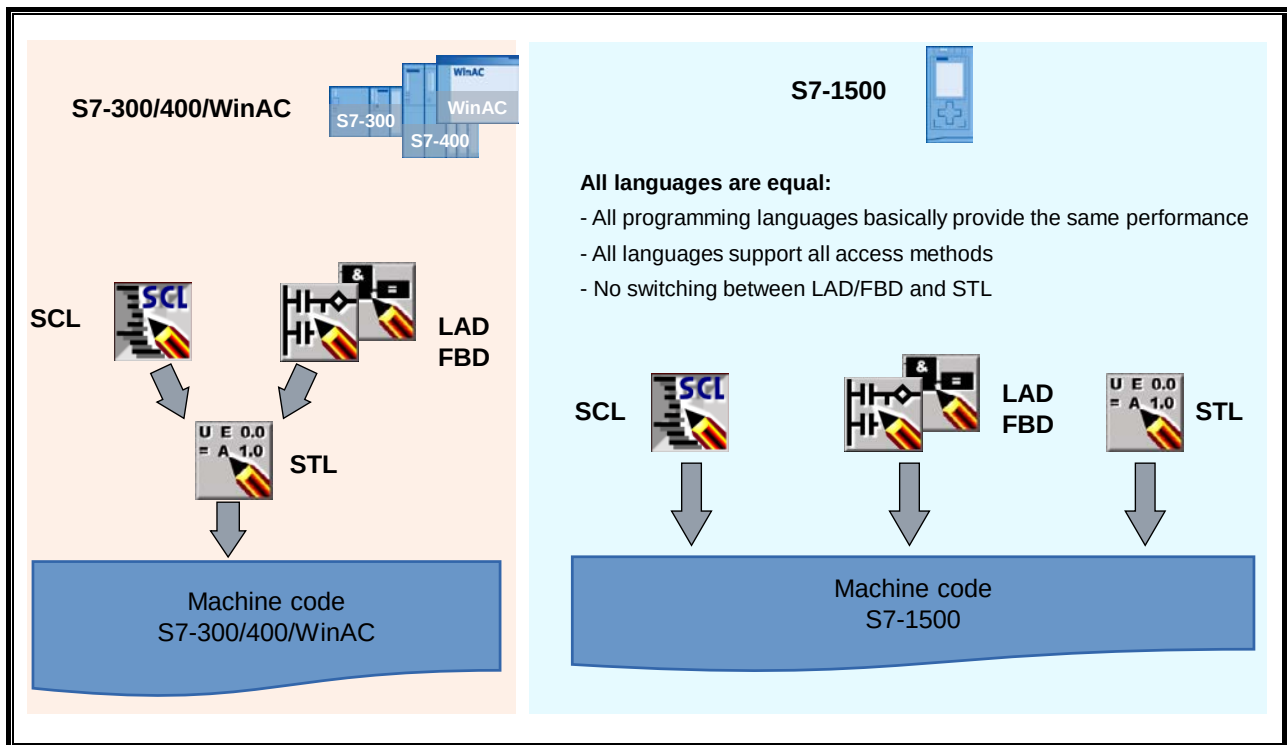
Name	Data type	Default value	Start value	Monitor value	Retain
Static					
WeightStore_A	*UDT_WeightStore*				
new_var	Real	0.0	3.14	3.14	
WeightStore_B	*UDT_WeightStore*				
max_Quant	Int	10	10	10	
curr_Quant	Int	0	0	3	
PartWeight	Array [1..10] of Int				
PartWeight[1]	Int	0	0	272	
PartWeight[2]	Int	0	0	245	
PartWeight[3]	Int	0	0	310	
PartWeight[4]	Int				
PartWeight[5]	Int				
PartWeight[6]	Int				
PartWeight[7]	Int				
PartWeight[8]	Int				
PartWeight[9]	Int				
PartWeight[10]	Int	0	0	0	
WeightStore_C	*UDT_WeightStore*				

Task Description

During running operation or in the CPU RUN state, the S7-1500 allows the downloading of data blocks into the CPU that were structurally changed or that were expanded by adding variables (tags) without overwriting the monitoring values (actual values) of the originally already existing variables. In this way the loading of program expansions when a system is running is made possible without losing current system data in the process.

In this chapter, you are to become familiar with this "Downloading without re-initialization" of structurally changed data blocks.

8.2. Programming Languages: Overview



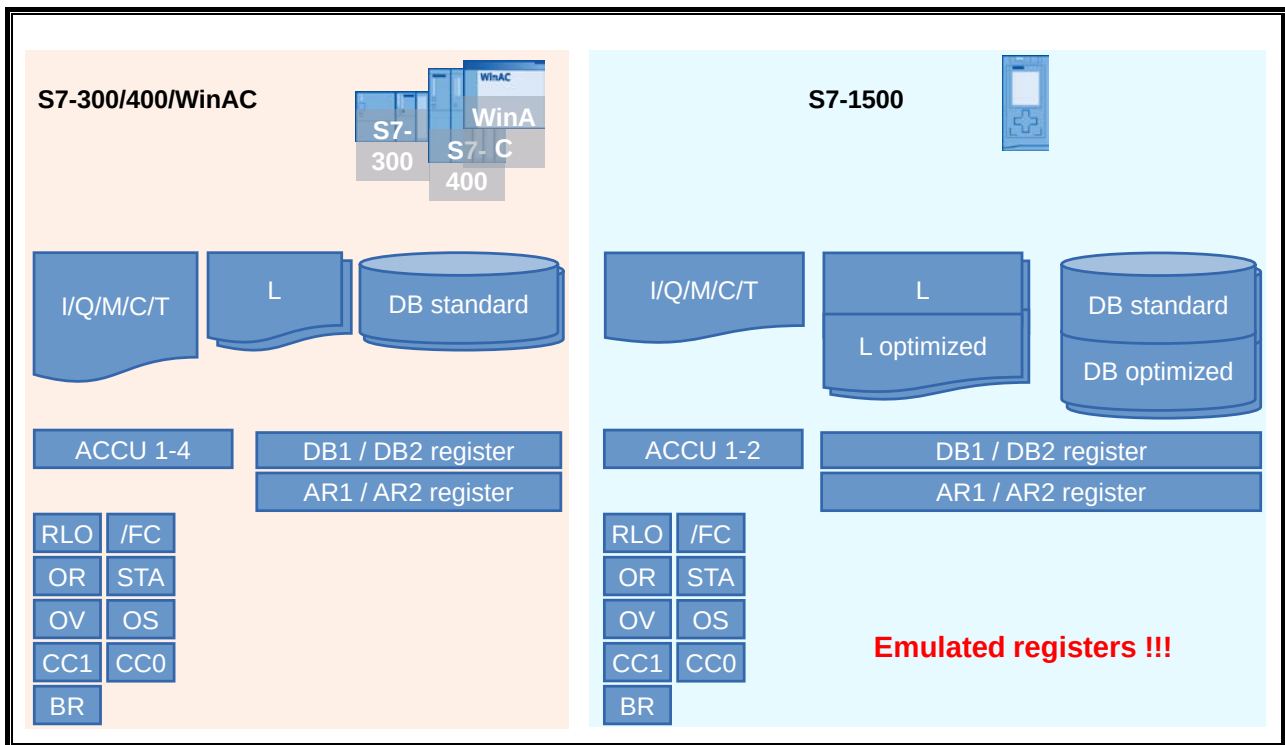
Overview of Programming Languages

While for S7-300/400, the programs of the programming languages LAD/FBD/SCL were first of all compiled in STL and then from STL into the machine code of the controller, for S7-1500, all programming languages have their own compiler which compiles the respective language directly into the machine code of the controller.

The table shown below gives an overview of which programming languages are available for the various controllers and which extended access methods they offer.

Programming languages	Controller			Additional access methods		
	S7-300/400 	S7-1200 	S7-1500 	Indexed Example: my_array[#idx]	AT construct (Standard blocks)	Slice access Example: My_word.x1
LAD / FBD	✓	✓	✓	S7-1200 S7-1500		 
STL	✓	✗	✓	S7-1500		
SCL	✓	✓	✓	All ✓	All ✓	S7-1200 S7-1500
Graph	✓	✗	✓	S7-1500		

8.3. System Architecture of S7-1500



System Architecture of S7-1500

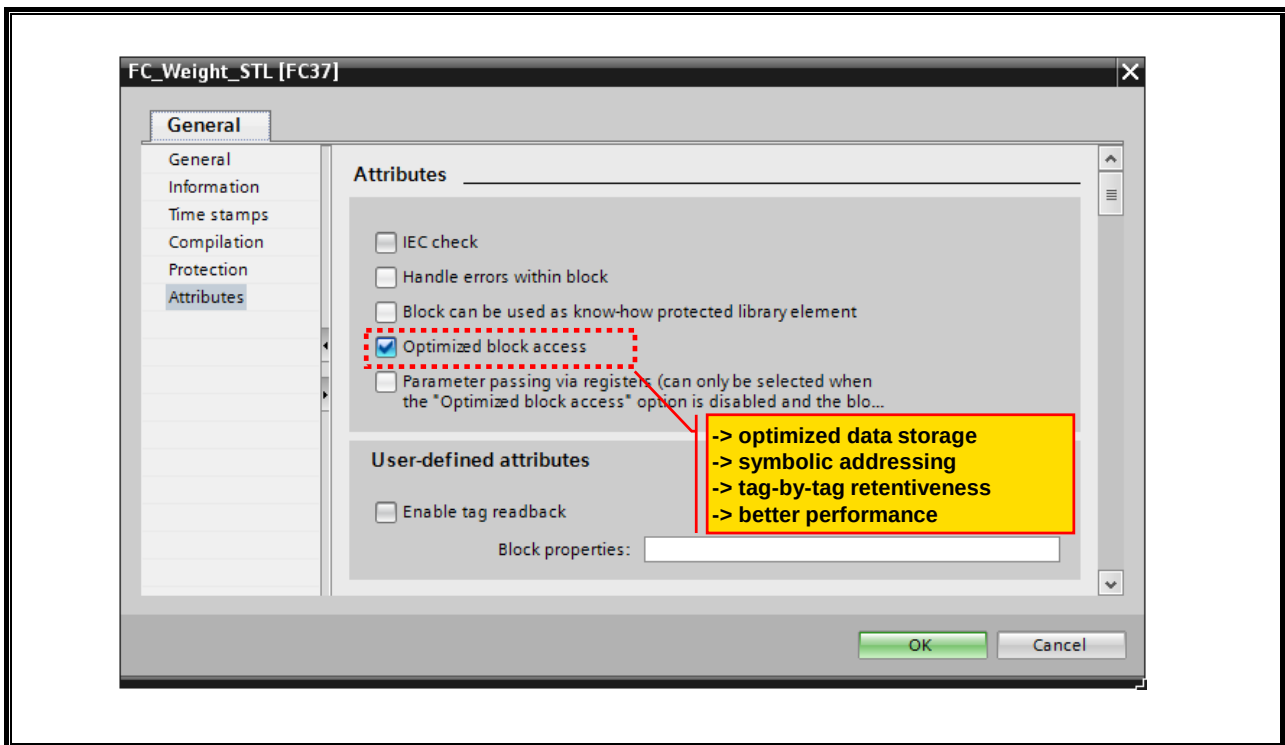
With the exception of S7-400 instructions which explicitly refer to ACCU 3/4 of the CPU, all instructions of S7-300/400 are supported by S7-1500.

During the execution of the instructions, the CPU registers of S7-300 are emulated whereby the instructions show the same runtime behavior.

Limitations in STL:

No variables of the dimension 64 bits can be loaded into the emulated registers, that is, that these variables cannot be processed with elementary STL instructions. For this, blocks are available in the global library "Long Functions".

8.4. Block Attribute: Optimized Block Access



Attribute "Optimized Block Access"

The "Optimized block access" attribute can be assigned to all logic and data blocks. Blocks that are created as optimized have the following advantages:

For Data Blocks:

- The retentive behavior of the variables (tags) can be adjusted variable-by-variable
- With S7-1200, these DBs are created memory-optimized
- With S7-1500, these DBs are created access-optimized
- The variables of the data block can only be accessed symbolically

For Logic Blocks:

- The logic of the block is compiled runtime-optimized
- No access to CPU registers
- No indirect addressing via pointer with absolute addresses

Block can be used as Know-how Protected Library Element

This attribute is assigned by the Block Editor when no global operands (inputs, outputs, memory bits, SIMATIC timers/counters, data blocks, global constants) are used in the program of the block and the block thus fulfills the requirements of IEC61131-3.

Communication HMI Device ↔ DB in CPU

If the HMI device is integrated in the project, it can symbolically access a DB of the CPU, that is, the DB can also be created as "optimized".

If the HMI device is not integrated in the project (because it is a third-party device or an older panel still configured with ProTool), it can only access a DB of the CPU absolutely, that is, the DB must not be created as "optimized" in the CPU.

8.4.1. Comparison: Standard / Optimized Block Accesses

	Standard Block Access (S7-1200/1500, compatible with S7-300/400)	Optimized Block Access (only S7-1200/1500)
Data storage	Any, depending on the user declaration	Managed and optimized by the system: The user can set up any structure, without burdening the memory capacity.
Performance	The access is not always as fast as it could be for S7-1200/1500, since the data could be "unfavorably" located depending on the declaration.	The access always takes place as quickly as possible, since the data storage is optimized by the system and is independent of the declaration.
Prone to errors	Absolute accesses (e.g. from HMI or through indirect addressing) can lead to inconsistencies with declaration changes.	Lower, since generally symbolic accesses are used. Declaration changes do not lead to access errors since, e.g., HMI accesses take place symbolically.
Retentiveness	Block-by-block	Variable-by-variable (Tag-by-tag)

Recommendation: Mix standard and optimized blocks as little as possible. The system automatically converts the data models during access between the blocks which, however, uses up resources!

Data Storage

For the declaration of global variables (tags) within global data blocks or of static variables (tags) within function blocks, the Block Editor inserts "fill bytes", if necessary, for blocks with "standard access". This occurs, for example, when variables of the data types BOOL and INT are declared in alternate sequence because INT variables are always placed on even absolute addresses for blocks with "standard access".

Performance

If both the data block and the logic block which accesses the data block are created with the attribute "Optimized block access", the accesses take place with optimized performance.

Prone to Errors

Since optimized blocks work exclusively with symbolic variables, they are less prone to error. Also, the readability of a program improves because of the use of symbolic variables.

8.4.2. "Standard" and "Optimized" Block Access

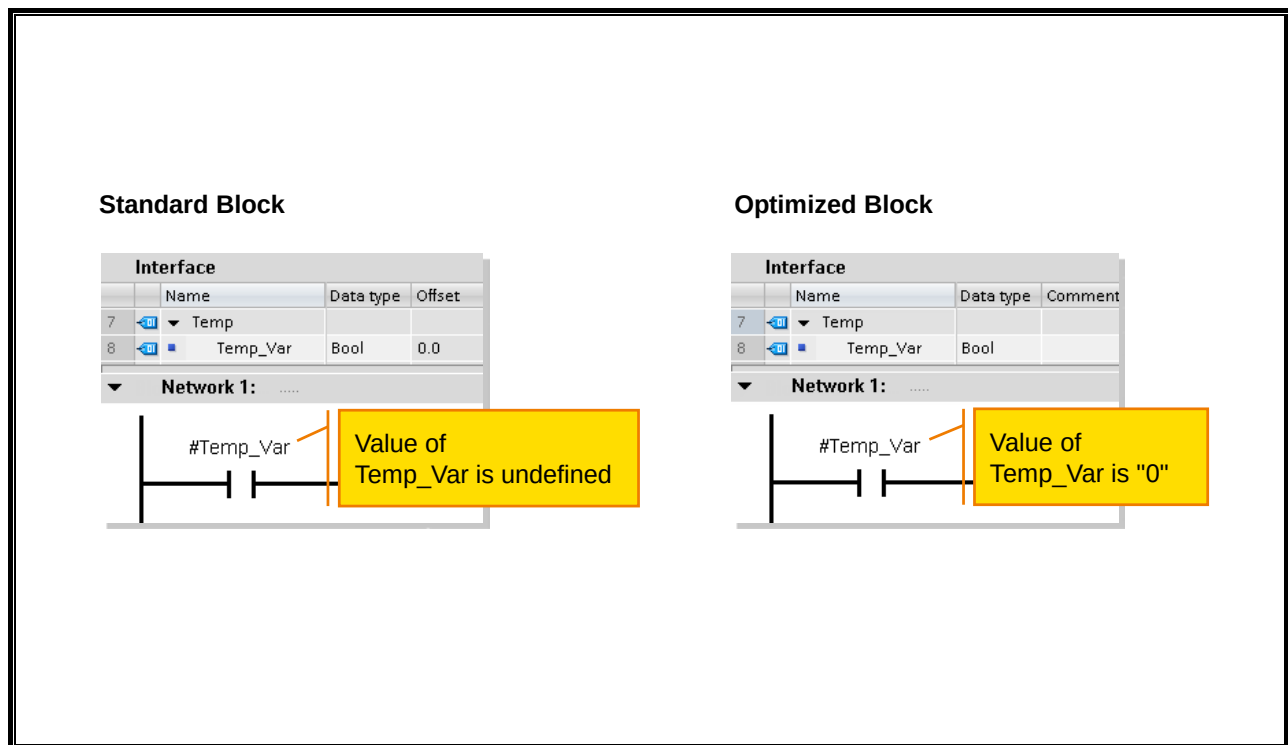
Access Methods	Standard DB S7-1200/1500 (compatible with S7-300/400)	DB with "Optimized block access" attribute (only S7-1200/1500)
Symbolic	✓	✓
Indexed (Arrays)	✓	✓
Slice Accesses	✓	✓
AT Construct	✓	✗ * Planned
Absolute	✓	✗
Indirect (Pointer)	✓	✗
Download without re-initialization	✗	✓ * Planned for S7-1200

Standard and Optimized Block Access

The table shows that blocks with "Optimized block access" provide the possibility of "Downloading without re-initialization" expanded data and function blocks, however, do not offer any other additional programming possibilities.

The advantages of the optimized blocks is rather that data blocks are created as memory-optimized (S7-1200) or runtime-optimized with variable-by-variable retentiveness, and that optimized logic (code) blocks have more performance and are less prone to error in the access to these data blocks.

8.4.3. Optimized Blocks (All): Automatic Initialization of Temporary Variables

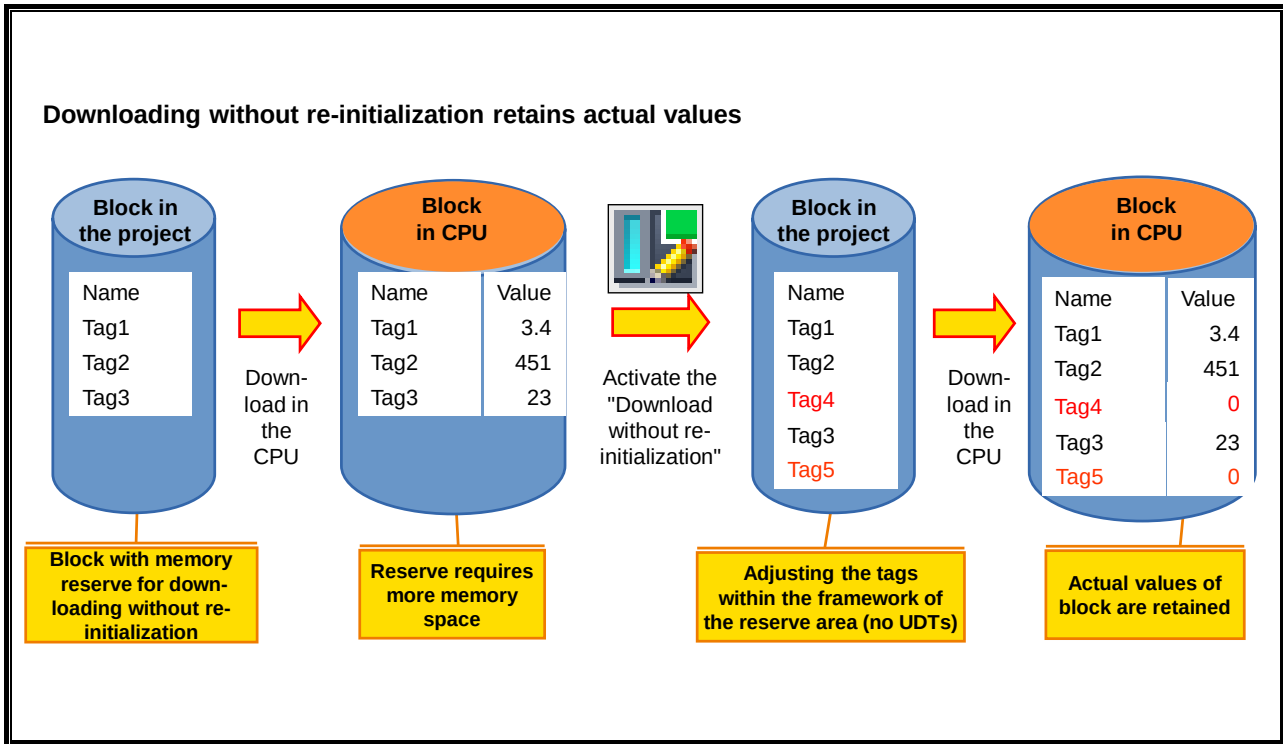


Automatic Initialization of Local Temporary Variables

In standard logic blocks, the local temporary variables are assigned with undefined values at the beginning of block processing.

In optimized logic blocks, the local temporary variables are automatically initialized during the block call or are pre-assigned with the value '0'.

8.4.4. Principle: "Download without Re-initialization" Structurally Changed Data Blocks



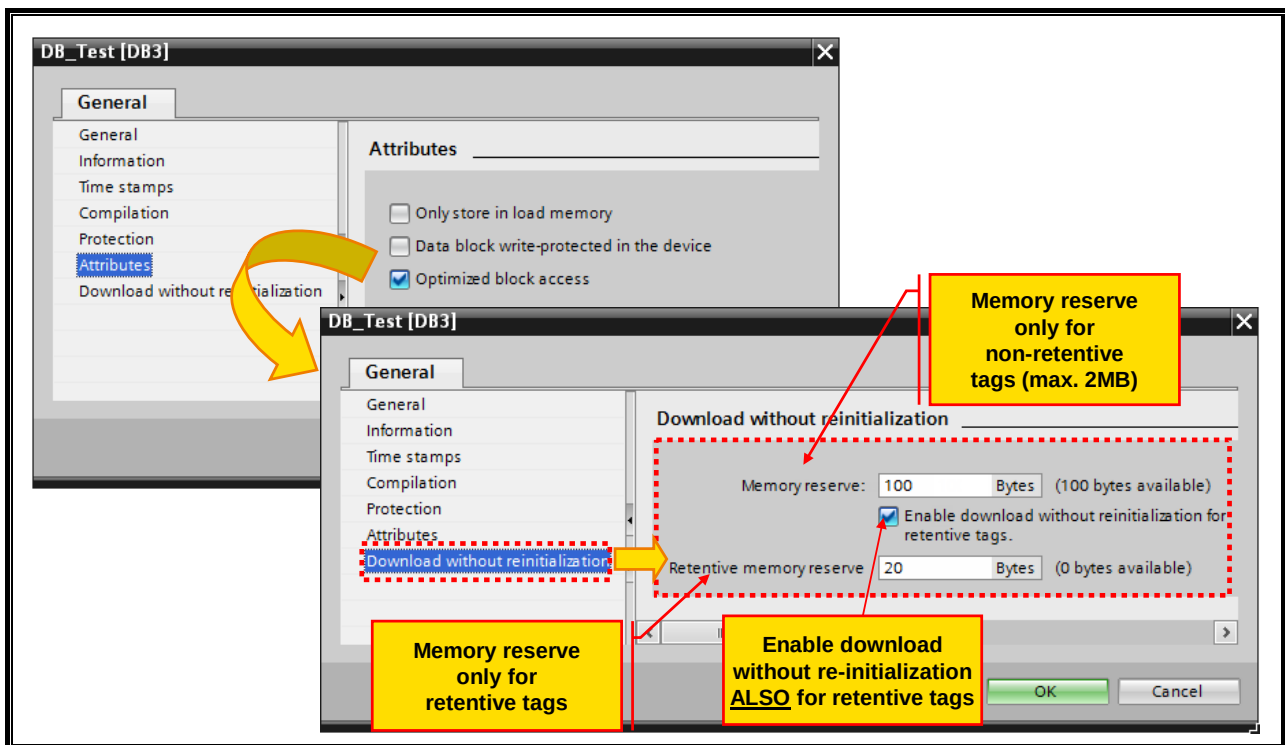
Download without Re-initialization

In order to make structural changes to data blocks or function blocks even while the system is running, modified blocks can be downloaded into the CPU without switching the CPU to STOP and without affecting the current online values in the CPU. The controlled system is not affected by this "Download without re-initialization".

Functional Principle:

Basically, every function block and data block with the attribute "Optimized block access" includes by default a memory reserve which can be used for changes made later on. All later, additionally declared tags are created in this memory reserve. During the subsequent download, the values of already loaded tags (variables) are not affected.

8.4.5. Memory Reserve of Data Blocks



Memory Reserve of Data Blocks

The memory reserve for the "Download without reinitialization" can only be used...

- for data blocks with the attribute "Optimized block access"
- after an expansion of the data block with additional tags (not after deleting already existing tags).

The "Download without reinitialization" of a data block that is to be expanded by **non-retentive** tags, does not have to be explicitly activated (enabled). As well, a memory reserve of 100 bytes is automatically created for this by default.

The "Download without reinitialization" of a data block that is to be expanded by **retentive** tags, on the other hand, must be explicitly activated via "Enable download without reinitialization (also) for retentive tags". As well, the memory reserve used for this "Retentive memory reserve" must be explicitly set up.

8.4.6. Data Block Initialization after Structural Changes

The diagram illustrates the process of Data Block (DB) initialization after structural changes. It shows three states of the **DB_Test** table:

- Initial State:**

	Name	Data type	Start value	Monitor value	Retain	Comment
1	Static					
2	Value1	Int	0	23		
3	Value2	Int	0	45	☒	

Annotations: "Start value changed and DB downloaded in CPU" (pointing to the Start value column), "no initialization" (pointing to the Monitor value column).
- Structural Change:**

Annotation: "Structural change: Name change: Download DB in CPU is only possible with re-initialization" (pointing to the table).

Annotation: "Initialization of all tags" (pointing to the Monitor value column).
- Restart:**

Annotation: "Restart carried out" (pointing to the table).

Annotation: "Initialization of non-retentive tags" (pointing to the Monitor value column).

Downloading Data Blocks with Re-initialization

No matter whether a DB was created with standard or optimized block access, a change of start values is not a structural change. Therefore, it can be done and downloaded into the CPU without a re-initialization of the data block tags (variables) being necessary.

On the other hand, after structural changes, it is only possible to download the data block by re-initializing all tags.

Structural changes are:

- Name changes
- Changes of the retentive behavior
- Adding / removing tags

DB Re-initialization during Restart

No matter whether a DB is created with standard or optimized block access, all monitor values (actual values) of non-retentive tags are overwritten with their start values when the CPU is restarted.

8.4.7. Data Blocks "Download without Re-initialization"

1 Before change:
Enable "Download without re-initialization"
(this is not an action here, rather a property of the DB!)

2 Add tag(s) and download DB in CPU via

No initialization

Download without reinitialization

Memory reserve: 100 Bytes (96 bytes available)

☒ Enable download without reinitialization for retentive tags.

Retentive memory reserve: 20 Bytes (18 bytes available)

Data Blocks "Download without Reinitialization"

The "Download without reinitialization" of a data block can only be used if the data block has been expanded by additional retentive and/or non-retentive tags and this expansion corresponds with the created memory reserve.

The function cannot be used if already existing retentive and/or non-retentive tags were deleted in the data block.

How to Proceed

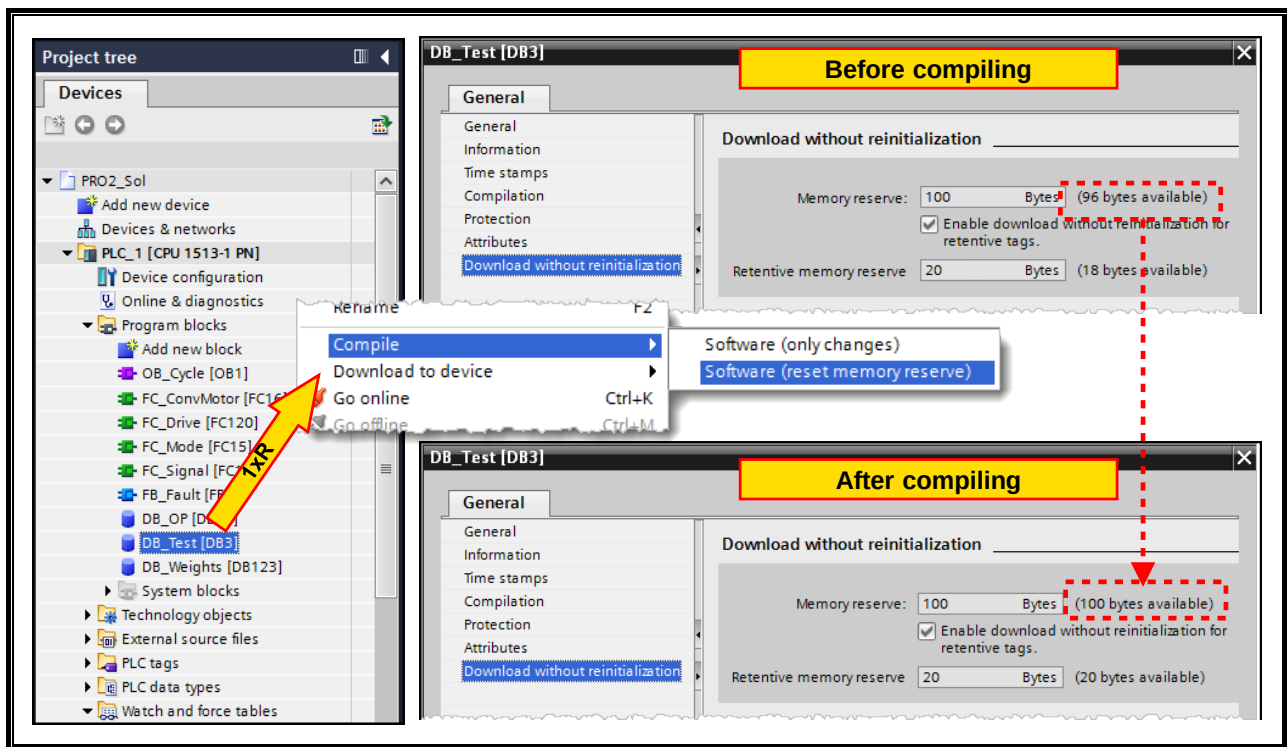


- Before adding tags, the "Download without reinitialization" icon must be pressed in the editor. That way, the data block is not downloaded into the CPU but is merely given the status or the property "Download without reinitialization". In this status, the already existing tags can no longer be edited and new retentive and/or non-retentive tags can be additionally declared.



- The data block expanded in this way can then be downloaded into the CPU without re-initialization using the "Download in CPU" icon or from the Project tree.

8.4.8. Resetting the Data Block Memory Reserve



Reset Memory Reserve

After a data block with the option "Reset memory reserve" is compiled, the tags created in the memory reserve are integrated in the regular part of the data block making the memory reserve completely available once again.

The subsequent download of the data block into the CPU can then only be made with a re-initialization of the tags.

8.4.9. Downloading without Re-initialization for Structurally Changed Function Blocks

Before change:
Enable "Download without re-initialization" (this is not an action here, rather a property of the FB!)

2
Add tag(s) and download FB in CPU via

Download without reinitialization

Memory reserve: 100 Bytes (96 bytes available)
☒ Enable download without reinitialization for retentive tags.
 Retentive memory reserve: 20 Bytes (18 bytes available)

Name	Data type	Default value	Retain
Input			
IN1	Int	0	Non-retain
Output			
OUT1	Real	0.0	Set in IDB
InOut			
Static			
STAT1	Bool	false	Retain
Temp			

Function Blocks "Download without Reinitialization"

The "Download without reinitialization" of a function block can only be used if it has been expanded by additional retentive and/or non-retentive parameters and/or tags and this expansion corresponds with the created memory reserve.

The function cannot be used if already existing retentive and/or non-retentive parameters and/or tags were deleted in the function block.

Functional Principle:

Basically, every function block and data block with the attribute "Optimized block access" includes by default a memory reserve which can be used for changes made later on. All later, additionally declared tags and/or parameters are created in this memory reserve.

For structurally changed function blocks, the changes are automatically also updated in the associated instance DBs. During a consistent "Download without reinitialization" of the expanded function block, the associated instance data blocks are also transferred through a consistent "Download without reinitialization" into the CPU.

In the blocks that call the affected function block, the FB calls are automatically updated. Even these blocks that reference the function block are downloaded into the CPU during a consistent download of the expanded function block.

8.4.10. Exercise 1: Expanding a Data Block and Downloading it into the CPU without Re-initialization

Before the expansion:
Enable "Download without re-initialization"
(this is not an action here, rather a property of the DB!)

1

Add tag *new_var* and download DB into the CPU via

2

3
Tags were not re-initialized

Name	Data type	Default value	Start value	Monitor value	Retain
Static					
WeightStore_A	"UDT_WeightStore"				
new_var	Real	0.0	3.14	3.14	
WeightStore_B	"UDT_WeightStore"				
max_Quant	Int	10	10	10	
curr_Quant	Int	0	0	3	
PartWeight	Array [1..10] of Int				
PartWeight[1]	Int	0	0	272	
PartWeight[2]	Int	0	0	245	
PartWeight[3]	Int	0	0	310	
PartWeight[4]	Int	0	0	0	
PartWeight[5]	Int	0	0	0	
PartWeight[6]	Int	0	0	0	
PartWeight[7]	Int	0	0	0	
PartWeight[8]	Int	0	0	0	
PartWeight[9]	Int	0	0	0	
PartWeight[10]	Int	0	0	0	
WeightStore_C	"UDT_WeightStore"				

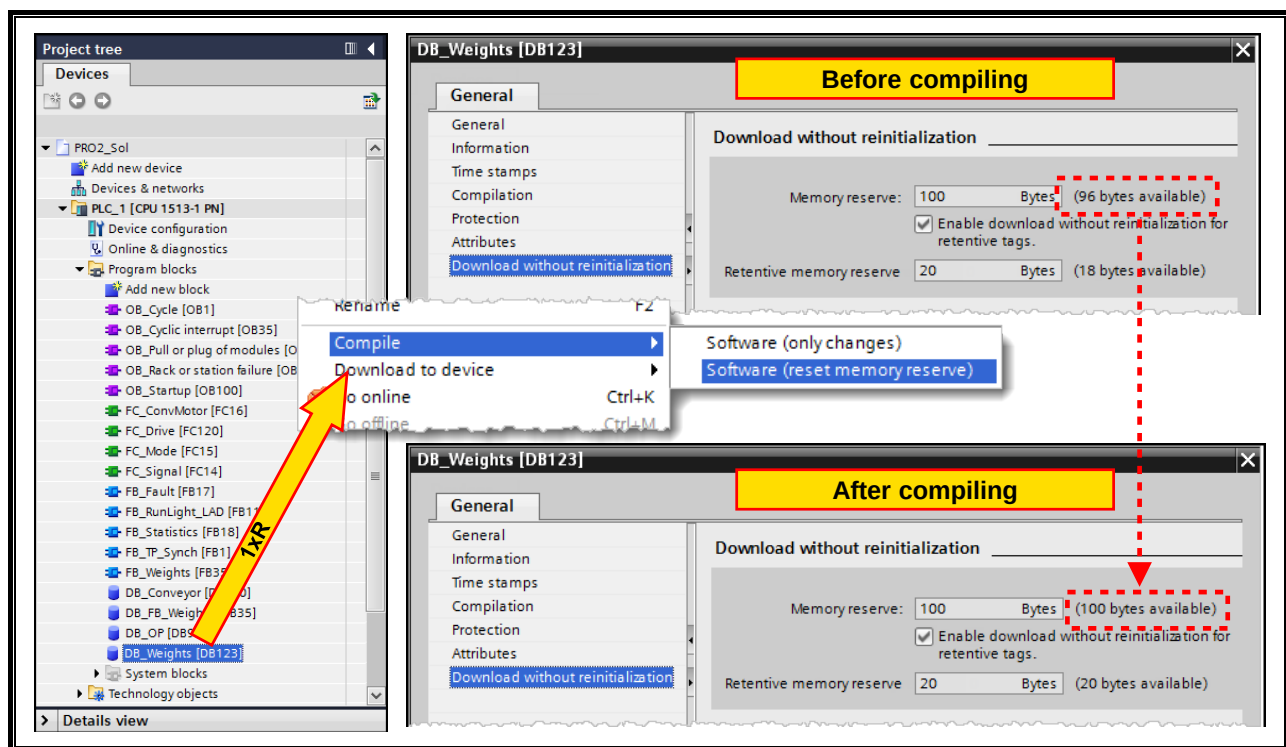
Task

The "DB_Weights" (DB123) is to be expanded by a tag and then downloaded into the CPU without re-initialization.

What to Do

1. In the Properties of the data block "DB_Weights" (DB123), activate the attribute "Optimized block access".
2. In the DB Properties, also activate the option "Download without reinitialization" and declare a memory reserve of 20 bytes in the non-retentive area.
3. Close the Properties dialog and download the changed data block into the CPU (this download sequence is only possible with re-initialization)
4. Open "DB_Weights" (DB123) and start the "Monitor All" function. Let several parts be transported so that their weights are saved in the DB.
5. Press the "Download without reinitialization" icon in order to enable the download without re-initialization.
6. Between the already existing tags *WeightStore_A* and *WeightStore_B*, insert the new, additional tag *new_var* of the data type REAL with the start value 3.14.
7. Download this expanded data block into the CPU and check whether the previously saved part weight values were also retained after the downloading of the data block.
8. Save your project.

8.4.11. Exercise 2: Resetting the Data Block Memory Reserve



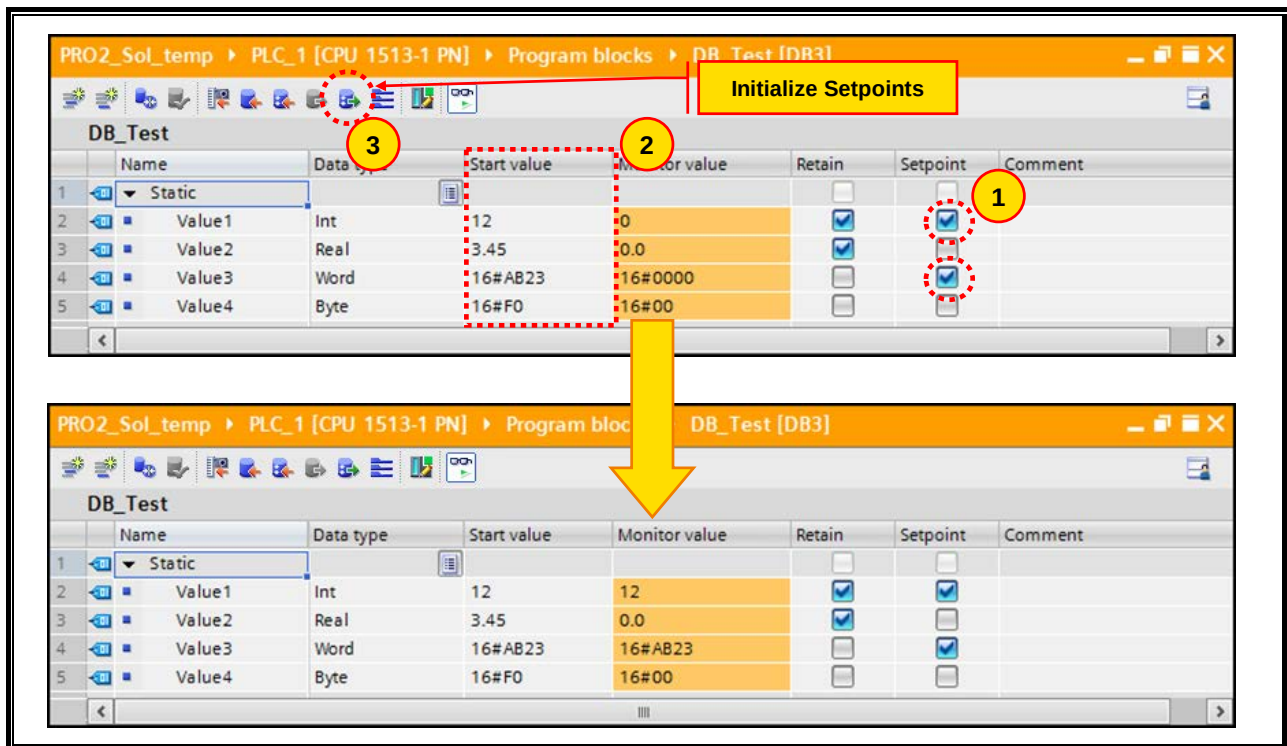
Task:

The tag that was created during the data block expansion is to be removed from the memory reserve and integrated in the regular part of the data block. Then, the data block is to be downloaded into the CPU with re-initialization.

What to Do:

1. Open the Properties dialog of "DB_Weights" (DB123) and check how many bytes of memory reserve are still available (see picture upper right).
2. "Compile" the data block with the option "Software (reset memory reserve)".
3. Open the Properties dialog of "DB_Weights" (DB123) once more and check whether all bytes of the memory reserve are once again available (see picture bottom right).
4. Download the data block into the CPU. Accept the message that this loading process is only possible with re-initialization of the data block.
5. Save your project.

8.5. Initializing Setpoints in the Online Program



Initializing Setpoints in the Online Program

All tags that are selected (highlighted) as "Setpoint" can be initialized online in the CPU. Online, the monitor(ing) values are overwritten with the offline editable Start values. The CPU remains in the "RUN" state. The modified monitor(ing) values (actual values) are adopted once at the next cycle control point. This applies both for retentive and for non-retentive tags. The program execution then continues with the new tag values. Prerequisite is an online connection to the CPU, the structure of the data block is identical online and offline, and, one or more tags are selected as "Setpoint".

Procedure:

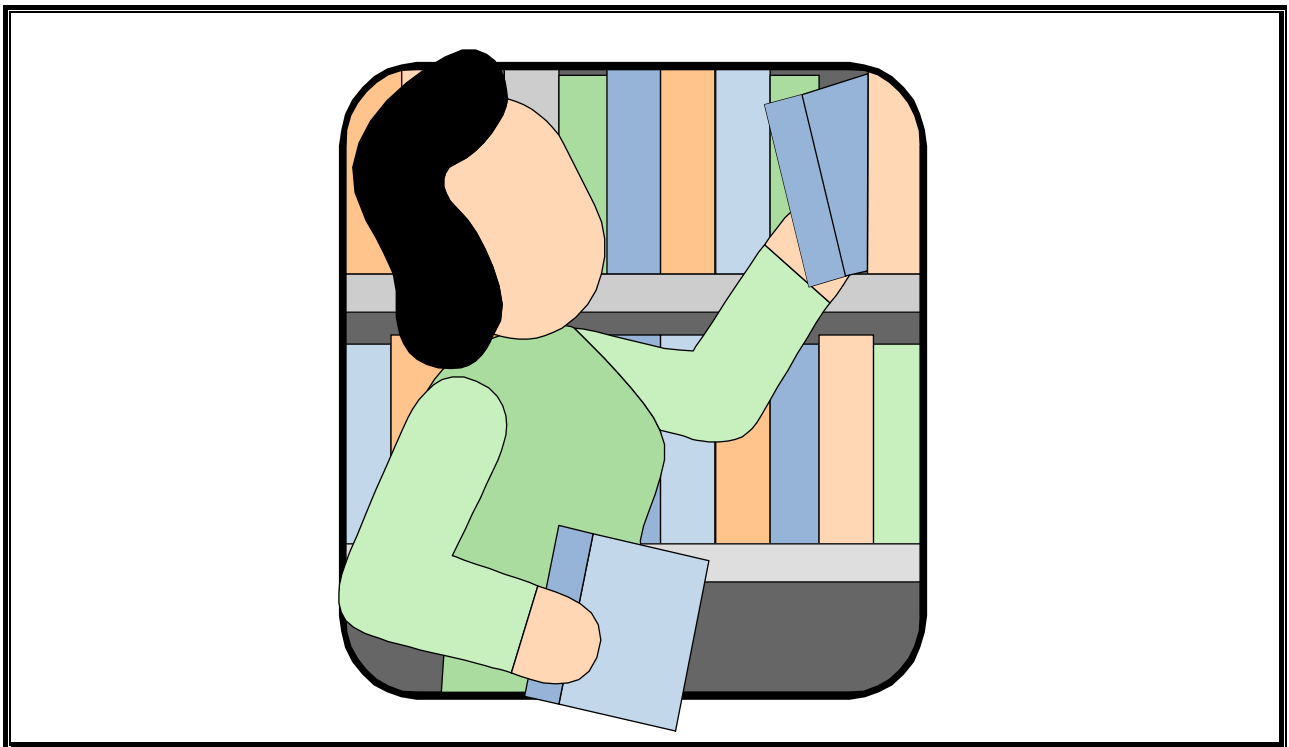
1. Open global data block or instance data block,
2. enter the desired values in the "Start value" column,
3. initialize the selected tags by clicking on the "Initialize setpoints" icon.

Note:

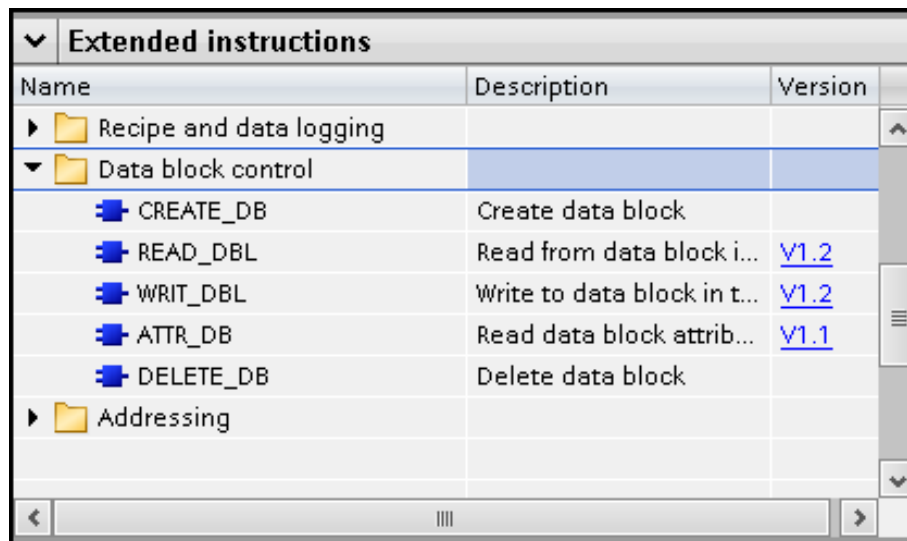
For global data blocks, the selection "Initialize setpoints" can be set and reset without having to reload the block for those tags (variables) that are not of the data type UDT.

For instance data blocks, only the static tags can be selected with "Initialize setpoints".

8.6. Additional Information



8.6.1. Data Block Control



Extended instructions		
Name	Description	Version
▶ Recipe and data logging		
▼ Data block control		
CREATE_DB	Create data block	
READ_DBL	Read from data block i...	V1.2
WRIT_DBL	Write to data block in t...	V1.2
ATTR_DB	Read data block attrib...	V1.1
DELETE_DB	Delete data block	
▶ Addressing		

CREATE_DB

...creates a new data block in the load and/or work memory.

READ_DBL

...copies a DB or a part of a DB, that is located in the load memory (Micro Memory Card), into the data area of a target DB in the work memory of the CPU.

WRT_DBL

...copies a DB or a part of a DB, that is located in the work memory of the CPU, into the data area of a target DB that is located in the load memory (Micro Memory Card).

ATTR_DB

...supplies information about a data block (DB) located in the work memory of the CPU:

- DB unlinked / linked
- DB write-protected yes/no
- DB retentive yes/no
- DB in the load/work memory, both

DELETE_DB

...deletes a data block (DB) in the work memory that was created by user program with the instruction "CREATE_DB".

9

Contents

9.	HMI Alarm Messages	9-2
9.1.	Task Description: Configuring Discrete and Analog Alarm Messages	9-3
9.2.	Tasks of an Alarm (Message) System	9-4
9.3.	Structure of an Alarm (Message)	9-5
9.3.1.	Alarm Classes	9-6
9.4.	Alarm (Message) Procedures	9-7
9.4.1.	Configuring Discrete Alarms	9-8
9.4.2.	Configuring Analog Alarms	9-9
9.4.3.	Displaying Alarm (Messages)	9-10
9.4.4.	Exercise 1: Configuring a Discrete Alarm	9-11
9.4.5.	Exercise 2: Configuring an Analog Alarm	9-12
9.4.6.	Exercise 3: Configuring an Alarm Window	9-13

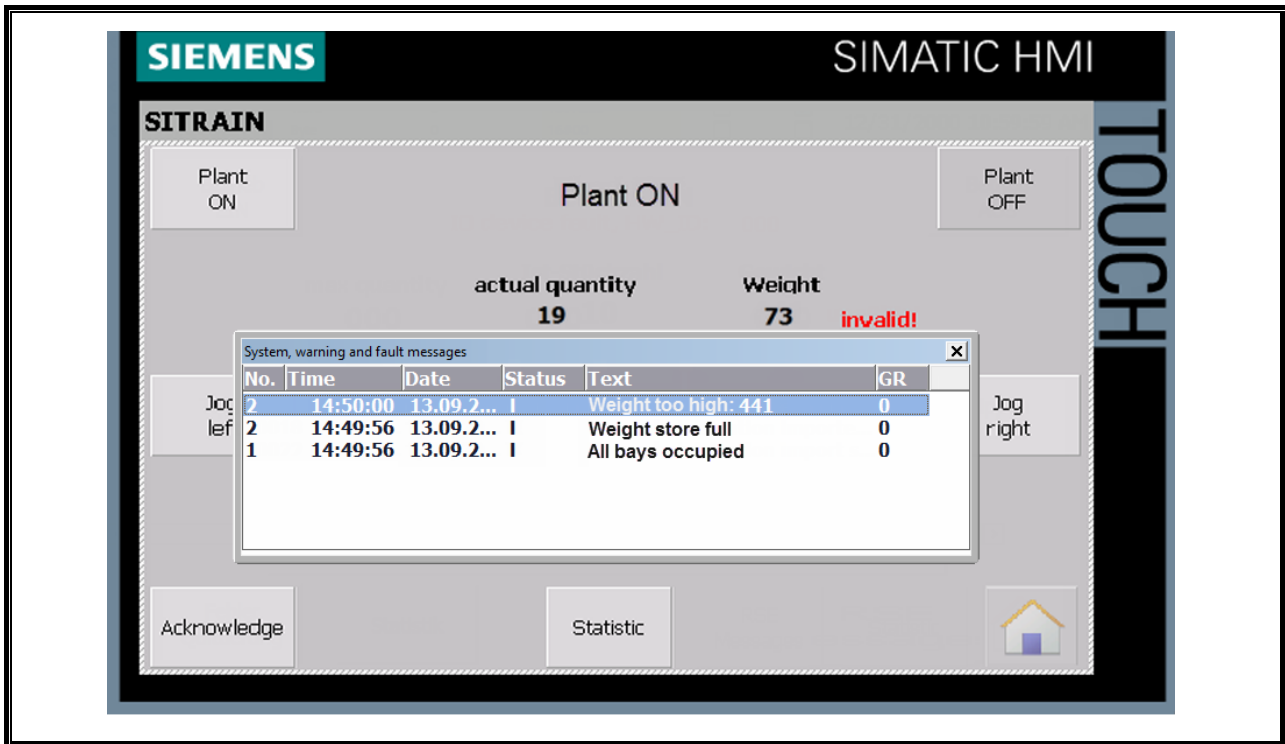
9. HMI Alarm Messages

At the end of the chapter the participant will ...

- ... be familiar with the various alarm message procedures
- ... be able to create trigger or monitoring tags
- ... be able to configure discrete and analog alarm messages



9.1. Task Description: Configuring Discrete and Analog Alarm Messages



Function Up Until Now:

When "P_Operation" (Q0.1) is switched on, the parts to be transported are weighed at the light barrier bay and then transported to the next free bay.

If the weight of the part (can be set with the slide control "S_Slider1" (IW10) on the simulator) is not within the allowable range of 100kg to 400kg, the part transportation cannot be started.

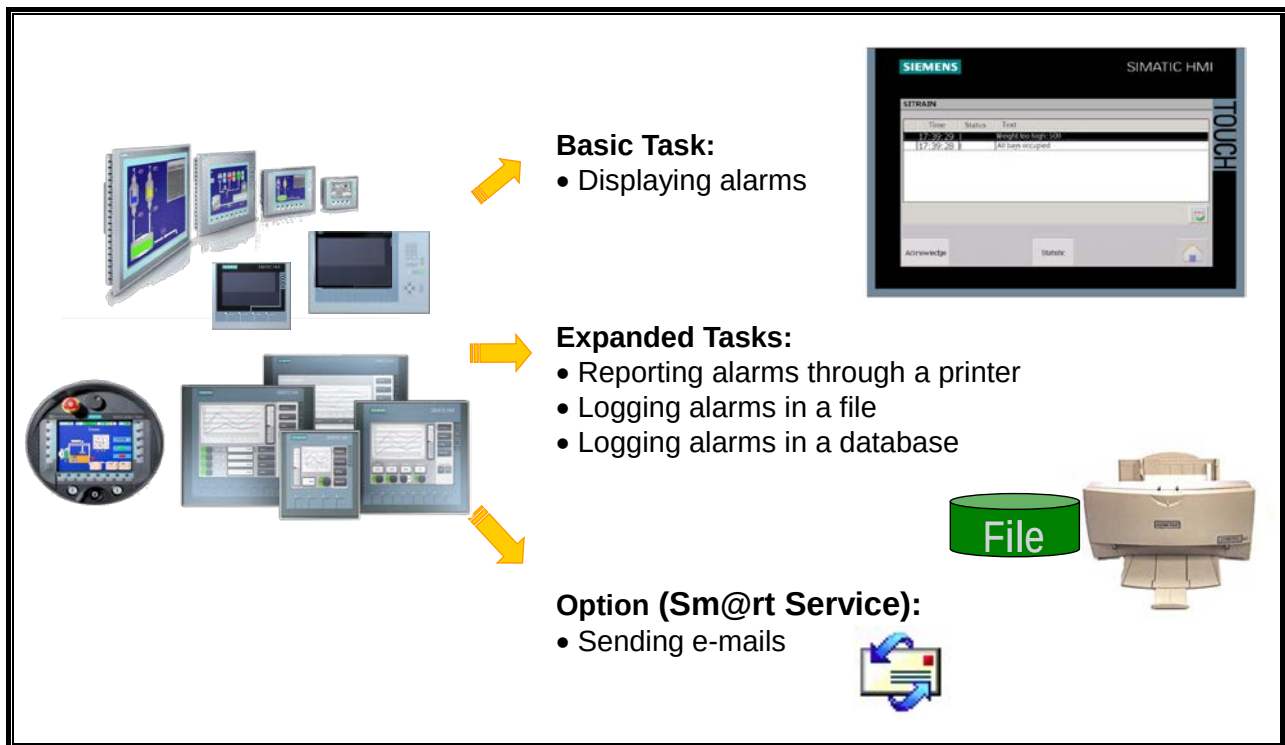
If none of the Bays 1 to 3 is free or the weight storage for saving the part weights is full, part transportation can also not be started.

Task Description

An alarm window is to be configured in which the following messages are output:

- If the lower weight limit of 100kg is exceeded, the alarm (message) "Weight too low: <actual weight>" is to be output; if the upper weight limit of 400kg is exceeded the alarm (message) "Weight too high: <actual weight>" is to be output.
- If none of the Bays 1 to 3 is free, the alarm (message) "All bays occupied" is to be output.
- If the weight storage cannot save any further part weights, the alarm (message) "Weight store full" is to be output.

9.2. Tasks of an Alarm (Message) System

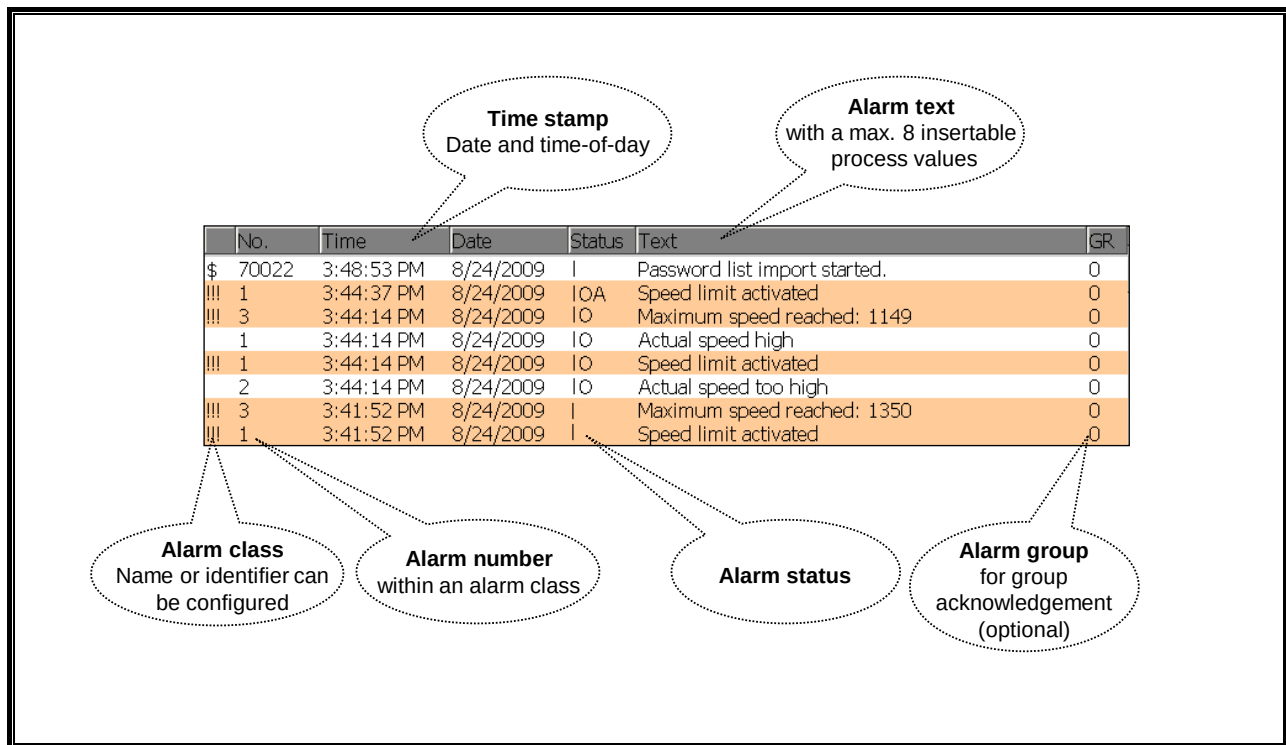


Tasks of an Alarm (Message) System

Alarm messages are output dependent on events or states that occur in the plant or in the process.

- **Displays:**
The alarms are displayed on the operator panel. This task is supported by every operator panel.
- **Reporting:**
The alarms are output to a printer. This task is only supported by operator panels that also support reporting (protocols).
- **Logging:**
The alarms are stored for further processing and analysis in a file (flash or hard drive).
- **Sending e-mails:**
With the "Sm@rt Service" option package, the "SendEmail" system function can also be configured to the message.

9.3. Structure of an Alarm (Message)



Alarm Classes

Each alarm is assigned to an alarm class that has specific properties. In addition to the following standard alarm classes, you can also declare your own alarm classes.

- **Diagnosis Events** are generated by the controller and report diagnostic events with standard texts (such as, the exceeding of the measuring range of an analog input module)
- **System Alarms** are alarms with standard texts that are output by the operating panel (such as, connection setup to the controller ...)
- **Error Alarms** must always be acknowledged and can be freely configured by the user. That is, you define the alarm event (→ discrete or analog alarms) as well as the alarm text.
- **Warnings** do not have to be acknowledged and like error alarms can be freely configured by you.

Alarm Statuses

The following alarm statuses exist:

- Alarm statuses that do not need to be acknowledged:
 - **I:** incoming
 - **IO:** incoming and outgoing
- Alarm statuses that must be acknowledged:
 - **I:** incoming
 - **IA:** incoming, still existing (pending) and already acknowledged
 - **IO:** incoming, outgoing, but not yet acknowledged
 - **IOA:** incoming, outgoing and later acknowledged
 - **IAO:** incoming, acknowledged and outgoing

9.3.1. Alarm Classes

Predefined alarm classes

- **Diagnosis events**
 - Error/diagnostic alarms of the controller
 - Are generated by the connected controller
- **System**
 - Status alarm messages of the operator panel
 - Are generated by the operator panel
- **Errors (for unrestricted use)**
 - For discrete and analog alarms that have to be acknowledged
- **Warnings (for unrestricted use)**
 - For discrete and analog alarms that do not have to be acknowledged

Display name	Name	Acknowledgment model	Log	Back...	Back...	Back...	Back...
S7	Diagnosis events	Alarm without acknowledgment	<No log>	2...	2...	2...	2...
I	Errors	Alarm with simple acknowledgment	<No log>	2...	2...	2...	2...
\$	System	Alarm without acknowledgment	<No log>	2...	2...	2...	2...
W	Warnings	Alarm without acknowledgment	<No log>	2...	2...	2...	2...
III	ConveyorFault	Alarm with simple acknowledgment	<No log>	2...	2...	2...	2...
<Add new>							

Alarm Classes

Each alarm must be assigned to an alarm class. Alarm classes primarily define the need for acknowledgement and the appearance of alarms in the display on the operator panel. Beyond that, they are used to group alarm messages for different forms of representation.

WinCC offers predefined alarm classes as well as the opportunity to configure your own alarm classes.

Predefined alarm classes "Errors" and "Warnings" for unrestricted use

- **Errors**
for discrete and analog alarms that indicate critical or dangerous operating and process statuses.
Alarms of this alarm class must always be acknowledged.
- **Warnings**
for discrete and analog alarms that indicate regular operating statuses, process statuses and process sequences.
Alarms of this alarm class are operator information and do not have to be acknowledged.

Display Name

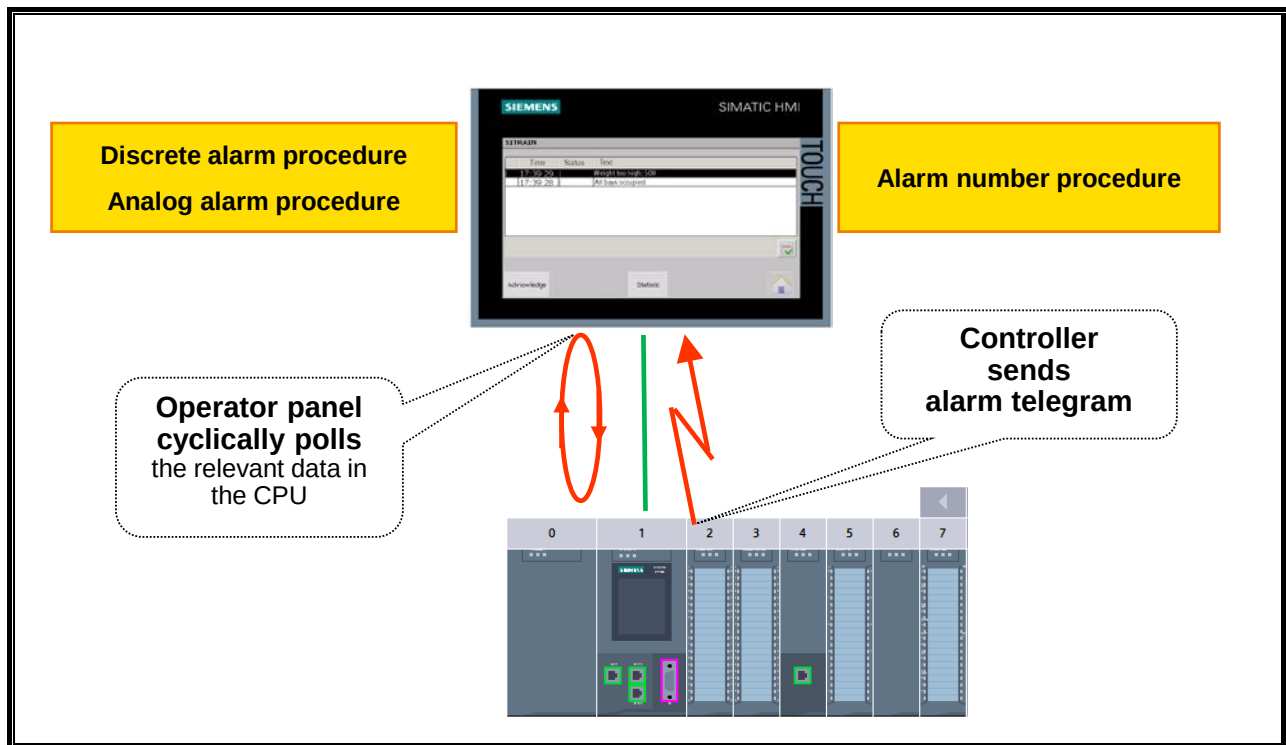
Here, an individual name can be set for each alarm class that can then also be displayed in the alarm display when several alarm classes are represented in an alarm display. These display names can also be individually adjusted for the predefined alarm classes.

Acknowledgement Concept

The need to acknowledge an alarm depends on whether the alarm class to which the alarm was assigned is required to be acknowledged.

The operator acknowledges an alarm on the operator panel by pressing the relevant acknowledgement button.

9.4. Alarm (Message) Procedures



Alarm Procedures

WinCC flexible supports the following alarm procedures:

- **Discrete Alarm Procedure:**

The operator panel displays an alarm if a certain bit of a so-called trigger tag is set in the controller. For this, the operator panel must read out the trigger tag cyclically from the S7 controller.

- **Analog Alarm Procedure:**

The operator panel displays an alarm if a certain tag exceeds specified upper or lower limit values. For this, the operator panel must read out the tag cyclically from the S7 controller.

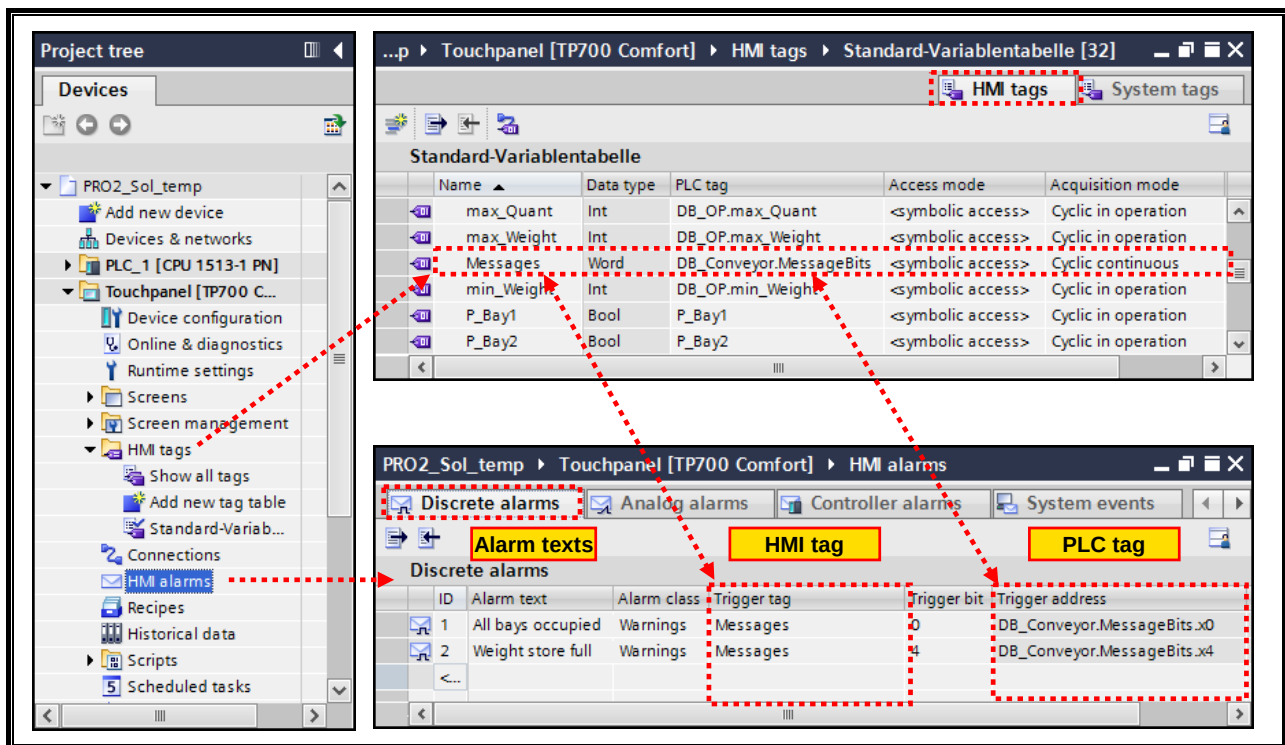
- **Alarm Number Procedure:**

The operator panel displays an alarm if the controller sends a telegram with the associated (message) number to the operator panel. For this, the ALARM_S block must be configured in the S7 program.

Benefits:

- the alarms are displayed in exactly the same sequence as they actually occur in the process
- the alarms have a CPU time stamp and not an operator panel time stamp
- reduced bus load because communication only takes place in case of an alarm
- no alarms are lost

9.4.1. Configuring Discrete Alarms



Discrete Alarms

A discrete alarm is triggered when a defined bit of a tag changes its state from 0 to 1 (rising edge).

ID

Freely selectable alarm number that is also displayed in the alarm display

Alarm Text

The alarm text can be a maximum of 254 characters long and can be formatted character by character. Tag values and/or texts from text lists can be inserted in the alarm text. The current value of the tag used is updated or once again read in when an alarm occurs.

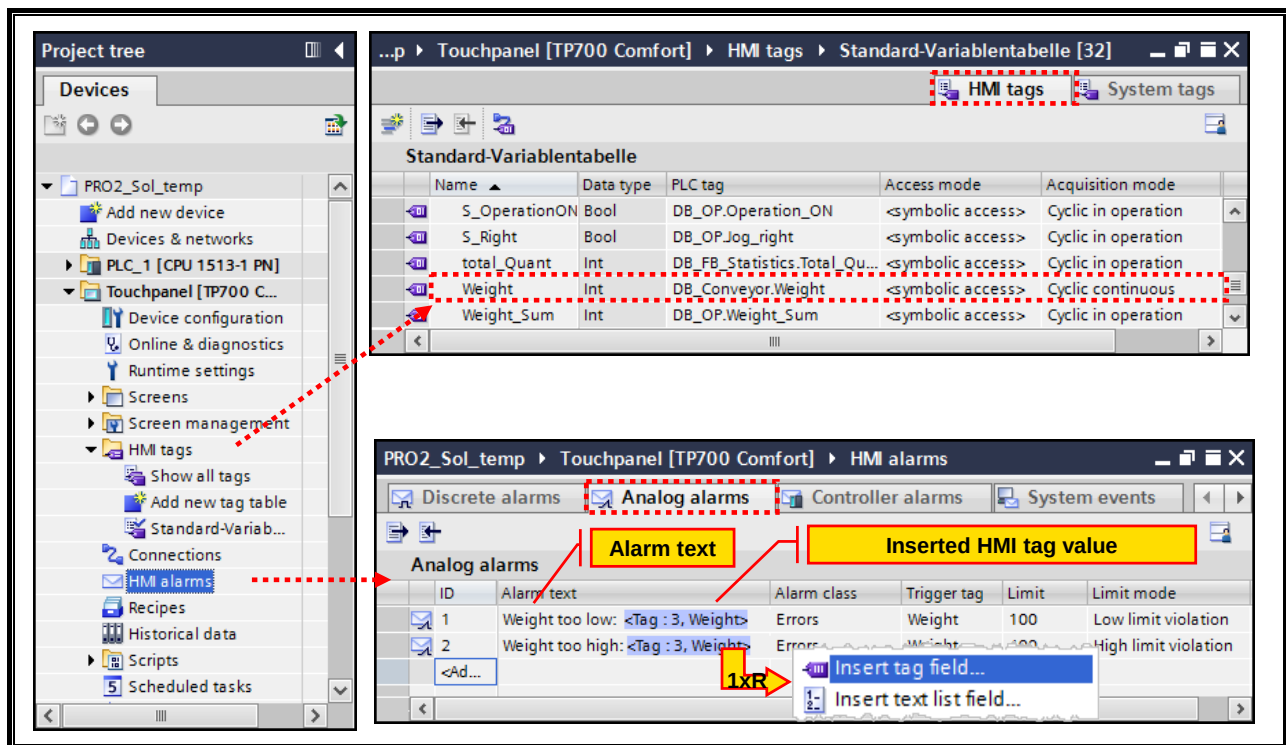
It is possible to insert the value of a tag in the alarm text via "Insert tag field...".

Trigger Tags

The trigger tag is the tag whose bits are monitored for rising edges. Each alarm is assigned to a specific bit of this tag.

- **Data type of Trigger Tags**
The trigger tag must be of the data type WORD, DWORD, INT or DINT.
- **Acquisition mode: Cyclic continuous**
means that these tags are constantly updated or read in the background, not only as required ("Cyclic in operation").

9.4.2. Configuring Analog Alarms



Analog Alarms

An analog alarm is triggered when the value of a specific tag falls below or exceeds a limit.

Alarm Text

The alarm text can be a maximum of 254 characters long and can be formatted character by character. Tag values and/or texts from text lists can be inserted in the alarm text. The current value of the tag used is updated or once again read in when an alarm occurs.

It is possible to insert the value of a tag in the alarm text via "Insert tag field..."

Trigger Tag

The trigger tag is the tag whose value is monitored. Each alarm is assigned to a specific tag with a specific limit and whether this limit must fall below or must be exceeded.

Limit

The limit for which the trigger tag is monitored can be specified as a constant or, in turn, via a tag whose data type must match that of the trigger tag.

Dead Zone

When a process value swings around the limit, the associated alarm for this fault is triggered several times. To prevent this, a dead zone can be configured.

9.4.3. Displaying Alarm (Messages)

- **Alarm View**
 - Alarms from the alarm buffer or log
 - Can be configured **in every screen**
 - Only visible when the relevant screen is open
- **Alarm Window**
 - Alarms from the alarm buffer
 - Can **only** be configured **in "Global screen"**
 - Window appears when a corresponding alarm occurs
 - In addition, the **alarm indicator** can be configured

Alarm View

Several alarm views can be configured for different alarm classes and in various screens or various windows. The alarm view is only visible when the relevant screen is called.

- Display of currently pending alarms or alarms from the alarm buffer or alarm log
- Alarm selection according to alarm class
- Can be configured in every screen
- Is only visible when the relevant screen is open
- Selectable display type: Table | Line

Alarm Window

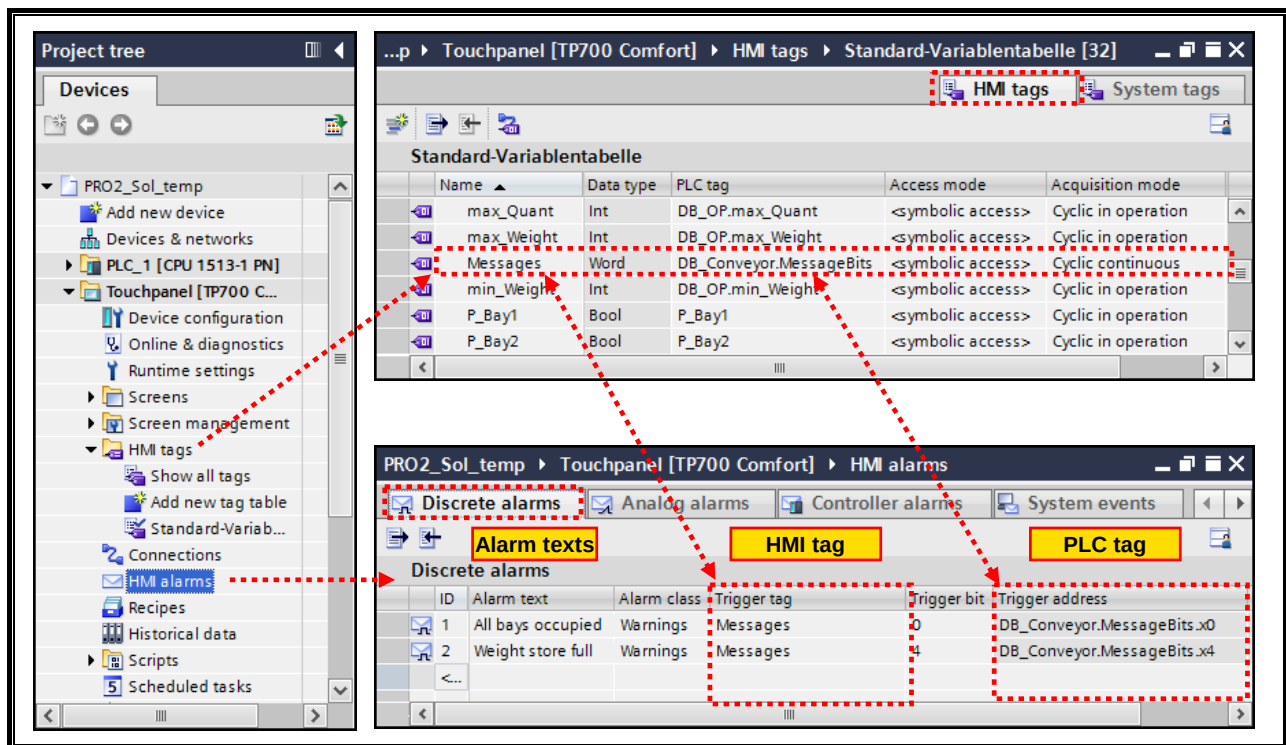
The alarm window can only be configured in the screen "Global screen". Just as with the alarm view you can select the classes to be displayed. In all other screens, you can select whether or not the alarm window is to appear when an alarm "comes".

- Display of currently pending alarms or alarms from the alarm buffer or alarm log
- Alarm selection according to alarm class
- Can only be configured in the screen "Global screen"
- Window can automatically appear when a new entry is made in the window (for example, a new alarm has come).

Alarm Indicator

Just as the alarm window, the alarm indicator can only be configured in the "Global screen". It is a graphic symbol that points to pending alarms or alarms to be acknowledged in all screens, depending on the configuration. With the help of the alarm indicator, a defined alarm window can be shown.

9.4.4. Exercise 1: Configuring a Discrete Alarm



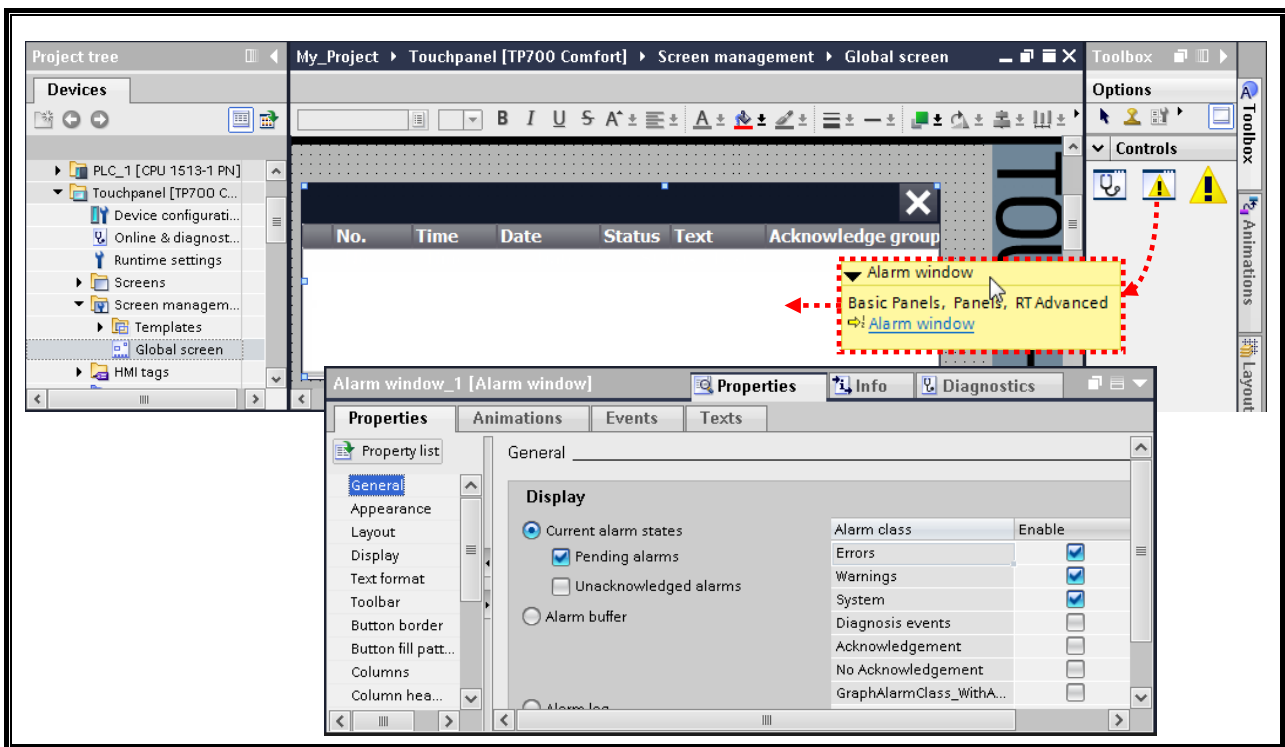
Task

The discrete alarms shown in the lower right picture are to be configured. Later, they will be displayed in an alarm window.

What to Do

- Expand the "DB_Conveyor" (DB100) block by adding the tag *Messages* [data type WORD]
- Configure the required discrete alarms (see picture) as alarms of the alarm class "Warnings". Declare the newly created DB variable "DB_Conveyor".*MessageBits* as the trigger tag.
Select the "trigger bits" as follows:
 - Trigger bit 0 or "DB_Conveyor".*MessageBits.X0* for the message "All bays occupied"
 - Trigger bit 4 or "DB_Conveyor".*MessageBits.X4* for the message "Weight store full"
- Program the management of these trigger bits in "FC_Signal" (FC14):
 - As long as all bays are occupied, the trigger bit "DB_Conveyor".*MessageBits.X0* must be assigned Status '1'.
 - The status of the variable "DB_Conveyor".*WeightStore_full* must also be assigned to the trigger bit "DB_Conveyor".*MessageBits.X4*.
- Save your project.

9.4.6. Exercise 3: Configuring an Alarm Window



Task

Now that you have configured 2 discrete and 2 analog alarms, you now still have to configure where these alarms are to be displayed. This is to happen in an alarm window that – as soon as one of the alarms is pending – is inserted into every touchpanel screen.

What to Do

1. In the "Screen management", open the "Global screen"
2. There, configure an alarm window and, in its Properties, define that the pending alarms of the alarm classes "Errors", "Warnings" and "System" are to be displayed.
3. Download the modified program into the touchpanel.
4. Download all modified S7 blocks into the CPU.
5. Check whether the alarms are displayed as required.
6. Save your project.

Contents

10. System Diagnostics and Error Handling	10-2
10.1. Task Description: Displaying an I/O-Device Failure on the Touchpanel	10-3
10.2. Functional Principle of the CPU System Diagnostics	10-4
10.3. Overview: Diagnostic Possibilities	10-5
10.4. Diagnostics using the PG with STEP 7	10-6
10.5. CPU System Diagnostics	10-7
10.5.1. CPU System Diagnostics: Functional Principle	10-8
10.5.2. Parameterizing CPU System Diagnostics	10-9
10.5.3. Displaying System Diagnostics Alarms in STEP7	10-10
10.5.4. Exercise 1: Displaying System Diagnostics Alarms on the CPU-Display	10-11
10.5.5. Displaying System Diagnostics Alarms on the HMI Device with "System Diagnostics View"	10-12
10.5.6. Exercise 2: Displaying System Diagnostics on the HMI Device	10-13
10.6. Diagnostics with the CPU Web Server	10-14
10.6.1. PN-CPUs: Accessing the Web Service	10-15
10.6.2. PN-CPUs: Web Service: "Start Page"	10-16
10.6.3. PN-CPUs: Web Service: Example "Diagnostic Buffer"	10-17
10.6.4. Parameterizing the CPU Web Server	10-18
10.6.5. Exercise 3: Activating and Parameterizing the Web Server	10-19
10.7. Diagnostics using the S7 Program	10-20
10.7.1. Start Information of the Error OBs	10-21
10.7.2. Global Error Handling with Asynchronous Error OBs	10-22
10.7.3. Diagnosis of Asynchronous Errors with "DeviceStates"	10-23
10.7.4. Global Handling of Synchronous Errors with Error OBs: Principle	10-24
10.7.5. Global Handling of Synchronous Errors with Error OBs: CPU Behavior	10-25
10.7.6. Local Handling of Synchronous Errors	10-26
10.7.7. Exercise 4: Integrating Local Error Handling	10-27
10.7.8. Task Description: Displaying an IO-Device Error and Time-of-day Synchronization with the CPU	10-29
10.7.9. Exercise 5: Displaying an IO-Device Error	10-30
10.7.10. Exercise 6: Determining the Device Downtime	10-31
10.7.11. Exercise 7: HMI Devices Time-of-day Synchronization with the CPU	10-32
10.8. Diagnosis of the Operator Panel Status using the CPU Program	10-33
10.8.1. Configuring the Life Bit	10-34
10.8.2. Exercise 8: Diagnosis of the Online Connection TP <--> CPU	10-35

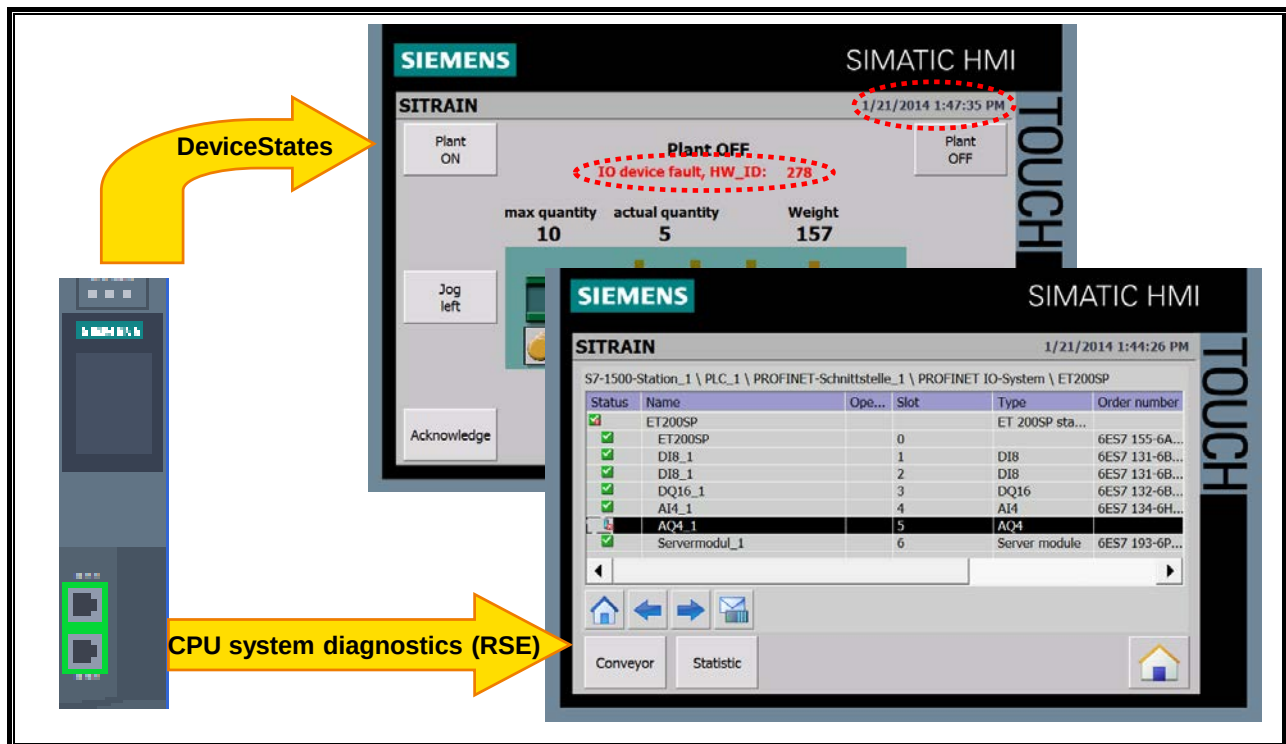
10. System Diagnostics and Error Handling

At the end of the chapter the participant will ...

- ... be able to explain the function of system diagnostics
- ... be familiar with and be able to configure the function
CPU system diagnostics
- ... be able to use the CPU web server for diagnostics
- ... be able to program diagnostic functions in the S7 program
- ... be able to evaluate the TP's "life bit" through the program



10.1. Task Description: Displaying an I/O-Device Failure on the Touchpanel



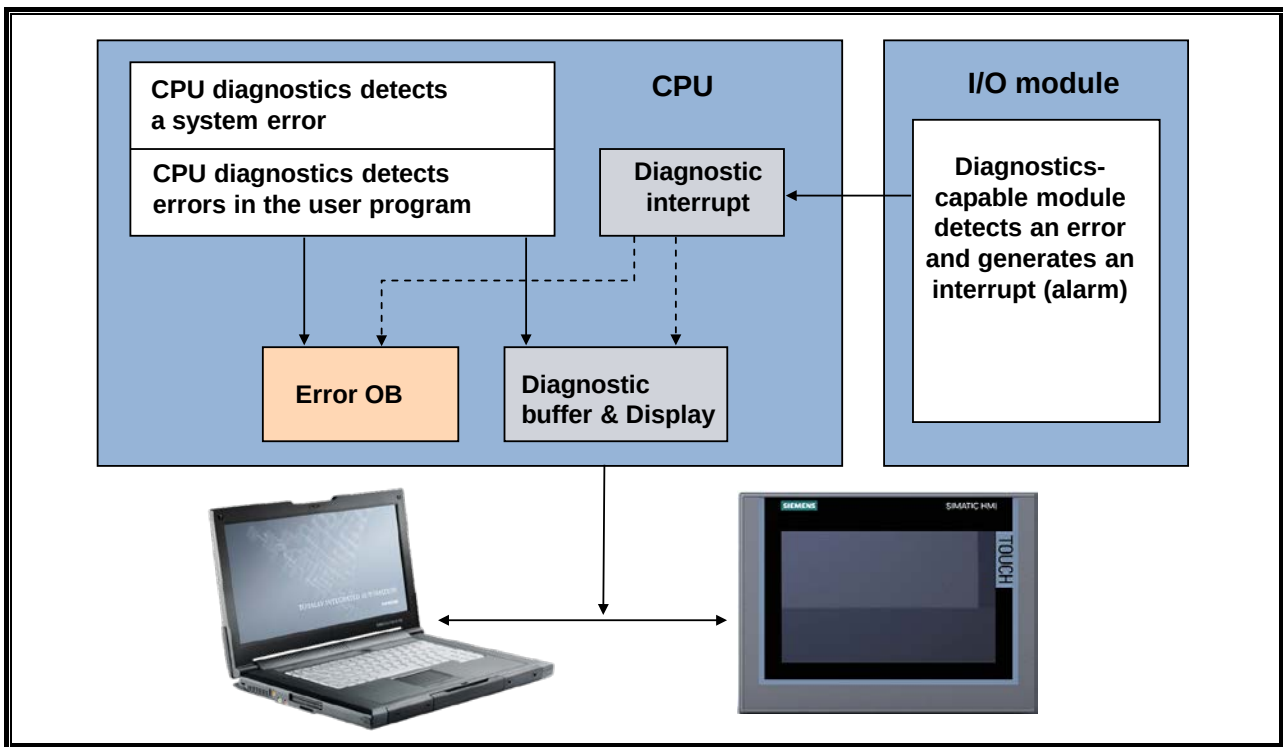
Task Description

In case of an I/O-Device failure, this is to be displayed on the touchpanel in a new screen called "Diagnostics".

Furthermore, the current date and time that is synchronized with the CPU is to be displayed in every touchpanel screen at the upper right-hand corner.

In addition, the I/O-Device failure is to be determined through the program.

10.2. Functional Principle of the CPU System Diagnostics



Standard Reaction of the CPU when it detects an Error

If the CPU's operating system detects an error, this error is entered in the diagnostic buffer of the CPU and the error OB assigned to the error is called. In this OB program, the user has the opportunity to program a CPU reaction (switch CPU to STOP, send message etc.) to the error that occurred. After the error OB is executed, the execution of the user program resumes at the point of interruption.

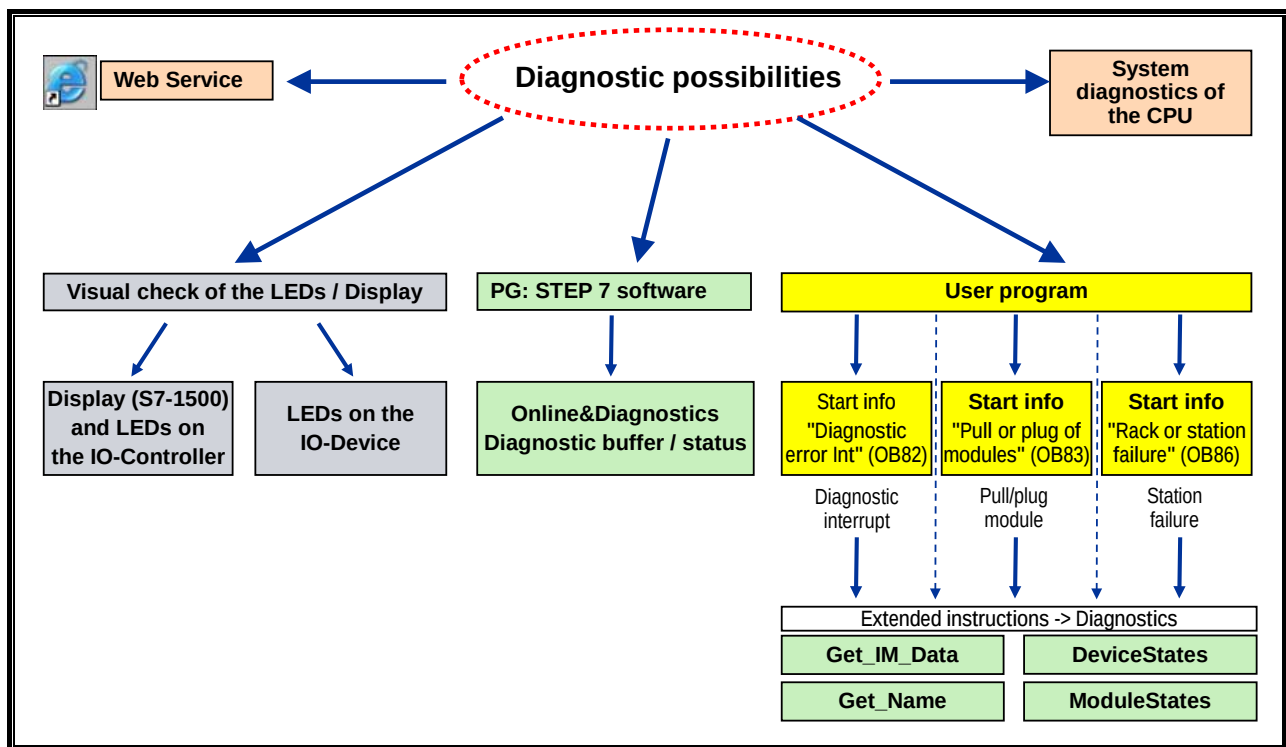
Exception:

With a programming error (OB121), the CPU goes into the STOP state if there is no error OB.

Caution:

The operating system does not analyze whether the error handling (programmed error OB) really takes care of the triggering error. **For that reason, particular care has to be taken when programming an error handling so that no dangerous system states result in case of a failure.** If required, the CPU can be sent into a programmed STOP through the use of the function "STP". (Task Card Instructions → Basic Instructions → Program control... → Runtime control).

10.3. Overview: Diagnostic Possibilities



STEP 7 Software

Under "Online & diagnostics" you can read out the general diagnostic status and the diagnostics buffer with information about the error that occurred.

User Program

OB86 (Rack or station failure)

When a distributed I/O module or a central rack expansion (IM) fails and recovers, OB86 is called. The HW_ID of the failed module is then stored in the start information of the OB.

OB83 (Pull or plug of modules)

If a configured module is removed or inserted, OB83 is called. The HW_ID of the failed module is then stored in the start information of the OB.

OB82 (Diagnostic error interrupt)

When there are errors on an I/O module, OB82 is called if the module is diagnostics-capable and the diagnostic interrupt was also activated in the hardware configuration. The HW_ID of the failed module is then stored in the start information of the OB.

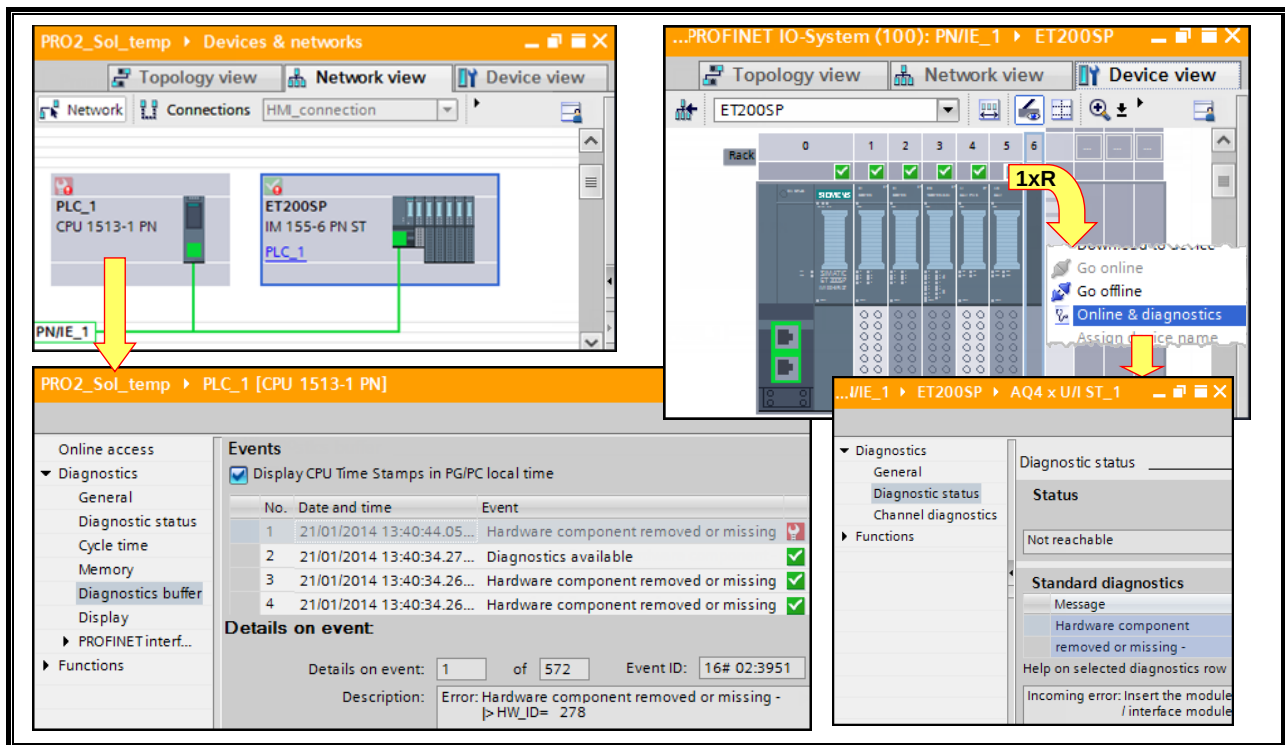
System Diagnostics

The CPU system diagnostics - in S7-300/400 also known as "Reporting system errors" (RSE) - is an always activated message function which automatically generates error messages and sends them to all connected HMI devices.

Web Service

All PROFINET-capable S7-CPU's provide a web presence for diagnostics which can be accessed through a commercially available web browser (such as, Internet Explorer, Netscape, Mozilla Firefox or Opera). These HTML pages offer readable access to device status, tags and enables the execution of some device functions.

10.4. Diagnostics using the PG with STEP 7



Diagnostics Buffer

In the picture you can see an error message for an IO Device that has failed. In this case, the IO Device with the Hardware ID 278 has failed.

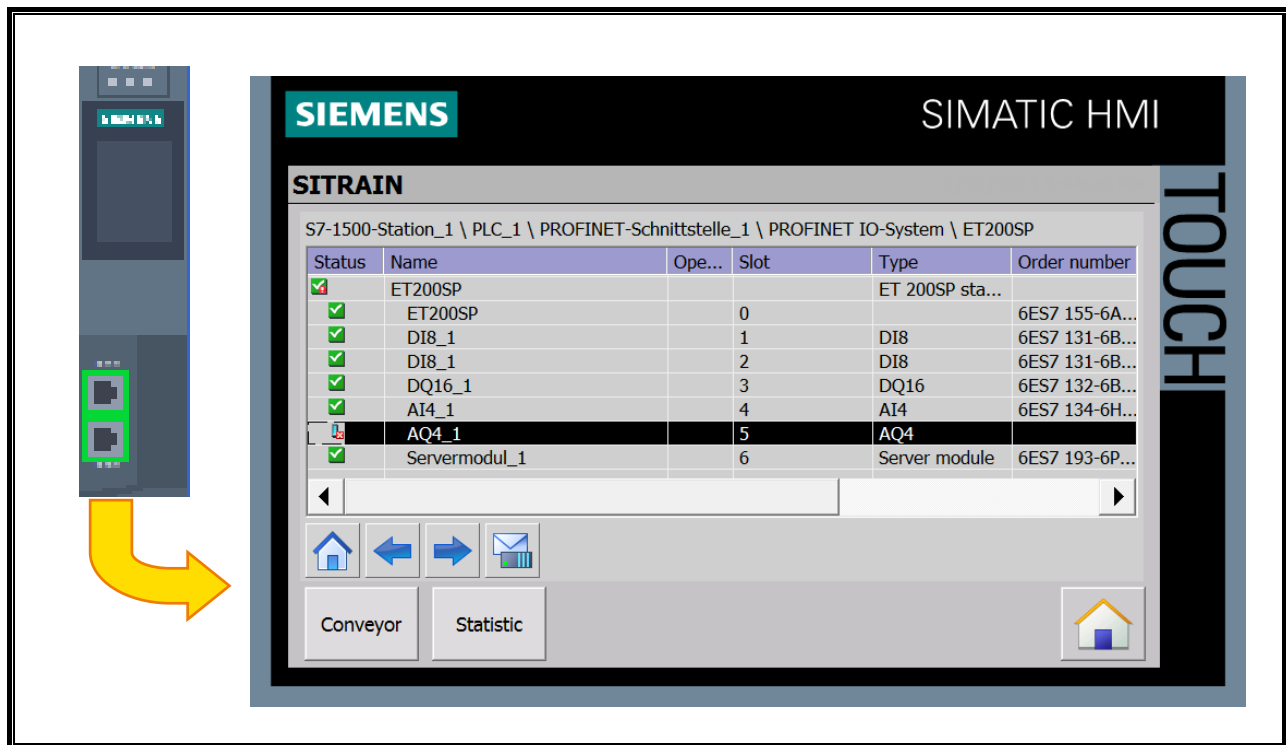
The "HW_ID" was automatically assigned by STEP7 during the configuration of the hardware and is stored in the Network view for the selected device in the Inspector window "Properties"/ "System constants" tabs or, for the entire hardware, under the PLC-tags in the "System constants" tab.

Online & Diagnostics

With an existing online connection, you can recognize the failed or faulty devices by the red crossed-out symbol in the Project tree (not shown in the picture) and in the Devices & networks editor both in the Network view and in the Device view.

By activating "Online & diagnostics", further information about the failed device / module can be queried.

10.5. CPU System Diagnostics



System Diagnostics

STEP7 offers a simple way to output system errors to one or more display devices in the form of messages with a description of the error that has occurred and information on the error location.

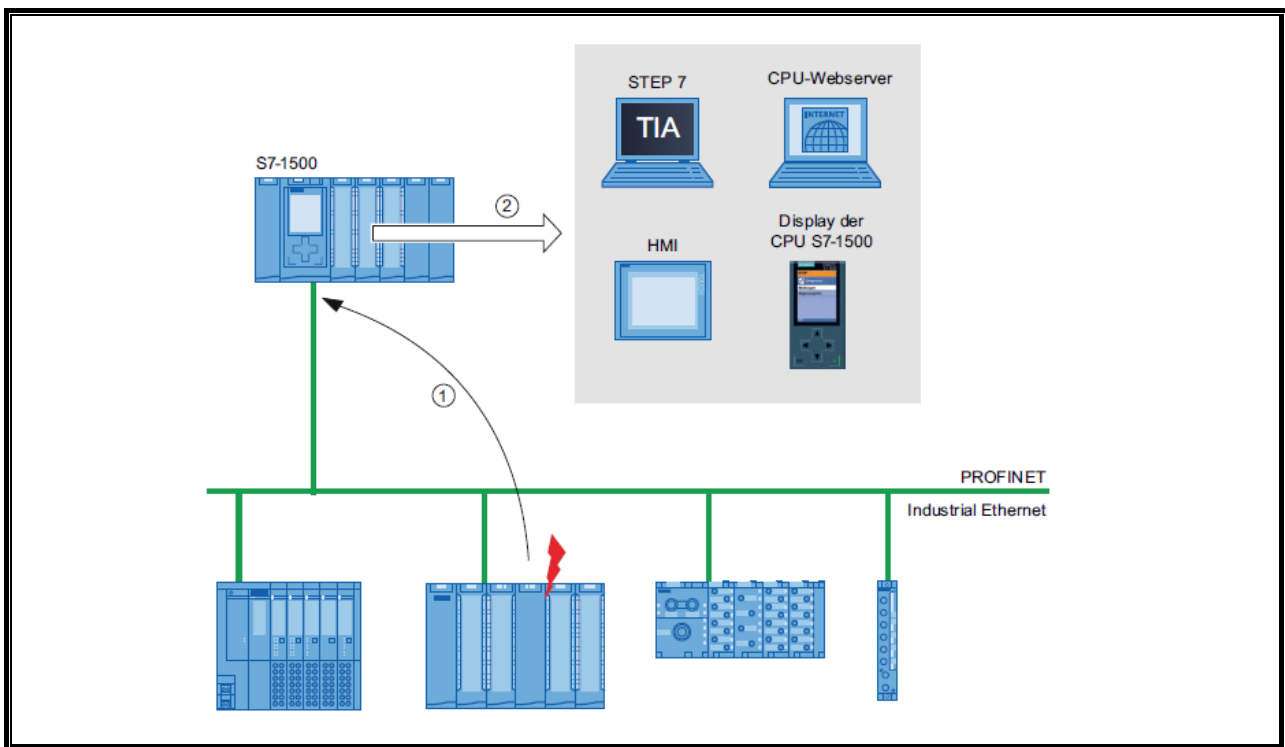
This alarm procedure is organized through alarm numbers that are automatically assigned by STEP7 during programming. When a fault occurs, the operator panel receives a message with an alarm number and the time stamp of the CPU. Based on this information, the operator panel can clearly associate the message with an alarm text. As well, the CPU saves the state of the alarm (came in, went out, acknowledged) and possible process parameters. In this way, an exact diagnosis of the cause of the fault is possible.

Advantages

Compared to the frequently used discrete alarm procedure, the system diagnostics offers the following advantages:

- The system diagnostics is an active alarm procedure. If an alarm occurs, the CPU actively sends an appropriate message to all connected operator panels and doesn't wait until the operator panels make inquiries.
- The process values always coincide exactly with the values at the time of the alarm. This is not guaranteed in the discrete alarm procedure.
- The timestamp specifies exactly when an event occurs, even then, when the operator panel is only connected later on.

10.5.1. CPU System Diagnostics: Functional Principle



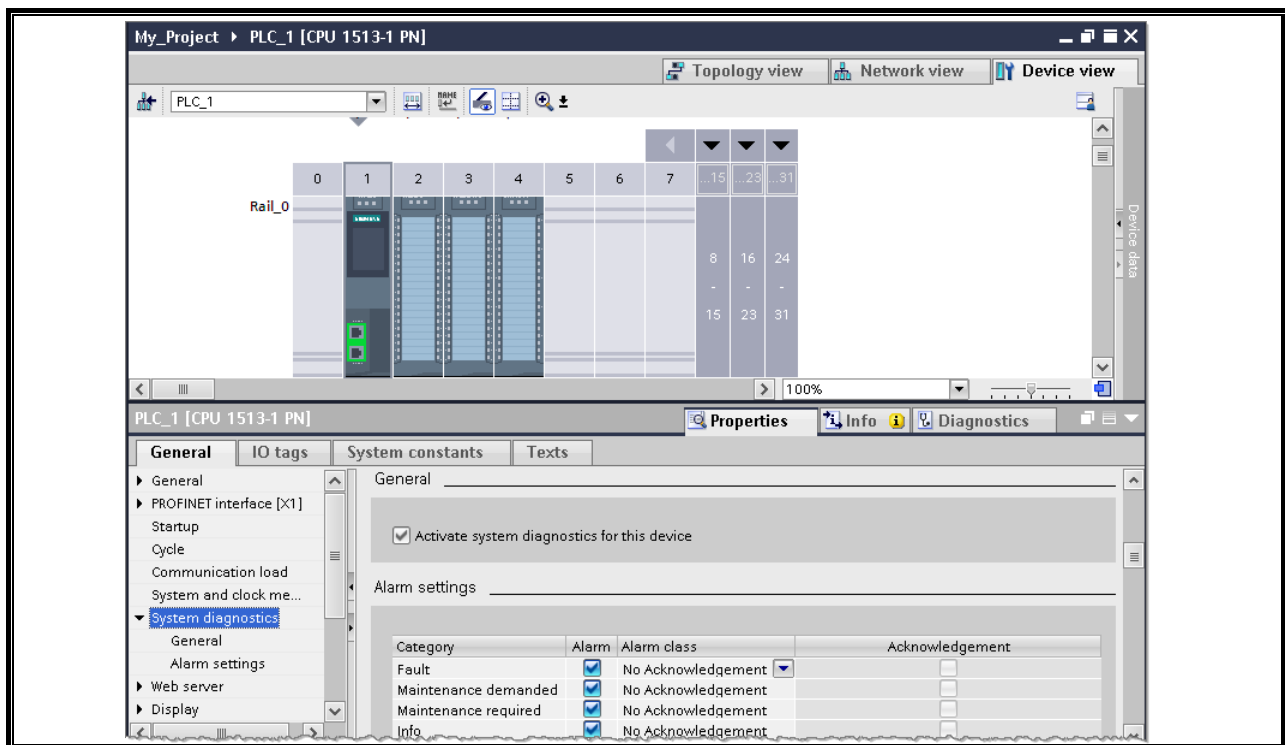
CPU System Diagnostics

The CPU system diagnostics is integrated in the firmware of the CPU S7-1500 and works independent of the cyclic user program. Therefore, it is also available when the CPU is in the STOP state.

1. If a device or a module detects an error, it signals the CPU this with an interrupt signal.
2. Detected faults are immediately signaled to the higher-level HMI devices, the Webserver and the Display of the CPU S7-1500. In that way, the system diagnostics is always consistent with the actual system state.

All clients of a system are provided with diagnostic information via a uniform mechanism. Regardless of the display medium, the same system diagnostic information is used.

10.5.2. Parameterizing CPU System Diagnostics



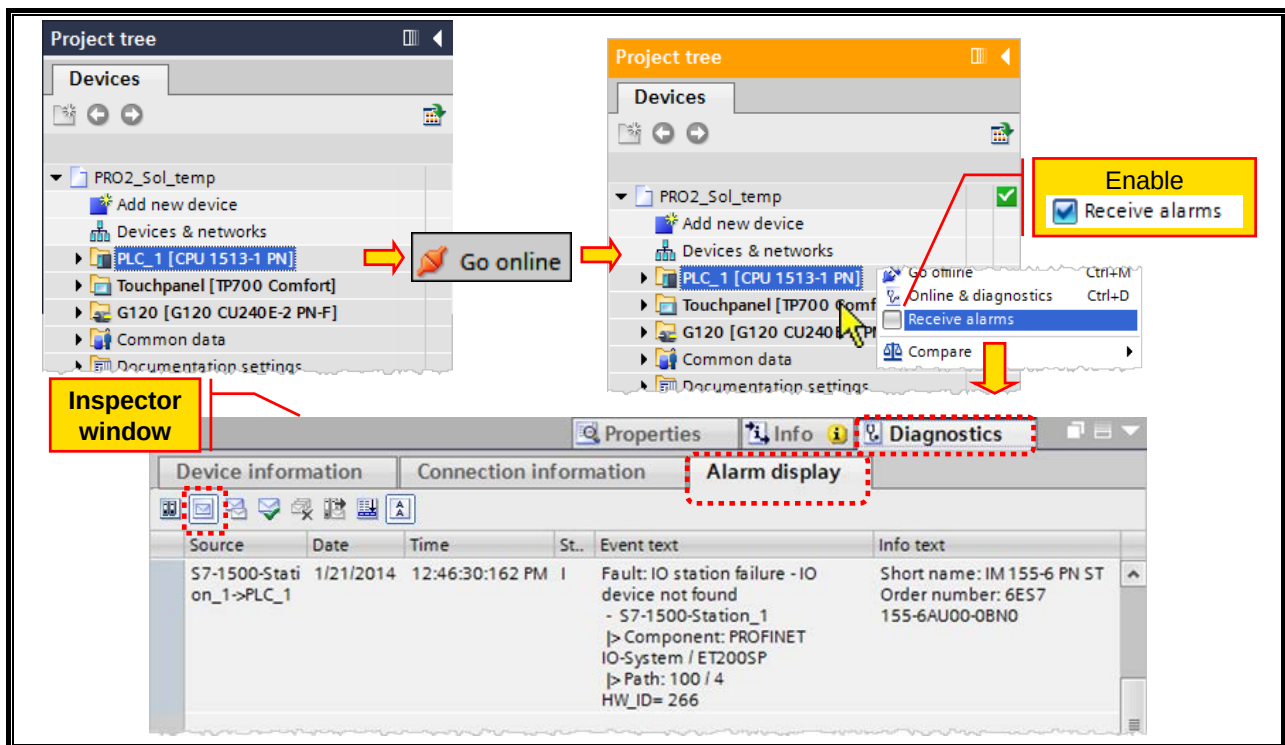
CPU System Diagnostics

The system diagnostics is always activated and cannot be deactivated.

Alarm Settings

In the Alarm settings you can define which alarm categories are to be output and whether these need to be acknowledged.

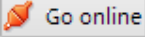
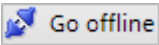
10.5.3. Displaying System Diagnostics Alarms in STEP7



Task

Monitor the CPU alarm messages in the Inspector window of STEP7.

What to Do

1. Select the CPU and establish an online connection using .
2. Enable the receiving of sent alarms in STEP7 as shown in the picture.
3. Provoke an alarm message using System diagnostics, for example, by pulling an ET200SP-DI module. The alarm display occurs in the Inspector window in the "Diagnostics"/"Alarm display" tabs.
4. Terminate the online connection. .

10.5.4. Exercise 1: Displaying System Diagnostics Alarms on the CPU-Display

The alarms can also be viewed on the CPU-Display.



Main menu item:



Diagnostics
Display diagnostic buffer
Display CPU alarms

Task

Monitor the CPU alarm messages using the CPU-Display.

What to Do:

1. Select the menu item "Diagnostics"

With the cursor keys  and  select the menu item.

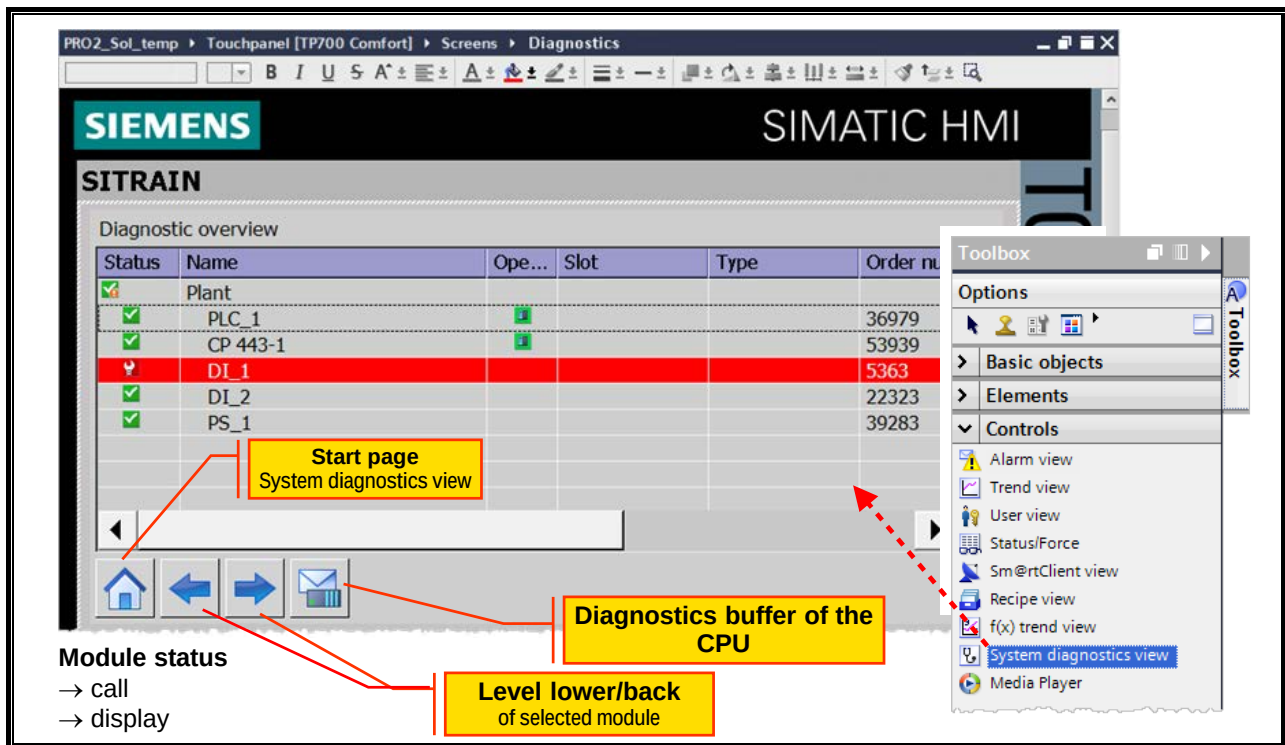
2. Look at Alarm Messages

With the cursor keys  and  select the menu item "Alarms".



With  you get back to the main menu.

10.5.5. Displaying System Diagnostics Alarms on the HMI Device with "System Diagnostics View"



View System Diagnostics

This tabular view shows all available devices of one level. Symbols in the first column give information about the current status of the device.



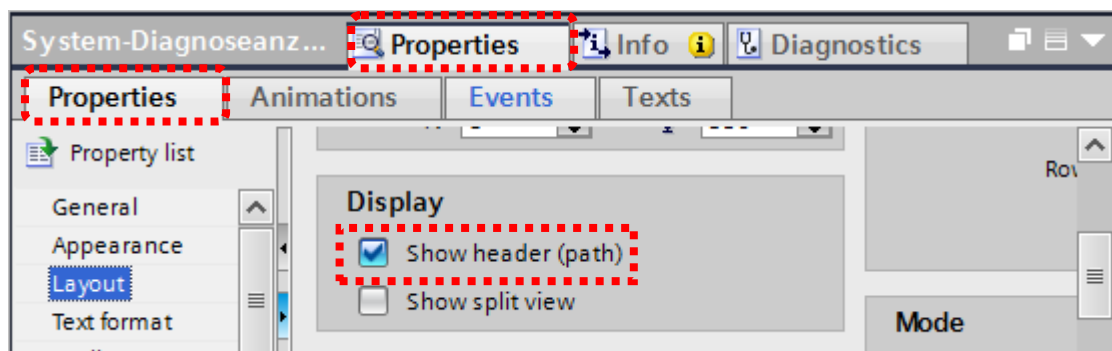
Level Lower

By double-clicking on a device or by pressing the button, either the subordinate devices or the Details view is opened.

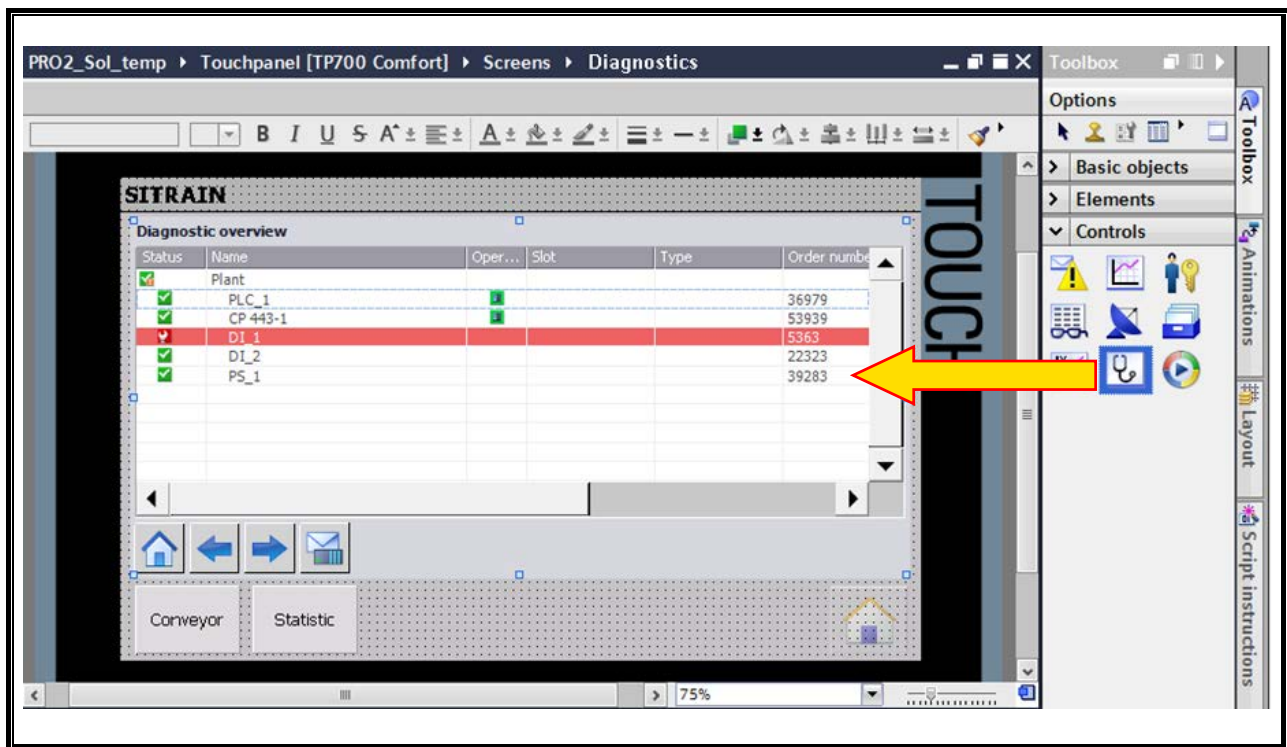
- Higher-level components (for example, PN interface)
→ subordinate components/modules are displayed
- Diagnostics buffer or module without subordinate components
→ Details view of the Diagnostics buffer entry or the module is displayed

Splitting the View in the System Diagnostics View

The system diagnostics view also offers a split view of the display. In that way, associated details can immediately be seen in the lower area of the display.



10.5.6. Exercise 2: Displaying System Diagnostics on the HMI Device



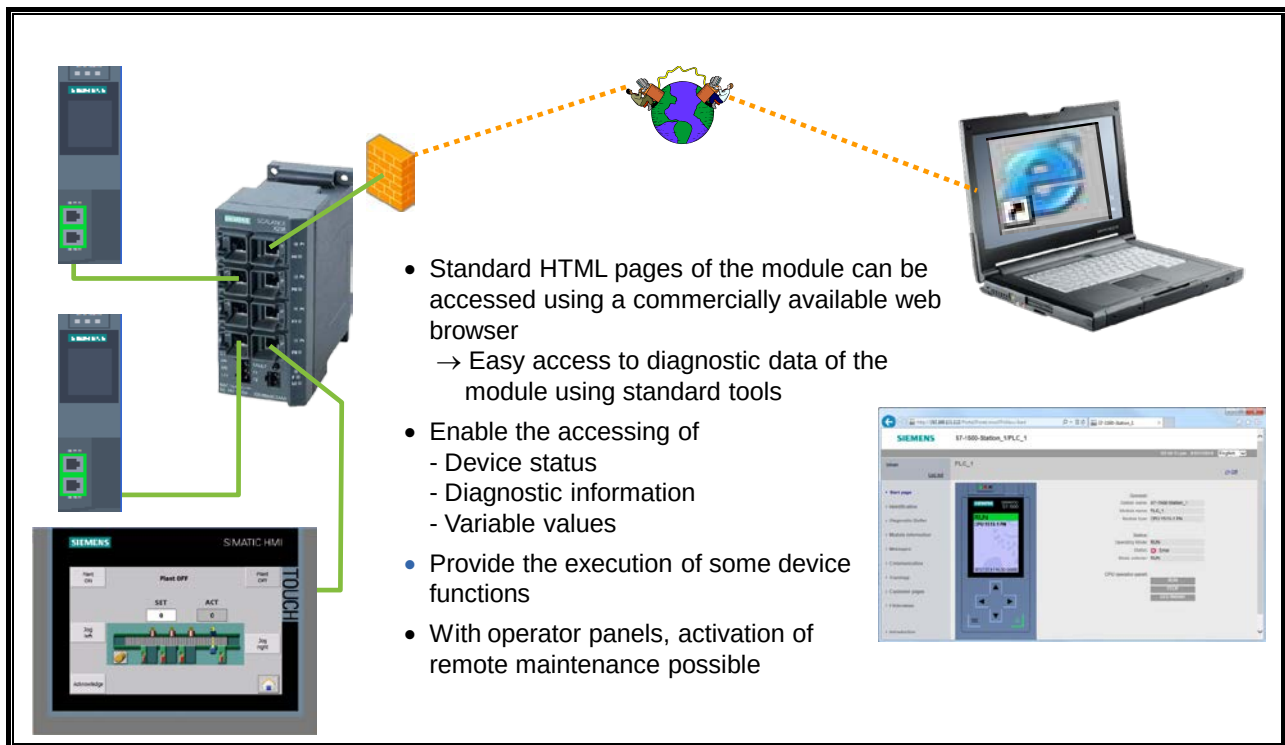
Task: Displaying System Diagnostics on the HMI Device

The information of the CPU system diagnostics is to be displayed on the touchpanel with the "System diagnostics" control which is prepared for this.

What to Do

1. Insert a new touchpanel screen and rename it "Diagnostics".
2. There, configure a "System diagnostics view" using drag & drop (see picture)
3. As screen navigation, configure the buttons "Conveyor" and "Statistic".
For this, drag the touchpanel screens "Conveyor" and "Statistic" one after the other from the Project view into the opened "Diagnostics" screen using drag & drop.
In the other touchpanel screens, also configure an appropriate project navigation in order to be able to select the here newly created "Diagnostics" screen.
4. Download the modified configuration into the touchpanel.
5. Check the new function by provoking an IO-Device error by pulling and inserting an ET200SP module.
6. Save your project.

10.6. Diagnostics with the CPU Web Server



Web Service

All S7-CPU with PROFINET interface provide a web server for diagnostics which can be accessed through a commercially available web browser (such as, Internet Explorer, Netscape, Mozilla Firefox or Opera). These HTML pages offer readable access to device status, tags and enables the execution of some device functions. For operator panels, these standard HTML pages can even be modified (user-designed) and so a write access to variables can also be programmed here.

Security

Not every web service offers security functions for access protection and user management (CPUs, for example, none). If necessary, an additional safety measure has to be implemented to prevent unauthorized access, for example, by using a firewall or similar. In any case, you should discuss this topic with network experts!!!

10.6.1. PN-CPU: Accessing the Web Service



Accessing the Web Service

The address in the web browser is the IP address of the CPU.

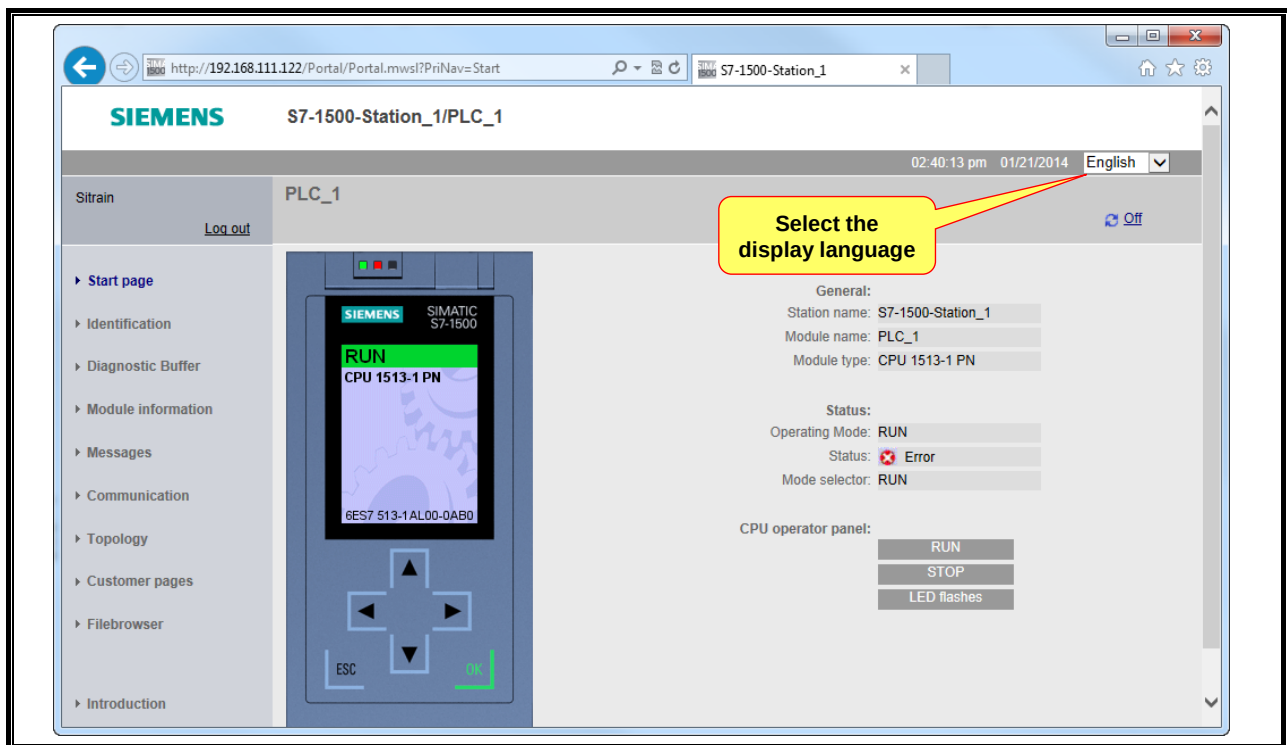
A general welcome page then appears.

After leaving the Welcome page “Intro”, you get to the “Start page”.

For Pocket-PC or PDA:

IP address of the CPU followed by **/basic** (for example, <http://192.168.111.100/basic>)

10.6.2. PN-CPU: Web Service: "Start Page"



Start Page

Through the navigation that appears on the left, different information can be queried. Furthermore, the language can be selected.

10.6.3. PN-CPU: Web Service: Example "Diagnostic Buffer"

The screenshot displays the SIMATIC TIA Portal Web Service interface for a PN-CPU. The browser window shows the URL `http://192.168.111.122/Portal/Portal.mwsl?PriNav=Diag` and the station name `S7-1500-Station_1`. The page title is `02:41:47 pm 01/21/2014 English`. The interface includes a sidebar with navigation links and a main content area titled **Diagnostic Buffer**. The diagnostic buffer entries are displayed in a table with columns: Number, Time, Date, State, and Event. The table shows 8 entries, with the first entry highlighted. The event description for the first entry is: `Communication initiated request: WARM RESTART Pending startup inhibit(s): - No startup inhibit set - CPU changes from STARTUP to RUN mode`. Below the table, there is a **Details: 1** section showing CPU info and event ID 16# 02:400E.

Number	Time	Date	State	Event
1	02:38:47.229 pm	01/21/2014	incoming event	Communication initiated request: WARM RESTART Pending startup inhibit(s): - No startup inhibit set - CPU changes from STARTUP to RUN mode
2	02:38:47.225 pm	01/21/2014	incoming event	IO station failure - IO device not found -
3	02:37:50.752 pm	01/21/2014	incoming event	Hardware component removed or missing - -
4	02:37:50.013 pm	01/21/2014	incoming event	Communication initiated request: WARM RESTART Pending startup inhibit(s): - No startup inhibit set - CPU changes from STOP to STARTUP mode
5	02:37:46.494 pm	01/21/2014	outgoing event	IO station failure - IO device not found -
6	02:37:46.483 pm	01/21/2014	outgoing event	Hardware component removed or missing - (outgoing) -
7	02:37:45.334 pm	01/21/2014	incoming event	Communication initiated request: STOP Pending startup inhibit(s): - No startup inhibit set - CPU changes from RUN to STOP mode
8	12:40:44.058 pm	01/21/2014	incoming event	Hardware component removed or missing - -

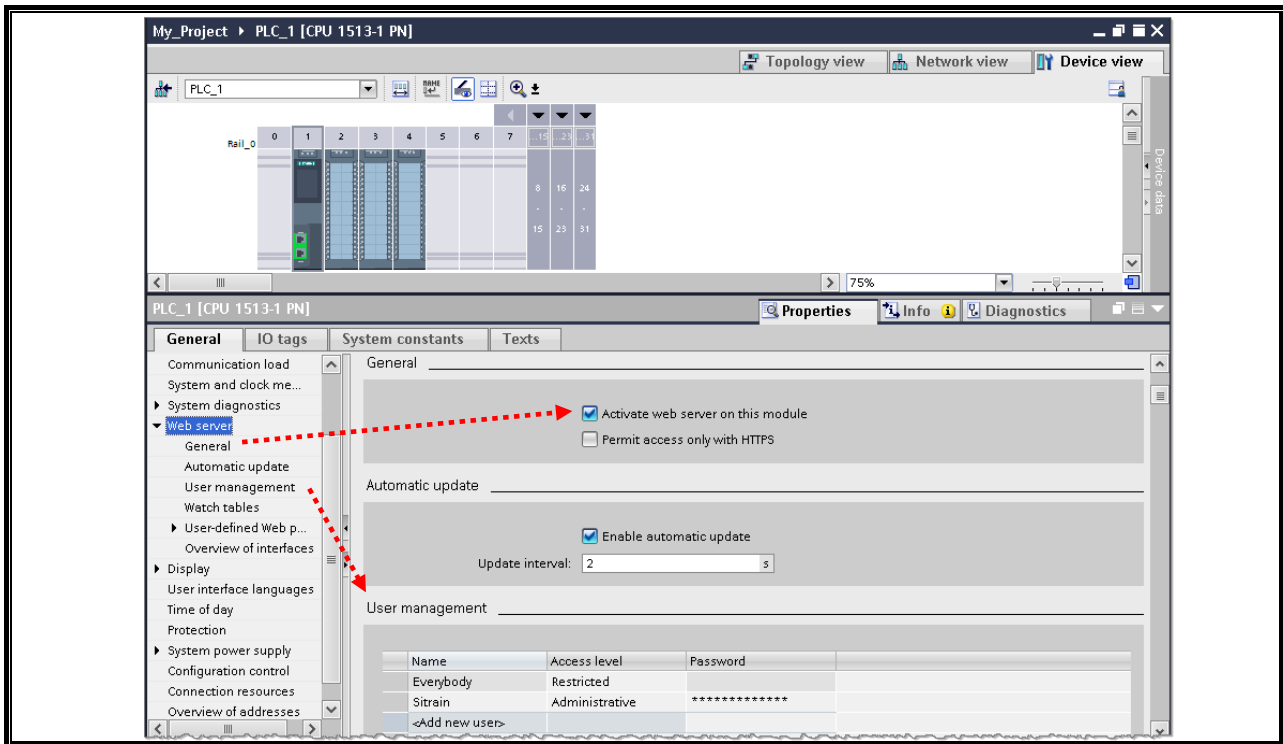
Details: 1
 CPU info: Communication initiated request: WARM RESTART Pending startup inhibit(s): - No startup inhibit set
 CPU changes from STARTUP to RUN mode |> Component: PLC_1.CPU Exec Unit_1 |> Path: - |> HW_ID= 00052 |> Operating mode control
 Event ID: 16# 02:400E

Diagnostic Buffer

You can set the number of entries of the diagnostic buffer:

- In CPU STOP, all available entries are read out of the CPU and displayed here.
- In CPU RUN, the number of read and displayed entries depends on the CPU or the configuration. For S7-400 in general and S7-300 as of Firmware V3.1, this number can be parameterized in the properties of the CPU. For S7-300 up to Firmware V3.1, this number is defined by an internal module setting (=10).

10.6.4. Parameterizing the CPU Web Server



Web Server Activation

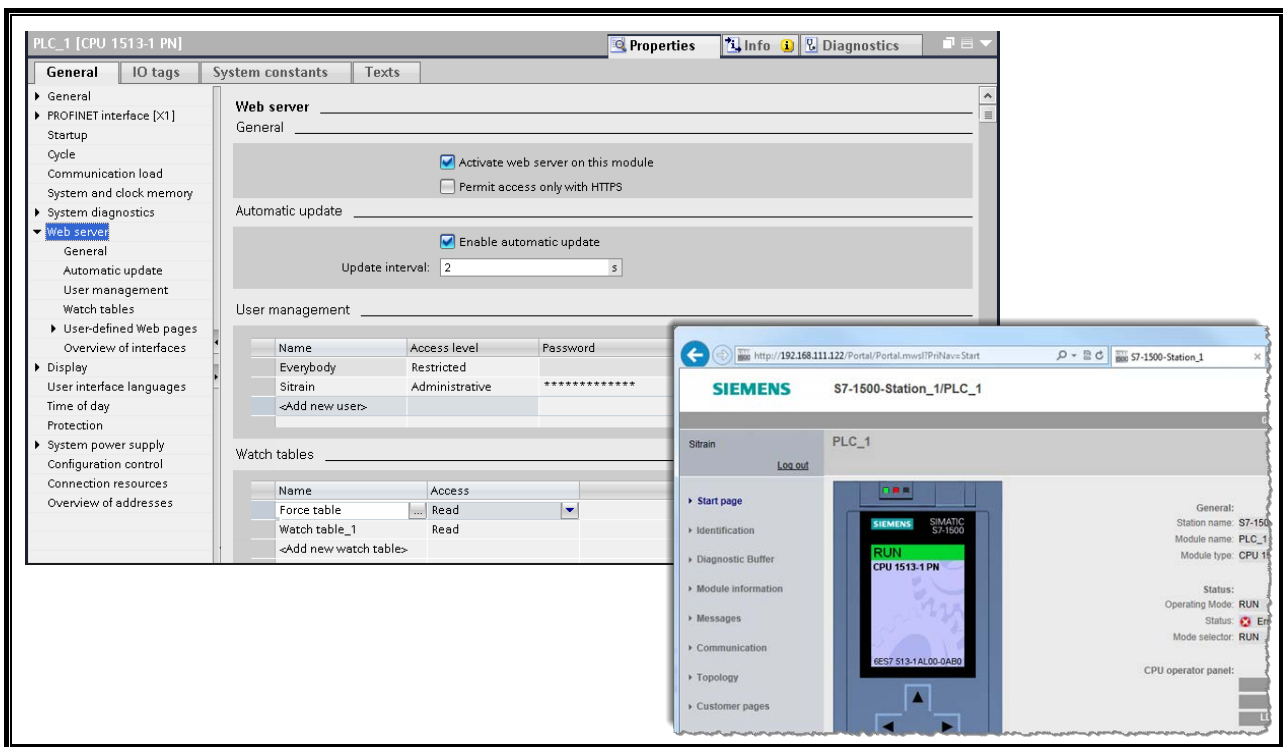
In the default settings of the CPU, the web service is deactivated. You activate the service as shown in the picture.

User Management

In the User Management, the user "Everybody" with the access level "Minimum" is set up by default. When accessing the CPU Web server, this user can only read the Start page of the web server.

Users whose access level is "Administrative" can, on the other hand, query all information of the CPU web server and also have – depending on the web site called – write access to the CPU.

10.6.5. Exercise 3: Activating and Parameterizing the Web Server



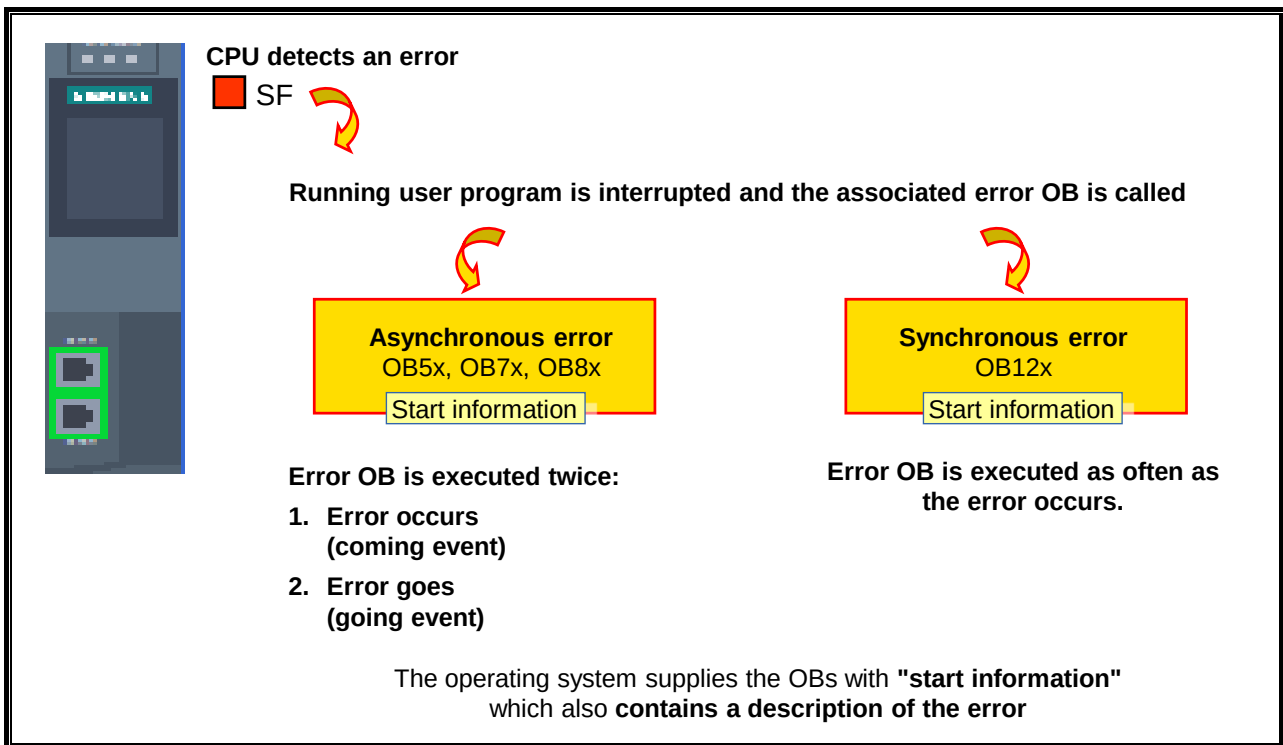
Task:

The CPU Web server is to be activated and set up so that it can be used.

What to Do

1. Open the Properties of the CPU.
2. Under the item Web server → General, activate the Web server on the CPU,
3. Activate (enable) the Automatic update and set the Update interval.
4. Create a new user with the name 'Admin' who is given administrator rights (authorize for all accesses).
5. Give the new user "Admin" a password.
6. Under the item Watch tables, select the already existing watch table "Force table".
7. Save, compile and download the modified hardware configuration.
8. Test the changes by calling the CPU Web server on the Field-PG using the Internet Explorer.

10.7. Diagnostics using the S7 Program



Standard Reaction of the CPU when it detects an Error

If the CPU's operating system detects an error, this error is entered in the diagnostic buffer of the CPU and the error OB assigned to the error is called. In this OB program, the user has the opportunity to program a CPU reaction (switch CPU to STOP, send message etc.) to the error that occurred.

Asynchronous Errors

Asynchronous errors occur regardless of the program execution and accordingly cannot be traced to a defined point in the program:

- for example, module failure
- for example, I/O fault (exceeding the measuring range, wire break)

These OBs are executed exactly twice:

1. When the error occurs (with a start information that contains the identifier "(in)coming")
2. When the error goes (with a start information that contains the identifier "(out)going")

Synchronous Errors

Synchronous errors occur through the program execution and accordingly can be traced to a defined point in the program:

- for example, indirect access to a non-existent Array element
- for example, direct access to non-existent / defective I/O

These OBs are executed as often as the error occurs. If, for example, in a cyclically executed block, there is an indirect access to a non-existent Array element, the error OB "Programming error" (OB121) is also called and executed cyclically.

10.7.1. Start Information of the Error OBs

L-Byte			
0 / 1	Event class and ID	Error code	Management information
2 / 3	Priority class	OB No.	
4 / 5	Error info (such as, block type in which the error occurred)		Error information
6 / 7	Error info (such as, address at which the error occurred)		
8 / 9	Error info (such as, block number of the faulty block)		
10 / 11	Error info (such as, relative address of the error-causing instruction)		
12 / 13	Year	Month	Start time
14 / 15	Day	Hours	
16 / 17	Minutes	Seconds	
18 / 19	1/10 Second, 1/100 Second	1 /1000 Second, Weekday	

Start Information

When the operating system calls organization blocks, the user is provided with a uniform system start information in the local data stack. The start information or the pre-declared variables are OB-specific and have a length of 20 bytes in total.

The start information as well as their absolute L-stack addresses is only completely available for those OBs where the block attribute "Optimized block access" is not activated.

Access to Start Information

Since the start information variables are supplied with defined values by the operating system when the OB is called, the user program can read them without them having been written to beforehand. The access to the variables can be symbolic or absolute whereby the symbolic access is recommended for reasons of program readability and the avoidance of addressing errors.

Notes

With the help of the instruction "RD_SINFO", the start information for OBs with "Optimized block access" can also be read out.

The user can change and/or supplement the standard declaration table. However, you must make sure that the dimensions or the memory space requirements and with that the initial (start) addresses of the given variables are not changed. Accordingly, additional variables may only be declared at the end, after the standard variables.

The meanings of the variables are explained to you in the online Help or in the Standard and System Functions manual.

10.7.2. Global Error Handling with Asynchronous Error OBs

Type of Error	Example	Error OB	Priority
Time Error: Max. allowed cycle time exceeded once System reaction with OB: RUN without OB: STOP Max. allowed cycle time exceeded by more than double System reaction with OB: STOP	Exceeding the max. allowed cycle time, Delayed call of a time OB	"Time error interrupt" (OB80)	S7-1200*/1500: Can be set: 2 to 26
Diagnostic Interrupt System reaction w/o OB: RUN	Wire break at diagnostics-capable module, power supply error	"Diagnostic error Int." (OB82)	S7-1200*/1500: Can be set: 2 to 26
Insert / Remove Interrupt System reaction w/o OB: RUN	Insert / Remove a module	"Pull or plug modules" (OB83)	S7-1200*/1500: Can be set: 2 to 26
Rack Failure System reaction w/o OB: RUN	Failure of a DP-slave or an IO-Device	"Rack or station failure" (OB86)	S7-1200*/1500: Can be set: 2 to 26

(*= cannot be set)

Any event leads to an entry in the diagnostics buffer!

Asynchronous Errors

Asynchronous errors occur asynchronous to the program execution and accordingly cannot be traced to a defined point in the program.

Time Errors

They occur when the current cycle time exceeds the cycle monitoring time set in the hardware configuration.

Diagnostic Interrupts

They are triggered by diagnostics-capable modules, such as, analog modules in case of a fault (for example, wire break).

Insert/Remove Interrupts

These interrupts are triggered when modules are inserted or removed (only S7-400). When a module is inserted, the operating system checks whether the correct module type was used. With this function, it is possible to insert/remove modules while the system is running.

Rack Failure

A rack failure is detected with the failure of a rack, a subnet or a station of distributed I/O.

10.7.3. Diagnosis of Asynchronous Errors with "DeviceStates"

Parameter	Declaration	Data type	Memory area	Description
LADDR	Input	HW_IOSYSTEM	I, Q, M, L or constant	Hardware identifier of the PROFINET IO or DP master system (see description below)
MODE	Input	UINT	I, Q, M, D, L or constant	Selection of status information to be read (see description below)
RET_VAL	Return	INT	I, Q, M, D, L	Status of instruction (see description below)
STATE	InOut	VARIANT	I, Q, M, D, L	Buffer for status of the IO devices or the DP slaves (see description below)

Description

The instruction "DeviceStates" supplies status information for all modules in an IO-System. Beyond that it displays whether the status information to be read applies to at least one of the IO-Devices or DP-Slaves. The instruction can be called in the cyclic OB as well as in an Interrupt OB (such as, OB82 - diagnostic interrupt).

Parameter MODE

Through the parameter MODE you can select which status information is to be read

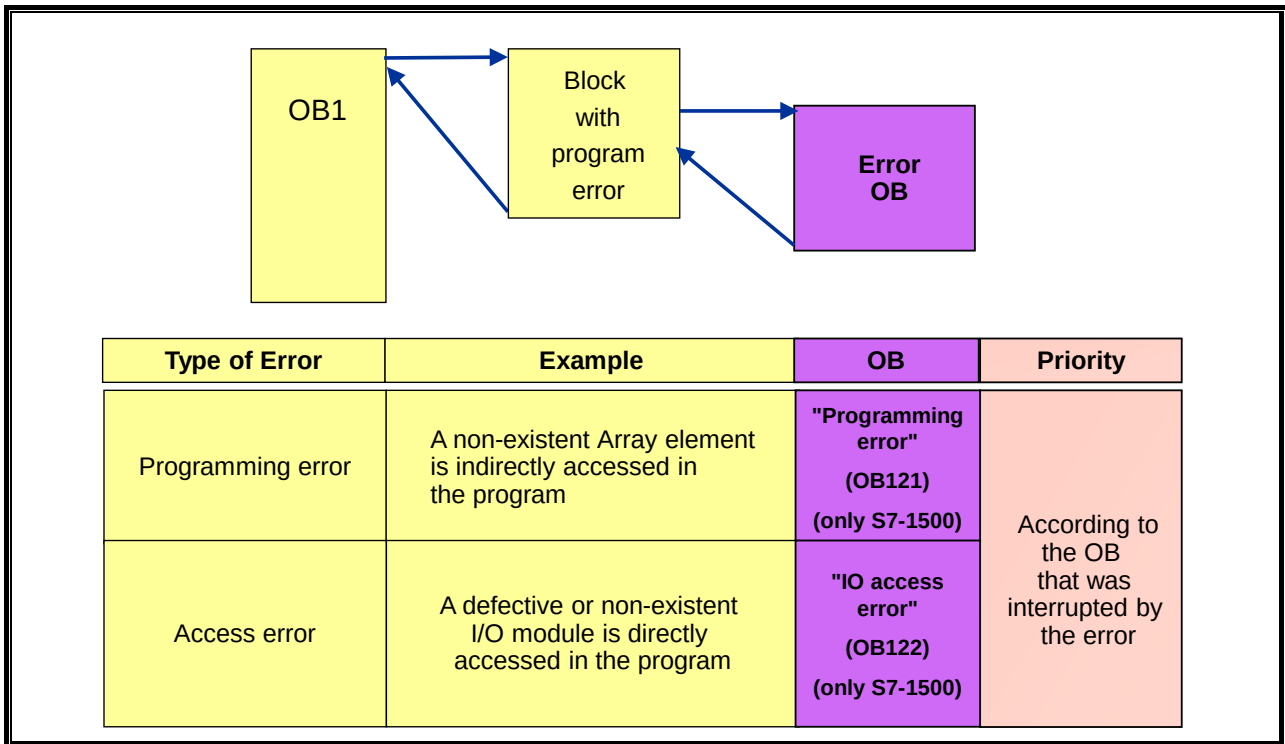
- 1: IO-Devices/DP-Slaves are configured
- 2: IO-Devices/DP-Slaves faulty
- 3: IO-Devices/DP-Slaves deactivated
- 4: IO-Devices/DP-Slaves exist
- 5: IO-Devices/DP-Slaves, in which a problem has occurred (such as, "Maintenance necessary", "Not accessible", "Error occurred")

Parameter STATE (Array [0...1023] of BOOL)

If the status selected via MODE applies to an IO-Device/DP-Slave, the following bits are set to "1" at parameter STATE:

- Bit 0 = 1: Group display. Bit n of at least one IO-Device/DP-Slave was set to "1".
- Bit n = 1: The status selected via MODE applies to the IO-Device/DP-Slave.
- For a PROFINET-IO System, Bit n corresponds to the device number of the respective IO-Device (see Properties of the PROFINET interface in the Device view and Network view)
- For a PROFIBUS-DP System, Bit n corresponds to the PROFIBUS address of the DP-Slave (see Properties of the DP-Slave in the Device view and Network view)

10.7.4. Global Handling of Synchronous Errors with Error OBs: Principle



Synchronous Errors

Synchronous errors occur synchronously to the program execution and accordingly can be traced to a defined point in the program.

With a programming error, the programming error OB (OB121) is called; with an access error, the I/O access error OB (OB122) is called.

If, in case of a programming error, OB121 is not available in the CPU, the CPU switches to the STOP state.

10.7.5. Global Handling of Synchronous Errors with Error OBs: CPU Behavior

Type of error	Example	OB	Priority
S7-1200 Programming error Access error System reaction w/o OB: RUN	Access to non-existent DB Direct access to non-existent or defective I/O module	no OB exists	✘
Programming error System reaction w/o OB: STOP	Access to non-existent DB	"Programming error" (OB121)	Can be set: 2 to 26
S7-1500 Access error System reaction w/o OB: RUN	Direct access to non-existent or defective I/O module	"IO access error" (OB122)	

All events lead to an entry in the diagnostic buffer!

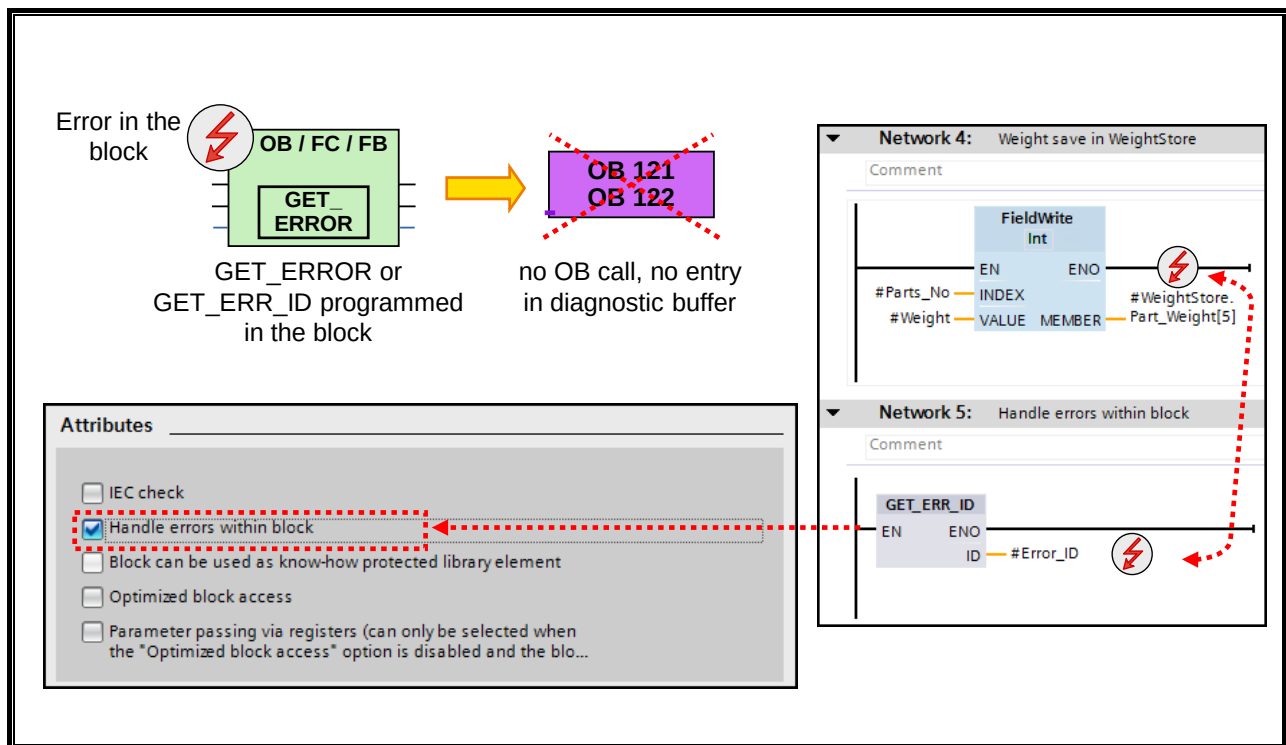
Synchronous Errors

Synchronous errors occur synchronously to the program execution and accordingly can be traced to a defined point in the program.

Priority:

The priority of synchronous error OBs can be set from 2 to 26.

10.7.6. Local Handling of Synchronous Errors



Local Error Handling of Synchronous Errors

If the "Handle errors within block" is programmed, then if this block is integrated in another program, the adjustment of possibly already existing error OBs is not necessary. Thus the "local error handling" contributes to designing the block as **re-usable**.

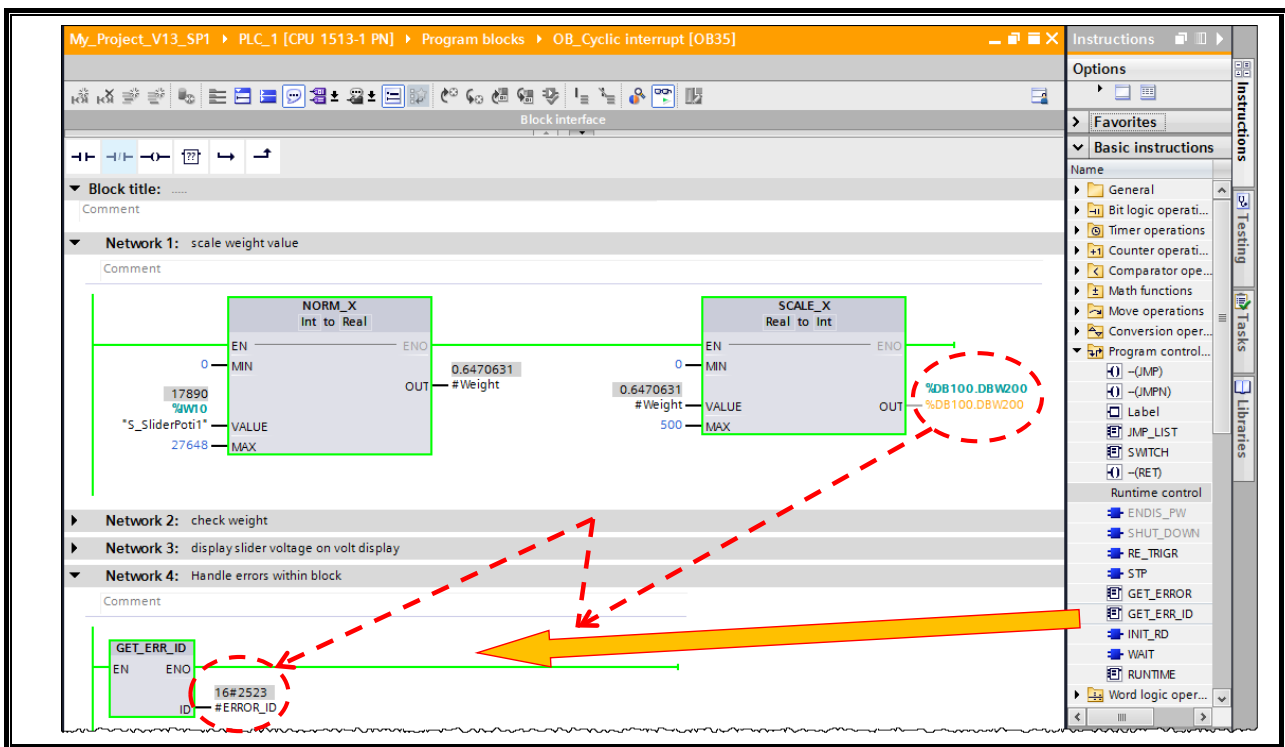
Local error handling with the "GET_ERROR" or "GET_ERROR_ID" block has to be programmed separately in every block and has no impact on the calling and called blocks.

With the programming of the call of "GET_ERROR" or "GET_ERROR_ID", the block receives the attribute "Handle errors within block".

If the call of "GET_ERROR" or "GET_ERROR_ID" is programmed after the instruction at which an error could occur, GET_ERROR supplies an "ERROR_STRUCT" with a detailed error description as a result, and "GET_ERROR_ID" supplies an error number whose meaning can be read in the online help. Furthermore, the call of the associated error OB (OB121 or OB122) is suppressed.

If the call of "GET_ERROR" or "GET_ERROR_ID" is programmed before the instruction at which an error could occur, the blocks do not supply an error description and only the call of the associated error OB (OB121 or OB122) is suppressed.

10.7.7. Exercise 4: Integrating Local Error Handling



Task

In "OB_Cyclic interrupt" (OB35), program an access to a non-existent data block (as shown in the picture) and test the system reaction without and with local error handling.

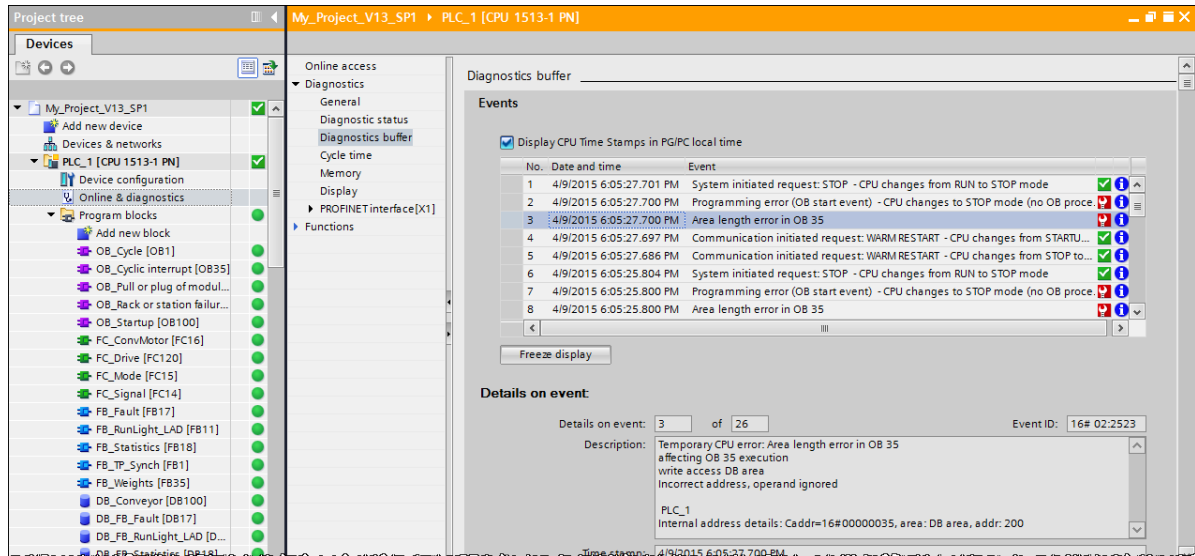
What to Do

1. In "OB_Cyclic interrupt" (OB35), program an access to a non-existent data block as shown in the picture (without NW4 Handle errors within block)
2. Download the modified block into the CPU.
3. Perform a CPU restart, by switching the CPU mode selector switch to STOP and then back to RUN.

Exercise is continued on the next page

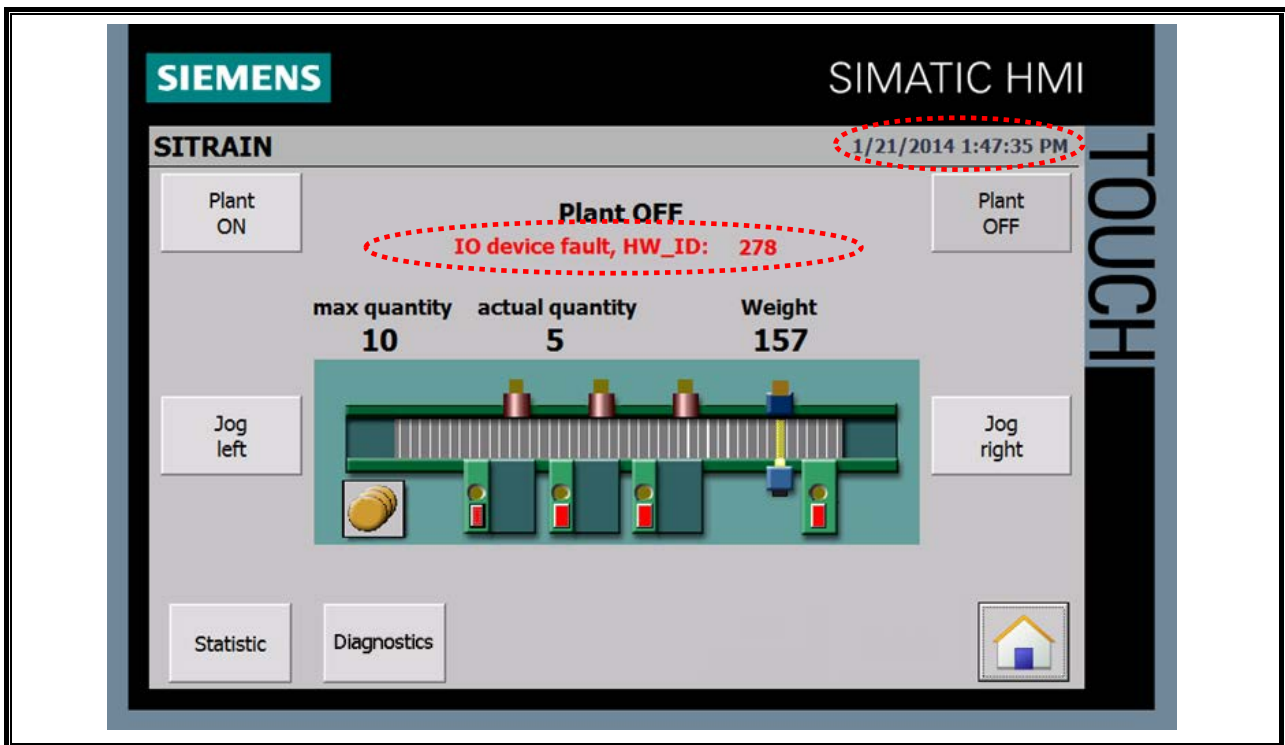
4. Read out the diagnostic buffer. Proceed as follows:

- Select your S7-1500 station
- "Go Online"
- Start "Online & diagnostics"
- There, select the diagnostic buffer in the "Diagnostics" tab



5. In "OB_Cyclic interrupt" (OB35), after the access to the non-existent data block, program a new network with local error handling. For this, insert a new network and call the instruction GET_ERROR_ID (you will find the instruction in "Basic instructions → Program control operati..."). Pass the instruction a local, temporary variable at the Output ID.
6. Download the extended block into the CPU and perform a restart.
7. The CPU should now stay in RUN in spite of programming errors. Open OB35, activate the function "Monitor block".
8. At the Output ID of the instruction, an Error-ID is output. Read about its meaning in the online help (select instruction → F1).

10.7.8. Task Description: Displaying an IO-Device Error and Time-of-day Synchronization with the CPU



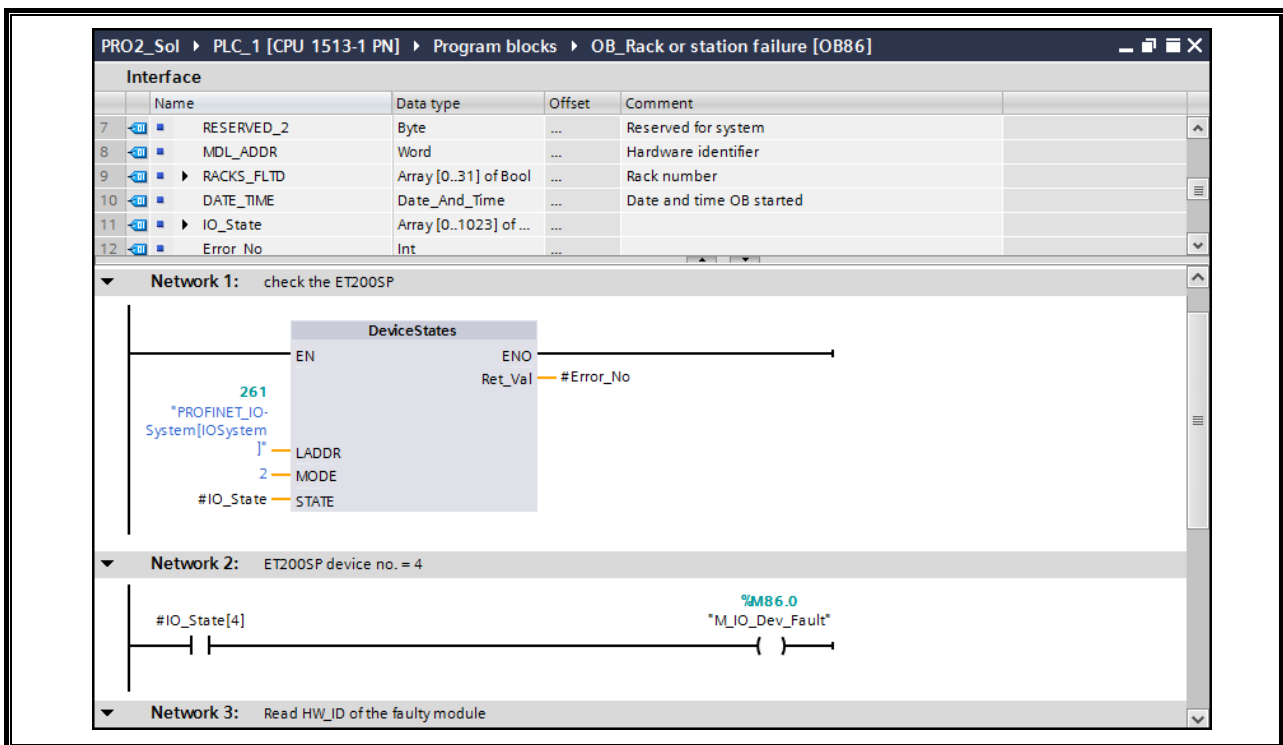
Task Description

As soon as an IO-Device has failed or an error has occurred on the IO-Device, the message shown in the picture – with the HW_ID of the IO-Device concerned or the module concerned – is to appear in the touchpanel screen "Conveyor".

Furthermore, the current date and time that is synchronized with the CPU is to be displayed in every touchpanel screen at the upper right-hand corner.

The message indicating the HW_ID is already configured in the touchpanel. All you have to do is create an S7 program which assigns the tag "M_IO_Dev_Fault" (M86.0) status '1' as long as the message is to be displayed, and which assigns the tag "MW_IO_DevHW_ID" (MW88) the HW_ID of the station concerned or the module concerned.

10.7.9. Exercise 5: Displaying an IO-Device Error



Task

Create an S7 program that, in case of station failure, or removal / insertion or CPU restart...

- ...assigns the tag "M_IO_Dev_Fault" (M86.0) status '1' as long as the message is to be displayed. (IO_State[4] since the device number of the Profinet interface of the ET200SP is equal to 4 – see Properties of the Profinet interface)
- ...assigns the tag "MW_IO_DevHW_ID" (MW88) the HW_ID of the station concerned or the module concerned. ("MW_IO_DevHW_ID" == #MDL_ADDR)

Note:

The text with No. "IO device fault, HW_ID: xxx" (see picture on the previous page) shown in the touchpanel screen "Conveyor" is already configured and is displayed as long as "M_IO_Dev_Fault" (M86.0) has status '1'. For this, the No. xxx must be stored in the tag "MW_IO_DevHW_ID" (MW88).

What to Do

- Create the new blocks "OB_Pull or plug of module" (OB83) and "OB_Rack or station failure" (OB86).
- In these newly created OBs and in "OB_Startup" (OB100), program the required functions.
NW3: In the online Help of the blocks, read in which tags of the start information the HW-ID is stored.
- Download all program changes into the CPU.
- Check the function by provoking an IO-Device error by pulling and inserting an ET200SP module or by switching off the ET200SP. (Switch under the touchpanel)
- Save your project.

10.7.10. Exercise 6: Determining the Device Downtime

The screenshot shows a SIMATIC Manager interface. At the top, a hardware rack is displayed with three components: an ET200SP IM 155-6 PN ST (labeled PLC_1), a PLC_1 CPU 1513-1 PN, and a Touchpanel TP700 Comfort. A red lightning bolt icon is next to the ET200SP, and a green line labeled 'PN/IE_1' connects it to the CPU. Below the rack, the 'Watch and force tables' window is open, showing 'WatchTable_1'. The table contains the following data:

	Name	Address	Display format	Monitor value
1	"M_IO_Dev_Fault"	%M86.0	Bool	FALSE
2	"MW_IO_DevHW_ID"	%MW88	Hex	16#0116
3	"MD_DeviceFail"	%MD90	TIME_OF_DAY	TOD#12:40:22.753
4	"MD_DeviceReturn"	%MD94	TIME_OF_DAY	TOD#12:50:33.270
5	"MD_FailDuration"	%MD98	Time	T#10M_10S_517MS

Task

The length of time that an IO-Device is down is to be determined and stored in the memory double-word "MD_FailDuration" (MD98).

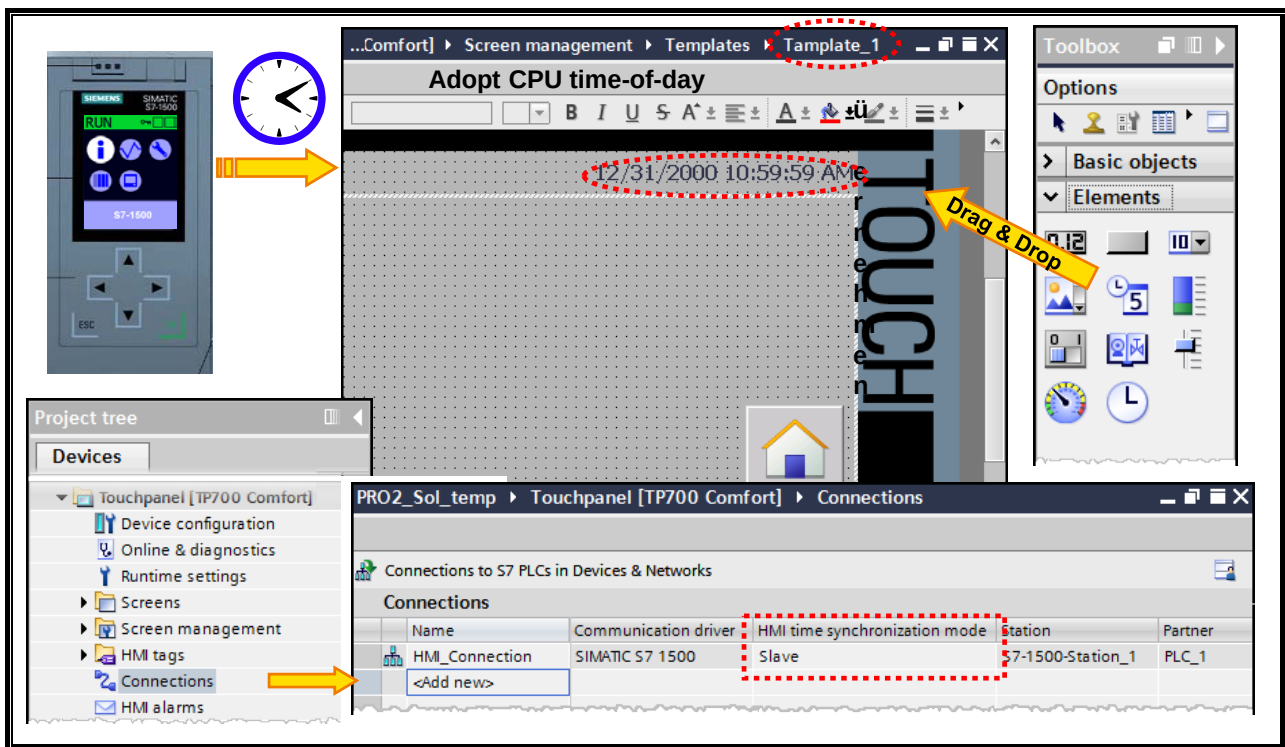
What to Do

1. After failure or error and subsequent recovery of the ET200SP, the downtime is to be displayed on the PG. Simulate a module failure on the ET200SP by pulling and then reinserting a module.
2. Expand the program of the relevant OB in such a way that the above-mentioned functions are fulfilled.
3. With the PG, check the program function by means of a suitable Watch table (see picture).
4. Save your project.

Solution Hints:

To calculate the slave downtime use the "Extended instruction" T_CONV" and parameterize the block in such a way that the time-of-day of data type "TOD" (Time of Day) can be extracted from a time stamp of the data type "DT" (Date and Time).

10.7.11. Exercise 7: HMI Devices Time-of-day Synchronization with the CPU



Task

The current date and time that is synchronized with the CPU is to be displayed in every touchpanel screen at the upper right-hand corner.

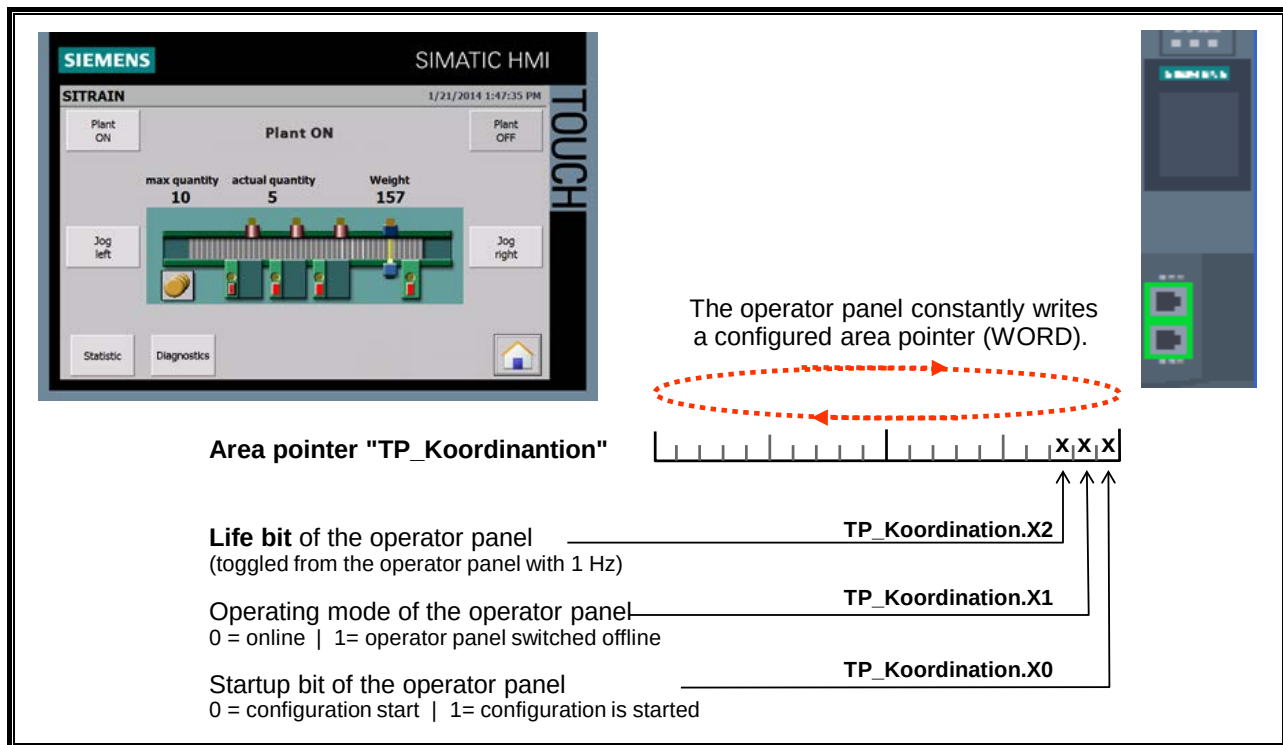
What to Do

1. Open the "Connections" of the touchpanel and there under "HMI time synchronization mode" select the mode "Slave". With that, the touchpanel synchronizes its own time with that of the CPU.
2. In the "Screen management" in the screen "Template_1", configure a date/time output field.
3. Download the modified configuration into the touchpanel and check the function.
4. Save your project.

Note:

So that the times (of day) match, the summer time (daylight savings time) either has to be switched on or switched off for both devices and the same time zones have to set. In the CPU this is (Settings → Time of day) and in the touchpanel (Settings → Date/Time).

10.8. Diagnosis of the Operator Panel Status using the CPU Program



General

In some systems it is important that the CPU program knows whether connected operator panels are currently functioning or, if necessary, have failed or are switched off.

Operator panels are capable of cyclically writing to a tag (in the picture the tag "TP_Koordination") in order to store certain status information.

Life Bit

As long as a connection exists between the operator panel and the S7-CPU, the operator panel toggles this bit with a frequency of about 1Hz. With this, it can be evaluated in the CPU program whether the connection to the operator panel still exists.

Operating Mode

As soon as the operator panel is switched offline, all connections are automatically switched offline and the operating mode bit is set to = 1.

During normal operation of the operator panel (operator panel online and connection online), the operating mode bit is set to = 0.

In this way, the CPU program can detect whether the operator panel (!!!) was switched offline.

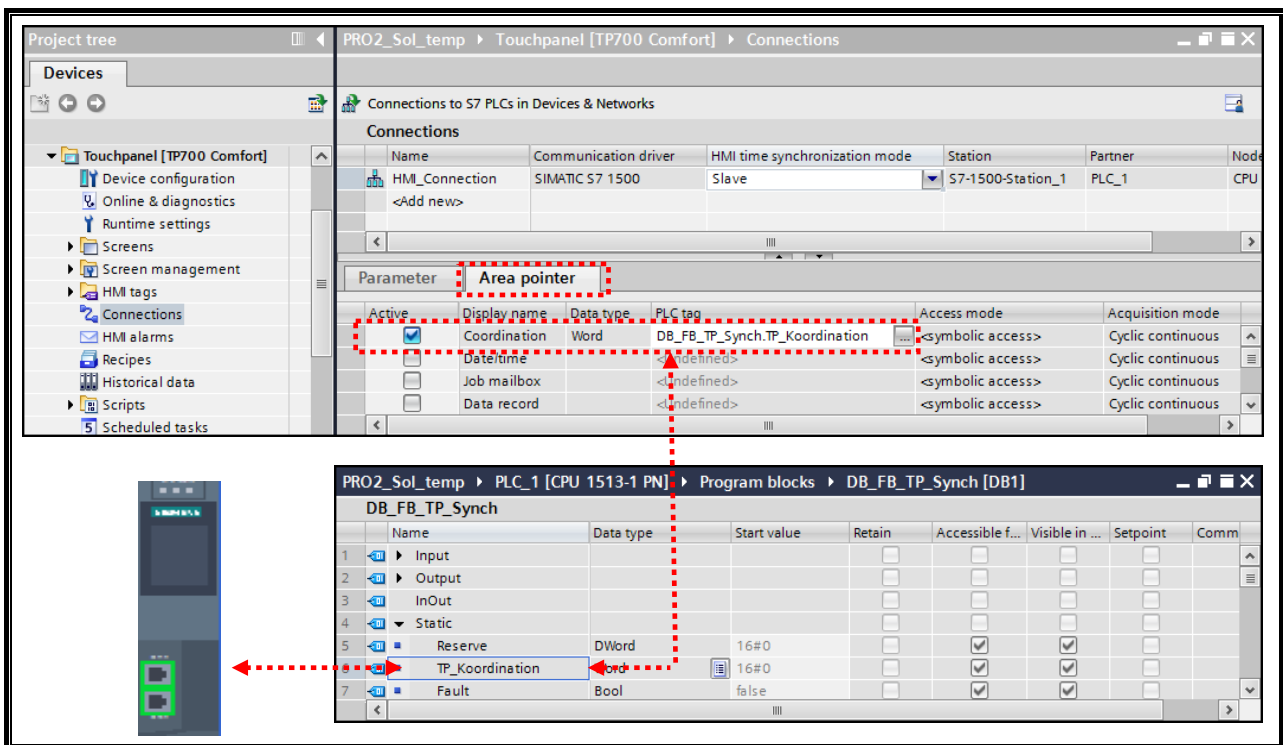
Note:

This bit does not give any direct information about the current status (on/offline) of the connection. This bit is not set when the individual connections are switched offline. This can only be evaluated through the life bit.

Startup Bit

The startup bit is briefly set to "0" by the operator panel during the configuration start. After ending the startup operation, the bit is continuously "1" until the next configuration start takes place.

10.8.1. Configuring the Life Bit



Principle

The operator panel cyclically writes status information into a tag (also called "area pointer") of the CPU which can be evaluated using the CPU program. This status information contains, among other things, a so-called "life bit".

The tag that serves as an "area pointer" can be created in the memory bit area or in a data block. In the example shown, the tag is stored in an instance data block since it is declared as a local static variable within an FB.

TP Configuration

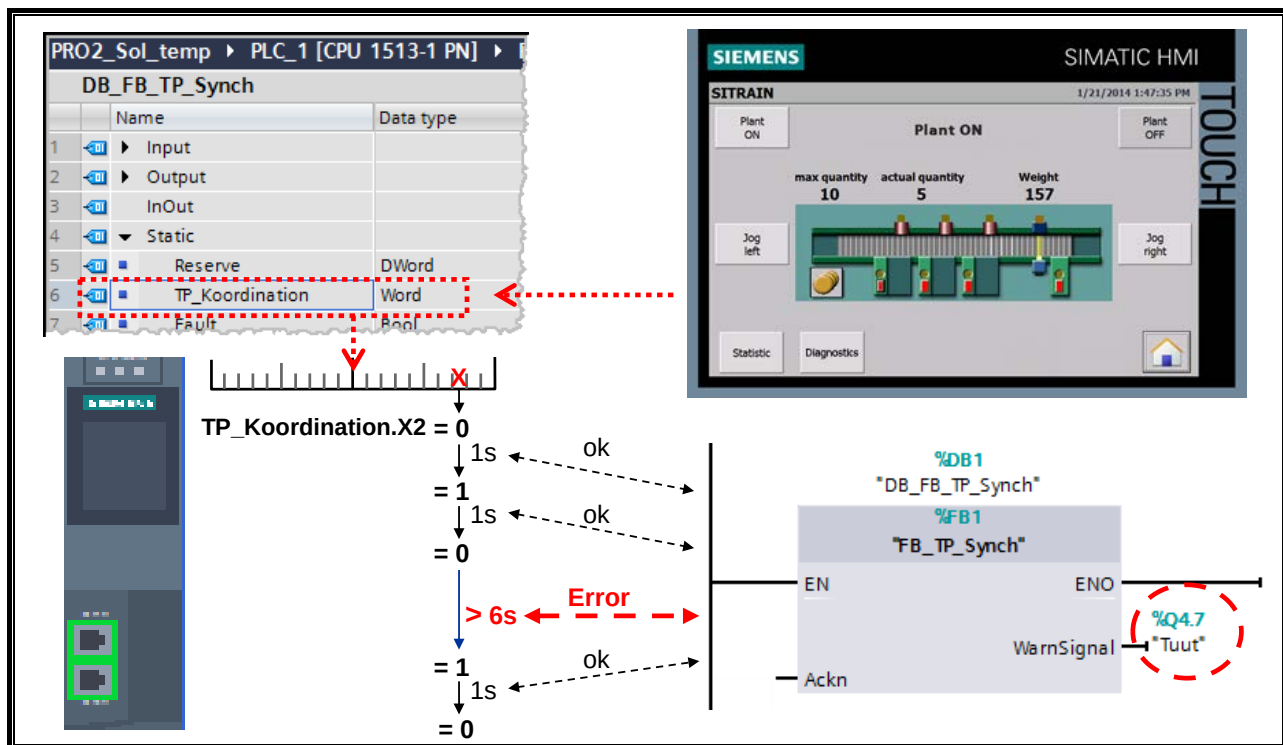
The writing of the status information into the CPU is configured via a so-called "area pointer" – called "TP_Koordination" in the example shown – which must be activated in the "Connections" editor of the operator panel.

By activating the area pointer (in the "Active" column) and defining the PLC tag that is to be used as the area pointer, the writing of the status information is activated.

CPU Program

Toggleing the "life bit" can be evaluated by means of timers in which the rising and the falling edge of the life bit is used to start or retrigger the timer. Should the programmed time period completely run out for one of the timers, the connection between the touchpanel and the CPU is faulty.

10.8.2. Exercise 8: Diagnosis of the Online Connection TP <--> CPU



Task

As long as the life bit or the local static tag `TP_Koordination.X2` declared in "FB_TP_Synch" is toggled from the operator panel with a frequency of 1Hz there is no connection fault and no reaction of the part of the CPU program is necessary. If the life bit doesn't change its state for longer than 6 seconds, there is a connection fault and "FB_TP_Synch" switches on the conveyor model horn "Tuut"(Q4.7) as a warning. This can be acknowledged by the operator via the simulator switch "S_Acknowledge" (I 1.0) or on the touchpanel.

What to Do

1. Create the new "FB_TP_Synch" (FB1) block, provide it with the interface shown in the picture and also create the static tag `TP_Koordination` (data type WORD) in the FB which is to serve as the area pointer.
2. Program the evaluation of the "life bit": As soon as the life bit or the bit `TP_Koordination.X2` does not change its state for longer than 6 seconds, the horn is to be switched on – acknowledgeable via "S_Acknowledge" (I 0.7) or on the touchpanel. For this, use IEC timers which you declare as multiple instances internally in the block.
3. Under *Touchpanel* → *Connections* → *Area pointer*, activate the "Coordination" and specify the tag "TP_Koordination" as the PLC tag which is stored in the instance DB of "FB_TP_Synch" (FB1) (see picture on the previous page).

Note

You can simulate the operator panel failure by, for example, exiting the runtime on the operator panel or by switching off the power supply of the HMI device.

11

Contents

11.	Structured Control Language SCL	11-2
11.1.	Task Description: Storing Weight Values in a DB Tag	11-3
11.2.	Program Creation with SCL	11-4
11.3.	Advantages of the SCL Programming Language	11-5
11.4.	Creating an SCL Block.....	11-6
11.4.1.	Editing an SCL Block	11-7
11.4.2.	Operators	11-8
11.4.3.	Control Structures	11-9
11.4.4.	Direct Addressing (Examples)	11-10
11.4.5.	Indirect Addressing (Examples)	11-11
11.4.6.	Calling an SCL Block	11-12
11.4.7.	Monitoring SCL Blocks.....	11-13
11.5.	Exercise 1: Copying the SCL Block from the Library.....	11-14
11.5.1.	Exercise 2: Commissioning "FC_Weight_SCL" (FC38).....	11-15

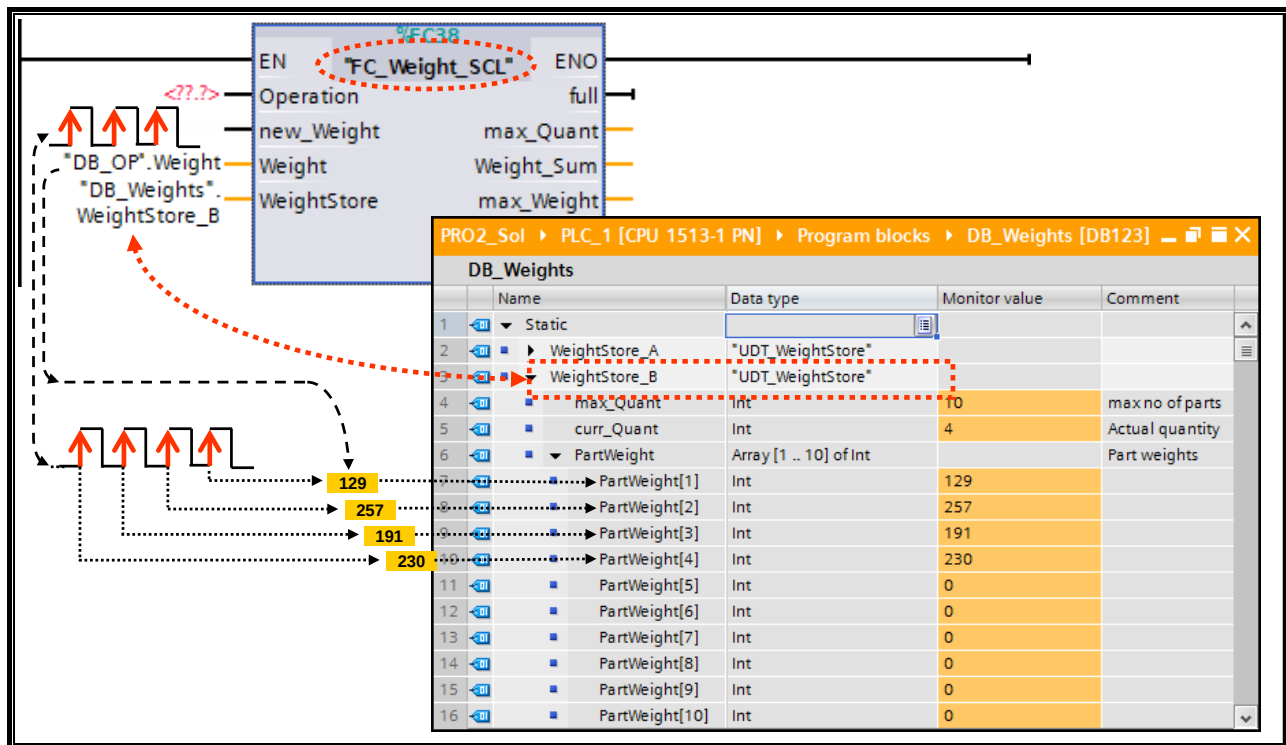
11. Structured Control Language SCL

At the end of the chapter the participant will ...

- ... be familiar with the application areas of SCL
- ... be able to commission a given SCL block



11.1. Task Description: Storing Weight Values in a DB Tag



Task Description

When "P_Operation" (Q0.1) is switched on, whenever a part with a valid weight ($100 \leq \text{weight} \leq 400$) passes the light barrier, the currently set weight value on the simulator slide control "S_Slider1" (IW10) is to be stored in the "DB_Weights" (DB35) data block in the *Weight_Store2.Part_Weight[x]* variable.

Through its output *#full*, the block signals when *Act_No* $\geq$ *Setp_No* (can be set on the touchpanel). Further part transportations and with that the storing of further part weights is then no longer possible (interlocks in "FC_ConvMotor" (FC16) and "FC_Signal" (FC14)).

Only after "P_Operation" (Q0.1) has been switched off and subsequently switched back on again thus resetting *Act_No* to 0 and overwriting all stored *Part_Weights* with 0, can new part transportations be restarted.

11.2. Program Creation with SCL

Structured Control Language as text-based structured high-level language

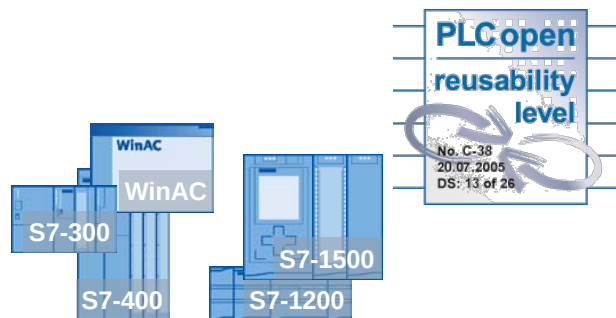
```

3  [*]
4  // Example FOR
5  FOR #index := 0 TO "GRENZE" DO
6    // Statement section
7    #counter:=#counter + #index;

8    "Speed" := #counter;
9  END_FOR;
10 // Example IF
11 IF...
12 // Example CASE
13 CASE #trigger OF
14   1: // Statement section case 1
15     #trigger := 2;
16   2..4: // Statement section case 2 to 4
17     #trigger:= #trigger + 1 ;
18 ELSE
19   // Statement section ELSE
20   #trigger:=0;
21 END_CASE;
22 "Speed_Control" := #trigger;
23 (*...
24 IF #trigger = 0 THEN
25   // Statement section IF
26   #trigger := 1;
27 END_IF;
28

```

- Structured Text
- Complex calculations & algorithms
- Database operations, for example, sorting data
- Program code exchangeable between S7-1200 and S7-1500 as well as S7-300, S7-400 and WinAC



S7-SCL

SCL (Structured Control Language) is a PASCAL-similar high-level textual language. It simplifies the programming of mathematical algorithms and complex data processing tasks for PLCs. SCL therefore also enables S7 PLCs to be used for more complex tasks such as closed-loop control or statistical evaluation.

SCL offers the functional scope of a high-level language such as:

- Loops
- Alternatives
- Branch distributors, etc.

combined with PLC-specific functions such as:

- Bit accesses to the I/O, bit memories, timers, counters etc.
- Access to the symbol table
- STEP7 block accesses

11.3. Advantages of the SCL Programming Language

Solution in STL for S7-300/400

```

9  L   P##WeightStore
10 LAR1
11 L W [ AR1 , P#0.0 ]
12 T   #DBNo
13 OPN DB [ #DBNo]
14 L D [ AR1 , P#2.0 ]
15 LAR1
16
17 L DBW [ AR1 , P#2.0 ]
18 +   1
19 T DBW [ AR1 , P#2.0 ]
20
21 T   #part_no
22 L   P#2.0
23 *D
24 L   P#4.0
25 +D
26 TAR1
27 +D
28 T   #pointer
29 L   #Weight
30 T DBW [ #pointer]

```

Solution in SCL for S7-300/400 AND S7-1200/1500

```

#WeightStore."curr_Quant":=#WeightStore.curr_Quant+1;
#part_no:= #WeightStore."curr_Quant";
#WeightStore."PartWeight"[#part_no]:=#Weight;

```

Solution in STL for S7-1500

```

3  L   #WeightStore.curr_Quant
4  +   1
5  T   #WeightStore.curr_Quant
6  T   #part_no
7
8  L   #Weight
9  T   #WeightStore.PartWeight[#part_no]

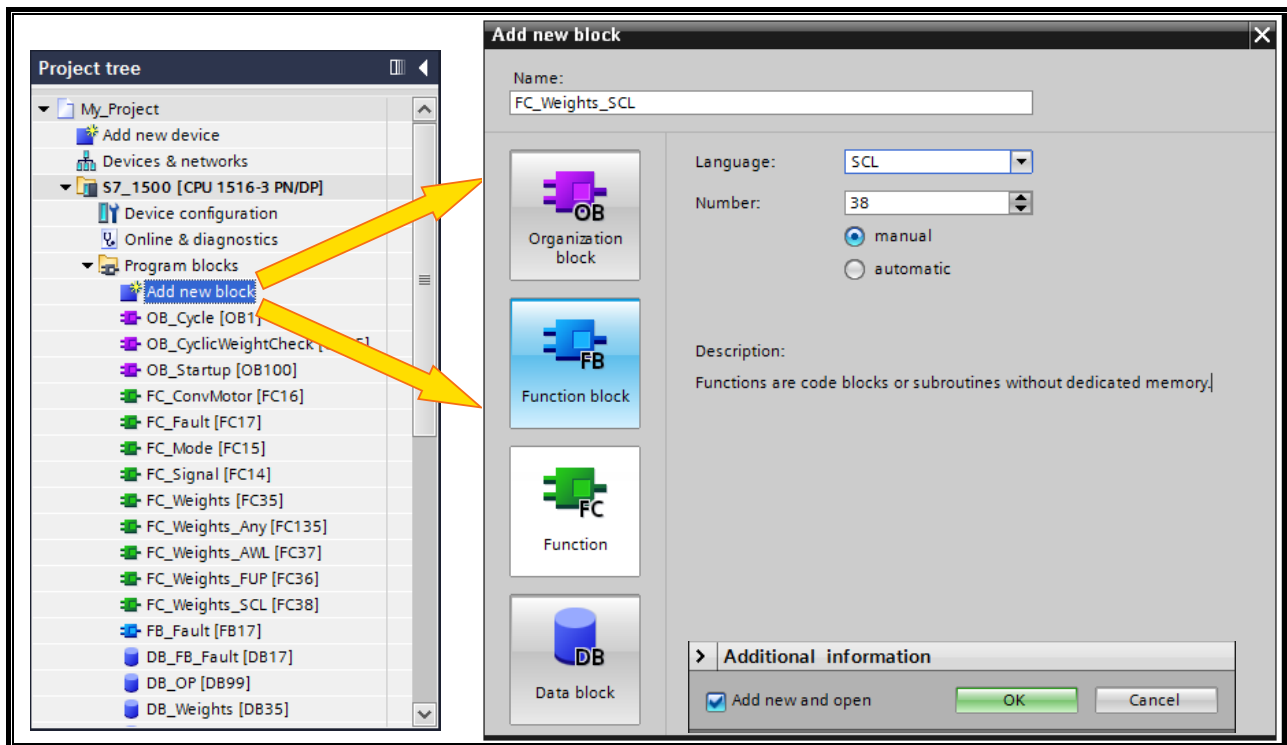
```

Solution in LAD/FBD for S7-1200/1500

Advantages of SCL

- Easy to learn programming language, especially for beginners
- Easy to read (understandable) programs are created.
- Easier programming of complex algorithms and processing of complex data structures
- System integration in S7 languages, such as STL, LAD and FBD.
- Relatively easy for PLC technicians to understand through similarity with S7 languages.

11.4. Creating an SCL Block



Creating an SCL Block

SCL blocks are inserted in the same manner as blocks in the programming languages LAD/FBD/STL. Depending on the task, SCL blocks can be of the FC or FB block type.

Even the internal structure in the declaration and statement section is identical to the LAD/FBD/STL blocks:

Declaration Section:

The IN, OUT and INOUT parameters as well as the local temporary and local static variables of the block are declared in the declaration section of a block.

Statement Section:

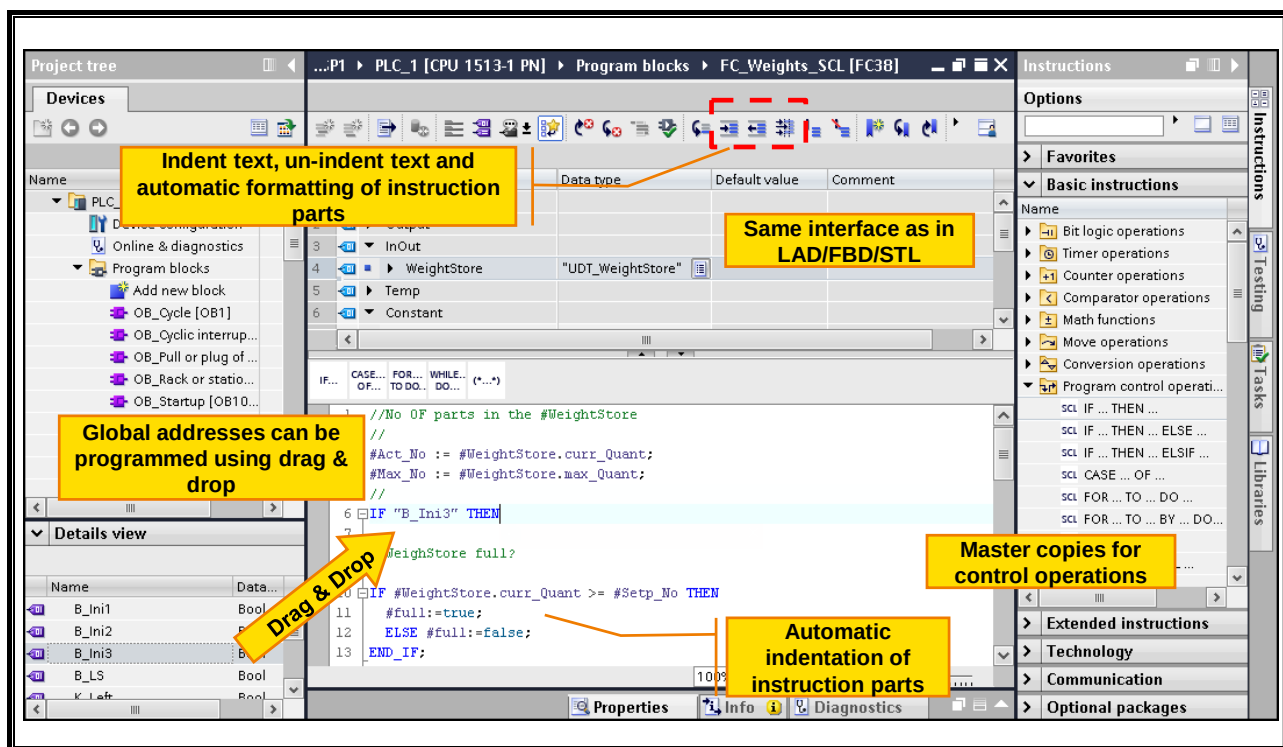
The statement section contains the instructions that are executed after a code (logic) block (OB, FB, FC) is called. These instructions are used to process data and operands.

Note

In programming instructions, the following points must be taken into consideration:

- Each instruction must be completed with a semicolon.
- All identifiers (names) used in the statement section must be declared.

11.4.1. Editing an SCL Block



Statement Section

The statement section contains the instructions that are executed after a code (logic) block (OB, FB, FC) is called. These instructions are used to process data and operands.

Subdivision

The individual instructions can basically be divided into three groups:

- Value assignments:
 - They are used to assign an expression or a value to a variable.
- Control instructions:
 - They are used to branch within a program or to repeat groups of instructions.
- Subroutine call:
 - They are used to process functions and function blocks.

Note

In programming instructions, the following points must be taken into consideration:

- Each instruction must be completed with a semicolon.
- All identifiers (names) used in the statement section must be declared.

Alignment:

- Selected texts are aligned by adding / removing spaces and tabs
- Alignment is determined by keywords (IF, FOR,...) and previous line
- With the help of the  button, selected texts can automatically be aligned.

11.4.2. Operators

Syntax: result := expression; Example: #Q_Average := (#Value1 + Value2) / 2;		
Logic operation	Description	Operator
Assignment	Assignment	:=
Parenthesis	(Expression)	(,)
Binary logic operation	Negation AND OR Exclusive-OR	NOT AND, & OR XOR
Comparison	Less than, less than or equal to, greater than, greater than or equal to, equal to, not equal to	<, <=, >, >=, =, <>
Math	Plus, Minus (sign) Addition, Subtraction Multiplication, Division Exponentiation	+,- +,- *, /, DIV, MOD **

Expressions

Expressions consist of operands, operators and round brackets (parenthesis). Within an expression the operators (e.g. +, -, *, /, etc.), that is, the active components of an expression, are linked with the passive elements, such as constants, variables and function values, in order to form a new value. An expression therefore stands for the value it represents. SCL permits the formation of standard expressions, that is, mathematical, logical and comparative expressions. Variables from data blocks, arrays, structures and CPU memory areas (inputs, outputs, etc.) can be enlisted for the formation.

Operators and Operands

Expressions consist of operators and operands. Most SCL operators link two operands (e.g. A + B) and are therefore termed *binary* operators. The others work with only one operand and are thus called *unary* operators.

The result of an expression can

- be assigned to a variable (e.g. A := B + C;)
- be used as a condition for a control statement (e.g. IF A < B DO ...)
- be used as an actual parameter for the call of a function or a function block (e.g. FB20 (Input := A + B))

11.4.3. Control Structures

There are master copies for the control structures (so-called code snippets)!

	Keyword	Functionality
Program branching	IF	Program branching with Boolean value
	CASE	Program branching with INT value
Program loops (Abort possible)	FOR	Program loop with run variable
	WHILE	Program loop with execution condition
	REPEAT	Program loop with abort condition
Loop aborts	CONTINUE	Abort current loop pass
	EXIT	Exit program loop
Block abort	RETURN	Exit the block

Control Instructions

These are used for changing the sequence in which the instructions are normally processed.

A choice from the various alternatives in the program execution can be made with conditional instructions (IF and CASE instructions).

Loop instructions (FOR, WHILE and REPEAT instructions) are used to repeatedly execute instructions.

Jump instructions (CONTINUE, EXIT and GOTO instructions) permit the sequence of processing to be interrupted and to jump to a resumption point

FB and FC Calls

According to the principle of structured programming, other function blocks (FB and SFB) and functions can also be called from an SCL block. Callable blocks are:

Other functions and function blocks that were created in SCL or in another STEP7 language (STL, LAD, etc.).

Standard functions and standard function blocks that are supplied with SCL. System functions (SFC) and system function blocks (SFB) that are available in the operating system of the CPU.

11.4.4. Direct Addressing (Examples)

Direct addressing for inputs, outputs, memory bits, I/O areas and data blocks is similar to LAD/FBD/STL!			
	Area	Notation examples	Examples
Absolute Addressing	Bit	%DBz.DBXy.x , %Iy.x	%DB5.DBX0.7 , %I0.0
	Byte	%DBz.DBB y , %MBy	%DB5.DBB2 , %QB2
	Word	%DBz.DBW y , %QWy:P	%DB5.DBW4 , %MW20
	Doubleword	%DBz.DBD y , %MDy	%DB5.DBD8 , %ID30:P
Symbolic Addressing		"<DB-Symbol>".<Variable-Name>	"Motor".Setvalue
	Only for S7-1200/1500	<Variable-Name>.%X<Bitnumber> <Variable-Name>.%B<Bytenumber> <Variable-Name>.%W<Wordnumber>	"Motor".Alarms.%X1 (Bit 1 of the variable "Alarms" in DB "Motor")

Direct Addressing

In the statement section of a block, the actions are described that are to be executed with local operands (parameters or local variables) or global variables (PLC tags or variables in a global data block). The global variables can be addressed either absolutely or symbolically.

Absolute Addressing

Absolute addressing addresses operands relative to the beginning of the operand area; e.g. %I1.0 (input bit 0 in byte 1). The absolute addressing for SCL is similar to that of the basic languages (LAD, FBD and STL) - only a percent sign must be written in front of the address so that the compiler knows that it is dealing with an absolute address. The access to global data operands is only done completely addressed for SCL. Inputs, outputs and memory bits are automatically given a symbolic name after entry (e.g. "Tag_4").



Note, that for SCL, there can be no separator (space or tabulator) between the operand notation and the operand address.

Symbolic Addressing

Symbolic addressing addresses operands and variables with a name. The name is assigned in the symbol table for global data; for local data it is assigned in the declaration section of the block. The symbolic addressing for SCL corresponds to that of the basic languages.

11.4.5. Indirect Addressing (Examples)

Indirect addressing, both for I/Q/M/Peripheral access, and for variables in data blocks!			
	Area	Instruction	Examples
Indirect Addressing	Bit	PEEK_BOOL POKE_BOOL	#OUT1 :=PEEK_BOOL(area:=#Memory_area, dbNumber:=#DB_Number, byteOffset:=#Byte_addr, bitOffset:=Bit_addr);
	BYTE	PEEK POKE	
	WORD	PEEK_WORD POKE_WORD	
	DWORD	PEEK_DWORD POKE_DWORD	POKE_WORD(area:= Memory_area, dbNumber:=#DB_Number, byteOffset:=#Byte_addr, value:=w#16#12);
	Area	PEEK_BLK POKE_BLK	
	Array element	"<DB-Symbol>".<Array-Symbol>[Index]	"Motor".Value[#Index]

Indirect Addressing

The instructions PEEK (reading) and POKE (writing) are used for indirect addressing.

Memory_area, *DB_Number*, *Byte_addr* and *Bit_addr* are constants or, at runtime, changable variables or expressions.

The following operand areas (area) can be addressed in this way:

- 16#81: Input
- 16#82: Output
- 16#83: Memory bit
- 16#84: DB
- 16#1: Peripheral input for PEEK or Peripheral output for POKE (only S7-1500)

If the memory area is not 16#84, then the DB_Number must be specified with 0.

Example Poke_BLK:

```
POKE_BLK(AREA_SRC := "Tag_Source_Area",           //Memory area Source
          DBNUMBER_SRC := "Tag_Source_DBNumber",   //DB-Number Source
          BYTEOFFSET_SRC := "Tag_Source_Byte"),     //Byte number Source
          AREA_DEST := "Tag_Destination_Area",      //Memory area Destination
          DBNUMBER_DEST := "Tag_Destination_DBNumber", //DB-Number Destination
          BYTEOFFSET_DEST := "Tag_Destination_Byte", //Byte number Destination
          COUNT := "Tag_Count");                   //Number of bytes
```

Array Elements

Array elements can be addressed with a variable Index at runtime.

11.4.6. Calling an SCL Block

Input parameters are identified with := and output parameters with =>		
	FC Call	FB Call
FBD		
STL (only S7-300 / S7-400 / S7-1500)	<pre> 1 CALL "FC_FirstWord" 2 Sentence := "DB_Test".Sentence 3 FirstWord := "DB_Test".FirstWord 4 NOP 0 </pre>	<pre> 1 CALL "FB_StringWord", "DB_FB99" 2 Sentence := "DB_Test".Sentence 3 Words := "DB_Test".Words 4 No_of_Words := "DB_Test".no_of_words 5 NOP 0 </pre>
SCL	<pre> 9 10 □ "FC_FirstWord" (Sentence := "DB_Test".Sentence, 11 FirstWord => "DB_Test".FirstWord); 12 </pre>	<pre> 1 2 □ "DB_FB99" (Sentence := "DB_Test".Sentence, 3 Words => "DB_Test".Words, 4 No_of_Words => "DB_Test".no_of_words); 5 </pre>

Calling an SCL Block

Depending on the task, SCL blocks can be created as FC or FB. According to the principle of structured programming, other function blocks (FB and SFB) and functions can also be called from an SCL block. In that way, other functions and function blocks that were created in SCL or in another STEP7 language (STL, LAD, etc.) can be called.

11.4.7. Monitoring SCL Blocks

By selecting variables, the values are displayed in a tooltip

Display of the instruction result

Line is grayed out for not executed instructions

Display of all instruction values

When the line is selected, the values are displayed in a popup

```

1 //No OF parts in the #WeightStore
2 //
3 #Act_No := #WeightStore.Act_No;
4 #Max_No := #WeightStore.Max_No;
5 //
6 //
7 //WeightStore full?
8 //
9 IF #WeightStore.Act_No >= #Setp_No THEN
10   #full:=true;  #WeightStore.Act_No = 2
11 ELSE #full:=false;
12 END_IF;
13 //
14 //
15 //Increase Act_No and store the current weight in WeightStore
16 //
17 IF #full=false AND ... THEN ... END_IF;
18 //
19 //
20 //Initialize WeightStore
21 //
22 IF #Init THEN
23   #WeightStore.Act_No:=0;
24   FOR #count := 1 TO #WeightStore.Max_No DO
25     #WeightStore.Part Weight[#count] := 0;
26   END_FOR;
27 END_IF;

```

Result	
#full	FALSE
#full	FALSE
Result	FALSE
#Init	TRUE
#WeightStore.Act_No	0
#count	1
#WeightStore.Part Weight[1]	0
#count	1

Monitoring an SCL Block

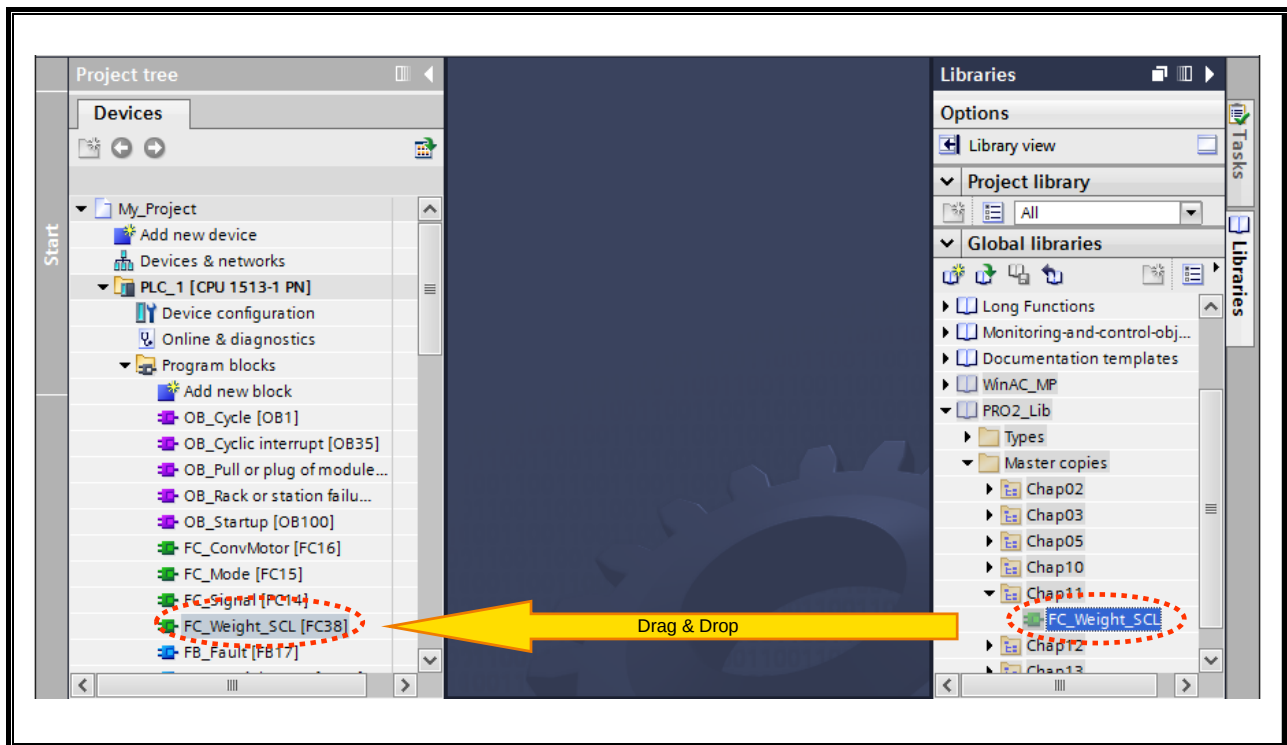
Just as in LAD/FBD/STL, SCL blocks can be monitored. All values of control instructions are displayed when these are "opened and revealed".

The values of not executed instructions are grayed out.

Features of Selected Tags (Variables):

- In the online mode, a tooltip displays the value of the tag (variable).
- The value of IN-OUT variables is shown before and after the call of the function.

11.5. Exercise 1: Copying the SCL Block from the Library



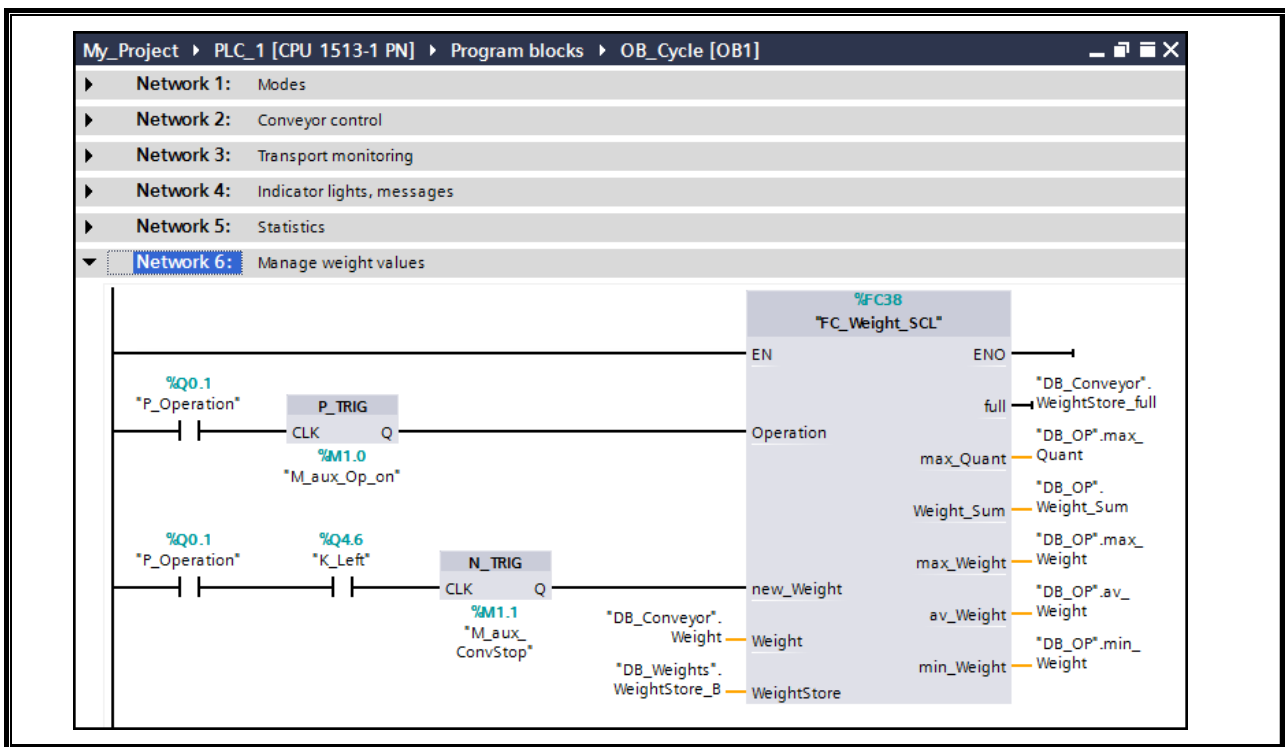
Task

For the subsequent commissioning, you are to copy the prepared block "FC_Weight_SCL" (FC38) from the "PRO2_Lib" global library into your project.

What to Do

1. In the "Global libraries", open the library <Drive>:\Archives\TIA\PRO2_Lib
2. Using drag & drop, copy the "FC_Weight_SCL" (FC38) block into your project.
3. Save your project.

11.5.1. Exercise 2: Commissioning "FC_Weight_SCL" (FC38)



Task

The new "FC_Weight_SCL" (FC38) fulfills the same function as "FB_Weights" (FB35) did previously and is now to be integrated into the entire program by you.

What to Do

1. In OB1, instead of "FB_Weights" (FB35) call the new "FC_Weight_SCL" (FC38) and parameterize it the same as you did the FB previously (see picture).
2. Download the modified S7 program into the CPU.
3. Check whether "FC_Weight_SCL" (FC38) fulfills all functions of the weight management just as "FB_Weights" (FB35) did before.
4. Save your project.

Contents

12

12.	S7-GRAPH.....	12-2
12.1.	The Conveyor Model as Production Line.....	12-3
12.2.	S7-GRAPH: Programming Sequential Function Operations	12-4
12.3.	Creating an S7-GRAPH Block	12-5
12.4.	Sequence View of an S7-GRAPH Block.....	12-6
12.5.	Interlock and Supervision.....	12-7
12.6.	Testing an S7-GRAPH Sequencer	12-8
12.7.	Exercise 1: Copying an S7-GRAPH Block from the Library	12-9
12.8.	Exercise 2: Commissioning the S7-GRAPH Block	12-10
12.9.	Additional Information	12-11
12.9.1.	Additional Exercise: Returning Rejected Parts.....	12-12

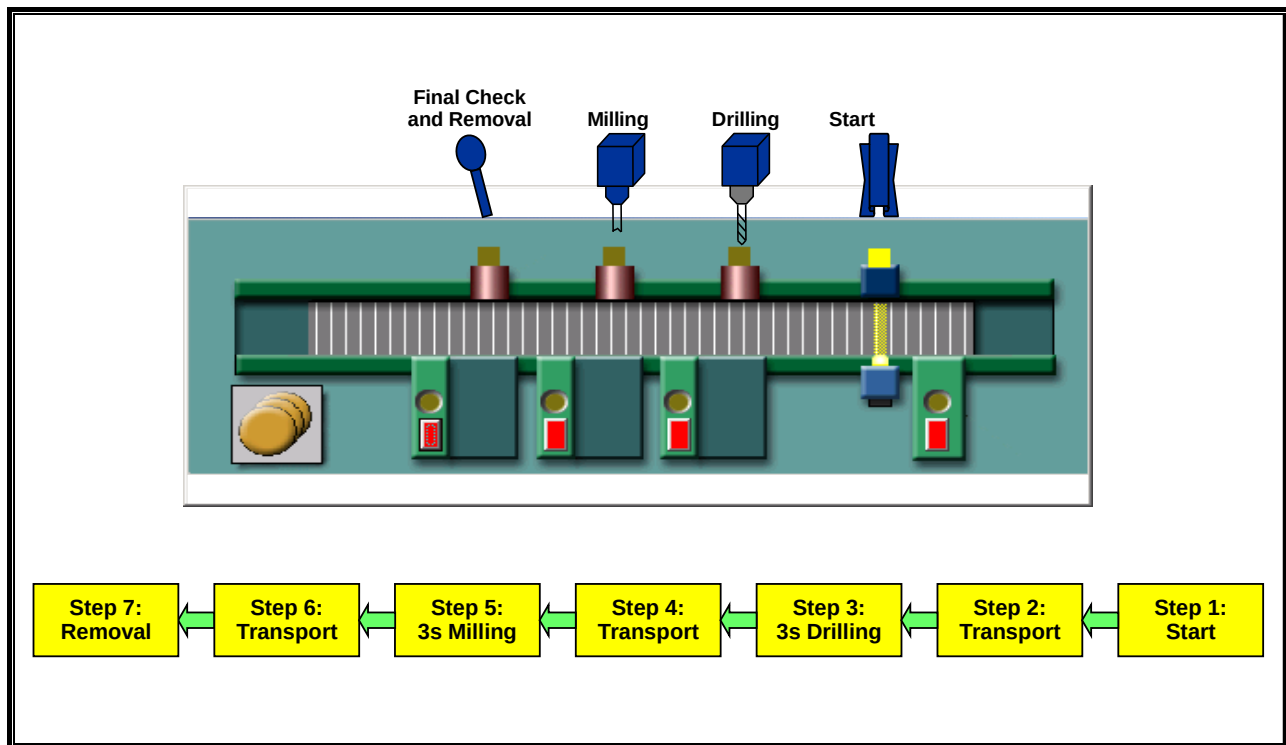
12. S7-GRAPH

At the end of the chapter the participant will ...

- ... be familiar with the application areas of S7-GRAPH
- ... be familiar with the functional principle of S7-GRAPH
- ... be able to commission a given S7-GRAPH block



12.1. The Conveyor Model as Production Line



Task Description

When "P_Operation" (Q0.1) is switched on, the conveyor model is no longer to transport parts from the light barrier bay to the next free bay. Instead, the conveyor model is to transport parts in sequence to the Bays 3, 2 and 1. All other, prevailing functions (parts statistics, saving the part weights in a weight storage, indicator light control) are to remain unchanged.

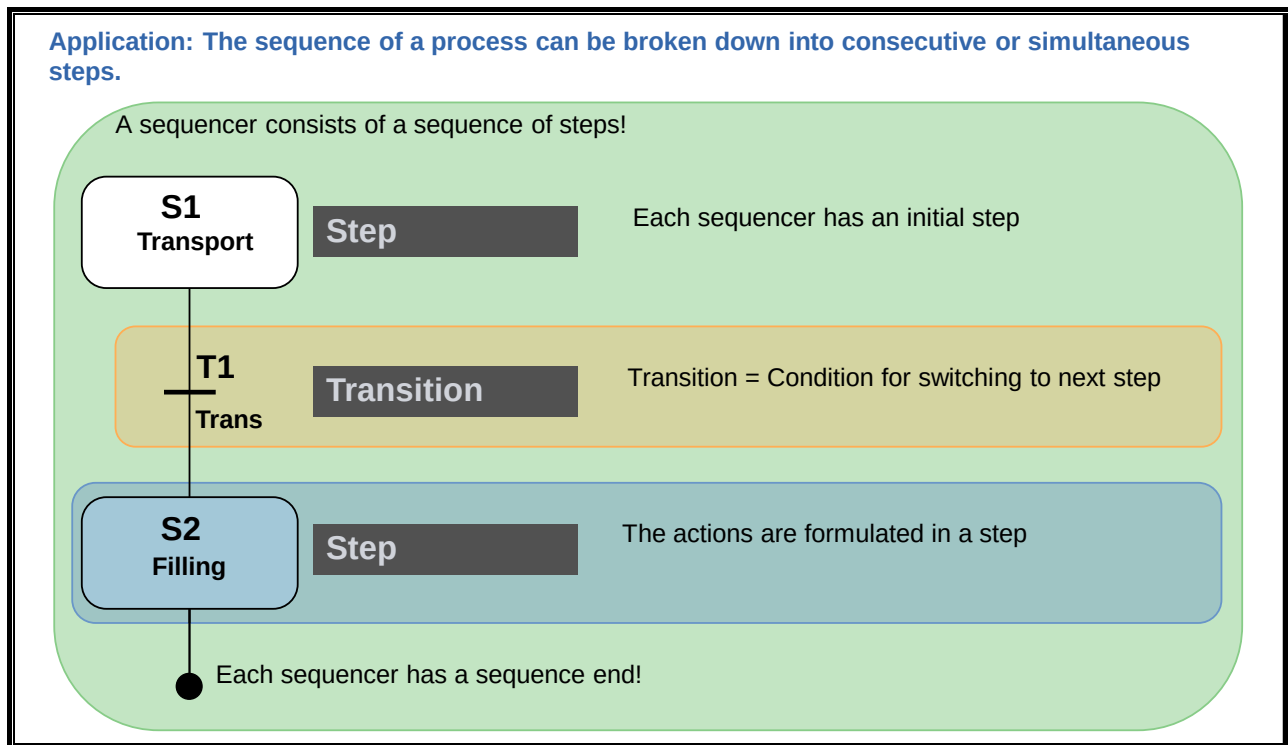
Parts Machining

Each piece is mounted onto a tool holder at the **Start** bay (light barrier bay). After pressing the associated bay pushbutton, the part is transported to the **Drilling** bay (Bay 3) and drilled for 3 seconds, then transported to the **Milling** bay (Bay 2) and milled for 3 seconds and finally transported to the **Removal** bay (Bay 1) where it is removed from the production line.

Sequential Control System

The process can be broken down into individual steps which are executed in a certain, timely sequence. Tasks of this kind are programmed as *sequential control systems*. The term sequential control system therefore does not stand for a particular PLC operating mode but for a kind of structuring and programming of the user program. Typical application examples where it is advantageous to program them as sequential control systems are automatic assembly machines, car washes, packaging machines or traffic light systems – in other words, those sequences that traditionally were solved with sequence processors.

12.2. S7-GRAPH: Programming Sequential Function Operations



S7-GRAPH

The S7-GRAPH programming language is used to control sequential sequences. The process is split into single steps with their own function scope. The sequence is represented graphically and can be documented with pictures and text.

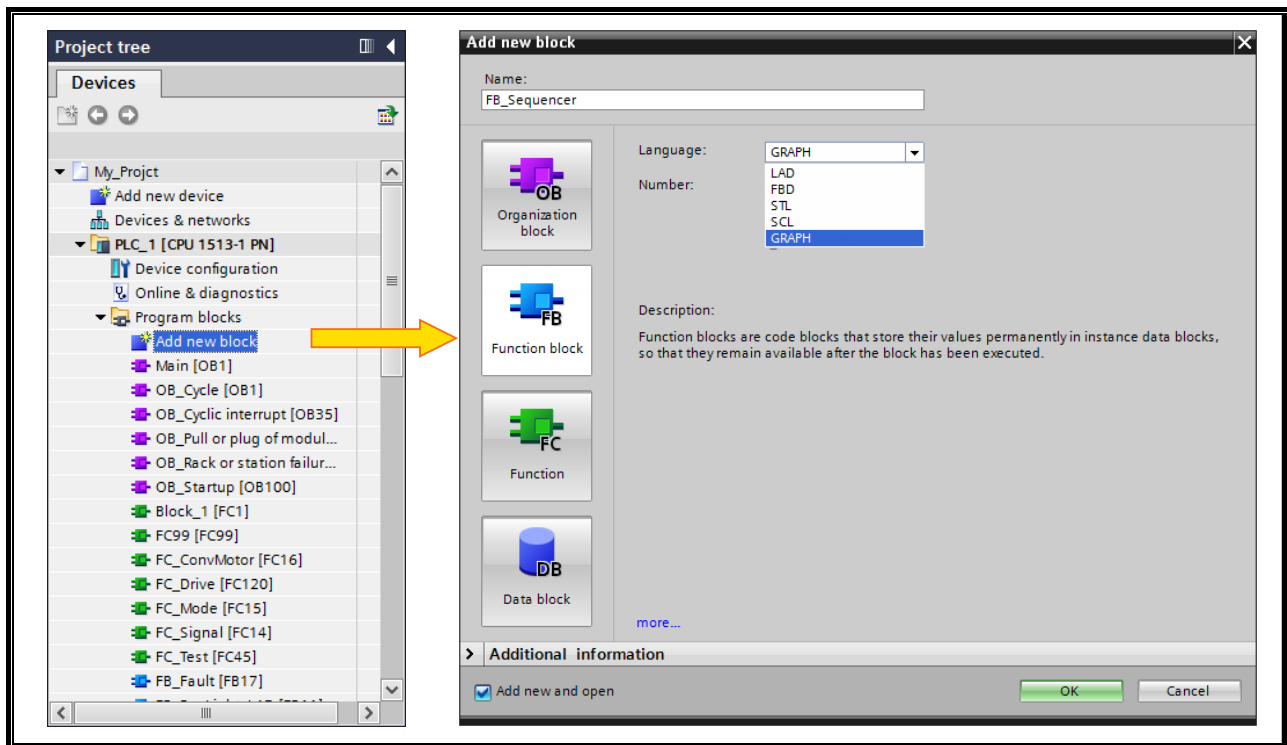
The actions to be performed are defined in the single steps; the conditions for moving on to the next step are defined by transitions. The definitions of these, as well as interlocks and supervisions are written in a subset of the STEP 7 programming languages LAD und FBD.

Sequencer

Steps and transitions form a sequencer. The sequencer is stored in an FB. An instance DB, that contains the sequencer data, is assigned to this FB. At least three blocks are necessary for an executable program:

- the FB, that contains the sequencer(s)
- an instance DB with the sequencer data
- an organization block, a function block or a function containing the FB call. The parameters and the number of the instance DB are passed in the FB call.

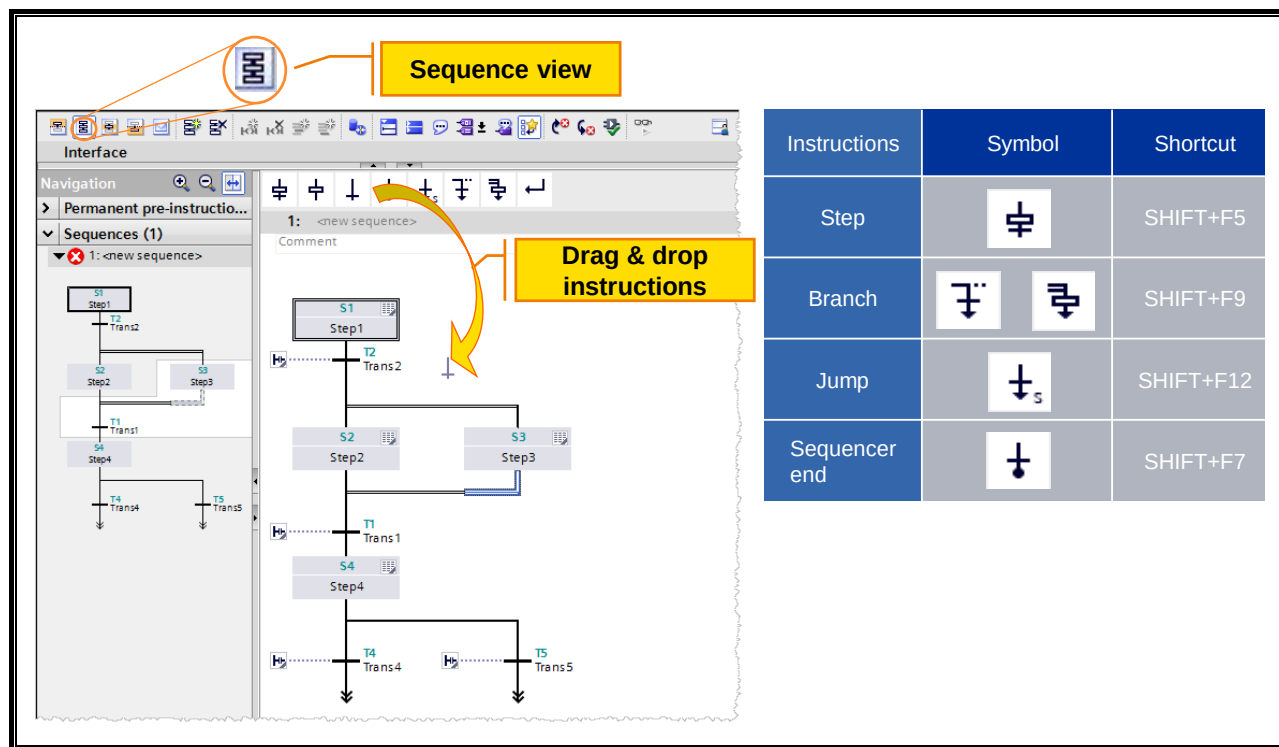
12.3. Creating an S7-GRAPH Block



Creating an S7-GRAPH Block

S7-GRAPH blocks are created exactly like LAD/FBD/STL blocks. However, you must make sure that S7-GRAPH blocks are of the FB block type.

12.4. Sequence View of an S7-GRAPH Block



Instructions	Symbol	Shortcut
Step		SHIFT+F5
Branch		SHIFT+F9
Jump		SHIFT+F12
Sequencer end		SHIFT+F7

Overview

In creating a sequencer, you are supported by functions for graphic programming. You can easily create a sequencer's structure without programming knowledge by arranging the S7-GRAPH structure elements in such a way that a visual representation of the sequencer results.

Structure of a Sequencer

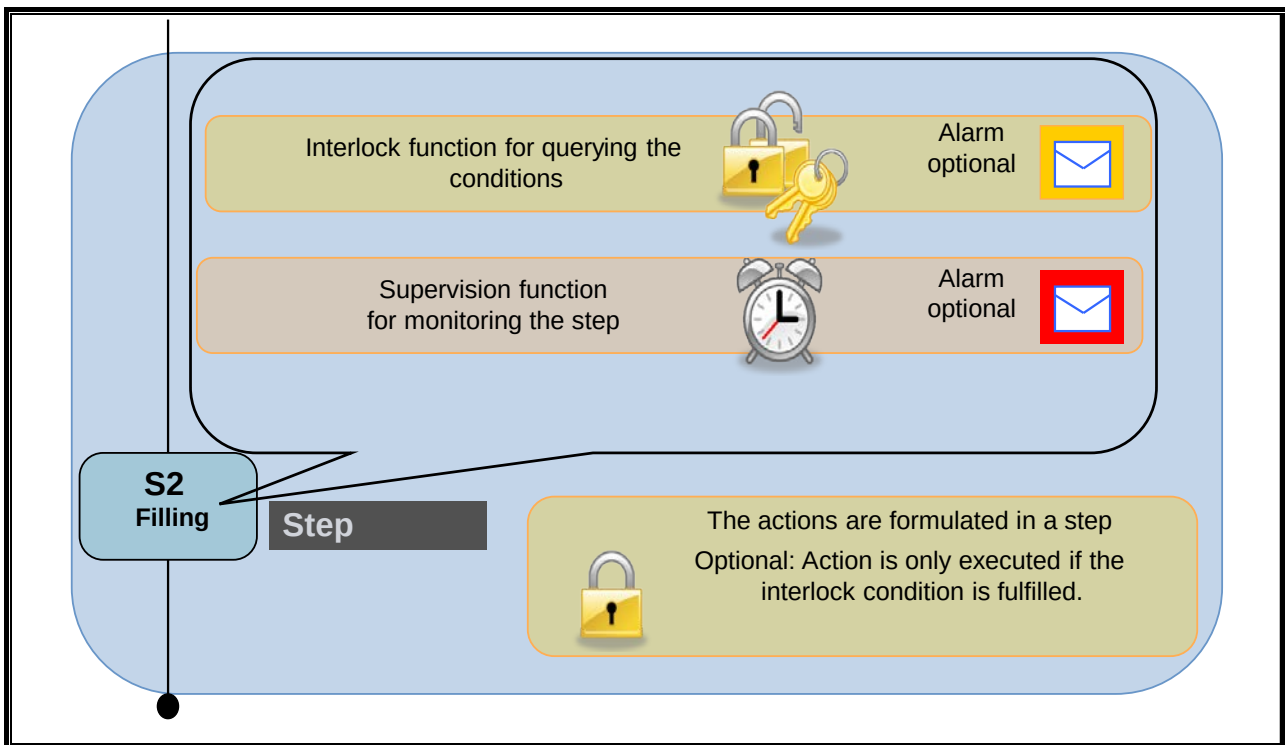
A sequencer consists of a sequence of steps and transitions. This sequence can be linear or branched.

- In the steps, the commands to the system are formulated.
- The transitions contain the conditions for the switching from one step to the next.

The following symbols show the elements that can make up a sequencer. You can select these symbols from the toolbar.

Symbol	Name
	Step and transition
	Step
	Transition
	Sequence end
	Jump
	Open alternative branch
	Open simultaneous branch
	Close branch

12.5. Interlock and Supervision



Interlock

An interlock is a programmable condition within a step. If the condition is fulfilled (true), this is the good case: There is no fault. A protective guard is closed, for example. If you have not programmed an interlock in a step, this is handled as a good case. Then, an action that is linked with the interlock is executed. The move to the next step occurs independent of the interlock. When a step is deactivated, an existing interlock is automatically canceled.

Supervision

A supervision is a programmable condition within a step. If the condition is not fulfilled (false), this is the good case. A container is emptied, for example. If a fault has occurred, the bad case has occurred, with which the condition "Container empty" is fulfilled. A fulfilled supervision leads to a fault alarm. You can define the properties and contents of alarms in the "Alarms" pane in the area navigation of the programming window.

The move to the next step only occurs if the supervision error no longer exists and the following transition is fulfilled. If a step is deactivated, the existing supervision error is automatically canceled. With that, a step that is not active can also not be faulty.

12.6. Testing an S7-GRAPH Sequencer

Overview in the navigation

Actions

Logic of the transition

Task card "Testing"

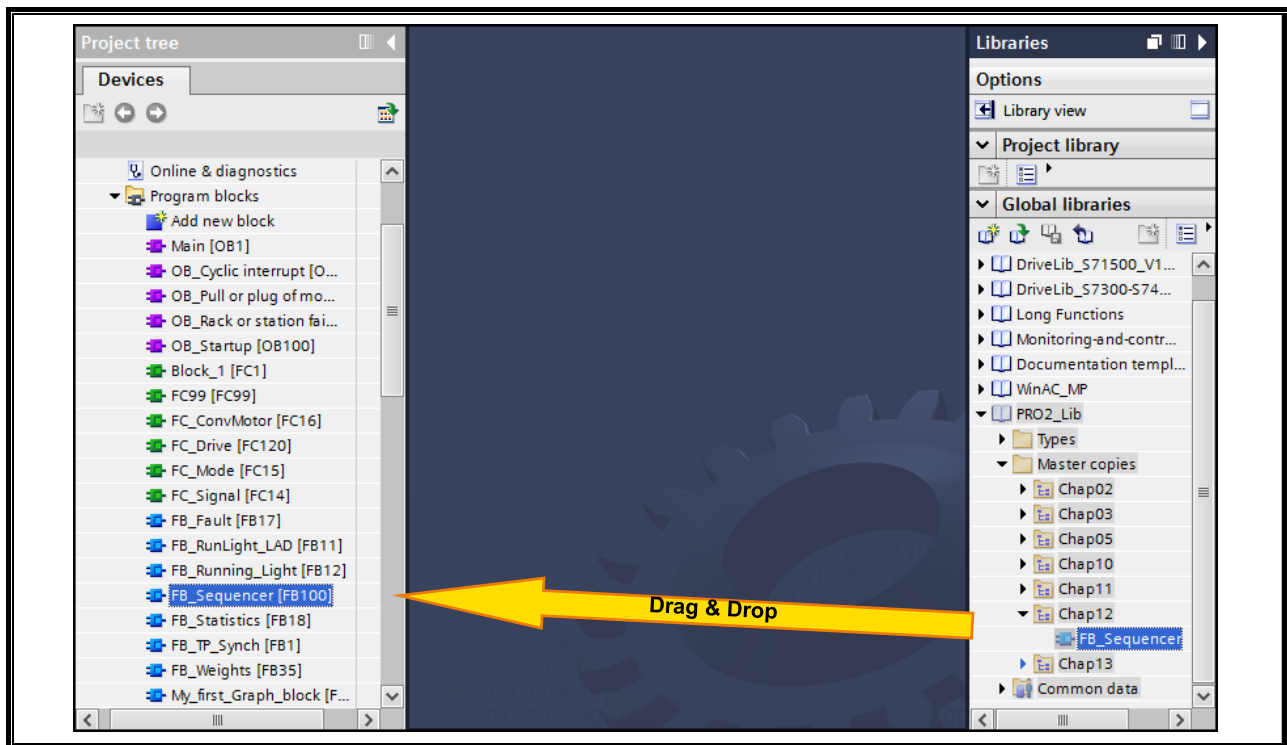
Track active step: The active step is automatically displayed

Additional Test settings

Test Functions

- Display active steps or faulty steps
- Status display and Modify variable
- Switch between the operating modes: Manual, Automatic and Jog

12.7. Exercise 1: Copying an S7-GRAPH Block from the Library



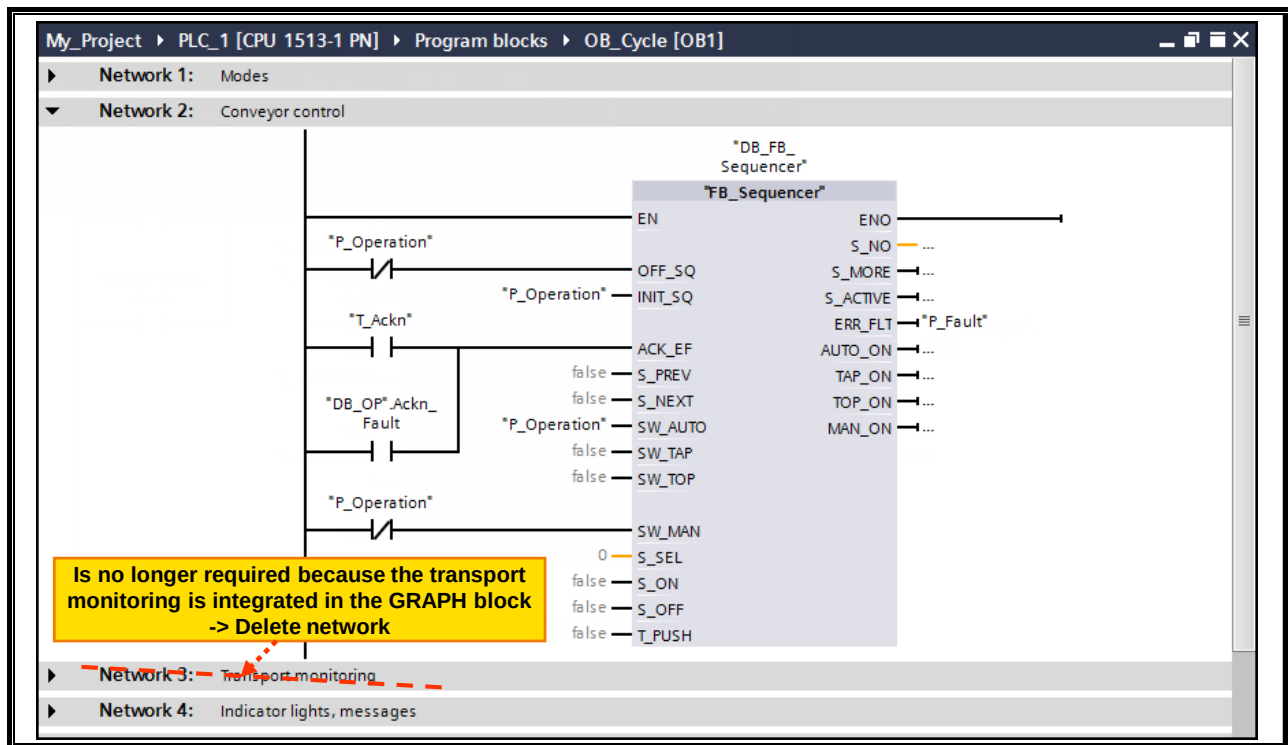
Task

You are to copy the prepared block "FB_Sequencer" (FB100) from the global library "PRO2_Lib" into your project.

What to Do

1. In the "Global Libraries", open the library <Drive>:\Courses\TIA\PRO2_Lib
2. Using drag & drop, copy the prepared block "FB_Sequencer" (FB100) into your project (see picture).

12.8. Exercise 2: Commissioning the S7-GRAPH Block



Task Description

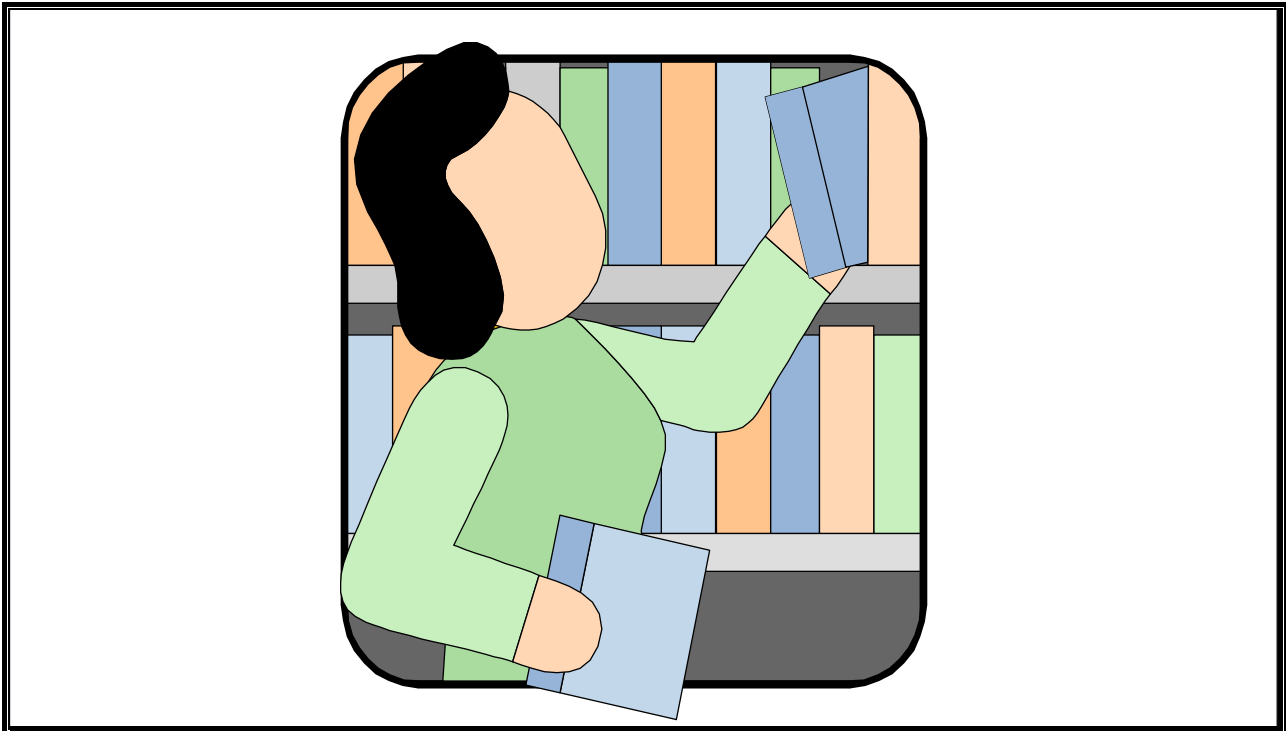
When "P_Operation" (Q0.1) is switched on, the conveyor model is no longer to transport the parts from the light barrier bay to the individual bays. Instead, it is to machine parts as described at the beginning of this chapter. The other, prevailing functions are to remain unchanged.

The machining sequence is already programmed in the "FB_Sequencer" (FB100) block that you have already copied from the library into your project. Now you are to integrate the block into your entire program.

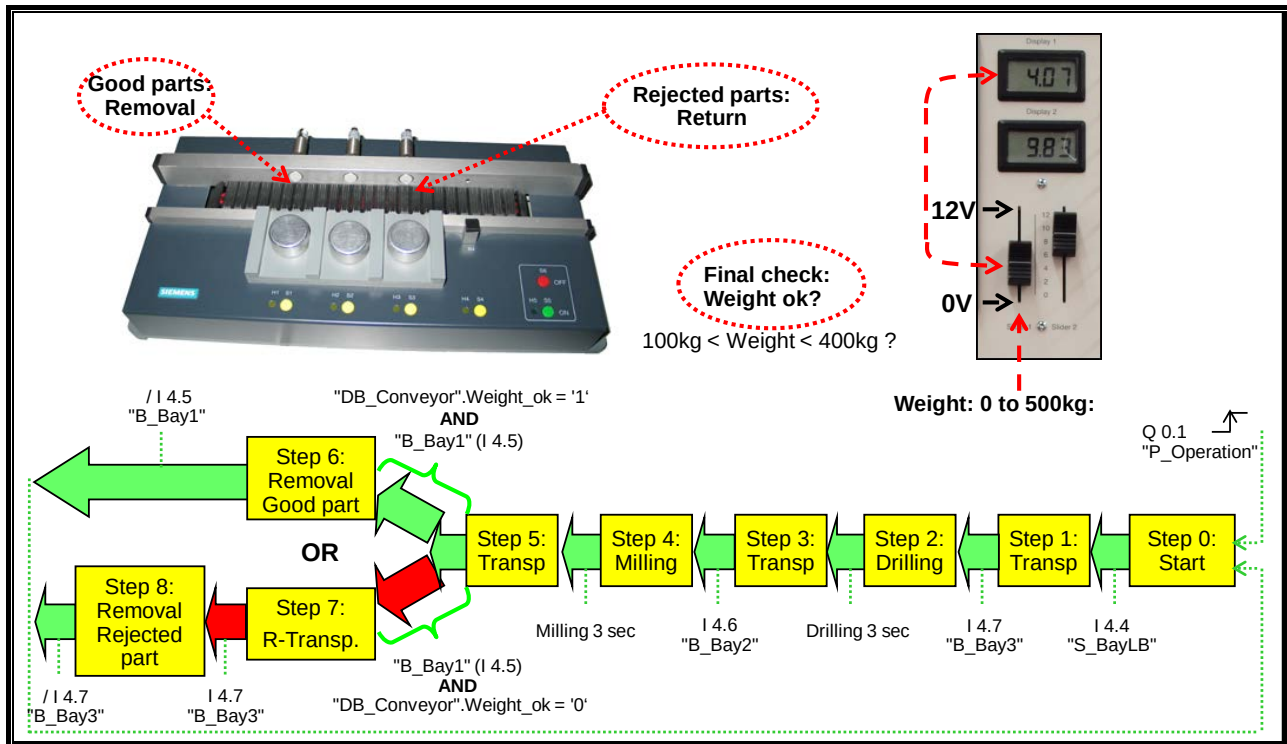
What to Do

1. Delete the call of "FC_ConvMotor" (FC16).
2. Program the call of "FB_Sequencer" (FB100) and pass the data block "DB_FB_Sequencer" (DB101) as the instance data block and assign the parameters with the operands shown in the picture.
3. In OB1, delete the network with the call of "FC_Fault" (FC17) since the monitoring of the transport sequences is integrated in "FB_Sequencer" (FB100).
4. Download all blocks into the CPU and check how the program functions and, if necessary, change further blocks in order to retain a logical function (weight control, counting parts, etc.).
5. Save your project.

12.9. Additional Information



12.9.1. Additional Exercise: Returning Rejected Parts



Function Up Until Now

After **Milling** at Bay 2, the processed part is transported to Bay 1 for **Removal**, where it is weighed and removed from the production line – up until now regardless of the measured weight or the weight set on the simulator potentiometer.

Task

The function of the current program is to be expanded as follows:

Rejected parts, whose weight is less than 100kg or greater than 400kg, must be transported back from Bay 1 to Bay 3 or to the proximity switch "B_Bay3" (I 4.7) for scrapping and removed there. Good parts with a valid weight remain at Bay 1 "B_Bay1" for **Removal**.

What to Do

Expand "FB_Sequencer" (FB100) with the additional step "R-Transp." (return transport). This step is to be executed as an alternative step to the already existing step "Removal" when the part weight does not lie in the required range (see picture).

Contents

13.	Integrating and Commissioning a Drive with Startdrive	13-2
13.1.	Task Description: G120 as an Additional Conveyor Drive.....	13-3
13.2.	Communication Standard PROFIdrive	13-4
13.2.1.	CPU - Drive Communication: CPU - G120	13-5
13.2.2.	Standard Telegrams	13-6
13.2.3.	Structure of the Control Word (CDS0 - ControlDataSet for Fieldbus Control)	13-7
13.2.4.	Structure of the Status Word.....	13-8
13.2.5.	Setpoint / Actual Value → Speed Values	13-9
13.3.	Inserting a Drive into the Project.....	13-10
13.3.1.	Networking a Drive.....	13-11
13.3.2.	Parameterizing the Module Address and Module Name	13-12
13.3.3.	Configuring a Power Unit	13-13
13.3.4.	Parameterizing the Process Data Area (PZD).....	13-14
13.3.5.	Assigning the Device Name ONLINE (Module Initialization)	13-15
13.4.	Parameterizing the Drive: with the "Commissioning Wizard"	13-16
13.5.	Online Commissioning: Activating / Deactivating the Control Panel	13-17
13.5.1.	Operating the Control Panel	13-18
13.5.2.	Monitoring Control and Status Word(s) Online	13-19
13.5.3.	Monitoring Active Messages Online	13-19
13.5.4.	Exercise 1: Reading-out the Firmware Version of the Drive.....	13-20
13.5.5.	Exercise 2: Restoring the Factory Settings.....	13-21
13.5.6.	Exercise 3: Inserting and Networking the Drive in the Offline Project	13-22
13.5.7.	Exercise 4: Configuring and Parameterizing the Drive	13-23
13.5.8.	Exercise 5: Assigning the PROFINET Device Name Online	13-24
13.5.9.	Exercise 6: Parameterizing the Drive OFFLINE with the Commissioning Wizard.....	13-25
13.5.10.	Exercise 7: Downloading the Parameterization into the Drive.....	13-30
13.5.11.	Exercise 8: Operating the Drive via the Control Panel	13-32
13.5.12.	Exercise 9: Commissioning a Program Expansion.....	13-33
13.6.	Additional Information	13-34
13.6.1.	Changing Parameters in the Inverter.....	13-35
13.6.2.	G120 Reset to Factory Settings via BOP-2	13-36

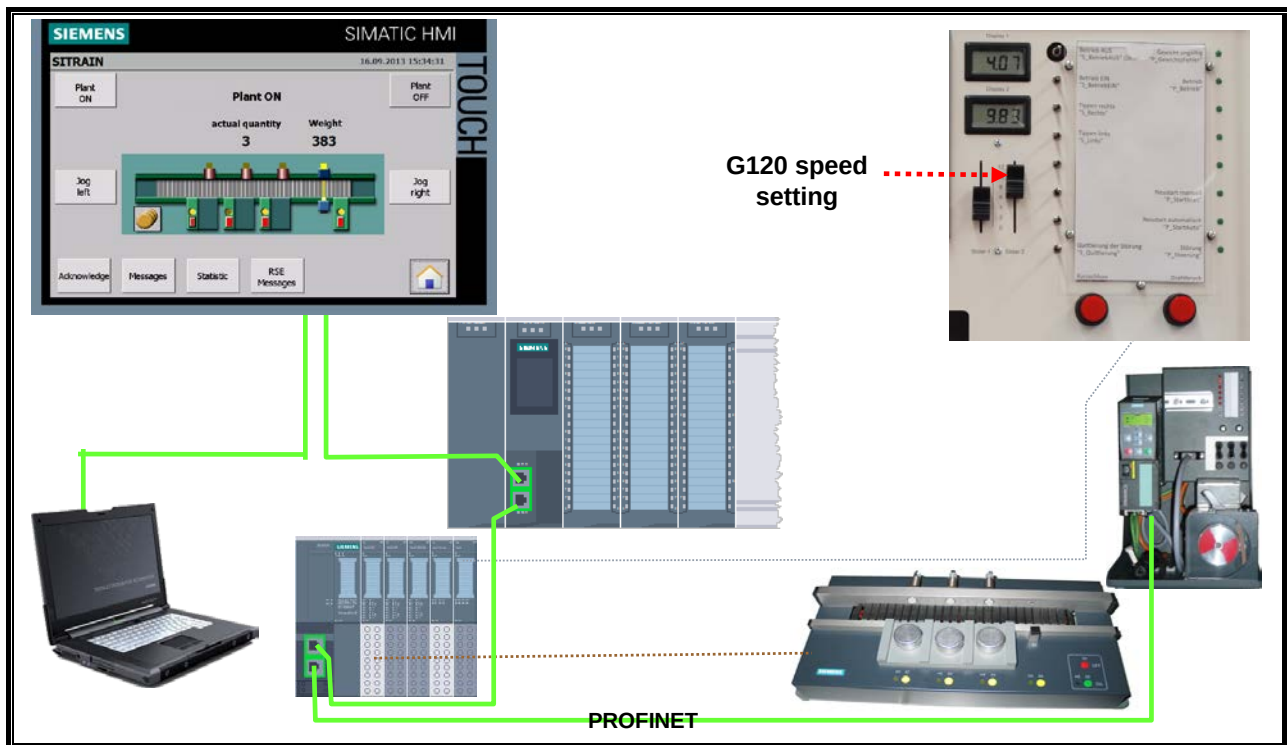
13. Integrating and Commissioning a Drive with Startdrive

At the end of the chapter the participant will ...



- ... be able to parameterize and test a drive with Startdrive
- ... be able to integrate a drive in the device configuration
- ... be able to reset (restore) the inverter to factory settings
- ... be able to set basic parameters via Startdrive
- ... be able to control the drive via a PLC

13.1. Task Description: G120 as an Additional Conveyor Drive



Task Description

Imagine that the drive is being used as an additional drive for the conveyor model.

The drive is to be controlled as follows:

- When "P_Operation" (Q0.1) is switched on (activated operation), the G120 drive is automatically also switched on and off parallel to the conveyor motor.
- The speed of the G120 can be set via the right slide control (potentiometer) on the simulator.

13.2. Communication Standard PROFIdrive



The diagram illustrates the integration of a SIMATIC S7-1500 drive unit with industrial communication systems. On the left is a physical drive unit. In the center, the PROFIBUS and PROFINET logos are shown. A yellow arrow labeled 'PROFIdrive' points from these logos to the PI (PROFIBUS & PROFINET International) logo on the right.

- Consistent industrial communication
- Comprehensive range of applications
- PROFIdrive is the standard profile for drives technology in conjunction with the PROFIBUS and PROFINET communication systems
- Proven method for the easy and integrated connection of drives and controllers of different manufacturers

Communication via PROFIBUS and PROFINET

PROFIBUS and PROFINET are the solutions which provide absolute consistency and are highly application-oriented. The main reason that PROFIBUS and PROFINET stand out from other industrial communication systems is because they span such an extraordinary breadth of applications.

One of the most important applications in industrial automation is drives technology.

Open technologies need, for their maintenance, ongoing development and market penetration, a company-independent institution that can serve as a working platform. For this purpose, the PROFIBUS Nutzerorganisation e.V. (PNO) [PROFIBUS User Organization] was founded in 1989 as a non-profit organization representing the interests of manufacturers, users and institutes (world's largest interest association in the field of industrial communication).

PNO is a member of the international umbrella organization PI (PROFIBUS & PROFINET International) founded in 1995.

PROFIdrive

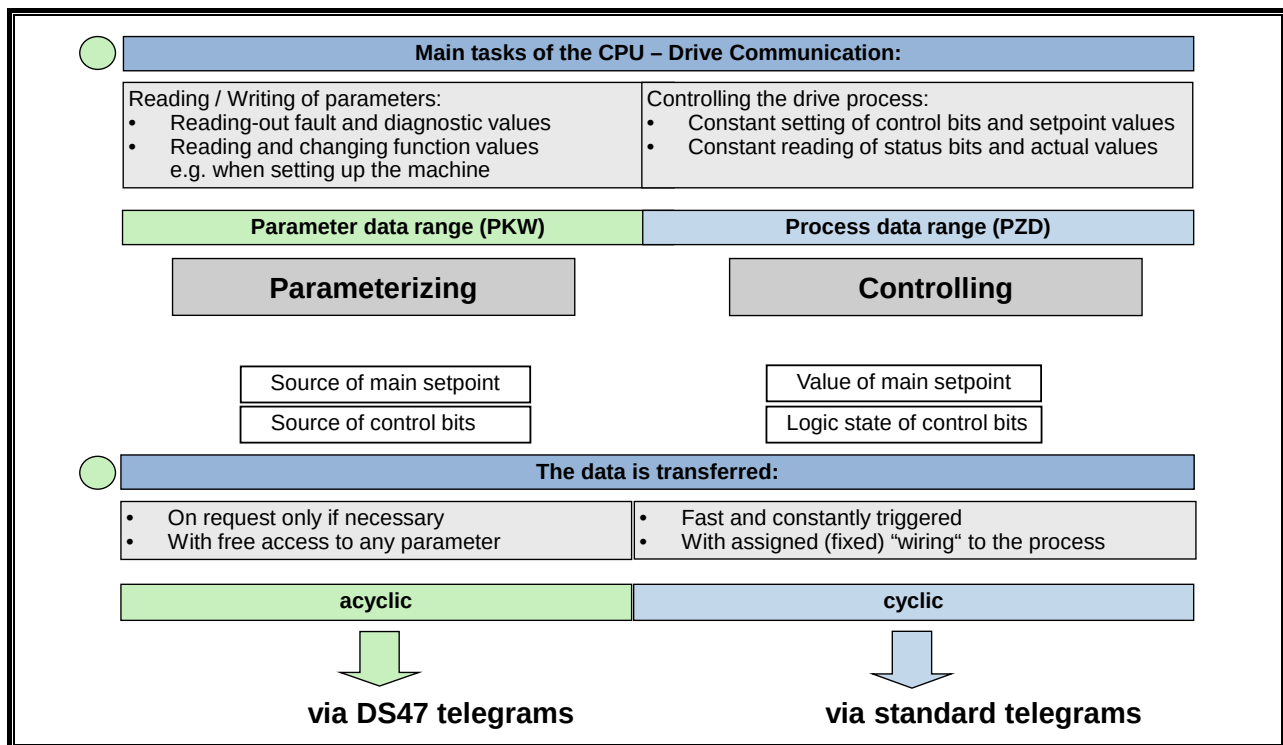
PROFIdrive is the standard profile for drives technology in conjunction with the communication systems PROFIBUS and PROFINET.

→ open "application profile" for connecting drives and controllers of different manufacturers via communication systems



The control word and status word of the G120 comply with the specifications of PROFIdrive profile Version 2.0 or Version 3.0 for the "speed control" operating mode.

13.2.1. CPU - Drive Communication: CPU - G120



Process Control

Fast data transmission of short data telegrams to all participating stations with the maximum speed available on the bus whereby all participants can receive different data. In drive systems these are typically setpoints, control commands, status replies and actual values (measured values).

Operating Control

In addition to this permanently available data, there is data that is only needed in particular cases. It would therefore be senseless to permanently put load on the bus if this data is only needed once per hour or per day, for example, when starting up the machine (Class 1 Master). Another reason for an expanded communication need could be the commissioning, optimization or diagnosis of machine components from a central location. In this case, (such as a fault) a detailed access of an Engineering Tool to the system components or the drive is enabled. Since, as a rule, only one affected device/component is directly addressed, this window is only made available once per bus cycle for one bus station and not simultaneously for every station.

Cyclic Data Exchange:

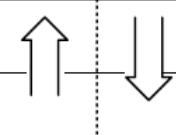
This is used to fulfill the demand for fast, permanent data exchange.

Acyclic Data Exchange:

The acyclic exchange of entire data sets (for example, DS47, DS100) between master and slave occurs according to standardized procedures which are implemented with FBs from the libraries (for example, Drive ES SIMATIC)..

The initiation of a data transmission for the acyclic communication of a Class 1 Master with a slave is always triggered by the user program. That is, the user program decides whether a data exchange with a slave is necessary or not based on further conditions. Only when there is a need, does a data transmission request get signaled to the DP master which then executes it.

13.2.2. Standard Telegrams

Telegram	1		20		352		354		999	
PZD1	STW1	ZSW1	STW1	ZSW1	STW1	ZSW1	STW1	ZSW1	STW1 <4>	ZSW1 <4>
PZD2	NSOLL_A	NIST_A	NSOLL_A	NIST_A_GL	NSOLL_A	NIST_A_GL	NSOLL_A	NIST_A_GL	Receive telegram length freely selectable via central PROFIdrive configuration in the master	Transmit telegram length freely selectable via central PROFIdrive configuration in the master
PZD3				IAIST_GL	<3>	IAIST_GL	<3>	IAIST_GL		
PZD4				MIST_GL	<3>	MIST_GL	<3>	MIST_GL		
PZD5				PIST_GL	<3>	WARN_CODE	<3>	WARN_CODE		
PZD6				<2>	<3>	FAULT_CODE	<3>	FAULT_CODE		
PZD7										
PZD8										
PZD9										
PZD10										
PZD11										
PZD12										

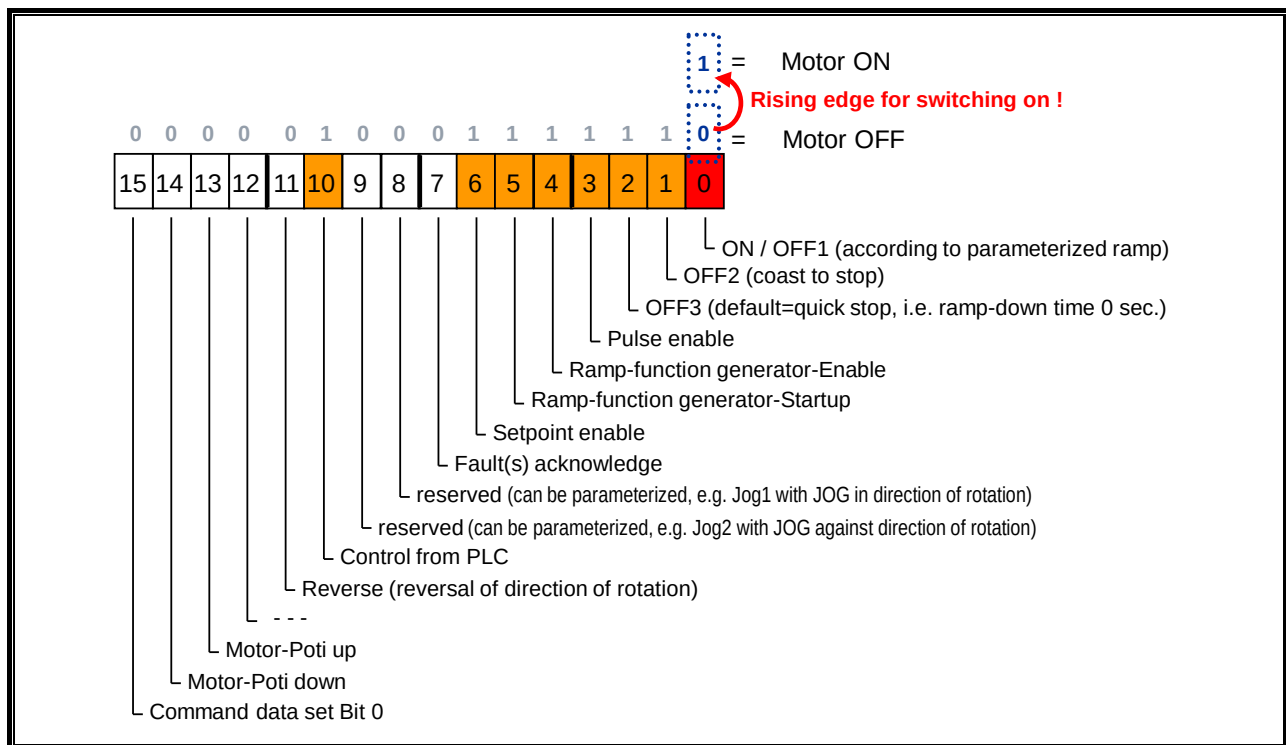
Telegrams 1, 20	Manufacturer-independent Standard Tel.	automatic configuration in drive
Telegrams 352 to 391	Siemens-specific Standard Telegrams	automatic configuration in drive
Telegram 999	Free telegram	manual configuration required

Master-Slave Communication via Standard Telegram

The best way to implement a cyclic data exchange between Controller [Master] (CPU) and PN-Device [Slave] (G120) is using a standard telegram, which typically is also used for simple speed controls. Between the Master and Slave, two input and output words with the following contents are exchanged:

- Output word 1: Control word
- Output word 2: Main setpoint value
- Input word 1: Status word
- Input word 2: Main actual value

13.2.3. Structure of the Control Word (CDS0 - ControlDataSet for Fieldbus Control)



The inverter can be controlled in various ways. The G120 can either be operated via the field bus or via the terminal strip.

This is controlled through a relevant Control Data Set (CDS).

default for G120 = two CDS:

- Control data set CDS0: Control of the inverter via field bus
- Control data set CDS1: Control of the inverter via terminal strip

Control Word

The control word (Bit 0 to 10) complies with the PROFIdrive profile standard. Bits 11 to 15 are inverter-specific.



For reasons of safety in the case of a wire break, the inverter is always switched off when the relevant OFF bit has signal 0.

ON/OFF1

A rising edge of this bit is generally required for switching on the inverter, and bits OFF2 and OFF3 must have status 1.

When it is switched off using this bit, the motor is braked by the ramp down of the ramp-function generator (Parameter 1121) and the inverter then switches itself off.

OFF2

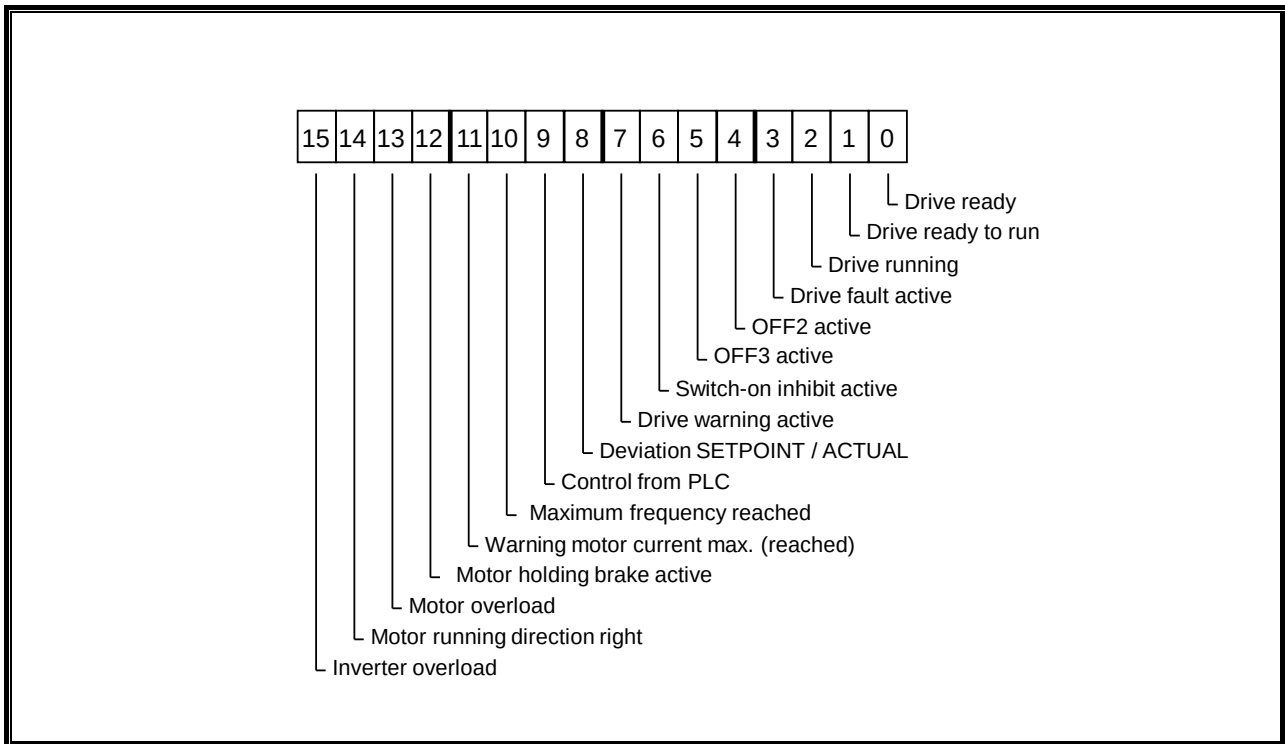
The inverter switches itself off immediately and as a result, the motor coasts down without braking (coasts to a standstill). To switch the inverter on again, this bit must be set to 1 again and a rising edge is necessary at ON/OFF1.

OFF3

The motor is braked by the OFF3 ramp down and the inverter remains on. This function is often used as an EMERGENCY STOP.

→ default for OFF3 = 0 seconds (in case of larger motors, this cannot always be achieved)

13.2.4. Structure of the Status Word



Status Word

The status word (Bit 0 to 10) complies with the PROFIdrive profile standard.

Bits 11 to 15 are inverter-specific.

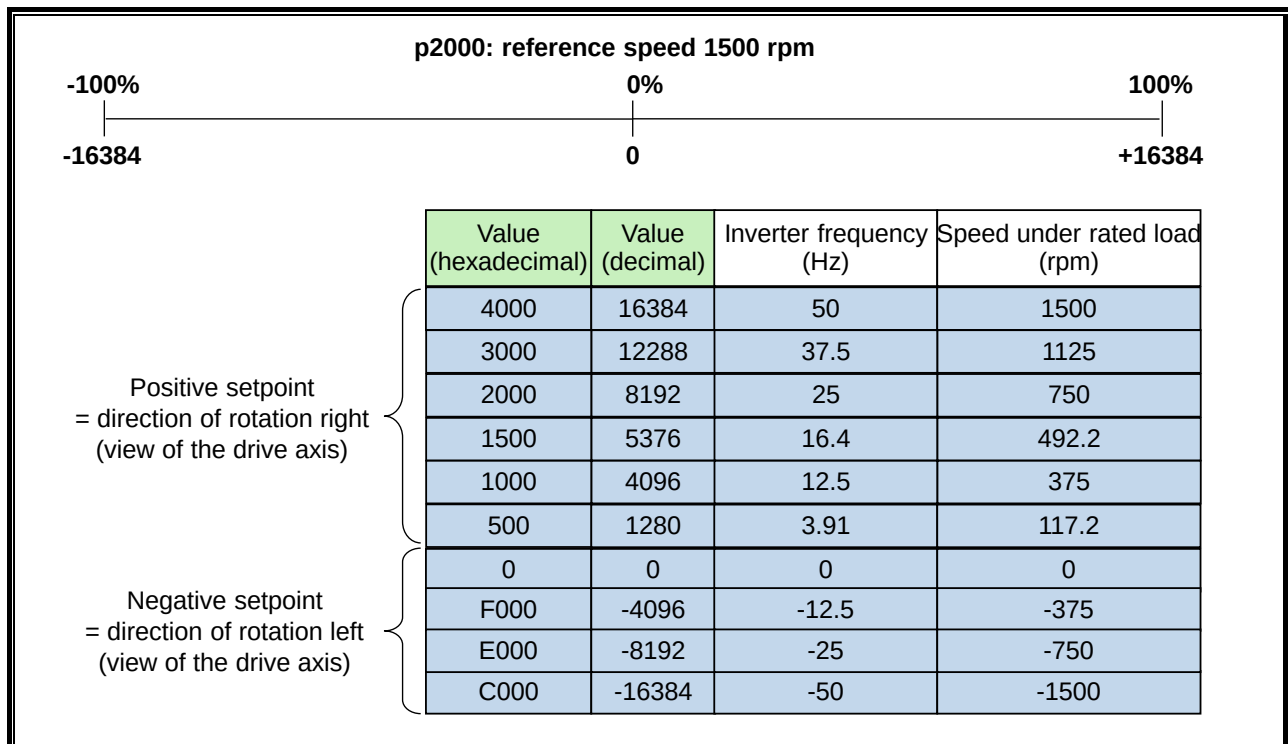
Switch-on Inhibit

This status is reached following the error elimination and acknowledgement of a drive fault. A subsequent switch on again is only possible with ON/OFF1 "0" → "1".

Warning, Message

Messages (bit 13, bit 15) and Warnings (bit 11)

13.2.5. Setpoint / Actual Value → Speed Values

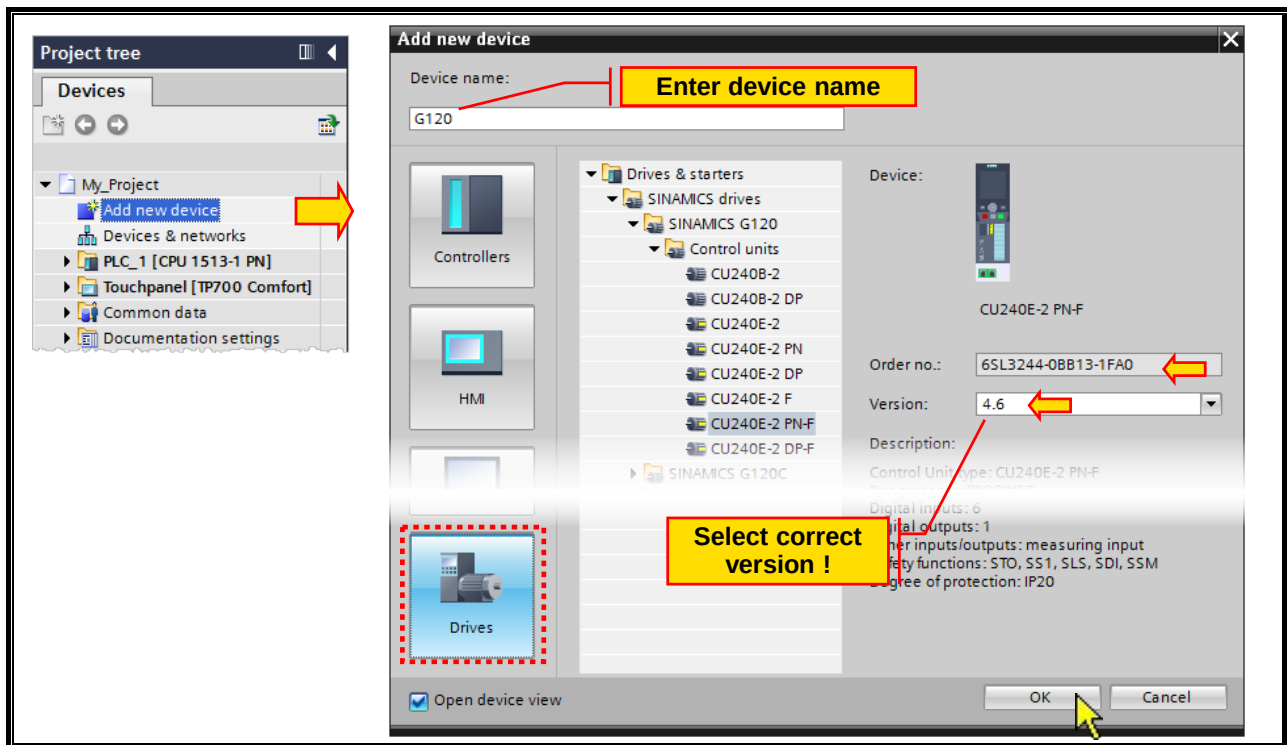


Scaling the Speed Values

Via the parameter p2000, a reference speed is defined which all setpoint and actual values use as a reference.

The range of -100% to 100% of the setpoint or actual value, in turn, is scaled to the range 16384 (CH00_{HEX}) to +16384 (4000_{HEX}).

13.3. Inserting a Drive into the Project

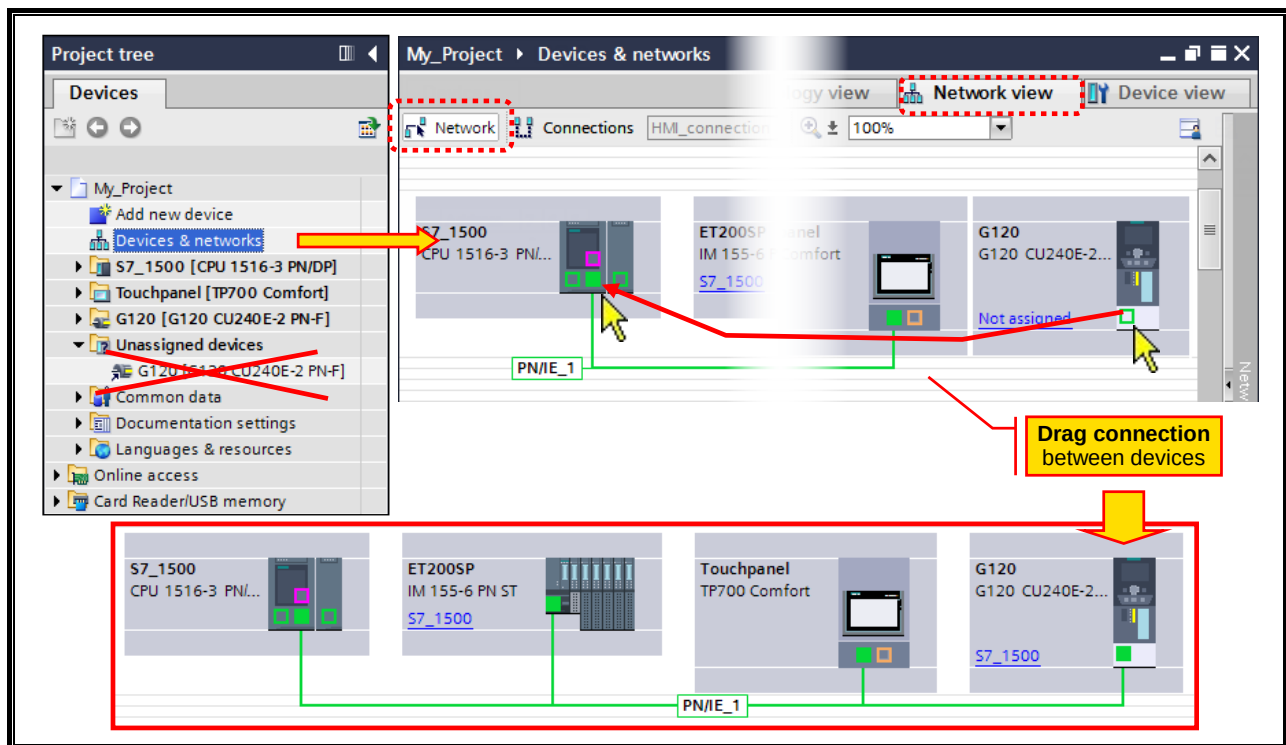


"Startdrive" Option Package

If the "Startdrive" option package is installed in TIA Portal, drives can be added, configured and assigned parameters as a new device.

Otherwise, a drive can be inserted in the project via GSD file as a distributed I/O station using the Hardware catalog. However, then only the communication with the CPU can be configured in the TIA Portal, no hardware configuration, no parameter assignment.

13.3.1. Networking a Drive



Editor

"Devices & networks" → "Network view"

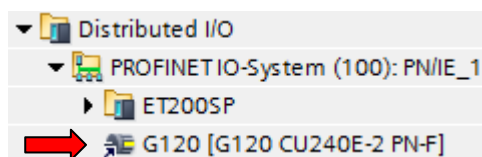
Networking

Click on the interface of the one module, hold the left mouse button and drag onto the interface of the other module.

→ here in the picture: PN interface of the G120 dragged onto PN interface of the CPU

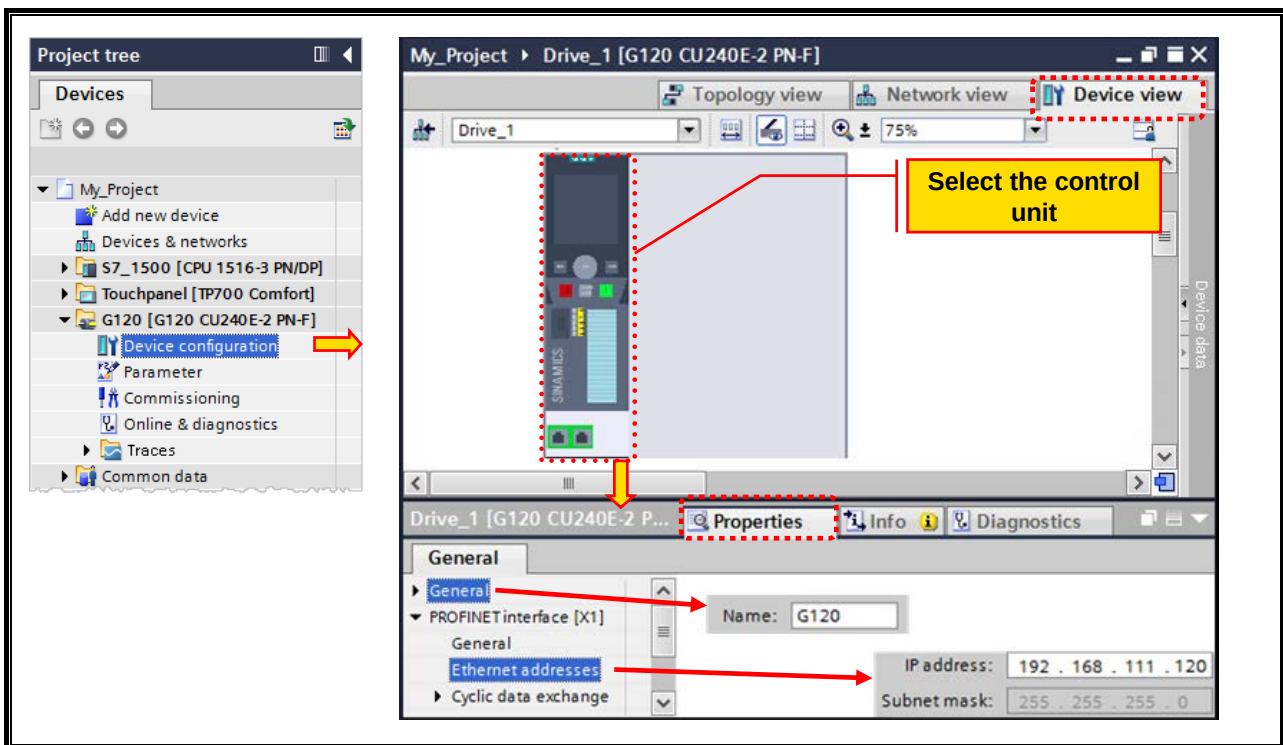
Not Assigned Devices

This section then disappears since the device is now assigned to the CPU.



The drive now also appears at the CPU.

13.3.2. Parameterizing the Module Address and Module Name



Name of the Module

The name is the assignment CPU ↔ module.

Through this, the PROFINET-IO is identified by the PROFINET-Controller.

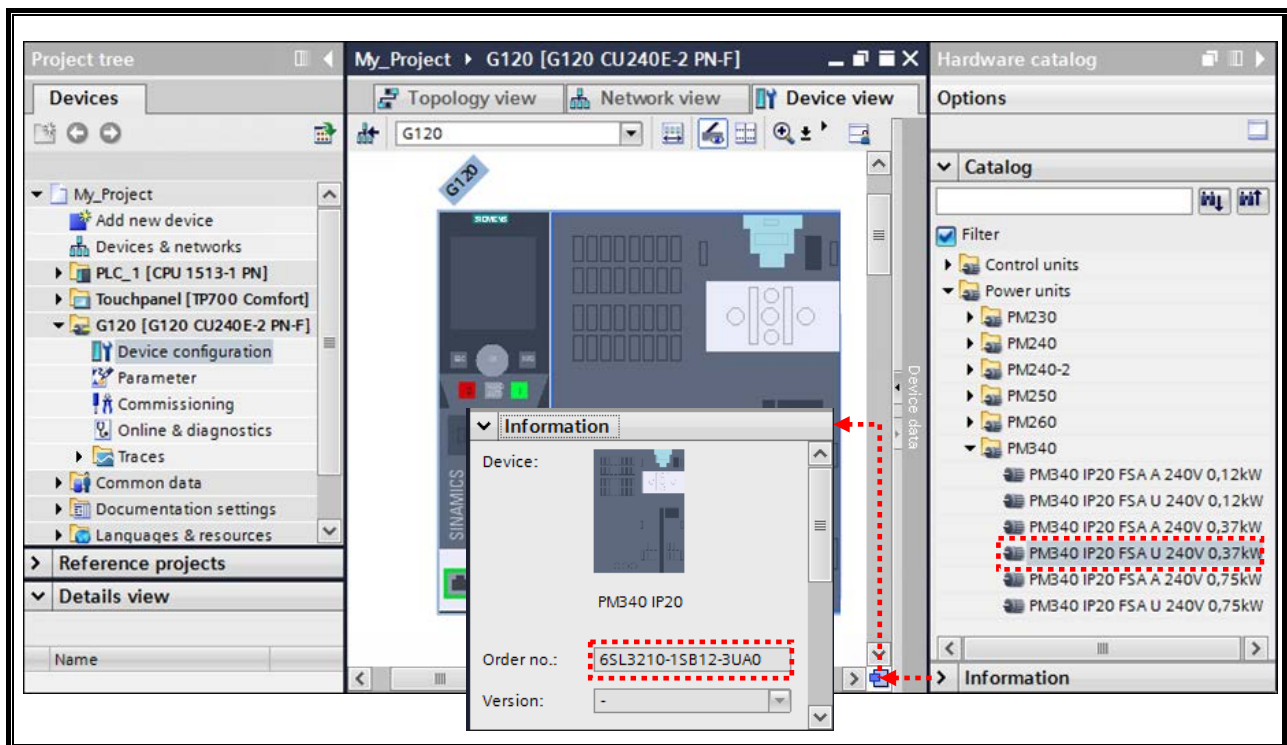


This name must be assigned to the PROFINET-IO module online; otherwise, the CPU (PROFINET-I/O Controller) cannot identify the module.

Address Assignment of the Distributed IO-Module

For PROFINET-IO, the parameterized IP address of the module is assigned by the CPU through the configured device name.

13.3.3. Configuring a Power Unit

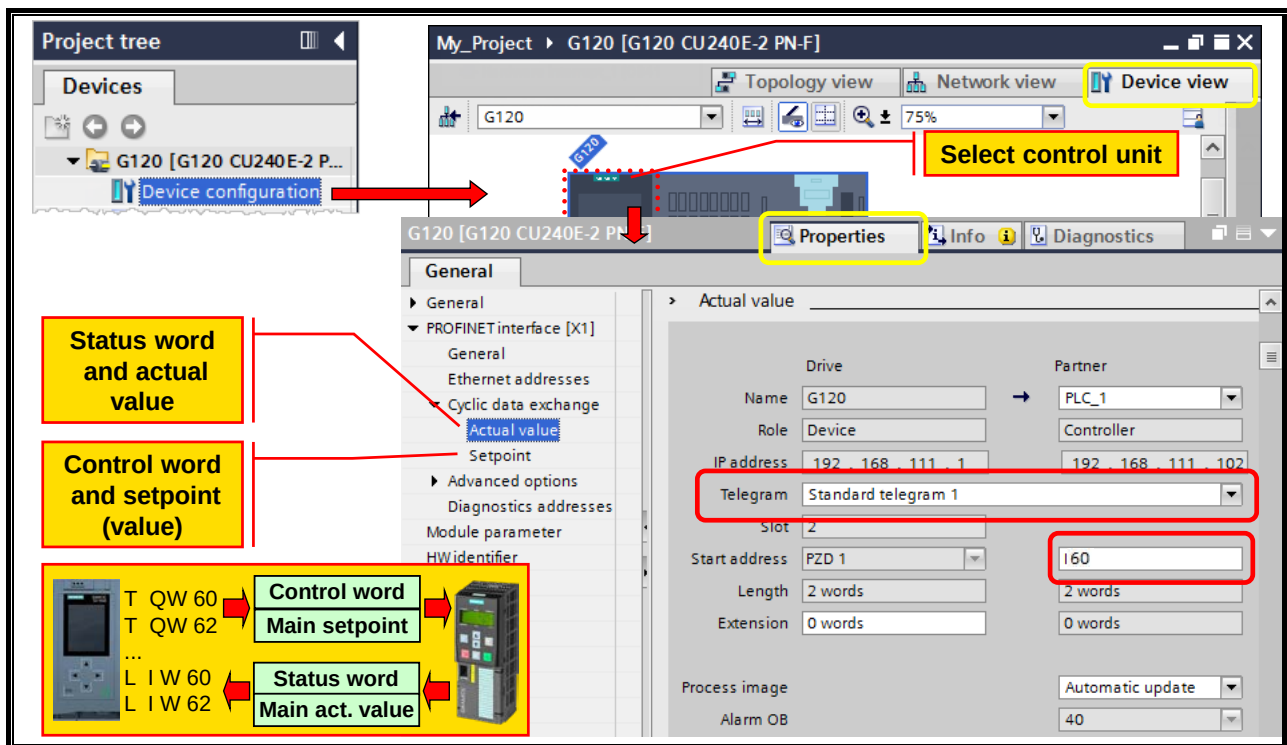


Configuring a Power Unit:

A drive consists of two components

- Control unit → already configured with the added device
- Power unit → must be configured

13.3.4. Parameterizing the Process Data Area (PZD)



Process Data Area

Communication using Standard telegram 1 is based on a cyclic process data exchange that is very easy to program.



The control word and status word comply with the specifications of PROFIdrive profile Version 2.0 or Version 3.0 for the "speed control" operating mode.

The control word and, if necessary, the main setpoint are sent from the CPU to the drive. In the response telegram, the drive returns the status word and the main actual value.

Since only two words are exchanged with the drive, simple load and transfer operations are all that are required in the program. When a double word is transferred, data consistency is also assured.

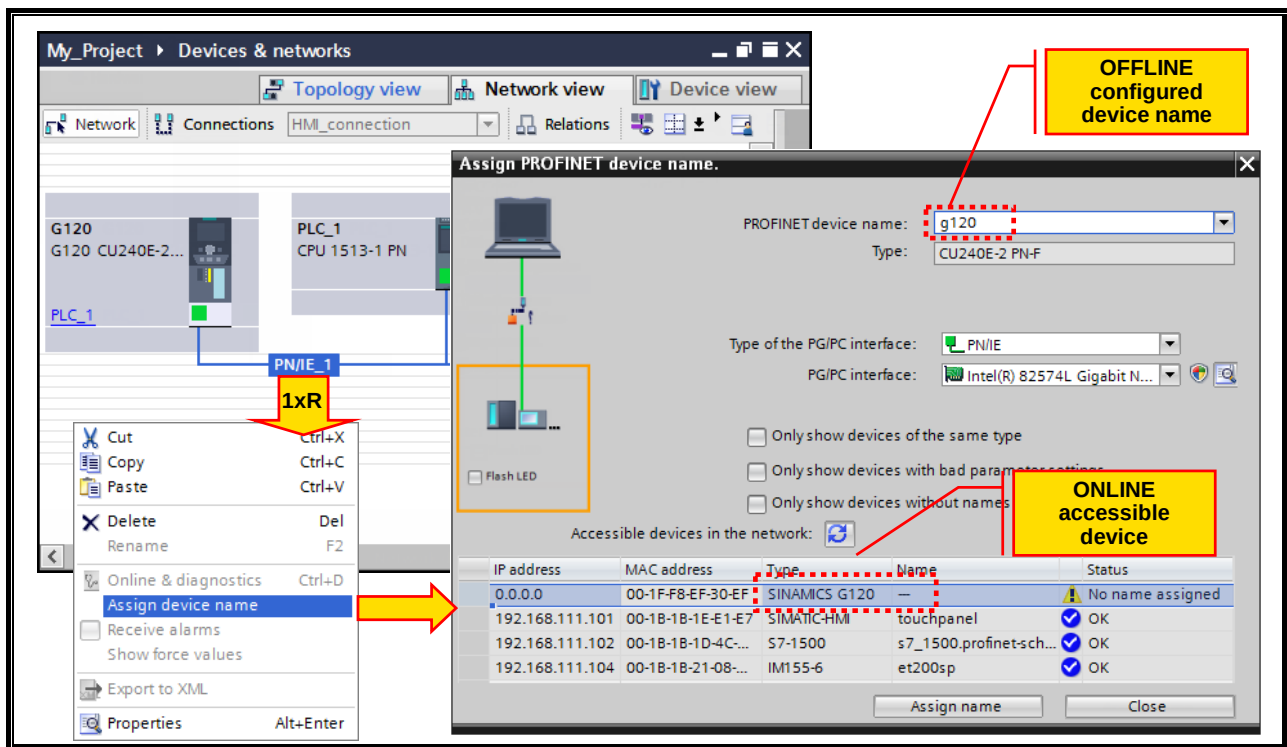
Control Word

The control word contains 16 binary signals for controlling (On/Off, direction of rotation) the drive.

Main Setpoint

According to standard parameterization of the drive, the main setpoint specifies the setpoint speed.

13.3.5. Assigning the Device Name ONLINE (Module Initialization)



Device Name

The PROFINET device name that was configured OFFLINE must be assigned to the drive that actually exists ONLINE.

Name of the Module

The name is the assignment CPU ↔ module.

Through this, the PROFINET-IO is identified by the PROFINET-Controller.

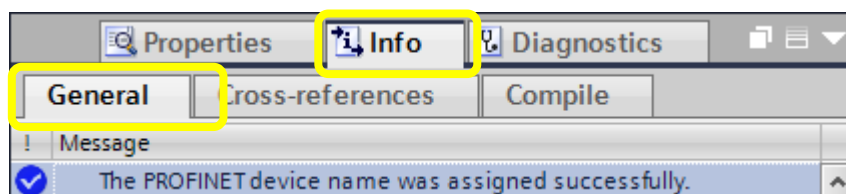


This name must be assigned to the PROFINET-IO module online; otherwise, the CPU (PROFINET-I/O Controller) cannot identify the module.

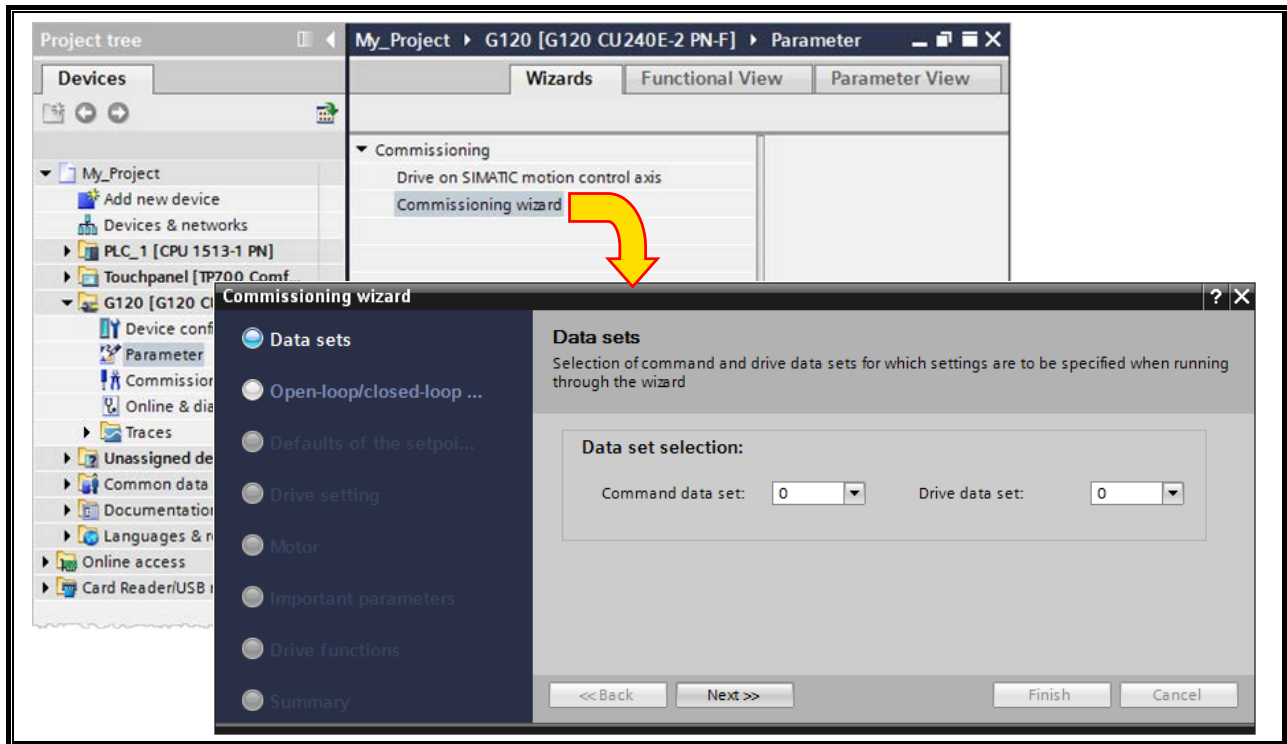
Name Assignment

Currently, the menu item is only included in the context menu of the network connection; not with the drive.

Checking the Result in the Inspector Window



13.4. Parameterizing the Drive: with the "Commissioning Wizard"



Commissioning Wizard

The most important settings for the drive can be made with the menu-driven Commissioning Wizard. These are:

- Data sets
- Open-loop/closed-loop parameters
- Defaults of the setpoints / command sources
- Drive settings
- Motor (data)
- Important parameters (dynamic data)
- Drive functions

13.5. Online Commissioning: Activating / Deactivating the Control Panel

Prerequisite:

- Online connection PG ↔ Drive exists
- No fault is active (if necessary, acknowledge first)

The screenshot shows the 'Project tree' on the left with 'G120 [G120 CU240E-2 PN-F]' selected. A red arrow points to the 'Go online' button. Another red arrow points from 'Commissioning' in the project tree to the 'Control panel' option in the 'Commissioning wizard'. A third red arrow points from the 'Acknowledge fault' button in the 'Drive status' section to the 'Acknowledge faults' button. A final red arrow points from the 'Activate' button in the 'Master control' section to the 'Activate' button.

As soon as the drive has a valid IP address and thus is accessible online, the parameterization and functions of commissioning are available.

Online Accesses

As soon as the drive has been assigned a valid IP address in the subnet used, the functions of commissioning are available here.

Device in the Project Tree

If the drive was inserted as a new device, it is assigned the default IP address 192.168.0.1.

Go online is only possible when the IP address of the device configuration corresponds to the subnet used and is also assigned to the drive.

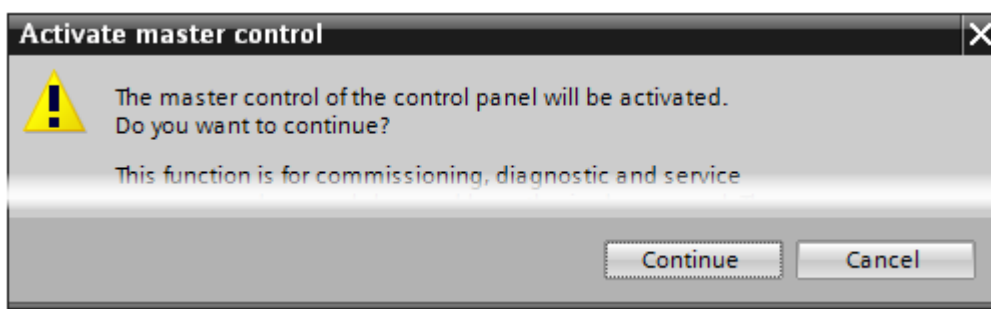
Control Panel

This can only be activated when an online connection exists and no fault exists.

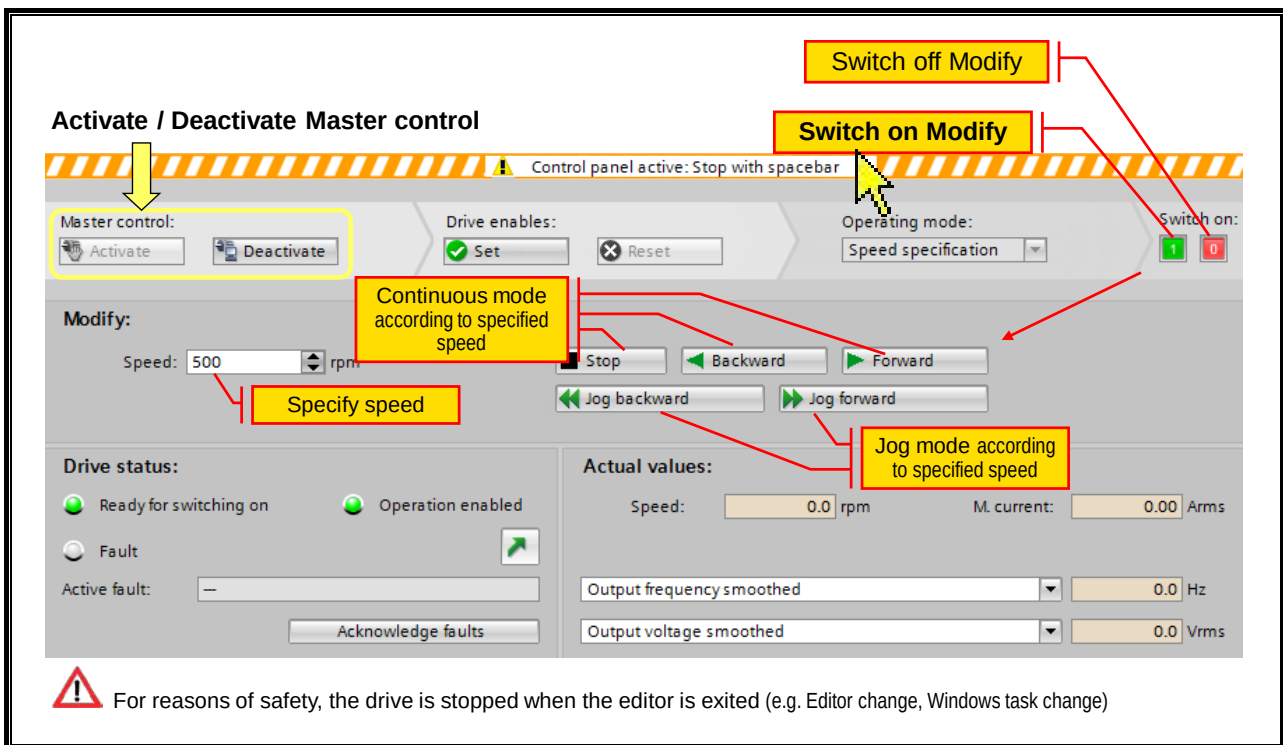


Caution!

Activation terminates a possibly existing connection to the CPU and gives the operator full control of the drive. Therefore it is important to have at your disposal a hardware solution for an Emergency Stop.



13.5.1. Operating the Control Panel



If the master control is "fetched to the PG", "Modify" can be switched on.

Operation is enabled and the buttons for modifying are active.

Safety Shutdown

The drive is stopped and...

- operating enables are cancelled, when the "Commissioning" editor is exited through a Windows task change.

⇒  **Set**

- the master control is deactivated with a fault message, when a switch is made to the editor of another device.

⇒  **Acknowledge faults** and  **Activate**

13.5.2. Monitoring Control and Status Word(s) Online

Select Control word 1 and Status word 1

Control word 1

- 0 ON/OFF1 [0=No, 1=Yes]
- 1 OC / OFF2 [0=No, 1=Yes]
- 2 OC / OFF3 [0=No, 1=Yes]
- 3 Operation enable [0=No, 1=Yes]
- 4 Ramp-function generator enable [0=No, 1=Yes]
- 5 Continue ramp-function generator [0=No, 1=Yes]
- 6 Speed setpoint enable [0=No, 1=Yes]
- 7 Acknowledge fault [0=No, 1=Yes]
- 8 Jog bit 0 [0=No, 1=Yes]
- 9 Jog bit 1 [0=No, 1=Yes]
- 10 Master ctrl by PLC [0=No, 1=Yes]
- 11 Direction reversal (setpoint) [0=No, 1=Yes]

Status word 1

- 0 Rdy for switch on [0=No, 1=Yes]
- 1 Ready [0=No, 1=Yes]
- 2 Operation enabled [0=No, 1=Yes]
- 3 Fault present [0=No, 1=Yes]
- 4 Coast down active (OFF2) [0=Yes, 1=No]
- 5 Quick Stop active (OFF3) [0=Yes, 1=No]
- 6 Switching on inhibited active [0=No, 1=Yes]
- 7 Alarm present [0=No, 1=Yes]
- 8 Deviation setpoint/actual speed [0=Yes, 1=No]
- 9 Control request [0=No, 1=Yes]
- 10 Maximum speed reached [0=No, 1=Yes]
- 11 I, M, P limit reached [0=Yes, 1=No]
- 12 Motor holding brake open [0=No, 1=Yes]
- 13 Alarm motor overtemperature [0=Yes, 1=No]
- 14 Motor rotates forwards [0=No, 1=Yes]
- 15 Alarm drive converter overload [0=Yes, 1=No]

Project tree

- My_Project
 - Add new device
 - Devices & networks
 - PLC_1 [CPU 1513-1 PN]
 - Touchpanel [TP700 Comfort]
 - G120 [G120 CU240E-2 PN-F]
 - Device configuration
 - Parameter
 - Commissioning
 - Online & diagnostics
 - Traces
 - Common data

Go online

My_Project > G120 [G120]

- Online access
- Diagnostics
 - Diagnostics general
 - Active messages
 - Message history
 - Control/status word
 - Drive enable signals
 - Safety diagnostics
- Functions
 - Backing up/reset

13.5.3. Monitoring Active Messages Online

Active messages

Faults

	Fault buffer	Fault code	Message
1	Fault 1	1030	Sign-of-life failure for master control
2			
3			

Alarms

	Alarm buffer	Alarm code	Message
1	Alarm 1	8526	PROFINET: No cyclic connection
2			

Project tree

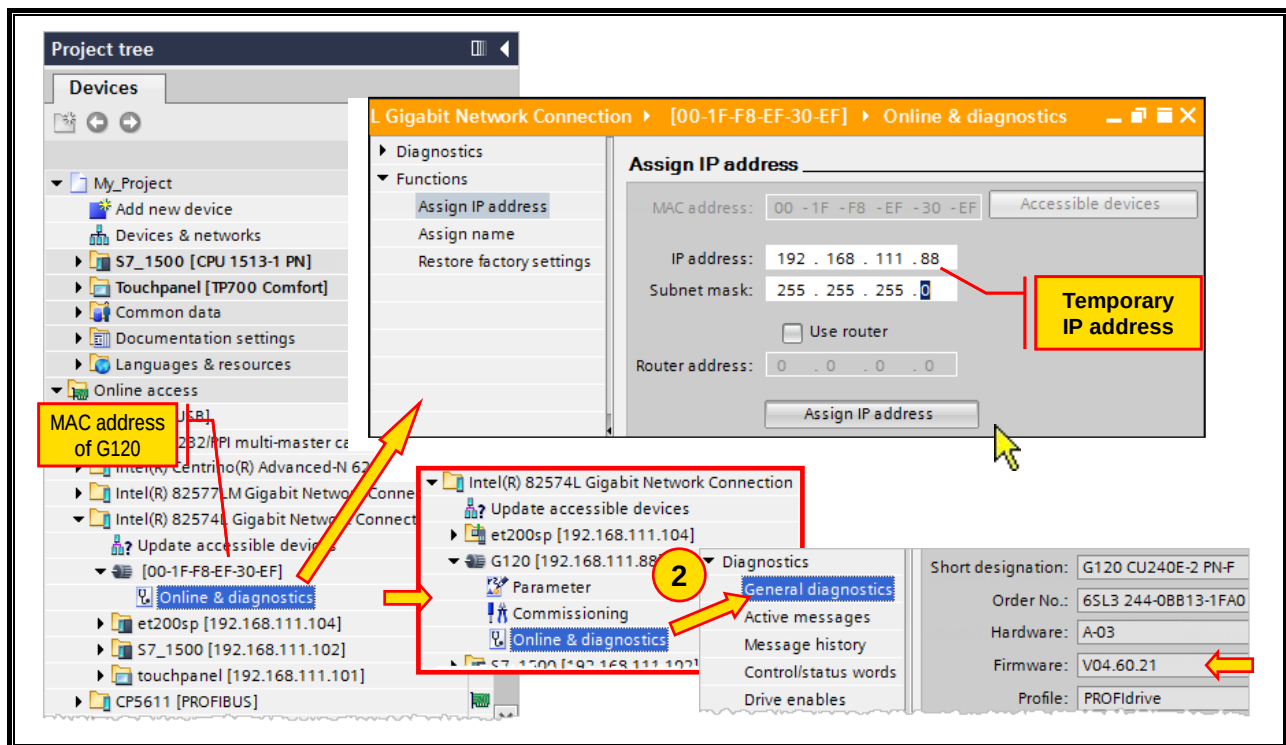
- My_Project
 - Add new device
 - Devices & networks
 - PLC_1 [CPU 1513-1 PN]
 - Touchpanel [TP700 Comfort]
 - G120 [G120 CU240E-2 PN-F]
 - Device configuration
 - Parameter
 - Commissioning
 - Online & diagnostics
 - Traces
 - Common data

Go online

My_Project > G120 [G120]

- Online access
- Diagnostics
 - Diagnostics general
 - Active messages
 - Message history
 - Control/status word
 - Drive enable signals
 - Safety diagnostics
- Functions
 - Backing up/reset

13.5.4. Exercise 1: Reading-out the Firmware Version of the Drive



Task

In order to be able to configure the drive in the project in the next exercises, you must know the firmware version of the training unit. Since the firmware of the drive can be upgraded, the version of the firmware specified on the outside of the drive does not necessarily reflect reality. For that reason, the current version of the firmware is now to be read out online from the drive.

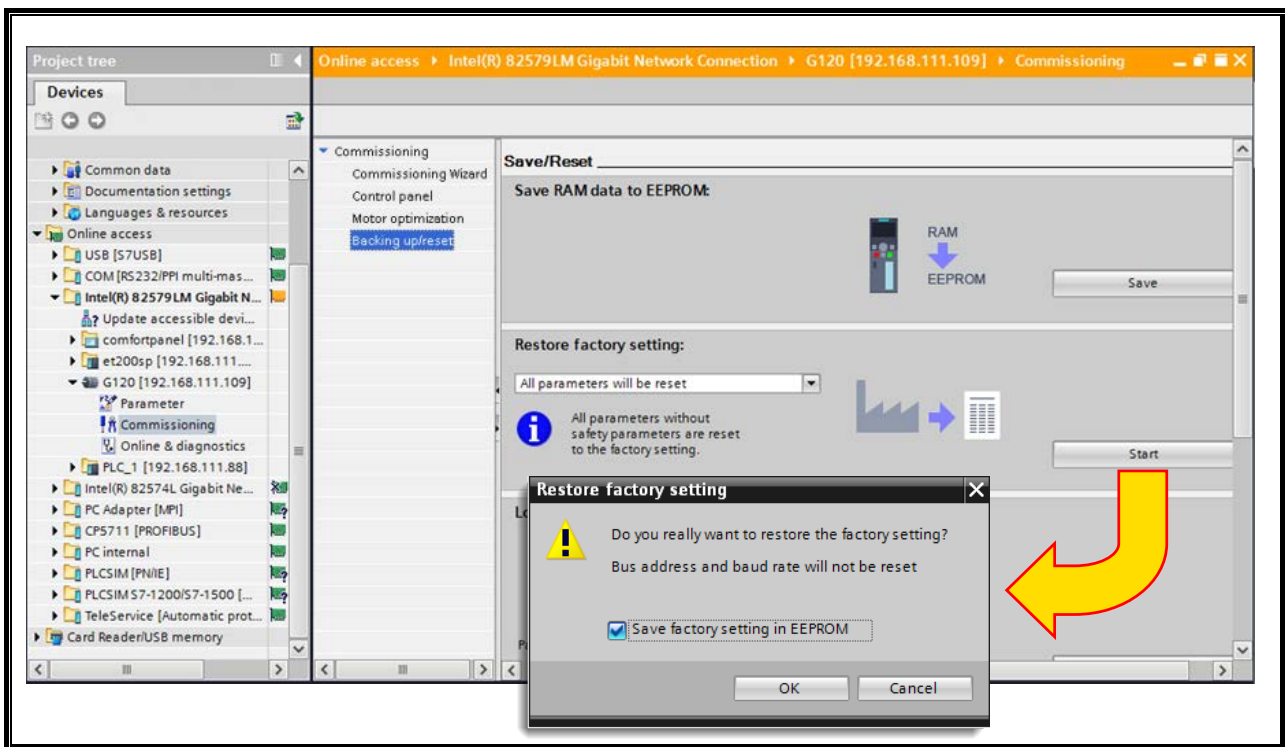
Problem

Should the drive only have a MAC address (see Project tree, to the left in the picture) and neither a PROFINET device name nor an IP address, the firmware version cannot be read out, since an IP address is required for this diagnostic service.

What to Do

1. In case the drive does not yet have an IP address:
Assign a temporary IP address as shown in the picture and update the list "Update accessible devices".
2. In the list of "accessible devices", open the drive and activate "Online & diagnostics".
3. In the window that appears on the right, activate the function "General diagnostics" and make note of the firmware version of the inverter.
4. Close the window.

13.5.5. Exercise 2: Restoring the Factory Settings



Task

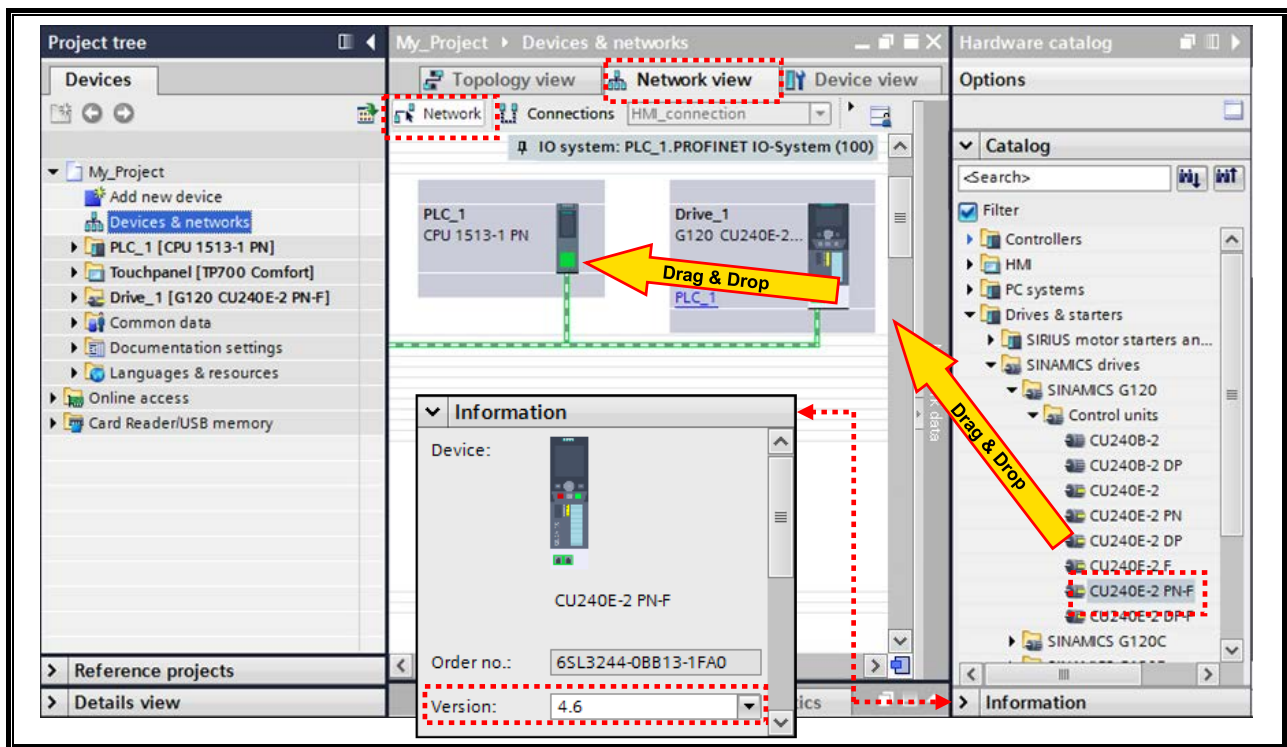
The drive is to be reset to its factory settings.

With the "Restore factory settings" function, the PROFINET device name and the IP address are retained.

What to Do

1. Under the Online access, select the interface Intel(R) 82574L Gigabit.... and update the list "Update accessible devices".
2. Open the drive and there activate the function "Commissioning"
3. In the window that appears on the right, activate the function "Backing up/reset" and there start the function "Restore factory setting".
4. In the dialog that then appears, activate the option "Save factory setting in EEPROM" so that the drive retains the factory setting even after a power recovery.
5. Close the window.

13.5.6. Exercise 3: Inserting and Networking the Drive in the Offline Project



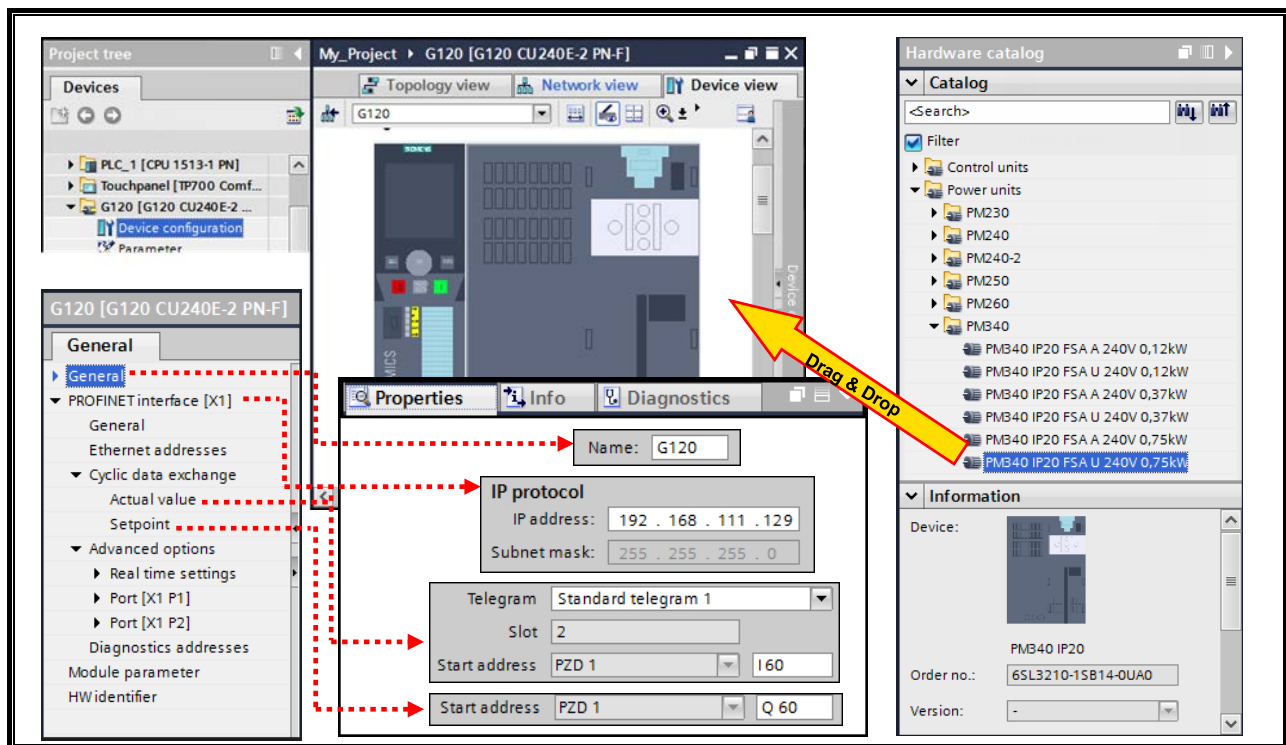
Task

The drive is to be inserted in the project and networked with the CPU via PROFINET.

What to Do

1. Open the "Devices & networks" editor and there activate the Network view.
2. In the Task Cards, open the Hardware catalog.
3. Insert the G120 drive by dragging the CU240E-2PN-F control unit into the Network view using drag & drop. Pay attention to the order number and the version in the "Information".
4. Network the drive with the CPU by dragging the PROFINET interface of the drive to the PROFINET interface of the CPU using drag & drop.
5. Save your project.

13.5.7. Exercise 4: Configuring and Parameterizing the Drive



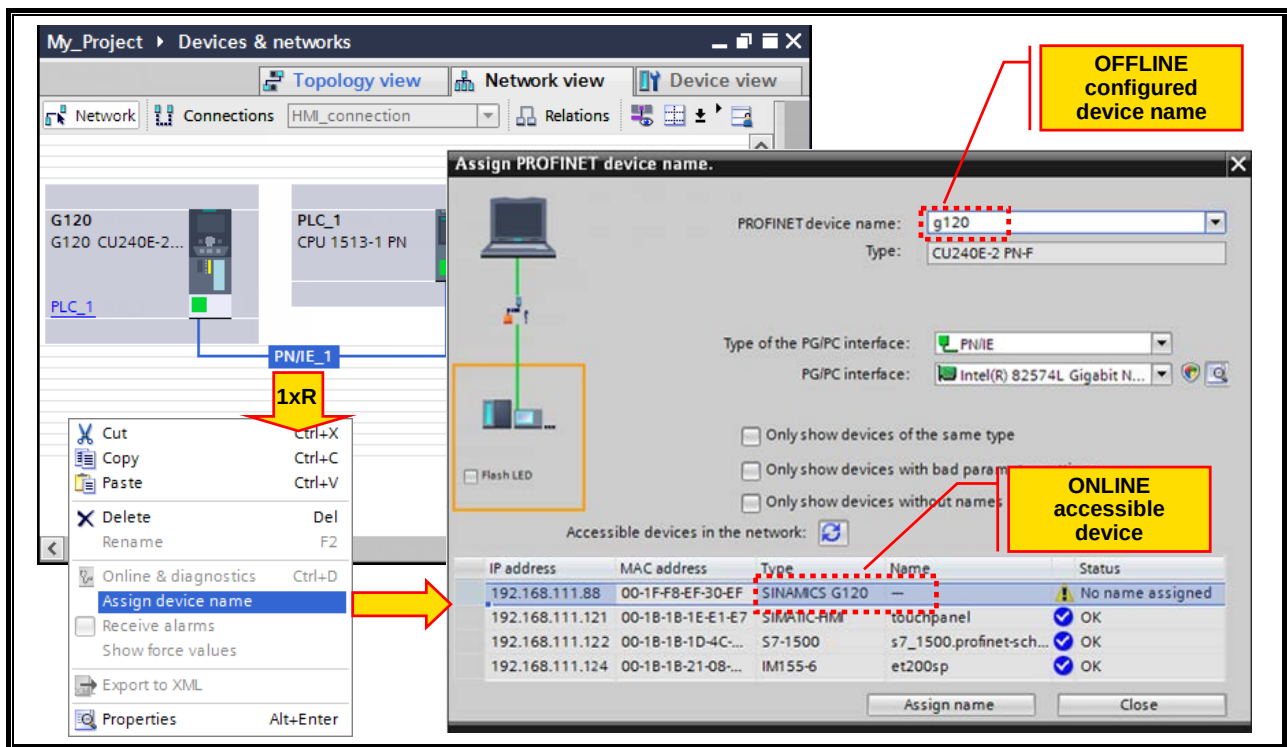
Task

So that the configuration of the drive is complete, a power unit still has to be added. Furthermore, the control unit still has to be parameterized.

What to Do:

1. Open the Device configuration of the drive.
2. Open the Hardware catalog and, using drag & drop, add the power unit. Pay attention to its order number and version in the "Information".
3. Select the control unit and, in the Inspector window under "Properties", parameterize the module with the parameters shown in the picture.
4. Save your project.

13.5.8. Exercise 5: Assigning the PROFINET Device Name Online



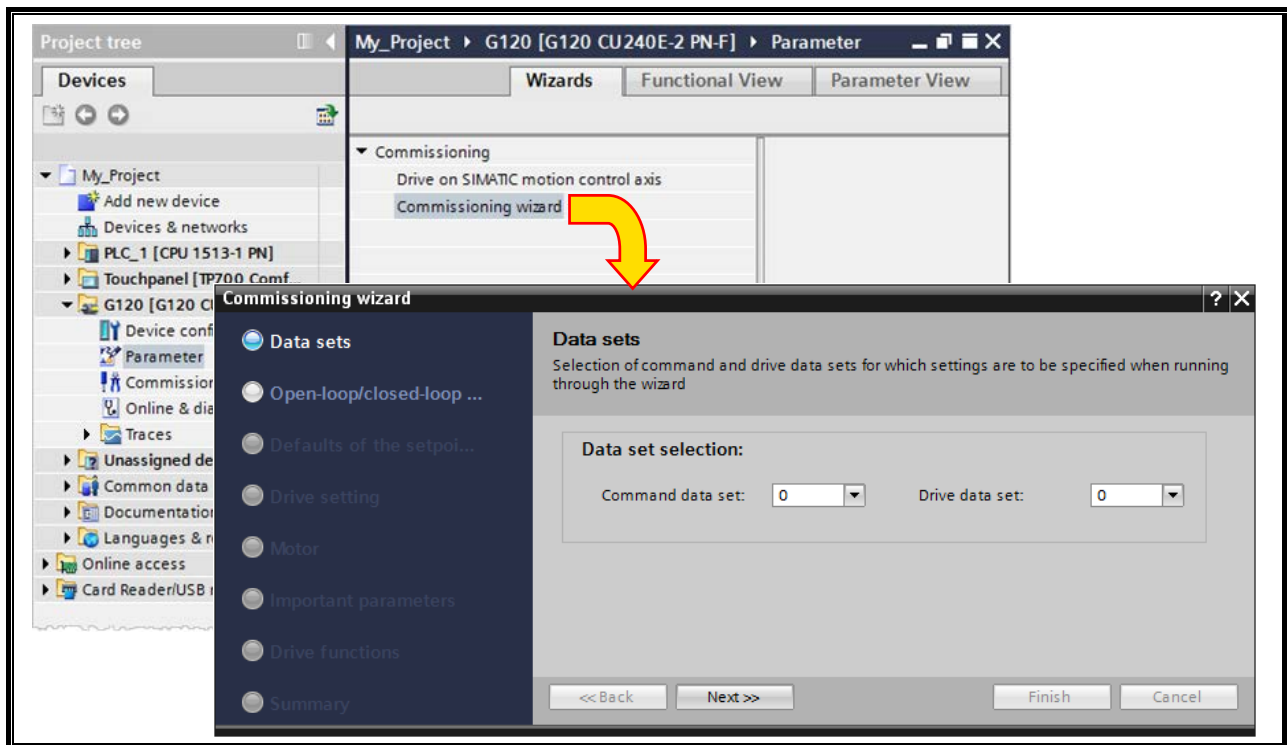
Task

ONLINE, assign the OFFLINE configured PROFINET device name and the IP address to the drive.

What to Do

1. Open the Devices & networks editor and there activate the Network view.
2. Select the PLC_1 IO-System (100) and via the right mouse button, activate the function "Assign device name"
3. In the dialog that follows, complete it as shown in the picture and assign the name.
4. Check whether the name assignment was successful by "Updating the accessible devices" under the Online access.
5. Save your project.

13.5.9. Exercise 6: Parameterizing the Drive OFFLINE with the Commissioning Wizard



Task

OFFLINE set the most important drive parameters and use the "Commissioning Wizard" for this.

What to Do

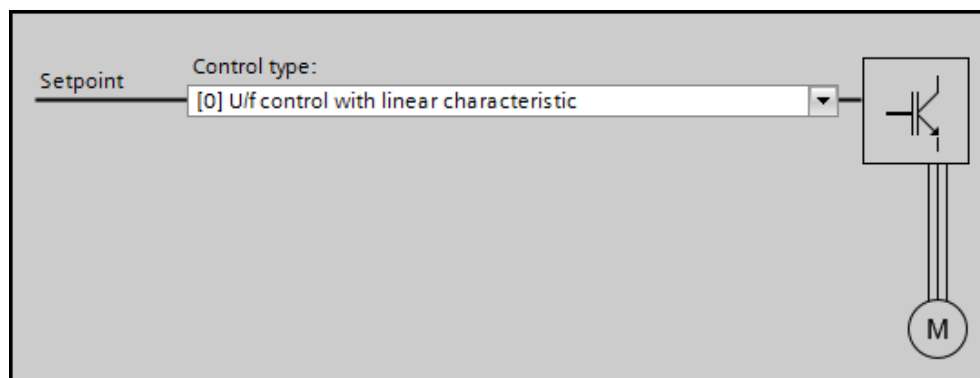
1. Start the Commissioning Wizard as shown in the picture. Implement the parameterization as shown in the following.

- **Data sets**

Data set selection:

Command data set: Drive data set:

- **Open-loop and closed-loop control type**



- Defaults of the setpoints / command sources

Current I/O configuration:

[7] Fieldbus with data set changeover

Select the default of the I/O configuration:

No change

 Note: If changed, all the existing drive-internal interconnections to the I/O terminals are deleted and reconnected in accordance with the selected I/O configuration.

Accept

Current interconnections of the I/O terminals:

DI 0:	p1055[1] BI: Jog bit 0	^
DI 1:	p1056[1] BI: Jog bit 1	
AI 0:	–	
AI 1:	–	v

Current telegram configuration:

[1] Standard telegram 1, PZD-2/2

- Drive setting

Standard:

[0] IEC-Motor (50 Hz, SI units)

You can see which standard your motor complies with on the rating plate:

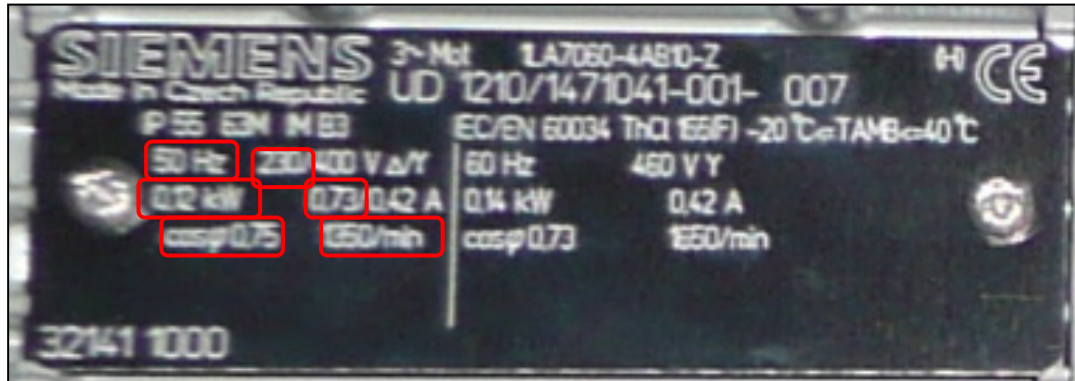
- [0] IEC-Motor (50 Hz, SI units)
- [1] NEMA motor (60 Hz, US units)
- [2] NEMA motor (60 Hz, SI units)

Power unit application:

[0] Load duty cycle with high overload for vector drives

- **Motor**

Take the motor data from the motor's nameplate and enter it in the Motor dialog:



Motor

Specification of motor type and motor data

Motor configuration
Enter motor data

Select motor type
[17] 1LA7 induction motor

Select the connection type for your motor and 87 Hz operation:
Delta ☐ Motor 87 Hz operation

Motor data

Parameter	Parameter text	Value	Unit
p304[0]	Rated motor voltage	230	Vrms
p305[0]	Rated motor current	0.73	Arms
p307[0]	Rated motor power	0.12	kW
p308[0]	Rated motor power factor	0.750	
p310[0]	Rated motor frequency	50.00	Hz
p311[0]	Rated motor speed	1350.0	rpm
p335[0]	Motor cooling type	[0] Non-ventil...	

☐ Parallel motor connection Number: 1

<< Back Next >> Finish Cancel

- **Important parameters**

Change the ramp-function generator times:

Set the values for the most important parameters:

Current limit: Arms

Minimum speed: rpm

Maximum speed: rpm

Ramp-function generator ramp-up time: s

Ramp-function generator ramp-down time: s

OFF3 ramp-down time: s

- **Drive functions**

A motor identification is recommended for the first commissioning.
 Stationary measurement is recommended for V/f and vector control.
 Stationary measurement and rotating measurement are recommended for vector control (only rotating measurement is not sufficient).

Motor identification:

Calculation of the motor parameters

☐ No calculation

☒ Complete calculation

 Includes the calculation of the motor, open-loop control and closed-loop control parameters depending on the entered motor data.

- Summary

Summary

Please check the entered data and complete the configuration

The following drive data has been entered:

Open-loop/closed-loop control type:
Open-loop/closed-loop control operating mode: [0] U/f control with linear characteristic

Defaults of the setpoints/command sources:
Macro drive unit: [7] Fieldbus with data set changeover

Drive setting:
IEC/NEMA mot stds: [0] IEC-Motor (50 Hz, SI units)
Power unit application: [0] Load duty cycle with high overload for vector drives

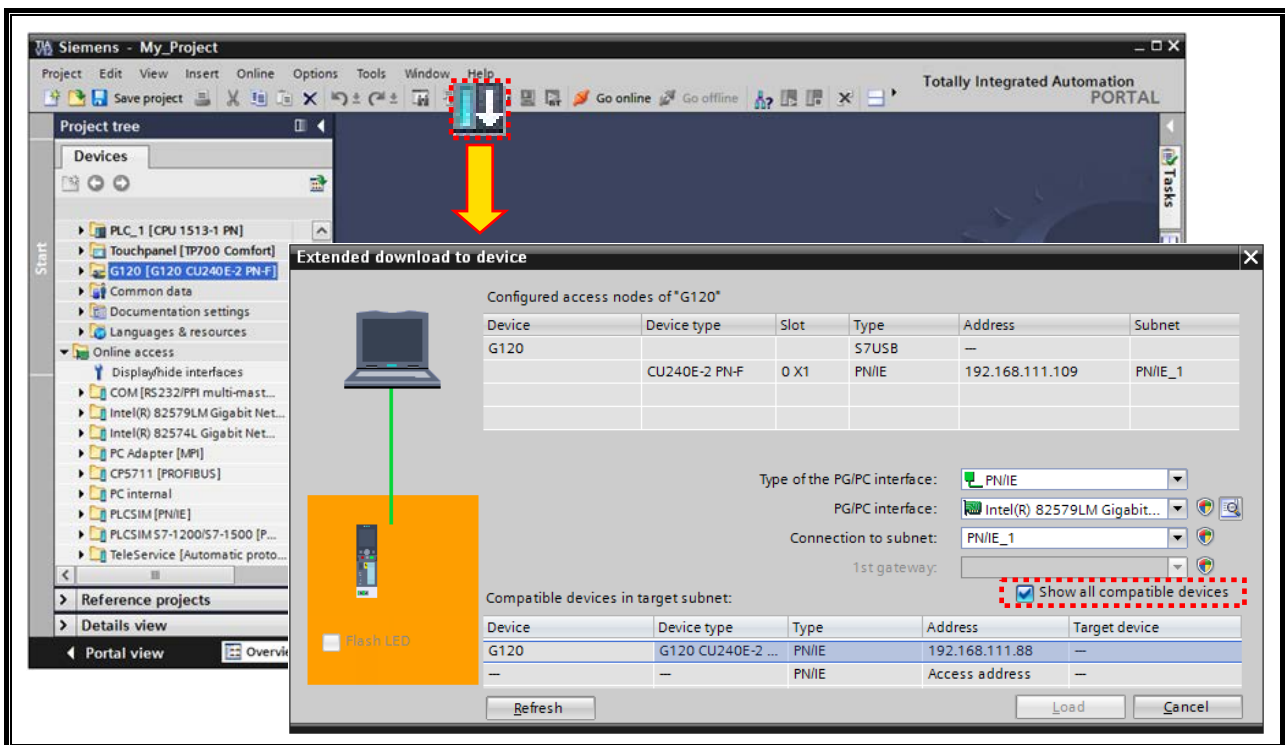
Motor:
Motor type selection: [17] 1LA7 induction motor
Motor connection type: Delta
Motor 87 Hz operation: No
Number of motors connected in parallel: 1
Rated motor voltage: 230 Vrms
Rated motor current: 0.73 Arms
Rated motor power: 0.12 kW
Rated motor power factor: 0.750
Rated motor frequency: 50.00 Hz
Rated motor speed: 1350.0 rpm
Motor cooling type: [0] Non-ventilated

Important parameters:
Current limit: 1.10 Arms
Minimum speed: 0.000 rpm
Maximum speed: 1500.000 rpm
Ramp-function generator ramp-up time: 1.000 s
Ramp-function generator ramp-down time: 1.000 s

<< Back Next >> **Finish** Cancel

2. Close the Commissioning Wizard (with Finish) and save your project.

13.5.10. Exercise 7: Downloading the Parameterization into the Drive



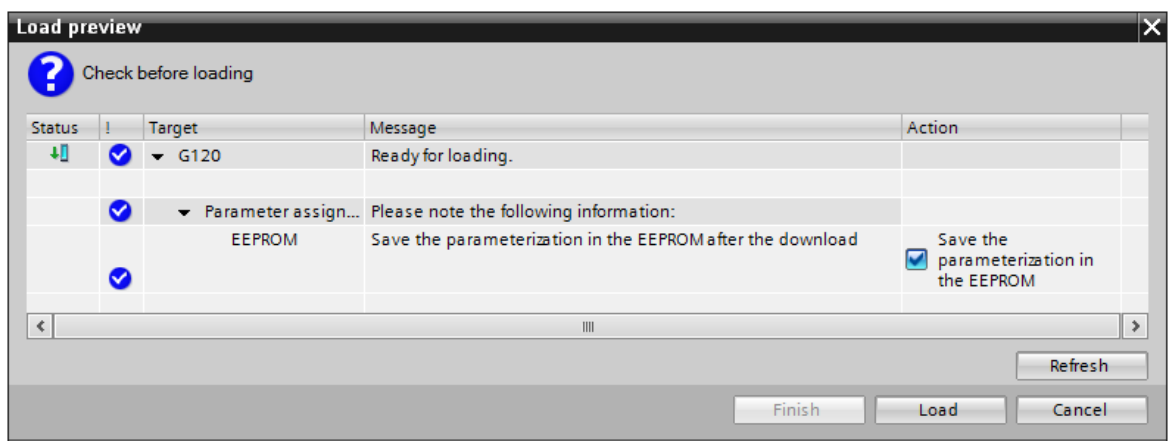
Task

Now that you have implemented the basic parameterization offline, it is now to be downloaded into the drive online.

What to Do

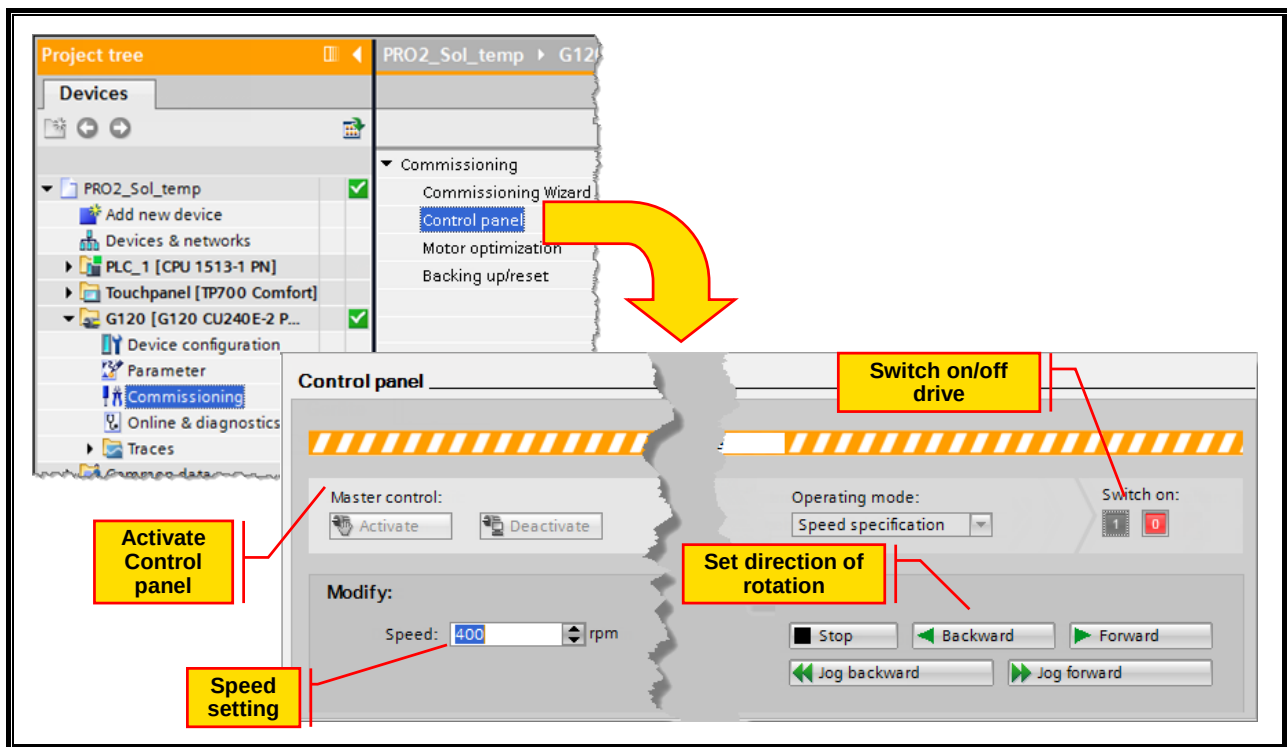
1. In the Project tree, select the drive and start the download of the hardware configuration and parameter assignment using the button shown in the picture.
2. The dialog "Extended download to device" appears which contains the list of "Compatible devices in target subnet" in the lower part. Should the drive of your training area not be listed there, activate the option "Show all compatible devices" (see picture).
3. In the list of "Compatible devices in target subnet", select the drive of your training area and start the "Download".

So that the drive doesn't lose the parameterization even after a power failure, answer the following dialog as follows:



4. Then, download the device configuration into the CPU so that the drive is passed the parameterized IP address.
(With Profinet, the distributed I/O is recognized by the CPU by means of the device name and in this way is assigned the parameterized IP address. For that reason, when the IP address is changed, this information must also be passed to the CPU.)

13.5.11. Exercise 8: Operating the Drive via the Control Panel

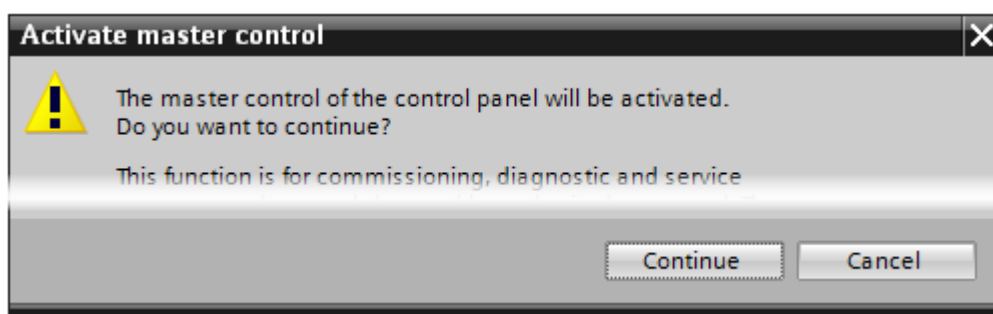


Task

Since the drive is already configured and has been assigned an IP address, it can be operated via the Control panel.

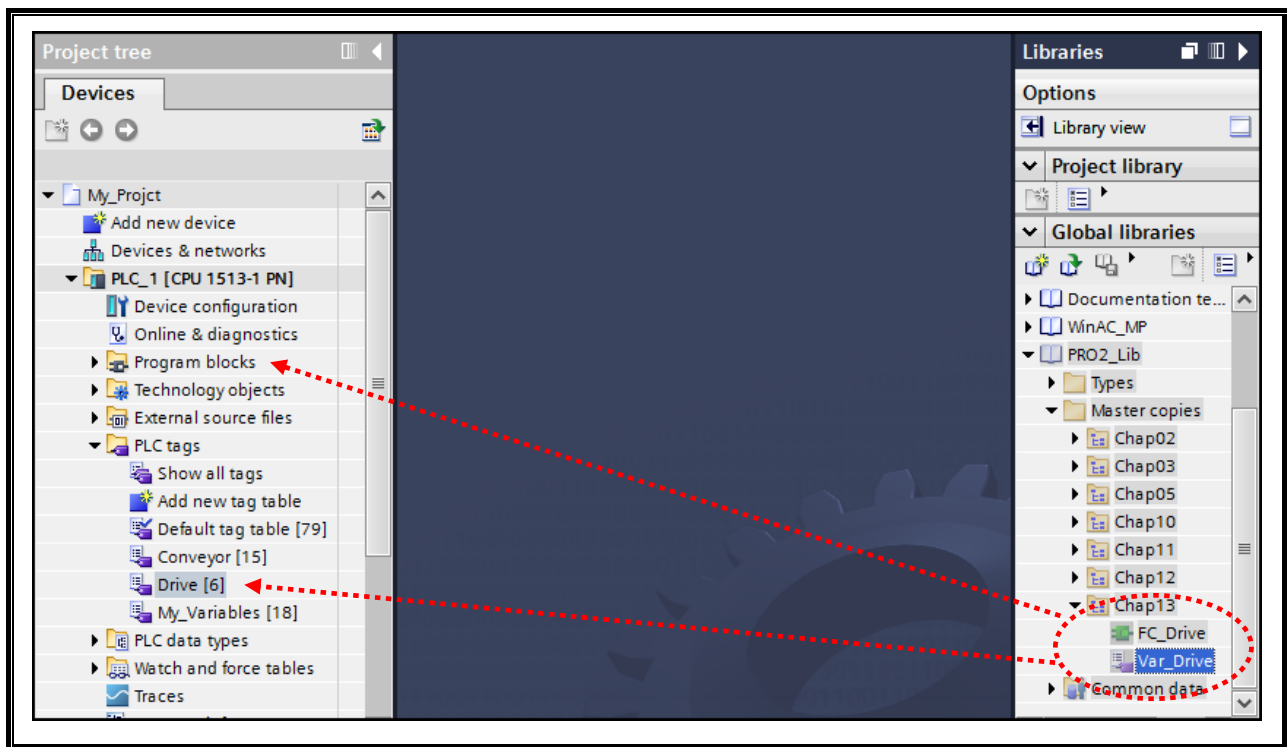
What to Do

1. In the Project tree under "G120" select the function "Commissioning" and there the "Control panel" (see picture)
2. Activate the "Master control" (see picture) and acknowledge the dialog that appears with "Continue"



3. In the control panel, specify a speed and switch on the drive.
4. Before the motor can turn with the desired speed, one of the two directions "Forward" or "Backward" still has to be selected.
5. Let the motor move "Forward" and "Backward" with different speeds.
6. Switch the motor off again and deactivate the master control.
7. Close the "Commissioning" editor and disconnect the online connection to the G120 (Go Offline).

13.5.12. Exercise 9: Commissioning a Program Expansion



Task

Imagine that the G120 serves as the drive for the conveyor model when parts are transported from the light barrier bay automatically to the Bays 1, 2 and 3.

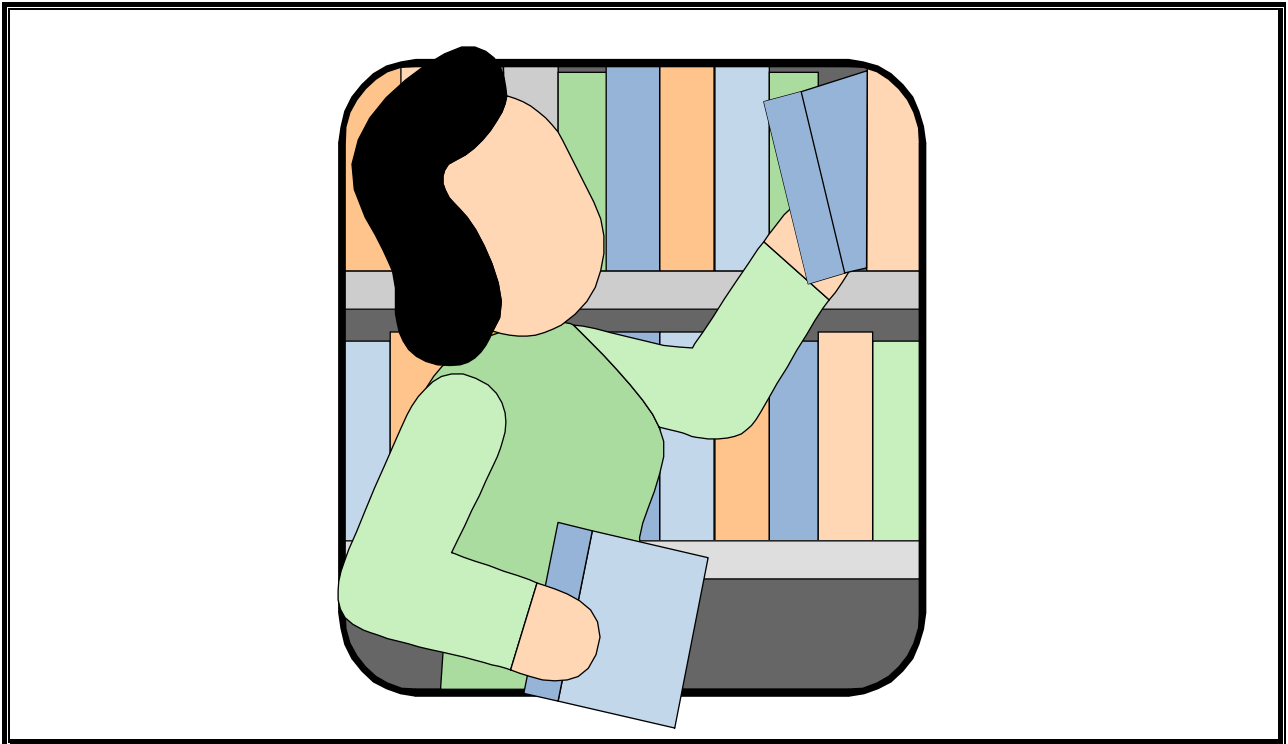
Accordingly, the G120 must be switched on and off parallel to the conveyor model motor when "P_Operation" (Q0.1) is switched on. The G120 speed can be set using the right slide control potentiometer.

The described function is already implemented in the block "FC_Drive" (FC120) and is now to be copied from a library and commissioned.

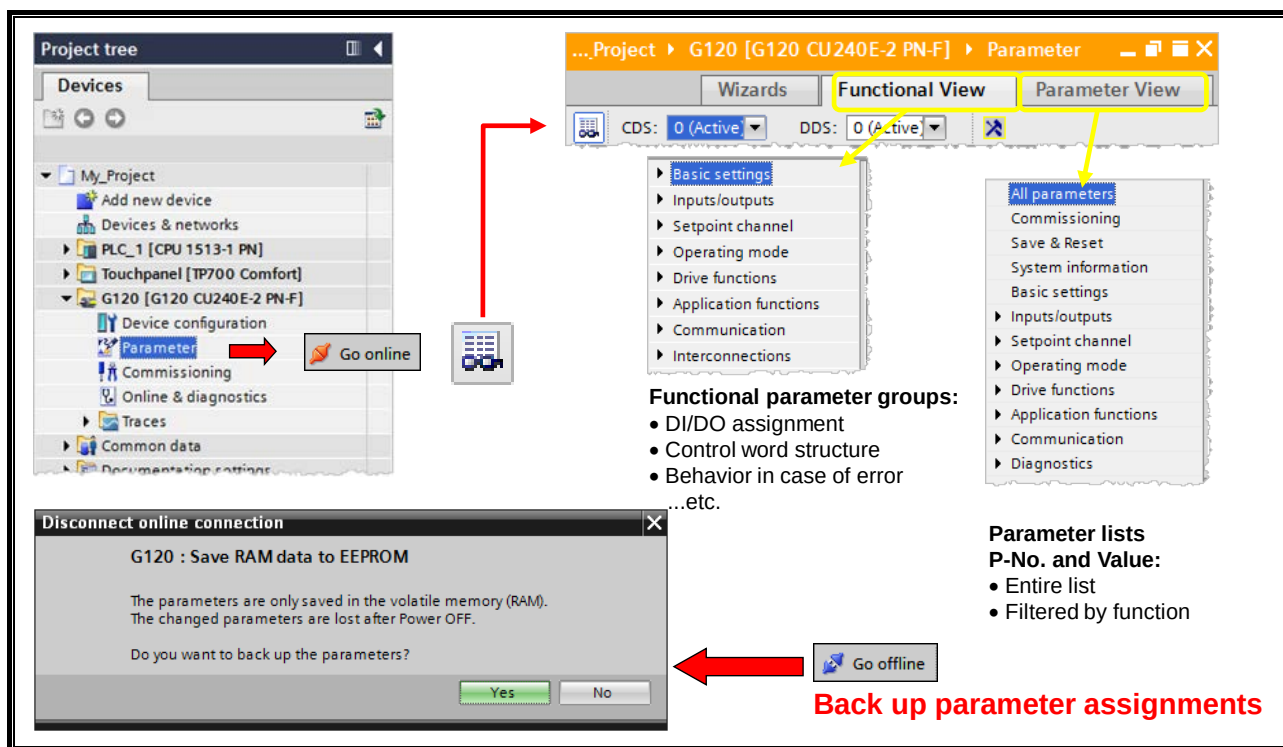
What to Do

1. Open the library C:\Archives\TIA\PRO2_Lib
2. Using drag & drop, copy the library elements "FC_Drive" (FC120) and "Var_Drive" (tag table) into your project.
3. Program the call of "FC_Drive" (FC120) in OB1.
4. Download all blocks into the CPU and check whether the G120 is switched on and switched off parallel to the conveyor model motor when "P_Operation" (Q0.1) is switched on.
5. Check whether the speed of the G120 can be changed using the right slide control potentiometer of the simulator.
6. Save your project.

13.6. Additional Information



13.6.1. Changing Parameters in the Inverter



Parameters can be changed through the "Parameter" editor.

Calling the "Parameter" Editor from the Device in the Project Tree



This function must be activated so that the online parameters of the drive are also visible. You can see this in that the parameters are displayed on an orange-colored background.

When the online connection is to be terminated, a prompt comes up questioning whether the data is to be copied to EEPROM (nonvolatile backup of the parameter changes).

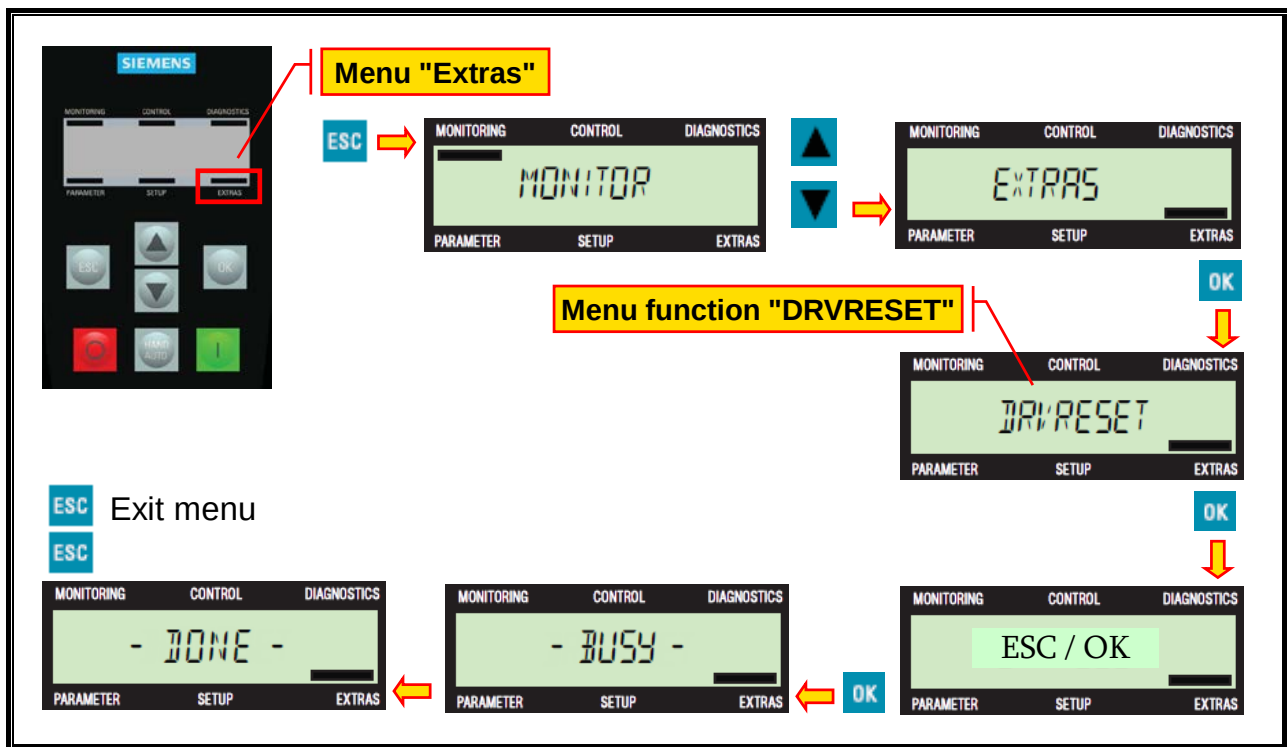


If the change is only to be tried for test purposes, you can answer the query for copying with NO and after a PowerOFF/ON, the inverter then works with its previous values.

Calling the "Parameter" Editor from the Online Access

The change of these parameters is immediately written to the EEPROM.

13.6.2. G120 Reset to Factory Settings via BOP-2



Contents

14. Training and Support	14-2
14.1. Any Questions on our Training Courses Offered??	14-3
14.2. www.siemens.com/sitrain	14-4
14.3. Learning path: SIMATIC S7 Programming based on TIA Portal	14-6
14.4. The Industry Online Support – the most important innovations.....	14-7
14.5. The Principle of Navigation	14-8
14.6. Complete product information.....	14-9
14.7. mySupport – Overview.....	14-10
14.8. Support Request	14-11
14.9. Support Request	14-12
14.10. Forum - the communication platform for Siemens Industry products	14-13
14.10.1. Conferences and Forum management	14-13
14.10.2. Interactions in the Forum	14-15
14.11. Task and Checkpoint	14-17

14. Training and Support



14.1. Any Questions on our Training Courses Offered??

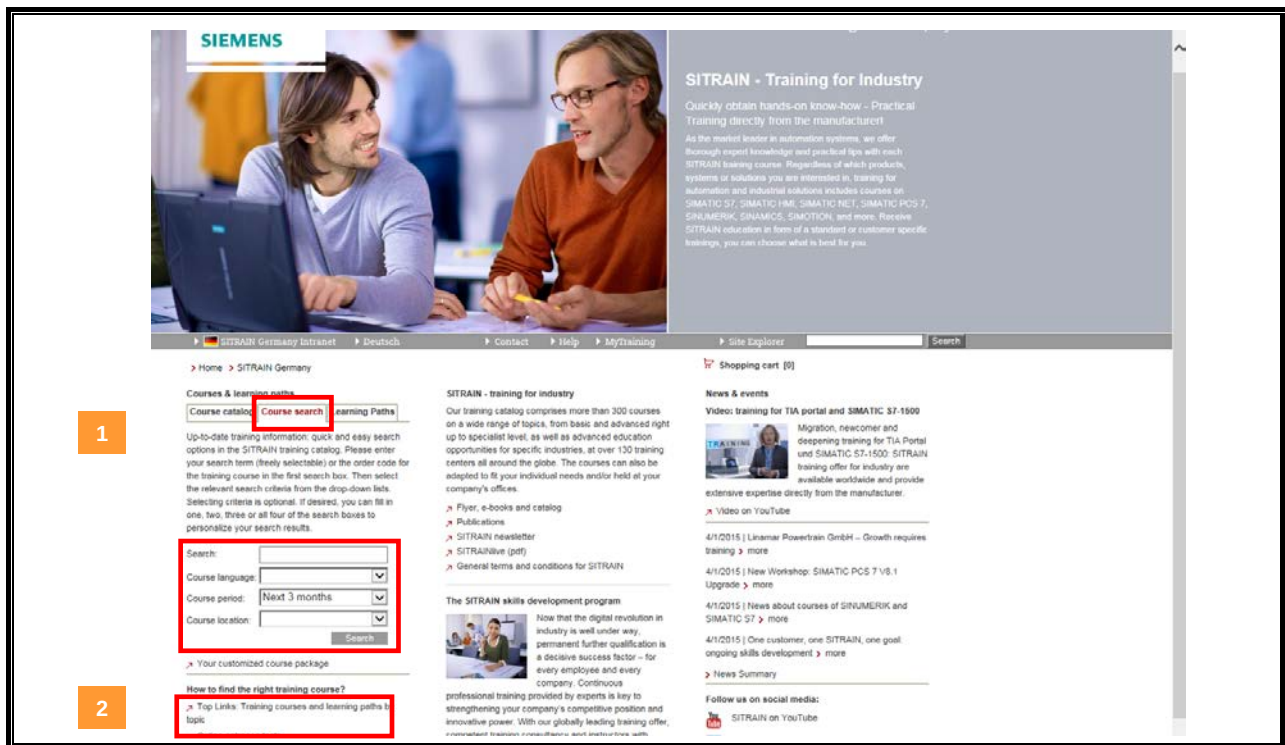


We'll help you!
... on the Internet:
www.siemens.com/sitrain
or with e-mail:
info@sitrain.com

General Information

We'll be glad to help you regarding any questions on our training courses offered.

14.2. www.siemens.com/sitrain



The complete range of courses offered can be accessed via the following links:

www.siemens.de/sitrain or

www.siemens.com/sitrain

Course Search

1

The course search permits the user to find the required courses by applying different search filters such as keyword, target group, etc. The filters can also be combined.

Course Catalog

The course catalog permits you to find the required course via learning paths or via the Siemens Mall structure.

Top Links

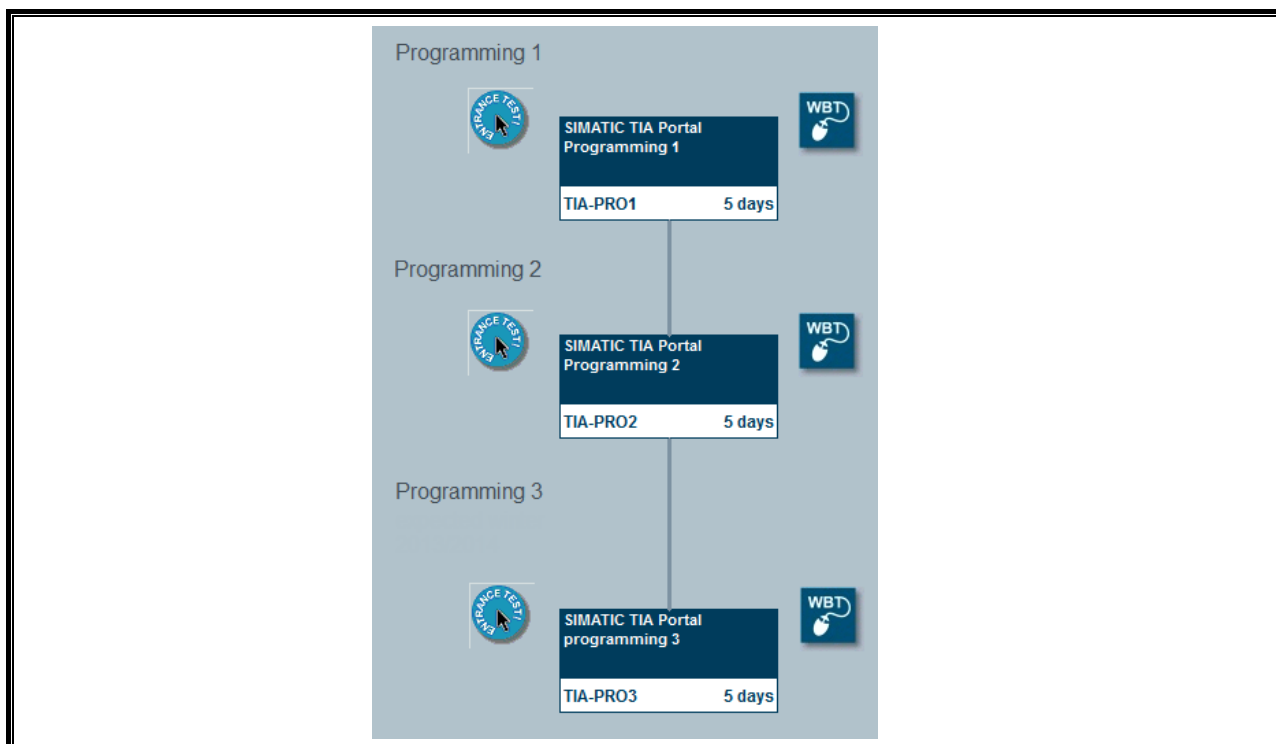
2

Various courses, e.g. SIMATIC S7-1500 solution line, etc., can be reached directly via the top links.

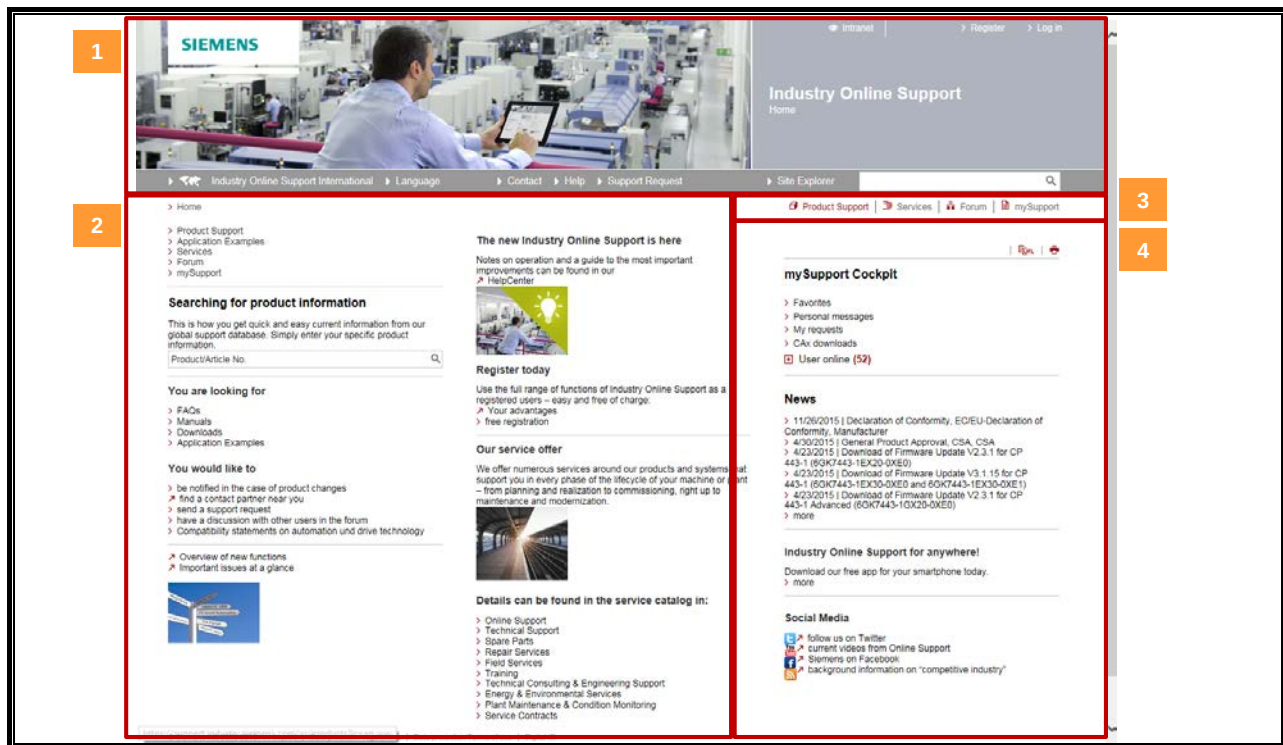
Top links

- > AS Learning Academy for Factory (FA) and Process Automation (PA) [Intra](#)
- > Certified Competency in Industry
- > CNC Training with SINUMERIK
- > Courses for Industrial Communication - SIMATIC NET
- > Energy Management at SITRAIN
- > Identification Systems
- > KNX Building Technologies
- > LD-Qualifying - Enter the World of Large Drives! [Intra](#)
- > Low Voltage Distribution and Electrical Installation Technology
- > MD-Qualifying [Intra](#)
- > Process Analytics
- > Process Instrumentation [Intra](#)
- > Siemens Authorized CNC Training Partners
- > Siemens Authorized SIMATIC Training Partners
- > Siemens COMOS Training
- > SIMATIC HMI, the leading Human Machine Interface solution
- > SIMATIC PCS 7
- > SIMATIC S7 Training based on SIMATIC S7-1200 in the TIA Portal
- > SIMATIC S7 Trainings based on SIMATIC S7-1500 in the TIA Portal
- > SIMATIC S7 Trainings based on SIMATIC S7-300/-400 with STEP 7 V5.x
- > SIMOTION
- > SITRAIN Courses Safety Integrated
- > SITRAIN Learning Services [Intra](#)
- > SMS-CP - Siemens Mechatronic System Certification Program
- > Solution Partner Program
- > The right training course for every drive: SITRAIN training
- > Totally Integrated Power

14.3. Learning path: SIMATIC S7 Programming based on TIA Portal



14.4. The Industry Online Support – the most important innovations



The most important functions are always in the same place on all the pages:

1

The menu bar links to the main areas of the site. You can subscribe and register at any time to benefit from the features the personalized mySupport option offers.

2

Links to our service offerings are in the center. On the start page, you will find up-to-date information and links, which quickly brings you to your destination in other areas of Online Support.

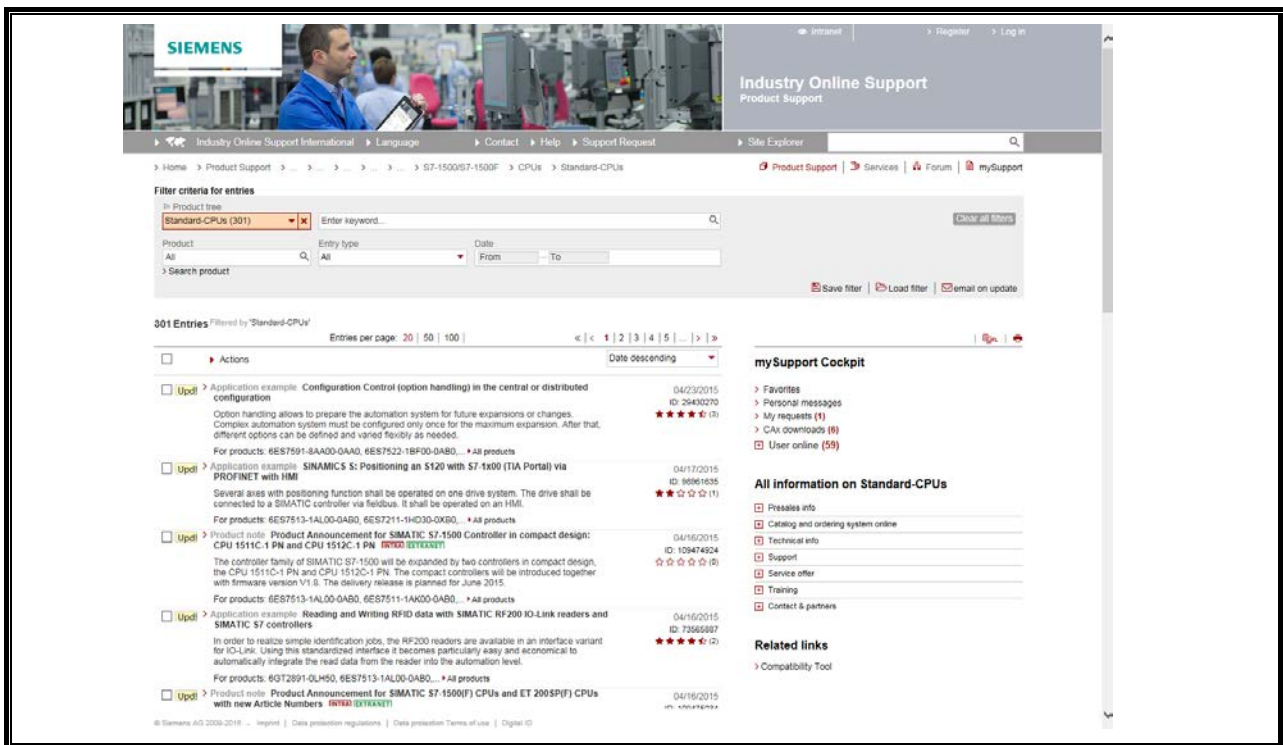
3

Links from the menu bar are repeated at the top of the page: Product Support, Services, Forum and mySupport.

4

On every page, you will find your personal mySupport cockpit. There, for example, you can see when the status of your support inquiry changes.

14.5. The Principle of Navigation



Here, you will find information about all the current and discontinued products, such as:

- Frequently Asked Questions (FAQ)
- Manuals and Operating Instructions
- Downloads
- Product Notes (product announcements, discontinuation, etc.)
- Certificates
- Characteristics
- Application Examples

You will not only be able to access these articles through the product tree, but also through a central filter bar. The integration of various search filters will give you access to relevant information after only a few clicks. The product tree has been moved to an equivalent filter. This has the effect that several filter steps can be combined clearly and comprehensibly.

Based on the preview numbers you can see the expected set of results before using a filter. This makes finding relevant information considerably easier and more efficient.

For example, you can customize your search by combining the product tree, a search keyword and a document type in your search. There will be no hidden search parameters, all the settings and results will be clearly displayed.

14.6. Complete product information

The screenshot displays the Siemens Industry Online Support website. The top navigation bar includes links for Industry Online Support International, Language, Contact, Help, Support Request, and Site Explorer. The breadcrumb trail shows the path: Home > Product Support > ... > PLC > Modular Controller SIMATIC S7 > S7-1500/S7-1500F > CPUs.

The main content area is titled "Filter criteria for entries" and includes a search bar with the text "Enter keyword...". Below the search bar, there are filters for "Product" (set to "6ES7513-1AL00-0AB0") and "Entry type" (set to "All"). A "Search product" button is also present.

The search results show a list of 263 entries, filtered by "6ES7513-1AL00-0AB0". The first entry is for the "6ES7513-1AL00-0AB0" CPU 1513-1 PN, 300KB PROG., 1.5MB DATA. The entry details include a description of the product and its features, such as "SIMATIC S7-1500, CPU 1513-1 PN, CENTRAL PROCESSING UNIT WITH WORKING MEMORY 300 KB FOR PROGRAM AND 1.5 MB FOR DATA, 1. INTERFACE, PROFINET IRT WITH 2 PORT SWITCH, 40 NS BIT-PERFORMANCE, SIMATIC MEMORY CARD NECESSARY".

On the right side of the page, there is a "mySupport Cockpit" section with links for Favorites, Personal messages, My requests, CAX downloads, and User online (68). Below this, there is an "All information on CPUs" section with links for Presales info, Catalog and ordering system online, Technical info, Support, Service offer, Training, and Contact & network.

A powerful function of the Industry Online Support is the direct access to complete product information. You can use it if you are looking for a quick and easy access to all the technical information about a Siemens Industry product. For example, for comparing products, if you are expanding your system or replacing individual components, this is how to do it:

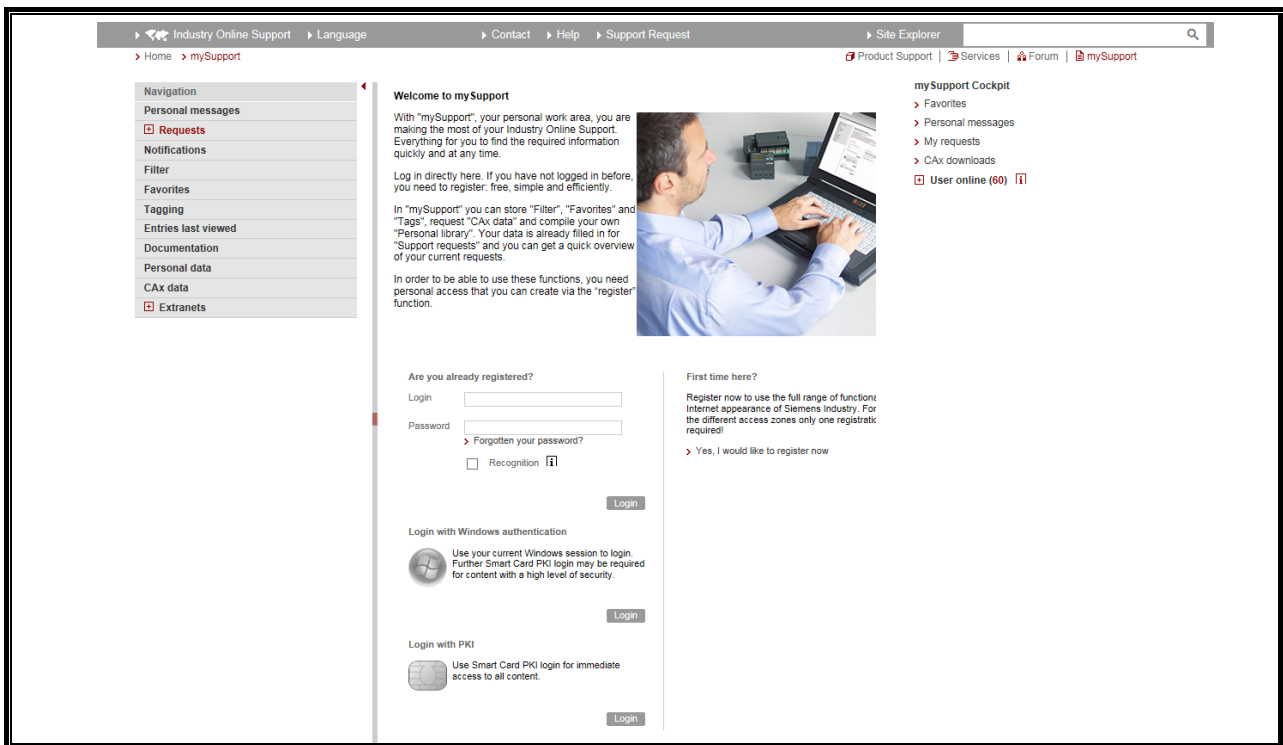
In the Product Support area, there is the central navigation bar.

To select a product, simply select the filter "Product." Enter an order number or a product name here. You will be supported by a dynamic display of suitable products (list of suggestions).

One more click and the details of the selected product will be displayed – always up to date:

- Product life cycle, consisting of milestones with dates (e.g. delivery release, discontinuation of the product, ...). You will find out whether the selected product is a current product or whether the product is already in the discontinuation phase.
- Successor products for discontinued products and new developments will be suggested. If there is a successor product, you will get a direct link to the product information of this product.
- Technical data – clear, compact and complete. You get all the available technical data concerning the selected product here – dimensions, operating voltage or the number of inputs/outputs, etc.

14.7. mySupport – Overview



mySupport

The mySupport area will always remain your personal workplace; with this feature you can make the best of your Industry Online Support experience.

The most important thing, if you're already working with mySupport, you can take all your previous personal data and information you've filed away with you to the Industry Online Support.

In this area, you can compile the information that is important for your daily work – we provide you with the suitable tools. Create your own folder structures and file information such as bookmarks. There are numerous options, whether you want to file items by project or by products.

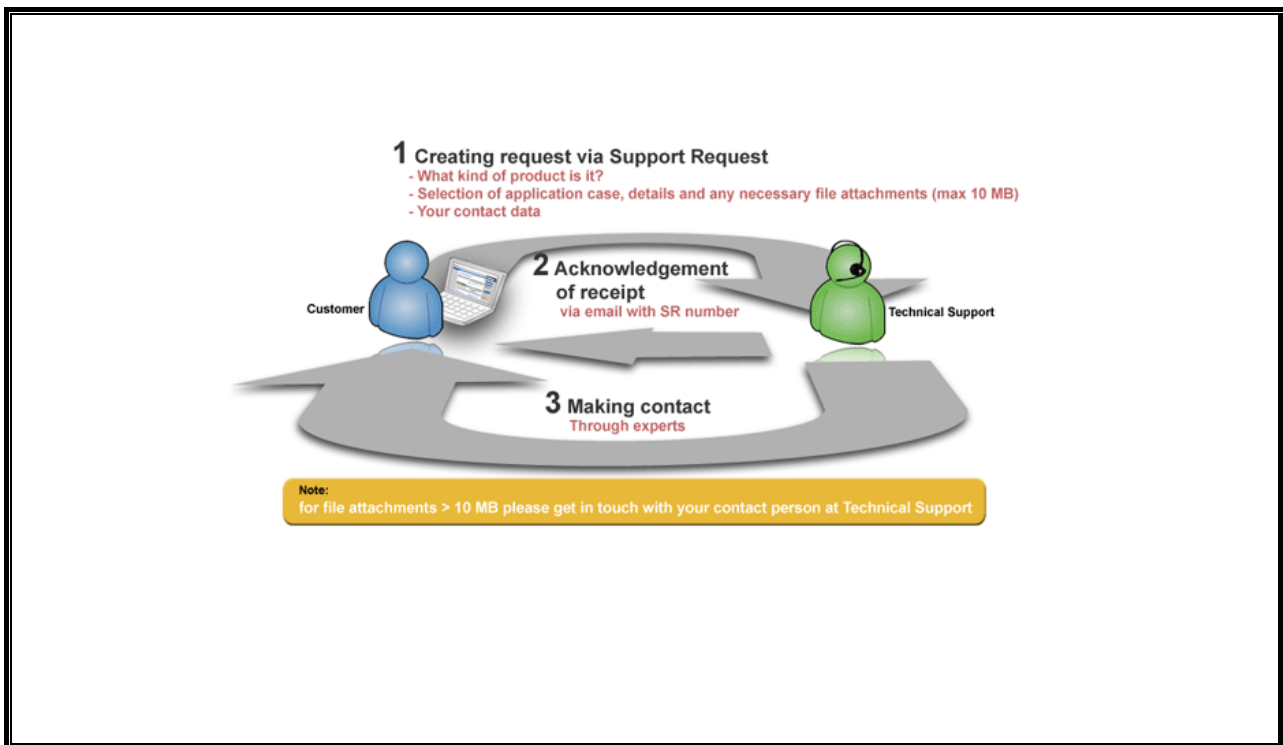
Moreover, you can now add notes, comments and tags (keywords). The system automatically creates a “Tag Cloud” based on your entries so you can access information quickly and easily by means of your own terms. The operation is consistent throughout mySupport so that you will easily find your way around. “Drag & drop” is also possible.

As soon as you are logged on, the mySupport cockpit is always at your side. It will immediately show you when the status of a support request changes, or when you receive new personal messages. You also have direct access to your personal keywords in the tag cloud, to the entries last visited, and you can see which user is online.

Here, just a few highlights:

- The previous MyDocumentationManager is now completely integrated into mySupport under the name of “mySupport-Dokumentation.” The function category “Documentation” contains all the functions of the MyDocumentationManager and provides a few innovations, too.
- The Service & Support Newsletter has been completely revamped. An individual messaging system will more than replace it.

14.8. Support Request



Support Request

To create a Support Request, different options are available to you in Online Support:

- You will find the "Support Request" option in the menu on all Online Support pages.
- Alternatively, you can create a new request in mySupport in the "Requests" category.
- Or directly click on the following link:

<http://www.siemens.com/automation/support-request>

Tips for creating a request:

- Select your product and use case as accurately as possible; try to avoid selecting "Other". By doing so, you ensure optimum support by our experts and appropriate suggested solutions.
- Did other users have a similar problem? This step already offers frequent problems and solutions. Take a look – it will be worth your while!
- Describe your problem with as much detail as possible. Pictures or explanatory attachments allow our experts to consider your problem from all sides and develop solutions. You can upload multiple attachments up to 10 MB per file.
- Before each sending, verify your personal contact information and the data you have entered. The final step additionally offers the option to print the summary.

As a logged in user, you can track the status of your requests online. To do so, navigate to "My requests" in the "Requests" category in mySupport.

14.9. Support Request

Industry Online Support | Language | Contact | Help | Support Request | Sit

> Home > mySupport > Requests

Navigation

- Personal messages
- Requests
- My requests
- Create new request
- Notifications
- Filter
- Favorites
- Tagging
- Entries last viewed
- Documentation
- Personal data
- CAx data
- Extranets

My requests

Search in "My requests"

for search

Status

Actions ✖ New request ☐ Show details...

Items per page: 10 | 20 | 30

☐	SR number	Product and subject	Status	Created on
☐	1-3871916175	STEP7 Professional V13	Closed	2/12/2015 8:49 AM

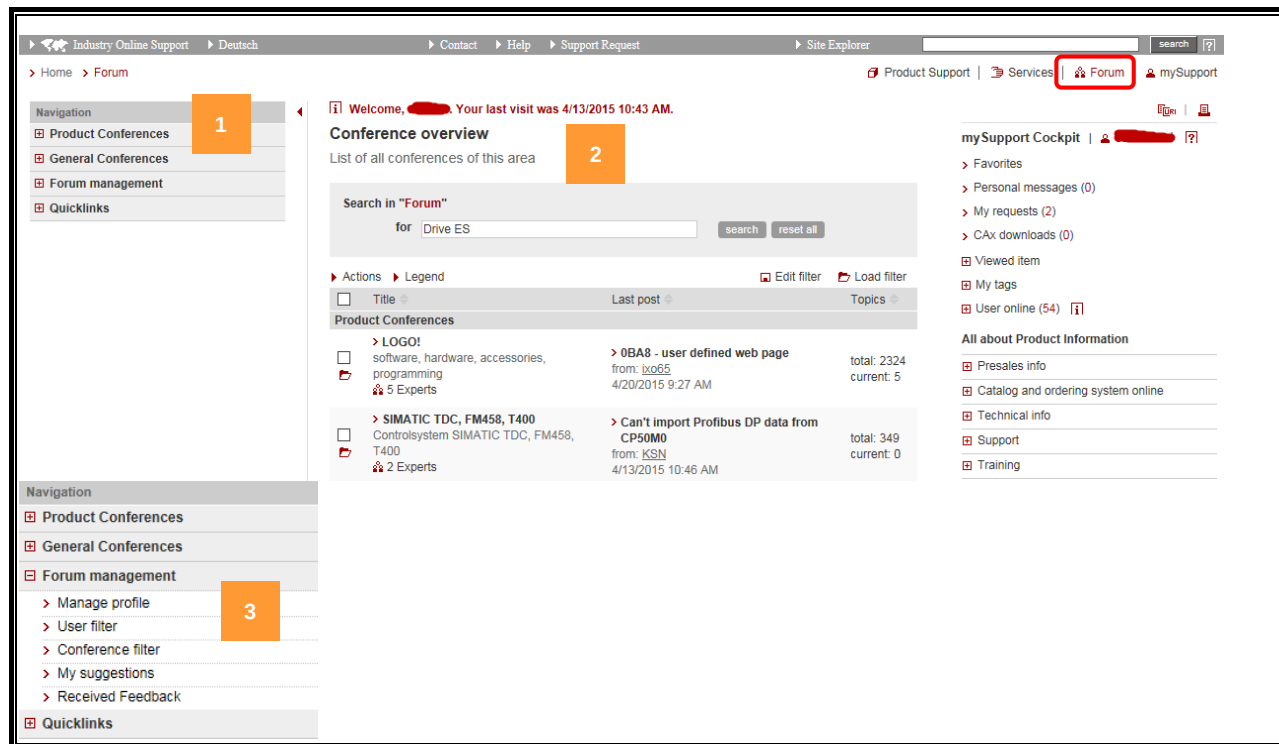
☐ Show details...

☐ Add note

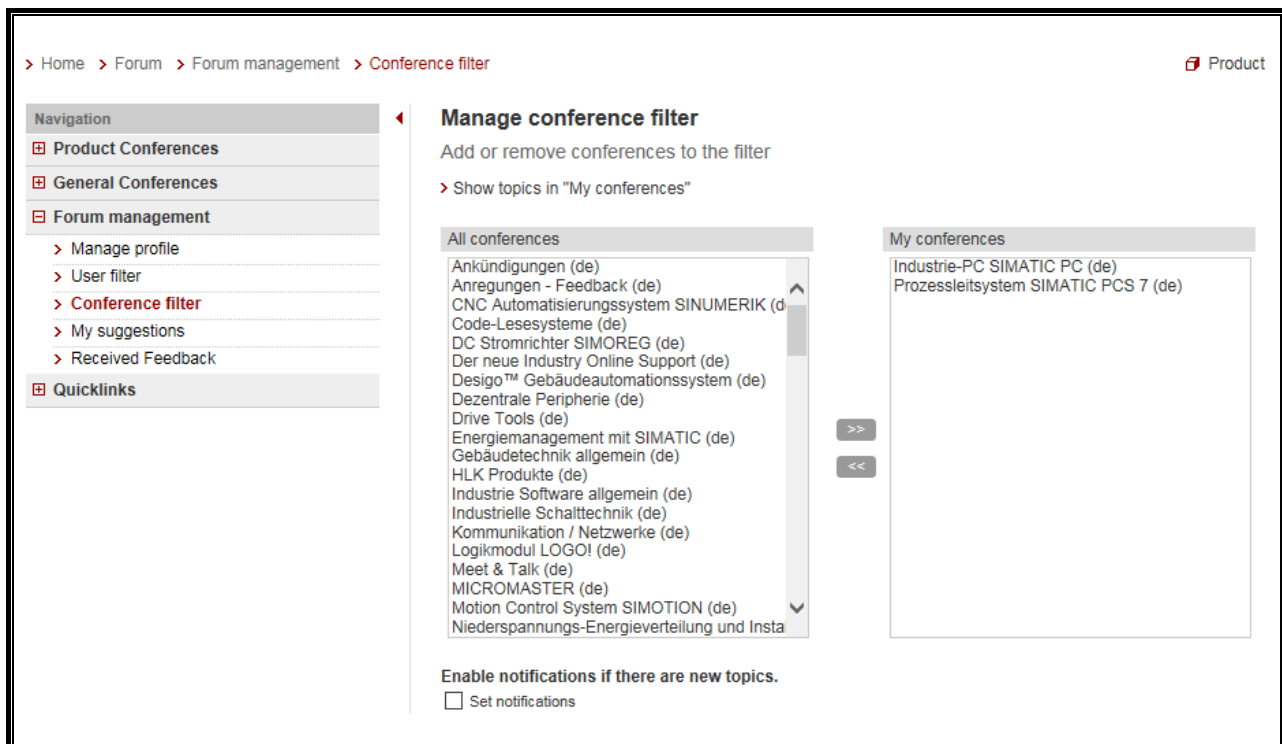
Items per page: 10 | 20 | 30

14.10. Forum - the communication platform for Siemens Industry products

14.10.1. Conferences and Forum management



- 1 On the left side, you will find the so-called conference tree. It allows you to navigate through the individual discussion areas.
- 2 The conference overview is the central discussion area of the Technical Forum. This is where the community meets to discuss technical questions about Siemens Industry products.
- 3 In forum management, you will find your personal control center for the Technical Forum. It allows you to manage your specific profile data and filters.



Conference filter

Add conferences to your personal filter of preferred conferences.

This allows you to enable a notification that informs you when new topics are started in these conferences.

In Quicklinks, the Technical Forum additionally offers an overview page that contains all topics of your preferred conferences.

Managing profile

Profile management provides interesting information and functions:

- You get an overview of your activities in the Technical Forum.
- You can view your rank, any special permissions and your ranking progress.
- You can store a signature and a personal description for your profile in the forum.
- You have direct access to the quick links to get an overview of all topics you have contributed to.

User filter

Have you found a user in the Technical Forum who posts entries that are particularly interesting? Then add this user to your list of "preferred users".

This allows you to enable a notification that informs you when the user has posted a new entry.

In Quicklinks, the Technical Forum additionally offers an overview page that contains all topics of your preferred users.

14.10.2. Interactions in the Forum

The screenshot shows the SIMATIC PCS 7 forum interface. At the top, the title "Process Control Systems SIMATIC PCS 7" is displayed. Below it is a search bar with the text "Search in 'Process Control Systems SIMATIC PCS 7'" and a "for" input field. To the right of the search bar are "search" and "reset all" buttons. Below the search bar is a navigation bar with links: "Actions", "Legend", "Experts", "New topic" (highlighted with a red asterisk icon), "Edit filter", and "Load filter". Below the navigation bar, it shows "4748 Entries" and "Entries per page: 10". A pagination bar shows "1" as the current page, with links for "2", "3", "4", "5", and "..." (highlighted with a red asterisk icon). Below the pagination bar, a topic is listed: "> Missing OS in backup PCS7 6.0 Sp2" from "paco arreaga" on "2/26/2015 3:41 AM". To the left of the topic title is a checkbox with a red checkmark icon. To the right of the topic title are the numbers "2" and "81", and a rating of five stars (four are filled, one is empty) with "(1)" below it. Three yellow callout boxes with black text are overlaid on the image: "Status: solved" points to the checkbox; "Creating a new topic in a conference" points to the "New topic" link; "Topic from conference overview" points to the topic title; and "Rating of the topic" points to the star rating.

Process Control Systems SIMATIC PCS 7

Search in "Process Control Systems SIMATIC PCS 7"

for

▶ Actions ▶ Legend ▶ Experts **✱ New topic**

4748 Entries Entries per page: 10 | 7

< | < 1 | 2 | 3 | 4 | 5 | ... > | >

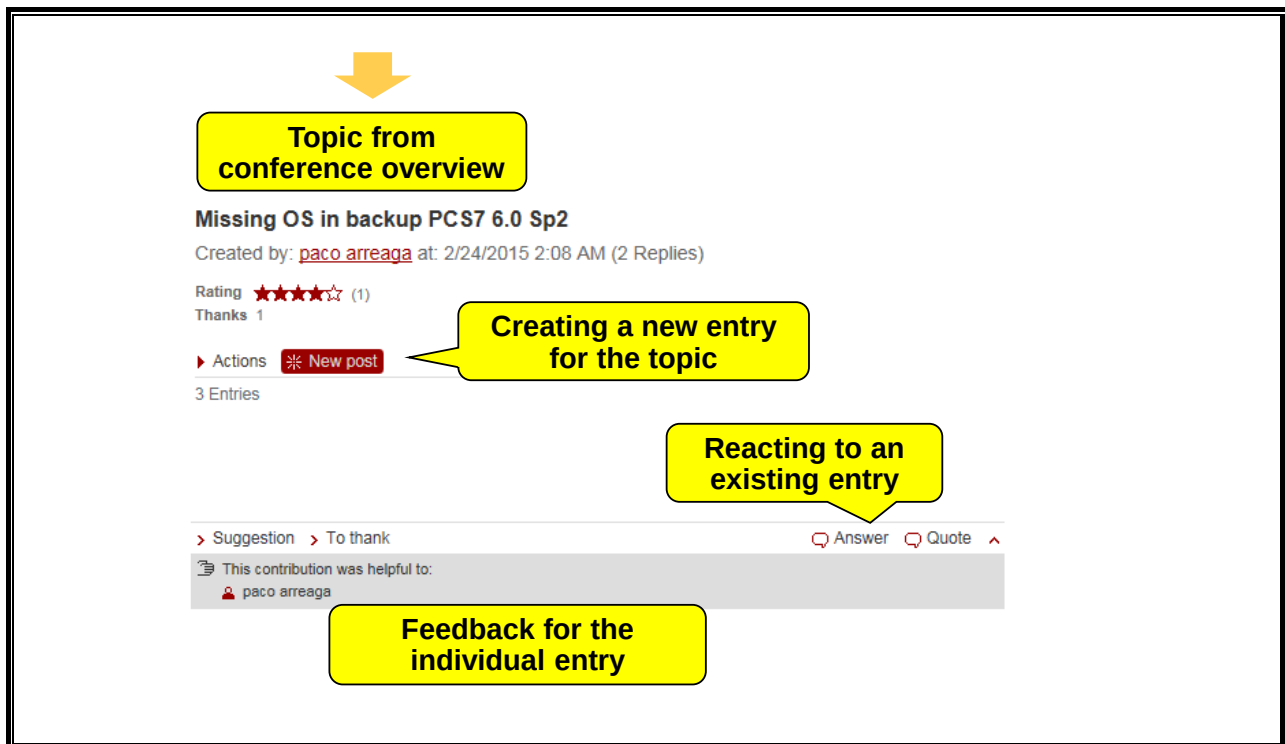
Creating a new topic in a conference

Status: solved

Topic from conference overview

Rating of the topic

☐ > Missing OS in backup PCS7 6.0 Sp2 from: [paco arreaga](#) from: [paco arreaga](#) 2/26/2015 3:41 AM 2 81 ★★★★★ (1)



Creating a new entry

Do you want to create or format a new entry? The entry editor provides all the necessary functions.

- You can upload and publish in the forum a file with "Add attachment".
- You would like to check before the publication how your entry will actually look? A preview is available for this purpose.
- You would like to look at the topic again to which you create an entry? Please, you used the link over the input area (right mouse button > open in a new tab or window)

Posting / replying to an entry

Do you want to participate in an existing discussion with your own entry? Click on "Reply" and post your personal entry to support other users in answering the question.

- Use the "Reply" link to go to the entry editor and create a reply without quoting the entry.
- If you want to quote the entry, possibly only excerpts of it, use the "Quote" link. The content of the quoted entry is then displayed accordingly in the entry editor.

Rating an entry / saying thank you

Do you find an entry particularly interesting? Use the available functions and rate the entry or say thank you to provide personal feedback. Ratings and thank yours are the rewards our community members get for the support they provide. When you rate an author or entry, this will be added to the already existing ratings. The average value of all ratings is displayed.

Aside from feedback to the author of the entry, you also draw other readers' attention to particularly valuable entries and helpful authors.

14.11. Task and Checkpoint

Task: Software compatibility

Goal

Find out which current version of virus scanners is compatible with your engineering software.

Use all information sources available:

- Readme files in the installation folder
- The compatibility tool of the Industry Online Support
- Entries in the Product support
- Entries in the Forum
- Create a Support Request.

Checkpoint



Let's think about this:

- Name some reasons for registration in MySupport.
- What do you think is the best way to have always the latest version of the required manuals for your job with you?